AI Quest: every question in the app (42172 Introduction to AI)

Generated 2026-11-10 from the app's question bank. Each question shows its type, difficulty and source, then the answer and the worked solution.
Types: MCQ (one answer), MULTI (select all that apply), TF (true/false), BLANK (fill one blank), BLANKS (fill several), MATCH (matching).
Part A of each topic = hand-written questions (lectures, workshops, key-points sheets, class exercises, Books 1-3, AIMA). Part B = computer-generated calculation drills (fresh numbers for every skill).

Contents

- 🧭 Classical Search (Lecture 2): 120 hand-written + 176 calculation drills
- 🌳 Decision Trees (Lecture 4): 62 hand-written + 80 calculation drills
- 📉 Supervised Learning (Lecture 4): 126 hand-written + 192 calculation drills
- 🏰 MDPs & Reinforcement Learning (Workshop 6): 104 hand-written + 144 calculation drills
- 🎲 Probability & Bayesian Networks (Lecture 11): 126 hand-written + 144 calculation drills
- 🧱 CNNs & Computer Vision (Lecture 7): 102 hand-written + 128 calculation drills
- 📦 Object Detection (IoU & NMS) (Lecture 7): 55 hand-written + 80 calculation drills
- 🔁 Sequence Models & NLP (Lecture 8): 99 hand-written + 128 calculation drills
- 🎯 Attention & Transformers (Lecture 8): 63 hand-written + 112 calculation drills
- 🧠 Neural Networks (Workshop 5): 79 hand-written + 112 calculation drills
- ⛰️ Local Search & GA (Lecture 3): 81 hand-written + 80 calculation drills
- ✨ Generative AI (Lecture 9): 78 hand-written + 96 calculation drills
- 🔐 Logic & Inference (Lecture 10): 121 hand-written + 80 calculation drills
- 🕸️ Knowledge Representation (Lecture 12): 87 hand-written + 0 calculation drills
- 🧮 Maths Toolkit (Book 3) (Book 3 · Part I): 45 hand-written + 0 calculation drills
- 📘 Book 2 mixed practice paper: answer key at the end

Total: 2900 questions.

🧭 Classical Search (Lecture 2)

Part A: hand-written questions (120)

SEARCH A1. [MCQ · Easy]

Which of these is NOT a component of a classical search problem as defined in Lecture 2?

A. A goal state and goal test
B. A transition model RESULT(s, a)
C. An action cost function
D. A reward function that changes over time

Answer: D. A reward function that changes over time

Option notes:

- A: It is a component.
- B: It is a component.
- C: It is a component.

Takeaway: Search problem = states, initial, goal, actions, transition model, action cost.

Source: Lecture 2, slide 11

SEARCH A2. [TF · Easy]

Classical search assumes a fully observable, deterministic and known environment, so the solution is a fixed sequence of actions.

Answer: True

Why: Lecture 2 states exactly these assumptions.

Takeaway: Uncertainty (partial observability, non-determinism) needs other tools (MDPs, Bayesian networks).

Source: Lecture 2, slide 12

SEARCH A3. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

In the Romania example, the blue path A-S-F-B costs 450 and A-S-R-P-B costs 418. Which statement is correct?

A. Only the optimal path counts as a solution
B. A-S-F-B is optimal because it uses fewer actions
C. Both are solutions, but only A-S-R-P-B is the optimal solution
D. Neither is a solution until the goal test is defined on paths

Answer: C. Both are solutions, but only A-S-R-P-B is the optimal solution

Option notes:

- A: Any action sequence from start to goal is a solution.
- B: Optimality is about path COST, not number of actions.
- D: The goal test checks states, and both reach Bucharest.

Takeaway: Solution = any path to the goal; optimal = least path cost.

Source: Lecture 2, slide 8-9

SEARCH A4. [BLANK · Easy]

In Lecture 2 notation, the function that returns the state reached by doing action a in state s is called the _____ model.

Answer: transition
(Also accepted: Transition, transition model)

Takeaway: RESULT(s, a) is the transition model.

Source: Lecture 2, slide 17

SEARCH A5. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Romania path A-S-R-P-B: road costs A→S 140, S→R 80, R→P 97, P→B 101. Give the path cost, and the cost of the alternative A-S-F-B (S→F 99, F→B 211).

(1) A-S-R-P-B = ____
(2) A-S-F-B = ____

Answer: A-S-R-P-B = 418; A-S-F-B = 450

Working:

- 140+80+97+101 = 418
- 140+99+211 = 450

Takeaway: Path cost = sum of step costs.

Source: Lecture 2, slide 9

SEARCH A6. [MCQ · Medium]

What is the frontier in a search spanning tree?

A. The nodes at the maximum depth only
B. The set of leaf nodes that are candidates for expansion
C. The path from the root to the goal
D. The set of all nodes already expanded

Answer: B. The set of leaf nodes that are candidates for expansion

Option notes:

- A: Frontier nodes can be at any depth.
- C: That is the solution.
- D: That is the explored set (graph search).

Takeaway: Frontier = leaves waiting to be expanded.

Source: Lecture 2, slide 26

SEARCH A7. [TF · Medium]

Tree search never expands the same state twice.

Answer: False

Why: That's GRAPH search. Tree search does not check whether a node was expanded before, so Arad can reappear under Sibiu.

Takeaway: Graph search = tree search + explored set.

Source: Lecture 2, slide 28-29

SEARCH A8. [MULTI · Medium]

Which statements about graph search (Lecture 2) are correct? (Select ALL that apply.)

A. It does not add a child that is already in the frontier or explored set
B. The goal test in the lecture pseudocode happens when a node is chosen for expansion
C. It can loop forever on Arad → Sibiu → Arad
D. It keeps an explored set
E. It is the same as tree search with a FIFO frontier

Answer: A. It does not add a child that is already in the frontier or explored set; B. The goal test in the lecture pseudocode happens when a node is chosen for expansion; D. It keeps an explored set

Option notes:

- A: Correct: As in the pseudocode.
- B: Correct: Line: if the node contains a goal state then return.
- C: Incorrect: That loop is the tree-search problem it avoids.
- D: Correct: That is what distinguishes it.
- E: Incorrect: FIFO frontier is BFS; tree vs graph is about repeated states.

Takeaway: Graph search remembers where it has been.

Source: Lecture 2, slide 28-29

SEARCH A9. [BLANK · Easy]

Expanding a node means using the _____ function to generate its children and connecting them to the current node.

Answer: transition
(Also accepted: Transition, successor)

Takeaway: Expand = generate children via the transition model.

Source: Lecture 2, slide 24

SEARCH A10. [MCQ · Hard]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

A search tree for Romania built with TREE search contains Arad → Sibiu → Arad. Why?

A. The graph has a self-loop on Arad
B. Arad has a lower h than Sibiu
C. Tree search does not check whether a state has already been expanded
D. Because the frontier is a FIFO queue

Answer: C. Tree search does not check whether a state has already been expanded

Option notes:

- A: There is a two-way road Arad–Sibiu, not a self-loop.
- B: Heuristics don't create nodes; the transition model does.
- D: Any strategy can produce repeats in tree search.

Takeaway: Repeated states are the price of forgetting.

Source: Lecture 2, slide 25

SEARCH A11. [MATCH · Medium]

Match each performance measure to its analogy (Lecture 2, slide 31).

1. Completeness
2. Cost optimality
3. Time complexity
4. Space complexity

A. Will I find the shortest/cheapest route?
B. Will I eventually reach the destination?
C. How much searching do I need to do?
D. How much do I need to remember?
E. How confident am I that the route is safe?

Answer: 1 → B, 2 → A, 3 → C, 4 → D

Working:

- Completeness → Will I eventually reach the destination?
- Cost optimality → Will I find the shortest/cheapest route?
- Time complexity → How much searching do I need to do?
- Space complexity → How much do I need to remember?

Takeaway: Complete = finds a solution; optimal = finds the cheapest.

Source: Lecture 2, slide 31

SEARCH A12. [MCQ · Hard]

According to the Lecture 2 table, which algorithm has space complexity O(bm)?

A. Depth-first search
B. Greedy best-first search
C. Breadth-first search
D. Uniform-cost search

Answer: A. Depth-first search

Option notes:

- B: Greedy: O(b^m).
- C: BFS needs O(b^d) space.
- D: UCS: O(b^(1+⌊C*/ε⌋)).

Takeaway: DFS only stores one path plus siblings.

Source: Lecture 2, slide 53

SEARCH A13. [TF · Medium]

Cost optimality means the algorithm is guaranteed to find a solution whenever one exists.

Answer: False

Why: That is COMPLETENESS. Cost optimality = the solution found has the lowest cost.

Takeaway: Complete ≠ optimal.

Source: Lecture 2, slide 30

SEARCH A14. [MATCH · Exam+]

Match each algorithm to its entry in the Lecture 2 performance table (Complete? / Cost-optimal?).

1. BFS
2. DFS
3. Greedy best-first
4. A* (admissible h)

A. Complete: No, Optimal: No
B. Complete: Yes*, Optimal: Yes (time O(b^d))
C. Complete: Not always, Optimal: No
D. Complete: No, Optimal: Yes
E. Complete: Yes*, Optimal: Yes

Answer: 1 → E, 2 → A, 3 → C, 4 → B

Working:

- BFS → Complete: Yes*, Optimal: Yes
- DFS → Complete: No, Optimal: No
- Greedy best-first → Complete: Not always, Optimal: No
- A* (admissible h) → Complete: Yes*, Optimal: Yes (time O(b^d))

Takeaway: Only BFS, UCS and A* are cost-optimal in the table.

Source: Lecture 2, slide 53

SEARCH A15. [BLANK · Medium]

In the complexity O(b^d), b is the branching factor and d is the _____ of the shallowest solution.

Answer: depth
(Also accepted: Depth)

Takeaway: b = branching factor, d = solution depth, m = max depth.

Source: Lecture 2, slide 53

SEARCH A16. [MCQ · Easy]

Which data structure does BFS use for its frontier?

A. A FIFO queue
B. A LIFO stack
C. A priority queue ordered by h
D. A priority queue ordered by g

Answer: A. A FIFO queue

Option notes:

- B: That's DFS.
- C: That's greedy best-first.
- D: That's UCS.

Takeaway: BFS: First In, First Out.

Source: Lecture 2, slide 33

SEARCH A17. [TF · Medium]

BFS always finds a solution with the minimal number of actions, even with an infinite state space.

Answer: True

Why: Lecture 2, slide 33 states this.

Takeaway: Fewest actions = cheapest only if all step costs are equal.

Source: Lecture 2, slide 33

SEARCH A18. [TF · Medium]

DFS is cost-optimal because it explores each path to the end before backing up.

Answer: False

Why: DFS returns the FIRST solution it finds, even if it is not the cheapest.

Takeaway: DFS: memory-light, not optimal.

Source: Lecture 2, slide 36

SEARCH A19. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): A — B; A — C; B — D; B — E; C — F; C — G

Binary tree A has children B, C; B has D, E; C has F, G. List the BFS expansion order (letters, comma-separated).

Answer: A,B,C,D,E,F,G
(Also accepted: A, B, C, D, E, F, G, ABCDEFG)

Takeaway: BFS expands level by level.

Source: Lecture 2, slide 34

SEARCH A20. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): A — B; A — C; B — D; B — E; C — F; C — G; D — H; D — I; E — J; E — K; F — L; F — M; G — N; G — O

In the depth-3 binary tree A..O of Lecture 2 (A; B,C; D,E,F,G; H..O) with goal M, DFS (left child first) expands nodes in which order? (comma-separated, ending with M)

Answer: A,B,D,H,I,E,J,K,C,F,L,M
(Also accepted: A, B, D, H, I, E, J, K, C, F, L, M)

Takeaway: DFS dives left first, backs up when a branch is exhausted.

Source: Lecture 2, slide 37

SEARCH A21. [MATCH · Medium]

Match each algorithm to its evaluation function (as a best-first search, Lecture 2 summary).

1. BFS
2. DFS
3. Uniform-cost search
4. Greedy best-first
5. A*

A. depth of the node
B. h(n), the cost to the goal
C. path cost g(n)
D. negative depth
E. h(n) − g(n)
F. g(n) + h(n)

Answer: 1 → A, 2 → D, 3 → C, 4 → B, 5 → F

Working:

- BFS → depth of the node
- DFS → negative depth
- Uniform-cost search → path cost g(n)
- Greedy best-first → h(n), the cost to the goal
- A* → g(n) + h(n)

Takeaway: Every algorithm here is 'best-first' with a different f.

Source: Lecture 2, slide 54

SEARCH A22. [BLANK · Easy]

Uniform-cost search ranks frontier nodes using f(n) = _____.

Answer: g(n)
(Also accepted: g, path cost)

Takeaway: UCS: no heuristic term.

Source: UCS Romania fill-in-blanks Q1

SEARCH A23. [BLANKS · Easy · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Road costs from Arad: Zerind 75, Timișoara 118, Sibiu 140. UCS expands _____ next with f = _____.

(1) node = ____
(2) f = ____

Answer: node = Zerind; f = 75

Takeaway: Lowest cumulative path cost first.

Source: UCS Romania fill-in-blanks Q2

SEARCH A24. [BLANK · Easy · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Arad → Sibiu costs 140 and Sibiu → Râmnicu Vâlcea costs 80. g(Râmnicu Vâlcea) = _____.

Answer: 220

Takeaway: Add every road cost from the start.

Source: UCS Romania fill-in-blanks Q3

SEARCH A25. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

A frontier route to Făgăraș has g = 239 and one to Râmnicu Vâlcea has g = 220 (h(F) = 176, h(RV) = 193). Which does UCS expand first and why?

A. Făgăraș, because 239 + 176 = 415 is the smallest f
B. Both together, since they are siblings
C. Făgăraș, because h = 176 < 193
D. Râmnicu Vâlcea, because 220 < 239

Answer: D. Râmnicu Vâlcea, because 220 < 239

Option notes:

- A: That mixes in h; UCS uses g only (and A* would pick RV with 413).
- B: One node is popped at a time.
- C: UCS ignores h.

Takeaway: Heuristic distance is irrelevant to UCS.

Source: UCS Romania fill-in-blanks Q4

SEARCH A26. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

When Bucharest is removed from the UCS frontier in the Romania example, f(Bucharest) = _____.

Answer: 418

Takeaway: For UCS, the goal's value is its total path cost.

Source: UCS Romania fill-in-blanks Q5

SEARCH A27. [TF · Hard]

UCS can stop as soon as the goal is added to the frontier, because any later path must be longer.

Answer: False

Why: Bucharest is first generated via Făgăraș (450); the cheaper route via Pitești (418) is found later. UCS tests the goal when it is SELECTED.

Takeaway: Goal test on expansion makes UCS optimal.

Source: Lecture 2, slide 43

SEARCH A28. [MULTI · Medium]

Which statements about uniform-cost search are correct (Lecture 2)? (Select ALL that apply.)

A. It is complete and cost optimal
B. It considers paths in order of increasing cost
C. It uses a priority queue ordered by path cost

D. It can get stuck going down a single infinite path
E. It uses the straight-line distance heuristic

Answer: A. It is complete and cost optimal; B. It considers paths in order of increasing cost; C. It uses a priority queue ordered by path cost

Option notes:

- A: Correct: Slide 41.
- B: Correct: Slide 41.
- C: Correct: Slide 43.
- D: Incorrect: It never gets caught down a single infinite path.
- E: Incorrect: No heuristic in UCS.

Takeaway: UCS = Dijkstra-style cheapest-first.

Source: Lecture 2, slide 41-43

SEARCH A29. [BLANK · Easy]

Greedy best-first search ranks each frontier node n using f(n) = _____.

Answer: h(n)
(Also accepted: h)

Takeaway: Ignore the path cost already travelled.

Source: GBFS Romania fill-in-blanks Q1

SEARCH A30. [BLANK · Easy · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

The straight-line distance from Arad to Bucharest is h(Arad) = 366. For greedy search, f(Arad) = _____.

Answer: 366

Takeaway: For greedy, f and h are the same.

Source: GBFS Romania fill-in-blanks Q2

SEARCH A31. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380,

Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

From Arad: h(Sibiu) = 253, h(Timișoara) = 329, h(Zerind) = 374. Greedy search expands which node next?

A. Zerind
B. Timișoara
C. Arad again
D. Sibiu

Answer: D. Sibiu

Option notes:

- A: Zerind has the LOWEST road cost (75) but the highest h.
- B: 329 > 253.
- C: Arad has already been expanded.

Takeaway: Choose the smallest heuristic value.

Source: GBFS Romania fill-in-blanks Q3

SEARCH A32. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

After expanding Sibiu: h(Făgăraș) = 176, h(Râmnicu Vâlcea) = 193, h(Oradea) = 380. The next node is _____ because its f-value is _____.

(1) node = ____
(2) f = ____

Answer: node = Făgăraș; f = 176

Takeaway: Greedy always chooses the lowest h.

Source: GBFS Romania fill-in-blanks Q4

SEARCH A33. [BLANKS · Medium]

One frontier node has g = 400, h = 50; another has g = 100, h = 80. Greedy selects the node with h = _____, ignoring _____ (one symbol).

(1) h = ____
(2) ignored = ____

Answer: h = 50; ignored = g

Takeaway: The greedy evaluation has no accumulated path-cost term.

Source: GBFS Romania fill-in-blanks Q5

SEARCH A34. [TF · Medium]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Greedy best-first search on Romania returns the optimal route to Bucharest.

Answer: False

Why: It returns A-S-F-B (450 km), not the optimal 418 km.

Takeaway: Greedy is not cost-optimal.

Source: Lecture 2, slide 44-45, 53

SEARCH A35. [BLANKS · Easy]

Build the A* evaluation function: f(n) = _____ + _____.

(1) first term = ____
(2) second term = ____

Answer: first term = g(n); second term = h(n)

Takeaway: A* uses both the accumulated path cost and the heuristic estimate.

Source: A Romania fill-in-blanks Q1*

SEARCH A36. [BLANKS · Easy · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

At Arad, g(Arad) = 0 and h(Arad) = 366. f(Arad) = _____ + _____ = _____.

(1) g = ____
(2) h = ____
(3) f = ____

Answer: g = 0; h = 366; f = 366

Takeaway: Apply f(n) = g(n) + h(n).

Source: A Romania fill-in-blanks Q2*

SEARCH A37. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

From Arad: Sibiu g=140, h=253; Timișoara g=118, h=329; Zerind g=75, h=374. A* expands _____ next because its f-value is _____.

(1) node = ____
(2) f = ____

Answer: node = Sibiu; f = 393

Takeaway: Timișoara 447, Zerind 449.

Source: A Romania fill-in-blanks Q3*

SEARCH A38. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Via Sibiu: Râmnicu Vâlcea g=220, h=193; Făgăraș g=239, h=176. A* chooses _____ because 220 + 193 = _____.

(1) node = ____
(2) f = ____

Answer: node = Râmnicu Vâlcea; f = 413

Takeaway: f(Făgăraș) = 415 > 413.

Source: A Romania fill-in-blanks Q4*

SEARCH A39. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176,

Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Complete the optimal A* route: Arad → Sibiu → _____ → _____ → Bucharest; total cost = _____.

(1) city 3 = ____
(2) city 4 = ____
(3) cost = ____

Answer: city 3 = Râmnicu Vâlcea; city 4 = Pitești; cost = 418

Working:

- g(RV) = 220, g(P) = 220 + 97 = 317, g(B) = 317 + 101 = 418.

Takeaway: Final road costs 97 (RV→P) and 101 (P→B).

Source: A Romania fill-in-blanks Q5*

SEARCH A40. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

```
Node | g | h | f = g + h
Sibiu | 140 | 253 | 393
Timișoara | 118 | 329 | 447
Zerind | 75 | 374 | 449
Râmnicu Vâlcea (via S) | 220 | 193 | 413
Făgăraș (via S) | 239 | 176 | 415
Pitești (via RV) | 317 | 100 | 417
Bucharest (via F) | 450 | 0 | 450
Bucharest (via P) | 418 | 0 | 418
```

After expanding Arad, Sibiu and Râmnicu Vâlcea, the A* frontier contains Făgăraș 415, Pitești 417, Timișoara 447, Zerind 449, Craiova 526, Sibiu-again 553. Which node is expanded next?

A. Timișoara (447), because it has waited longest
B. Pitești (417)
C. Bucharest, because it is the goal
D. Făgăraș (415)

Answer: D. Făgăraș (415)

Option notes:

- A: A* uses f, not waiting time.
- B: 417 > 415.
- C: Bucharest is not in the frontier yet.

Takeaway: Always pick the global minimum f in the frontier.

Source: Lecture 2, slide 48

SEARCH A41. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

```
Node | g | h | f = g + h
Sibiu | 140 | 253 | 393
Timișoara | 118 | 329 | 447
Zerind | 75 | 374 | 449
Râmnicu Vâlcea (via S) | 220 | 193 | 413
Făgăraș (via S) | 239 | 176 | 415
Pitești (via RV) | 317 | 100 | 417
Bucharest (via F) | 450 | 0 | 450
Bucharest (via P) | 418 | 0 | 418
```

In the Romania A* run, when Făgăraș is expanded it adds Bucharest with f = 450 + 0 = 450, so A* returns the 450 km route.

Answer: False

Why: Bucharest(450) waits in the frontier; Pitești (417) is expanded first and finds Bucharest with f = 418, which is popped before 450.

Takeaway: A* returns a goal only when it is selected with the smallest f.

Source: Lecture 2, slide 48

SEARCH A42. [MCQ · Easy]

What does it mean for a heuristic to be admissible?

A. It never underestimates the cost to reach the goal
B. It never overestimates the cost to reach the goal
C. It equals the true cost exactly
D. h(n) ≤ c(n,a,n′) + h(n′) for every successor

Answer: B. It never overestimates the cost to reach the goal

Option notes:

- A: Reversed: admissible heuristics may underestimate.
- C: That's the perfect heuristic h*, a special case.
- D: That is CONSISTENCY.

Takeaway: Admissible = optimistic.

Source: Lecture 2, slide 50

SEARCH A43. [TF · Medium]

Every admissible heuristic is also consistent.

Answer: False

Why: Reversed. Every CONSISTENT heuristic is admissible, not vice versa.

Takeaway: Consistency is the stronger property.

Source: Lecture 2, slide 52

SEARCH A44. [TF · Medium]

Every consistent heuristic is also admissible.

Answer: True

Why: Lecture 2, slide 52.

Takeaway: Consistent ⇒ admissible.

Source: Lecture 2, slide 52

SEARCH A45. [TF · Hard]

The graph-search version of A* is guaranteed optimal if h is merely admissible.

Answer: False

Why: Lecture 2: tree-search A* is optimal if h is admissible; GRAPH-search A* needs a CONSISTENT h.

Takeaway: Admissible → tree A*; consistent → graph A*.

Source: Lecture 2, slide 52

SEARCH A46. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Why is the straight-line distance (SLD) to Bucharest admissible?

A. Because it overestimates slightly, which makes A* faster
B. A straight line is never longer than any road route between two cities
C. Because SLD is always exactly equal to the road distance
D. Because it is consistent with the road costs by definition of A*

Answer: B. A straight line is never longer than any road route between two cities

Option notes:

- A: Overestimating would break admissibility.
- C: It is usually SHORTER, not equal.
- D: Admissibility comes from geometry, not from A*.

Takeaway: SLD ≤ true road distance.

Source: Lecture 2, slide 50

SEARCH A47. [BLANK · Medium]

Consistency condition: h(n) ≤ c(n, a, n′) + _____.

Answer: h(n′)
(Also accepted: h(n'), h(n’), h(n2))

Takeaway: Triangle inequality.

Source: Lecture 2, slide 51

SEARCH A48. [MCQ · Hard · calculation]

h(A) = 9, c(A,B) = 3, h(B) = 5, and h*(A) = 10. Which is true?

A. h is neither admissible nor consistent
B. h is both admissible and consistent
C. h is admissible at A but violates consistency on edge A→B
D. h is consistent at A but not admissible

Answer: C. h is admissible at A but violates consistency on edge A→B

Option notes:

- A: 9 ≤ 10 is admissible.
- B: 9 > 8 breaks consistency.
- D: 9 ≤ 10 so admissible; 9 > 3 + 5 = 8 so not consistent.

Takeaway: Check both conditions separately.

Source: Lecture 2, slide 50-51

SEARCH A49. [MULTI · Exam+]

Select ALL true statements about A*. (Select ALL that apply.)

A. A* with an inadmissible heuristic is still always optimal
B. f(n) estimates the cost of the cheapest path from start through n to the goal
C. With an admissible heuristic, (tree-search) A* is cost-optimal
D. A* expands the node with the lowest h
E. A* with h(n) = 0 behaves like uniform-cost search

Answer: B. f(n) estimates the cost of the cheapest path from start through n to the goal; C. With an admissible heuristic, (tree-search) A* is cost-optimal; E. A* with h(n) = 0 behaves like uniform-cost search

Option notes:

- A: Incorrect: Overestimates can make A* skip the optimal path.
- B: Correct: Slide 47.
- C: Correct: Slide 50/52.
- D: Incorrect: That's greedy; A* uses g + h.
- E: Correct: f = g + 0 = g.

Takeaway: h = 0 turns A* into UCS.

Source: Lecture 2, slide 47-52

SEARCH A50. [MATCH · Medium]

Match each algorithm to its frontier data structure / ordering.

1. BFS
2. DFS

3. UCS
4. A*

A. FIFO queue
B. LIFO stack
C. priority queue ordered by g(n) + h(n)
D. priority queue ordered by g(n)
E. priority queue ordered by depth²

Answer: 1 → A, 2 → B, 3 → D, 4 → C

Working:

- BFS → FIFO queue
- DFS → LIFO stack
- UCS → priority queue ordered by g(n)
- A* → priority queue ordered by g(n) + h(n)

Takeaway: UCS and A* both use priority queues; only the key differs.

Source: Lecture 2, slide 33-49

SEARCH A51. [MCQ · Medium]

Which method fits this scenario: you need the cheapest route, have no heuristic information, and step costs differ?

A. Uniform-cost search
B. Depth-first search
C. Greedy best-first search
D. Breadth-first search

Answer: A. Uniform-cost search

Option notes:

- B: Not cost-optimal.
- C: Needs a heuristic and is not optimal.
- D: BFS minimises the number of actions, not cost, when costs differ.

Takeaway: No heuristic + varying costs → UCS.

Source: Lecture 2, slide 33-41

SEARCH A52. [MCQ · Hard]

A* and UCS both return the optimal route. What is A*'s main advantage when h is good?

A. It never needs to revisit states
B. It needs no priority queue
C. It finds cheaper paths than UCS
D. It usually expands far fewer nodes by focusing toward the goal

Answer: D. It usually expands far fewer nodes by focusing toward the goal

Option notes:

- A: Revisiting depends on graph vs tree search, not A* vs UCS.
- B: It uses one, ordered by g + h.
- C: Both are optimal: same cost.

Takeaway: Same answer, less work.

Source: Lecture 2, slide 41-53

SEARCH A53. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Greedy best-first search ranks each frontier node n using only its estimated distance to Bucharest. f(n) = ___.

Answer: h(n)
(Also accepted: h, h (n), h_sld(n), hsld(n))

Takeaway: Greedy: f = h.

Source: Book 2 (Exam examples), Q1, p. 7

SEARCH A54. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Greedy best-first on Romania. At Arad, h(Arad) = 366. f(Arad) = ___.

Answer: 366

Takeaway: For greedy search f and h are the same.

Source: Book 2 (Exam examples), Q2, p. 7

SEARCH A55. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380,

Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Greedy from Arad: h(Sibiu) = 253, h(Timisoara) = 329, h(Zerind) = 374. Greedy expands ___ next.

Answer: Sibiu

Working:

- 253 < 329 < 374, so Sibiu.

Takeaway: Smallest h wins.

Source: Book 2 (Exam examples), Q3, p. 8

SEARCH A56. [BLANKS · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Greedy, after expanding Sibiu: h(Fagaras) = 176, h(Rimnicu Vilcea) = 193, h(Oradea) = 380. The next node is ___ because its f-value is ___.

(1) node = ____
(2) f = ____

Answer: node = Fagaras; f = 176

Takeaway: Greedy picks the smallest h.

Source: Book 2 (Exam examples), Q4, p. 8

SEARCH A57. [BLANKS · Medium]

One frontier node has g = 400, h = 50; another has g = 100, h = 80. Greedy selects the node with h = ___, ignoring ___.

(1) h = ____
(2) what it ignores = ____

Answer: h = 50; what it ignores = g(n)

Takeaway: Greedy ignores the cost already travelled.

Source: Book 2 (Exam examples), Q5, p. 8

SEARCH A58. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)

Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

UCS ranks each frontier node using the exact cost accumulated from Arad. f(n) = ___.

Answer: g(n)
(Also accepted: g, g (n))

Takeaway: UCS has no heuristic term.

Source: Book 2 (Exam examples), Q6, p. 8

SEARCH A59. [BLANKS · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Road costs from Arad: Zerind 75, Timisoara 118, Sibiu 140. UCS expands ___ next with f = ___.

(1) node = ____
(2) f = ____

Answer: node = Zerind; f = 75

Takeaway: Lowest cumulative path cost.

Source: Book 2 (Exam examples), Q7, p. 8

SEARCH A60. [BLANK · Easy · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Arad→Sibiu costs 140 and Sibiu→Rimnicu Vilcea costs 80. g(Rimnicu Vilcea) = 140 + 80 = ___.

Answer: 220

Takeaway: Add every road cost from the start.

Source: Book 2 (Exam examples), Q8, p. 8

SEARCH A61. [BLANKS · Medium]

A UCS frontier route to Fagaras has g = 239; to Rimnicu Vilcea g = 220. UCS expands ___ first because ___ < ___.

(1) node = ____
(2) smaller g = ____
(3) larger g = ____

Answer: node = Rimnicu Vilcea; smaller g = 220; larger g = 239

Takeaway: Heuristic distances are irrelevant for UCS.

Source: Book 2 (Exam examples), Q9, p. 8

SEARCH A62. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

The cheapest route from Arad to Bucharest costs 418 (via Sibiu, Rimnicu Vilcea, Pitesti). When UCS removes Bucharest from the frontier, f(Bucharest) = ___.

Answer: 418

Takeaway: For UCS the goal's value is its total path cost; it is returned when selected.

Source: Book 2 (Exam examples), Q10, p. 8

SEARCH A63. [BLANKS · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

A* ranks each frontier node by combining the path cost from Arad with the estimated remaining distance. f(n) = ___ + ___.

(1) first term = ____
(2) second term = ____

Answer: first term = g(n); second term = h(n)

Takeaway: A*: f = g + h.

Source: Book 2 (Exam examples), Q11, p. 8

SEARCH A64. [BLANKS · Easy · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

A* on Romania: g(Arad) = 0 and h(Arad) = 366. f(Arad) = ___ + ___ = ___.

(1) g = ____
(2) h = ____
(3) f = ____

Answer: g = 0; h = 366; f = 366

Takeaway: At the start g = 0.

Source: Book 2 (Exam examples), Q12, p. 9

SEARCH A65. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

A* from Arad: Sibiu g = 140, h = 253; Timisoara g = 118, h = 329; Zerind g = 75, h = 374. A* expands ___ next because its f-value is ___.

(1) node = ____
(2) f = ____

Answer: node = Sibiu; f = 393

Working:

- Sibiu 140 + 253 = 393
- Timisoara 118 + 329 = 447
- Zerind 75 + 374 = 449
- Smallest: Sibiu 393.

Takeaway: Compare g + h, not g or h alone.

Source: Book 2 (Exam examples), Q13, p. 9

SEARCH A66. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

A* via Sibiu: Rimnicu Vilcea g = 220, h = 193; Fagaras g = 239, h = 176. A* chooses ___ because 220 + 193 = ___.

(1) node = ____
(2) f = ____

Answer: node = Rimnicu Vilcea; f = 413

Takeaway: Fagaras has 239 + 176 = 415, so Rimnicu Vilcea (413) wins.

Source: Book 2 (Exam examples), Q14, p. 9

SEARCH A67. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Complete the optimal A* route: Arad → Sibiu → ___ → ___ → Bucharest; total cost ___.

(1) 3rd city = ____
(2) 4th city = ____
(3) cost = ____

Answer: 3rd city = Rimnicu Vilcea; 4th city = Pitesti; cost = 418

Working:

- Arad 366 → S 393, T 447, Z 449
- Sibiu 393 → R 413, F 415, O 671
- Rimnicu 413 → P 317 + 100 = 417, C 526
- Fagaras 415 → B 450
- Pitesti 417 → B 418 replaces 450
- Pop Bucharest 418: done.

Takeaway: 220 + 97 = 317 to Pitesti, 317 + 101 = 418.

Source: Book 2 (Exam examples), Q15, p. 9

SEARCH A68. [BLANKS · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (5); A — C (2); A — D (6); C — G (7); B — D (1); D — G (2)

```
node | S | A | B | C | D | G
h | 6 | 5 | 3 | 6 | 2 | 0
```

Book 2 §2.2: arrows go left to right. Successors generated alphabetically; ties broken alphabetically.

Run UCS from S on the graph shown (successors alphabetical, ties alphabetical). Give the expansion order, the path returned and its cost.

(1) expansion order = ____
(2) path = ____
(3) cost = ____

Answer: expansion order = S, A, C, B, D, G; path = S-B-D-G; cost = 8

Working:

- Expand S: A 2, B 5.
- Expand A (2): C 4, D 8.
- Expand C (4): G 11.
- Expand B (5): D via B g = 6 < 8, replace.
- Expand D (6): G 8 < 11, replace.
- Pop G (8): path S-B-D-G, cost 8.

Takeaway: UCS replaces a frontier entry when a cheaper path appears (D: 8 → 6, G: 11 → 8).

Source: Book 2 (Exam examples), Q16, p. 9

SEARCH A69. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (5); A — C (2); A — D (6); C — G (7); B — D (1); D — G (2)

```
node | S | A | B | C | D | G
h | 6 | 5 | 3 | 6 | 2 | 0
```

Book 2 §2.2: arrows go left to right. Successors generated alphabetically; ties broken alphabetically.

Run greedy best-first search from S on the graph shown. Expansion order, path and cost?

(1) expansion order = ____
(2) path = ____
(3) cost = ____

Answer: expansion order = S, B, D, G; path = S-B-D-G; cost = 8

Working:

- From S: A h 5, B h 3 → B.
- From B: D h 2 → D.
- From D: G h 0 → G.

Takeaway: Here greedy happens to find the optimum. That is luck, not a guarantee.

Source: Book 2 (Exam examples), Q17, p. 9

SEARCH A70. [BLANKS · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (5); A — C (2); A — D (6); C — G (7); B — D (1); D — G (2)

```
node | S | A | B | C | D | G
h | 6 | 5 | 3 | 6 | 2 | 0
```

Book 2 §2.2: arrows go left to right. Successors generated alphabetically; ties broken alphabetically.

Run A* from S on the graph shown. Give f of each node when it is expanded.

(1) f(S) = ____
(2) f(A) = ____
(3) f(B) = ____
(4) f(D) = ____
(5) f(G) = ____

Answer: f(S) = 6; f(A) = 7; f(B) = 8; f(D) = 8; f(G) = 8

Working:

- S: 0 + 6 = 6
- A: 2 + 5 = 7 → adds C 4 + 6 = 10, D 8 + 2 = 10
- B: 5 + 3 = 8 → D via B 6 + 2 = 8 replaces 10
- D: 8 → G 8 + 0 = 8
- G: 8, path S-B-D-G

Takeaway: B's cheaper path to D (g = 6, f = 8) beats D via A (f = 10).

Source: Book 2 (Exam examples), Q18, p. 10

SEARCH A71. [BLANKS · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (5); A — C (2); A — D (6); C — G (7); B — D (1); D — G (2)

```
node | S | A | B | C | D | G
h | 6 | 5 | 3 | 6 | 2 | 0
```

Book 2 §2.2: arrows go left to right. Successors generated alphabetically; ties broken alphabetically.

Run BFS from S on the graph shown (goal test when generated). Path and cost?

(1) path = ____
(2) cost = ____

Answer: path = S-A-C-G; cost = 11

Working:

- Queue S → A, B.
- Pop A: C, D.
- Pop B: D already queued.
- Pop C: generates G → stop.
- Path S-A-C-G = 2 + 2 + 7 = 11 (3 actions).

Takeaway: BFS minimises the number of actions, not cost.

Source: Book 2 (Exam examples), Q19, p. 10

SEARCH A72. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (5); A — C (2); A — D (6); C — G (7); B — D (1); D — G (2)

```
node | S | A | B | C | D | G
h | 6 | 5 | 3 | 6 | 2 | 0
```

Book 2 §2.2: arrows go left to right. Successors generated alphabetically; ties broken alphabetically.

On the graph shown, which statement about h is true?

A. Admissible but not consistent
B. Neither admissible nor consistent
C. Consistent (hence admissible)
D. Consistent but not admissible

Answer: C. Consistent (hence admissible)

Option notes:

- A: Every edge satisfies h(n) ≤ c + h(n′), e.g. B→D 3 ≤ 1 + 2.
- B: True costs to G: S 8, A 8, B 3, C 7, D 2; every h ≤ h*.
- D: Impossible: consistent always implies admissible.

Takeaway: Consistent ⇒ admissible, never the other way round.

Source: Book 2 (Exam examples), Q20, p. 10

SEARCH A73. [MCQ · Hard]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (5); A — C (2); A — D (6); C — G (7); B — D (1); D — G (2)

```
node | S | A | B | C | D | G
h | 6 | 5 | 3 | 6 | 2 | 0
```

Book 2 §2.2: arrows go left to right. Successors generated alphabetically; ties broken alphabetically.

Same graph, but change h(A) to 7 and h(D) to 0. Now h is:

A. Not admissible
B. Admissible and consistent
C. Admissible but not consistent

Answer: C. Admissible but not consistent

Working:

- h*(A) = 8 ≥ 7 ✓, h*(D) = 2 ≥ 0 ✓ → admissible.
- Edge A→D cost 6: 7 ≤ 6 + 0? No → not consistent.

Option notes:

- A: h*(A) = 8 ≥ 7 and h*(D) = 2 ≥ 0, so it never overestimates.
- B: Edge A→D: h(A) = 7 > 6 + h(D) = 6 breaks consistency.

Takeaway: Check admissibility against true costs, consistency against every edge.

Source: Book 2 (Exam examples), Q21, p. 10

SEARCH A74. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): A — B; A — C; B — D; B — E; C — F; C — G; D — H; D — I; E — J; E — K; F — L; F — M; G — N; G — O

Depth-3 binary tree A..O (A's children B, C; B: D, E; C: F, G; ...). Goal G (the node). BFS expansion order (left child first) until G is expanded:

Answer: A, B, C, D, E, F, G
(Also accepted: ABCDEFG)

Takeaway: BFS goes level by level.

Source: Book 2 (Exam examples), Q22, p. 10

SEARCH A75. [BLANK · Medium]

[Diagram as text] Edges (label in brackets): A — B; A — C; B — D; B — E; C — F; C — G; D — H; D — I; E — J; E — K; F — L; F — M; G — N; G — O

Depth-3 binary tree A..O (A's children B, C; B: D, E; D: H, I; E: J, K; C: F, G; F: L, M; G: N, O). Goal G. DFS expansion order (left child first) until G is expanded:

Answer: A, B, D, H, I, E, J, K, C, F, L, M, G
(Also accepted: ABDHIEJKCFLMG)

Takeaway: DFS dives to the bottom of the left subtree first.

Source: Book 2 (Exam examples), Q23, p. 10

SEARCH A76. [MCQ · Easy]

Which frontier data structure produces the DFS order?

A. Priority queue on h
B. Priority queue on g
C. LIFO stack
D. FIFO queue

Answer: C. LIFO stack

Option notes:

- A: That gives greedy best-first.
- B: That gives UCS.
- D: That gives BFS.

Takeaway: DFS = stack, BFS = queue, best-first = priority queue.

Source: Book 2 (Exam examples), Q24, p. 10

SEARCH A77. [TF · Easy]

UCS is complete and cost-optimal when all step costs are positive.

Answer: True

Why: UCS expands in order of g, so the first goal popped is cheapest.

Takeaway: UCS: complete + optimal (costs ≥ ε > 0).

Source: Book 2 (Exam examples), Q25, p. 10

SEARCH A78. [TF · Medium]

Greedy best-first search is cost-optimal if the heuristic is admissible.

Answer: False

Why: Greedy ignores g. Admissibility makes A* optimal, not greedy (Romania: 450 vs 418).

Takeaway: Admissible h helps A*, not greedy.

Source: Book 2 (Exam examples), Q26, p. 10

SEARCH A79. [TF · Easy]

BFS always returns the cheapest path.

Answer: False

Why: It returns the path with fewest actions; cheapest only if all step costs are equal.

Takeaway: BFS = fewest steps.

Source: Book 2 (Exam examples), Q27, p. 10

SEARCH A80. [TF · Medium]

Every admissible heuristic is consistent.

Answer: False

Why: The converse is true: every consistent heuristic is admissible.

Takeaway: Consistent ⇒ admissible only.

Source: Book 2 (Exam examples), Q28, p. 11

SEARCH A81. [TF · Medium]

Graph-search A* is guaranteed optimal with a consistent heuristic.

Answer: True

Why: Graph search needs consistency; tree search needs only admissibility.

Takeaway: Graph A* → consistent h.

Source: Book 2 (Exam examples), Q29, p. 11

SEARCH A82. [TF · Easy]

DFS uses less memory than BFS.

Answer: True

Why: O(bm) versus O(b^d).

Takeaway: DFS memory is linear in depth.

Source: Book 2 (Exam examples), Q30, p. 11

SEARCH A83. [TF · Easy]

A tree search algorithm keeps an explored set.

Answer: False

Why: That is graph search. Tree search can revisit states.

Takeaway: Explored set = graph search.

Source: Book 2 (Exam examples), Q31, p. 11

SEARCH A84. [MATCH · Easy]

Match the algorithm to its evaluation function.

1. UCS
2. Greedy best-first
3. A*

A. h(n)
B. depth(n)
C. g(n) + h(n)
D. g(n)

Answer: 1 → D, 2 → A, 3 → C

Working:

- UCS → g(n)
- Greedy best-first → h(n)
- A* → g(n) + h(n)

Takeaway: UCS g, Greedy h, A* g + h.

Source: Book 2 (Exam examples), Q32, p. 11

SEARCH A85. [MATCH · Easy]

Match the algorithm to its frontier.

1. BFS
2. DFS
3. A*

A. LIFO stack
B. priority queue
C. hash set
D. FIFO queue

Answer: 1 → D, 2 → A, 3 → B

Working:

- BFS → FIFO queue
- DFS → LIFO stack
- A* → priority queue

Takeaway: Queue, stack, priority queue.

Source: Book 2 (Exam examples), Q33, p. 11

SEARCH A86. [MULTI · Easy]

Which are components of a search-problem definition? (Select ALL that apply.)

A. initial state
B. state space
C. goal test
D. transition model RESULT(s, a)

E. action cost function
F. learning rate

Answer: A. initial state; B. state space; C. goal test; D. transition model RESULT(s, a); E. action cost function

Option notes:

- A: Correct: where the agent starts
- B: Correct: the set of all states
- C: Correct: checks whether a state is a goal
- D: Correct: what each action produces
- E: Correct: c(s, a, s′)
- F: Incorrect: that belongs to gradient descent, not search

Takeaway: Six parts: states, start, goal test, actions, transition, cost.

Source: Book 2 (Exam examples), Q34, p. 11

SEARCH A87. [MCQ · Easy]

The classic search setting assumes the environment is:

A. unknown
B. partially observable, stochastic
C. fully observable, stochastic
D. fully observable, deterministic, known

Answer: D. fully observable, deterministic, known

Option notes:

- A: That is reinforcement learning.
- B: That is the Bayesian network setting.
- C: That is the MDP setting.

Takeaway: Then a fixed sequence of actions is a solution.

Source: Book 2 (Exam examples), Q35, p. 11

SEARCH A88. [BLANK · Medium]

In UCS the goal test is performed when a node is ___ from the frontier (not when it is generated).

Answer: removed
(Also accepted: popped, selected, selected for expansion, popped from the frontier, removed from the frontier, expanded, taken, dequeued)

Takeaway: Late goal test: Bucharest is generated at 450 but popped at 418.

Source: Book 2 (Exam examples), Q36, p. 11

SEARCH A89. [MCQ · Easy]

Why is the straight-line distance heuristic admissible?

A. It is always exactly equal to the road distance
B. It is computed from g(n)
C. A straight line is never longer than the road distance, so it never overestimates
D. It is consistent with UCS

Answer: C. A straight line is never longer than the road distance, so it never overestimates

Option notes:

- A: Then it would be perfect, not just admissible; it is usually shorter.
- B: h is independent of the path travelled.
- D: UCS does not use a heuristic.

Takeaway: Admissible = never overestimates.

Source: Book 2 (Exam examples), Q37, p. 11

SEARCH A90. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

In the Romania problem, how many cities (states) are there?

A. 23
B. 20
C. 10
D. 14

Answer: B. 20

Option notes:

- A: 23 is close to the number of roads, not cities.
- C: Only part of the map.
- D: Count again: 20 cities on the map.

Takeaway: 20 states, 20 × 20 adjacency matrix.

Source: Book 2 (Exam examples), Q38, p. 11

SEARCH A91. [BLANK · Medium]

A* with h(n) = 0 for every n behaves exactly like ___.

Answer: uniform-cost search
(Also accepted: UCS, uniform cost search, uniform-cost, uniform cost)

Takeaway: f = g + 0 = g.

Source: Book 2 (Exam examples), Q39, p. 11

SEARCH A92. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176,

Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374 Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

The slides represent the Romania transition function as a 20 × 20 ___ matrix.

Answer: adjacency
(Also accepted: adjacency matrix, 0/1 adjacency, 0/1, binary adjacency)

Takeaway: Entry 1 = road between the two cities.

Source: Book 2 (Exam examples), Q40, p. 11

SEARCH A93. [MCQ · Easy · calculation]

A* frontier: node X has g = 120, h = 90; node Y has g = 150, h = 50. Which is expanded next?

A. Y
B. cannot tell
C. tie
D. X

Answer: A. Y

Option notes:

- B: Compute f = g + h.
- C: 210 ≠ 200.
- D: f(X) = 210 > f(Y) = 200.

Takeaway: A* picks the smaller f.

Source: Book 2 (Exam examples), Mixed practice paper Q1, p. 54

SEARCH A94. [TF · Medium]

Greedy best-first search with an admissible heuristic is guaranteed to find the cheapest path.

Answer: False

Why: Greedy ignores g (Romania gives 450 instead of 418).

Takeaway: Only A* benefits from admissibility.

Source: Book 2 (Exam examples), Mixed practice paper Q2, p. 54

SEARCH A95. [BLANK · Easy]

A heuristic satisfying $h(n) \leq c(n, a, n') + h(n')$ for all edges is called ___.

Answer: consistent
(Also accepted: monotone, consistent (monotone))

Takeaway: Triangle inequality.

Source: Book 2 (Exam examples), Mixed practice paper Q3, p. 54

SEARCH A96. [MULTI · Medium]

Which are true? (Select ALL that apply.)

A. Q-learning is model-free
B. BFS uses a FIFO queue
C. UCS tests the goal when generating a node

D. ReLU(z) = max(0, z)
E. k-fold CV should use the test set

Answer: A. Q-learning is model-free; B. BFS uses a FIFO queue; D. ReLU(z) = max(0, z)

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: UCS tests on pop
- D: Correct:
- E: Incorrect: training set only

Takeaway: Mixed facts.

Source: Book 2 (Exam examples), Mixed practice paper Q40, p. 55

SEARCH A97. [BLANK · Easy]

What does A* become if h(n) = 0 everywhere?

Answer: UCS
(Also accepted: uniform-cost search, uniform cost search)

Takeaway: f = g.

Source: Book 3 (AI from zero), Practice 9.1, p. 36

SEARCH A98. [BLANKS · Medium · calculation]

Frontier: P (g = 50, h = 40), Q (g = 70, h = 10). Which node does each algorithm expand?

(1) UCS = ____
(2) Greedy = ____
(3) A* = ____

Answer: UCS = P; Greedy = Q; A* = Q

Takeaway: UCS g; greedy h; A* g + h (P 90, Q 80).

Source: Book 3 (AI from zero), Practice 9.2, p. 36

SEARCH A99. [TF · Medium]

h = twice the straight-line distance is admissible.

Answer: False

Why: It can overestimate the true road distance.

Takeaway: Admissible = never overestimates.

Source: Book 3 (AI from zero), Practice 9.3, p. 36

SEARCH A100. [MCQ · Medium]

Why does UCS test for the goal when a node is popped?

A. popping is faster than generating
B. the goal has h = 0
C. a cheaper path to the goal may still be found after the goal is first generated
D. to save memory

Answer: C. a cheaper path to the goal may still be found after the goal is first generated

Option notes:

- A: Speed is not the reason.
- B: UCS uses no h.
- D: Not the reason.

Takeaway: Romania: 450 first, then 418.

Source: Book 3 (AI from zero), Practice 9.4, p. 36

SEARCH A101. [BLANK · Medium · calculation]

BFS memory with b = 3, d = 10 is about ___ nodes.

Answer: 59049
(Also accepted: 59,049, 3^10)

Takeaway: b^d.

Source: Book 3 (AI from zero), Practice 9.5, p. 36

SEARCH A102. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

BFS on Romania (successors alphabetical, goal test when generated) returns which path?

A. Arad–Timisoara–Lugoj–Mehadia–Drobeta–Craiova–Pitesti–Bucharest
B. Arad–Zerind–Oradea–Sibiu–Fagaras–Bucharest
C. Arad–Sibiu–Rimnicu–Pitesti–Bucharest (418)
D. Arad–Sibiu–Fagaras–Bucharest (3 actions, cost 450)

Answer: D. Arad–Sibiu–Fagaras–Bucharest (3 actions, cost 450)

Working:

- Queue [A] → pop A: add S, T, Z.
- Pop S: add F, O, R. Pop T: add L. Pop Z.
- Pop F: generates Bucharest → stop. Cost 140 + 99 + 211 = 450.

Option notes:

- A: Too many actions for BFS.
- B: BFS finds a 3-action path first.
- C: That is the cheapest, found by UCS/A*.

Takeaway: BFS = fewest actions.

Source: Book 3 (AI from zero), Chapter 9, p. 31

SEARCH A103. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80);

Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Romania: the slide lists seven Arad→Bucharest paths. A–Z–O–S–R–P–B costs 75 + 71 + 151 + 80 + 97 + 101 = ___.

Answer: 575

Takeaway: Add every road.

Source: Book 1 (Compiled lecture notes), Chapter 2, p. 11

SEARCH A104. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

UCS on Romania expands in order of g: Arad 0, Zerind 75, Timisoara 118, Sibiu 140, Oradea 146, Rimnicu 220, Lugoj 229, Fagaras 239, Mehadia ___, Pitesti 317, Craiova 366, Drobeta 374, Bucharest 418.

Answer: 299

Takeaway: Lugoj 229 + 70 = 299.

Source: Book 1 (Compiled lecture notes), Chapter 2, p. 11

SEARCH A105. [MATCH · Hard]

Match the algorithm to its time complexity (b branching, d shallowest goal depth, m max depth).

1. BFS
2. DFS
3. UCS

A. O(log n)
B. O(b^d)
C. O(b^(1 + ⌊C*/ε⌋))
D. O(b^m)

Answer: 1 → B, 2 → D, 3 → C

Working:

- BFS → O(b^d)
- DFS → O(b^m)

- UCS → O(b^(1 + ⌊C*/ε⌋))

Takeaway: UCS depends on C*/ε.

Source: Book 1 (Compiled lecture notes), Chapter 2, p. 11

SEARCH A106. [TF · Hard]

Consistency implies admissibility because applying h(n) ≤ c + h(n′) along the optimal path sums to h(n) ≤ h*(n).

Answer: True

Why: Telescoping: $h(n_0) \le c_1 + \ldots + c_k + h(G) = h^*(n)$.

Takeaway: Consistent ⇒ admissible.

Source: Book 3 (AI from zero), Chapter 9, p. 31

SEARCH A107. [BLANK · Medium · calculation]

7 | 2 | 4
5 | | 6
8 | 3 | 1

```
  | 1 | 2
3 | 4 | 5
6 | 7 | 8
```

8-puzzle (shown). h_1 = number of misplaced tiles (blank not counted). h_1(start) = ___.

Answer: 8

Takeaway: Every tile is out of place.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: 8-puzzle heuristics

SEARCH A108. [BLANK · Hard · calculation]

7 | 2 | 4
5 | | 6
8 | 3 | 1

```
  | 1 | 2
3 | 4 | 5
6 | 7 | 8
```

8-puzzle (shown). h_2 = sum of Manhattan distances of the tiles from their goal squares. h_2(start) = ___.

Answer: 18

Working:

- Tile 1: 3, tile 2: 1, tile 3: 2, tile 4: 2
- Tile 5: 2, tile 6: 3, tile 7: 3, tile 8: 2
- Sum = 18

Takeaway: Tiles 1..8: 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18. The true solution cost is 26.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: 8-puzzle heuristics

SEARCH A109. [MULTI · Medium]

8-puzzle heuristics h_1 (misplaced tiles) and h_2 (Manhattan distance). Which are true? (Select ALL that apply.)

A. A* with h_2 never expands more nodes than with h_1 (ignoring ties)
B. both are admissible
C. h_1 overestimates when tiles are far away
D. h_2 dominates h_1 ($h_2(n) \geq h_1(n)$ for every n)

Answer: A. A* with h_2 never expands more nodes than with h_1 (ignoring ties); B. both are admissible; D. h_2 dominates h_1 ($h_2(n) \geq h_1(n)$ for every n)

Option notes:

- A: Correct: a dominating admissible heuristic is better
- B: Correct: a tile needs at least one move if misplaced, and at least its Manhattan distance
- C: Incorrect: it never overestimates
- D: Correct: each misplaced tile contributes ≥ 1 to h_2

Takeaway: (Extra, not in your slides) Both come from relaxed problems.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: heuristic dominance

SEARCH A110. [MCQ · Medium]

(Extra, not in your slides) Where do h_1 and h_2 for the 8-puzzle come from?

A. random guesses that happen to work
B. learning from solved puzzles
C. exact solution costs of relaxed versions of the problem (fewer restrictions on moves)
D. running BFS from the goal

Answer: C. exact solution costs of relaxed versions of the problem (fewer restrictions on moves)

Option notes:

- A: They are principled.
- B: Possible, but not how h_1/h_2 arise.
- D: That gives h*, not h_1/h_2.

Takeaway: Relaxed-problem cost is an admissible heuristic.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: relaxed problems

SEARCH A111. [TF · Medium]

(Extra, not in your slides) If h_a and h_b are both admissible, then h(n) = max(h_a(n), h_b(n)) is admissible and dominates both.

Answer: True

Why: The max of underestimates is still an underestimate, and it is at least as large as each.

Takeaway: Combine heuristics with max.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: composite heuristics

SEARCH A112. [BLANK · Hard · calculation]

(Extra, not in your slides) A* generates N = 52 nodes to find a solution at depth d = 5. Effective branching factor b* satisfies $N + 1 = 1 + b^* + b^{*2} + \ldots + b^{*5}$. b* ≈ ___ (2 d.p.).

Answer: 1.92

Takeaway: A good heuristic has b* close to 1.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: effective branching factor

SEARCH A113. [BLANKS · Hard · calculation]

(Extra, not in your slides) b = 10, d = 5. Nodes generated by iterative deepening: 5·10 + 4·100 + 3·1,000 + 2·10,000 + 1·100,000 = ___; by BFS: 10 + 100 + … + 100,000 = ___.

(1) IDS = ____
(2) BFS = ____

Answer: IDS = 123450; BFS = 111110

Takeaway: Repeating the shallow levels costs little: IDS is the preferred uninformed method for large spaces.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: iterative deepening

SEARCH A114. [MATCH · Medium]

(Extra, not in your slides) Match the uninformed search to its property.

1. iterative deepening DFS
2. depth-limited search
3. bidirectional search

A. DFS memory O(bd) with BFS completeness/optimality for unit costs
B. always expands the deepest node with no limit
C. DFS with a fixed depth limit ℓ; incomplete if ℓ < d
D. two searches meet in the middle: about 2·b^(d/2) nodes

Answer: 1 → A, 2 → C, 3 → D

Working:

- iterative deepening DFS → DFS memory O(bd) with BFS completeness/optimality for unit costs
- depth-limited search → DFS with a fixed depth limit ℓ; incomplete if ℓ < d
- bidirectional search → two searches meet in the middle: about 2·b^(d/2) nodes

Takeaway: AIMA's uninformed family.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: uninformed search

SEARCH A115. [BLANKS · Medium · calculation]

AIMA state-space sizes: the two-cell vacuum world has ___ states; the 8-puzzle has ___ reachable states (9!/2).

(1) vacuum = ____
(2) 8-puzzle = ____

Answer: vacuum = 8; 8-puzzle = 181440

Takeaway: Vacuum: 2 positions × 2^2 dirt patterns.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: example problems

SEARCH A116. [MCQ · Hard · calculation]

(Extra, not in your slides) 8-queens, incremental formulation (add queens one by one to any empty square): about how many sequences must be explored?

A. 2,057
B. 92

C. $8^8 \approx 16.8$ million
D. 1.8×10^{14} $(64 \cdot 63 \cdot \ldots \cdot 57)$

Answer: D. 1.8×10^{14} $(64 \cdot 63 \cdot \ldots \cdot 57)$

Option notes:

- A: That is the improved formulation (one queen per column, never attacked).
- B: That is the number of solutions.
- C: That is the complete-state space for local search.

Takeaway: Formulation matters enormously.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: 8-queens formulations

SEARCH A117. [TF · Hard]

(Extra, not in your slides) Uniform-cost search can get stuck in an infinite loop if there are zero-cost actions leading back to the same state.

Answer: True

Why: Completeness needs every step cost $\geq \varepsilon > 0$.

Takeaway: UCS needs positive costs.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: UCS

SEARCH A118. [MCQ · Medium]

(Extra, not in your slides) Weighted A* uses $f(n) = g(n) + W \cdot h(n)$ with $W > 1$. Compared with A*, it:

A. is exactly greedy search
B. behaves like UCS
C. is always optimal
D. usually expands fewer nodes but may return a path costing up to W times the optimum

Answer: D. usually expands fewer nodes but may return a path costing up to W times the optimum

Option notes:

- A: That is $W \to \infty$.
- B: That is $W = 0$.
- C: Inflating h breaks admissibility.

Takeaway: Trade optimality for speed.

*Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: weighted A**

SEARCH A119. [TF · Medium]

With a consistent heuristic, the f-values along any path expanded by A* are non-decreasing.

Answer: True

Why: $f(n') = g(n) + c + h(n') \geq g(n) + h(n) = f(n)$.

Takeaway: This is why graph-search A* is optimal with consistent h.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: A optimality**

SEARCH A120. [MULTI · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101);

Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

AIMA's Romania problem (same as Lecture 2). Which are true? (Select ALL that apply.)

A. A* with h_SLD finds the 418 route via Rimnicu Vilcea and Pitesti
B. it has 20 cities
C. greedy best-first with h_SLD finds Arad–Sibiu–Fagaras–Bucharest (450)
D. h_SLD can overestimate the road distance

Answer: A. A* with h_SLD finds the 418 route via Rimnicu Vilcea and Pitesti; B. it has 20 cities; C. greedy best-first with h_SLD finds Arad–Sibiu–Fagaras–Bucharest (450)

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Incorrect: a straight line is never longer than a road

Takeaway: Book and lecture agree.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 3: Romania

Part B: calculation drills (176)

SEARCH B1. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (2); A — E (1); B — D (1); B — C (1); C — F (2); D — H (7); E — H (4); F — G (9); H — G (3); B — F (7)
Heuristic values: S h=13, A h=11, B h=7, C h=6, D h=8, E h=5, F h=6, H h=2, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,E,C,D,F,H,G
(Also accepted: S, A, B, E, C, D, F, H, G, SABECDFHG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:2, B:9} → pop lowest = A.
- Step 3: frontier {B:9, E:9} → pop lowest = B.
- Step 4: frontier {E:9, C:10, D:10, F:10} → pop lowest = E.
- Step 5: frontier {C:10, D:10, F:10, H:13} → pop lowest = C.
- Step 6: frontier {D:10, F:10, H:13} → pop lowest = D.
- Step 7: frontier {F:10, H:13} → pop lowest = F.
- Step 8: frontier {H:13, G:19} → pop lowest = H.

- Step 9: frontier {G:16} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,E,C,D,F,H,G. Path found: S-A-E-H-G with cost 16.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B2. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (8); A — E (5); A — C (3); B — E (2); A — D (8); C — H (4); C — F (6); D — F (5); D — H (3); E — F (6); F — G (3); H — G (3); B — F (8)
Heuristic values: S h=14, A h=8, B h=6, C h=6, D h=5, E h=5, F h=2, H h=1, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,C,E,D,H,F,G
(Also accepted: S, A, B, C, E, D, H, F, G, SABCEDHFG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:8, B:9} → pop lowest = A.
- Step 3: frontier {B:9, C:11, D:13, E:16} → pop lowest = B.
- Step 4: frontier {C:11, E:11, D:13, F:17} → pop lowest = C.
- Step 5: frontier {E:11, D:13, H:15, F:17} → pop lowest = E.
- Step 6: frontier {D:13, H:15, F:17} → pop lowest = D.
- Step 7: frontier {H:15, F:17} → pop lowest = H.
- Step 8: frontier {F:17, G:18} → pop lowest = F.
- Step 9: frontier {G:18} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,C,E,D,H,F,G. Path found: S-A-C-H-G with cost 18.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B3. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (7); A — D (4); B — C (5); B — D (4); B — E (1); C — F (9); D — F (8); E — H (6); F — G (9); H — G (2)
Heuristic values: S h=11, A h=11, B h=8, C h=12, D h=7, E h=7, F h=5, H h=1, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,E,D,C,H,G
(Also accepted: S, A, B, E, D, C, H, G, SABEDCHG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:7, B:8} → pop lowest = A.
- Step 3: frontier {B:8, D:11} → pop lowest = B.
- Step 4: frontier {E:9, D:11, C:13} → pop lowest = E.
- Step 5: frontier {D:11, C:13, H:15} → pop lowest = D.
- Step 6: frontier {C:13, H:15, F:19} → pop lowest = C.
- Step 7: frontier {H:15, F:19} → pop lowest = H.
- Step 8: frontier {G:17, F:19} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,E,D,C,H,G. Path found: S-B-E-H-G with cost 17.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B4. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (3); A — E (1); A — C (8); B — D (6); C — F (6); D — F (9); D — H (4); E — H (3); F — G (1); H — G (3); B — H (2)
Heuristic values: S h=5, A h=4, B h=2, C h=3, D h=3, E h=4, F h=0, H h=2, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,E,H,G
(Also accepted: S, A, B, E, H, G, SABEHG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:2, B:3} → pop lowest = A.
- Step 3: frontier {B:3, E:3, C:10} → pop lowest = B.
- Step 4: frontier {E:3, H:5, D:9, C:10} → pop lowest = E.
- Step 5: frontier {H:5, D:9, C:10} → pop lowest = H.
- Step 6: frontier {G:8, D:9, C:10} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,E,H,G. Path found: S-B-H-G with cost 8.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B5. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — C (6); A — D (4); B — D (3); A — E (8); C — H (5); C — F (4); D — H (7); D — F (8); E — H (9); E — F (3); F — G (3); H — G (2); A — F (6)
Heuristic values: S h=9, A h=8, B h=6, C h=5, D h=5, E h=3, F h=2, H h=1, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,D,C,F,E,G
(Also accepted: S, A, B, D, C, F, E, G, SABDCFEG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:1, B:2} → pop lowest = A.
- Step 3: frontier {B:2, D:5, C:7, F:7, E:9} → pop lowest = B.
- Step 4: frontier {D:5, C:7, F:7, E:9} → pop lowest = D.
- Step 5: frontier {C:7, F:7, E:9, H:12} → pop lowest = C.
- Step 6: frontier {F:7, E:9, H:12} → pop lowest = F.
- Step 7: frontier {E:9, G:10, H:12} → pop lowest = E.
- Step 8: frontier {G:10, H:12} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,D,C,F,E,G. Path found: S-A-F-G with cost 10.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B6. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (1); A — C (1); B — D (2); B — E (5); C — F (9); D — F (6); E — F (3); F — G (7); B — F (7)
Heuristic values: S h=10, A h=10, B h=11, C h=14, D h=14, E h=9, F h=4, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,D,E,F,A,C,G
(Also accepted: S, B, D, E, F, A, C, G, SBDEFACG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {B:1, A:9} → pop lowest = B.
- Step 3: frontier {D:3, E:6, F:8, A:9} → pop lowest = D.
- Step 4: frontier {E:6, F:8, A:9} → pop lowest = E.
- Step 5: frontier {F:8, A:9} → pop lowest = F.
- Step 6: frontier {A:9, G:15, C:17} → pop lowest = A.
- Step 7: frontier {C:10, G:15} → pop lowest = C.
- Step 8: frontier {G:15} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,D,E,F,A,C,G. Path found: S-B-F-G with cost 15.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B7. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (4); A — D (3); A — E (8); B — D (2); B — E (9); B — C (9); C — H (5); D — H (1); D — F (1); E — H (5); E — F (2); F — G (6); H — G (4); B — H (6)
Heuristic values: S h=8, A h=5, B h=5, C h=3, D h=4, E h=6, F h=3, H h=2, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,D,H,C,E,F,G
(Also accepted: S, A, B, D, H, C, E, F, G, SABDHCEFG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:4, B:7} → pop lowest = A.
- Step 3: frontier {B:7, D:7, E:12} → pop lowest = B.
- Step 4: frontier {D:7, E:12, H:13, C:16} → pop lowest = D.
- Step 5: frontier {H:8, E:12, F:12, C:16} → pop lowest = H.
- Step 6: frontier {C:9, E:12, F:12, G:12} → pop lowest = C.
- Step 7: frontier {E:12, F:12, G:12} → pop lowest = E.
- Step 8: frontier {F:12, G:12} → pop lowest = F.
- Step 9: frontier {G:12} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,D,H,C,E,F,G. Path found: S-A-D-H-G with cost 12.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B8. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (4); A — C (3); B — D (7); B — E (2); C — F (3); D — F (2); E — F (1); F — G (7); B — F (9)
Heuristic values: S h=12, A h=10, B h=7, C h=9, D h=5, E h=5, F h=5, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,A,E,F,C,D,G
(Also accepted: S, B, A, E, F, C, D, G, SBAEFCDG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {B:4, A:5} → pop lowest = B.
- Step 3: frontier {A:5, E:6, D:11, F:13} → pop lowest = A.
- Step 4: frontier {E:6, C:8, D:11, F:13} → pop lowest = E.

- Step 5: frontier {F:7, C:8, D:11} → pop lowest = F.
- Step 6: frontier {C:8, D:9, G:14} → pop lowest = C.
- Step 7: frontier {D:9, G:14} → pop lowest = D.
- Step 8: frontier {G:14} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,A,E,F,C,D,G. Path found: S-B-E-F-G with cost 14.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B9. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (5); A — C (4); B — C (6); B — D (9); B — E (8); C — F (4); D — F (4); E — F (9); F — G (7); A — F (9)
Heuristic values: S h=12, A h=8, B h=9, C h=8, D h=8, E h=10, F h=6, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,C,E,F,D,G
(Also accepted: S, A, B, C, E, F, D, G, SABCEFDG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:5, B:5} → pop lowest = A.
- Step 3: frontier {B:5, C:9, F:14} → pop lowest = B.
- Step 4: frontier {C:9, E:13, D:14, F:14} → pop lowest = C.
- Step 5: frontier {E:13, F:13, D:14} → pop lowest = E.
- Step 6: frontier {F:13, D:14} → pop lowest = F.
- Step 7: frontier {D:14, G:20} → pop lowest = D.
- Step 8: frontier {G:20} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,C,E,F,D,G. Path found: S-A-C-F-G with cost 20.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B10. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (7); A — D (9); A — C (5); B — D (1); A — E (6); C — F (6); D — F (2); E — F (2); F — G (8); B — F (7)
Heuristic values: S h=13, A h=10, B h=12, C h=12, D h=7, E h=7, F h=7, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,E,B,F,C,D,G
(Also accepted: S, A, E, B, F, C, D, G, SAEBFCDG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:4, B:7} → pop lowest = A.
- Step 3: frontier {E:5, B:7, C:9, D:13} → pop lowest = E.
- Step 4: frontier {B:7, F:7, C:9, D:13} → pop lowest = B.
- Step 5: frontier {F:7, C:9, D:13} → pop lowest = F.
- Step 6: frontier {C:9, D:9, G:15} → pop lowest = C.
- Step 7: frontier {D:9, G:15} → pop lowest = D.
- Step 8: frontier {G:15} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,E,B,F,C,D,G. Path found: S-A-E-F-G with cost 15.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B11. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (8); A — C (4); A — E (9); B — C (2); B — E (5); A — D (2); C — F (6); C — H (3); D — H (5); D — F (2); E — F (7); E — H (6); F — G (4); H — G (7); B — H (2)
Heuristic values: S h=7, A h=7, B h=6, C h=7, D h=5, E h=5, F h=3, H h=3, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,H,E,A,D,F,G
(Also accepted: S, B, C, H, E, A, D, F, G, SBCHEADFG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {B:1, A:8} → pop lowest = B.
- Step 3: frontier {C:3, H:3, E:6, A:8} → pop lowest = C.
- Step 4: frontier {H:3, E:6, A:7, F:9} → pop lowest = H.
- Step 5: frontier {E:6, A:7, D:8, F:9, G:10} → pop lowest = E.
- Step 6: frontier {A:7, D:8, F:9, G:10} → pop lowest = A.
- Step 7: frontier {D:8, F:9, G:10} → pop lowest = D.
- Step 8: frontier {F:9, G:10} → pop lowest = F.
- Step 9: frontier {G:10} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,H,E,A,D,F,G. Path found: S-B-H-G with cost 10.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B12. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (7); A — C (9); A — D (9); B — E (9); B — C (3); C — F (1); D — F (7); E — F (5); F — G (8); B — F (2)
Heuristic values: S h=14, A h=14, B h=8, C h=6, D h=13, E h=11, F h=4, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,F,C,D,E,G
(Also accepted: S, A, B, F, C, D, E, G, SABFCDEG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:5, B:7} → pop lowest = A.
- Step 3: frontier {B:7, C:14, D:14} → pop lowest = B.
- Step 4: frontier {F:9, C:10, D:14, E:16} → pop lowest = F.
- Step 5: frontier {C:10, D:14, E:14, G:17} → pop lowest = C.
- Step 6: frontier {D:14, E:14, G:17} → pop lowest = D.
- Step 7: frontier {E:14, G:17} → pop lowest = E.
- Step 8: frontier {G:17} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,F,C,D,E,G. Path found: S-B-F-G with cost 17.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B13. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (3); A — D (7); B — C (9); A — E (7); C — H (1); C — F (3); D — F (8); E — H (3); E — F (6); F — G (2); H — G (4)
Heuristic values: S h=12, A h=8, B h=9, C h=3, D h=3, E h=6, F h=1, H h=3, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,A,C,F,D,G
(Also accepted: S, B, A, C, F, D, G, SBACFDG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {B:3, A:9} → pop lowest = B.
- Step 3: frontier {A:9, C:10} → pop lowest = A.
- Step 4: frontier {C:10, D:16, E:18} → pop lowest = C.
- Step 5: frontier {F:13, D:16, E:18, H:18} → pop lowest = F.

- Step 6: frontier {D:14, G:15, E:18, H:18} → pop lowest = D.
- Step 7: frontier {G:15, E:18, H:18} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,A,C,F,D,G. Path found: S-B-C-F-G with cost 15.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B14. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (1); A — D (9); A — C (2); B — C (2); B — E (6); C — F (8); C — H (1); D — H (1); E — F (3); F — G (5); H — G (6); B — F (7)
Heuristic values: S h=8, A h=5, B h=8, C h=6, D h=6, E h=5, F h=3, H h=4, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,H,A,D,E,F,G
(Also accepted: S, B, C, H, A, D, E, F, G, SBCHADEFG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {B:1, A:8} → pop lowest = B.
- Step 3: frontier {C:3, E:7, A:8, F:8} → pop lowest = C.
- Step 4: frontier {H:4, A:5, E:7, F:8} → pop lowest = H.
- Step 5: frontier {A:5, D:7, E:7, F:8, G:10} → pop lowest = A.
- Step 6: frontier {D:7, E:7, F:8, G:10} → pop lowest = D.
- Step 7: frontier {E:7, F:8, G:10} → pop lowest = E.
- Step 8: frontier {F:8, G:10} → pop lowest = F.
- Step 9: frontier {G:10} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,H,A,D,E,F,G. Path found: S-B-C-H-G with cost 10.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B15. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (4); S — A (7); A — E (9); B — D (9); A — C (8); C — F (1); D — F (9); E — F (9); F — G (2); B — F (9)
Heuristic values: S h=14, A h=7, B h=9, C h=1, D h=1, E h=10, F h=1, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,A,D,F,C,G
(Also accepted: S, B, A, D, F, C, G, SBADFCG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {B:4, A:7} → pop lowest = B.
- Step 3: frontier {A:7, D:13, F:13} → pop lowest = A.
- Step 4: frontier {D:13, F:13, C:15, E:16} → pop lowest = D.
- Step 5: frontier {F:13, C:15, E:16} → pop lowest = F.
- Step 6: frontier {C:14, G:15, E:16} → pop lowest = C.
- Step 7: frontier {G:15, E:16} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,A,D,F,C,G. Path found: S-B-F-G with cost 15.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B16. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — D (5); B — E (9); B — D (8); A — C (1); C — F (3); D — F (6); E — F (8); F — G (1); A — F (7)
Heuristic values: S h=5, A h=3, B h=14, C h=2, D h=6, E h=7, F h=0, G h=0

Run Uniform-Cost Search (f(n) = g(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,C,F,D,G
(Also accepted: S, A, C, F, D, G, SACFDG)

Working:

- Evaluation function: f(n) = g(n).
- Step 1: frontier {S:0} → pop lowest = S.
- Step 2: frontier {A:1, B:9} → pop lowest = A.
- Step 3: frontier {C:2, D:6, F:8, B:9} → pop lowest = C.
- Step 4: frontier {F:5, D:6, B:9} → pop lowest = F.
- Step 5: frontier {D:6, G:6, B:9, E:13} → pop lowest = D.
- Step 6: frontier {G:6, B:9, E:13} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,C,F,D,G. Path found: S-A-C-F-G with cost 6.

Takeaway: UCS looks Back: always pop the smallest g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B17. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (4); A — C (7); A — E (9); B — D (7); B — E (8); C — F (8); C — H (2); D — F (7); E — H (6); F — G (1); H — G (1); B — H (4)
Heuristic values: S h=6, A h=9, B h=2, C h=2, D h=2, E h=5, F h=0, H h=0, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,H,G
(Also accepted: S, B, H, G, SBHG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:6} → pop lowest = S.
- Step 2: frontier {B:2, A:9} → pop lowest = B.
- Step 3: frontier {H:0, E:5, D:6, A:9} → pop lowest = H.
- Step 4: frontier {G:0, C:2, E:5, D:6, A:9} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,H,G. Path found: S-B-H-G with cost 9.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B18. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (4); A — C (7); A — E (1); B — C (6); B — D (9); C — F (9); D — F (7); E — F (6); F — G (9)
Heuristic values: S h=16, A h=11, B h=14, C h=12, D h=12, E h=10, F h=8, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,E,F,G
(Also accepted: S, A, E, F, G, SAEFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:16} → pop lowest = S.
- Step 2: frontier {A:11, B:14} → pop lowest = A.
- Step 3: frontier {E:10, C:12, B:14} → pop lowest = E.
- Step 4: frontier {F:8, C:12, B:14} → pop lowest = F.
- Step 5: frontier {G:0, C:12, B:14, D:15} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,E,F,G. Path found: S-A-E-F-G with cost 20.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B19. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — D (1); A — E (9); B — D (7); B — E (2); A — C (1); C — F (7); D — F (6); E — F (5); F — G (3)
Heuristic values: S h=12, A h=5, B h=9, C h=7, D h=6, E h=6, F h=1, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,D,F,G
(Also accepted: S, A, D, F, G, SADFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:12} → pop lowest = S.
- Step 2: frontier {A:5, B:9} → pop lowest = A.
- Step 3: frontier {D:6, E:6, C:7, B:9} → pop lowest = D.
- Step 4: frontier {F:1, E:6, C:7, B:9} → pop lowest = F.
- Step 5: frontier {G:0, E:6, C:7, B:9} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,D,F,G. Path found: S-A-D-F-G with cost 16.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B20. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — E (8); A — D (6); B — D (4); B — C (1); C — H (1); C — F (2); D — F (3); E — H (9); F — G (7); H — G (2); B — F (4)
Heuristic values: S h=4, A h=4, B h=2, C h=1, D h=6, E h=10, F h=4, H h=1, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,H,G
(Also accepted: S, B, C, H, G, SBCHG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:4} → pop lowest = S.
- Step 2: frontier {B:2, A:4} → pop lowest = B.
- Step 3: frontier {C:1, A:4, F:4, D:6} → pop lowest = C.
- Step 4: frontier {H:1, A:4, F:4, D:6} → pop lowest = H.
- Step 5: frontier {G:0, A:4, F:4, D:6, E:10} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,H,G. Path found: S-B-C-H-G with cost 6.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B21. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (2); A — D (1); A — E (2); B — C (4); B — D (6); C — F (8); D — F (2); E — F (9); F — G (3); A — F (4)
Heuristic values: S h=6, A h=3, B h=6, C h=8, D h=3, E h=3, F h=2, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node

is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,F,G
(Also accepted: S, A, F, G, SAFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:6} → pop lowest = S.
- Step 2: frontier {A:3, B:6} → pop lowest = A.
- Step 3: frontier {F:2, D:3, E:5, B:6} → pop lowest = F.
- Step 4: frontier {G:0, D:3, E:5, B:6, C:8} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,F,G. Path found: S-A-F-G with cost 9.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B22. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (2); A — E (1); B — D (6); B — E (8); B — C (6); C — F (5); C — H (1); D — H (7); E — F (9); E — H (4); F — G (7); H — G (2)
Heuristic values: S h=8, A h=8, B h=5, C h=1, D h=1, E h=4, F h=6, H h=1, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,H,G
(Also accepted: S, B, C, H, G, SBCHG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:8} → pop lowest = S.
- Step 2: frontier {B:5, A:8} → pop lowest = B.
- Step 3: frontier {C:1, E:4, A:8, D:8} → pop lowest = C.
- Step 4: frontier {H:1, E:4, F:6, A:8, D:8} → pop lowest = H.
- Step 5: frontier {G:0, E:4, F:6, A:8, D:8} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,H,G. Path found: S-B-C-H-G with cost 12.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B23. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (1); A — D (8); B — E (1); B — C (8); C — H (9); C — F (6); D — F (9); D — H (1); E — F (5); F — G (5); H — G (5); B — F (3)
Heuristic values: S h=6, A h=7, B h=5, C h=9, D h=8, E h=5, F h=4, H h=3, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node

is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,F,G
(Also accepted: S, B, F, G, SBFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:6} → pop lowest = S.
- Step 2: frontier {B:5, A:7} → pop lowest = B.
- Step 3: frontier {F:4, E:5, A:7, C:9} → pop lowest = F.
- Step 4: frontier {G:0, E:5, A:7, D:8, C:9} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,F,G. Path found: S-B-F-G with cost 9.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B24. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (9); A — C (4); B — D (4); B — E (1); C — F (3); C — H (8); D — H (6); D — F (3); E — H (2); E — F (2); F — G (4); H — G (4)
Heuristic values: S h=12, A h=6, B h=4, C h=6, D h=4, E h=4, F h=2, H h=3, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,E,F,G
(Also accepted: S, B, E, F, G, SBEFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:12} → pop lowest = S.
- Step 2: frontier {B:4, A:6} → pop lowest = B.
- Step 3: frontier {E:3, D:4, A:6} → pop lowest = E.
- Step 4: frontier {F:2, H:3, D:4, A:6} → pop lowest = F.
- Step 5: frontier {G:0, H:3, D:4, A:6, C:6} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,E,F,G. Path found: S-B-E-F-G with cost 16.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B25. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (2); A — C (9); A — D (9); B — E (1); B — C (5); C — F (2); C — H (2); D — F (5); D — H (9); E — F (6); E — H (3); F — G (4); H — G (8); B — H (6)
Heuristic values: S h=10, A h=13, B h=9, C h=5, D h=5, E h=7, F h=3, H h=5, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node

is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,F,G
(Also accepted: S, B, C, F, G, SBCFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:10} → pop lowest = S.
- Step 2: frontier {B:9, A:13} → pop lowest = B.
- Step 3: frontier {C:5, H:5, E:7, A:13} → pop lowest = C.
- Step 4: frontier {F:3, H:5, E:7, A:13} → pop lowest = F.
- Step 5: frontier {G:0, D:5, H:5, E:7, A:13} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,F,G. Path found: S-B-C-F-G with cost 12.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B26. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (3); A — E (7); B — C (4); B — D (4); C — H (5); D — H (6); E — H (6); E — F (5); F — G (9); H — G (8)
Heuristic values: S h=19, A h=17, B h=16, C h=10, D h=12, E h=12, F h=5, H h=6, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,H,G
(Also accepted: S, B, C, H, G, SBCHG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:19} → pop lowest = S.
- Step 2: frontier {B:16, A:17} → pop lowest = B.
- Step 3: frontier {C:10, D:12, A:17} → pop lowest = C.
- Step 4: frontier {H:6, D:12, A:17} → pop lowest = H.
- Step 5: frontier {G:0, D:12, E:12, A:17} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,H,G. Path found: S-B-C-H-G with cost 24.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B27. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — E (4); A — C (8); B — C (2); B — E (9); A — D (1); C — H (5); D — F (5); D — H (3); E — H (6); E — F (4); F — G (4); H — G (5); A — F (5)
Heuristic values: S h=8, A h=5, B h=9, C h=7, D h=7, E h=7, F h=3, H h=4, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,F,G
(Also accepted: S, A, F, G, SAFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:8} → pop lowest = S.
- Step 2: frontier {A:5, B:9} → pop lowest = A.
- Step 3: frontier {F:3, E:5, C:7, D:7, B:9} → pop lowest = F.
- Step 4: frontier {G:0, E:5, C:7, D:7, B:9} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,F,G. Path found: S-A-F-G with cost 10.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B28. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (7); A — E (9); A — D (2); B — D (5); B — C (2); C — F (3); D — F (3); E — F (5); F — G (6)
Heuristic values: S h=15, A h=12, B h=6, C h=7, D h=7, E h=7, F h=5, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,F,G
(Also accepted: S, B, C, F, G, SBCFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:15} → pop lowest = S.
- Step 2: frontier {B:6, A:12} → pop lowest = B.
- Step 3: frontier {C:7, D:7, A:12} → pop lowest = C.
- Step 4: frontier {F:5, D:7, A:12} → pop lowest = F.
- Step 5: frontier {G:0, E:6, D:7, A:12} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,F,G. Path found: S-B-C-F-G with cost 17.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B29. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (5); A — D (1); A — C (7); B — E (7); C — F (6); D — F (6); E — F (9); F — G (7)
Heuristic values: S h=19, A h=9, B h=18, C h=11, D h=10, E h=15, F h=4, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,D,F,G
(Also accepted: S, A, D, F, G, SADFG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:19} → pop lowest = S.
- Step 2: frontier {A:9, B:18} → pop lowest = A.
- Step 3: frontier {D:10, C:11, B:18} → pop lowest = D.
- Step 4: frontier {F:4, C:11, B:18} → pop lowest = F.
- Step 5: frontier {G:0, C:11, E:15, B:18} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,D,F,G. Path found: S-A-D-F-G with cost 20.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B30. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (6); A — E (2); B — C (4); B — D (3); C — F (3); D — F (6); D — H (7); E — F (9); F — G (6); H — G (5); B — H (7)
Heuristic values: S h=11, A h=12, B h=7, C h=5, D h=5, E h=9, F h=5, H h=3, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,H,G
(Also accepted: S, B, H, G, SBHG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:11} → pop lowest = S.
- Step 2: frontier {B:7, A:12} → pop lowest = B.
- Step 3: frontier {H:3, C:5, D:11, A:12} → pop lowest = H.
- Step 4: frontier {G:0, C:5, D:11, A:12} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,H,G. Path found: S-B-H-G with cost 15.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B31. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (3); A — D (7); A — E (5); B — D (9); B — C (1); C — F (1); C — H (1); D — H (8); E — F (8); E — H (7); F — G (8); H — G (8); B — F (1)
Heuristic values: S h=14, A h=13, B h=7, C h=5, D h=13, E h=13, F h=6, H h=5, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,H,G
(Also accepted: S, B, C, H, G, SBCHG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:14} → pop lowest = S.
- Step 2: frontier {B:7, A:13} → pop lowest = B.
- Step 3: frontier {C:5, F:6, A:13, D:13} → pop lowest = C.
- Step 4: frontier {H:5, F:6, A:13, D:13} → pop lowest = H.
- Step 5: frontier {G:0, F:6, E:8, A:13, D:13} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,H,G. Path found: S-B-C-H-G with cost 16.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B32. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (3); A — E (1); A — C (7); B — E (7); B — C (6); B — D (9); C — F (4); C — H (4); D — F (7); E — F (3); E — H (3); F — G (7); H — G (1)
Heuristic values: S h=14, A h=10, B h=8, C h=7, D h=9, E h=3, F h=6, H h=0, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,E,H,G
(Also accepted: S, B, E, H, G, SBEHG)

Working:

- Evaluation function: f(n) = h(n).
- Step 1: frontier {S:14} → pop lowest = S.
- Step 2: frontier {B:8, A:10} → pop lowest = B.
- Step 3: frontier {E:3, C:7, D:9, A:10} → pop lowest = E.
- Step 4: frontier {H:0, F:6, C:7, D:9, A:10} → pop lowest = H.
- Step 5: frontier {G:0, F:6, C:7, D:9, A:10} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,E,H,G. Path found: S-B-E-H-G with cost 19.

Takeaway: Greedy looks Forward: always pop the smallest h, ignoring g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B33. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (5); A — D (9); B — E (5); B — C (1); C — F (6); D — F (6); E — F (9); F — G (3)

Heuristic values: S h=12, A h=14, B h=7, C h=6, D h=8, E h=7, F h=2, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,F,A,G
(Also accepted: S, B, C, F, A, G, SBCFAG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:12} → pop lowest = S.
- Step 2: frontier {B:16, A:19} → pop lowest = B.
- Step 3: frontier {C:16, A:19, E:21} → pop lowest = C.
- Step 4: frontier {F:18, A:19, E:21} → pop lowest = F.
- Step 5: frontier {A:19, G:19, E:21, D:30} → pop lowest = A.
- Step 6: frontier {G:19, E:21, D:22} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,F,A,G. Path found: S-B-C-F-G with cost 19.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B34. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (8); A — C (3); B — D (9); B — E (5); C — H (1); D — H (4); E — H (8); E — F (4); F — G (1); H — G (9); B — H (6)
Heuristic values: S h=11, A h=11, B h=6, C h=10, D h=10, E h=1, F h=0, H h=6, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,E,F,G
(Also accepted: S, B, E, F, G, SBEFG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:11} → pop lowest = S.
- Step 2: frontier {B:14, A:19} → pop lowest = B.
- Step 3: frontier {E:14, A:19, H:20, D:27} → pop lowest = E.
- Step 4: frontier {F:14, A:19, H:20, D:27} → pop lowest = F.
- Step 5: frontier {G:15, A:19, H:20, D:27} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,E,F,G. Path found: S-B-E-F-G with cost 15.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B35. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (7); A — E (7); A — D (3); B — D (1); A — C (4); C — F (5); D — F (9); E — F (6); F — G (3); A — F (5)
Heuristic values: S h=16, A h=5, B h=6, C h=4, D h=9, E h=7, F h=2, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,A,F,C,D,G
(Also accepted: S, B, A, F, C, D, G, SBAFCDG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:16} → pop lowest = S.
- Step 2: frontier {B:13, A:14} → pop lowest = B.
- Step 3: frontier {A:14, D:17} → pop lowest = A.
- Step 4: frontier {F:16, C:17, D:17, E:23} → pop lowest = F.
- Step 5: frontier {C:17, D:17, G:17, E:23} → pop lowest = C.
- Step 6: frontier {D:17, G:17, E:23} → pop lowest = D.
- Step 7: frontier {G:17, E:23} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,A,F,C,D,G. Path found: S-A-F-G with cost 17.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B36. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (5); A — E (1); A — D (7); B — D (1); B — C (3); C — H (5); C — F (8); D — H (5); D — F (1); E — H (6); E — F (9); F — G (7); H — G (8)
Heuristic values: S h=16, A h=10, B h=7, C h=7, D h=7, E h=13, F h=4, H h=6, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,D,C,F,H,G
(Also accepted: S, B, D, C, F, H, G, SBDCFHG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:16} → pop lowest = S.
- Step 2: frontier {B:12, A:18} → pop lowest = B.
- Step 3: frontier {D:14, C:15, A:18} → pop lowest = D.
- Step 4: frontier {C:15, F:15, H:17, A:18} → pop lowest = C.
- Step 5: frontier {F:15, H:15, A:18} → pop lowest = F.
- Step 6: frontier {H:15, A:18, G:18, E:33} → pop lowest = H.

- Step 7: frontier {G:17, A:18, E:28} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,D,C,F,H,G. Path found: S-B-C-H-G with cost 17.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B37. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (1); A — E (6); B — E (8); B — C (4); A — D (4); C — F (7); D — F (9); E — H (3); F — G (1); H — G (2); B — F (6)
Heuristic values: S h=9, A h=8, B h=5, C h=7, D h=5, E h=5, F h=0, H h=1, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,E,H,G
(Also accepted: S, A, E, H, G, SAEHG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:9} → pop lowest = S.
- Step 2: frontier {A:9, B:14} → pop lowest = A.
- Step 3: frontier {E:8, D:12, B:14} → pop lowest = E.
- Step 4: frontier {H:9, D:12, B:14} → pop lowest = H.
- Step 5: frontier {G:10, D:12, B:14} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,E,H,G. Path found: S-A-E-H-G with cost 10.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B38. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (6); A — E (5); B — E (6); B — C (6); B — D (2); C — H (6); D — H (2); D — F (3); E — H (9); E — F (1); F — G (3); H — G (8); A — F (5)
Heuristic values: S h=11, A h=5, B h=5, C h=8, D h=5, E h=2, F h=2, H h=7, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,A,E,F,G
(Also accepted: S, B, A, E, F, G, SBAEFG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:11} → pop lowest = S.
- Step 2: frontier {B:11, A:14} → pop lowest = B.
- Step 3: frontier {A:14, E:14, D:16, C:20} → pop lowest = A.

- Step 4: frontier {E:13, D:16, F:16, C:20} → pop lowest = E.
- Step 5: frontier {F:14, D:16, C:20, H:27} → pop lowest = F.
- Step 6: frontier {G:15, D:16, C:20, H:27} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,A,E,F,G. Path found: S-A-E-F-G with cost 15.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B39. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (5); A — E (6); B — D (8); A — C (9); C — F (1); C — H (7); D — H (8); E — F (6); F — G (8); H — G (5); A — F (5)
Heuristic values: S h=10, A h=7, B h=12, C h=5, D h=12, E h=10, F h=6, H h=3, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,F,C,B,G
(Also accepted: S, A, F, C, B, G, SAFCBG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:10} → pop lowest = S.
- Step 2: frontier {A:11, B:17} → pop lowest = A.
- Step 3: frontier {F:15, B:17, C:18, E:22} → pop lowest = F.
- Step 4: frontier {C:15, B:17, G:17, E:22} → pop lowest = C.
- Step 5: frontier {B:17, G:17, H:20, E:22} → pop lowest = B.
- Step 6: frontier {G:17, H:20, E:22, D:23} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,F,C,B,G. Path found: S-A-F-G with cost 17.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B40. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (9); A — C (3); A — D (5); B — E (6); B — C (7); C — F (4); D — H (9); E — F (3); F — G (6); H — G (5)
Heuristic values: S h=13, A h=9, B h=10, C h=8, D h=9, E h=8, F h=4, H h=4, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,C,F,G
(Also accepted: S, A, C, F, G, SACFG)

Working:

- Evaluation function: f(n) = g(n) + h(n).

- Step 1: frontier {S:13} → pop lowest = S.
- Step 2: frontier {A:14, B:19} → pop lowest = A.
- Step 3: frontier {C:16, B:19, D:19} → pop lowest = C.
- Step 4: frontier {F:16, B:19, D:19} → pop lowest = F.
- Step 5: frontier {G:18, B:19, D:19, E:23} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,C,F,G. Path found: S-A-C-F-G with cost 18.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B41. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (5); A — D (3); A — E (1); B — D (4); A — C (1); C — H (9); C — F (5); D — F (5); D — H (1); E — F (9); E — H (2); F — G (4); H — G (8)
Heuristic values: S h=17, A h=7, B h=10, C h=8, D h=7, E h=6, F h=3, H h=5, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,A,D,H,E,F,C,G
(Also accepted: S, B, A, D, H, E, F, C, G, SBADHEFCG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:17} → pop lowest = S.
- Step 2: frontier {B:15, A:16} → pop lowest = B.
- Step 3: frontier {A:16, D:16} → pop lowest = A.
- Step 4: frontier {D:16, E:16, C:18} → pop lowest = D.
- Step 5: frontier {H:15, E:16, F:17, C:18} → pop lowest = H.
- Step 6: frontier {E:16, F:17, C:18, G:18} → pop lowest = E.
- Step 7: frontier {F:17, C:18, G:18} → pop lowest = F.
- Step 8: frontier {C:18, G:18} → pop lowest = C.
- Step 9: frontier {G:18} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,A,D,H,E,F,C,G. Path found: S-B-D-H-G with cost 18.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B42. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (5); S — A (8); A — D (9); A — C (9); B — C (3); B — E (1); C — F (5); D — F (2); E — F (4); F — G (7); B — F (7)
Heuristic values: S h=12, A h=11, B h=10, C h=10, D h=7, E h=7, F h=4, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,E,F,G
(Also accepted: S, B, E, F, G, SBEFG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:12} → pop lowest = S.
- Step 2: frontier {B:15, A:19} → pop lowest = B.
- Step 3: frontier {E:13, F:16, C:18, A:19} → pop lowest = E.
- Step 4: frontier {F:14, C:18, A:19} → pop lowest = F.
- Step 5: frontier {G:17, C:18, A:19, D:19} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,E,F,G. Path found: S-B-E-F-G with cost 17.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B43. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (5); A — C (2); B — D (1); A — E (1); C — H (5); D — F (3); E — H (9); E — F (7); F — G (3); H — G (6); A — F (2)
Heuristic values: S h=7, A h=3, B h=5, C h=3, D h=3, E h=4, F h=2, H h=5, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,B,D,F,C,E,G
(Also accepted: S, A, B, D, F, C, E, G, SABDFCEG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:7} → pop lowest = S.
- Step 2: frontier {A:8, B:8} → pop lowest = A.
- Step 3: frontier {B:8, F:9, C:10, E:10} → pop lowest = B.
- Step 4: frontier {D:7, F:9, C:10, E:10} → pop lowest = D.
- Step 5: frontier {F:9, C:10, E:10} → pop lowest = F.
- Step 6: frontier {C:10, E:10, G:10} → pop lowest = C.
- Step 7: frontier {E:10, G:10, H:19} → pop lowest = E.
- Step 8: frontier {G:10, H:19} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,B,D,F,C,E,G. Path found: S-A-F-G with cost 10.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B44. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (4); A — C (6); B — D (3); B — E (4); C — H (4); C — F (6); D — F (8); D — H (6); E — F (4); E — H (8); F — G (2); H — G (1)
Heuristic values: S h=12, A h=12, B h=5, C h=5, D h=5, E h=7, F h=1, H h=0, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,D,H,A,C,F,G
(Also accepted: S, B, D, H, A, C, F, G, SBDHACFG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:12} → pop lowest = S.
- Step 2: frontier {B:13, A:16} → pop lowest = B.
- Step 3: frontier {D:13, A:16, E:19} → pop lowest = D.
- Step 4: frontier {H:15, A:16, F:16, E:19} → pop lowest = H.
- Step 5: frontier {A:16, F:16, G:16, E:19, C:28} → pop lowest = A.
- Step 6: frontier {C:15, F:16, G:16, E:19} → pop lowest = C.
- Step 7: frontier {F:16, G:16, E:19} → pop lowest = F.
- Step 8: frontier {G:16, E:19} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,D,H,A,C,F,G. Path found: S-B-D-H-G with cost 16.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B45. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (5); A — C (7); B — C (6); B — E (8); A — D (8); C — H (7); C — F (4); D — F (5); E — H (2); F — G (7); H — G (2); A — H (7)
Heuristic values: S h=9, A h=8, B h=11, C h=6, D h=9, E h=2, F h=6, H h=1, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,H,B,E,G
(Also accepted: S, A, H, B, E, G, SAHBEG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:9} → pop lowest = S.
- Step 2: frontier {A:15, B:16} → pop lowest = A.
- Step 3: frontier {H:15, B:16, C:20, D:22} → pop lowest = H.
- Step 4: frontier {B:16, G:16, E:18, C:20, D:22} → pop lowest = B.
- Step 5: frontier {E:15, G:16, C:19, D:22} → pop lowest = E.
- Step 6: frontier {G:16, C:19, D:22} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,H,B,E,G. Path found: S-A-H-G with cost 16.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B46. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (4); A — C (8); B — D (3); B — C (3); A — E (7); C — F (3); D — F (6); E — F (5); F — G (5)
Heuristic values: S h=10, A h=15, B h=9, C h=4, D h=7, E h=9, F h=3, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,C,F,D,G
(Also accepted: S, B, C, F, D, G, SBCFDG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:10} → pop lowest = S.
- Step 2: frontier {B:13, A:16} → pop lowest = B.
- Step 3: frontier {C:11, D:14, A:16} → pop lowest = C.
- Step 4: frontier {F:13, D:14, A:16} → pop lowest = F.
- Step 5: frontier {D:14, G:15, A:16, E:24} → pop lowest = D.
- Step 6: frontier {G:15, A:16, E:24} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,C,F,D,G. Path found: S-B-C-F-G with cost 15.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B47. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (1); A — C (7); A — E (9); B — E (6); B — D (2); C — F (3); C — H (2); D — F (8); E — H (6); E — F (3); F — G (9); H — G (9); A — H (1)
Heuristic values: S h=10, A h=6, B h=9, C h=11, D h=6, E h=6, F h=8, H h=8, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,B,D,F,G
(Also accepted: S, B, D, F, G, SBDFG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:10} → pop lowest = S.
- Step 2: frontier {B:10, A:15} → pop lowest = B.
- Step 3: frontier {D:9, A:15, E:18} → pop lowest = D.
- Step 4: frontier {F:13, A:15, E:18} → pop lowest = F.
- Step 5: frontier {G:14, A:15, E:18, C:19} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,B,D,F,G. Path found: S-B-D-F-G with cost 14.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B48. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (6); A — D (1); B — E (8); B — C (2); C — F (8); D — F (9); E — F (7); F — G (9); A — F (1)
Heuristic values: S h=13, A h=8, B h=17, C h=10, D h=6, E h=6, F h=6, G h=0

Run A* search (f(n) = g(n) + h(n)) from S to G on the graph below (edge costs on edges, h under each node). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

List the nodes in the order they are selected for expansion, including G, as letters separated by commas (e.g. S,A,C,G).

Answer: S,A,D,F,G
(Also accepted: S, A, D, F, G, SADFG)

Working:

- Evaluation function: f(n) = g(n) + h(n).
- Step 1: frontier {S:13} → pop lowest = S.
- Step 2: frontier {A:14, B:20} → pop lowest = A.
- Step 3: frontier {D:13, F:13, B:20} → pop lowest = D.
- Step 4: frontier {F:13, B:20} → pop lowest = F.
- Step 5: frontier {G:16, B:20, E:24, C:25} → pop lowest = G (goal test passes when selected, stop).
- Expansion order: S,A,D,F,G. Path found: S-A-F-G with cost 16.

Takeaway: A* looks Both ways: pop the smallest g + h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B49. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (4); A — C (7); A — E (9); B — D (7); B — E (8); C — F (8); C — H (2); D — F (7); E — H (6); F — G (1); H — G (1); B — H (4)
Heuristic values: S h=6, A h=9, B h=2, C h=2, D h=2, E h=5, F h=0, H h=0, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-H-G; Cost = 9

Working:

- Step 1: frontier {S:6} → expand S.
- Step 2: frontier {B:6, A:13} → expand B.
- Step 3: frontier {H:8, A:13, D:17, E:17} → expand H.
- Step 4: frontier {G:9, C:12, A:13, D:17, E:17} → expand G.
- G is selected with g = 9. Follow parent pointers: S-B-H-G.
- Cost check: 4 + 4 + 1 = 9.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B50. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (4); A — C (7); A — E (1); B — C (6); B — D (9); C — F (9); D — F (7); E — F (6); F — G (9)
Heuristic values: S h=16, A h=11, B h=14, C h=12, D h=12, E h=10, F h=8, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-E-F-G; Cost = 20

Working:

- Step 1: frontier {S:16} → expand S.
- Step 2: frontier {A:15, B:15} → expand A.
- Step 3: frontier {B:15, E:15, C:23} → expand B.
- Step 4: frontier {E:15, C:19, D:25} → expand E.
- Step 5: frontier {C:19, F:19, D:25} → expand C.
- Step 6: frontier {F:19, D:25} → expand F.
- Step 7: frontier {G:20, D:25} → expand G.
- G is selected with g = 20. Follow parent pointers: S-A-E-F-G.
- Cost check: 4 + 1 + 6 + 9 = 20.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B51. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — D (1); A — E (9); B — D (7); B — E (2); A — C (1); C — F (7); D — F (6); E — F (5); F — G (3)
Heuristic values: S h=12, A h=5, B h=9, C h=7, D h=6, E h=6, F h=1, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-D-F-G; Cost = 16

Working:

- Step 1: frontier {S:12} → expand S.
- Step 2: frontier {A:11, B:15} → expand A.
- Step 3: frontier {D:13, C:14, B:15, E:21} → expand D.
- Step 4: frontier {C:14, F:14, B:15, E:21} → expand C.
- Step 5: frontier {F:14, B:15, E:21} → expand F.
- Step 6: frontier {B:15, G:16, E:21} → expand B.
- Step 7: frontier {E:14, G:16} → expand E.

- Step 8: frontier {G:16} → expand G.
- G is selected with g = 16. Follow parent pointers: S-A-D-F-G.
- Cost check: 6 + 1 + 6 + 3 = 16.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B52. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — E (8); A — D (6); B — D (4); B — C (1); C — H (1); C — F (2); D — F (3); E — H (9); F — G (7); H — G (2); B — F (4)
Heuristic values: S h=4, A h=4, B h=2, C h=1, D h=6, E h=10, F h=4, H h=1, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-C-H-G; Cost = 6

Working:

- Step 1: frontier {S:4} → expand S.
- Step 2: frontier {B:4, A:5} → expand B.
- Step 3: frontier {C:4, A:5, F:10, D:16} → expand C.
- Step 4: frontier {A:5, H:5, F:9, D:16} → expand A.
- Step 5: frontier {H:5, F:9, D:13, E:15} → expand H.
- Step 6: frontier {G:6, F:9, D:13, E:15} → expand G.
- G is selected with g = 6. Follow parent pointers: S-B-C-H-G.
- Cost check: 2 + 1 + 1 + 2 = 6.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B53. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (2); A — D (1); A — E (2); B — C (4); B — D (6); C — F (8); D — F (2); E — F (9); F — G (3); A — F (4)
Heuristic values: S h=6, A h=3, B h=6, C h=8, D h=3, E h=3, F h=2, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-D-F-G; Cost = 8

Working:

- Step 1: frontier {S:6} → expand S.
- Step 2: frontier {A:5, B:12} → expand A.
- Step 3: frontier {D:6, F:8, E:9, B:12} → expand D.

- Step 4: frontier {F:7, E:9, B:12} → expand F.
- Step 5: frontier {G:8, E:9, B:12, C:21} → expand G.
- G is selected with g = 8. Follow parent pointers: S-A-D-F-G.
- Cost check: 2 + 1 + 2 + 3 = 8.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B54. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (2); A — E (1); B — D (6); B — E (8); B — C (6); C — F (5); C — H (1); D — H (7); E — F (9); E — H (4); F — G (7); H — G (2)
Heuristic values: S h=8, A h=8, B h=5, C h=1, D h=1, E h=4, F h=6, H h=1, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-C-H-G; Cost = 12

Working:

- Step 1: frontier {S:8} → expand S.
- Step 2: frontier {B:8, A:10} → expand B.
- Step 3: frontier {A:10, C:10, D:12, E:15} → expand A.
- Step 4: frontier {C:10, D:12, E:12} → expand C.
- Step 5: frontier {H:11, D:12, E:12, F:20} → expand H.
- Step 6: frontier {D:12, E:12, G:12, F:20} → expand D.
- Step 7: frontier {E:12, G:12, F:20} → expand E.
- Step 8: frontier {G:12, F:20} → expand G.
- G is selected with g = 12. Follow parent pointers: S-B-C-H-G.
- Cost check: 3 + 6 + 1 + 2 = 12.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B55. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (1); A — D (8); B — E (1); B — C (8); C — H (9); C — F (6); D — F (9); D — H (1); E — F (5); F — G (5); H — G (5); B — F (3)
Heuristic values: S h=6, A h=7, B h=5, C h=9, D h=8, E h=5, F h=4, H h=3, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-E-F-G; Cost = 8

Working:

- Step 1: frontier {S:6} → expand S.
- Step 2: frontier {B:6, A:8} → expand B.
- Step 3: frontier {E:7, A:8, F:8, C:18} → expand E.
- Step 4: frontier {F:7, A:8, C:18} → expand F.
- Step 5: frontier {A:8, G:8, C:18, D:20} → expand A.
- Step 6: frontier {G:8, D:17, C:18} → expand G.
- G is selected with g = 8. Follow parent pointers: S-B-E-F-G.
- Cost check: 1 + 1 + 1 + 5 = 8.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B56. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (9); A — C (4); B — D (4); B — E (1); C — F (3); C — H (8); D — H (6); D — F (3); E — H (2); E — F (2); F — G (4); H — G (4)
Heuristic values: S h=12, A h=6, B h=4, C h=6, D h=4, E h=4, F h=2, H h=3, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-E-F-G; Cost = 16

Working:

- Step 1: frontier {S:12} → expand S.
- Step 2: frontier {B:13, A:14} → expand B.
- Step 3: frontier {E:13, A:14, D:17} → expand E.
- Step 4: frontier {A:14, F:14, H:15, D:17} → expand A.
- Step 5: frontier {F:14, H:15, D:17, C:18} → expand F.
- Step 6: frontier {H:15, G:16, D:17, C:18} → expand H.
- Step 7: frontier {G:16, D:17, C:18} → expand G.
- G is selected with g = 16. Follow parent pointers: S-B-E-F-G.
- Cost check: 9 + 1 + 2 + 4 = 16.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B57. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (2); A — C (9); A — D (9); B — E (1); B — C (5); C — F (2); C — H (2); D — F (5); D — H (9); E — F (6); E — H (3); F — G (4); H — G (8); B — H (6)
Heuristic values: S h=10, A h=13, B h=9, C h=5, D h=5, E h=7, F h=3, H h=5, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-E-F-G; Cost = 12

Working:

- Step 1: frontier {S:10} → expand S.
- Step 2: frontier {B:10, A:15} → expand B.
- Step 3: frontier {E:9, C:11, H:12, A:15} → expand E.
- Step 4: frontier {H:10, C:11, F:11, A:15} → expand H.
- Step 5: frontier {C:11, F:11, G:13, A:15, D:19} → expand C.
- Step 6: frontier {F:11, G:13, A:15, D:19} → expand F.
- Step 7: frontier {G:12, A:15, D:15} → expand G.
- G is selected with g = 12. Follow parent pointers: S-B-E-F-G.
- Cost check: 1 + 1 + 6 + 4 = 12.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B58. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (3); A — E (7); B — C (4); B — D (4); C — H (5); D — H (6); E — H (6); E — F (5); F — G (9); H — G (8)
Heuristic values: S h=19, A h=17, B h=16, C h=10, D h=12, E h=12, F h=5, H h=6, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-E-F-G; Cost = 21

Working:

- Step 1: frontier {S:19} → expand S.
- Step 2: frontier {A:20, B:23} → expand A.
- Step 3: frontier {E:19, B:23} → expand E.
- Step 4: frontier {F:17, H:19, B:23} → expand F.
- Step 5: frontier {H:19, G:21, B:23} → expand H.
- Step 6: frontier {G:21, B:23, C:28, D:31} → expand G.
- G is selected with g = 21. Follow parent pointers: S-A-E-F-G.
- Cost check: 3 + 4 + 5 + 9 = 21.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B59. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — E (4); A — C (8); B — C (2); B — E (9); A — D (1); C — H (5); D — F (5); D — H (3); E — H (6); E — F (4); F — G (4); H — G (5); A — F (5)
Heuristic values: S h=8, A h=5, B h=9, C h=7, D h=7, E h=7, F h=3, H h=4, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-F-G; Cost = 10

Working:

- Step 1: frontier {S:8} → expand S.
- Step 2: frontier {A:6, B:18} → expand A.
- Step 3: frontier {F:9, D:10, E:10, C:16, B:18} → expand F.
- Step 4: frontier {D:10, E:10, G:10, C:16, B:18} → expand D.
- Step 5: frontier {E:10, G:10, H:10, C:16, B:18} → expand E.
- Step 6: frontier {G:10, H:10, C:16, B:18} → expand G.
- G is selected with g = 10. Follow parent pointers: S-A-F-G.
- Cost check: 1 + 5 + 4 = 10.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B60. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (7); A — E (9); A — D (2); B — D (5); B — C (2); C — F (3); D — F (3); E — F (5); F — G (6)
Heuristic values: S h=15, A h=12, B h=6, C h=7, D h=7, E h=7, F h=5, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-C-F-G; Cost = 17

Working:

- Step 1: frontier {S:15} → expand S.
- Step 2: frontier {B:12, A:19} → expand B.
- Step 3: frontier {C:15, D:18, A:19} → expand C.
- Step 4: frontier {F:16, D:18, A:19} → expand F.
- Step 5: frontier {G:17, D:18, A:19, E:22} → expand G.
- G is selected with g = 17. Follow parent pointers: S-B-C-F-G.
- Cost check: 6 + 2 + 3 + 6 = 17.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B61. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (5); A — D (1); A — C (7); B — E (7); C — F (6); D — F (6); E — F (9); F — G (7)
Heuristic values: S h=19, A h=9, B h=18, C h=11, D h=10, E h=15, F h=4, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-D-F-G; Cost = 20

Working:

- Step 1: frontier {S:19} → expand S.
- Step 2: frontier {A:15, B:23} → expand A.
- Step 3: frontier {D:17, B:23, C:24} → expand D.
- Step 4: frontier {F:17, B:23, C:24} → expand F.
- Step 5: frontier {G:20, B:23, C:24, E:37} → expand G.
- G is selected with g = 20. Follow parent pointers: S-A-D-F-G.
- Cost check: 6 + 1 + 6 + 7 = 20.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B62. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (6); A — E (2); B — C (4); B — D (3); C — F (3); D — F (6); D — H (7); E — F (9); F — G (6); H — G (5); B — H (7)
Heuristic values: S h=11, A h=12, B h=7, C h=5, D h=5, E h=9, F h=5, H h=3, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-C-F-G; Cost = 14

Working:

- Step 1: frontier {S:11} → expand S.
- Step 2: frontier {B:10, A:18} → expand B.
- Step 3: frontier {C:10, H:13, D:17, A:18} → expand C.
- Step 4: frontier {F:13, H:13, D:17, A:18} → expand F.
- Step 5: frontier {H:13, G:14, D:17, A:18, E:26} → expand H.
- Step 6: frontier {G:14, D:17, A:18, E:26} → expand G.
- G is selected with g = 14. Follow parent pointers: S-B-C-F-G.
- Cost check: 3 + 2 + 3 + 6 = 14.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B63. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (3); A — D (7); A — E (5); B — D (9); B — C (1); C — F (1); C — H (1); D — H (8); E — F (8); E — H (7); F — G (8); H — G (8); B — F (1)
Heuristic values: S h=14, A h=13, B h=7, C h=5, D h=13, E h=13, F h=6, H h=5, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a

cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-F-G; Cost = 15

Working:

- Step 1: frontier {S:14} → expand S.
- Step 2: frontier {B:13, A:16} → expand B.
- Step 3: frontier {C:12, F:13, A:16, D:28} → expand C.
- Step 4: frontier {F:13, H:13, A:16, D:28} → expand F.
- Step 5: frontier {H:13, G:15, A:16, E:23, D:28} → expand H.
- Step 6: frontier {G:15, A:16, E:23, D:28} → expand G.
- G is selected with g = 15. Follow parent pointers: S-B-F-G.
- Cost check: 6 + 1 + 8 = 15.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B64. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (3); A — E (1); A — C (7); B — E (7); B — C (6); B — D (9); C — F (4); C — H (4); D — F (7); E — F (3); E — H (3); F — G (7); H — G (1)
Heuristic values: S h=14, A h=10, B h=8, C h=7, D h=9, E h=3, F h=6, H h=0, G h=0

Run A* search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-E-H-G; Cost = 16

Working:

- Step 1: frontier {S:14} → expand S.
- Step 2: frontier {A:13, B:16} → expand A.
- Step 3: frontier {E:15, B:16, C:17} → expand E.
- Step 4: frontier {H:15, B:16, C:17, F:21} → expand H.
- Step 5: frontier {B:16, G:16, C:17, F:21} → expand B.
- Step 6: frontier {G:16, C:17, D:18, F:21} → expand G.
- G is selected with g = 16. Follow parent pointers: S-A-E-H-G.
- Cost check: 3 + 9 + 3 + 1 = 16.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B65. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (2); A — D (8); A — C (1); B — D (1); A — E (6); C — F (9); D — F (1); E — F (9); F — G (7)
Heuristic values: S h=12, A h=11, B h=13, C h=14, D h=14, E h=14, F h=5, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-D-F-G; Cost = 16

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:2, A:9} → expand B.
- Step 3: frontier {D:8, A:9} → expand D.
- Step 4: frontier {A:9, F:9} → expand A.
- Step 5: frontier {F:9, C:10, E:10} → expand F.
- Step 6: frontier {C:10, E:10, G:16} → expand C.
- Step 7: frontier {E:10, G:16} → expand E.
- Step 8: frontier {G:16} → expand G.
- G is selected with g = 16. Follow parent pointers: S-B-D-F-G.
- Cost check: 2 + 6 + 1 + 7 = 16.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B66. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (4); A — D (3); B — E (2); B — C (4); C — F (6); D — F (5); E — F (9); F — G (1); A — F (1)
Heuristic values: S h=5, A h=1, B h=5, C h=6, D h=3, E h=9, F h=0, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-F-G; Cost = 6

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {A:4, B:4} → expand A.
- Step 3: frontier {B:4, F:5, D:7} → expand B.
- Step 4: frontier {F:5, E:6, D:7, C:8} → expand F.
- Step 5: frontier {E:6, G:6, D:7, C:8} → expand E.
- Step 6: frontier {G:6, D:7, C:8} → expand G.
- G is selected with g = 6. Follow parent pointers: S-A-F-G.
- Cost check: 4 + 1 + 1 = 6.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B67. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (4); A — E (5); A — D (4); B — C (6); B — D (5); C — H (6); D — F (4); D — H (1); E — F (2); F — G (2); H — G (9); B — H (5)
Heuristic values: S h=8, A h=8, B h=7, C h=11, D h=3, E h=2, F h=1, H h=6, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-D-F-G; Cost = 12

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:1, A:4} → expand B.
- Step 3: frontier {A:4, C:6, D:6, H:6} → expand A.
- Step 4: frontier {C:6, D:6, H:6, E:10} → expand C.
- Step 5: frontier {D:6, H:6, E:10} → expand D.
- Step 6: frontier {H:6, E:10, F:10} → expand H.
- Step 7: frontier {E:10, F:10, G:15} → expand E.
- Step 8: frontier {F:10, G:15} → expand F.
- Step 9: frontier {G:12} → expand G.
- G is selected with g = 12. Follow parent pointers: S-B-D-F-G.
- Cost check: 1 + 5 + 4 + 2 = 12.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B68. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (5); A — E (6); B — C (6); B — D (6); C — F (3); D — F (8); D — H (2); E — H (7); F — G (7); H — G (7); B — H (5)
Heuristic values: S h=9, A h=12, B h=7, C h=5, D h=8, E h=8, F h=6, H h=4, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-H-G; Cost = 18

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {A:5, B:6} → expand A.
- Step 3: frontier {B:6, E:11} → expand B.
- Step 4: frontier {E:11, H:11, C:12, D:12} → expand E.
- Step 5: frontier {H:11, C:12, D:12} → expand H.
- Step 6: frontier {C:12, D:12, G:18} → expand C.

- Step 7: frontier {D:12, F:15, G:18} → expand D.
- Step 8: frontier {F:15, G:18} → expand F.
- Step 9: frontier {G:18} → expand G.
- G is selected with g = 18. Follow parent pointers: S-B-H-G.
- Cost check: 6 + 5 + 7 = 18.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B69. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (6); A — C (9); A — E (3); B — D (4); B — C (3); C — F (1); C — H (1); D — F (8); E — H (1); E — F (5); F — G (2); H — G (9); B — F (1)
Heuristic values: S h=10, A h=7, B h=1, C h=2, D h=5, E h=4, F h=1, H h=2, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-F-G; Cost = 12

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {A:6, B:9} → expand A.
- Step 3: frontier {B:9, E:9, C:15} → expand B.
- Step 4: frontier {E:9, F:10, C:12, D:13} → expand E.
- Step 5: frontier {F:10, H:10, C:12, D:13} → expand F.
- Step 6: frontier {H:10, C:11, G:12, D:13} → expand H.
- Step 7: frontier {C:11, G:12, D:13} → expand C.
- Step 8: frontier {G:12, D:13} → expand G.
- G is selected with g = 12. Follow parent pointers: S-B-F-G.
- Cost check: 9 + 1 + 2 = 12.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B70. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (7); A — C (8); B — D (7); B — E (3); C — F (8); D — F (9); E — F (5); F — G (9)
Heuristic values: S h=18, A h=20, B h=16, C h=14, D h=17, E h=17, F h=6, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-E-F-G; Cost = 25

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {A:7, B:8} → expand A.
- Step 3: frontier {B:8, C:15} → expand B.
- Step 4: frontier {E:11, C:15, D:15} → expand E.
- Step 5: frontier {C:15, D:15, F:16} → expand C.
- Step 6: frontier {D:15, F:16} → expand D.
- Step 7: frontier {F:16} → expand F.
- Step 8: frontier {G:25} → expand G.
- G is selected with g = 25. Follow parent pointers: S-B-E-F-G.
- Cost check: 8 + 3 + 5 + 9 = 25.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B71. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (7); A — D (4); B — D (3); B — C (7); B — E (6); C — F (8); D — F (1); E — F (9); F — G (9); B — F (4)
Heuristic values: S h=8, A h=12, B h=12, C h=13, D h=7, E h=7, F h=6, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-F-G; Cost = 14

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:1, A:7} → expand B.
- Step 3: frontier {D:4, F:5, A:7, E:7, C:8} → expand D.
- Step 4: frontier {F:5, A:7, E:7, C:8} → expand F.
- Step 5: frontier {A:7, E:7, C:8, G:14} → expand A.
- Step 6: frontier {E:7, C:8, G:14} → expand E.
- Step 7: frontier {C:8, G:14} → expand C.
- Step 8: frontier {G:14} → expand G.
- G is selected with g = 14. Follow parent pointers: S-B-F-G.
- Cost check: 1 + 4 + 9 = 14.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B72. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (5); A — E (8); A — C (1); B — E (1); B — C (4); B — D (2); C — H (2); C — F (4); D — H (9); D — F (5); E — F (7); F — G (9); H — G (7)
Heuristic values: S h=11, A h=5, B h=8, C h=8, D h=11, E h=10, F h=4, H h=5, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is

replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-C-H-G; Cost = 15

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:2, A:5} → expand B.
- Step 3: frontier {E:3, A:5, C:6, D:10} → expand E.
- Step 4: frontier {A:5, C:6, D:10, F:10} → expand A.
- Step 5: frontier {C:6, D:10, F:10} → expand C.
- Step 6: frontier {H:8, D:10, F:10} → expand H.
- Step 7: frontier {D:10, F:10, G:15} → expand D.
- Step 8: frontier {F:10, G:15} → expand F.
- Step 9: frontier {G:15} → expand G.
- G is selected with g = 15. Follow parent pointers: S-B-C-H-G.
- Cost check: 2 + 4 + 2 + 7 = 15.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B73. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (1); A — C (7); B — C (5); B — D (9); B — E (1); C — F (8); D — F (5); E — F (7); F — G (5)
Heuristic values: S h=11, A h=10, B h=12, C h=12, D h=7, E h=7, F h=4, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-E-F-G; Cost = 14

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:1, A:2} → expand B.
- Step 3: frontier {A:2, E:2, C:6, D:10} → expand A.
- Step 4: frontier {E:2, C:6, D:10} → expand E.
- Step 5: frontier {C:6, F:9, D:10} → expand C.
- Step 6: frontier {F:9, D:10} → expand F.
- Step 7: frontier {D:10, G:14} → expand D.
- Step 8: frontier {G:14} → expand G.
- G is selected with g = 14. Follow parent pointers: S-B-E-F-G.
- Cost check: 1 + 1 + 7 + 5 = 14.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B74. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (8); A — C (2); B — C (7); B — E (7); A — D (6); C — F (9); D — F (2); E — F (4); F — G (9); B — F (1)
Heuristic values: S h=8, A h=14, B h=7, C h=9, D h=9, E h=8, F h=6, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-F-G; Cost = 11

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:1, A:8} → expand B.
- Step 3: frontier {F:2, C:7, A:8, E:8} → expand F.
- Step 4: frontier {D:4, E:6, C:7, A:8, G:11} → expand D.
- Step 5: frontier {E:6, C:7, A:8, G:11} → expand E.
- Step 6: frontier {C:7, A:8, G:11} → expand C.
- Step 7: frontier {A:8, G:11} → expand A.
- Step 8: frontier {G:11} → expand G.
- G is selected with g = 11. Follow parent pointers: S-B-F-G.
- Cost check: 1 + 1 + 9 = 11.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B75. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (8); A — C (2); B — D (6); B — E (2); C — F (3); D — F (6); D — H (6); E — F (2); E — H (9); F — G (8); H — G (4)
Heuristic values: S h=12, A h=11, B h=7, C h=6, D h=6, E h=9, F h=5, H h=3, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-E-F-G; Cost = 18

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:6, A:8} → expand B.
- Step 3: frontier {A:8, E:8, D:12} → expand A.
- Step 4: frontier {E:8, C:10, D:12} → expand E.
- Step 5: frontier {C:10, F:10, D:12, H:17} → expand C.
- Step 6: frontier {F:10, D:12, H:17} → expand F.
- Step 7: frontier {D:12, H:17, G:18} → expand D.

- Step 8: frontier {H:17, G:18} → expand H.
- Step 9: frontier {G:18} → expand G.
- G is selected with g = 18. Follow parent pointers: S-B-E-F-G.
- Cost check: 6 + 2 + 2 + 8 = 18.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B76. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (3); S — B (4); A — E (9); A — C (5); B — D (4); B — E (8); C — F (1); D — F (4); E — F (5); F — G (8); A — F (8)
Heuristic values: S h=13, A h=10, B h=14, C h=8, D h=8, E h=8, F h=5, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-C-F-G; Cost = 17

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {A:3, B:4} → expand A.
- Step 3: frontier {B:4, C:8, F:11, E:12} → expand B.
- Step 4: frontier {C:8, D:8, F:11, E:12} → expand C.
- Step 5: frontier {D:8, F:9, E:12} → expand D.
- Step 6: frontier {F:9, E:12} → expand F.
- Step 7: frontier {E:12, G:17} → expand E.
- Step 8: frontier {G:17} → expand G.
- G is selected with g = 17. Follow parent pointers: S-A-C-F-G.
- Cost check: 3 + 5 + 1 + 8 = 17.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B77. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (4); A — D (2); A — E (8); B — D (9); A — C (6); C — H (4); C — F (2); D — H (3); E — F (2); E — H (2); F — G (8); H — G (1)
Heuristic values: S h=7, A h=4, B h=9, C h=4, D h=2, E h=2, F h=4, H h=0, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-D-H-G; Cost = 9

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:3, A:4} → expand B.
- Step 3: frontier {A:4, D:12} → expand A.
- Step 4: frontier {D:6, C:10, E:12} → expand D.
- Step 5: frontier {H:8, C:10, E:12} → expand H.
- Step 6: frontier {G:9, C:10, E:10} → expand G.
- G is selected with g = 9. Follow parent pointers: S-A-D-H-G.
- Cost check: 4 + 2 + 2 + 1 = 9.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B78. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (3); A — E (9); B — E (7); B — C (6); B — D (1); C — F (5); D — F (4); E — F (2); F — G (4)
Heuristic values: S h=8, A h=14, B h=7, C h=5, D h=6, E h=6, F h=2, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-D-F-G; Cost = 12

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:3, A:8} → expand B.
- Step 3: frontier {D:4, A:8, C:9, E:10} → expand D.
- Step 4: frontier {A:8, F:8, C:9, E:10} → expand A.
- Step 5: frontier {F:8, C:9, E:10} → expand F.
- Step 6: frontier {C:9, E:10, G:12} → expand C.
- Step 7: frontier {E:10, G:12} → expand E.
- Step 8: frontier {G:12} → expand G.
- G is selected with g = 12. Follow parent pointers: S-B-D-F-G.
- Cost check: 3 + 1 + 4 + 4 = 12.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B79. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (7); A — E (7); A — C (1); B — E (2); B — D (2); C — F (6); D — F (2); E — F (1); F — G (5); A — F (4)
Heuristic values: S h=15, A h=8, B h=7, C h=7, D h=6, E h=3, F h=4, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-A-F-G; Cost = 16

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {A:7, B:8} → expand A.
- Step 3: frontier {B:8, C:8, F:11, E:14} → expand B.
- Step 4: frontier {C:8, D:10, E:10, F:11} → expand C.
- Step 5: frontier {D:10, E:10, F:11} → expand D.
- Step 6: frontier {E:10, F:11} → expand E.
- Step 7: frontier {F:11} → expand F.
- Step 8: frontier {G:16} → expand G.
- G is selected with g = 16. Follow parent pointers: S-A-F-G.
- Cost check: 7 + 4 + 5 = 16.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B80. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (4); A — E (3); A — D (2); B — D (4); B — C (1); C — H (5); D — H (7); D — F (6); E — H (2); F — G (6); H — G (1); B — H (3)
Heuristic values: S h=5, A h=4, B h=3, C h=3, D h=4, E h=2, F h=4, H h=0, G h=0

Run Uniform-Cost Search from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.

Give (1) the path returned, as letters joined with hyphens (e.g. S-A-G), and (2) its total path cost (integer).

(1) Path = ____
(2) Cost = ____

Answer: Path = S-B-H-G; Cost = 7

Working:

- Step 1: frontier {S:0} → expand S.
- Step 2: frontier {B:3, A:4} → expand B.
- Step 3: frontier {A:4, C:4, D:6, H:6} → expand A.
- Step 4: frontier {C:4, D:6, H:6, E:8} → expand C.
- Step 5: frontier {D:6, H:6, E:8} → expand D.
- Step 6: frontier {H:6, E:8, F:12} → expand H.
- Step 7: frontier {G:7, E:8, F:12} → expand G.
- G is selected with g = 7. Follow parent pointers: S-B-H-G.
- Cost check: 3 + 3 + 1 = 7.

Takeaway: With an admissible h, A* returns the same optimal cost as UCS, usually expanding fewer nodes.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B81. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (2); A — E (1); B — D (1); B — C (1); C — F (2); D — H (7); E — H (4); F — G (9); H — G (3); B — F (7)
Heuristic values: S h=13, A h=11, B h=7, C h=6, D h=8, E h=5, F h=6, H h=2, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-B-F-G (cost 19)
B. No path: greedy gets stuck
C. S-A-...-G found by following the lowest edge cost
D. S-A-E-H-G (cost 16)

Answer: A. S-B-F-G (cost 19)

Working:

- Step 1: frontier {S:13} → expand S (smallest h).
- Step 2: frontier {B:7, A:11} → expand B (smallest h).
- Step 3: frontier {C:6, F:6, D:8, A:11} → expand C (smallest h).
- Step 4: frontier {F:6, D:8, A:11} → expand F (smallest h).
- Step 5: frontier {G:0, D:8, A:11} → expand G (smallest h).
- Greedy returns S-B-F-G (cost 19). Optimal cost (UCS) is 16, so greedy is NOT optimal here.

Option notes:

- B: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- C: Choosing by edge cost is not the greedy rule; greedy uses h only.
- D: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B82. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (8); A — E (5); A — C (3); B — E (2); A — D (8); C — H (4); C — F (6); D — F (5); D — H (3); E — F (6); F — G (3); H — G (3); B — F (8)
Heuristic values: S h=14, A h=8, B h=6, C h=6, D h=5, E h=5, F h=2, H h=1, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-B-F-G (cost 20)
B. S-A-C-H-G (cost 18)
C. No path: greedy gets stuck
D. S-A-...-G found by following the lowest edge cost

Answer: A. S-B-F-G (cost 20)

Working:

- Step 1: frontier {S:14} → expand S (smallest h).
- Step 2: frontier {B:6, A:8} → expand B (smallest h).
- Step 3: frontier {F:2, E:5, A:8} → expand F (smallest h).
- Step 4: frontier {G:0, D:5, E:5, C:6, A:8} → expand G (smallest h).
- Greedy returns S-B-F-G (cost 20). Optimal cost (UCS) is 18, so greedy is NOT optimal here.

Option notes:

- B: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.
- C: On a finite graph with explored-set checking, greedy graph search still reaches G here.

- D: Choosing by edge cost is not the greedy rule; greedy uses h only.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B83. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (7); A — D (4); B — C (5); B — D (4); B — E (1); C — F (9); D — F (8); E — H (6); F — G (9); H — G (2)
Heuristic values: S h=11, A h=11, B h=8, C h=12, D h=7, E h=7, F h=5, H h=1, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-A-...-G found by following the lowest edge cost
B. No path: greedy gets stuck
C. S-B-D-F-G (cost 29)
D. S-B-E-H-G (cost 17)

Answer: C. S-B-D-F-G (cost 29)

Working:

- Step 1: frontier {S:11} → expand S (smallest h).
- Step 2: frontier {B:8, A:11} → expand B (smallest h).
- Step 3: frontier {D:7, E:7, A:11, C:12} → expand D (smallest h).
- Step 4: frontier {F:5, E:7, A:11, C:12} → expand F (smallest h).
- Step 5: frontier {G:0, E:7, A:11, C:12} → expand G (smallest h).
- Greedy returns S-B-D-F-G (cost 29). Optimal cost (UCS) is 17, so greedy is NOT optimal here.

Option notes:

- A: Choosing by edge cost is not the greedy rule; greedy uses h only.
- B: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- D: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B84. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (3); A — E (1); A — C (8); B — D (6); C — F (6); D — F (9); D — H (4); E — H (3); F — G (1); H — G (3); B — H (2)
Heuristic values: S h=5, A h=4, B h=2, C h=3, D h=3, E h=4, F h=0, H h=2, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. No path: greedy gets stuck
B. S-B-H-G (cost 8)
C. S-A-...-G found by following the lowest edge cost

Answer: B. S-B-H-G (cost 8)

Working:

- Step 1: frontier {S:5} → expand S (smallest h).
- Step 2: frontier {B:2, A:4} → expand B (smallest h).
- Step 3: frontier {H:2, A:4, D:6} → expand H (smallest h).
- Step 4: frontier {G:0, A:4, E:4, D:6} → expand G (smallest h).
- Greedy returns S-B-H-G (cost 8). Optimal cost (UCS) is 8.

Option notes:

- A: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- C: Choosing by edge cost is not the greedy rule; greedy uses h only.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B85. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — C (6); A — D (4); B — D (3); A — E (8); C — H (5); C — F (4); D — H (7); D — F (8); E — H (9); E — F (3); F — G (3); H — G (2); A — F (6)
Heuristic values: S h=9, A h=8, B h=6, C h=5, D h=5, E h=3, F h=2, H h=1, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-A-...-G found by following the lowest edge cost
B. No path: greedy gets stuck
C. S-A-F-G (cost 10)
D. S-B-D-H-G (cost 14)

Answer: D. S-B-D-H-G (cost 14)

Working:

- Step 1: frontier {S:9} → expand S (smallest h).
- Step 2: frontier {B:6, A:8} → expand B (smallest h).
- Step 3: frontier {D:5, A:8} → expand D (smallest h).
- Step 4: frontier {H:1, F:2, A:8} → expand H (smallest h).
- Step 5: frontier {G:0, F:2, E:3, C:5, A:8} → expand G (smallest h).
- Greedy returns S-B-D-H-G (cost 14). Optimal cost (UCS) is 10, so greedy is NOT optimal here.

Option notes:

- A: Choosing by edge cost is not the greedy rule; greedy uses h only.
- B: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- C: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B86. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (1); A — C (1); B — D (2); B — E (5); C — F (9); D — F (6); E — F (3); F — G (7); B — F (7)
Heuristic values: S h=10, A h=10, B h=11, C h=14, D h=14, E h=9, F h=4, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a

frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. No path: greedy gets stuck
B. S-B-F-G (cost 15)
C. S-A-...-G found by following the lowest edge cost

Answer: B. S-B-F-G (cost 15)

Working:

- Step 1: frontier {S:10} → expand S (smallest h).
- Step 2: frontier {A:10, B:11} → expand A (smallest h).
- Step 3: frontier {B:11, C:14} → expand B (smallest h).
- Step 4: frontier {F:4, E:9, D:10, C:14} → expand F (smallest h).
- Step 5: frontier {G:0, E:9, D:10, C:14} → expand G (smallest h).
- Greedy returns S-B-F-G (cost 15). Optimal cost (UCS) is 15.

Option notes:

- A: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- C: Choosing by edge cost is not the greedy rule; greedy uses h only.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B87. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (4); A — D (3); A — E (8); B — D (2); B — E (9); B — C (9); C — H (5); D — H (1); D — F (1); E — H (5); E — F (2); F — G (6); H — G (4); B — H (6)
Heuristic values: S h=8, A h=5, B h=5, C h=3, D h=4, E h=6, F h=3, H h=2, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. No path: greedy gets stuck
B. S-A-D-H-G (cost 12)
C. S-B-...-G found by following the lowest edge cost

Answer: B. S-A-D-H-G (cost 12)

Working:

- Step 1: frontier {S:8} → expand S (smallest h).
- Step 2: frontier {A:5, B:5} → expand A (smallest h).
- Step 3: frontier {D:4, B:5, E:6} → expand D (smallest h).
- Step 4: frontier {H:2, F:3, B:5, E:6} → expand H (smallest h).
- Step 5: frontier {G:0, C:3, F:3, B:5, E:6} → expand G (smallest h).
- Greedy returns S-A-D-H-G (cost 12). Optimal cost (UCS) is 12.

Option notes:

- A: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- C: Choosing by edge cost is not the greedy rule; greedy uses h only.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B88. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (4); A — C (3); B — D (7); B — E (2); C — F (3); D — F (2); E — F (1); F — G (7); B — F (9)
Heuristic values: S h=12, A h=10, B h=7, C h=9, D h=5, E h=5, F h=5, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-B-F-G (cost 20)
B. S-B-E-F-G (cost 14)
C. S-A-...-G found by following the lowest edge cost
D. No path: greedy gets stuck

Answer: A. S-B-F-G (cost 20)

Working:

- Step 1: frontier {S:12} → expand S (smallest h).
- Step 2: frontier {B:7, A:10} → expand B (smallest h).
- Step 3: frontier {D:5, E:5, F:5, A:10} → expand D (smallest h).
- Step 4: frontier {E:5, F:5, A:10} → expand E (smallest h).
- Step 5: frontier {F:5, A:10} → expand F (smallest h).
- Step 6: frontier {G:0, C:9, A:10} → expand G (smallest h).
- Greedy returns S-B-F-G (cost 20). Optimal cost (UCS) is 14, so greedy is NOT optimal here.

Option notes:

- B: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.
- C: Choosing by edge cost is not the greedy rule; greedy uses h only.
- D: On a finite graph with explored-set checking, greedy graph search still reaches G here.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B89. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (5); A — C (4); B — C (6); B — D (9); B — E (8); C — F (4); D — F (4); E — F (9); F — G (7); A — F (9)
Heuristic values: S h=12, A h=8, B h=9, C h=8, D h=8, E h=10, F h=6, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-A-F-G (cost 21)
B. S-B-...-G found by following the lowest edge cost
C. No path: greedy gets stuck
D. S-A-C-F-G (cost 20)

Answer: A. S-A-F-G (cost 21)

Working:

- Step 1: frontier {S:12} → expand S (smallest h).
- Step 2: frontier {A:8, B:9} → expand A (smallest h).
- Step 3: frontier {F:6, C:8, B:9} → expand F (smallest h).

- Step 4: frontier {G:0, C:8, B:9, D:10, E:10} → expand G (smallest h).
- Greedy returns S-A-F-G (cost 21). Optimal cost (UCS) is 20, so greedy is NOT optimal here.

Option notes:

- B: Choosing by edge cost is not the greedy rule; greedy uses h only.
- C: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- D: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B90. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (7); A — D (9); A — C (5); B — D (1); A — E (6); C — F (6); D — F (2); E — F (2); F — G (8); B — F (7)
Heuristic values: S h=13, A h=10, B h=12, C h=12, D h=7, E h=7, F h=7, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. No path: greedy gets stuck
B. S-B-...-G found by following the lowest edge cost
C. S-A-E-F-G (cost 15)

Answer: C. S-A-E-F-G (cost 15)

Working:

- Step 1: frontier {S:13} → expand S (smallest h).
- Step 2: frontier {A:10, B:12} → expand A (smallest h).
- Step 3: frontier {E:6, D:7, B:12, C:12} → expand E (smallest h).
- Step 4: frontier {D:7, F:7, B:12, C:12} → expand D (smallest h).
- Step 5: frontier {F:7, B:12, C:12} → expand F (smallest h).
- Step 6: frontier {G:0, B:12, C:12} → expand G (smallest h).
- Greedy returns S-A-E-F-G (cost 15). Optimal cost (UCS) is 15.

Option notes:

- A: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- B: Choosing by edge cost is not the greedy rule; greedy uses h only.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B91. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (8); A — C (4); A — E (9); B — C (2); B — E (5); A — D (2); C — F (6); C — H (3); D — H (5); D — F (2); E — F (7); E — H (6); F — G (4); H — G (7); B — H (2)
Heuristic values: S h=7, A h=7, B h=6, C h=7, D h=5, E h=5, F h=3, H h=3, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-B-H-G (cost 10)
B. S-A-...-G found by following the lowest edge cost
C. No path: greedy gets stuck

Answer: A. S-B-H-G (cost 10)

Working:

- Step 1: frontier {S:7} → expand S (smallest h).
- Step 2: frontier {B:6, A:7} → expand B (smallest h).
- Step 3: frontier {H:3, E:6, A:7, C:7} → expand H (smallest h).
- Step 4: frontier {G:0, D:5, E:6, A:7, C:7} → expand G (smallest h).
- Greedy returns S-B-H-G (cost 10). Optimal cost (UCS) is 10.

Option notes:

- B: Choosing by edge cost is not the greedy rule; greedy uses h only.
- C: On a finite graph with explored-set checking, greedy graph search still reaches G here.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B92. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (7); A — C (9); A — D (9); B — E (9); B — C (3); C — F (1); D — F (7); E — F (5); F — G (8); B — F (2)
Heuristic values: S h=14, A h=14, B h=8, C h=6, D h=13, E h=11, F h=4, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-B-F-G (cost 17)
B. S-A-...-G found by following the lowest edge cost
C. No path: greedy gets stuck

Answer: A. S-B-F-G (cost 17)

Working:

- Step 1: frontier {S:14} → expand S (smallest h).
- Step 2: frontier {B:8, A:14} → expand B (smallest h).
- Step 3: frontier {F:4, C:6, E:11, A:14} → expand F (smallest h).
- Step 4: frontier {G:0, C:6, E:11, D:13, A:14} → expand G (smallest h).
- Greedy returns S-B-F-G (cost 17). Optimal cost (UCS) is 17.

Option notes:

- B: Choosing by edge cost is not the greedy rule; greedy uses h only.
- C: On a finite graph with explored-set checking, greedy graph search still reaches G here.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B93. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (3); A — D (7); B — C (9); A — E (7); C — H (1); C — F (3); D — F (8); E — H (3); E — F (6); F — G (2); H — G (4)
Heuristic values: S h=12, A h=8, B h=9, C h=3, D h=3, E h=6, F h=1, H h=3, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. No path: greedy gets stuck
B. S-A-D-F-G (cost 19)
C. S-B-C-F-G (cost 15)
D. S-B-...-G found by following the lowest edge cost

Answer: B. S-A-D-F-G (cost 19)

Working:

- Step 1: frontier {S:12} → expand S (smallest h).
- Step 2: frontier {A:8, B:9} → expand A (smallest h).
- Step 3: frontier {D:2, E:6, B:9} → expand D (smallest h).
- Step 4: frontier {F:1, E:6, B:9} → expand F (smallest h).
- Step 5: frontier {G:0, C:3, E:6, B:9} → expand G (smallest h).
- Greedy returns S-A-D-F-G (cost 19). Optimal cost (UCS) is 15, so greedy is NOT optimal here.

Option notes:

- A: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- C: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.
- D: Choosing by edge cost is not the greedy rule; greedy uses h only.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B94. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (1); A — D (9); A — C (2); B — C (2); B — E (6); C — F (8); C — H (1); D — H (1); E — F (3); F — G (5); H — G (6); B — F (7)
Heuristic values: S h=8, A h=5, B h=8, C h=6, D h=6, E h=5, F h=3, H h=4, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-B-...-G found by following the lowest edge cost
B. S-B-C-H-G (cost 10)
C. S-A-C-F-G (cost 23)
D. No path: greedy gets stuck

Answer: C. S-A-C-F-G (cost 23)

Working:

- Step 1: frontier {S:8} → expand S (smallest h).
- Step 2: frontier {A:5, B:8} → expand A (smallest h).
- Step 3: frontier {C:6, B:8, D:8} → expand C (smallest h).
- Step 4: frontier {F:3, H:4, B:8, D:8} → expand F (smallest h).
- Step 5: frontier {G:0, H:4, E:5, B:8, D:8} → expand G (smallest h).
- Greedy returns S-A-C-F-G (cost 23). Optimal cost (UCS) is 10, so greedy is NOT optimal here.

Option notes:

- A: Choosing by edge cost is not the greedy rule; greedy uses h only.

- B: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.
- D: On a finite graph with explored-set checking, greedy graph search still reaches G here.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B95. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — B (4); S — A (7); A — E (9); B — D (9); A — C (8); C — F (1); D — F (9); E — F (9); F — G (2); B — F (9)
Heuristic values: S h=14, A h=7, B h=9, C h=1, D h=1, E h=10, F h=1, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. No path: greedy gets stuck
B. S-A-C-F-G (cost 18)
C. S-B-...-G found by following the lowest edge cost
D. S-B-F-G (cost 15)

Answer: B. S-A-C-F-G (cost 18)

Working:

- Step 1: frontier {S:14} → expand S (smallest h).
- Step 2: frontier {A:7, B:9} → expand A (smallest h).
- Step 3: frontier {C:1, B:9, E:10} → expand C (smallest h).
- Step 4: frontier {F:1, B:9, E:10} → expand F (smallest h).
- Step 5: frontier {G:0, B:9, D:9, E:10} → expand G (smallest h).
- Greedy returns S-A-C-F-G (cost 18). Optimal cost (UCS) is 15, so greedy is NOT optimal here.

Option notes:

- A: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- C: Choosing by edge cost is not the greedy rule; greedy uses h only.
- D: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B96. [MCQ · Medium · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — D (5); B — E (9); B — D (8); A — C (1); C — F (3); D — F (6); E — F (8); F — G (1); A — F (7)
Heuristic values: S h=5, A h=3, B h=14, C h=2, D h=6, E h=7, F h=0, G h=0

Run Greedy Best-First Search (f(n) = h(n)) from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which path does it return?

A. S-B-...-G found by following the lowest edge cost
B. S-A-F-G (cost 9)
C. No path: greedy gets stuck
D. S-A-C-F-G (cost 6)

Answer: B. S-A-F-G (cost 9)

Working:

- Step 1: frontier {S:5} → expand S (smallest h).
- Step 2: frontier {A:3, B:14} → expand A (smallest h).
- Step 3: frontier {F:0, C:2, D:6, B:14} → expand F (smallest h).
- Step 4: frontier {G:0, C:2, D:6, E:7, B:14} → expand G (smallest h).
- Greedy returns S-A-F-G (cost 9). Optimal cost (UCS) is 6, so greedy is NOT optimal here.

Option notes:

- A: Choosing by edge cost is not the greedy rule; greedy uses h only.
- C: On a finite graph with explored-set checking, greedy graph search still reaches G here.
- D: This is the cheapest path (UCS/A*). Greedy ignores g, so it can return a different path.

Takeaway: Greedy is fast but not cost-optimal: it never looks at g.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B97. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (4); A — C (7); A — E (9); B — D (7); B — E (8); C — F (8); C — H (2); D — F (7); E — H (6); F — G (1); H — G (1); B — H (4)
Heuristic values: S h=6, A h=9, B h=2, C h=2, D h=2, E h=5, F h=0, H h=0, G h=0

During A* from S, after 1 expansion(s) (S), the frontier is {B:6, A:13} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. B with f = 6
B. G immediately, since G is in the frontier
C. A with f = 13

Answer: A. B with f = 6

Working:

- List f = g + h for every frontier node: B:6, A:13.
- Smallest f = 6 → expand B.

Option notes:

- B: A goal in the frontier is only returned when it is selected (smallest f).
- C: A has f = 13, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B98. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (4); A — C (7); A — E (1); B — C (6); B — D (9); C — F (9); D — F (7); E — F (6); F — G (9)
Heuristic values: S h=16, A h=11, B h=14, C h=12, D h=12, E h=10, F h=8, G h=0

During A* from S, after 2 expansion(s) (S, A), the frontier is {B:15, E:15, C:23} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. E with f = 15
B. E because it has the smallest h = 10
C. B with f = 15
D. C with f = 23

Answer: C. B with f = 15

Working:

- List f = g + h for every frontier node: B:15, E:15, C:23.
- Smallest f = 15 → expand B.

Option notes:

- A: E has f = 15, which is not the smallest (or loses the alphabetical tie-break).
- B: That is the greedy choice (f = h). A* uses f = g + h.
- D: C has f = 23, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B99. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — D (1); A — E (9); B — D (7); B — E (2); A — C (1); C — F (7); D — F (6); E — F (5); F — G (3)
Heuristic values: S h=12, A h=5, B h=9, C h=7, D h=6, E h=6, F h=1, G h=0

During A* from S, after 3 expansion(s) (S, A, D), the frontier is {C:14, F:14, B:15, E:21} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. B with f = 15
B. C with f = 14
C. F because it has the smallest h = 1
D. F with f = 14

Answer: B. C with f = 14

Working:

- List f = g + h for every frontier node: C:14, F:14, B:15, E:21.
- Smallest f = 14 → expand C.

Option notes:

- A: B has f = 15, which is not the smallest (or loses the alphabetical tie-break).
- C: That is the greedy choice (f = h). A* uses f = g + h.
- D: F has f = 14, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B100. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — E (8); A — D (6); B — D (4); B — C (1); C — H (1); C — F (2); D — F (3); E — H (9); F — G (7); H — G (2); B — F (4)
Heuristic values: S h=4, A h=4, B h=2, C h=1, D h=6, E h=10, F h=4, H h=1, G h=0

During A* from S, after 3 expansion(s) (S, B, C), the frontier is {A:5, H:5, F:9, D:16} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. A with f = 5
B. H with f = 5
C. H because it has the smallest h = 1
D. F with f = 9

Answer: A. A with f = 5

Working:

- List f = g + h for every frontier node: A:5, H:5, F:9, D:16.
- Smallest f = 5 → expand A.

Option notes:

- B: H has f = 5, which is not the smallest (or loses the alphabetical tie-break).
- C: That is the greedy choice (f = h). A* uses f = g + h.
- D: F has f = 9, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B101. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (2); A — D (1); A — E (2); B — C (4); B — D (6); C — F (8); D — F (2); E — F (9); F — G (3); A — F (4)
Heuristic values: S h=6, A h=3, B h=6, C h=8, D h=3, E h=3, F h=2, G h=0

During A* from S, after 2 expansion(s) (S, A), the frontier is {D:6, F:8, E:9, B:12} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. E with f = 9
B. F with f = 8
C. F because it has the smallest h = 2
D. D with f = 6

Answer: D. D with f = 6

Working:

- List f = g + h for every frontier node: D:6, F:8, E:9, B:12.
- Smallest f = 6 → expand D.

Option notes:

- A: E has f = 9, which is not the smallest (or loses the alphabetical tie-break).
- B: F has f = 8, which is not the smallest (or loses the alphabetical tie-break).
- C: That is the greedy choice (f = h). A* uses f = g + h.

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B102. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (2); A — E (1); B — D (6); B — E (8); B — C (6); C — F (5); C — H (1); D — H (7); E — F (9); E — H (4); F — G (7); H — G (2)
Heuristic values: S h=8, A h=8, B h=5, C h=1, D h=1, E h=4, F h=6, H h=1, G h=0

During A* from S, after 2 expansion(s) (S, B), the frontier is {A:10, C:10, D:12, E:15} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. A with f = 10
B. C because it has the smallest h = 1
C. C with f = 10
D. D with f = 12

Answer: A. A with f = 10

Working:

- List f = g + h for every frontier node: A:10, C:10, D:12, E:15.
- Smallest f = 10 → expand A.

Option notes:

- B: That is the greedy choice (f = h). A* uses f = g + h.
- C: C has f = 10, which is not the smallest (or loses the alphabetical tie-break).
- D: D has f = 12, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B103. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (1); A — D (8); B — E (1); B — C (8); C — H (9); C — F (6); D — F (9); D — H (1); E — F (5); F — G (5); H — G (5); B — F (3)
Heuristic values: S h=6, A h=7, B h=5, C h=9, D h=8, E h=5, F h=4, H h=3, G h=0

During A* from S, after 1 expansion(s) (S), the frontier is {B:6, A:8} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. B with f = 6
B. G immediately, since G is in the frontier
C. A with f = 8

Answer: A. B with f = 6

Working:

- List f = g + h for every frontier node: B:6, A:8.
- Smallest f = 6 → expand B.

Option notes:

- B: A goal in the frontier is only returned when it is selected (smallest f).
- C: A has f = 8, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B104. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (9); A — C (4); B — D (4); B — E (1); C — F (3); C — H (8); D — H (6); D — F (3); E — H (2); E — F (2); F — G (4); H — G (4)
Heuristic values: S h=12, A h=6, B h=4, C h=6, D h=4, E h=4, F h=2, H h=3, G h=0

During A* from S, after 3 expansion(s) (S, B, E), the frontier is {A:14, F:14, H:15, D:17} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. A with f = 14
B. H with f = 15
C. F with f = 14
D. F because it has the smallest h = 2

Answer: A. A with f = 14

Working:

- List f = g + h for every frontier node: A:14, F:14, H:15, D:17.
- Smallest f = 14 → expand A.

Option notes:

- B: H has f = 15, which is not the smallest (or loses the alphabetical tie-break).
- C: F has f = 14, which is not the smallest (or loses the alphabetical tie-break).
- D: That is the greedy choice (f = h). A* uses f = g + h.

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B105. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (2); A — C (9); A — D (9); B — E (1); B — C (5); C — F (2); C — H (2); D — F (5); D — H (9); E — F (6); E — H (3); F — G (4); H — G (8); B — H (6)
Heuristic values: S h=10, A h=13, B h=9, C h=5, D h=5, E h=7, F h=3, H h=5, G h=0

During A* from S, after 2 expansion(s) (S, B), the frontier is {E:9, C:11, H:12, A:15} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. H with f = 12
B. C because it has the smallest h = 5
C. E with f = 9
D. C with f = 11

Answer: C. E with f = 9

Working:

- List f = g + h for every frontier node: E:9, C:11, H:12, A:15.
- Smallest f = 9 → expand E.

Option notes:

- A: H has f = 12, which is not the smallest (or loses the alphabetical tie-break).

- B: That is the greedy choice (f = h). A* uses f = g + h.
- D: C has f = 11, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B106. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (3); A — E (7); B — C (4); B — D (4); C — H (5); D — H (6); E — H (6); E — F (5); F — G (9); H — G (8)
Heuristic values: S h=19, A h=17, B h=16, C h=10, D h=12, E h=12, F h=5, H h=6, G h=0

During A* from S, after 2 expansion(s) (S, A), the frontier is {E:19, B:23} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. E with f = 19
B. G immediately, since G is in the frontier
C. B with f = 23

Answer: A. E with f = 19

Working:

- List f = g + h for every frontier node: E:19, B:23.
- Smallest f = 19 → expand E.

Option notes:

- B: A goal in the frontier is only returned when it is selected (smallest f).
- C: B has f = 23, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B107. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — E (4); A — C (8); B — C (2); B — E (9); A — D (1); C — H (5); D — F (5); D — H (3); E — H (6); E — F (4); F — G (4); H — G (5); A — F (5)
Heuristic values: S h=8, A h=5, B h=9, C h=7, D h=7, E h=7, F h=3, H h=4, G h=0

During A* from S, after 3 expansion(s) (S, A, F), the frontier is {D:10, E:10, G:10, C:16, B:18} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. G because it has the smallest h = 0
B. E with f = 10
C. G with f = 10
D. D with f = 10

Answer: D. D with f = 10

Working:

- List f = g + h for every frontier node: D:10, E:10, G:10, C:16, B:18.
- Smallest f = 10 → expand D.

Option notes:

- A: That is the greedy choice (f = h). A* uses f = g + h.
- B: E has f = 10, which is not the smallest (or loses the alphabetical tie-break).
- C: G has f = 10, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B108. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (7); A — E (9); A — D (2); B — D (5); B — C (2); C — F (3); D — F (3); E — F (5); F — G (6)
Heuristic values: S h=15, A h=12, B h=6, C h=7, D h=7, E h=7, F h=5, G h=0

During A* from S, after 2 expansion(s) (S, B), the frontier is {C:15, D:18, A:19} (values are f = g + h).
Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. A with f = 19
B. C with f = 15
C. G immediately, since G is in the frontier
D. D with f = 18

Answer: B. C with f = 15

Working:

- List f = g + h for every frontier node: C:15, D:18, A:19.
- Smallest f = 15 → expand C.

Option notes:

- A: A has f = 19, which is not the smallest (or loses the alphabetical tie-break).
- C: A goal in the frontier is only returned when it is selected (smallest f).
- D: D has f = 18, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B109. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (5); A — D (1); A — C (7); B — E (7); C — F (6); D — F (6); E — F (9); F — G (7)
Heuristic values: S h=19, A h=9, B h=18, C h=11, D h=10, E h=15, F h=4, G h=0

During A* from S, after 3 expansion(s) (S, A, D), the frontier is {F:17, B:23, C:24} (values are f = g + h).
Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. G immediately, since G is in the frontier
B. B with f = 23
C. C with f = 24
D. F with f = 17

Answer: D. F with f = 17

Working:

- List f = g + h for every frontier node: F:17, B:23, C:24.
- Smallest f = 17 → expand F.

Option notes:

- A: A goal in the frontier is only returned when it is selected (smallest f).
- B: B has f = 23, which is not the smallest (or loses the alphabetical tie-break).
- C: C has f = 24, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B110. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (6); A — E (2); B — C (4); B — D (3); C — F (3); D — F (6); D — H (7); E — F (9); F — G (6); H — G (5); B — H (7)
Heuristic values: S h=11, A h=12, B h=7, C h=5, D h=5, E h=9, F h=5, H h=3, G h=0

During A* from S, after 3 expansion(s) (S, B, C), the frontier is {F:13, H:13, D:17, A:18} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. H with f = 13
B. D with f = 17
C. H because it has the smallest h = 3
D. F with f = 13

Answer: D. F with f = 13

Working:

- List f = g + h for every frontier node: F:13, H:13, D:17, A:18.
- Smallest f = 13 → expand F.

Option notes:

- A: H has f = 13, which is not the smallest (or loses the alphabetical tie-break).
- B: D has f = 17, which is not the smallest (or loses the alphabetical tie-break).
- C: That is the greedy choice (f = h). A* uses f = g + h.

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B111. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (3); A — D (7); A — E (5); B — D (9); B — C (1); C — F (1); C — H (1); D — H (8); E — F (8); E — H (7); F — G (8); H — G (8); B — F (1)
Heuristic values: S h=14, A h=13, B h=7, C h=5, D h=13, E h=13, F h=6, H h=5, G h=0

During A* from S, after 3 expansion(s) (S, B, C), the frontier is {F:13, H:13, A:16, D:28} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. H with f = 13
B. A with f = 16
C. H because it has the smallest h = 5
D. F with f = 13

Answer: D. F with f = 13

Working:

- List f = g + h for every frontier node: F:13, H:13, A:16, D:28.
- Smallest f = 13 → expand F.

Option notes:

- A: H has f = 13, which is not the smallest (or loses the alphabetical tie-break).
- B: A has f = 16, which is not the smallest (or loses the alphabetical tie-break).
- C: That is the greedy choice (f = h). A* uses f = g + h.

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B112. [MCQ · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (3); A — E (1); A — C (7); B — E (7); B — C (6); B — D (9); C — F (4); C — H (4); D — F (7); E — F (3); E — H (3); F — G (7); H — G (1)
Heuristic values: S h=14, A h=10, B h=8, C h=7, D h=9, E h=3, F h=6, H h=0, G h=0

During A* from S, after 3 expansion(s) (S, A, E), the frontier is {H:15, B:16, C:17, F:21} (values are f = g + h). Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Which node is expanded next, and with what f?

A. B with f = 16
B. H with f = 15
C. C with f = 17
D. F with f = 21

Answer: B. H with f = 15

Working:

- List f = g + h for every frontier node: H:15, B:16, C:17, F:21.
- Smallest f = 15 → expand H.

Option notes:

- A: B has f = 16, which is not the smallest (or loses the alphabetical tie-break).
- C: C has f = 17, which is not the smallest (or loses the alphabetical tie-break).
- D: F has f = 21, which is not the smallest (or loses the alphabetical tie-break).

Takeaway: Always compute f = g + h for EVERY frontier node, not just the newest children.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B113. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (7); A — E (9); A — C (9); B — D (8); B — C (7); C — F (8); C — H (5); D — H (5); E — H (4); E — F (7); F — G (3); H — G (5); A — H (5)
Heuristic values: S h=10, A h=9, B h=13, C h=7, D h=7, E h=5, F h=1, H h=3, G h=0

Using the cheapest path from S to D, what is the A* evaluation f(D)?

A. 22
B. 12
C. 19
D. 7

Answer: C. 19

Working:

- Cheapest g(D) from S = 12.
- h(D) = 7 (read from the graph).
- f(D) = g + h = 12 + 7 = 19.

Option notes:

- A: Used the true remaining cost h*(n) instead of the given h(n).
- B: That is g(n) only (UCS value).
- D: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B114. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (3); A — D (3); B — C (6); B — E (9); C — H (6); D — H (4); E — H (3); C — F (5); F — G (1); H — G (7)
Heuristic values: S h=11, A h=9, B h=11, C h=3, D h=3, E h=5, F h=0, H h=5, G h=0

Using the cheapest path from S to G, what is the A* evaluation f(G)?

A. 1
B. 22
C. 0
D. 15

Answer: D. 15

Working:

- Cheapest g(G) from S = 15.
- h(G) = 0 (read from the graph).
- f(G) = g + h = 15 + 0 = 15.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- C: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B115. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (2); A — C (2); B — E (5); B — D (9); C — F (1); D — F (5); E — F (4); F — G (7)
Heuristic values: S h=10, A h=6, B h=14, C h=5, D h=8, E h=10, F h=4, G h=0

Using the cheapest path from S to A, what is the A* evaluation f(A)?

A. 15
B. 11

C. 6
D. 5

Answer: B. 11

Working:

- Cheapest g(A) from S = 5.
- h(A) = 6 (read from the graph).
- f(A) = g + h = 5 + 6 = 11.

Option notes:

- A: Used the true remaining cost h*(n) instead of the given h(n).
- C: That is h(n) only (greedy value).
- D: That is g(n) only (UCS value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B116. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (8); A — C (1); B — E (2); B — C (1); B — D (6); C — H (9); C — F (2); D — H (3); E — F (8); F — G (5); H — G (7)
Heuristic values: S h=12, A h=6, B h=4, C h=3, D h=8, E h=5, F h=4, H h=6, G h=0

Using the cheapest path from S to B, what is the A* evaluation f(B)?

A. 16
B. 12
C. 8
D. 4

Answer: B. 12

Working:

- Cheapest g(B) from S = 8.
- h(B) = 4 (read from the graph).
- f(B) = g + h = 8 + 4 = 12.

Option notes:

- A: Used the true remaining cost h*(n) instead of the given h(n).
- C: That is g(n) only (UCS value).
- D: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B117. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (8); A — D (8); A — E (4); B — E (6); B — C (2); C — F (4); C — H (5); D — H (7); E — F (2); E — H (6); F — G (5); H — G (4); B — H (2)
Heuristic values: S h=11, A h=11, B h=5, C h=5, D h=4, E h=4, F h=3, H h=3, G h=0

Using the cheapest path from S to A, what is the A* evaluation f(A)?

A. 18
B. 7
C. 21
D. 11

Answer: A. 18

Working:

- Cheapest g(A) from S = 7.
- h(A) = 11 (read from the graph).
- f(A) = g + h = 7 + 11 = 18.

Option notes:

- B: That is g(n) only (UCS value).
- C: Used the true remaining cost h*(n) instead of the given h(n).
- D: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B118. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — B (4); S — A (6); A — E (8); A — D (4); B — C (5); B — D (7); C — F (7); D — F (3); E — F (5); F — G (5)
Heuristic values: S h=10, A h=9, B h=14, C h=11, D h=5, E h=5, F h=3, G h=0

Using the cheapest path from S to A, what is the A* evaluation f(A)?

A. 15
B. 21
C. 6
D. 9

Answer: A. 15

Working:

- Cheapest g(A) from S = 6.
- h(A) = 9 (read from the graph).
- f(A) = g + h = 6 + 9 = 15.

Option notes:

- B: Used the true remaining cost h*(n) instead of the given h(n).
- C: That is g(n) only (UCS value).
- D: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B119. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (5); A — C (7); B — C (1); B — E (3); A — D (2); C — H (1); C — F (4); D — F (4); E — F (6); F — G (7); H — G (4); B — H (5)
Heuristic values: S h=10, A h=8, B h=6, C h=4, D h=7, E h=7, F h=4, H h=3, G h=0

Using the cheapest path from S to C, what is the A* evaluation f(C)?

A. 7
B. 11
C. 4
D. 12

Answer: B. 11

Working:

- Cheapest g(C) from S = 7.
- h(C) = 4 (read from the graph).
- f(C) = g + h = 7 + 4 = 11.

Option notes:

- A: That is g(n) only (UCS value).
- C: That is h(n) only (greedy value).
- D: Used the true remaining cost h*(n) instead of the given h(n).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B120. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — B (5); S — A (1); A — E (8); A — D (8); B — C (1); B — E (9); C — F (3); C — H (1); D — F (1); D — H (4); E — H (4); F — G (9); H — G (1)
Heuristic values: S h=6, A h=3, B h=7, C h=1, D h=3, E h=3, F h=4, H h=0, G h=0

Using the cheapest path from S to F, what is the A* evaluation f(F)?

A. 15
B. 10
C. 14
D. 4

Answer: C. 14

Working:

- Cheapest g(F) from S = 10.
- h(F) = 4 (read from the graph).
- f(F) = g + h = 10 + 4 = 14.

Option notes:

- A: Used the true remaining cost h*(n) instead of the given h(n).
- B: That is g(n) only (UCS value).
- D: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B121. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (8); A — E (4); B — C (9); B — E (6); B — D (6); C — F (1); D — F (2); E — F (6); F — G (6); A — F
Heuristic values: S h=10, A h=8, B h=9, C h=6, D h=7, E h=6, F h=5, G h=0
Other labels: 9

Using the cheapest path from S to G, what is the A* evaluation f(G)?

A. 3
B. 24
C. 0
D. 17

Answer: D. 17

Working:

- Cheapest g(G) from S = 17.

- h(G) = 0 (read from the graph).
- f(G) = g + h = 17 + 0 = 17.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- C: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B122. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (9); A — C (3); A — D (8); B — C (5); B — E (6); C — H (2); D — H (7); D — F (7); E — H (6); E — F (8); F — G (9); H — G (3); A — H
Heuristic values: S h=10, A h=6, B h=7, C h=4, D h=4, E h=5, F h=8, H h=2, G h=0
Other labels: 7

Using the cheapest path from S to G, what is the A* evaluation f(G)?

A. 3
B. 0
C. 24
D. 17

Answer: D. 17

Working:

- Cheapest g(G) from S = 17.
- h(G) = 0 (read from the graph).
- f(G) = g + h = 17 + 0 = 17.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: That is h(n) only (greedy value).
- C: Arithmetic slip: recompute each step carefully.

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B123. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — B (5); S — A (6); A — D (2); B — E (4); B — D (9); B — C (5); C — F (1); C — H (6); D — H (8); E — F (8); F — G (1); H — G (8); A — F (4)
Heuristic values: S h=7, A h=3, B h=6, C h=1, D h=5, E h=6, F h=0, H h=7, G h=0

Using the cheapest path from S to G, what is the A* evaluation f(G)?

A. 18
B. 0
C. -3
D. 11

Answer: D. 11

Working:

- Cheapest g(G) from S = 11.
- h(G) = 0 (read from the graph).

- f(G) = g + h = 11 + 0 = 11.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: That is h(n) only (greedy value).
- C: Arithmetic slip: recompute each step carefully.

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B124. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (5); A — D (8); B — C (9); B — E (2); C — F (6); D — F (4); E — F (8); F — G (3); A — F (6)
Heuristic values: S h=11, A h=8, B h=8, C h=6, D h=5, E h=5, F h=1, G h=0

Using the cheapest path from S to B, what is the A* evaluation f(B)?

A. 5
B. 8
C. 18
D. 13

Answer: D. 13

Working:

- Cheapest g(B) from S = 5.
- h(B) = 8 (read from the graph).
- f(B) = g + h = 5 + 8 = 13.

Option notes:

- A: That is g(n) only (UCS value).
- B: That is h(n) only (greedy value).
- C: Used the true remaining cost h*(n) instead of the given h(n).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B125. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — E (7); A — D (4); B — C (6); B — E (2); C — H (6); D — F (8); D — H (7); E — F (7); E — H (1); F — G (3); H — G (3); A — H (1)
Heuristic values: S h=6, A h=2, B h=4, C h=6, D h=6, E h=3, F h=2, H h=2, G h=0

Using the cheapest path from S to G, what is the A* evaluation f(G)?

A. 17
B. -4
C. 10
D. 0

Answer: C. 10

Working:

- Cheapest g(G) from S = 10.
- h(G) = 0 (read from the graph).
- f(G) = g + h = 10 + 0 = 10.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- D: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B126. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (3); A — D (3); A — C (8); B — C (8); B — E (4); C — F (3); D — F (2); E — F (9); F — G (6)
Heuristic values: S h=15, A h=10, B h=10, C h=8, D h=7, E h=13, F h=4, G h=0

Using the cheapest path from S to D, what is the A* evaluation f(D)?

A. 11
B. 18
C. 19
D. 7

Answer: B. 18

Working:

- Cheapest g(D) from S = 11.
- h(D) = 7 (read from the graph).
- f(D) = g + h = 11 + 7 = 18.

Option notes:

- A: That is g(n) only (UCS value).
- C: Used the true remaining cost h*(n) instead of the given h(n).
- D: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B127. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (1); A — E (1); A — D (6); B — D (1); B — C (5); C — F (8); C — H (1); D — H (9); E — H (3); E — F (2); F — G (4); H — G (4); B — F (8)
Heuristic values: S h=6, A h=4, B h=6, C h=4, D h=6, E h=6, F h=3, H h=2, G h=0

Using the cheapest path from S to C, what is the A* evaluation f(C)?

A. 6
B. 11
C. 4
D. 10

Answer: D. 10

Working:

- Cheapest g(C) from S = 6.
- h(C) = 4 (read from the graph).
- f(C) = g + h = 6 + 4 = 10.

Option notes:

- A: That is g(n) only (UCS value).
- B: Used the true remaining cost h*(n) instead of the given h(n).

- C: That is h(n) only (greedy value).

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B128. [MCQ · Easy · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (3); A — E (3); A — D (3); B — E (3); A — C; C — F (5); D — F (1); E — F (2); F — G (9)
Heuristic values: S h=9, A h=10, B h=10, C h=11, D h=9, E h=9, F h=8, G h=0
Other labels: 9

Using the cheapest path from S to G, what is the A* evaluation f(G)?

A. 0
B. 2
C. 23
D. 16

Answer: D. 16

Working:

- Cheapest g(G) from S = 16.
- h(G) = 0 (read from the graph).
- f(G) = g + h = 16 + 0 = 16.

Option notes:

- A: That is h(n) only (greedy value).
- B: Arithmetic slip: recompute each step carefully.
- C: Arithmetic slip: recompute each step carefully.

Takeaway: A*: f = g + h. UCS: f = g. Greedy: f = h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B129. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (4); A — C (7); A — E (9); B — D (7); B — E (8); C — F (8); C — H (2); D — F (7); E — H (6); F — G (1); H — G (1); B — H (4)
Heuristic values: S h=8, A h=5, B h=4, C h=2, D h=2, E h=4, F h=0, H h=0, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 8 | 9 | OK
  A | 5 | 10 | OK
  B | 4 | 5 | OK
  C | 2 | 3 | OK
  D | 5 | 8 | OK
  E | 4 | 7 | OK
  F | 0 | 1 | OK

H | 0 | 1 | OK
G | 0 | 0 | OK

- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B130. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (4); A — C (7); A — E (1); B — C (6); B — D (9); C — F (9); D — F (7); E — F (6); F — G (9)
Heuristic values: S h=14, A h=10, B h=14, C h=17, D h=17, E h=14, F h=6, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 14 | 20 | OK
  A | 10 | 16 | OK
  B | 14 | 21 | OK
  C | 17 | 18 | OK
  D | 11 | 16 | OK
  E | 14 | 15 | OK
  F | 6 | 9 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B131. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — D (1); A — E (9); B — D (7); B — E (2); A — C (1); C — F (7); D — F (6); E — F (5); F — G (3)
Heuristic values: S h=9, A h=9, B h=7, C h=7, D h=7, E h=4, F h=2, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 9 | 16 | OK
  A | 9 | 10 | OK
  B | 7 | 10 | OK
  C | 7 | 10 | OK
  D | 7 | 9 | OK

E | 4 | 8 | OK
F | 2 | 3 | OK
G | 0 | 0 | OK

- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B132. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — E (8); A — D (6); B — D (4); B — C (1); C — H (1); C — F (2); D — F (3); E — H (9); F — G (7); H — G (2); B — F (4)
Heuristic values: S h=4, A h=4, B h=2, C h=2, D h=7, E h=10, F h=7, H h=1, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: False

Why: h(F) = 7 > h*(F) = 5: it overestimates.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 4 | 6 | OK
  A | 4 | 7 | OK
  B | 2 | 4 | OK
  C | 2 | 3 | OK
  D | 7 | 8 | OK
  E | 10 | 11 | OK
  F | 7 | 5 | OVER
  H | 1 | 2 | OK
  G | 0 | 0 | OK
- F overestimates → NOT admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B133. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (2); A — D (1); A — E (2); B — C (4); B — D (6); C — F (8); D — F (2); E — F (9); F — G (3); A — F (4)
Heuristic values: S h=4, A h=8, B h=8, C h=6, D h=3, E h=3, F h=2, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: False

Why: h(A) = 8 > h*(A) = 6: it overestimates.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 4 | 8 | OK
  A | 8 | 6 | OVER
  B | 8 | 11 | OK

C | 6 | 11 | OK
D | 3 | 5 | OK
E | 7 | 8 | OK
F | 2 | 3 | OK
G | 0 | 0 | OK

- A overestimates → NOT admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B134. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (2); A — E (1); B — D (6); B — E (8); B — C (6); C — F (5); C — H (1); D — H (7); E — F (9); E — H (4); F — G (7); H — G (2)
Heuristic values: S h=8, A h=7, B h=5, C h=2, D h=2, E h=5, F h=11, H h=1, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: False

Why: h(F) = 11 > h*(F) = 7: it overestimates.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 8 | 12 | OK
  A | 7 | 12 | OK
  B | 5 | 9 | OK
  C | 2 | 3 | OK
  D | 8 | 10 | OK
  E | 5 | 6 | OK
  F | 11 | 7 | OVER
  H | 1 | 2 | OK
  G | 0 | 0 | OK
- F overestimates → NOT admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B135. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (1); A — D (8); B — E (1); B — C (8); C — H (9); C — F (6); D — F (9); D — H (1); E — F (5); F — G (5); H — G (5); B — F (3)
Heuristic values: S h=6, A h=7, B h=6, C h=8, D h=9, E h=5, F h=3, H h=3, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 6 | 8 | OK

A | 7 | 9 | OK
B | 6 | 7 | OK
C | 8 | 11 | OK
D | 9 | 10 | OK
E | 5 | 6 | OK
F | 3 | 5 | OK
H | 3 | 5 | OK
G | 0 | 0 | OK

- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B136. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (9); A — C (4); B — D (4); B — E (1); C — F (3); C — H (8); D — H (6); D — F (3); E — H (2); E — F (2); F — G (4); H — G (4)
Heuristic values: S h=9, A h=7, B h=6, C h=4, D h=3, E h=3, F h=3, H h=3, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 9 | 16 | OK
  A | 7 | 11 | OK
  B | 6 | 7 | OK
  C | 4 | 7 | OK
  D | 3 | 7 | OK
  E | 3 | 6 | OK
  F | 3 | 4 | OK
  H | 3 | 4 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B137. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (2); A — C (9); A — D (9); B — E (1); B — C (5); C — F (2); C — H (2); D — F (5); D — H (9); E — F (6); E — H (3); F — G (4); H — G (8); B — H (6)
Heuristic values: S h=11, A h=12, B h=9, C h=5, D h=4, E h=9, F h=2, H h=6, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).

- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 11 | 12 | OK
  A | 12 | 14 | OK
  B | 9 | 11 | OK
  C | 5 | 6 | OK
  D | 4 | 6 | OK
  E | 9 | 10 | OK
  F | 2 | 4 | OK
  H | 6 | 8 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B138. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (3); A — E (7); B — C (4); B — D (4); C — H (5); D — H (6); E — H (6); E — F (5); F — G (9); H — G (8)
Heuristic values: S h=19, A h=17, B h=13, C h=11, D h=12, E h=12, F h=7, H h=5, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 19 | 21 | OK
  A | 17 | 18 | OK
  B | 13 | 17 | OK
  C | 11 | 13 | OK
  D | 12 | 14 | OK
  E | 8 | 14 | OK
  F | 7 | 9 | OK
  H | 5 | 8 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B139. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — E (4); A — C (8); B — C (2); B — E (9); A — D (1); C — H (5); D — F (5); D — H (3); E — H (6); E — F (4); F — G (4); H — G (5); A — F (5)
Heuristic values: S h=5, A h=7, B h=7, C h=9, D h=5, E h=5, F h=3, H h=4, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 5 | 10 | OK
  A | 7 | 9 | OK
  B | 7 | 11 | OK
  C | 9 | 10 | OK
  D | 5 | 8 | OK
  E | 6 | 8 | OK
  F | 3 | 4 | OK
  H | 4 | 5 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B140. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (7); A — E (9); A — D (2); B — D (5); B — C (2); C — F (3); D — F (3); E — F (5); F — G (6)
Heuristic values: S h=16, A h=7, B h=9, C h=7, D h=5, E h=5, F h=4, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 16 | 17 | OK
  A | 7 | 13 | OK
  B | 9 | 11 | OK
  C | 7 | 9 | OK
  D | 5 | 9 | OK
  E | 9 | 11 | OK
  F | 4 | 6 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B141. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (5); A — D (1); A — C (7); B — E (7); C — F (6); D — F (6); E — F (9); F — G (7)
Heuristic values: S h=12, A h=11, B h=20, C h=10, D h=12, E h=10, F h=6, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 12 | 20 | OK
  A | 11 | 14 | OK
  B | 20 | 23 | OK
  C | 10 | 13 | OK
  D | 12 | 13 | OK
  E | 10 | 16 | OK
  F | 6 | 7 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B142. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (6); A — E (2); B — C (4); B — D (3); C — F (3); D — F (6); D — H (7); E — F (9); F — G (6); H — G (5); B — H (7)
Heuristic values: S h=9, A h=12, B h=6, C h=8, D h=8, E h=13, F h=4, H h=4, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 9 | 14 | OK
  A | 12 | 19 | OK
  B | 6 | 11 | OK
  C | 8 | 9 | OK
  D | 7 | 12 | OK
  E | 13 | 15 | OK
  F | 4 | 6 | OK
  H | 4 | 5 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B143. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (3); A — D (7); A — E (5); B — D (9); B — C (1); C — F (1); C — H (1); D — H (8); E — F (8); E — H (7); F — G (8); H — G (8); B — F (1)

Heuristic values: S h=11, A h=14, B h=5, C h=7, D h=9, E h=9, F h=5, H h=7, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 11 | 15 | OK
  A | 14 | 18 | OK
  B | 5 | 9 | OK
  C | 7 | 9 | OK
  D | 9 | 16 | OK
  E | 12 | 15 | OK
  F | 5 | 8 | OK
  H | 7 | 8 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B144. [TF · Hard · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (3); A — E (1); A — C (7); B — E (7); B — C (6); B — D (9); C — F (4); C — H (4); D — F (7); E — F (3); E — H (3); F — G (7); H — G (1)
Heuristic values: S h=13, A h=9, B h=9, C h=6, D h=10, E h=3, F h=6, H h=0, G h=0

True or False: the heuristic h shown on the graph is admissible for reaching G.

Answer: True

Why: Every h(n) ≤ h*(n), the true cheapest cost to G.

Working:

- Admissible means h(n) ≤ h*(n) for every node (never overestimates).
- Compute the true cost-to-goal h*(n) for every node (shortest path to G):
- Node | h(n) | h*(n) | Check
  S | 13 | 16 | OK
  A | 9 | 13 | OK
  B | 9 | 11 | OK
  C | 6 | 8 | OK
  D | 10 | 11 | OK
  E | 3 | 4 | OK
  F | 6 | 7 | OK
  H | 0 | 1 | OK
  G | 0 | 0 | OK
- No node overestimates → admissible.

Takeaway: Admissible = never overestimates the true remaining cost.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B145. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (4); S — B (4); A — C (7); A — E (9); B — D (7); B — E (8); C — F (8); C — H (2); D — F (7); E — H (6); F — G (1); H — G (1); B — H (4)
Heuristic values: S h=8, A h=5, B h=4, C h=2, D h=2, E h=4, F h=0, H h=0, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: True

Why: Every edge satisfies the triangle inequality.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=4): 8 ≤ 4+5=9 ✓; 5 ≤ 4+8=12 ✓
- S–B (c=4): 8 ≤ 4+4=8 ✓; 4 ≤ 4+8=12 ✓
- A–C (c=7): 5 ≤ 7+2=9 ✓; 2 ≤ 7+5=12 ✓
- A–E (c=9): 5 ≤ 9+4=13 ✓; 4 ≤ 9+5=14 ✓
- B–D (c=7): 4 ≤ 7+5=12 ✓; 5 ≤ 7+4=11 ✓
- B–E (c=8): 4 ≤ 8+4=12 ✓; 4 ≤ 8+4=12 ✓
- C–F (c=8): 2 ≤ 8+0=8 ✓; 0 ≤ 8+2=10 ✓
- C–H (c=2): 2 ≤ 2+0=2 ✓; 0 ≤ 2+2=4 ✓
- D–F (c=7): 5 ≤ 7+0=7 ✓; 0 ≤ 7+5=12 ✓
- E–H (c=6): 4 ≤ 6+0=6 ✓; 0 ≤ 6+4=10 ✓
- F–G (c=1): 0 ≤ 1+0=1 ✓; 0 ≤ 1+0=1 ✓
- H–G (c=1): 0 ≤ 1+0=1 ✓; 0 ≤ 1+0=1 ✓
- B–H (c=4): 4 ≤ 4+0=4 ✓; 0 ≤ 4+4=8 ✓
- All pass → consistent (and therefore also admissible).

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B146. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (4); A — C (7); A — E (1); B — C (6); B — D (9); C — F (9); D — F (7); E — F (6); F — G (9)
Heuristic values: S h=14, A h=10, B h=14, C h=17, D h=17, E h=14, F h=6, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge E→A: h(E) = 14 > 1 + h(A) = 11.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=1): 14 ≤ 1+14=15 ✓; 14 ≤ 1+14=15 ✓
- S–A (c=4): 14 ≤ 4+10=14 ✓; 10 ≤ 4+14=18 ✓
- A–C (c=7): 10 ≤ 7+17=24 ✓; 17 ≤ 7+10=17 ✓
- A–E (c=1): 10 ≤ 1+14=15 ✓; 14 ≤ 1+10=11 ✗
- B–C (c=6): 14 ≤ 6+17=23 ✓; 17 ≤ 6+14=20 ✓
- B–D (c=9): 14 ≤ 9+11=20 ✓; 11 ≤ 9+14=23 ✓
- C–F (c=9): 17 ≤ 9+6=15 ✗; 6 ≤ 9+17=26 ✓

- D–F (c=7): 11 ≤ 7+6=13 ✓; 6 ≤ 7+11=18 ✓
- E–F (c=6): 14 ≤ 6+6=12 ✗; 6 ≤ 6+14=20 ✓
- F–G (c=9): 6 ≤ 9+0=9 ✓; 0 ≤ 9+6=15 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B147. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — D (1); A — E (9); B — D (7); B — E (2); A — C (1); C — F (7); D — F (6); E — F (5); F — G (3)
Heuristic values: S h=9, A h=9, B h=7, C h=7, D h=7, E h=4, F h=2, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge A→D: h(A) = 9 > 1 + h(D) = 8.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=6): 9 ≤ 6+9=15 ✓; 9 ≤ 6+9=15 ✓
- S–B (c=6): 9 ≤ 6+7=13 ✓; 7 ≤ 6+9=15 ✓
- A–D (c=1): 9 ≤ 1+7=8 ✗; 7 ≤ 1+9=10 ✓
- A–E (c=9): 9 ≤ 9+4=13 ✓; 4 ≤ 9+9=18 ✓
- B–D (c=7): 7 ≤ 7+7=14 ✓; 7 ≤ 7+7=14 ✓
- B–E (c=2): 7 ≤ 2+4=6 ✗; 4 ≤ 2+7=9 ✓
- A–C (c=1): 9 ≤ 1+7=8 ✗; 7 ≤ 1+9=10 ✓
- C–F (c=7): 7 ≤ 7+2=9 ✓; 2 ≤ 7+7=14 ✓
- D–F (c=6): 7 ≤ 6+2=8 ✓; 2 ≤ 6+7=13 ✓
- E–F (c=5): 4 ≤ 5+2=7 ✓; 2 ≤ 5+4=9 ✓
- F–G (c=3): 2 ≤ 3+0=3 ✓; 0 ≤ 3+2=5 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B148. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (2); S — A (1); A — E (8); A — D (6); B — D (4); B — C (1); C — H (1); C — F (2); D — F (3); E — H (9); F — G (7); H — G (2); B — F (4)
Heuristic values: S h=4, A h=4, B h=2, C h=2, D h=7, E h=10, F h=7, H h=1, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge E→A: h(E) = 10 > 4 + h(A) = 8.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.

- S–B (c=2): 4 ≤ 2+2=4 ✓; 2 ≤ 2+4=6 ✓
- S–A (c=1): 4 ≤ 1+4=5 ✓; 4 ≤ 1+4=5 ✓
- A–E (c=4): 4 ≤ 4+10=14 ✓; 10 ≤ 4+4=8 ✗
- A–D (c=6): 4 ≤ 6+7=13 ✓; 7 ≤ 6+4=10 ✓
- B–D (c=8): 2 ≤ 8+7=15 ✓; 7 ≤ 8+2=10 ✓
- B–C (c=1): 2 ≤ 1+2=3 ✓; 2 ≤ 1+2=3 ✓
- C–H (c=1): 2 ≤ 1+1=2 ✓; 1 ≤ 1+2=3 ✓
- C–F (c=2): 2 ≤ 2+7=9 ✓; 7 ≤ 2+2=4 ✗
- D–F (c=3): 7 ≤ 3+7=10 ✓; 7 ≤ 3+7=10 ✓
- E–H (c=9): 10 ≤ 9+1=10 ✓; 1 ≤ 9+10=19 ✓
- F–G (c=7): 7 ≤ 7+0=7 ✓; 0 ≤ 7+7=14 ✓
- H–G (c=2): 1 ≤ 2+0=2 ✓; 0 ≤ 2+1=3 ✓
- B–F (c=4): 2 ≤ 4+7=11 ✓; 7 ≤ 4+2=6 ✗
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B149. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (2); A — D (1); A — E (2); B — C (4); B — D (6); C — F (8); D — F (2); E — F (9); F — G (3); A — F (4)
Heuristic values: S h=4, A h=8, B h=8, C h=6, D h=3, E h=3, F h=2, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge A→S: h(A) = 8 > 2 + h(S) = 6.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=6): 4 ≤ 6+8=14 ✓; 8 ≤ 6+4=10 ✓
- S–A (c=2): 4 ≤ 2+8=10 ✓; 8 ≤ 2+4=6 ✗
- A–D (c=1): 8 ≤ 1+3=4 ✗; 3 ≤ 1+8=9 ✓
- A–E (c=2): 8 ≤ 2+7=9 ✓; 7 ≤ 2+8=10 ✓
- B–C (c=4): 8 ≤ 4+6=10 ✓; 6 ≤ 4+8=12 ✓
- B–D (c=6): 8 ≤ 6+3=9 ✓; 3 ≤ 6+8=14 ✓
- C–F (c=8): 6 ≤ 8+2=10 ✓; 2 ≤ 8+6=14 ✓
- D–F (c=2): 3 ≤ 2+2=4 ✓; 2 ≤ 2+3=5 ✓
- E–F (c=9): 7 ≤ 9+2=11 ✓; 2 ≤ 9+7=16 ✓
- F–G (c=3): 2 ≤ 3+0=3 ✓; 0 ≤ 3+2=5 ✓
- A–F (c=4): 8 ≤ 4+2=6 ✗; 2 ≤ 4+8=12 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B150. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (2); A — E (1); B — D (6); B — E (8); B — C (6); C — F (5); C — H (1); D — H (7); E — F (9); E — H (4); F — G (7); H — G (2)
Heuristic values: S h=8, A h=7, B h=5, C h=2, D h=2, E h=5, F h=11, H h=1, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge D→B: h(D) = 8 > 1 + h(B) = 6.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=3): 8 ≤ 3+5=8 ✓; 5 ≤ 3+8=11 ✓
- S–A (c=2): 8 ≤ 2+7=9 ✓; 7 ≤ 2+8=10 ✓
- A–E (c=6): 7 ≤ 6+5=11 ✓; 5 ≤ 6+7=13 ✓
- B–D (c=1): 5 ≤ 1+8=9 ✓; 8 ≤ 1+5=6 ✗
- B–E (c=8): 5 ≤ 8+5=13 ✓; 5 ≤ 8+5=13 ✓
- B–C (c=6): 5 ≤ 6+2=8 ✓; 2 ≤ 6+5=11 ✓
- C–F (c=5): 2 ≤ 5+11=16 ✓; 11 ≤ 5+2=7 ✗
- C–H (c=1): 2 ≤ 1+1=2 ✓; 1 ≤ 1+2=3 ✓
- D–H (c=9): 8 ≤ 9+1=10 ✓; 1 ≤ 9+8=17 ✓
- E–F (c=7): 5 ≤ 7+11=18 ✓; 11 ≤ 7+5=12 ✓
- E–H (c=4): 5 ≤ 4+1=5 ✓; 1 ≤ 4+5=9 ✓
- F–G (c=7): 11 ≤ 7+0=7 ✗; 0 ≤ 7+11=18 ✓
- H–G (c=2): 1 ≤ 2+0=2 ✓; 0 ≤ 2+1=3 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples**

SEARCH B151. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (1); A — D (8); B — E (1); B — C (8); C — H (9); C — F (6); D — F (9); D — H (1); E — F (5); F — G (5); H — G (5); B — F (3)
Heuristic values: S h=6, A h=7, B h=6, C h=8, D h=9, E h=5, F h=3, H h=3, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge D→H: h(D) = 9 > 5 + h(H) = 8.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=1): 6 ≤ 1+7=8 ✓; 7 ≤ 1+6=7 ✓
- S–B (c=1): 6 ≤ 1+6=7 ✓; 6 ≤ 1+6=7 ✓
- A–D (c=8): 7 ≤ 8+9=17 ✓; 9 ≤ 8+7=15 ✓
- B–E (c=1): 6 ≤ 1+5=6 ✓; 5 ≤ 1+6=7 ✓
- B–C (c=8): 6 ≤ 8+8=16 ✓; 8 ≤ 8+6=14 ✓
- C–H (c=9): 8 ≤ 9+3=12 ✓; 3 ≤ 9+8=17 ✓
- C–F (c=6): 8 ≤ 6+3=9 ✓; 3 ≤ 6+8=14 ✓

- D–F (c=9): 9 ≤ 9+3=12 ✓; 3 ≤ 9+9=18 ✓
- D–H (c=5): 9 ≤ 5+3=8 ✗; 3 ≤ 5+9=14 ✓
- E–F (c=1): 5 ≤ 1+3=4 ✗; 3 ≤ 1+5=6 ✓
- F–G (c=5): 3 ≤ 5+0=5 ✓; 0 ≤ 5+3=8 ✓
- H–G (c=5): 3 ≤ 5+0=5 ✓; 0 ≤ 5+3=8 ✓
- B–F (c=3): 6 ≤ 3+3=6 ✓; 3 ≤ 3+6=9 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B152. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (9); A — C (4); B — D (4); B — E (1); C — F (3); C — H (8); D — H (6); D — F (3); E — H (2); E — F (2); F — G (4); H — G (4)
Heuristic values: S h=9, A h=7, B h=6, C h=4, D h=3, E h=3, F h=3, H h=3, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge B→E: h(B) = 6 > 1 + h(E) = 4.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=8): 9 ≤ 8+7=15 ✓; 7 ≤ 8+9=17 ✓
- S–B (c=9): 9 ≤ 9+6=15 ✓; 6 ≤ 9+9=18 ✓
- A–C (c=4): 7 ≤ 4+4=8 ✓; 4 ≤ 4+7=11 ✓
- B–D (c=4): 6 ≤ 4+3=7 ✓; 3 ≤ 4+6=10 ✓
- B–E (c=1): 6 ≤ 1+3=4 ✗; 3 ≤ 1+6=7 ✓
- C–F (c=3): 4 ≤ 3+3=6 ✓; 3 ≤ 3+4=7 ✓
- C–H (c=8): 4 ≤ 8+3=11 ✓; 3 ≤ 8+4=12 ✓
- D–H (c=6): 3 ≤ 6+3=9 ✓; 3 ≤ 6+3=9 ✓
- D–F (c=3): 3 ≤ 3+3=6 ✓; 3 ≤ 3+3=6 ✓
- E–H (c=2): 3 ≤ 2+3=5 ✓; 3 ≤ 2+3=5 ✓
- E–F (c=2): 3 ≤ 2+3=5 ✓; 3 ≤ 2+3=5 ✓
- F–G (c=4): 3 ≤ 4+0=4 ✓; 0 ≤ 4+3=7 ✓
- H–G (c=4): 3 ≤ 4+0=4 ✓; 0 ≤ 4+3=7 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B153. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (1); S — A (2); A — C (9); A — D (9); B — E (1); B — C (5); C — F (2); C — H (2); D — F (5); D — H (9); E — F (6); E — H (3); F — G (4); H — G (8); B — H (6)
Heuristic values: S h=11, A h=12, B h=9, C h=5, D h=4, E h=9, F h=2, H h=6, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge S→B: h(S) = 11 > 1 + h(B) = 10.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=1): 11 ≤ 1+9=10 ✗; 9 ≤ 1+11=12 ✓
- S–A (c=2): 11 ≤ 2+12=14 ✓; 12 ≤ 2+11=13 ✓
- A–C (c=9): 12 ≤ 9+5=14 ✓; 5 ≤ 9+12=21 ✓
- A–D (c=9): 12 ≤ 9+4=13 ✓; 4 ≤ 9+12=21 ✓
- B–E (c=1): 9 ≤ 1+9=10 ✓; 9 ≤ 1+9=10 ✓
- B–C (c=5): 9 ≤ 5+5=10 ✓; 5 ≤ 5+9=14 ✓
- C–F (c=2): 5 ≤ 2+2=4 ✗; 2 ≤ 2+5=7 ✓
- C–H (c=5): 5 ≤ 5+6=11 ✓; 6 ≤ 5+5=10 ✓
- D–F (c=2): 4 ≤ 2+2=4 ✓; 2 ≤ 2+4=6 ✓
- D–H (c=9): 4 ≤ 9+6=15 ✓; 6 ≤ 9+4=13 ✓
- E–F (c=6): 9 ≤ 6+2=8 ✗; 2 ≤ 6+9=15 ✓
- E–H (c=3): 9 ≤ 3+6=9 ✓; 6 ≤ 3+9=12 ✓
- F–G (c=4): 2 ≤ 4+0=4 ✓; 0 ≤ 4+2=6 ✓
- H–G (c=8): 6 ≤ 8+0=8 ✓; 0 ≤ 8+6=14 ✓
- B–H (c=6): 9 ≤ 6+6=12 ✓; 6 ≤ 6+9=15 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B154. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (7); S — A (3); A — E (7); B — C (4); B — D (4); C — H (5); D — H (6); E — H (6); E — F (5); F — G (9); H — G (8)
Heuristic values: S h=19, A h=17, B h=13, C h=11, D h=12, E h=12, F h=7, H h=5, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge A→E: h(A) = 17 > 4 + h(E) = 12.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=7): 19 ≤ 7+13=20 ✓; 13 ≤ 7+19=26 ✓
- S–A (c=3): 19 ≤ 3+17=20 ✓; 17 ≤ 3+19=22 ✓
- A–E (c=4): 17 ≤ 4+8=12 ✗; 8 ≤ 4+17=21 ✓
- B–C (c=4): 13 ≤ 4+11=15 ✓; 11 ≤ 4+13=17 ✓
- B–D (c=7): 13 ≤ 7+12=19 ✓; 12 ≤ 7+13=20 ✓
- C–H (c=5): 11 ≤ 5+5=10 ✗; 5 ≤ 5+11=16 ✓
- D–H (c=6): 12 ≤ 6+5=11 ✗; 5 ≤ 6+12=18 ✓
- E–H (c=6): 8 ≤ 6+5=11 ✓; 5 ≤ 6+8=14 ✓
- E–F (c=5): 8 ≤ 5+7=12 ✓; 7 ≤ 5+8=13 ✓
- F–G (c=9): 7 ≤ 9+0=9 ✓; 0 ≤ 9+7=16 ✓
- H–G (c=8): 5 ≤ 8+0=8 ✓; 0 ≤ 8+5=13 ✓

- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B155. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (1); S — B (9); A — E (4); A — C (8); B — C (2); B — E (9); A — D (1); C — H (5); D — F (5); D — H (3); E — H (6); E — F (4); F — G (4); H — G (5); A — F (5)
Heuristic values: S h=5, A h=7, B h=7, C h=9, D h=5, E h=5, F h=3, H h=4, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge A→S: h(A) = 7 > 1 + h(S) = 6.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=1): 5 ≤ 1+7=8 ✓; 7 ≤ 1+5=6 ✗
- S–B (c=9): 5 ≤ 9+7=16 ✓; 7 ≤ 9+5=14 ✓
- A–E (c=4): 7 ≤ 4+6=10 ✓; 6 ≤ 4+7=11 ✓
- A–C (c=8): 7 ≤ 8+9=17 ✓; 9 ≤ 8+7=15 ✓
- B–C (c=1): 7 ≤ 1+9=10 ✓; 9 ≤ 1+7=8 ✗
- B–E (c=9): 7 ≤ 9+6=15 ✓; 6 ≤ 9+7=16 ✓
- A–D (c=2): 7 ≤ 2+5=7 ✓; 5 ≤ 2+7=9 ✓
- C–H (c=5): 9 ≤ 5+4=9 ✓; 4 ≤ 5+9=14 ✓
- D–F (c=5): 5 ≤ 5+3=8 ✓; 3 ≤ 5+5=10 ✓
- D–H (c=3): 5 ≤ 3+4=7 ✓; 4 ≤ 3+5=8 ✓
- E–H (c=6): 6 ≤ 6+4=10 ✓; 4 ≤ 6+6=12 ✓
- E–F (c=4): 6 ≤ 4+3=7 ✓; 3 ≤ 4+6=10 ✓
- F–G (c=4): 3 ≤ 4+0=4 ✓; 0 ≤ 4+3=7 ✓
- H–G (c=5): 4 ≤ 5+0=5 ✓; 0 ≤ 5+4=9 ✓
- A–F (c=5): 7 ≤ 5+3=8 ✓; 3 ≤ 5+7=12 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B156. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (7); A — E (9); A — D (2); B — D (5); B — C (2); C — F (3); D — F (3); E — F (5); F — G (6)
Heuristic values: S h=16, A h=7, B h=9, C h=7, D h=5, E h=5, F h=4, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge S→B: h(S) = 16 > 6 + h(B) = 15.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=6): 16 ≤ 6+9=15 ✗; 9 ≤ 6+16=22 ✓
- S–A (c=7): 16 ≤ 7+7=14 ✗; 7 ≤ 7+16=23 ✓
- A–E (c=2): 7 ≤ 2+9=11 ✓; 9 ≤ 2+7=9 ✓
- A–D (c=9): 7 ≤ 9+5=14 ✓; 5 ≤ 9+7=16 ✓
- B–D (c=5): 9 ≤ 5+5=10 ✓; 5 ≤ 5+9=14 ✓
- B–C (c=2): 9 ≤ 2+7=9 ✓; 7 ≤ 2+9=11 ✓
- C–F (c=3): 7 ≤ 3+4=7 ✓; 4 ≤ 3+7=10 ✓
- D–F (c=3): 5 ≤ 3+4=7 ✓; 4 ≤ 3+5=8 ✓
- E–F (c=5): 9 ≤ 5+4=9 ✓; 4 ≤ 5+9=14 ✓
- F–G (c=6): 4 ≤ 6+0=6 ✓; 0 ≤ 6+4=10 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B157. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (5); A — D (1); A — C (7); B — E (7); C — F (6); D — F (6); E — F (9); F — G (7)
Heuristic values: S h=12, A h=11, B h=20, C h=10, D h=12, E h=10, F h=6, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge B→S: h(B) = 20 > 5 + h(S) = 17.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–A (c=6): 12 ≤ 6+11=17 ✓; 11 ≤ 6+12=18 ✓
- S–B (c=5): 12 ≤ 5+20=25 ✓; 20 ≤ 5+12=17 ✗
- A–D (c=1): 11 ≤ 1+12=13 ✓; 12 ≤ 1+11=12 ✓
- A–C (c=7): 11 ≤ 7+10=17 ✓; 10 ≤ 7+11=18 ✓
- B–E (c=7): 20 ≤ 7+10=17 ✗; 10 ≤ 7+20=27 ✓
- C–F (c=6): 10 ≤ 6+6=12 ✓; 6 ≤ 6+10=16 ✓
- D–F (c=6): 12 ≤ 6+6=12 ✓; 6 ≤ 6+12=18 ✓
- E–F (c=9): 10 ≤ 9+6=15 ✓; 6 ≤ 9+10=19 ✓
- F–G (c=7): 6 ≤ 7+0=7 ✓; 0 ≤ 7+6=13 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B158. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (3); S — A (6); A — E (2); B — C (4); B — D (3); C — F (3); D — F (6); D — H (7); E — F (9); F — G (6); H — G (5); B — H (7)
Heuristic values: S h=9, A h=12, B h=6, C h=8, D h=8, E h=13, F h=4, H h=4, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge C→F: h(C) = 8 > 3 + h(F) = 7.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=3): 9 ≤ 3+6=9 ✓; 6 ≤ 3+9=12 ✓
- S–A (c=6): 9 ≤ 6+12=18 ✓; 12 ≤ 6+9=15 ✓
- A–E (c=4): 12 ≤ 4+13=17 ✓; 13 ≤ 4+12=16 ✓
- B–C (c=2): 6 ≤ 2+8=10 ✓; 8 ≤ 2+6=8 ✓
- B–D (c=3): 6 ≤ 3+7=10 ✓; 7 ≤ 3+6=9 ✓
- C–F (c=3): 8 ≤ 3+4=7 ✗; 4 ≤ 3+8=11 ✓
- D–F (c=6): 7 ≤ 6+4=10 ✓; 4 ≤ 6+7=13 ✓
- D–H (c=7): 7 ≤ 7+4=11 ✓; 4 ≤ 7+7=14 ✓
- E–F (c=9): 13 ≤ 9+4=13 ✓; 4 ≤ 9+13=22 ✓
- F–G (c=6): 4 ≤ 6+0=6 ✓; 0 ≤ 6+4=10 ✓
- H–G (c=5): 4 ≤ 5+0=5 ✓; 0 ≤ 5+4=9 ✓
- B–H (c=7): 6 ≤ 7+4=11 ✓; 4 ≤ 7+6=13 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B159. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (6); S — A (3); A — D (7); A — E (5); B — D (9); B — C (1); C — F (1); C — H (1); D — H (8); E — F (8); E — H (7); F — G (8); H — G (8); B — F (1)
Heuristic values: S h=11, A h=14, B h=5, C h=7, D h=9, E h=9, F h=5, H h=7, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge C→B: h(C) = 7 > 1 + h(B) = 6.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=6): 11 ≤ 6+5=11 ✓; 5 ≤ 6+11=17 ✓
- S–A (c=3): 11 ≤ 3+14=17 ✓; 14 ≤ 3+11=14 ✓
- A–D (c=7): 14 ≤ 7+9=16 ✓; 9 ≤ 7+14=21 ✓
- A–E (c=5): 14 ≤ 5+12=17 ✓; 12 ≤ 5+14=19 ✓
- B–D (c=9): 5 ≤ 9+9=18 ✓; 9 ≤ 9+5=14 ✓
- B–C (c=1): 5 ≤ 1+7=8 ✓; 7 ≤ 1+5=6 ✗
- C–F (c=1): 7 ≤ 1+5=6 ✗; 5 ≤ 1+7=8 ✓
- C–H (c=1): 7 ≤ 1+7=8 ✓; 7 ≤ 1+7=8 ✓
- D–H (c=8): 9 ≤ 8+7=15 ✓; 7 ≤ 8+9=17 ✓
- E–F (c=8): 12 ≤ 8+5=13 ✓; 5 ≤ 8+12=20 ✓
- E–H (c=7): 12 ≤ 7+7=14 ✓; 7 ≤ 7+12=19 ✓
- F–G (c=8): 5 ≤ 8+0=8 ✓; 0 ≤ 8+5=13 ✓

- H–G (c=8): 7 ≤ 8+0=8 ✓; 0 ≤ 8+7=15 ✓
- B–F (c=1): 5 ≤ 1+5=6 ✓; 5 ≤ 1+5=6 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B160. [TF · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (8); S — A (3); A — E (1); A — C (7); B — E (7); B — C (6); B — D (9); C — F (4); C — H (4); D — F (7); E — F (3); E — H (3); F — G (7); H — G (1)
Heuristic values: S h=13, A h=9, B h=9, C h=6, D h=10, E h=3, F h=6, H h=0, G h=0

True or False: the heuristic h on the graph is consistent, i.e. h(n) ≤ c(n, n′) + h(n′) for every edge (both directions).

Answer: False

Why: Edge S→A: h(S) = 13 > 3 + h(A) = 12.

Working:

- Check h(n) ≤ c(n,n′) + h(n′) on every edge, in both directions.
- S–B (c=8): 13 ≤ 8+9=17 ✓; 9 ≤ 8+13=21 ✓
- S–A (c=3): 13 ≤ 3+9=12 ✗; 9 ≤ 3+13=16 ✓
- A–E (c=9): 9 ≤ 9+3=12 ✓; 3 ≤ 9+9=18 ✓
- A–C (c=7): 9 ≤ 7+6=13 ✓; 6 ≤ 7+9=16 ✓
- B–E (c=7): 9 ≤ 7+3=10 ✓; 3 ≤ 7+9=16 ✓
- B–C (c=6): 9 ≤ 6+6=12 ✓; 6 ≤ 6+9=15 ✓
- B–D (c=1): 9 ≤ 1+10=11 ✓; 10 ≤ 1+9=10 ✓
- C–F (c=4): 6 ≤ 4+6=10 ✓; 6 ≤ 4+6=10 ✓
- C–H (c=7): 6 ≤ 7+0=7 ✓; 0 ≤ 7+6=13 ✓
- D–F (c=4): 10 ≤ 4+6=10 ✓; 6 ≤ 4+10=14 ✓
- E–F (c=3): 3 ≤ 3+6=9 ✓; 6 ≤ 3+3=6 ✓
- E–H (c=3): 3 ≤ 3+0=3 ✓; 0 ≤ 3+3=6 ✓
- F–G (c=7): 6 ≤ 7+0=7 ✓; 0 ≤ 7+6=13 ✓
- H–G (c=1): 0 ≤ 1+0=1 ✓; 0 ≤ 1+0=1 ✓
- At least one edge fails → NOT consistent.

Takeaway: Consistent ⇒ admissible, but admissible does not imply consistent. Graph-search A* is optimal with a consistent h.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B161. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (7); A — E (9); A — C (9); B — D (8); B — C (7); C — F (8); C — H (5); D — H (5); E — H (4); E — F (7); F — G (3); H — G (5); A — H (5)
Heuristic values: S h=10, A h=9, B h=13, C h=7, D h=7, E h=5, F h=1, H h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 12; Greedy cost = 12; A* cost = 12

Working:

- UCS order S,A,B,H,E,C,D,G → path S-A-H-G, cost 12.
- Greedy order S,A,H,G → path S-A-H-G, cost 12.
- A* order S,A,H,G → path S-A-H-G, cost 12.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 12
  Greedy | 4 | 12
  A* | 4 | 12

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B162. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (3); A — D (3); B — C (6); B — E (9); C — H (6); D — H (4); E — H (3); C — F (5); F — G (1); H — G (7)
Heuristic values: S h=11, A h=9, B h=11, C h=3, D h=3, E h=5, F h=0, H h=5, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 15; Greedy cost = 23; A* cost = 15

Working:

- UCS order S,B,A,C,D,E,F,G → path S-B-C-F-G, cost 15.
- Greedy order S,A,D,H,G → path S-A-D-H-G, cost 23.
- A* order S,B,C,F,G → path S-B-C-F-G, cost 15.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 15
  Greedy | 5 | 23
  A* | 5 | 15

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B163. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (5); S — B (2); A — C (2); B — E (5); B — D (9); C — F (1); D — F (5); E — F (4); F — G (7)
Heuristic values: S h=10, A h=6, B h=14, C h=5, D h=8, E h=10, F h=4, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry

is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 15; Greedy cost = 15; A* cost = 15

Working:

- UCS order S,B,A,C,E,F,D,G → path S-A-C-F-G, cost 15.
- Greedy order S,A,C,F,G → path S-A-C-F-G, cost 15.
- A* order S,A,C,F,G → path S-A-C-F-G, cost 15.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 15
  Greedy | 5 | 15
  A* | 5 | 15

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B164. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (8); A — C (1); B — E (2); B — C (1); B — D (6); C — H (9); C — F (2); D — H (3); E — F (8); F — G (5); H — G (7)
Heuristic values: S h=12, A h=6, B h=4, C h=3, D h=8, E h=5, F h=4, H h=6, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 15; Greedy cost = 16; A* cost = 16

Working:

- UCS order S,A,B,C,E,F,D,G → path S-A-C-F-G, cost 15.
- Greedy order S,B,C,F,G → path S-B-C-F-G, cost 16.
- A* order S,B,C,A,E,F,G → path S-B-C-F-G, cost 16.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 15
  Greedy | 5 | 16
  A* | 7 | 16

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B165. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (8); A — D (8); A — E (4); B — E (6); B — C (2); C — F (4); C — H (5); D — H (7); E — F (2); E — H (6); F — G (5); H — G (4); B — H (2)
Heuristic values: S h=11, A h=11, B h=5, C h=5, D h=4, E h=4, F h=3, H h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 14; Greedy cost = 14; A* cost = 14

Working:

- UCS order S,A,B,C,H,E,D,F,G → path S-B-H-G, cost 14.
- Greedy order S,B,H,G → path S-B-H-G, cost 14.
- A* order S,B,H,G → path S-B-H-G, cost 14.
- Algorithm | Nodes expanded | Cost
  UCS | 9 | 14
  Greedy | 4 | 14
  A* | 4 | 14

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B166. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (4); S — A (6); A — E (8); A — D (4); B — C (5); B — D (7); C — F (7); D — F (3); E — F (5); F — G (5)
Heuristic values: S h=10, A h=9, B h=14, C h=11, D h=5, E h=5, F h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 19; Greedy cost = 22; A* cost = 19

Working:

- UCS order S,B,A,C,D,E,F,G → path S-B-D-F-G, cost 19.
- Greedy order S,A,D,F,G → path S-A-D-F-G, cost 22.
- A* order S,A,E,B,D,F,C,G → path S-B-D-F-G, cost 19.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 19
  Greedy | 5 | 22
  A* | 8 | 19

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B167. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (7); S — B (5); A — C (7); B — C (1); B — E (3); A — D (2); C — H (1); C — F (4); D — F (4); E — F (6); F — G (7); H — G (4); B — H (5)

Heuristic values: S h=10, A h=8, B h=6, C h=4, D h=7, E h=7, F h=4, H h=3, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 12; Greedy cost = 14; A* cost = 12

Working:

- UCS order S,B,A,C,D,E,H,F,G → path S-B-C-H-G, cost 12.
- Greedy order S,B,H,G → path S-B-H-G, cost 14.
- A* order S,B,C,H,G → path S-B-C-H-G, cost 12.
- Algorithm | Nodes expanded | Cost
  UCS | 9 | 12
  Greedy | 4 | 14
  A* | 5 | 12

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B168. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (5); S — A (1); A — E (8); A — D (8); B — C (1); B — E (9); C — F (3); C — H (1); D — F (1); D — H (4); E — H (4); F — G (9); H — G (1)
Heuristic values: S h=6, A h=3, B h=7, C h=1, D h=3, E h=3, F h=4, H h=0, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 7; Greedy cost = 14; A* cost = 7

Working:

- UCS order S,A,E,B,H,C,G → path S-A-E-H-G, cost 7.
- Greedy order S,A,D,H,G → path S-A-D-H-G, cost 14.
- A* order S,A,E,H,G → path S-A-E-H-G, cost 7.
- Algorithm | Nodes expanded | Cost
  UCS | 7 | 7
  Greedy | 5 | 14
  A* | 5 | 7

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B169. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (2); S — B (8); A — E (4); B — C (9); B — E (6); B — D (6); C — F (1); D — F (2); E — F (6); F — G (6); A — F
Heuristic values: S h=10, A h=8, B h=9, C h=6, D h=7, E h=6, F h=5, G h=0
Other labels: 9

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 17; Greedy cost = 17; A* cost = 17

Working:

- UCS order S,A,B,E,F,C,D,G → path S-A-F-G, cost 17.
- Greedy order S,A,F,G → path S-A-F-G, cost 17.
- A* order S,A,F,B,E,G → path S-A-F-G, cost 17.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 17
  Greedy | 4 | 17
  A* | 6 | 17

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B170. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (9); A — C (3); A — D (8); B — C (5); B — E (6); C — H (2); D — H (7); D — F (7); E — H (6); E — F (8); F — G (9); H — G (3); A — H
Heuristic values: S h=10, A h=6, B h=7, C h=4, D h=4, E h=5, F h=8, H h=2, G h=0
Other labels: 7

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 17; Greedy cost = 17; A* cost = 17

Working:

- UCS order S,A,B,C,H,E,D,G → path S-A-H-G, cost 17.
- Greedy order S,A,H,G → path S-A-H-G, cost 17.
- A* order S,A,B,C,H,G → path S-A-H-G, cost 17.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 17
  Greedy | 4 | 17
  A* | 6 | 17

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B171. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (5); S — A (6); A — D (2); B — E (4); B — D (9); B — C (5); C — F (1); C — H (6); D — H (8); E — F (8); F — G (1); H — G (8); A — F (4)
Heuristic values: S h=7, A h=3, B h=6, C h=1, D h=5, E h=6, F h=0, H h=7, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 11; Greedy cost = 11; A* cost = 11

Working:

- UCS order S,B,A,D,E,C,F,G → path S-A-F-G, cost 11.
- Greedy order S,A,F,G → path S-A-F-G, cost 11.
- A* order S,A,F,B,C,G → path S-A-F-G, cost 11.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 11
  Greedy | 4 | 11
  A* | 6 | 11

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B172. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (9); S — B (5); A — D (8); B — C (9); B — E (2); C — F (6); D — F (4); E — F (8); F — G (3); A — F (6)
Heuristic values: S h=11, A h=8, B h=8, C h=6, D h=5, E h=5, F h=1, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 18; Greedy cost = 18; A* cost = 18

Working:

- UCS order S,B,E,A,C,F,D,G → path S-B-E-F-G, cost 18.
- Greedy order S,A,F,G → path S-A-F-G, cost 18.
- A* order S,B,A,F,E,G → path S-A-F-G, cost 18.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 18

Greedy | 4 | 18
A* | 6 | 18

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B173. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (6); S — B (6); A — E (7); A — D (4); B — C (6); B — E (2); C — H (6); D — F (8); D — H (7); E — F (7); E — H (1); F — G (3); H — G (3); A — H (1)
Heuristic values: S h=6, A h=2, B h=4, C h=6, D h=6, E h=3, F h=2, H h=2, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 10; Greedy cost = 10; A* cost = 10

Working:

- UCS order S,A,B,H,E,D,G → path S-A-H-G, cost 10.
- Greedy order S,A,H,G → path S-A-H-G, cost 10.
- A* order S,A,H,B,G → path S-A-H-G, cost 10.
- Algorithm | Nodes expanded | Cost
  UCS | 7 | 10
  Greedy | 4 | 10
  A* | 5 | 10

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B174. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (3); A — D (3); A — C (8); B — C (8); B — E (4); C — F (3); D — F (2); E — F (9); F — G (6)
Heuristic values: S h=15, A h=10, B h=10, C h=8, D h=7, E h=13, F h=4, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 19; Greedy cost = 19; A* cost = 19

Working:

- UCS order S,B,E,A,C,D,F,G → path S-A-D-F-G, cost 19.
- Greedy order S,A,D,F,G → path S-A-D-F-G, cost 19.
- A* order S,B,A,D,F,C,G → path S-A-D-F-G, cost 19.

- Algorithm | Nodes expanded | Cost
  UCS | 8 | 19
  Greedy | 5 | 19
  A* | 7 | 19

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B175. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — A (8); S — B (1); A — E (1); A — D (6); B — D (1); B — C (5); C — F (8); C — H (1); D — H (9); E — H (3); E — F (2); F — G (4); H — G (4); B — F (8)
Heuristic values: S h=6, A h=4, B h=6, C h=4, D h=6, E h=6, F h=3, H h=2, G h=0

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 11; Greedy cost = 16; A* cost = 11

Working:

- UCS order S,B,D,C,H,A,E,F,G → path S-B-C-H-G, cost 11.
- Greedy order S,A,E,H,G → path S-A-E-H-G, cost 16.
- A* order S,B,D,C,H,G → path S-B-C-H-G, cost 11.
- Algorithm | Nodes expanded | Cost
  UCS | 9 | 11
  Greedy | 5 | 16
  A* | 6 | 11

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

SEARCH B176. [BLANKS · Exam+ · calculation]

[Diagram as text] Edges (label in brackets): S — B (9); S — A (3); A — E (3); A — D (3); B — E (3); A — C; C — F (5); D — F (1); E — F (2); F — G (9)
Heuristic values: S h=9, A h=10, B h=10, C h=11, D h=9, E h=9, F h=8, G h=0
Other labels: 9

Run UCS, Greedy Best-First and A* from S to G. Conventions (as in Lecture 2 pseudocode): graph search; the goal test is applied when a node is selected for expansion; ties are broken alphabetically; a frontier entry is replaced if a cheaper path to it is found (UCS/A*); edges are two-way.
Give the path cost returned by each, in the order UCS, Greedy, A* (integers).

(1) UCS cost = ____
(2) Greedy cost = ____
(3) A* cost = ____

Answer: UCS cost = 16; Greedy cost = 23; A* cost = 16

Working:

- UCS order S,A,C,D,F,B,E,G → path S-A-D-F-G, cost 16.

- Greedy order S,A,E,F,G → path S-A-E-F-G, cost 23.
- A* order S,A,D,F,E,G → path S-A-D-F-G, cost 16.
- Algorithm | Nodes expanded | Cost
  UCS | 8 | 16
  Greedy | 5 | 23
  A* | 6 | 16

Takeaway: UCS and A* (admissible h) are optimal; Greedy is not guaranteed to be.

Source: Lecture 2 (Classic search) + UCS/GBFS/A Romania examples*

🌳 Decision Trees (Lecture 4)

Part A: hand-written questions (62)

TREES A1. [MCQ · Easy]

In a decision tree, what do internal nodes represent?

A. Learning rates
B. Training examples
C. Tests on attribute values
D. Final class predictions

Answer: C. Tests on attribute values

Option notes:

- A: Trees don't use a learning rate.
- B: Examples flow through the tree; they aren't nodes.
- D: Those are leaves.

Takeaway: Internal node = attribute test; leaf = output.

Source: Lecture 4, slide 34

TREES A2. [MATCH · Medium]

Match each decision-tree algorithm to its split criterion.

1. ID3
2. C4.5
3. CART

A. Information gain
B. Gini / MSE with binary splits
C. Mean absolute percentage error
D. Gain ratio + pruning

Answer: 1 → A, 2 → D, 3 → B

Working:

- ID3 → Information gain
- C4.5 → Gain ratio + pruning
- CART → Gini / MSE with binary splits

Takeaway: ID3 → IG, C4.5 → gain ratio, CART → Gini.

Source: Lecture 4, slide 27

TREES A3. [MULTI · Medium]

Which settings help control overfitting of a sklearn decision tree? (Select ALL that apply.)

A. min_samples_leaf
B. random_state
C. ccp_alpha
D. criterion='entropy'
E. max_depth

Answer: A. min_samples_leaf; C. ccp_alpha; E. max_depth

Option notes:

- A: Correct: Prevents tiny leaves.
- B: Incorrect: Only makes results reproducible.
- C: Correct: Pruning strength.
- D: Incorrect: Changes the split measure, not complexity.
- E: Correct: Limits tree depth.

Takeaway: Deep trees overfit; limit or prune them.

Source: Lecture 4, slides 28-30

TREES A4. [TF · Easy]

Decision trees can be used for both classification and regression.

Answer: True

Why: Lecture 4: DecisionTreeClassifier and DecisionTreeRegressor (MSE / variance reduction).

Takeaway: Regression trees store constant numeric predictions in leaves.

Source: Lecture 4 (Decision trees)

TREES A5. [BLANK · Medium]

[Diagram as text] Labels: Patrons? · None · Some · Full · No · Yes · WaitEstimate? · >60 No · 30-60 Alternate? · 10-30 Hungry? · 0-10 Yes

```
[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

In the restaurant tree (Lecture 4), Patrons = Some leads directly to the leaf WillWait = _____.

Answer: Yes
(Also accepted: yes, True, true)

Takeaway: Some → all 4 examples positive.

Source: Lecture 4, slide 34

TREES A6. [MCQ · Hard]

For DecisionTreeRegressor, what does score() return and what do leaves store?

A. R^2 on the given data; constant numeric predictions
B. Accuracy; class labels
C. F1; probabilities
D. MSE; class probabilities

Answer: A. R^2 on the given data; constant numeric predictions

Option notes:

- B: That's the classifier.
- C: Classification metrics.
- D: score() is R^2 for regressors.

Takeaway: Regressor leaves = constants; score = R^2.

Source: Lecture 4, slide 29

TREES A7. [TF · Easy]

The entropy of a pure node (all examples in one class) is 0.

Answer: True

Why: No uncertainty → H = 0.

Takeaway: Pure = 0, 50/50 = 1.

Source: Lecture 4 (Decision trees)

TREES A8. [TF · Medium]

For a binary class variable, the maximum possible entropy is 0.5.

Answer: False

Why: 0.5 is the maximum GINI. Maximum binary entropy is 1 bit at 50/50.

Takeaway: Entropy max 1, Gini max 0.5.

Source: Lecture 4 (Decision trees)

TREES A9. [MCQ · Easy]

Which node has the LOWEST entropy?

A. [3+, 3−]
B. [9+, 1−]
C. [6+, 4−]
D. [5+, 5−]

Answer: B. [9+, 1−]

Option notes:

- A: 50/50 → 1.
- C: Closer to 50/50 → higher entropy.
- D: 50/50 → maximum entropy (1).

Takeaway: The more homogeneous, the lower the entropy (90/10 ≈ 0.47).

Source: Lecture 4, slide 36

TREES A10. [BLANK · Medium · calculation]

Entropy of [2+, 4−] in bits (4 decimal places).

Answer: 0.9183
(Also accepted: 0.918)

Takeaway: $-(1/3)\log_2(1/3) - (2/3)\log_2(2/3)$.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A11. [BLANK · Easy]

In Lecture 4 notation, B(q) is the entropy of a Boolean variable that is true with probability q. B(1) = _____ (all samples positive).

Answer: 0
(Also accepted: 0.0)

Takeaway: Certain variable → entropy 0.

Source: Lecture 4, slide 38

TREES A12. [MCQ · Hard · calculation]

Using natural log instead of $\log_2$ for H([5+, 5−]) gives…

A. 2.0
B. 0.693, a different unit (nats), so values won't match the expected bits
C. 0.5
D. 1.0, the same as $\log_2$

Answer: B. 0.693, a different unit (nats), so values won't match the expected bits

Option notes:

- A: Entropy of two classes never exceeds 1 bit.
- C: That's the Gini of a 50/50 node.
- D: ln 2 ≈ 0.693, not 1.

Takeaway: Always use $\log_2$ for entropy here.

Source: Lecture 4 (Decision trees)

TREES A13. [BLANK · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Restaurant data: IG(Patrons) = 1 − [2/12·0 + 4/12·0 + 6/12·0.9183] = _____ bits (4 dp).

Answer: 0.5409
(Also accepted: 0.541)

Takeaway: Weighted child entropy 0.4591.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A14. [BLANK · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Restaurant data: every Type branch (French 1/1, Italian 1/1, Thai 2/2, Burger 2/2) is 50/50. IG(Type) = _____.

Answer: 0
(Also accepted: 0.0, 0.0000)

Takeaway: Weighted child entropy = 1 = parent entropy.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A15. [TF · Medium]

Information gain is computed by subtracting the plain (unweighted) average of the child entropies from the parent entropy.

Answer: False

Why: Children must be weighted by $|S_v|/|S|$.

Takeaway: Weighted, always.

Source: Lecture 4 (Decision trees)

TREES A16. [MCQ · Medium]

ID3 chooses the attribute with…

A. the highest information gain
B. the highest weighted child entropy
C. the lowest information gain
D. the most branches

Answer: A. the highest information gain

Option notes:

- B: That would be the WORST split.
- C: Lower IG = less uncertainty removed.
- D: Number of values is not the criterion (it can even bias IG).

Takeaway: Max IG = min weighted child entropy.

Source: Lecture 4 (Decision trees)

TREES A17. [MULTI · Hard · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Parent [6+, 6−] split by Patrons into None [0+,2−], Some [4+,0−], Full [2+,4−]. Select ALL correct values. (Select ALL that apply.)

A. H(parent) = 1.0
B. Weighted child entropy = 0.4591
C. IG = 0.4591
D. Weighted child Gini = 0.2222
E. H(None) = H(Some) = 0

Answer: A. H(parent) = 1.0; B. Weighted child entropy = 0.4591; D. Weighted child Gini = 0.2222; E. H(None) = H(Some) = 0

Option notes:

- A: Correct: 50/50.
- B: Correct: 6/12 × 0.9183.
- C: Incorrect: IG = 1 − 0.4591 = 0.5409.
- D: Correct: 6/12 × 0.4444.
- E: Correct: Pure.

Takeaway: Do the parent, children, weight, subtract.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A18. [MCQ · Easy · calculation]

Gini index of [2+, 4−]?

A. 0.9183
B. 0.4444
C. 0.5556
D. 0.2222

Answer: B. 0.4444

Option notes:

- A: That's entropy.
- C: That's Σp^2 (forgot 1 −).
- D: That's the WEIGHTED Gini (× 6/12).

Takeaway: $1 - (1/3)^2 - (2/3)^2 = 4/9$.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A19. [TF · Medium]

When using the Gini index, we choose the split with the HIGHEST weighted child Gini.

Answer: False

Why: Choose the LOWEST weighted child Gini (= largest Gini reduction).

Takeaway: Impurity after the split should be small.

Source: Lecture 4 (Decision trees)

TREES A20. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Patrons split: weighted child Gini = _____ and Gini reduction = _____ (4 dp).

(1) weighted Gini = ____
(2) reduction = ____

Answer: weighted Gini = 0.2222; reduction = 0.2778

Takeaway: 0.5 − 0.2222 = 0.2778.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A21. [BLANK · Hard · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Lecture 4 also tries a CART binary split on Patrons: Some vs (None or Full). The Gini gain of that binary split is _____.

Answer: 0.25
(Also accepted: 0.250, 1/4)

Takeaway: Weighted Gini after split 0.25 → gain 0.5 − 0.25.

Source: Lecture 4, slide 41

TREES A22. [MATCH · Medium]

Match each quantity to its value for a binary node.

1. Maximum entropy
2. Maximum Gini
3. Entropy of a pure node
4. Entropy of a 90/10 node (approx.)

A. 0.69
B. 0
C. 0.47
D. 1
E. 0.5

Answer: 1 → D, 2 → E, 3 → B, 4 → C

Working:

- Maximum entropy → 1
- Maximum Gini → 0.5
- Entropy of a pure node → 0
- Entropy of a 90/10 node (approx.) → 0.47

Takeaway: Max entropy 1 bit; max Gini 0.5.

Source: Lecture 4, slide 36

TREES A23. [MCQ · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Type gives IG = 0 and weighted Gini 0.5; Patrons gives IG = 0.5409 and weighted Gini 0.2222. Which attribute is chosen?

A. Type by both criteria
B. Neither: the tree stops
C. Patrons by both criteria
D. Patrons by IG, Type by Gini

Answer: C. Patrons by both criteria

Option notes:

- A: Type removes no uncertainty.

- B: Patrons gives a clear gain.
- D: Lower weighted Gini (0.2222) is better → Patrons.

Takeaway: Highest IG and lowest weighted Gini both pick Patrons.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A24. [TF · Hard]

Information gain and Gini reduction always choose the same attribute.

Answer: False

Why: They usually agree but can disagree on close splits (different impurity curves).

Takeaway: Compute both when asked.

Source: Lecture 4 (Decision trees)

TREES A25. [MULTI · Medium]

Order of steps when comparing two attributes (the step-by-step file). Select ALL steps that are part of the process. (Select ALL that apply.)

A. Compute the parent's entropy and Gini
B. Pick the attribute with the most branches
C. Compute each branch's entropy/Gini
D. Weight each branch by |subset|/|parent|
E. Subtract from the parent to get IG / Gini reduction

Answer: A. Compute the parent's entropy and Gini; C. Compute each branch's entropy/Gini; D. Weight each branch by |subset|/|parent|; E. Subtract from the parent to get IG / Gini reduction

Option notes:

- A: Correct: Step 2 of the file.
- B: Incorrect: Not a criterion.
- C: Correct: Per subset.
- D: Correct: Weighted child value.
- E: Correct: Final comparison.

Takeaway: Parent → branches → weight → subtract → compare.

Source: Comparing two attributes to split (Type vs Patrons)

TREES A26. [TF · Easy]

A decision path runs from the root node to a leaf node and corresponds to an if/then rule.

Answer: True

Why: Lecture 4 slide 27/34.

Takeaway: Root → internal splits → leaf.

Source: Lecture 4 (Decision trees)

TREES A27. [BLANK · Medium]

The restaurant classification problem has a Boolean output variable called _____.

Answer: WillWait
(Also accepted: willwait, Will Wait)

Takeaway: Target: WillWait.

Source: Lecture 4, slide 33

TREES A28. [MULTI · Hard]

[Diagram as text] Labels: Patrons? · None · Some · Full · No · Yes · WaitEstimate? · >60 No · 30-60 Alternate? · 10-30 Hungry? · 0-10 Yes

Restaurant tree (Lecture 4): select ALL correct statements. (Select ALL that apply.)

A. Patrons = Some → test Hungry
B. WaitEstimate > 60 → No
C. Patrons = None → No
D. Patrons = Full → test WaitEstimate

Answer: B. WaitEstimate > 60 → No; C. Patrons = None → No; D. Patrons = Full → test WaitEstimate

Option notes:

- A: Incorrect: Some → Yes directly.
- B: Correct:
- C: Correct:
- D: Correct:

Takeaway: Read the tree top-down.

Source: Lecture 4, slide 34

TREES A29. [MCQ · Medium]

Which sklearn criterion values can DecisionTreeClassifier use?

A. "accuracy", "f1"
B. "gini", "entropy", "log_loss"
C. "rbf", "poly"
D. "squared_error", "absolute_error"

Answer: B. "gini", "entropy", "log_loss"

Option notes:

- A: Metrics, not split criteria.
- C: SVM kernels.
- D: Regressor criteria.

Takeaway: Classifier: gini/entropy/log_loss.

Source: Lecture 4, slide 28

TREES A30. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Helen's step-by-step sheet (Restaurant data, 6 yes / 6 no). Type splits 1/1, 1/1, 2/2, 2/2; Patrons splits None 0/2, Some 4/0, Full 2/4. Fill in (4 d.p.).

(1) IG(Type) = ____
(2) IG(Patrons) = ____

(3) Gini reduction Type = ____
(4) Gini reduction Patrons = ____

Answer: IG(Type) = 0; IG(Patrons) = 0.5409; Gini reduction Type = 0; Gini reduction Patrons = 0.2778

Working:

- Parent H = 1, Gini = 0.5.
- Type: every branch 50/50 → weighted H = 1 → IG 0; weighted Gini 0.5 → reduction 0.
- Patrons: H(Full) = B(1/3) = 0.9183; weighted 6/12 × 0.9183 = 0.4591; IG = 0.5409.
- Gini(Full) = 1 − 1/9 − 4/9 = 0.4444; weighted 0.2222; reduction 0.2778.

Takeaway: Patrons wins under both criteria.

Source: Book 2 (Exam examples), §4.1 step-by-step sheet, p. 16

TREES A31. [BLANK · Medium · calculation]

Attribute | Value: Yes / No
Outlook | Sunny 2/3, Overcast 4/0, Rain 3/2
Wind | Weak 6/2, Strong 3/3
Humidity | High 3/4, Normal 6/1
All 14 days | 9 Yes / 5 No

Play-tennis data (14 days, 9 yes / 5 no). H(parent) = ___ (3 d.p.).

Answer: 0.940

Working:

- $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$
- = 0.410 + 0.530 = 0.940

Takeaway: B(9/14).

Source: Book 2 (Exam examples), Q80, p. 17

TREES A32. [BLANK · Hard · calculation]

Attribute | Value: Yes / No
Outlook | Sunny 2/3, Overcast 4/0, Rain 3/2
Wind | Weak 6/2, Strong 3/3
Humidity | High 3/4, Normal 6/1
All 14 days | 9 Yes / 5 No

Play-tennis: IG(Outlook) = ___ (3 d.p.).

Answer: 0.247
(Also accepted: 0.246)

Working:

- Sunny 2Y3N: B(2/5) = 0.971; Overcast 4Y0N: 0; Rain 3Y2N: 0.971.
- Weighted = 5/14(0.971) + 0 + 5/14(0.971) = 0.694.
- IG = 0.940 − 0.694 = 0.247.

Takeaway: Sunny 0.971, Overcast 0, Rain 0.971.

Source: Book 2 (Exam examples), Q81, p. 17

TREES A33. [BLANK · Hard · calculation]

Attribute | Value: Yes / No
Outlook | Sunny 2/3, Overcast 4/0, Rain 3/2

Wind | Weak 6/2, Strong 3/3
Humidity | High 3/4, Normal 6/1
All 14 days | 9 Yes / 5 No

Play-tennis: IG(Wind) = ___ (3 d.p.).

Answer: 0.048

Working:

- Weighted = 8/14(0.811) + 6/14(1) = 0.892.
- IG = 0.940 − 0.892 = 0.048.

Takeaway: Weak B(6/8) = 0.811, Strong 1.0.

Source: Book 2 (Exam examples), Q82, p. 17

TREES A34. [BLANK · Hard · calculation]

Attribute | Value: Yes / No
Outlook | Sunny 2/3, Overcast 4/0, Rain 3/2
Wind | Weak 6/2, Strong 3/3
Humidity | High 3/4, Normal 6/1
All 14 days | 9 Yes / 5 No

Play-tennis: IG(Humidity) = ___ (3 d.p.).

Answer: 0.152
(Also accepted: 0.151)

Working:

- Weighted = 0.5(0.985) + 0.5(0.592) = 0.788.
- IG = 0.940 − 0.788 = 0.152.

Takeaway: High B(3/7) = 0.985, Normal B(6/7) = 0.592.

Source: Book 2 (Exam examples), Q83, p. 17

TREES A35. [MCQ · Medium]

Attribute | Value: Yes / No
Outlook | Sunny 2/3, Overcast 4/0, Rain 3/2
Wind | Weak 6/2, Strong 3/3
Humidity | High 3/4, Normal 6/1
All 14 days | 9 Yes / 5 No

Play-tennis: which attribute does ID3 choose at the root (Outlook 0.247, Humidity 0.152, Wind 0.048)?

A. Humidity
B. Cannot be decided
C. Wind
D. Outlook

Answer: D. Outlook

Option notes:

- A: Second.
- B: Pick the largest IG.
- C: Smallest IG.

Takeaway: ID3 = largest information gain.

Source: Book 2 (Exam examples), Q84, p. 17

TREES A36. [BLANKS · Hard · calculation]

Attribute | Value: Yes / No
Outlook | Sunny 2/3, Overcast 4/0, Rain 3/2
Wind | Weak 6/2, Strong 3/3
Humidity | High 3/4, Normal 6/1
All 14 days | 9 Yes / 5 No

Play-tennis Gini (3 d.p.): Gini(parent) = ___; Gini reduction for Outlook = ___; for Wind = ___.

(1) parent = ____
(2) Outlook = ____
(3) Wind = ____

Answer: parent = 0.459; Outlook = 0.116; Wind = 0.031

Working:

- Parent $1 - (9/14)^2 - (5/14)^2 = 0.459$.
- Outlook weighted $5/14(0.48) + 0 + 5/14(0.48) = 0.343 \rightarrow 0.116$.
- Wind weighted $8/14(0.375) + 6/14(0.5) = 0.429 \rightarrow 0.031$.

Takeaway: Gini = $1 - \Sigma p^2$.

Source: Book 2 (Exam examples), Q85, p. 17

TREES A37. [BLANKS · Medium · calculation]

8 examples (4+, 4−). Attribute X splits them into a: 3+,1− and b: 1+,3−. IG(X) = ___; Gini reduction = ___ (3 d.p.).

(1) IG = ____
(2) Gini reduction = ____

Answer: IG = 0.189; Gini reduction = 0.125

Working:

- $IG = 1 - [4/8(0.811) + 4/8(0.811)] = 0.189$.
- Gini reduction = $0.5 - 0.375 = 0.125$.

Takeaway: Both children have B(1/4) = 0.811, Gini 0.375.

Source: Book 2 (Exam examples), Q86, p. 17

TREES A38. [TF · Easy]

A branch containing only positive examples has entropy 1.

Answer: False

Why: A pure branch has entropy 0 (certain).

Takeaway: Pure = 0, 50/50 = 1.

Source: Book 2 (Exam examples), Q87, p. 17

TREES A39. [TF · Medium]

Information gain can be negative.

Answer: False

Why: IG is always ≥ 0.

Takeaway: Splitting never increases weighted entropy.

Source: Book 2 (Exam examples), Q88, p. 18

TREES A40. [MATCH · Medium]

Match the tree algorithm to its split criterion.

1. ID3
2. C4.5
3. CART

A. information gain
B. mean absolute error only
C. gain ratio + pruning
D. Gini/MSE with binary splits

Answer: 1 → A, 2 → C, 3 → D

Working:

- ID3 → information gain
- C4.5 → gain ratio + pruning
- CART → Gini/MSE with binary splits

Takeaway: ID3 IG, C4.5 gain ratio, CART Gini.

Source: Book 2 (Exam examples), Q89, p. 18

TREES A41. [BLANK · Easy · calculation]

Entropy of a node with 3 positive and 1 negative example: ___ (3 d.p.).

Answer: 0.811

Takeaway: B(3/4).

Source: Book 2 (Exam examples), Mixed practice paper Q6, p. 54

TREES A42. [BLANK · Medium · calculation]

Parent 5+/5−. A split gives L: 4+/1− and R: 1+/4−. Information gain ___ (3 d.p.).

Answer: 0.278

Working:

- Weighted child entropy 0.722
- IG = 1 − 0.722 = 0.278

Takeaway: Parent H = 1; each child B(0.2) = 0.722.

Source: Book 2 (Exam examples), Mixed practice paper Q7, p. 54

TREES A43. [BLANK · Easy · calculation]

Gini impurity of a node with 2 positive and 6 negative examples: ___.

Answer: 0.375

Takeaway: $1 - (0.25^2 + 0.75^2)$.

Source: Book 2 (Exam examples), Mixed practice paper Q8, p. 54

TREES A44. [BLANKS · Easy · calculation]

Node with 3 yes, 1 no. Entropy and Gini (3 d.p.)?

(1) entropy = ____
(2) Gini = ____

Answer: entropy = 0.811; Gini = 0.375

Takeaway: B(0.25), 1 − 0.5625 − 0.0625.

Source: Book 3 (AI from zero), Practice 12.1, p. 51

TREES A45. [BLANK · Medium · calculation]

Maximum entropy for 4 equally likely classes = ___ bits.

Answer: 2

Takeaway: $\log_2 4$.

Source: Book 3 (AI from zero), Practice 12.4, p. 51

TREES A46. [MCQ · Medium]

Gini impurity is the probability that:

A. two examples have the same label
B. the tree overfits
C. a random example is positive
D. two examples drawn at random (with replacement) have different labels

Answer: D. two examples drawn at random (with replacement) have different labels

Option notes:

- A: That is 1 − Gini.
- B: No.
- C: That is p.

Takeaway: 0 pure, 0.5 worst for two classes.

Source: Book 3 (AI from zero), Chapter 12, p. 48

TREES A47. [MCQ · Hard]

Why does C4.5 use gain ratio instead of plain information gain?

A. IG can be negative
B. gain ratio uses Gini
C. IG is biased towards attributes with many values
D. gain ratio is faster

Answer: C. IG is biased towards attributes with many values

Option notes:

- A: IG ≥ 0.
- B: It uses IG / split info.
- D: Not the reason.

Takeaway: Many-valued attributes look good under IG.

Source: Book 3 (AI from zero), Chapter 12, p. 48

TREES A48. [TF · Medium]

A regression-tree leaf predicts the average of its examples, and splits are chosen to reduce variance/MSE.

Answer: True

Why: CART regression.

Takeaway: Leaves store a constant.

Source: Book 3 (AI from zero), Chapter 12, p. 48

TREES A49. [BLANK · Medium · calculation]

An event with probability 1/8 carries ___ bits of surprise ($\log_2(1/p)$).

Answer: 3

Takeaway: Like 3 coin flips.

Source: Book 3 (AI from zero), Chapter 12, p. 48

TREES A50. [BLANK · Medium · calculation]

Entropy of a coin that comes up heads 99% of the time ≈ ___ bits (2 d.p.).

Answer: 0.08

Takeaway: Almost no uncertainty.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: entropy

TREES A51. [BLANK · Easy · calculation]

Entropy of a fair four-sided die = ___ bits.

Answer: 2

Takeaway: $\log_2 4$.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: entropy

TREES A52. [BLANK · Hard · calculation]

(Extra, not in your slides) How many distinct Boolean functions (decision trees' targets) exist over n = 6 Boolean attributes? 2^(2^6) = 2^64 ≈ ___ (write e.g. 1.8e19).

Answer: 1.8e19
(Also accepted: 1.8 × 10^19, 1.8x10^19)

Takeaway: The hypothesis space is huge, so we need a good bias.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: expressiveness

TREES A53. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

AIMA restaurant data (12 examples, 6+/6−): Gain(Patrons) ≈ ___ bits (3 d.p.) and Gain(Type) = ___.

(1) Patrons = ____
(2) Type = ____

Answer: Patrons = 0.541; Type = 0

Takeaway: Patrons is the root, exactly as in Lecture 4.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: restaurant example

TREES A54. [MCQ · Hard]

(Extra, not in your slides) Decision-tree pruning with a χ^2 test removes a split when:

A. the attribute is numeric
B. the tree is too shallow
C. the split is not statistically significant (it looks like noise)
D. it has the highest gain

Answer: C. the split is not statistically significant (it looks like noise)

Option notes:

- A: Not the criterion.
- B: Pruning makes it shallower.
- D: Keep those.

Takeaway: Avoid fitting noise.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: pruning

TREES A55. [BLANKS · Easy · calculation]

Step 1 of the sheet: parent set of 12 (6 positive: 1, 3, 4, 6, 8, 12; 6 negative). Entropy(Parent) = ___ bit; Gini(Parent) = ___ (4 d.p.).

(1) entropy = ____
(2) Gini = ____

Answer: entropy = 1.0000; Gini = 0.5000

Takeaway: p(pos) = p(neg) = 0.5.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

TREES A56. [BLANKS · Medium · calculation]

Type branches: French 2 (1+,1−), Italian 2 (1+,1−), Thai 4 (2+,2−), Burger 4 (2+,2−). Each branch entropy = ___; weighted child entropy = ___; IG(Type) = ___.

(1) branch H = ____
(2) weighted = ____
(3) IG = ____

Answer: branch H = 1; weighted = 1; IG = 0

Working:

- $H = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$
- $2/12 + 2/12 + 4/12 + 4/12 = 1.0$
- $IG = 1 - 1 = 0$

Takeaway: Every branch is 50/50.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

TREES A57. [BLANKS · Medium · calculation]

Type: each branch Gini = ___; weighted child Gini = ___; Gini reduction = ___ (4 d.p.).

(1) branch Gini = ____
(2) weighted = ____
(3) reduction = ____

Answer: branch Gini = 0.5000; weighted = 0.5000; reduction = 0.0000

Takeaway: No improvement.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

TREES A58. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Patrons branches: None 2 (0+,2−), Some 4 (4+,0−), Full 6 (2+,4−). H(None) = ___, H(Some) = ___, H(Full) = ___ (4 d.p.); weighted child entropy = ___; IG(Patrons) = ___.

(1) H(None) = ____
(2) H(Some) = ____
(3) H(Full) = ____
(4) weighted = ____
(5) IG = ____

Answer: H(None) = 0; H(Some) = 0; H(Full) = 0.9183; weighted = 0.4591; IG = 0.5409

Working:

- $H(\text{Full}) = -(1/3)\log_2(1/3) - (2/3)\log_2(2/3) = 0.9183$
- weighted = 6/12 × 0.9183 = 0.4591
- IG = 1 − 0.4591 = 0.5409

Takeaway: Pure branches have entropy 0.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

TREES A59. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Patrons? · [6+, 6−] · None · [0+, 2−] · Some · [4+, 0−] · Full · [2+, 4−]

```
[Diagram as text] Labels: Type? · [6+, 6−] · French · [1+, 1−] · Italian · [1+, 1−] · Thai · [2+, 2−] · Burger · [2+, 2−]
```

Restaurant data (Lecture 4, 12 examples, WillWait 6 yes / 6 no).

Patrons Gini: Gini(Full) = ___; weighted child Gini = ___; Gini reduction = ___ (4 d.p.).

(1) Gini(Full) = ____
(2) weighted = ____
(3) reduction = ____

Answer: Gini(Full) = 0.4444; weighted = 0.2222; reduction = 0.2778

Takeaway: $1 - (1/3)^2 - (2/3)^2$.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

TREES A60. [MCQ · Easy]

Final step of the sheet: which attribute is picked, and by which criteria?

A. Type, because it has more values
B. neither: the gains tie
C. Patrons, by both information gain (0.5409 > 0) and Gini reduction (0.2778 > 0)
D. Patrons by IG but Type by Gini

Answer: C. Patrons, by both information gain (0.5409 > 0) and Gini reduction (0.2778 > 0)

Option notes:

- A: More values does not mean more gain.
- B: They do not tie.
- D: Both criteria agree.

Takeaway: Pick the larger gain/reduction.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

TREES A61. [TF · Medium]

In the sheet, the weighted child entropy for Type is the plain average (1 + 1 + 1 + 1)/4.

Answer: False

Why: It weights by branch size: 2/12, 2/12, 4/12, 4/12 (here it happens to give the same 1.0).

Takeaway: Weights are |subset|/|parent|.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

TREES A62. [BLANKS · Hard · calculation]

Same method, new attribute. Parent 6+/6−. Attribute Hungry splits into Yes 7 (5+,2−) and No 5 (1+,4−). IG(Hungry) = ___ and Gini reduction = ___ (4 d.p.).

(1) IG = ____
(2) Gini reduction = ____

Answer: IG = 0.1957; Gini reduction = 0.1286

Working:

- H(Yes) = 0.8631, H(No) = 0.7219
- weighted = 7/12 × 0.8631 + 5/12 × 0.7219 = 0.8043 → IG = 0.1957
- Gini(Yes) = 1 − (5/7)² − (2/7)² = 0.4082; Gini(No) = 0.32
- weighted = 7/12 × 0.4082 + 5/12 × 0.32 = 0.3714 → reduction 0.1286

Takeaway: B(5/7) = 0.8631, B(1/5) = 0.7219.

Source: Comparing two attributes to split (Type vs Patrons) step-by-step sheet

Part B: calculation drills (80)

TREES B1. [BLANK · Easy · calculation]

A node has 6 positive and 9 negative examples. What is its entropy H(S) in bits (4 decimal places)?

Answer: 0.9710

Working:

- p(+) = 6/15, p(−) = 9/15.

- H(S [6+, 9−]) = −(6/15)·$\log_2$(6/15) = 0.5288 + −(9/15)·$\log_2$(9/15) = 0.4422 = 0.9710

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B2. [MCQ · Easy · calculation]

A node has 8 positive and 3 negative examples. What is its entropy H(S) in bits?

A. 0.545
B. 0.845
C. 0.397
D. 0.586

Answer: B. 0.845

Working:

- p(+) = 8/11, p(−) = 3/11.
- H(S [8+, 3−]) = −(8/11)·$\log_2$(8/11) = 0.3341 + −(3/11)·$\log_2$(3/11) = 0.5112 = 0.8454

Option notes:

- A: Not a valid shortcut; entropy uses −Σ p $\log_2$ p.
- C: That is the Gini index, not entropy.
- D: Used natural log (ln). Entropy here is in bits: use $\log_2$.

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B3. [BLANK · Easy · calculation]

A node has 5 positive and 6 negative examples. What is its entropy H(S) in bits (4 decimal places)?

Answer: 0.9940

Working:

- p(+) = 5/11, p(−) = 6/11.
- H(S [5+, 6−]) = −(5/11)·$\log_2$(5/11) = 0.5170 + −(6/11)·$\log_2$(6/11) = 0.4770 = 0.9940

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B4. [MCQ · Easy · calculation]

A node has 6 positive and 5 negative examples. What is its entropy H(S) in bits?

A. 0.6890
B. 0.4959
C. 0.9091
D. 0.9940

Answer: D. 0.9940

Working:

- p(+) = 6/11, p(−) = 5/11.
- H(S [6+, 5−]) = −(6/11)·$\log_2$(6/11) = 0.4770 + −(5/11)·$\log_2$(5/11) = 0.5170 = 0.9940

Option notes:

- A: Used natural log (ln). Entropy here is in bits: use $\log_2$.
- B: That is the Gini index, not entropy.

- C: Not a valid shortcut; entropy uses $-\Sigma\ p \log_2 p$.

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B5. [MCQ · Easy · calculation]

A node has 8 positive and 7 negative examples. What is its entropy H(S) in bits?

A. 0.997
B. 0.933
C. 0.691
D. 0.498

Answer: A. 0.997

Working:

- p(+) = 8/15, p(−) = 7/15.
- H(S [8+, 7−]) = $-(8/15)\cdot\log_2(8/15)$ = 0.4837 + $-(7/15)\cdot\log_2(7/15)$ = 0.5131 = 0.9968

Option notes:

- B: Not a valid shortcut; entropy uses $-\Sigma\ p \log_2 p$.
- C: Used natural log (ln). Entropy here is in bits: use $\log_2$.
- D: That is the Gini index, not entropy.

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B6. [BLANK · Easy · calculation]

A node has 3 positive and 7 negative examples. What is its entropy H(S) in bits (4 decimal places)?

Answer: 0.8813

Working:

- p(+) = 3/10, p(−) = 7/10.
- H(S [3+, 7−]) = $-(3/10)\cdot\log_2(3/10)$ = 0.5211 + $-(7/10)\cdot\log_2(7/10)$ = 0.3602 = 0.8813

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B7. [MCQ · Easy · calculation]

A node has 8 positive and 6 negative examples. What is its entropy H(S) in bits?

A. 0.985
B. 0.857
C. 0.683
D. 0.490

Answer: A. 0.985

Working:

- p(+) = 8/14, p(−) = 6/14.
- H(S [8+, 6−]) = $-(8/14)\cdot\log_2(8/14)$ = 0.4613 + $-(6/14)\cdot\log_2(6/14)$ = 0.5239 = 0.9852

Option notes:

- B: Not a valid shortcut; entropy uses $-\Sigma\ p \log_2 p$.
- C: Used natural log (ln). Entropy here is in bits: use $\log_2$.

- D: That is the Gini index, not entropy.

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B8. [MCQ · Easy · calculation]

A node has 6 positive and 3 negative examples. What is its entropy H(S) in bits?

A. 0.637
B. 0.667
C. 0.918
D. 0.444

Answer: C. 0.918

Working:

- p(+) = 6/9, p(−) = 3/9.
- H(S [6+, 3−]) = $-(6/9)\cdot\log_2(6/9) = 0.3900 + -(3/9)\cdot\log_2(3/9) = 0.5283 = 0.9183$

Option notes:

- A: Used natural log (ln). Entropy here is in bits: use $\log_2$.
- B: Not a valid shortcut; entropy uses $-\Sigma\ p\ \log_2\ p$.
- D: That is the Gini index, not entropy.

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B9. [BLANK · Easy · calculation]

A node has 3 positive and 7 negative examples. What is its entropy H(S) in bits (3 decimal places)?

Answer: 0.881

Working:

- p(+) = 3/10, p(−) = 7/10.
- H(S [3+, 7−]) = $-(3/10)\cdot\log_2(3/10) = 0.5211 + -(7/10)\cdot\log_2(7/10) = 0.3602 = 0.8813$

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B10. [BLANK · Easy · calculation]

A node has 6 positive and 8 negative examples. What is its entropy H(S) in bits (3 decimal places)?

Answer: 0.985

Working:

- p(+) = 6/14, p(−) = 8/14.
- H(S [6+, 8−]) = $-(6/14)\cdot\log_2(6/14) = 0.5239 + -(8/14)\cdot\log_2(8/14) = 0.4613 = 0.9852$

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B11. [BLANK · Easy · calculation]

A node has 6 positive and 5 negative examples. What is its entropy H(S) in bits (3 decimal places)?

Answer: 0.994

Working:

- p(+) = 6/11, p(−) = 5/11.
- H(S [6+, 5−]) = −(6/11)·$\log_2$(6/11) = 0.4770 + −(5/11)·$\log_2$(5/11) = 0.5170 = 0.9940

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B12. [MCQ · Easy · calculation]

A node has 5 positive and 8 negative examples. What is its entropy H(S) in bits?

A. 0.4734
B. 0.6663
C. 0.7692
D. 0.9612

Answer: D. 0.9612

Working:

- p(+) = 5/13, p(−) = 8/13.
- H(S [5+, 8−]) = −(5/13)·$\log_2$(5/13) = 0.5302 + −(8/13)·$\log_2$(8/13) = 0.4310 = 0.9612

Option notes:

- A: That is the Gini index, not entropy.
- B: Used natural log (ln). Entropy here is in bits: use $\log_2$.
- C: Not a valid shortcut; entropy uses −Σ p $\log_2$ p.

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B13. [BLANK · Easy · calculation]

A node has 8 positive and 7 negative examples. What is its entropy H(S) in bits (3 decimal places)?

Answer: 0.997

Working:

- p(+) = 8/15, p(−) = 7/15.
- H(S [8+, 7−]) = −(8/15)·$\log_2$(8/15) = 0.4837 + −(7/15)·$\log_2$(7/15) = 0.5131 = 0.9968

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B14. [BLANK · Easy · calculation]

A node has 4 positive and 6 negative examples. What is its entropy H(S) in bits (3 decimal places)?

Answer: 0.971

Working:

- p(+) = 4/10, p(−) = 6/10.
- H(S [4+, 6−]) = −(4/10)·$\log_2$(4/10) = 0.5288 + −(6/10)·$\log_2$(6/10) = 0.4422 = 0.9710

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B15. [BLANK · Easy · calculation]

A node has 8 positive and 6 negative examples. What is its entropy H(S) in bits (4 decimal places)?

Answer: 0.9852

Working:

- p(+) = 8/14, p(−) = 6/14.
- H(S [8+, 6−]) = $-(8/14)\cdot\log_2(8/14) = 0.4613 + -(6/14)\cdot\log_2(6/14) = 0.5239 = 0.9852$

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B16. [MCQ · Easy · calculation]

A node has 3 positive and 4 negative examples. What is its entropy H(S) in bits?

A. 0.8571
B. 0.6829
C. 0.9852
D. 0.4898

Answer: C. 0.9852

Working:

- p(+) = 3/7, p(−) = 4/7.
- H(S [3+, 4−]) = $-(3/7)\cdot\log_2(3/7) = 0.5239 + -(4/7)\cdot\log_2(4/7) = 0.4613 = 0.9852$

Option notes:

- A: Not a valid shortcut; entropy uses $-\Sigma\ p\ \log_2\ p$.
- B: Used natural log (ln). Entropy here is in bits: use $\log_2$.
- D: That is the Gini index, not entropy.

Takeaway: Entropy: 0 for a pure node, 1 bit at 50/50 (binary).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B17. [MCQ · Easy · calculation]

A node contains class counts 6, 6. What is its Gini index?

A. 0.5000
B. 0.5750
C. 1.0000
D. 0.0000

Answer: A. 0.5000

Working:

- Gini(S) = $1 - (6/12)^2 - (6/12)^2 = 0.5000$

Option notes:

- B: Arithmetic slip: recompute each step carefully.
- C: That is entropy, not Gini.
- D: You squared the sum instead of summing the squares.

Takeaway: Gini = $1 - \Sigma\ p_i^2$. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B18. [MCQ · Easy · calculation]

A node contains class counts 3, 6. What is its Gini index?

A. 0.5556
B. 0.9183
C. 0.0000
D. 0.4444

Answer: D. 0.4444

Working:

- Gini(S) = 1 − (3/9)² − (6/9)² = 0.4444

Option notes:

- A: Forgot the '1 −'.
- B: That is entropy, not Gini.
- C: You squared the sum instead of summing the squares.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B19. [MCQ · Easy · calculation]

A node contains class counts 7, 6, 5. What is its Gini index?

A. 0.6605
B. 0.0000
C. 0.3395
D. 1.5715

Answer: A. 0.6605

Working:

- Gini(S) = 1 − (7/18)² − (6/18)² − (5/18)² = 0.6605

Option notes:

- B: You squared the sum instead of summing the squares.
- C: Forgot the '1 −'.
- D: That is entropy, not Gini.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B20. [MCQ · Easy · calculation]

A node contains class counts 3, 6. What is its Gini index?

A. 0.0000
B. 0.5556
C. 0.9183
D. 0.4444

Answer: D. 0.4444

Working:

- Gini(S) = 1 − (3/9)² − (6/9)² = 0.4444

Option notes:

- A: You squared the sum instead of summing the squares.
- B: Forgot the '1 −'.
- C: That is entropy, not Gini.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B21. [MCQ · Easy · calculation]

A node contains class counts 5, 5. What is its Gini index?

A. 0.5750
B. 0.0000
C. 1.0000
D. 0.5000

Answer: D. 0.5000

Working:

- Gini(S) = 1 − $(5/10)^2$ − $(5/10)^2$ = 0.5000

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: You squared the sum instead of summing the squares.
- C: That is entropy, not Gini.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B22. [MCQ · Easy · calculation]

A node contains class counts 9, 4. What is its Gini index?

A. 0.5740
B. 0.4260
C. 0.8905
D. 0.0000

Answer: B. 0.4260

Working:

- Gini(S) = 1 − $(9/13)^2$ − $(4/13)^2$ = 0.4260

Option notes:

- A: Forgot the '1 −'.
- C: That is entropy, not Gini.
- D: You squared the sum instead of summing the squares.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B23. [MCQ · Easy · calculation]

A node contains class counts 4, 6. What is its Gini index?

A. 0.5200
B. 0.0000

C. 0.4800
D. 0.9710

Answer: C. 0.4800

Working:

- Gini(S) = 1 − (4/10)² − (6/10)² = 0.4800

Option notes:

- A: Forgot the '1 −'.
- B: You squared the sum instead of summing the squares.
- D: That is entropy, not Gini.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B24. [MCQ · Easy · calculation]

A node contains class counts 4, 7, 5. What is its Gini index?

A. 0.0000
B. 0.6484
C. 1.5462
D. 0.3516

Answer: B. 0.6484

Working:

- Gini(S) = 1 − (4/16)² − (7/16)² − (5/16)² = 0.6484

Option notes:

- A: You squared the sum instead of summing the squares.
- C: That is entropy, not Gini.
- D: Forgot the '1 −'.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B25. [MCQ · Easy · calculation]

A node contains class counts 4, 4. What is its Gini index?

A. 0.0000
B. 1.0000
C. 0.5750
D. 0.5000

Answer: D. 0.5000

Working:

- Gini(S) = 1 − (4/8)² − (4/8)² = 0.5000

Option notes:

- A: You squared the sum instead of summing the squares.
- B: That is entropy, not Gini.
- C: Arithmetic slip: recompute each step carefully.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B26. [MCQ · Easy · calculation]

A node contains class counts 6, 8. What is its Gini index?

A. 0.4898
B. 0.0000
C. 0.5102
D. 0.9852

Answer: A. 0.4898

Working:

- Gini(S) = 1 − (6/14)² − (8/14)² = 0.4898

Option notes:

- B: You squared the sum instead of summing the squares.
- C: Forgot the '1 −'.
- D: That is entropy, not Gini.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B27. [MCQ · Easy · calculation]

A node contains class counts 4, 3. What is its Gini index?

A. 0.9852
B. 0.4898
C. 0.5102
D. 0.0000

Answer: B. 0.4898

Working:

- Gini(S) = 1 − (4/7)² − (3/7)² = 0.4898

Option notes:

- A: That is entropy, not Gini.
- C: Forgot the '1 −'.
- D: You squared the sum instead of summing the squares.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B28. [MCQ · Easy · calculation]

A node contains class counts 8, 4. What is its Gini index?

A. 0.5556
B. 0.0000
C. 0.9183
D. 0.4444

Answer: D. 0.4444

Working:

- Gini(S) = 1 − (8/12)² − (4/12)² = 0.4444

Option notes:

- A: Forgot the '1 −'.
- B: You squared the sum instead of summing the squares.
- C: That is entropy, not Gini.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B29. [MCQ · Easy · calculation]

A node contains class counts 5, 8. What is its Gini index?

A. 0.5266
B. 0.0000
C. 0.9612
D. 0.4734

Answer: D. 0.4734

Working:

- Gini(S) = 1 − $(5/13)^2$ − $(8/13)^2$ = 0.4734

Option notes:

- A: Forgot the '1 −'.
- B: You squared the sum instead of summing the squares.
- C: That is entropy, not Gini.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B30. [MCQ · Easy · calculation]

A node contains class counts 3, 5, 3. What is its Gini index?

A. 1.5395
B. 0.3554
C. 0.0000
D. 0.6446

Answer: D. 0.6446

Working:

- Gini(S) = 1 − $(3/11)^2$ − $(5/11)^2$ − $(3/11)^2$ = 0.6446

Option notes:

- A: That is entropy, not Gini.
- B: Forgot the '1 −'.
- C: You squared the sum instead of summing the squares.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B31. [MCQ · Easy · calculation]

A node contains class counts 8, 4, 4. What is its Gini index?

A. 0.0000
B. 1.5000

C. 0.6250
D. 0.3750

Answer: C. 0.6250

Working:

- Gini(S) = 1 − (8/16)² − (4/16)² − (4/16)² = 0.6250

Option notes:

- A: You squared the sum instead of summing the squares.
- B: That is entropy, not Gini.
- D: Forgot the '1 −'.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B32. [MCQ · Easy · calculation]

A node contains class counts 9, 8, 4. What is its Gini index?

A. 0.0000
B. 0.6349
C. 1.5100
D. 0.3651

Answer: B. 0.6349

Working:

- Gini(S) = 1 − (9/21)² − (8/21)² − (4/21)² = 0.6349

Option notes:

- A: You squared the sum instead of summing the squares.
- C: That is entropy, not Gini.
- D: Forgot the '1 −'.

Takeaway: Gini = 1 − Σ p_i^2. Pure node = 0; binary maximum = 0.5.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B33. [BLANK · Medium · calculation]

[Diagram as text] Labels: Price? · [7+, 6−] · Yes · [4+, 4−] · No · [3+, 2−]

Splitting a node [7+, 6−] on Price gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.007

Working:

- H(parent [7+, 6−]) = $-(7/13)\cdot\log_2(7/13) = 0.4809 + -(6/13)\cdot\log_2(6/13) = 0.5148 = 0.9957$
- H(Yes [4+, 4−]) = $-(4/8)\cdot\log_2(4/8) = 0.5000 + -(4/8)\cdot\log_2(4/8) = 0.5000 = 1.0000$
- H(No [3+, 2−]) = $-(3/5)\cdot\log_2(3/5) = 0.4422 + -(2/5)\cdot\log_2(2/5) = 0.5288 = 0.9710$
- Weighted child entropy = (8/13)·1.0000 + (5/13)·0.9710 = 0.9888
- IG(Price) = 0.9957 − 0.9888 = 0.0069

Takeaway: IG = H(parent) − Σ ($|S_v|/|S|$)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B34. [BLANK · Medium · calculation]

[Diagram as text] Labels: Hungry? · [5+, 4−] · Yes · [3+, 1−] · No · [2+, 3−]

Splitting a node [5+, 4−] on Hungry gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.091

Working:

- H(parent [5+, 4−]) = −(5/9)·$\log_2$(5/9) = 0.4711 + −(4/9)·$\log_2$(4/9) = 0.5200 = 0.9911
- H(Yes [3+, 1−]) = −(3/4)·$\log_2$(3/4) = 0.3113 + −(1/4)·$\log_2$(1/4) = 0.5000 = 0.8113
- H(No [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- Weighted child entropy = (4/9)·0.8113 + (5/9)·0.9710 = 0.9000
- IG(Hungry) = 0.9911 − 0.9000 = 0.0911

Takeaway: IG = H(parent) − Σ (|S_v|/|S|)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B35. [BLANK · Medium · calculation]

[Diagram as text] Labels: Patrons? · [9+, 5−] · v1 · [2+, 0−] · v2 · [4+, 2−] · v3 · [3+, 3−]

Splitting a node [9+, 5−] on Patrons gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.118

Working:

- H(parent [9+, 5−]) = −(9/14)·$\log_2$(9/14) = 0.4098 + −(5/14)·$\log_2$(5/14) = 0.5305 = 0.9403
- H(v1 [2+, 0−]) = −(2/2)·$\log_2$(2/2) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(v2 [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- H(v3 [3+, 3−]) = −(3/6)·$\log_2$(3/6) = 0.5000 + −(3/6)·$\log_2$(3/6) = 0.5000 = 1.0000
- Weighted child entropy = (2/14)·0.0000 + (6/14)·0.9183 + (6/14)·1.0000 = 0.8221
- IG(Patrons) = 0.9403 − 0.8221 = 0.1182

Takeaway: IG = H(parent) − Σ (|S_v|/|S|)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B36. [MCQ · Medium · calculation]

[Diagram as text] Labels: Price? · [5+, 5−] · Red · [3+, 1−] · Green · [1+, 3−] · Blue · [1+, 1−]

Splitting a node [5+, 5−] on Price gives the branches shown. What is the information gain?

A. 0.849
B. 0.126
C. 0.100
D. 0.151

Answer: D. 0.151

Working:

- H(parent [5+, 5−]) = −(5/10)·$\log_2$(5/10) = 0.5000 + −(5/10)·$\log_2$(5/10) = 0.5000 = 1.0000
- H(Red [3+, 1−]) = −(3/4)·$\log_2$(3/4) = 0.3113 + −(1/4)·$\log_2$(1/4) = 0.5000 = 0.8113
- H(Green [1+, 3−]) = −(1/4)·$\log_2$(1/4) = 0.5000 + −(3/4)·$\log_2$(3/4) = 0.3113 = 0.8113
- H(Blue [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- Weighted child entropy = (4/10)·0.8113 + (4/10)·0.8113 + (2/10)·1.0000 = 0.8490

- IG(Price) = 1.0000 − 0.8490 = 0.1510

Option notes:

- A: That is the weighted child entropy; IG = H(parent) − weighted child entropy.
- B: Took a plain average of child entropies; you must weight by $|S_v|/|S|$.
- C: That is the Gini reduction, not information gain.

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B37. [MCQ · Medium · calculation]

[Diagram as text] Labels: Price? · [3+, 6−] · Red · [3+, 2−] · Green · [0+, 2−] · Blue · [0+, 2−]

Splitting a node [3+, 6−] on Price gives the branches shown. What is the information gain?

A. 0.539
B. 0.178
C. 0.379
D. 0.595

Answer: C. 0.379

Working:

- H(parent [3+, 6−]) = $-(3/9)\cdot\log_2(3/9)$ = 0.5283 + $-(6/9)\cdot\log_2(6/9)$ = 0.3900 = 0.9183
- H(Red [3+, 2−]) = $-(3/5)\cdot\log_2(3/5)$ = 0.4422 + $-(2/5)\cdot\log_2(2/5)$ = 0.5288 = 0.9710
- H(Green [0+, 2−]) = 0 (pure part: $0\cdot\log_2 0 = 0$) + $-(2/2)\cdot\log_2(2/2)$ = 0.0000 = 0.0000
- H(Blue [0+, 2−]) = 0 (pure part: $0\cdot\log_2 0 = 0$) + $-(2/2)\cdot\log_2(2/2)$ = 0.0000 = 0.0000
- Weighted child entropy = $(5/9)\cdot 0.9710 + (2/9)\cdot 0.0000 + (2/9)\cdot 0.0000$ = 0.5394
- IG(Price) = 0.9183 − 0.5394 = 0.3789

Option notes:

- A: That is the weighted child entropy; IG = H(parent) − weighted child entropy.
- B: That is the Gini reduction, not information gain.
- D: Took a plain average of child entropies; you must weight by $|S_v|/|S|$.

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B38. [MCQ · Medium · calculation]

[Diagram as text] Labels: Patrons? · [7+, 5−] · Low · [1+, 2−] · Medium · [4+, 1−] · High · [2+, 2−]

Splitting a node [7+, 5−] on Patrons gives the branches shown. What is the information gain?

A. 0.075
B. 0.116
C. 0.100
D. 0.864

Answer: B. 0.116

Working:

- H(parent [7+, 5−]) = $-(7/12)\cdot\log_2(7/12)$ = 0.4536 + $-(5/12)\cdot\log_2(5/12)$ = 0.5263 = 0.9799
- H(Low [1+, 2−]) = $-(1/3)\cdot\log_2(1/3)$ = 0.5283 + $-(2/3)\cdot\log_2(2/3)$ = 0.3900 = 0.9183
- H(Medium [4+, 1−]) = $-(4/5)\cdot\log_2(4/5)$ = 0.2575 + $-(1/5)\cdot\log_2(1/5)$ = 0.4644 = 0.7219
- H(High [2+, 2−]) = $-(2/4)\cdot\log_2(2/4)$ = 0.5000 + $-(2/4)\cdot\log_2(2/4)$ = 0.5000 = 1.0000
- Weighted child entropy = $(3/12)\cdot 0.9183 + (5/12)\cdot 0.7219 + (4/12)\cdot 1.0000$ = 0.8637

- IG(Patrons) = 0.9799 − 0.8637 = 0.1162

Option notes:

- A: That is the Gini reduction, not information gain.
- C: Took a plain average of child entropies; you must weight by $|S_v|/|S|$.
- D: That is the weighted child entropy; IG = H(parent) − weighted child entropy.

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B39. [MCQ · Medium · calculation]

[Diagram as text] Labels: Size? · [6+, 6−] · v1 · [2+, 1−] · v2 · [0+, 3−] · v3 · [4+, 2−]

Splitting a node [6+, 6−] on Size gives the branches shown. What is the information gain?

A. 0.689
B. 0.388
C. 0.311
D. 0.167

Answer: C. 0.311

Working:

- H(parent [6+, 6−]) = $-(6/12)\cdot\log_2(6/12) = 0.5000 + -(6/12)\cdot\log_2(6/12) = 0.5000 = 1.0000$
- H(v1 [2+, 1−]) = $-(2/3)\cdot\log_2(2/3) = 0.3900 + -(1/3)\cdot\log_2(1/3) = 0.5283 = 0.9183$
- H(v2 [0+, 3−]) = 0 (pure part: $0\cdot\log_2 0 = 0$) + $-(3/3)\cdot\log_2(3/3) = 0.0000 = 0.0000$
- H(v3 [4+, 2−]) = $-(4/6)\cdot\log_2(4/6) = 0.3900 + -(2/6)\cdot\log_2(2/6) = 0.5283 = 0.9183$
- Weighted child entropy = (3/12)·0.9183 + (3/12)·0.0000 + (6/12)·0.9183 = 0.6887
- IG(Size) = 1.0000 − 0.6887 = 0.3113

Option notes:

- A: That is the weighted child entropy; IG = H(parent) − weighted child entropy.
- B: Took a plain average of child entropies; you must weight by $|S_v|/|S|$.
- D: That is the Gini reduction, not information gain.

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B40. [BLANK · Medium · calculation]

[Diagram as text] Labels: Patrons? · [9+, 3−] · Red · [4+, 2−] · Green · [5+, 1−]

Splitting a node [9+, 3−] on Patrons gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.027

Working:

- H(parent [9+, 3−]) = $-(9/12)\cdot\log_2(9/12) = 0.3113 + -(3/12)\cdot\log_2(3/12) = 0.5000 = 0.8113$
- H(Red [4+, 2−]) = $-(4/6)\cdot\log_2(4/6) = 0.3900 + -(2/6)\cdot\log_2(2/6) = 0.5283 = 0.9183$
- H(Green [5+, 1−]) = $-(5/6)\cdot\log_2(5/6) = 0.2192 + -(1/6)\cdot\log_2(1/6) = 0.4308 = 0.6500$
- Weighted child entropy = (6/12)·0.9183 + (6/12)·0.6500 = 0.7842
- IG(Patrons) = 0.8113 − 0.7842 = 0.0271

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B41. [MCQ · Medium · calculation]

[Diagram as text] Labels: Outlook? · [5+, 3−] · v1 · [1+, 1−] · v2 · [2+, 0−] · v3 · [2+, 2−]

Splitting a node [5+, 3−] on Outlook gives the branches shown. What is the information gain?

A. 0.750
B. 0.204
C. 0.288
D. 0.094

Answer: B. 0.204

Working:

- H(parent [5+, 3−]) = −(5/8)·$\log_2$(5/8) = 0.4238 + −(3/8)·$\log_2$(3/8) = 0.5306 = 0.9544
- H(v1 [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- H(v2 [2+, 0−]) = −(2/2)·$\log_2$(2/2) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(v3 [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- Weighted child entropy = (2/8)·1.0000 + (2/8)·0.0000 + (4/8)·1.0000 = 0.7500
- IG(Outlook) = 0.9544 − 0.7500 = 0.2044

Option notes:

- A: That is the weighted child entropy; IG = H(parent) − weighted child entropy.
- C: Took a plain average of child entropies; you must weight by $|S_v|/|S|$.
- D: That is the Gini reduction, not information gain.

Takeaway: IG = H(parent) − Σ ($|S_v|/|S|$)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B42. [BLANK · Medium · calculation]

[Diagram as text] Labels: Patrons? · [9+, 3−] · Some · [6+, 2−] · Full · [3+, 1−]

Splitting a node [9+, 3−] on Patrons gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.000

Working:

- H(parent [9+, 3−]) = −(9/12)·$\log_2$(9/12) = 0.3113 + −(3/12)·$\log_2$(3/12) = 0.5000 = 0.8113
- H(Some [6+, 2−]) = −(6/8)·$\log_2$(6/8) = 0.3113 + −(2/8)·$\log_2$(2/8) = 0.5000 = 0.8113
- H(Full [3+, 1−]) = −(3/4)·$\log_2$(3/4) = 0.3113 + −(1/4)·$\log_2$(1/4) = 0.5000 = 0.8113
- Weighted child entropy = (8/12)·0.8113 + (4/12)·0.8113 = 0.8113
- IG(Patrons) = 0.8113 − 0.8113 = 0.0000

Takeaway: IG = H(parent) − Σ ($|S_v|/|S|$)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B43. [BLANK · Medium · calculation]

[Diagram as text] Labels: Patrons? · [4+, 8−] · Low · [2+, 4−] · Medium · [2+, 4−]

Splitting a node [4+, 8−] on Patrons gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.000

Working:

- H(parent [4+, 8−]) = −(4/12)·$\log_2$(4/12) = 0.5283 + −(8/12)·$\log_2$(8/12) = 0.3900 = 0.9183

- H(Low [2+, 4−]) = −(2/6)·$\log_2$(2/6) = 0.5283 + −(4/6)·$\log_2$(4/6) = 0.3900 = 0.9183
- H(Medium [2+, 4−]) = −(2/6)·$\log_2$(2/6) = 0.5283 + −(4/6)·$\log_2$(4/6) = 0.3900 = 0.9183
- Weighted child entropy = (6/12)·0.9183 + (6/12)·0.9183 = 0.9183
- IG(Patrons) = 0.9183 − 0.9183 = 0.0000

Takeaway: IG = H(parent) − Σ (|S_v|/|S|)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B44. [MCQ · Medium · calculation]

[Diagram as text] Labels: Outlook? · [8+, 6−] · Some · [3+, 3−] · Full · [5+, 3−]

Splitting a node [8+, 6−] on Outlook gives the branches shown. What is the information gain?

A. 0.974
B. 0.018
C. 0.011
D. 0.008

Answer: C. 0.011

Working:

- H(parent [8+, 6−]) = −(8/14)·$\log_2$(8/14) = 0.4613 + −(6/14)·$\log_2$(6/14) = 0.5239 = 0.9852
- H(Some [3+, 3−]) = −(3/6)·$\log_2$(3/6) = 0.5000 + −(3/6)·$\log_2$(3/6) = 0.5000 = 1.0000
- H(Full [5+, 3−]) = −(5/8)·$\log_2$(5/8) = 0.4238 + −(3/8)·$\log_2$(3/8) = 0.5306 = 0.9544
- Weighted child entropy = (6/14)·1.0000 + (8/14)·0.9544 = 0.9740
- IG(Outlook) = 0.9852 − 0.9740 = 0.0113

Option notes:

- A: That is the weighted child entropy; IG = H(parent) − weighted child entropy.
- B: Arithmetic slip: recompute each step carefully.
- D: Took a plain average of child entropies; you must weight by |S_v|/|S|.

Takeaway: IG = H(parent) − Σ (|S_v|/|S|)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B45. [MCQ · Medium · calculation]

[Diagram as text] Labels: Price? · [6+, 5−] · Yes · [2+, 3−] · No · [4+, 2−]

Splitting a node [6+, 5−] on Price gives the branches shown. What is the information gain?

A. 0.052
B. 0.035
C. 0.049
D. 0.942

Answer: A. 0.052

Working:

- H(parent [6+, 5−]) = −(6/11)·$\log_2$(6/11) = 0.4770 + −(5/11)·$\log_2$(5/11) = 0.5170 = 0.9940
- H(Yes [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- H(No [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- Weighted child entropy = (5/11)·0.9710 + (6/11)·0.9183 = 0.9422
- IG(Price) = 0.9940 − 0.9422 = 0.0518

Option notes:

- B: That is the Gini reduction, not information gain.

- C: Took a plain average of child entropies; you must weight by $|S_v|/|S|$.
- D: That is the weighted child entropy; IG = H(parent) − weighted child entropy.

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B46. [BLANK · Medium · calculation]

[Diagram as text] Labels: Student? · [4+, 8−] · Some · [2+, 3−] · Full · [1+, 3−] · None · [1+, 2−]

Splitting a node [4+, 8−] on Student gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.014

Working:

- H(parent [4+, 8−]) = $-(4/12)\cdot\log_2(4/12) = 0.5283$ + $-(8/12)\cdot\log_2(8/12) = 0.3900 = 0.9183$
- H(Some [2+, 3−]) = $-(2/5)\cdot\log_2(2/5) = 0.5288$ + $-(3/5)\cdot\log_2(3/5) = 0.4422 = 0.9710$
- H(Full [1+, 3−]) = $-(1/4)\cdot\log_2(1/4) = 0.5000$ + $-(3/4)\cdot\log_2(3/4) = 0.3113 = 0.8113$
- H(None [1+, 2−]) = $-(1/3)\cdot\log_2(1/3) = 0.5283$ + $-(2/3)\cdot\log_2(2/3) = 0.3900 = 0.9183$
- Weighted child entropy = $(5/12)\cdot0.9710 + (4/12)\cdot0.8113 + (3/12)\cdot0.9183 = 0.9046$
- IG(Student) = 0.9183 − 0.9046 = 0.0137

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B47. [BLANK · Medium · calculation]

[Diagram as text] Labels: Student? · [9+, 5−] · Sunny · [6+, 0−] · Overcast · [3+, 5−]

Splitting a node [9+, 5−] on Student gives the branches shown. What is the information gain (3 decimal places)?

Answer: 0.395

Working:

- H(parent [9+, 5−]) = $-(9/14)\cdot\log_2(9/14) = 0.4098$ + $-(5/14)\cdot\log_2(5/14) = 0.5305 = 0.9403$
- H(Sunny [6+, 0−]) = $-(6/6)\cdot\log_2(6/6) = 0.0000$ + 0 (pure part: $0\cdot\log_2 0 = 0$) = 0.0000
- H(Overcast [3+, 5−]) = $-(3/8)\cdot\log_2(3/8) = 0.5306$ + $-(5/8)\cdot\log_2(5/8) = 0.4238 = 0.9544$
- Weighted child entropy = $(6/14)\cdot0.0000 + (8/14)\cdot0.9544 = 0.5454$
- IG(Student) = 0.9403 − 0.5454 = 0.3949

Takeaway: IG = H(parent) − Σ $(|S_v|/|S|)\cdot H(S_v)$. Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B48. [MCQ · Medium · calculation]

[Diagram as text] Labels: Colour? · [5+, 4−] · Small · [2+, 1−] · Large · [3+, 3−]

Splitting a node [5+, 4−] on Colour gives the branches shown. What is the information gain?

A. 0.032
B. 0.973
C. 0.012
D. 0.018

Answer: D. 0.018

Working:

- H(parent [5+, 4−]) = −(5/9)·$\log_2$(5/9) = 0.4711 + −(4/9)·$\log_2$(4/9) = 0.5200 = 0.9911
- H(Small [2+, 1−]) = −(2/3)·$\log_2$(2/3) = 0.3900 + −(1/3)·$\log_2$(1/3) = 0.5283 = 0.9183
- H(Large [3+, 3−]) = −(3/6)·$\log_2$(3/6) = 0.5000 + −(3/6)·$\log_2$(3/6) = 0.5000 = 1.0000
- Weighted child entropy = (3/9)·0.9183 + (6/9)·1.0000 = 0.9728
- IG(Colour) = 0.9911 − 0.9728 = 0.0183

Option notes:

- A: Took a plain average of child entropies; you must weight by $|S_v|/|S|$.
- B: That is the weighted child entropy; IG = H(parent) − weighted child entropy.
- C: That is the Gini reduction, not information gain.

Takeaway: IG = H(parent) − Σ ($|S_v|/|S|$)·H(S_v). Always weight by branch size.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B49. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Colour? · [4+, 5−] · Red · [2+, 1−] · Green · [1+, 1−] · Blue · [1+, 3−]

For the split of [4+, 5−] on Colour shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4938; Weighted child Gini = 0.4259; Gini reduction = 0.0679

Working:

- Gini(parent) = 1 − (4/9)² − (5/9)² = 0.4938
- Gini(Red) = 1 − (2/3)² − (1/3)² = 0.4444
- Gini(Green) = 1 − (1/2)² − (1/2)² = 0.5000
- Gini(Blue) = 1 − (1/4)² − (3/4)² = 0.3750
- Weighted child Gini = (3/9)·0.4444 + (2/9)·0.5000 + (4/9)·0.3750 = 0.4259
- Gini reduction = 0.4938 − 0.4259 = 0.0679

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B50. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Colour? · [3+, 5−] · Low · [2+, 0−] · Medium · [1+, 2−] · High · [0+, 3−]

For the split of [3+, 5−] on Colour shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4688; Weighted child Gini = 0.1667; Gini reduction = 0.3021

Working:

- Gini(parent) = 1 − (3/8)² − (5/8)² = 0.4688
- Gini(Low) = 1 − (2/2)² − (0/2)² = 0.0000
- Gini(Medium) = 1 − (1/3)² − (2/3)² = 0.4444
- Gini(High) = 1 − (0/3)² − (3/3)² = 0.0000
- Weighted child Gini = (2/8)·0.0000 + (3/8)·0.4444 + (3/8)·0.0000 = 0.1667

- Gini reduction = 0.4688 − 0.1667 = 0.3021

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B51. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Colour? · [7+, 3−] · Red · [4+, 2−] · Green · [3+, 1−]

For the split of [7+, 3−] on Colour shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4200; Weighted child Gini = 0.4167; Gini reduction = 0.0033

Working:

- Gini(parent) = 1 − (7/10)² − (3/10)² = 0.4200
- Gini(Red) = 1 − (4/6)² − (2/6)² = 0.4444
- Gini(Green) = 1 − (3/4)² − (1/4)² = 0.3750
- Weighted child Gini = (6/10)·0.4444 + (4/10)·0.3750 = 0.4167
- Gini reduction = 0.4200 − 0.4167 = 0.0033

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B52. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Colour? · [8+, 3−] · v1 · [3+, 0−] · v2 · [2+, 3−] · v3 · [3+, 0−]

For the split of [8+, 3−] on Colour shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.3967; Weighted child Gini = 0.2182; Gini reduction = 0.1785

Working:

- Gini(parent) = 1 − (8/11)² − (3/11)² = 0.3967
- Gini(v1) = 1 − (3/3)² − (0/3)² = 0.0000
- Gini(v2) = 1 − (2/5)² − (3/5)² = 0.4800
- Gini(v3) = 1 − (3/3)² − (0/3)² = 0.0000
- Weighted child Gini = (3/11)·0.0000 + (5/11)·0.4800 + (3/11)·0.0000 = 0.2182
- Gini reduction = 0.3967 − 0.2182 = 0.1785

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B53. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Price? · [9+, 8−] · Low · [3+, 4−] · Medium · [6+, 4−]

For the split of [9+, 8−] on Price shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4983; Weighted child Gini = 0.4840; Gini reduction = 0.0142

Working:

- Gini(parent) = 1 − (9/17)² − (8/17)² = 0.4983
- Gini(Low) = 1 − (3/7)² − (4/7)² = 0.4898
- Gini(Medium) = 1 − (6/10)² − (4/10)² = 0.4800
- Weighted child Gini = (7/17)·0.4898 + (10/17)·0.4800 = 0.4840
- Gini reduction = 0.4983 − 0.4840 = 0.0142

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B54. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Outlook? · [5+, 8−] · Sunny · [3+, 1−] · Overcast · [2+, 7−]

For the split of [5+, 8−] on Outlook shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4734; Weighted child Gini = 0.3547; Gini reduction = 0.1187

Working:

- Gini(parent) = 1 − (5/13)² − (8/13)² = 0.4734
- Gini(Sunny) = 1 − (3/4)² − (1/4)² = 0.3750
- Gini(Overcast) = 1 − (2/9)² − (7/9)² = 0.3457
- Weighted child Gini = (4/13)·0.3750 + (9/13)·0.3457 = 0.3547
- Gini reduction = 0.4734 − 0.3547 = 0.1187

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B55. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Student? · [3+, 3−] · v1 · [0+, 1−] · v2 · [1+, 1−] · v3 · [2+, 1−]

For the split of [3+, 3−] on Student shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.5000; Weighted child Gini = 0.3889; Gini reduction = 0.1111

Working:

- Gini(parent) = 1 − (3/6)² − (3/6)² = 0.5000
- Gini(v1) = 1 − (0/1)² − (1/1)² = 0.0000
- Gini(v2) = 1 − (1/2)² − (1/2)² = 0.5000
- Gini(v3) = 1 − (2/3)² − (1/3)² = 0.4444
- Weighted child Gini = (1/6)·0.0000 + (2/6)·0.5000 + (3/6)·0.4444 = 0.3889

- Gini reduction = 0.5000 − 0.3889 = 0.1111

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B56. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Student? · [8+, 7−] · Red · [4+, 2−] · Green · [3+, 5−] · Blue · [1+, 0−]

For the split of [8+, 7−] on Student shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4978; Weighted child Gini = 0.4278; Gini reduction = 0.0700

Working:

- Gini(parent) = 1 − (8/15)² − (7/15)² = 0.4978
- Gini(Red) = 1 − (4/6)² − (2/6)² = 0.4444
- Gini(Green) = 1 − (3/8)² − (5/8)² = 0.4688
- Gini(Blue) = 1 − (1/1)² − (0/1)² = 0.0000
- Weighted child Gini = (6/15)·0.4444 + (8/15)·0.4688 + (1/15)·0.0000 = 0.4278
- Gini reduction = 0.4978 − 0.4278 = 0.0700

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B57. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Colour? · [3+, 6−] · Red · [1+, 3−] · Green · [0+, 3−] · Blue · [2+, 0−]

For the split of [3+, 6−] on Colour shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4444; Weighted child Gini = 0.1667; Gini reduction = 0.2778

Working:

- Gini(parent) = 1 − (3/9)² − (6/9)² = 0.4444
- Gini(Red) = 1 − (1/4)² − (3/4)² = 0.3750
- Gini(Green) = 1 − (0/3)² − (3/3)² = 0.0000
- Gini(Blue) = 1 − (2/2)² − (0/2)² = 0.0000
- Weighted child Gini = (4/9)·0.3750 + (3/9)·0.0000 + (2/9)·0.0000 = 0.1667
- Gini reduction = 0.4444 − 0.1667 = 0.2778

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B58. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Price? · [4+, 6−] · v1 · [2+, 2−] · v2 · [2+, 3−] · v3 · [0+, 1−]

For the split of [4+, 6−] on Price shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4800; Weighted child Gini = 0.4400; Gini reduction = 0.0400

Working:

- Gini(parent) = 1 − (4/10)² − (6/10)² = 0.4800
- Gini(v1) = 1 − (2/4)² − (2/4)² = 0.5000
- Gini(v2) = 1 − (2/5)² − (3/5)² = 0.4800
- Gini(v3) = 1 − (0/1)² − (1/1)² = 0.0000
- Weighted child Gini = (4/10)·0.5000 + (5/10)·0.4800 + (1/10)·0.0000 = 0.4400
- Gini reduction = 0.4800 − 0.4400 = 0.0400

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B59. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Size? · [7+, 3−] · Small · [2+, 1−] · Large · [5+, 2−]

For the split of [7+, 3−] on Size shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4200; Weighted child Gini = 0.4190; Gini reduction = 0.0010

Working:

- Gini(parent) = 1 − (7/10)² − (3/10)² = 0.4200
- Gini(Small) = 1 − (2/3)² − (1/3)² = 0.4444
- Gini(Large) = 1 − (5/7)² − (2/7)² = 0.4082
- Weighted child Gini = (3/10)·0.4444 + (7/10)·0.4082 = 0.4190
- Gini reduction = 0.4200 − 0.4190 = 0.0010

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B60. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Patrons? · [3+, 4−] · v1 · [1+, 2−] · v2 · [0+, 1−] · v3 · [2+, 1−]

For the split of [3+, 4−] on Patrons shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4898; Weighted child Gini = 0.3810; Gini reduction = 0.1088

Working:

- Gini(parent) = 1 − (3/7)² − (4/7)² = 0.4898
- Gini(v1) = 1 − (1/3)² − (2/3)² = 0.4444

- Gini(v2) = 1 − (0/1)² − (1/1)² = 0.0000
- Gini(v3) = 1 − (2/3)² − (1/3)² = 0.4444
- Weighted child Gini = (3/7)·0.4444 + (1/7)·0.0000 + (3/7)·0.4444 = 0.3810
- Gini reduction = 0.4898 − 0.3810 = 0.1088

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B61. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Patrons? · [8+, 7−] · Red · [5+, 6−] · Green · [3+, 1−]

For the split of [8+, 7−] on Patrons shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.4978; Weighted child Gini = 0.4636; Gini reduction = 0.0341

Working:

- Gini(parent) = 1 − (8/15)² − (7/15)² = 0.4978
- Gini(Red) = 1 − (5/11)² − (6/11)² = 0.4959
- Gini(Green) = 1 − (3/4)² − (1/4)² = 0.3750
- Weighted child Gini = (11/15)·0.4959 + (4/15)·0.3750 = 0.4636
- Gini reduction = 0.4978 − 0.4636 = 0.0341

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B62. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Outlook? · [5+, 5−] · Low · [0+, 2−] · Medium · [3+, 2−] · High · [2+, 1−]

For the split of [5+, 5−] on Outlook shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.5000; Weighted child Gini = 0.3733; Gini reduction = 0.1267

Working:

- Gini(parent) = 1 − (5/10)² − (5/10)² = 0.5000
- Gini(Low) = 1 − (0/2)² − (2/2)² = 0.0000
- Gini(Medium) = 1 − (3/5)² − (2/5)² = 0.4800
- Gini(High) = 1 − (2/3)² − (1/3)² = 0.4444
- Weighted child Gini = (2/10)·0.0000 + (5/10)·0.4800 + (3/10)·0.4444 = 0.3733
- Gini reduction = 0.5000 − 0.3733 = 0.1267

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B63. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Hungry? · [3+, 3−] · v1 · [1+, 1−] · v2 · [2+, 1−] · v3 · [0+, 1−]

For the split of [3+, 3−] on Hungry shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.5000; Weighted child Gini = 0.3889; Gini reduction = 0.1111

Working:

- Gini(parent) = 1 − (3/6)² − (3/6)² = 0.5000
- Gini(v1) = 1 − (1/2)² − (1/2)² = 0.5000
- Gini(v2) = 1 − (2/3)² − (1/3)² = 0.4444
- Gini(v3) = 1 − (0/1)² − (1/1)² = 0.0000
- Weighted child Gini = (2/6)·0.5000 + (3/6)·0.4444 + (1/6)·0.0000 = 0.3889
- Gini reduction = 0.5000 − 0.3889 = 0.1111

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B64. [BLANKS · Medium · calculation]

[Diagram as text] Labels: Income? · [8+, 8−] · Low · [3+, 6−] · Medium · [5+, 2−]

For the split of [8+, 8−] on Income shown, give (1) Gini(parent), (2) the weighted child Gini after the split, (3) the Gini reduction. Use 4 decimal places.

(1) Gini(parent) = ____
(2) Weighted child Gini = ____
(3) Gini reduction = ____

Answer: Gini(parent) = 0.5000; Weighted child Gini = 0.4286; Gini reduction = 0.0714

Working:

- Gini(parent) = 1 − (8/16)² − (8/16)² = 0.5000
- Gini(Low) = 1 − (3/9)² − (6/9)² = 0.4444
- Gini(Medium) = 1 − (5/7)² − (2/7)² = 0.4082
- Weighted child Gini = (9/16)·0.4444 + (7/16)·0.4082 = 0.4286
- Gini reduction = 0.5000 − 0.4286 = 0.0714

Takeaway: Choose the split with the LOWEST weighted Gini (= highest Gini reduction).

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B65. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [6+, 9−] · Low · [0+, 4−] · Medium · [4+, 1−] · High · [2+, 4−]

```
[Diagram as text] Labels: Student? · [6+, 9−] · a · [2+, 7−] · b · [4+, 2−]
```

Two candidate splits of [6+, 9−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

Answer: IG(Colour) = 0.363; IG(Student) = 0.145; Chosen attribute = Colour

Working:

- H(parent [6+, 9−]) = −(6/15)·log₂(6/15) = 0.5288 + −(9/15)·log₂(9/15) = 0.4422 = 0.9710
- H(Low [0+, 4−]) = 0 (pure part: 0·log₂0 = 0) + −(4/4)·log₂(4/4) = 0.0000 = 0.0000
- H(Medium [4+, 1−]) = −(4/5)·log₂(4/5) = 0.2575 + −(1/5)·log₂(1/5) = 0.4644 = 0.7219
- H(High [2+, 4−]) = −(2/6)·log₂(2/6) = 0.5283 + −(4/6)·log₂(4/6) = 0.3900 = 0.9183
- Weighted child entropy = (4/15)·0.0000 + (5/15)·0.7219 + (6/15)·0.9183 = 0.6080
- IG(Colour) = 0.9710 − 0.6080 = 0.3630
- H(Student=a [2+, 7−]) = −(2/9)·log₂(2/9) = 0.4822 + −(7/9)·log₂(7/9) = 0.2820 = 0.7642
- H(Student=b [4+, 2−]) = −(4/6)·log₂(4/6) = 0.3900 + −(2/6)·log₂(2/6) = 0.5283 = 0.9183
- Weighted child entropy (Student) = 0.8258 → IG(Student) = 0.9710 − 0.8258 = 0.1451
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.2844, Student = 0.3852 → Gini prefers Colour (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B66. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Price? · [8+, 3−] · Red · [3+, 2−] · Green · [5+, 1−]

```
[Diagram as text] Labels: Student? · [8+, 3−] · a · [3+, 0−] · b · [4+, 2−] · c · [1+, 1−]
```

Two candidate splits of [8+, 3−] are shown: Price and Student. Give IG(Price), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Price) = _____
(2) IG(Student) = _____
(3) Chosen attribute = _____

Answer: IG(Price) = 0.049; IG(Student) = 0.163; Chosen attribute = Student

Working:

- H(parent [8+, 3−]) = −(8/11)·log₂(8/11) = 0.3341 + −(3/11)·log₂(3/11) = 0.5112 = 0.8454
- H(Red [3+, 2−]) = −(3/5)·log₂(3/5) = 0.4422 + −(2/5)·log₂(2/5) = 0.5288 = 0.9710
- H(Green [5+, 1−]) = −(5/6)·log₂(5/6) = 0.2192 + −(1/6)·log₂(1/6) = 0.4308 = 0.6500
- Weighted child entropy = (5/11)·0.9710 + (6/11)·0.6500 = 0.7959
- IG(Price) = 0.8454 − 0.7959 = 0.0495
- H(Student=a [3+, 0−]) = −(3/3)·log₂(3/3) = 0.0000 + 0 (pure part: 0·log₂0 = 0) = 0.0000
- H(Student=b [4+, 2−]) = −(4/6)·log₂(4/6) = 0.3900 + −(2/6)·log₂(2/6) = 0.5283 = 0.9183
- H(Student=c [1+, 1−]) = −(1/2)·log₂(1/2) = 0.5000 + −(1/2)·log₂(1/2) = 0.5000 = 1.0000
- Weighted child entropy (Student) = 0.6827 → IG(Student) = 0.8454 − 0.6827 = 0.1626
- Highest IG → split on Student.
- Gini check: weighted Gini Price = 0.3697, Student = 0.3333 → Gini prefers Student (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B67. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Student? · [5+, 6−] · Some · [1+, 2−] · Full · [2+, 1−] · None · [2+, 3−]

```
[Diagram as text] Labels: Patrons? · [5+, 6−] · a · [2+, 5−] · b · [3+, 1−]
```

Two candidate splits of [5+, 6−] are shown: Student and Patrons. Give IG(Student), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Student) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

Answer: IG(Student) = 0.052; IG(Patrons) = 0.150; Chosen attribute = Patrons

Working:

- H(parent [5+, 6−]) = $-(5/11)\cdot\log_2(5/11) = 0.5170 + -(6/11)\cdot\log_2(6/11) = 0.4770 = 0.9940$
- H(Some [1+, 2−]) = $-(1/3)\cdot\log_2(1/3) = 0.5283 + -(2/3)\cdot\log_2(2/3) = 0.3900 = 0.9183$
- H(Full [2+, 1−]) = $-(2/3)\cdot\log_2(2/3) = 0.3900 + -(1/3)\cdot\log_2(1/3) = 0.5283 = 0.9183$
- H(None [2+, 3−]) = $-(2/5)\cdot\log_2(2/5) = 0.5288 + -(3/5)\cdot\log_2(3/5) = 0.4422 = 0.9710$
- Weighted child entropy = $(3/11)\cdot0.9183 + (3/11)\cdot0.9183 + (5/11)\cdot0.9710 = 0.9422$
- IG(Student) = $0.9940 - 0.9422 = 0.0518$
- H(Patrons=a [2+, 5−]) = $-(2/7)\cdot\log_2(2/7) = 0.5164 + -(5/7)\cdot\log_2(5/7) = 0.3467 = 0.8631$
- H(Patrons=b [3+, 1−]) = $-(3/4)\cdot\log_2(3/4) = 0.3113 + -(1/4)\cdot\log_2(1/4) = 0.5000 = 0.8113$
- Weighted child entropy (Patrons) = 0.8443 → IG(Patrons) = $0.9940 - 0.8443 = 0.1498$
- Highest IG → split on Patrons.
- Gini check: weighted Gini Student = 0.4606, Patrons = 0.3961 → Gini prefers Patrons (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B68. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [6+, 5−] · Sunny · [1+, 0−] · Overcast · [3+, 2−] · Rain · [2+, 3−]

```
[Diagram as text] Labels: Income? · [6+, 5−] · a · [4+, 2−] · b · [2+, 3−]
```

Two candidate splits of [6+, 5−] are shown: Colour and Income. Give IG(Colour), IG(Income) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Income) = ____
(3) Chosen attribute = ____

Answer: IG(Colour) = 0.111; IG(Income) = 0.052; Chosen attribute = Colour

Working:

- H(parent [6+, 5−]) = $-(6/11)\cdot\log_2(6/11) = 0.4770 + -(5/11)\cdot\log_2(5/11) = 0.5170 = 0.9940$
- H(Sunny [1+, 0−]) = $-(1/1)\cdot\log_2(1/1) = 0.0000 + 0$ (pure part: $0\cdot\log_2 0 = 0$) $= 0.0000$
- H(Overcast [3+, 2−]) = $-(3/5)\cdot\log_2(3/5) = 0.4422 + -(2/5)\cdot\log_2(2/5) = 0.5288 = 0.9710$
- H(Rain [2+, 3−]) = $-(2/5)\cdot\log_2(2/5) = 0.5288 + -(3/5)\cdot\log_2(3/5) = 0.4422 = 0.9710$
- Weighted child entropy = $(1/11)\cdot0.0000 + (5/11)\cdot0.9710 + (5/11)\cdot0.9710 = 0.8827$
- IG(Colour) = $0.9940 - 0.8827 = 0.1113$
- H(Income=a [4+, 2−]) = $-(4/6)\cdot\log_2(4/6) = 0.3900 + -(2/6)\cdot\log_2(2/6) = 0.5283 = 0.9183$
- H(Income=b [2+, 3−]) = $-(2/5)\cdot\log_2(2/5) = 0.5288 + -(3/5)\cdot\log_2(3/5) = 0.4422 = 0.9710$
- Weighted child entropy (Income) = 0.9422 → IG(Income) = $0.9940 - 0.9422 = 0.0518$
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.4364, Income = 0.4606 → Gini prefers Colour (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B69. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Hungry? · [8+, 7−] · Small · [1+, 4−] · Large · [7+, 3−]

```
[Diagram as text] Labels: Patrons? · [8+, 7−] · a · [2+, 3−] · b · [4+, 2−] · c · [2+, 2−]
```

Two candidate splits of [8+, 7−] are shown: Hungry and Patrons. Give IG(Hungry), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Hungry) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

Answer: IG(Hungry) = 0.169; IG(Patrons) = 0.039; Chosen attribute = Hungry

Working:

- H(parent [8+, 7−]) = −(8/15)·$\log_2$(8/15) = 0.4837 + −(7/15)·$\log_2$(7/15) = 0.5131 = 0.9968
- H(Small [1+, 4−]) = −(1/5)·$\log_2$(1/5) = 0.4644 + −(4/5)·$\log_2$(4/5) = 0.2575 = 0.7219
- H(Large [7+, 3−]) = −(7/10)·$\log_2$(7/10) = 0.3602 + −(3/10)·$\log_2$(3/10) = 0.5211 = 0.8813
- Weighted child entropy = (5/15)·0.7219 + (10/15)·0.8813 = 0.8282
- IG(Hungry) = 0.9968 − 0.8282 = 0.1686
- H(Patrons=a [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- H(Patrons=b [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- H(Patrons=c [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- Weighted child entropy (Patrons) = 0.9576 → IG(Patrons) = 0.9968 − 0.9576 = 0.0392
- Highest IG → split on Hungry.
- Gini check: weighted Gini Hungry = 0.3867, Patrons = 0.4711 → Gini prefers Hungry (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B70. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Income? · [3+, 7−] · v1 · [0+, 2−] · v2 · [1+, 3−] · v3 · [2+, 2−]

```
[Diagram as text] Labels: Price? · [3+, 7−] · a · [1+, 2−] · b · [2+, 5−]
```

Two candidate splits of [3+, 7−] are shown: Income and Price. Give IG(Income), IG(Price) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Income) = ____
(2) IG(Price) = ____
(3) Chosen attribute = ____

Answer: IG(Income) = 0.157; IG(Price) = 0.002; Chosen attribute = Income

Working:

- H(parent [3+, 7−]) = −(3/10)·$\log_2$(3/10) = 0.5211 + −(7/10)·$\log_2$(7/10) = 0.3602 = 0.8813
- H(v1 [0+, 2−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(2/2)·$\log_2$(2/2) = 0.0000 = 0.0000
- H(v2 [1+, 3−]) = −(1/4)·$\log_2$(1/4) = 0.5000 + −(3/4)·$\log_2$(3/4) = 0.3113 = 0.8113
- H(v3 [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- Weighted child entropy = (2/10)·0.0000 + (4/10)·0.8113 + (4/10)·1.0000 = 0.7245
- IG(Income) = 0.8813 − 0.7245 = 0.1568
- H(Price=a [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- H(Price=b [2+, 5−]) = −(2/7)·$\log_2$(2/7) = 0.5164 + −(5/7)·$\log_2$(5/7) = 0.3467 = 0.8631

- Weighted child entropy (Price) = 0.8797 → IG(Price) = 0.8813 − 0.8797 = 0.0016
- Highest IG → split on Income.
- Gini check: weighted Gini Income = 0.3500, Price = 0.4190 → Gini prefers Income (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B71. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Price? · [8+, 6−] · Red · [2+, 2−] · Green · [6+, 4−]

```
[Diagram as text] Labels: Colour? · [8+, 6−] · a · [7+, 0−] · b · [0+, 4−] · c · [1+, 2−]
```

Two candidate splits of [8+, 6−] are shown: Price and Colour. Give IG(Price), IG(Colour) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Price) = ____
(2) IG(Colour) = ____
(3) Chosen attribute = ____

Answer: IG(Price) = 0.006; IG(Colour) = 0.788; Chosen attribute = Colour

Working:

- H(parent [8+, 6−]) = −(8/14)·$\log_2$(8/14) = 0.4613 + −(6/14)·$\log_2$(6/14) = 0.5239 = 0.9852
- H(Red [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- H(Green [6+, 4−]) = −(6/10)·$\log_2$(6/10) = 0.4422 + −(4/10)·$\log_2$(4/10) = 0.5288 = 0.9710
- Weighted child entropy = (4/14)·1.0000 + (10/14)·0.9710 = 0.9793
- IG(Price) = 0.9852 − 0.9793 = 0.0060
- H(Colour=a [7+, 0−]) = −(7/7)·$\log_2$(7/7) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(Colour=b [0+, 4−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(4/4)·$\log_2$(4/4) = 0.0000 = 0.0000
- H(Colour=c [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- Weighted child entropy (Colour) = 0.1968 → IG(Colour) = 0.9852 − 0.1968 = 0.7885
- Highest IG → split on Colour.
- Gini check: weighted Gini Price = 0.4857, Colour = 0.0952 → Gini prefers Colour (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B72. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Outlook? · [6+, 3−] · Red · [4+, 3−] · Green · [2+, 0−]

```
[Diagram as text] Labels: Price? · [6+, 3−] · a · [3+, 0−] · b · [1+, 1−] · c · [2+, 2−]
```

Two candidate splits of [6+, 3−] are shown: Outlook and Price. Give IG(Outlook), IG(Price) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Outlook) = ____
(2) IG(Price) = ____
(3) Chosen attribute = ____

Answer: IG(Outlook) = 0.152; IG(Price) = 0.252; Chosen attribute = Price

Working:

- H(parent [6+, 3−]) = −(6/9)·$\log_2$(6/9) = 0.3900 + −(3/9)·$\log_2$(3/9) = 0.5283 = 0.9183
- H(Red [4+, 3−]) = −(4/7)·$\log_2$(4/7) = 0.4613 + −(3/7)·$\log_2$(3/7) = 0.5239 = 0.9852

- H(Green [2+, 0−]) = −(2/2)·log₂(2/2) = 0.0000 + 0 (pure part: 0·log₂0 = 0) = 0.0000
- Weighted child entropy = (7/9)·0.9852 + (2/9)·0.0000 = 0.7663
- IG(Outlook) = 0.9183 − 0.7663 = 0.1520
- H(Price=a [3+, 0−]) = −(3/3)·log₂(3/3) = 0.0000 + 0 (pure part: 0·log₂0 = 0) = 0.0000
- H(Price=b [1+, 1−]) = −(1/2)·log₂(1/2) = 0.5000 + −(1/2)·log₂(1/2) = 0.5000 = 1.0000
- H(Price=c [2+, 2−]) = −(2/4)·log₂(2/4) = 0.5000 + −(2/4)·log₂(2/4) = 0.5000 = 1.0000
- Weighted child entropy (Price) = 0.6667 → IG(Price) = 0.9183 − 0.6667 = 0.2516
- Highest IG → split on Price.
- Gini check: weighted Gini Outlook = 0.3810, Price = 0.3333 → Gini prefers Price (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B73. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [3+, 7−] · Sunny · [2+, 3−] · Overcast · [1+, 4−]

```
[Diagram as text] Labels: Income? · [3+, 7−] · a · [0+, 2−] · b · [1+, 4−] · c · [2+, 1−]
```

Two candidate splits of [3+, 7−] are shown: Colour and Income. Give IG(Colour), IG(Income) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = _____
(2) IG(Income) = _____
(3) Chosen attribute = _____

Answer: IG(Colour) = 0.035; IG(Income) = 0.245; Chosen attribute = Income

Working:

- H(parent [3+, 7−]) = −(3/10)·log₂(3/10) = 0.5211 + −(7/10)·log₂(7/10) = 0.3602 = 0.8813
- H(Sunny [2+, 3−]) = −(2/5)·log₂(2/5) = 0.5288 + −(3/5)·log₂(3/5) = 0.4422 = 0.9710
- H(Overcast [1+, 4−]) = −(1/5)·log₂(1/5) = 0.4644 + −(4/5)·log₂(4/5) = 0.2575 = 0.7219
- Weighted child entropy = (5/10)·0.9710 + (5/10)·0.7219 = 0.8464
- IG(Colour) = 0.8813 − 0.8464 = 0.0349
- H(Income=a [0+, 2−]) = 0 (pure part: 0·log₂0 = 0) + −(2/2)·log₂(2/2) = 0.0000 = 0.0000
- H(Income=b [1+, 4−]) = −(1/5)·log₂(1/5) = 0.4644 + −(4/5)·log₂(4/5) = 0.2575 = 0.7219
- H(Income=c [2+, 1−]) = −(2/3)·log₂(2/3) = 0.3900 + −(1/3)·log₂(1/3) = 0.5283 = 0.9183
- Weighted child entropy (Income) = 0.6365 → IG(Income) = 0.8813 − 0.6365 = 0.2448
- Highest IG → split on Income.
- Gini check: weighted Gini Colour = 0.4000, Income = 0.2933 → Gini prefers Income (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B74. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Size? · [6+, 8−] · Small · [2+, 3−] · Large · [4+, 5−]

```
[Diagram as text] Labels: Patrons? · [6+, 8−] · a · [0+, 3−] · b · [4+, 2−] · c · [2+, 3−]
```

Two candidate splits of [6+, 8−] are shown: Size and Patrons. Give IG(Size), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Size) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

Answer: IG(Size) = 0.001; IG(Patrons) = 0.245; Chosen attribute = Patrons

Working:

- H(parent [6+, 8−]) = −(6/14)·$\log_2$(6/14) = 0.5239 + −(8/14)·$\log_2$(8/14) = 0.4613 = 0.9852
- H(Small [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- H(Large [4+, 5−]) = −(4/9)·$\log_2$(4/9) = 0.5200 + −(5/9)·$\log_2$(5/9) = 0.4711 = 0.9911
- Weighted child entropy = (5/14)·0.9710 + (9/14)·0.9911 = 0.9839
- IG(Size) = 0.9852 − 0.9839 = 0.0013
- H(Patrons=a [0+, 3−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(3/3)·$\log_2$(3/3) = 0.0000 = 0.0000
- H(Patrons=b [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- H(Patrons=c [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- Weighted child entropy (Patrons) = 0.7403 → IG(Patrons) = 0.9852 − 0.7403 = 0.2449
- Highest IG → split on Patrons.
- Gini check: weighted Gini Size = 0.4889, Patrons = 0.3619 → Gini prefers Patrons (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B75. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Patrons? · [6+, 5−] · Red · [4+, 2−] · Green · [2+, 3−]

```
[Diagram as text] Labels: Student? · [6+, 5−] · a · [3+, 1−] · b · [1+, 2−] · c · [2+, 2−]
```

Two candidate splits of [6+, 5−] are shown: Patrons and Student. Give IG(Patrons), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Patrons) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

Answer: IG(Patrons) = 0.052; IG(Student) = 0.085; Chosen attribute = Student

Working:

- H(parent [6+, 5−]) = −(6/11)·$\log_2$(6/11) = 0.4770 + −(5/11)·$\log_2$(5/11) = 0.5170 = 0.9940
- H(Red [4+, 2−]) = −(4/6)·$\log_2$(4/6) = 0.3900 + −(2/6)·$\log_2$(2/6) = 0.5283 = 0.9183
- H(Green [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- Weighted child entropy = (6/11)·0.9183 + (5/11)·0.9710 = 0.9422
- IG(Patrons) = 0.9940 − 0.9422 = 0.0518
- H(Student=a [3+, 1−]) = −(3/4)·$\log_2$(3/4) = 0.3113 + −(1/4)·$\log_2$(1/4) = 0.5000 = 0.8113
- H(Student=b [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- H(Student=c [2+, 2−]) = −(2/4)·$\log_2$(2/4) = 0.5000 + −(2/4)·$\log_2$(2/4) = 0.5000 = 1.0000
- Weighted child entropy (Student) = 0.9091 → IG(Student) = 0.9940 − 0.9091 = 0.0849
- Highest IG → split on Student.
- Gini check: weighted Gini Patrons = 0.4606, Student = 0.4394 → Gini prefers Student (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B76. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [5+, 8−] · Sunny · [1+, 2−] · Overcast · [1+, 4−] · Rain · [3+, 2−]

```
[Diagram as text] Labels: Student? · [5+, 8−] · a · [1+, 3−] · b · [4+, 5−]
```

Two candidate splits of [5+, 8−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

Answer: IG(Colour) = 0.098; IG(Student) = 0.025; Chosen attribute = Colour

Working:

- H(parent [5+, 8−]) = −(5/13)·$\log_2$(5/13) = 0.5302 + −(8/13)·$\log_2$(8/13) = 0.4310 = 0.9612
- H(Sunny [1+, 2−]) = −(1/3)·$\log_2$(1/3) = 0.5283 + −(2/3)·$\log_2$(2/3) = 0.3900 = 0.9183
- H(Overcast [1+, 4−]) = −(1/5)·$\log_2$(1/5) = 0.4644 + −(4/5)·$\log_2$(4/5) = 0.2575 = 0.7219
- H(Rain [3+, 2−]) = −(3/5)·$\log_2$(3/5) = 0.4422 + −(2/5)·$\log_2$(2/5) = 0.5288 = 0.9710
- Weighted child entropy = (3/13)·0.9183 + (5/13)·0.7219 + (5/13)·0.9710 = 0.8630
- IG(Colour) = 0.9612 − 0.8630 = 0.0982
- H(Student=a [1+, 3−]) = −(1/4)·$\log_2$(1/4) = 0.5000 + −(3/4)·$\log_2$(3/4) = 0.3113 = 0.8113
- H(Student=b [4+, 5−]) = −(4/9)·$\log_2$(4/9) = 0.5200 + −(5/9)·$\log_2$(5/9) = 0.4711 = 0.9911
- Weighted child entropy (Student) = 0.9358 → IG(Student) = 0.9612 − 0.9358 = 0.0255
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.4103, Student = 0.4573 → Gini prefers Colour (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B77. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [8+, 7−] · Yes · [0+, 4−] · No · [8+, 3−]

```
[Diagram as text] Labels: Student? · [8+, 7−] · a · [3+, 2−] · b · [5+, 1−] · c · [0+, 4−]
```

Two candidate splits of [8+, 7−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

Answer: IG(Colour) = 0.377; IG(Student) = 0.413; Chosen attribute = Student

Working:

- H(parent [8+, 7−]) = −(8/15)·$\log_2$(8/15) = 0.4837 + −(7/15)·$\log_2$(7/15) = 0.5131 = 0.9968
- H(Yes [0+, 4−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(4/4)·$\log_2$(4/4) = 0.0000 = 0.0000
- H(No [8+, 3−]) = −(8/11)·$\log_2$(8/11) = 0.3341 + −(3/11)·$\log_2$(3/11) = 0.5112 = 0.8454
- Weighted child entropy = (4/15)·0.0000 + (11/15)·0.8454 = 0.6199
- IG(Colour) = 0.9968 − 0.6199 = 0.3769
- H(Student=a [3+, 2−]) = −(3/5)·$\log_2$(3/5) = 0.4422 + −(2/5)·$\log_2$(2/5) = 0.5288 = 0.9710
- H(Student=b [5+, 1−]) = −(5/6)·$\log_2$(5/6) = 0.2192 + −(1/6)·$\log_2$(1/6) = 0.4308 = 0.6500
- H(Student=c [0+, 4−]) = 0 (pure part: 0·$\log_2$0 = 0) + −(4/4)·$\log_2$(4/4) = 0.0000 = 0.0000

- Weighted child entropy (Student) = 0.5837 → IG(Student) = 0.9968 − 0.5837 = 0.4131
- Highest IG → split on Student.
- Gini check: weighted Gini Colour = 0.2909, Student = 0.2711 → Gini prefers Student (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B78. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Size? · [4+, 6−] · Sunny · [3+, 3−] · Overcast · [1+, 3−]

```
[Diagram as text] Labels: Patrons? · [4+, 6−] · a · [1+, 1−] · b · [1+, 2−] · c · [2+, 3−]
```

Two candidate splits of [4+, 6−] are shown: Size and Patrons. Give IG(Size), IG(Patrons) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Size) = ____
(2) IG(Patrons) = ____
(3) Chosen attribute = ____

Answer: IG(Size) = 0.046; IG(Patrons) = 0.010; Chosen attribute = Size

Working:

- H(parent [4+, 6−]) = $-(4/10)\cdot\log_2(4/10) = 0.5288 + -(6/10)\cdot\log_2(6/10) = 0.4422 = 0.9710$
- H(Sunny [3+, 3−]) = $-(3/6)\cdot\log_2(3/6) = 0.5000 + -(3/6)\cdot\log_2(3/6) = 0.5000 = 1.0000$
- H(Overcast [1+, 3−]) = $-(1/4)\cdot\log_2(1/4) = 0.5000 + -(3/4)\cdot\log_2(3/4) = 0.3113 = 0.8113$
- Weighted child entropy = $(6/10)\cdot1.0000 + (4/10)\cdot0.8113 = 0.9245$
- IG(Size) = 0.9710 − 0.9245 = 0.0464
- H(Patrons=a [1+, 1−]) = $-(1/2)\cdot\log_2(1/2) = 0.5000 + -(1/2)\cdot\log_2(1/2) = 0.5000 = 1.0000$
- H(Patrons=b [1+, 2−]) = $-(1/3)\cdot\log_2(1/3) = 0.5283 + -(2/3)\cdot\log_2(2/3) = 0.3900 = 0.9183$
- H(Patrons=c [2+, 3−]) = $-(2/5)\cdot\log_2(2/5) = 0.5288 + -(3/5)\cdot\log_2(3/5) = 0.4422 = 0.9710$
- Weighted child entropy (Patrons) = 0.9610 → IG(Patrons) = 0.9710 − 0.9610 = 0.0100
- Highest IG → split on Size.
- Gini check: weighted Gini Size = 0.4500, Patrons = 0.4733 → Gini prefers Size (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B79. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Colour? · [8+, 6−] · Low · [1+, 4−] · Medium · [4+, 0−] · High · [3+, 2−]

```
[Diagram as text] Labels: Student? · [8+, 6−] · a · [3+, 3−] · b · [5+, 3−]
```

Two candidate splits of [8+, 6−] are shown: Colour and Student. Give IG(Colour), IG(Student) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Colour) = ____
(2) IG(Student) = ____
(3) Chosen attribute = ____

Answer: IG(Colour) = 0.381; IG(Student) = 0.011; Chosen attribute = Colour

Working:

- H(parent [8+, 6−]) = $-(8/14)\cdot\log_2(8/14) = 0.4613 + -(6/14)\cdot\log_2(6/14) = 0.5239 = 0.9852$
- H(Low [1+, 4−]) = $-(1/5)\cdot\log_2(1/5) = 0.4644 + -(4/5)\cdot\log_2(4/5) = 0.2575 = 0.7219$

- H(Medium [4+, 0−]) = −(4/4)·$\log_2$(4/4) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(High [3+, 2−]) = −(3/5)·$\log_2$(3/5) = 0.4422 + −(2/5)·$\log_2$(2/5) = 0.5288 = 0.9710
- Weighted child entropy = (5/14)·0.7219 + (4/14)·0.0000 + (5/14)·0.9710 = 0.6046
- IG(Colour) = 0.9852 − 0.6046 = 0.3806
- H(Student=a [3+, 3−]) = −(3/6)·$\log_2$(3/6) = 0.5000 + −(3/6)·$\log_2$(3/6) = 0.5000 = 1.0000
- H(Student=b [5+, 3−]) = −(5/8)·$\log_2$(5/8) = 0.4238 + −(3/8)·$\log_2$(3/8) = 0.5306 = 0.9544
- Weighted child entropy (Student) = 0.9740 → IG(Student) = 0.9852 − 0.9740 = 0.0113
- Highest IG → split on Colour.
- Gini check: weighted Gini Colour = 0.2857, Student = 0.4821 → Gini prefers Colour (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

TREES B80. [BLANKS · Exam+ · calculation]

[Diagram as text] Labels: Outlook? · [3+, 4−] · v1 · [1+, 3−] · v2 · [1+, 0−] · v3 · [1+, 1−]

```
[Diagram as text] Labels: Hungry? · [3+, 4−] · a · [2+, 3−] · b · [1+, 1−]
```

Two candidate splits of [3+, 4−] are shown: Outlook and Hungry. Give IG(Outlook), IG(Hungry) (3 dp) and the attribute chosen by information gain (one word).

(1) IG(Outlook) = ____
(2) IG(Hungry) = ____
(3) Chosen attribute = ____

Answer: IG(Outlook) = 0.236; IG(Hungry) = 0.006; Chosen attribute = Outlook

Working:

- H(parent [3+, 4−]) = −(3/7)·$\log_2$(3/7) = 0.5239 + −(4/7)·$\log_2$(4/7) = 0.4613 = 0.9852
- H(v1 [1+, 3−]) = −(1/4)·$\log_2$(1/4) = 0.5000 + −(3/4)·$\log_2$(3/4) = 0.3113 = 0.8113
- H(v2 [1+, 0−]) = −(1/1)·$\log_2$(1/1) = 0.0000 + 0 (pure part: 0·$\log_2$0 = 0) = 0.0000
- H(v3 [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- Weighted child entropy = (4/7)·0.8113 + (1/7)·0.0000 + (2/7)·1.0000 = 0.7493
- IG(Outlook) = 0.9852 − 0.7493 = 0.2359
- H(Hungry=a [2+, 3−]) = −(2/5)·$\log_2$(2/5) = 0.5288 + −(3/5)·$\log_2$(3/5) = 0.4422 = 0.9710
- H(Hungry=b [1+, 1−]) = −(1/2)·$\log_2$(1/2) = 0.5000 + −(1/2)·$\log_2$(1/2) = 0.5000 = 1.0000
- Weighted child entropy (Hungry) = 0.9793 → IG(Hungry) = 0.9852 − 0.9793 = 0.0060
- Highest IG → split on Outlook.
- Gini check: weighted Gini Outlook = 0.3571, Hungry = 0.4857 → Gini prefers Outlook (same choice).

Takeaway: IG: pick the highest. Weighted Gini: pick the lowest.

Source: Lecture 4 (Decision trees) + 'Step-by-step process of comparing two attributes to split'

📉 Supervised Learning (Lecture 4)

Part A: hand-written questions (126)

ML A1. [MATCH · Easy]

Match each learning type to its feedback.

1. Supervised
2. Unsupervised
3. Reinforcement

A. series of rewards and punishments
B. input-output pairs
C. no explicit feedback, find patterns
D. a teacher who only says 'faster'

Answer: 1 → B, 2 → C, 3 → A

Working:

- Supervised → input-output pairs
- Unsupervised → no explicit feedback, find patterns
- Reinforcement → series of rewards and punishments

Takeaway: Type of feedback → type of learning.

Source: Lecture 4, slide 6

ML A2. [MCQ · Easy]

Predicting tomorrow's temperature in °C is a…

A. classification problem
B. clustering problem
C. regression problem
D. reinforcement learning problem

Answer: C. regression problem

Option notes:

- A: Output is a continuous number.
- B: Labels exist.
- D: No rewards/actions.

Takeaway: Numeric → regression.

Source: Lecture 4, slide 12

ML A3. [TF · Medium]

Ockham's razor says choose the most complex hypothesis that fits the training data perfectly.

Answer: False

Why: Choose the SIMPLEST hypothesis consistent with the data.

Takeaway: Simpler hypotheses generalise better.

Source: Lecture 4, slide 10

ML A4. [MULTI · Medium]

Which statements about the i.i.d. assumption are correct? (Select ALL that apply.)

A. It requires all features to be independent of each other
B. Future examples are like past examples
C. All examples come from the same distribution
D. Each example is independent of the previous ones

Answer: B. Future examples are like past examples; C. All examples come from the same distribution; D. Each example is independent of the previous ones

Option notes:

- A: Incorrect: That's Naive Bayes' assumption, not i.i.d.
- B: Correct: Stationary assumption.
- C: Correct: Identically distributed.
- D: Correct: Independent.

Takeaway: i.i.d. is about EXAMPLES, not features.

Source: Lecture 4, slide 9

ML A5. [MCQ · Medium]

A degree-12 polynomial passes through every training point but wiggles wildly between them. This is…

A. generalisation
B. overfitting
C. regularisation
D. underfitting

Answer: B. overfitting

Option notes:

- A: It generalises poorly.
- C: Regularisation reduces this.
- D: Underfitting is too simple.

Takeaway: Fits noise → overfits.

Source: Lecture 4, slide 11

ML A6. [MCQ · Easy]

Precision is defined as…

A. (TP + TN) / All
B. TN / (TN + FP)
C. TP / (TP + FP)
D. TP / (TP + FN)

Answer: C. TP / (TP + FP)

Option notes:

- A: That's accuracy.
- B: That's specificity.
- D: That's recall.

Takeaway: Precision: of the predicted positives, how many are right.

Source: Lecture 4, slide 16

ML A7. [MCQ · Medium]

Missing a cancer diagnosis (false negative) is very costly. Which metric should you prioritise?

A. Accuracy
B. Recall
C. Precision
D. Specificity

Answer: B. Recall

Option notes:

- A: Misleading with imbalanced classes.
- C: Precision prioritises avoiding false positives.
- D: Measures true negatives.

Takeaway: Costly FN → recall; costly FP → precision.

Source: Lecture 4, slide 16

ML A8. [TF · Medium]

An AUC of 0.5 means the classifier ranks positives perfectly.

Answer: False

Why: AUC 0.5 = no discrimination (random). 1.0 = perfect.

Takeaway: AUC < 0.5 is worse than random.

Source: Lecture 4, slide 17

ML A9. [MATCH · Medium]

Match the metric to its formula.

1. Accuracy
2. Recall
3. Specificity
4. F1

A. (TP + TN) / All
B. TP / (TP + FN)
C. TN / (TN + FP)
D. TP / (TP + TN)
E. 2 × P × R / (P + R)

Answer: 1 → A, 2 → B, 3 → C, 4 → E

Working:

- Accuracy → (TP + TN) / All
- Recall → TP / (TP + FN)
- Specificity → TN / (TN + FP)
- F1 → 2 × P × R / (P + R)

Takeaway: Know each denominator.

Source: Lecture 4, slide 16

ML A10. [MULTI · Hard]

Select ALL correct statements about ROC/AUC (Lecture 4). (Select ALL that apply.)

A. AUC is threshold-independent
B. Lowering the threshold decreases both TPR and FPR
C. ROC plots TPR against FPR across thresholds
D. An AUC of 0.35 is worse than random
E. AUC equals the probability a random positive is ranked above a random negative

Answer: A. AUC is threshold-independent; C. ROC plots TPR against FPR across thresholds; D. An AUC of 0.35 is worse than random; E. AUC equals the probability a random positive is ranked above a random negative

Option notes:

- A: Correct: It summarises all thresholds.
- B: Incorrect: Lower threshold → more positives → both INCREASE.
- C: Correct: Definition.
- D: Correct: < 0.5.
- E: Correct: Interpretation on slide 17.

Takeaway: AUC = ranking ability.

Source: Lecture 4, slide 17

ML A11. [MCQ · Easy]

Which regression metric can be NEGATIVE?

A. RMSE
B. R^2
C. MAE
D. MSE

Answer: B. R^2

Option notes:

- A: Always ≥ 0.
- C: Always ≥ 0.
- D: Always ≥ 0.

Takeaway: $R^2 < 0$ → worse than predicting the mean.

Source: Lecture 4, slide 18

ML A12. [TF · Medium]

MAPE is a good metric when some actual values are 0.

Answer: False

Why: Division by y: undefined/huge when y is 0 or near 0.

Takeaway: Use MAE/RMSE there.

Source: Lecture 4, slide 18

ML A13. [MCQ · Medium]

Which metric penalises large errors MORE than MAE does?

A. MAPE
B. Accuracy
C. RMSE
D. R^2

Answer: C. RMSE

Option notes:

- A: MAPE is a relative absolute error.
- B: Classification metric.
- D: R^2 is a ratio of explained variance.

Takeaway: Squaring magnifies big errors.

Source: Lecture 4, slide 18

ML A14. [MATCH · Medium]

Match each regression metric to its range.

1. MAE
2. R^2
3. MAPE

A. $[0, \infty)$ %
B. $[-1, 1]$
C. $[0, \infty)$, same units as y
D. $(-\infty, 1]$

Answer: 1 → C, 2 → D, 3 → A

Working:

- MAE → $[0, \infty)$, same units as y
- R^2 → $(-\infty, 1]$
- MAPE → $[0, \infty)$ %

Takeaway: Only R^2 can be negative.

Source: Lecture 4, slide 18

ML A15. [MCQ · Medium]

When should the hold-out test set be used?

A. Before preprocessing to check data quality
B. For every grid-search candidate
C. To train the final model together with training data
D. Once, after hyperparameter tuning, for the final evaluation

Answer: D. Once, after hyperparameter tuning, for the final evaluation

Option notes:

- A: Fit preprocessing on training data only.
- B: That leaks test information into tuning.
- C: Test data must stay unseen.

Takeaway: Avoid data leakage.

Source: Lecture 4, slide 22, 30

ML A16. [BLANK · Easy]

In 5-fold cross-validation the average CV score is (S_1 + … + S_5) / _____.

Answer: 5

Takeaway: Average over the k folds.

Source: Lecture 4, slide 23

ML A17. [MULTI · Medium]

Which are steps of the Lecture 4 classifier workflow? (Select ALL that apply.)

A. Tune hyperparameters using test-set accuracy
B. Retrain the best model on the entire training set
C. Deploy and monitor for data drift
D. Preprocess: handle missing values, encode categoricals, scale
E. Hyperparameter tuning with k-fold CV on the training set

Answer: B. Retrain the best model on the entire training set; C. Deploy and monitor for data drift; D. Preprocess: handle missing values, encode categoricals, scale; E. Hyperparameter tuning with k-fold CV on the training set

Option notes:

- A: Incorrect: Leakage.
- B: Correct: Step 6.
- C: Correct: Step 8.
- D: Correct: Step 2.
- E: Correct: Step 4-5.

Takeaway: Train → tune (CV) → retrain → test once → deploy.

Source: Lecture 4, slide 21

ML A18. [TF · Medium]

StratifiedKFold keeps class proportions similar in each fold, which matters for classification.

Answer: True

Why: Lecture 4 uses StratifiedKFold for classifiers, KFold for regressors.

Takeaway: stratify=y also when splitting.

Source: Lecture 4, slide 30

ML A19. [MCQ · Hard]

GridSearchCV with refit=True. What is best_estimator_?

A. The parameter dictionary
B. The model with the best TEST score
C. An average of all fold models
D. The best parameter setting refitted on the whole training set

Answer: D. The best parameter setting refitted on the whole training set

Option notes:

- A: That's best_params_.
- B: Grid search never uses the test set.
- C: It refits one model.

Takeaway: best_params_ = settings; best_estimator_ = refitted model.

Source: Lecture 4, slide 30

ML A20. [MCQ · Medium]

Which statement correctly contrasts bagging and boosting?

A. Random forest is a boosting method
B. Both always use the same training sample
C. Bagging trains learners independently; boosting trains them sequentially, each correcting earlier errors
D. Bagging is sequential; boosting is parallel

Answer: C. Bagging trains learners independently; boosting trains them sequentially, each correcting earlier errors

Option notes:

- A: It is bagging.
- B: Bagging uses bootstrap samples.
- D: Reversed.

Takeaway: Independent vs dependent learners.

Source: Lecture 4, slide 43

ML A21. [MULTI · Medium]

Select ALL true statements about random forests. (Select ALL that apply.)

A. They reduce variance compared with a single tree
B. Each tree is trained on a bootstrap sample
C. A random subset of features is considered at each split
D. Classification uses majority vote
E. Trees are built sequentially to fix residuals

Answer: A. They reduce variance compared with a single tree; B. Each tree is trained on a bootstrap sample; C. A random subset of features is considered at each split; D. Classification uses majority vote

Option notes:

- A: Correct: Main benefit.
- B: Correct: Rows drawn with replacement.
- C: Correct: max_features.
- D: Correct: Regression averages.
- E: Incorrect: That's gradient boosting.

Takeaway: RF = bagging + random features.

Source: Lecture 4, slide 44

ML A22. [TF · Medium]

In gradient boosting, a smaller learning_rate usually needs more trees (n_estimators).

Answer: True

Why: Each tree contributes less, so more stages are needed.

Takeaway: Tune learning_rate and n_estimators together.

Source: Lecture 4, slide 47

ML A23. [MATCH · Hard]

Match the hyperparameter to its role.

1. n_estimators
2. max_features
3. learning_rate
4. bootstrap

A. sample rows with replacement
B. shrinkage of each tree's contribution
C. number of folds in CV
D. features checked at each split
E. number of trees / boosting stages

Answer: 1 → E, 2 → D, 3 → B, 4 → A

Working:

- n_estimators → number of trees / boosting stages
- max_features → features checked at each split
- learning_rate → shrinkage of each tree's contribution
- bootstrap → sample rows with replacement

Takeaway: Know what each knob does.

Source: Lecture 4, slide 44-47

ML A24. [TF · Easy]

Gradient descent moves the weights in the direction of the positive gradient.

Answer: False

Why: It moves AGAINST the gradient: $w \leftarrow w - \alpha \partial L/\partial w$.

Takeaway: Descent = minus.

Source: Lecture 4 (Supervised learning)

ML A25. [MCQ · Medium]

What happens if the learning rate α is too large?

A. The loss function becomes convex
B. Updates can overshoot the minimum and diverge
C. The model has more parameters
D. Training always converges faster to the same minimum

Answer: B. Updates can overshoot the minimum and diverge

Option notes:

- A: α doesn't change the loss shape.
- C: Unrelated.
- D: It can oscillate/diverge.

Takeaway: Too big: overshoot. Too small: slow.

Source: Lecture 4 (Supervised learning)

ML A26. [BLANKS · Medium · calculation]

$w_0 = 1$, $w_1 = 2$, $\alpha = 0.1$, example $(x, y) = (3, 10)$. Using $w_0 \leftarrow w_0 + \alpha(y - h)$, $w_1 \leftarrow w_1 + \alpha(y - h)x$, give new w_0 and w_1.

(1) w_0 = ____
(2) w_1 = ____

Answer: $w_0 = 1.3$; $w_1 = 2.9$

Takeaway: h = 7, error 3.

Source: Lecture 4 (Supervised learning)

ML A27. [MCQ · Medium]

Why can gradient descent find the optimal w for linear regression with squared loss?

A. Because the model is non-linear
B. Because the learning rate decays to zero
C. Because it checks every possible w
D. The loss is a continuous convex function, so there is a single global minimum

Answer: D. The loss is a continuous convex function, so there is a single global minimum

Option notes:

- A: It's linear.
- B: Decay helps convergence, but convexity guarantees the global optimum.
- C: It follows the gradient, not exhaustive search.

Takeaway: Convex → no local minima traps.

Source: Lecture 4, slide 53

ML A28. [MCQ · Easy]

What does the logistic (sigmoid) function output?

A. A value between −1 and 1
B. A value between 0 and 1, interpreted as a class probability
C. Any real number
D. Exactly 0 or 1

Answer: B. A value between 0 and 1, interpreted as a class probability

Option notes:

- A: That's tanh.
- C: That's the linear score z.
- D: That's a hard threshold.

Takeaway: $\sigma(z) \in (0, 1)$.

Source: Lecture 4 (Supervised learning)

ML A29. [MATCH · Medium]

Match each model to its target and typical evaluation.

1. Linear regression
2. Logistic regression

A. continuous target; MAE, RMSE, R^2
B. unlabelled data; silhouette score
C. categorical target; accuracy, F1, ROC-AUC

Answer: 1 → A, 2 → C

Working:

- Linear regression → continuous target; MAE, RMSE, R^2
- Logistic regression → categorical target; accuracy, F1, ROC-AUC

Takeaway: Linear → numbers; logistic → classes.

Source: Lecture 4, slide 55

ML A30. [TF · Medium]

For logistic regression with binary cross-entropy, the gradient w.r.t. weight w_j is $(p - y) \cdot x_j$.

Answer: True

Why: Slide 54.

Takeaway: Clean gradient: error × input.

Source: Lecture 4, slide 54

ML A31. [MCQ · Hard · calculation]

A model predicts p = 0.99 for an example whose true label is 0. The cross-entropy loss is…

A. near 0, because the model is confident
B. large ($-\ln 0.01 \approx 4.6$)
C. negative
D. exactly 0.99

Answer: B. large ($-\ln 0.01 \approx 4.6$)

Option notes:

- A: Confident AND wrong → large loss.
- C: Cross-entropy is ≥ 0.
- D: Loss = $-\ln(1 - p)$.

Takeaway: Cross-entropy punishes confident mistakes.

Source: Lecture 4 (Supervised learning)

ML A32. [MCQ · Easy]

Why should features be scaled before KNN?

A. Scaling turns KNN into a linear model
B. KNN needs normally distributed inputs
C. Distances would otherwise be dominated by features with large numeric ranges
D. Scaling reduces the number of neighbours

Answer: C. Distances would otherwise be dominated by features with large numeric ranges

Option notes:

- A: No.
- B: Not required.
- D: k is chosen separately.

Takeaway: Pipeline with StandardScaler.

Source: Lecture 4, slide 59

ML A33. [MULTI · Medium]

Select ALL true statements about KNN. (Select ALL that apply.)

A. It works for both classification and regression
B. It is sensitive to irrelevant features
C. It learns a separating hyperplane
D. It is instance-based / non-parametric
E. Prediction can be slow on large datasets

Answer: A. It works for both classification and regression; B. It is sensitive to irrelevant features; D. It is instance-based / non-parametric; E. Prediction can be slow on large datasets

Option notes:

- A: Correct: Vote / average.
- B: Correct: They distort distances.
- C: Incorrect: That's SVM.
- D: Correct: Stores training data.
- E: Correct: Distances to every point.

Takeaway: KNN = lazy learner.

Source: Lecture 4, slide 59-63

ML A34. [BLANK · Medium · calculation]

Point | area | bedrooms | price ($1000) | distance to (90, 2)
3 | 80 | 2 | 220 | 10.00
4 | 100 | 3 | 260 | 10.05
2 | 60 | 2 | 180 | 30.00
5 | 120 | 3 | 300 | 30.02
1 | 50 | 1 | 150 | 40.01
6 | 150 | 4 | 360 | 60.03
7 | 200 | 5 | 450 | 110.04

House example: the 3 nearest neighbours of (90, 2) have prices 220, 260 and 180 ($1000). KNN regression (k = 3, uniform weights) predicts _____ ($1000).

Answer: 220
(Also accepted: 220.0)

Takeaway: (220 + 260 + 180)/3.

Source: Lecture 4, slide 61

ML A35. [TF · Medium]

Minkowski distance with p = 1 is the Manhattan distance and with p = 2 is the Euclidean distance.

Answer: True

Why: Minkowski generalises both.

Takeaway: p in KNeighborsRegressor is the Minkowski power.

Source: Lecture 4, slide 63-65

ML A36. [MCQ · Medium]

Iris query (5.4, 3.9): its 3 nearest neighbours are all Setosa. KNN with k = 3 predicts…

A. The average of the three labels
B. Setosa
C. Versicolor
D. Undecided

Answer: B. Setosa

Option notes:

- A: Classification uses a vote.
- C: No votes.
- D: 3 votes to 0.

Takeaway: Majority vote.

Source: Lecture 4, slide 62

ML A37. [MCQ · Easy]

What are support vectors?

A. All training points
B. The training points closest to the separating hyperplane, which define the margin
C. Points misclassified by the model
D. The weight vector w

Answer: B. The training points closest to the separating hyperplane, which define the margin

Option notes:

- A: Only the closest ones matter.
- C: Those relate to slack in soft margins.
- D: w is the normal of the hyperplane.

Takeaway: Support vectors hold up the margin.

Source: Lecture 4, slide 67-69

ML A38. [BLANK · Medium · calculation]

Point | x_1 | x_2 | y
P1 | 3 | 3 | +1
P2 | 4 | 3 | +1
P3 | 1 | 1 | −1

Lecture 4 slide 71: separator w = (0.5, 0.5), b = −2.

Lecture 4 example: w = (0.5, 0.5), b = −2. The margin width 2/‖w‖ = _____ (3 dp).

Answer: 2.828
(Also accepted: 2.83)

Takeaway: ‖w‖ = 0.7071 → 2/0.7071.

Source: Lecture 4, slide 71

ML A39. [TF · Medium]

In a soft-margin SVM, a larger C allows more margin violations.

Answer: False

Why: Large C penalises errors MORE strongly → fewer violations, narrower margin.

Takeaway: Small C = softer margin.

Source: Lecture 4, slide 69

ML A40. [MULTI · Hard]

Select ALL true statements about kernels. (Select ALL that apply.)

A. The kernel trick avoids computing the high-dimensional coordinates explicitly
B. Kernels make SVM unable to handle non-linear data
C. RBF kernel: $K(x,z) = \exp(-\|x - z\|^2/(2\sigma^2))$
D. Polynomial kernel: $(x \cdot z + 1)^p$
E. Too high gamma in RBF gives very wiggly boundaries

Answer: A. The kernel trick avoids computing the high-dimensional coordinates explicitly; C. RBF kernel: $K(x,z) = \exp(-\|x - z\|^2/(2\sigma^2))$; D. Polynomial kernel: $(x \cdot z + 1)^p$; E. Too high gamma in RBF gives very wiggly boundaries

Option notes:

- A: Correct: Uses pairwise similarities.
- B: Incorrect: Opposite: they enable it.
- C: Correct: Slide 79.
- D: Correct: Slide 79.
- E: Correct: Slide 76.

Takeaway: Kernel = similarity.

Source: Lecture 4, slide 76-79

ML A41. [MCQ · Hard]

Data inside a circle $x_1^2 + x_2^2 \leq 1$ vs outside is not linearly separable in 2D. Lecture 4 maps it to $(x_1^2, x_2^2, \sqrt{2}x_1x_2)$. Why?

A. To reduce the number of features
B. To make the data i.i.d.
C. In the 3-D feature space the classes become linearly separable by a plane
D. To remove the support vectors

Answer: C. In the 3-D feature space the classes become linearly separable by a plane

Option notes:

- A: It increases them.
- B: Unrelated.
- D: Support vectors remain.

Takeaway: High-dimensional mapping → linear separation.

Source: Lecture 4, slide 82

ML A42. [MATCH · Medium]

Match the SVM term to its meaning.

1. Margin
2. Slack ξ_i
3. C
4. SVR epsilon

A. distance band 2/‖w‖ around the separator
B. amount a point violates the margin
C. number of support vectors
D. penalty on margin violations
E. width of the no-penalty tube

Answer: 1 → A, 2 → B, 3 → D, 4 → E

Working:

- Margin → distance band 2/‖w‖ around the separator
- Slack ξ_i → amount a point violates the margin
- C → penalty on margin violations
- SVR epsilon → width of the no-penalty tube

Takeaway: Soft margin = margin + slack × C.

Source: Lecture 4, slide 69-85

ML A43. [BLANK · Easy]

In a random forest classifier, the final prediction is made by majority _____ of the trees.

Answer: vote
(Also accepted: voting, votes)

Takeaway: Regressor: average.

Source: Lecture 4, slide 44

ML A44. [TF · Hard]

The out-of-bag (OOB) score of a random forest uses training rows that were not in a tree's bootstrap sample to estimate performance.

Answer: True

Why: oob_score=True in the Lecture 4 template.

Takeaway: Free validation estimate.

Source: Lecture 4, slide 45

ML A45. [BLANK · Easy · calculation]

| Actual + | Actual −
Predicted + | TP = 40 | FP = 10
Predicted − | FN = 20 | TN = 130

200 test cases (Book 2 §4.3).

TP = 40, FP = 10, FN = 20, TN = 130. Accuracy = ___ (2 d.p.).

Answer: 0.85

Working:

- (40 + 130)/200 = 0.85

Takeaway: (TP + TN) / all.

Source: Book 2 (Exam examples), Q90, p. 18

ML A46. [BLANK · Easy · calculation]

| Actual + | Actual −
Predicted + | TP = 40 | FP = 10
Predicted − | FN = 20 | TN = 130

200 test cases (Book 2 §4.3).

TP = 40, FP = 10, FN = 20, TN = 130. Precision = ___ (2 d.p.).

Answer: 0.80

Working:

- 40/50 = 0.80

Takeaway: TP / (TP + FP).

Source: Book 2 (Exam examples), Q91, p. 18

ML A47. [BLANK · Easy · calculation]

| Actual + | Actual −
Predicted + | TP = 40 | FP = 10
Predicted − | FN = 20 | TN = 130

200 test cases (Book 2 §4.3).

TP = 40, FP = 10, FN = 20, TN = 130. Recall = ___ (2 d.p.).

Answer: 0.67

Working:

- 40/60 = 0.667

Takeaway: TP / (TP + FN).

Source: Book 2 (Exam examples), Q92, p. 18

ML A48. [BLANK · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 40 | FP = 10
Predicted − | FN = 20 | TN = 130

200 test cases (Book 2 §4.3).

TP = 40, FP = 10, FN = 20, TN = 130. F1 = ___ (2 d.p.).

Answer: 0.73

Working:

- 2(0.8)(0.667)/(0.8 + 0.667) = 1.067/1.467 = 0.727

Takeaway: Harmonic mean of P and R.

Source: Book 2 (Exam examples), Q93, p. 18

ML A49. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 40 | FP = 10
Predicted − | FN = 20 | TN = 130

200 test cases (Book 2 §4.3).

TP = 40, FP = 10, FN = 20, TN = 130. Specificity = ___ and FPR = ___ (2 d.p.).

(1) specificity = ____
(2) FPR = ____

Answer: specificity = 0.93; FPR = 0.07

Working:

- 130/140 = 0.929
- 10/140 = 0.071

Takeaway: TN/(TN+FP); FPR = 1 − specificity.

Source: Book 2 (Exam examples), Q94, p. 18

ML A50. [BLANK · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 40 | FP = 10
Predicted − | FN = 20 | TN = 130

200 test cases (Book 2 §4.3).

TP = 40, FP = 10, FN = 20, TN = 130. Balanced accuracy = ___ (2 d.p.).

Answer: 0.80

Working:

- (0.667 + 0.929)/2 = 0.798

Takeaway: (TPR + TNR)/2.

Source: Book 2 (Exam examples), Q95, p. 18

ML A51. [MCQ · Easy]

For cancer screening, missing a sick patient is the costliest error. Which metric should you prioritise?

A. precision
B. recall
C. accuracy
D. specificity

Answer: B. recall

Option notes:

- A: Precision guards against false positives.
- C: Misleading when classes are imbalanced.
- D: That is about negatives.

Takeaway: False negatives costly → recall.

Source: Book 2 (Exam examples), Q96, p. 18

ML A52. [MCQ · Easy]

For a spam filter, deleting a genuine email is the costliest error. Prioritise:

A. recall
B. R^2
C. precision
D. FPR maximisation

Answer: C. precision

Option notes:

- A: Recall guards against misses.
- B: A regression metric.
- D: You want FPR low.

Takeaway: False positives costly → precision.

Source: Book 2 (Exam examples), Q97, p. 18

ML A53. [TF · Easy]

With 99% negatives, a model that always predicts 'negative' has 99% accuracy and is useful.

Answer: False

Why: Accuracy misleads on imbalanced data: recall is 0.

Takeaway: Imbalance → use F1/recall/balanced accuracy.

Source: Book 2 (Exam examples), Q98, p. 18

ML A54. [MCQ · Easy]

An AUC of 0.5 means:

A. perfect
B. random guessing
C. exactly 50% accuracy
D. always wrong

Answer: B. random guessing

Option notes:

- A: AUC 1 is perfect.
- C: AUC is not accuracy.
- D: That is AUC 0.

Takeaway: Diagonal ROC.

Source: Book 2 (Exam examples), Q99, p. 18

ML A55. [TF · Medium]

AUC depends on the chosen threshold.

Answer: False

Why: It summarises all thresholds.

Takeaway: AUC is threshold-independent.

Source: Book 2 (Exam examples), Q100, p. 18

ML A56. [BLANKS · Medium]

AUC equals the probability that the model ranks a random ___ instance above a random ___ instance.

(1) first = ____
(2) second = ____

Answer: first = positive; second = negative

Takeaway: Ranking interpretation.

Source: Book 2 (Exam examples), Q101, p. 18

ML A57. [BLANK · Easy · calculation]

y | 3 | 5 | 7 | 9
ŷ | 2.5 | 5.5 | 7 | 10
y − ŷ | 0.5 | -0.5 | 0 | -1

Actual y = (3, 5, 7, 9), predicted ŷ = (2.5, 5.5, 7, 10). MAE = ___.

Answer: 0.5

Working:

- (0.5 + 0.5 + 0 + 1)/4 = 0.5

Takeaway: Mean of |errors|.

Source: Book 2 (Exam examples), Q102, p. 19

ML A58. [BLANK · Medium · calculation]

y | 3 | 5 | 7 | 9
ŷ | 2.5 | 5.5 | 7 | 10
y − ŷ | 0.5 | -0.5 | 0 | -1

Actual y = (3, 5, 7, 9), predicted ŷ = (2.5, 5.5, 7, 10). RMSE = ___ (3 d.p.).

Answer: 0.612

Working:

- (0.25 + 0.25 + 0 + 1)/4 = 0.375
- √0.375 = 0.612

Takeaway: Square, mean, root.

Source: Book 2 (Exam examples), Q103, p. 19

ML A59. [BLANK · Medium · calculation]

y | 3 | 5 | 7 | 9
ŷ | 2.5 | 5.5 | 7 | 10
y − ŷ | 0.5 | -0.5 | 0 | -1

Actual y = (3, 5, 7, 9), predicted ŷ = (2.5, 5.5, 7, 10). R^2 = ___ (3 d.p.).

Answer: 0.925

Working:

- ȳ = 6; SStot = 9 + 1 + 1 + 9 = 20
- SSres = 1.5
- 1 − 1.5/20 = 0.925

Takeaway: 1 − SSres/SStot.

Source: Book 2 (Exam examples), Q104, p. 19

ML A60. [BLANK · Medium · calculation]

y | 3 | 5 | 7 | 9
ŷ | 2.5 | 5.5 | 7 | 10
y − ŷ | 0.5 | -0.5 | 0 | -1

Actual y = (3, 5, 7, 9), predicted ŷ = (2.5, 5.5, 7, 10). MAPE = ___ % (1 d.p.).

Answer: 9.4
(Also accepted: 9.4%)

Working:

- (100/4)(0.5/3 + 0.5/5 + 0 + 1/9) = 25(0.1667 + 0.1 + 0.1111) = 9.44%

Takeaway: Mean of |error|/actual × 100.

Source: Book 2 (Exam examples), Q105, p. 19

ML A61. [BLANK · Easy · calculation]

Actual price 500,000, predicted 480,000. MAPE = ___ %.

Answer: 4
(Also accepted: 4%)

Takeaway: 20,000/500,000 = 4%.

Source: Book 2 (Exam examples), Q106, p. 19

ML A62. [TF · Easy]

RMSE penalises large errors more than MAE.

Answer: True

Why: Errors are squared first.

Takeaway: RMSE ≥ MAE.

Source: Book 2 (Exam examples), Q107, p. 19

ML A63. [TF · Medium]

R^2 can be negative.

Answer: True

Why: When the model is worse than predicting the mean.

Takeaway: $R^2 \in (-\infty, 1]$.

Source: Book 2 (Exam examples), Q108, p. 19

ML A64. [MCQ · Easy]

MAPE is unsuitable when:

A. errors are large
B. some actual values are zero or near zero
C. data are scaled
D. N is small

Answer: B. some actual values are zero or near zero

Option notes:

- A: MAPE still works.
- C: MAPE is scale-independent.
- D: Not the issue.

Takeaway: Division by y.

Source: Book 2 (Exam examples), Q109, p. 19

ML A65. [BLANK · Medium · calculation]

Data (1, 3), (2, 5), (3, 7); model h(x) = w1·x + w0 with w0 = 0, w1 = 1. Squared loss Σ(y − h)² = ___.

Answer: 29

Working:

- Predictions 1, 2, 3
- Errors 2, 3, 4
- 4 + 9 + 16 = 29

Takeaway: Errors 2, 3, 4.

Source: Book 2 (Exam examples), Q110, p. 19

ML A66. [BLANKS · Hard · calculation]

Same data and model (w0 = 0, w1 = 1). ∂Loss/∂w0 = −2Σ(y − h) = ___; ∂Loss/∂w1 = −2Σ(y − h)x = ___.

(1) ∂L/∂w0 = _____
(2) ∂L/∂w1 = _____

Answer: ∂L/∂w0 = -18; ∂L/∂w1 = -40

Working:

- −2(2 + 3 + 4) = −18
- −2(2·1 + 3·2 + 4·3) = −40

Takeaway: Keep the minus sign.

Source: Book 2 (Exam examples), Q111, p. 19

ML A67. [BLANKS · Hard · calculation]

One gradient-descent step with α = 0.05 (gradients −18 and −40, w0 = 0, w1 = 1) gives w0 = ___, w1 = ___.

(1) w0 = _____
(2) w1 = _____

Answer: w0 = 0.9; w1 = 3.0

Working:

- 0 − 0.05(−18) = 0.9
- 1 − 0.05(−40) = 3.0

Takeaway: w ← w − α·gradient. New loss 12.83: it went down.

Source: Book 2 (Exam examples), Q112, p. 19

ML A68. [BLANKS · Hard · calculation]

Logistic regression, w = (0.5, −1, 2) (first weight is the bias), x = (1, 1, 0.5), y = 1. z = w·x = ___; h = ___; cross-entropy loss = ___ (3 d.p., natural log).

(1) z = _____
(2) h = _____
(3) loss = _____

Answer: z = 0.5; h = 0.622; loss = 0.474

Working:

- z = 0.5 − 1 + 1 = 0.5
- h = 1/(1 + e^−0.5) = 0.622

- $-\ln 0.622 = 0.474$

Takeaway: Loss = $-\ln h$ when $y = 1$.

Source: Book 2 (Exam examples), Q113, p. 19

ML A69. [BLANK · Medium · calculation]

Same logistic example ($h = 0.622$, $y = 1$). Gradient for the bias weight $(h - y) \cdot x0$ = ___ (3 d.p.).

Answer: -0.378

Takeaway: $x0 = 1$.

Source: Book 2 (Exam examples), Q114, p. 19

ML A70. [MCQ · Easy]

Which loss is used for logistic regression in the slides?

A. binary cross-entropy
B. squared error
C. MAE
D. hinge

Answer: A. binary cross-entropy

Option notes:

- B: Used for linear regression.
- C: A regression metric.
- D: SVM loss (Extra, not in your slides).

Takeaway: BCE.

Source: Book 2 (Exam examples), Q115, p. 19

ML A71. [TF · Easy]

Linear-regression squared loss is convex, so gradient descent can find its minimum.

Answer: True

Why: Bowl shape, one minimum.

Takeaway: Convex loss.

Source: Book 2 (Exam examples), Q116, p. 20

ML A72. [BLANK · Easy]

In $w_i \leftarrow w_i - \alpha \, \partial L/\partial w_i$, α is called the ___.

Answer: learning rate
(Also accepted: step size, learning-rate)

Takeaway: Step size.

Source: Book 2 (Exam examples), Q117, p. 20

ML A73. [BLANKS · Medium · calculation]

Point	x_1	x_2	label
A	1	1	+
B	3	1	−

C | 2 | 4 | +
D | 4 | 4 | −
E | 1 | 3 | +
query | 2 | 2 | ?

Query (2, 2). Euclidean distances (3 d.p.) to A(1,1), B(3,1), C(2,4), D(4,4), E(1,3):

(1) A = ____
(2) B = ____
(3) C = ____
(4) D = ____
(5) E = ____

Answer: A = 1.414; B = 1.414; C = 2.000; D = 2.828; E = 1.414

Takeaway: $\sqrt{(\Delta x^2 + \Delta y^2)}$.

Source: Book 2 (Exam examples), Q118, p. 20

ML A74. [MCQ · Medium · calculation]

Point | x_1 | x_2 | label
A | 1 | 1 | +
B | 3 | 1 | −
C | 2 | 4 | +
D | 4 | 4 | −
E | 1 | 3 | +
query | 2 | 2 | ?

Query (2, 2), k = 3 (Euclidean), points A(1,1)+, B(3,1)−, C(2,4)+, D(4,4)−, E(1,3)+. The prediction is:

A. + (A+, B−, E+)
B. cannot decide
C. − (B−, D−, C+)
D. tie

Answer: A. + (A+, B−, E+)

Option notes:

- B: Majority of A, B, E.
- C: D and C are not among the 3 nearest.
- D: k = 3 cannot tie with two classes.

Takeaway: Nearest three: A, B, E.

Source: Book 2 (Exam examples), Q119, p. 20

ML A75. [BLANK · Easy · calculation]

Manhattan distance from the query (2, 2) to D(4, 4) is ___.

Answer: 4

Takeaway: $|4 - 2| + |4 - 2|$.

Source: Book 2 (Exam examples), Q120, p. 20

ML A76. [TF · Easy]

kNN builds an explicit model during training.

Answer: False

Why: It stores the data (instance-based).

Takeaway: Lazy learner.

Source: Book 2 (Exam examples), Q121, p. 20

ML A77. [TF · Easy]

Features should be scaled before kNN.

Answer: True

Why: It is distance-based.

Takeaway: Scale first.

Source: Book 2 (Exam examples), Q122, p. 20

ML A78. [BLANKS · Medium · calculation]

Point | x_1 | x_2 | y
P1 | 3 | 3 | +1
P2 | 4 | 3 | +1
P3 | 1 | 1 | −1

Lecture 4 slide 71: separator w = (0.5, 0.5), b = −2.

SVM with w = (0.5, 0.5), b = −2. The functional margin of P2 = (4, 3), y = +1, is ___; its geometric margin is ___ (3 d.p.).

(1) functional = ____
(2) geometric = ____

Answer: functional = 1.5; geometric = 2.121

Working:

- 0.5(4) + 0.5(3) − 2 = 1.5
- ‖w‖ = 0.7071
- 1.5/0.7071 = 2.121

Takeaway: Geometric = functional/‖w‖.

Source: Book 2 (Exam examples), Q123, p. 20

ML A79. [MCQ · Medium]

Point | x_1 | x_2 | y
P1 | 3 | 3 | +1
P2 | 4 | 3 | +1
P3 | 1 | 1 | −1

Lecture 4 slide 71: separator w = (0.5, 0.5), b = −2.

SVM w = (0.5, 0.5), b = −2 with P1 (3,3)+, P2 (4,3)+, P3 (1,1)−. The support vectors are:

A. all three
B. P1 and P2
C. P1 and P3
D. P2 only

Answer: C. P1 and P3

Option notes:

- A: Only functional margin exactly 1.

- B: P2 is not on the margin.
- D: P2 has functional margin 1.5.

Takeaway: Support vectors sit on w·x + b = ±1.

Source: Book 2 (Exam examples), Q124, p. 20

ML A80. [BLANK · Medium · calculation]

The margin width 2/‖w‖ of the SVM with w = (0.5, 0.5) is ___ (3 d.p.).

Answer: 2.828

Takeaway: 2/0.7071.

Source: Book 2 (Exam examples), Q125, p. 20

ML A81. [MCQ · Medium]

In a soft-margin SVM, increasing C:

A. penalises violations more
B. always widens the margin
C. allows more violations
D. changes the kernel

Answer: A. penalises violations more

Option notes:

- B: Large C narrows it.
- C: That is smaller C.
- D: C is not the kernel.

Takeaway: Large C → fewer errors tolerated.

Source: Book 2 (Exam examples), Q126, p. 20

ML A82. [MCQ · Medium]

The kernel trick:

A. scales features
B. is only for linear data
C. computes K(x, z) instead of explicitly mapping to a high-dimensional space
D. removes support vectors

Answer: C. computes K(x, z) instead of explicitly mapping to a high-dimensional space

Option notes:

- A: That is preprocessing.
- B: It is for non-linear data.
- D: No.

Takeaway: Replace x·z by K(x, z).

Source: Book 2 (Exam examples), Q127, p. 20

ML A83. [MULTI · Easy]

Which are regression problems? (Select ALL that apply.)

A. AUD/USD exchange rate
B. house price

C. COVID positive/negative
D. Iris species
E. tomorrow's temperature

Answer: A. AUD/USD exchange rate; B. house price; E. tomorrow's temperature

Option notes:

- A: Correct: number
- B: Correct: number
- C: Incorrect: category
- D: Incorrect: category
- E: Correct: number

Takeaway: Regression = continuous output.

Source: Book 2 (Exam examples), Q128, p. 20

ML A84. [BLANKS · Easy]

The i.i.d. assumption means examples are ___ and ___.

(1) first = _____
(2) second = _____

Answer: first = independent; second = identically distributed

Takeaway: Stationary assumption.

Source: Book 2 (Exam examples), Q129, p. 21

ML A85. [BLANK · Easy]

Ockham's razor: choose the ___ hypothesis consistent with the training data.

Answer: simplest
(Also accepted: simple)

Takeaway: Simplest consistent hypothesis.

Source: Book 2 (Exam examples), Q130, p. 21

ML A86. [TF · Medium]

Hyperparameters should be tuned on the test set.

Answer: False

Why: Tune with k-fold CV on the training set; test once at the end.

Takeaway: Avoid leakage.

Source: Book 2 (Exam examples), Q131, p. 21

ML A87. [MATCH · Easy]

Match the ensemble.

1. Random forest
2. Gradient boosting

A. one deep tree with no pruning
B. trees built sequentially, each fixing remaining error
C. bootstrap samples, random feature subsets, majority vote

Answer: 1 → C, 2 → B

Working:

- Random forest → bootstrap samples, random feature subsets, majority vote
- Gradient boosting → trees built sequentially, each fixing remaining error

Takeaway: Bagging vs boosting.

Source: Book 2 (Exam examples), Q132, p. 21

ML A88. [MCQ · Medium]

Random forest mainly reduces:

A. number of features
B. bias
C. training time
D. variance

Answer: D. variance

Option notes:

- A: It samples features, but the goal is variance.
- B: Boosting targets bias.
- C: It trains many trees.

Takeaway: Averaging decorrelated trees.

Source: Book 2 (Exam examples), Q133, p. 21

ML A89. [MULTI · Easy]

The five supervised techniques in Module 3: (Select ALL that apply.)

A. ensembles/random forests
B. kNN
C. k-means
D. SVM
E. decision trees
F. linear and logistic regression

Answer: A. ensembles/random forests; B. kNN; D. SVM; E. decision trees; F. linear and logistic regression

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: unsupervised clustering
- D: Correct:
- E: Correct:
- F: Correct:

Takeaway: Trees, ensembles, regression, kNN, SVM.

Source: Book 2 (Exam examples), Q134, p. 21

ML A90. [BLANKS · Medium · calculation]

TP = 30, FP = 20, FN = 10. Precision ___, recall ___, F1 ___ (3 d.p.).

(1) precision = ____
(2) recall = ____

(3) F1 = ____

Answer: precision = 0.600; recall = 0.750; F1 = 0.667

Working:

- 30/50 = 0.600
- 30/40 = 0.750
- 2(0.6)(0.75)/1.35 = 0.667

Takeaway: F1 = 2PR/(P + R).

Source: Book 2 (Exam examples), Mixed practice paper Q9, p. 54

ML A91. [BLANKS · Medium · calculation]

Errors $y - \hat{y} = (2, -2, 1, -1)$. RMSE ___ (3 d.p.); MAE ___.

(1) RMSE = ____
(2) MAE = ____

Answer: RMSE = 1.581; MAE = 1.5

Takeaway: √(10/4); 6/4.

Source: Book 2 (Exam examples), Mixed practice paper Q10, p. 54

ML A92. [MCQ · Easy]

Which metric is threshold-independent?

A. AUC
B. recall
C. precision
D. accuracy

Answer: A. AUC

Option notes:

- B: Depends on the threshold.
- C: Depends on the threshold.
- D: Depends on the threshold.

Takeaway: AUC summarises all thresholds.

Source: Book 2 (Exam examples), Mixed practice paper Q11, p. 54

ML A93. [BLANKS · Medium · calculation]

TP = 30, FP = 20, FN = 10, TN = 40. Precision, recall, F1 (3 d.p.)?

(1) precision = ____
(2) recall = ____
(3) F1 = ____

Answer: precision = 0.600; recall = 0.750; F1 = 0.667

Takeaway: F1 = 0.9/1.35.

Source: Book 3 (AI from zero), Practice 11.1, p. 47

ML A94. [MCQ · Medium]

AUC = 0.3 means:

A. 30% accuracy
B. perfect
C. worse than random: predictions are inverted
D. fair

Answer: C. worse than random: predictions are inverted

Option notes:

- A: AUC is not accuracy.
- B: 1 is perfect.
- D: Fair is about 0.6.

Takeaway: Flip its answers.

Source: Book 3 (AI from zero), Practice 11.3, p. 47

ML A95. [MCQ · Easy]

Where is the test set used in the 10-step workflow?

A. in every CV fold
B. to pick hyperparameters
C. to scale features
D. once, at the end

Answer: D. once, at the end

Option notes:

- A: CV uses the training set.
- B: That is leakage.
- C: Also leakage.

Takeaway: Test once.

Source: Book 3 (AI from zero), Practice 11.5, p. 47

ML A96. [TF · Hard · calculation]

With precision 1 and recall 0.01, F1 ≈ 0.505.

Answer: False

Why: F1 = 2(1)(0.01)/1.01 = 0.0198; the plain average would be 0.505.

Takeaway: Harmonic mean stays low unless both are good.

Source: Book 3 (AI from zero), Chapter 11, p. 43

ML A97. [MATCH · Medium]

Slide ROC threshold table: match each threshold to its (FPR, TPR).

1. 0.9
2. 0.5
3. 0.1

A. (0.10, 0.60)
B. (0.01, 0.10)
C. (0.50, 0.50)
D. (0.60, 0.95)

Answer: 1 → B, 2 → A, 3 → D

Working:

- 0.9 → (0.01, 0.10)
- 0.5 → (0.10, 0.60)
- 0.1 → (0.60, 0.95)

Takeaway: Lower threshold: more positives caught, more false alarms.

Source: Book 1 (Compiled lecture notes), Chapter 4, p. 22

ML A98. [MCQ · Easy]

Which is a hyperparameter (chosen before training)?

A. the bias of a neuron
B. a support vector's α
C. k in kNN
D. a learned weight w_1

Answer: C. k in kNN

Option notes:

- A: Learned.
- B: Learned by the solver.
- D: Parameters are learned.

Takeaway: Depth, k, learning rate, C are hyperparameters.

Source: Book 3 (AI from zero), Chapter 11, p. 43

ML A99. [BLANKS · Easy · calculation]

h(x) = 2x + 1. Predict x = 4; residual if y = 10?

(1) $\hat{y}$ = ____
(2) residual = ____

Answer: $\hat{y}$ = 9; residual = 1

Takeaway: residual = y − $\hat{y}$.

Source: Book 3 (AI from zero), Practice 13.1, p. 55

ML A100. [BLANK · Easy · calculation]

z = 0. Logistic output = ___.

Answer: 0.5

Takeaway: σ(0).

Source: Book 3 (AI from zero), Practice 13.2, p. 55

ML A101. [BLANK · Medium · calculation]

h = 0.8, y = 0. Cross-entropy (natural log) = ___ (3 d.p.).

Answer: 1.609

Takeaway: −ln(1 − 0.8).

Source: Book 3 (AI from zero), Practice 13.3, p. 55

ML A102. [BLANK · Medium · calculation]

h = 0.8, y = 0, x_1 = 2, α = 0.1, w_1 = 1. New w_1 = ___.

Answer: 0.84

Takeaway: w − α(h − y)x.

Source: Book 3 (AI from zero), Practice 13.4, p. 55

ML A103. [BLANKS · Hard · calculation]

Logistic step: w = (0.5, −1, 2), x = (1, 1, 0.5), y = 1, h = 0.622, α = 1. New weights (3 d.p.)?

(1) w0 = ____
(2) w1 = ____
(3) w2 = ____

Answer: w0 = 0.878; w1 = -0.622; w2 = 2.189

Takeaway: Gradient (h − y)x = (−0.378, −0.378, −0.189).

Source: Book 3 (AI from zero), Chapter 13, p. 52

ML A104. [MCQ · Medium]

Why is the linear-regression loss squared?

A. because MAE is undefined
B. to make it a probability
C. so gradients are zero
D. errors become positive, big errors count more, and the loss is a smooth convex bowl

Answer: D. errors become positive, big errors count more, and the loss is a smooth convex bowl

Option notes:

- A: MAE is defined.
- B: It is not.
- C: No.

Takeaway: Convex = one minimum.

Source: Book 3 (AI from zero), Chapter 13, p. 52

ML A105. [BLANKS · Easy · calculation]

Manhattan distance (1, 5) to (4, 1) = ___; Euclidean = ___.

(1) Manhattan = ____
(2) Euclidean = ____

Answer: Manhattan = 7; Euclidean = 5

Takeaway: 3 + 4; √(9 + 16).

Source: Book 3 (AI from zero), Practice 14.1, p. 59

ML A106. [BLANKS · Medium · calculation]

w = (1, −1), b = 0, point (2, 5) with label −1. Functional and geometric margin (3 d.p.)?

(1) functional = ____
(2) geometric = ____

Answer: functional = 3; geometric = 2.121

Takeaway: −1(2 − 5) = 3; 3/√2.

Source: Book 3 (AI from zero), Practice 14.2, p. 59

ML A107. [BLANK · Easy · calculation]

‖w‖ = 0.5. Street (margin) width = ___.

Answer: 4

Takeaway: 2/‖w‖.

Source: Book 3 (AI from zero), Practice 14.3, p. 59

ML A108. [MCQ · Medium]

Effect of increasing C in a soft-margin SVM:

A. fewer training errors allowed, narrower margin
B. wider margin
C. switches to RBF
D. more support vectors always

Answer: A. fewer training errors allowed, narrower margin

Option notes:

- B: Smaller C widens.
- C: C is not the kernel.
- D: Not necessarily.

Takeaway: Large C = strict.

Source: Book 3 (AI from zero), Practice 14.4, p. 59

ML A109. [BLANK · Medium · calculation]

Point | x_1 | x_2 | y
P1 | 3 | 3 | +1
P2 | 4 | 3 | +1
P3 | 1 | 1 | −1

Lecture 4 slide 71: separator w = (0.5, 0.5), b = −2.

SVM w = (0.5, 0.5), b = −2. New point (2, 3): w·x + b = ___ → class +1.

Answer: 0.5

Takeaway: 1 + 1.5 − 2.

Source: Book 3 (AI from zero), Chapter 14, p. 56

ML A110. [MCQ · Hard]

1-D points −2, −1 (+), 0 (−), 1, 2 (+). Which mapping makes them linearly separable?

A. x ↦ 2x
B. x ↦ x + 1
C. no mapping can
D. x ↦ (x, x^2), separated by x^2 = 0.5

Answer: D. x ↦ (x, x^2), separated by x^2 = 0.5

Option notes:

- A: Still 1-D, still not separable.

- B: A shift does not help.
- C: x^2 works.

Takeaway: Kernels bend the street.

Source: Book 3 (AI from zero), Chapter 14, p. 56

ML A111. [MCQ · Easy]

For two classes, why choose an odd k in kNN?

A. the distance needs it
B. odd k is faster
C. to avoid tied votes
D. even k overfits

Answer: C. to avoid tied votes

Option notes:

- A: No.
- B: No.
- D: Not the reason.

Takeaway: Odd k, no ties.

Source: Book 3 (AI from zero), Chapter 14, p. 56

ML A112. [MCQ · Easy]

Is Random Forest bagging or boosting?

A. stacking
B. boosting
C. neither
D. bagging (plus random feature subsets)

Answer: D. bagging (plus random feature subsets)

Option notes:

- A: No learned combiner.
- B: Trees are independent.
- C: It is bagging.

Takeaway: Bootstrap + vote.

Source: Book 3 (AI from zero), Practice 15.1, p. 61

ML A113. [MCQ · Easy]

Which builds models sequentially?

A. kNN
B. bagging
C. random forest
D. boosting

Answer: D. boosting

Option notes:

- A: Not an ensemble.
- B: Parallel.

- C: Parallel.

Takeaway: Each fixes the last.

Source: Book 3 (AI from zero), Practice 15.2, p. 61

ML A114. [BLANK · Easy · calculation]

Three bagged regressors output 4, 6, 8. Prediction = ___.

Answer: 6

Takeaway: Average.

Source: Book 3 (AI from zero), Practice 15.3, p. 61

ML A115. [BLANKS · Medium · calculation]

Gradient boosting by hand (learning rate 1): true y = 10. Model 1 predicts 7; tree 2 predicts the residual as 2.5; tree 3 predicts 0.4. Prediction after tree 2 = ___; after tree 3 = ___.

(1) after tree 2 = ____
(2) after tree 3 = ____

Answer: after tree 2 = 9.5; after tree 3 = 9.9

Takeaway: Each tree fits the remaining residual.

Source: Book 3 (AI from zero), Chapter 15, p. 60

ML A116. [BLANK · Hard · calculation]

A bootstrap sample of N from N examples leaves out about ___ % of them (the out-of-bag examples).

Answer: 37
(Also accepted: 36.8, 37%)

Takeaway: $(1 - 1/N)^N \to e^{-1} \approx 0.368$.

Source: Book 3 (AI from zero), Chapter 15, p. 60

ML A117. [BLANK · Easy]

Five trees vote Yes, No, Yes, Yes, No. Random-forest prediction: ___.

Answer: Yes

Takeaway: Majority 3–2.

Source: Book 3 (AI from zero), Chapter 15, p. 60

ML A118. [BLANK · Hard · calculation]

(Extra, not in your slides) Five independent classifiers each err with probability 0.1. A majority vote errs only when ≥ 3 are wrong: probability ≈ ___ (4 d.p.).

Answer: 0.0086
(Also accepted: 0.00856)

Working:

- $C(5,3)(0.1)^3(0.9)^2 = 0.0081$
- $C(5,4)(0.1)^4(0.9) = 0.00045$
- $(0.1)^5 = 0.00001$
- Total 0.00856

Takeaway: Ensembles help when errors are independent.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: ensemble learning

ML A119. [BLANKS · Hard · calculation]

(Extra, not in your slides) kNN in a unit hypercube with N = 1,000,000 points and k = 10: the side ℓ of a neighbourhood containing k points is (k/N)^(1/n). For n = 2, $\ell \approx$ ___; for n = 200, $\ell \approx$ ___ (3 d.p.).

(1) n = 2 = ____
(2) n = 200 = ____

Answer: n = 2 = 0.003; n = 200 = 0.944

Takeaway: Curse of dimensionality: in high dimensions 'nearest' neighbours are far away.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: nearest neighbours

ML A120. [MCQ · Easy]

AIMA's three-way split uses the validation set to:

A. report the final accuracy
B. choose hyperparameters/models, keeping the test set untouched for the final estimate
C. train the weights
D. replace cross-validation entirely

Answer: B. choose hyperparameters/models, keeping the test set untouched for the final estimate

Option notes:

- A: That is the test set.
- C: That is the training set.
- D: k-fold CV is an alternative when data are scarce.

Takeaway: Peeking at the test set leaks.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: model selection

ML A121. [BLANK · Medium]

(Extra, not in your slides) Leave-one-out cross-validation is k-fold cross-validation with k = ___ (N examples).

Answer: N
(Also accepted: n)

Takeaway: Each example is the validation set once.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: cross-validation

ML A122. [MCQ · Medium]

(Extra, not in your slides) Regularisation adds a complexity penalty to the loss: Cost(h) = EmpLoss(h) + λ·Complexity(h). Larger λ gives:

A. simpler hypotheses (less overfitting, more bias)
B. zero training loss
C. more complex hypotheses
D. no change

Answer: A. simpler hypotheses (less overfitting, more bias)

Option notes:

- B: No.
- C: Opposite.
- D: λ weights the penalty.

Takeaway: L1 (lasso) also drives weights to exactly 0.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: regularisation

ML A123. [TF · Medium]

The perceptron learning rule converges to a perfect separator when the data are linearly separable.

Answer: True

Why: Perceptron convergence theorem.

Takeaway: Not separable → it oscillates.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: linear classifiers

ML A124. [MCQ · Medium]

Why replace the hard threshold with the logistic function?

A. it never misclassifies
B. it removes the need for weights
C. it is faster to compute
D. it is differentiable everywhere, so gradient descent works and outputs read as probabilities

Answer: D. it is differentiable everywhere, so gradient descent works and outputs read as probabilities

Option notes:

- A: It can.
- B: No.
- C: Not the reason.

Takeaway: Soft threshold.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: logistic regression

ML A125. [BLANK · Hard · calculation]

Univariate linear regression, L2 loss on one example (x = 2, y = 5) with $w_0 = 0$, $w_1 = 1$, $\alpha = 0.1$. The AIMA update $w_1 \leftarrow w_1 + \alpha(y - h)x$ gives w_1 = ___.

Answer: 1.6

Takeaway: h = 2, error 3, 1 + 0.1 × 3 × 2.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: linear regression

ML A126. [MULTI · Medium]

Properties of SVMs highlighted by AIMA: (Select ALL that apply.)

A. only support vectors determine the separator
B. they construct a maximum-margin separator
C. the kernel trick gives non-linear boundaries
D. they need gradient descent through a deep network

Answer: A. only support vectors determine the separator; B. they construct a maximum-margin separator; C. the kernel trick gives non-linear boundaries

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Incorrect: they solve a quadratic programme

Takeaway: Three SVM properties.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 19: SVM

Part B: calculation drills (192)

ML B1. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 11 | 12 | -1
2 | 9 | 10 | -1
3 | 9 | 12 | -3

Compute the MSE of these predictions.

A. 3.667
B. 8.333
C. 1.667
D. 11.000

Answer: A. 3.667

Working:

- Errors y − ŷ: -1, -1, -3.
- Squares: 1, 1, 9; sum = 11; MSE = sum / 3 = 3.667.

Option notes:

- B: Squared the sum instead of summing squares.
- C: That is MAE.
- D: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B2. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 9 | 7 | 2
2 | 12 | 14 | -2
3 | 12 | 11 | 1
4 | 1 | 2 | -1

Compute the MSE of these predictions.

A. 10.000
B. 0.000
C. 2.500
D. 1.500

Answer: C. 2.500

Working:

- Errors y − ŷ: 2, -2, 1, -1.
- Squares: 4, 4, 1, 1; sum = 10; MSE = sum / 4 = 2.500.

Option notes:

- A: Forgot to divide by n.
- B: Squared the sum instead of summing squares.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B3. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 6 | -2
2 | 9 | 12 | -3
3 | 4 | 4 | 0
4 | 9 | 10 | -1
5 | 12 | 10 | 2

Compute the MSE of these predictions.

A. 3.600
B. 18.000
C. 3.200
D. 1.600

Answer: A. 3.600

Working:

- Errors y − ŷ: -2, -3, 0, -1, 2.
- Squares: 4, 9, 0, 1, 4; sum = 18; MSE = sum / 5 = 3.600.

Option notes:

- B: Forgot to divide by n.
- C: Squared the sum instead of summing squares.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B4. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 2 | -1 | 3
2 | 11 | 8 | 3
3 | 11 | 10 | 1
4 | 4 | 4 | 0

Compute the MSE of these predictions.

A. 19.000
B. 4.750
C. 1.750
D. 12.250

Answer: B. 4.750

Working:

- Errors y − ŷ: 3, 3, 1, 0.
- Squares: 9, 9, 1, 0; sum = 19; MSE = sum / 4 = 4.750.

Option notes:

- A: Forgot to divide by n.
- C: That is MAE.
- D: Squared the sum instead of summing squares.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B5. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 3 | 5 | -2
2 | 9 | 12 | -3
3 | 6 | 6 | 0
4 | 12 | 12 | 0

Compute the MSE of these predictions.

A. 1.250
B. 3.250
C. 6.250
D. 13.000

Answer: B. 3.250

Working:

- Errors y − ŷ: -2, -3, 0, 0.
- Squares: 4, 9, 0, 0; sum = 13; MSE = sum / 4 = 3.250.

Option notes:

- A: That is MAE.
- C: Squared the sum instead of summing squares.
- D: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B6. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | 0 | 1
2 | 5 | 4 | 1
3 | 3 | 1 | 2

Compute the MSE of these predictions.

A. 6.000
B. 2.000

C. 5.333
D. 1.333

Answer: B. 2.000

Working:

- Errors $y - \hat{y}$: 1, 1, 2.
- Squares: 1, 1, 4; sum = 6; MSE = sum / 3 = 2.000.

Option notes:

- A: Forgot to divide by n.
- C: Squared the sum instead of summing squares.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B7. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 3 | 1 | 2
2 | 4 | 2 | 2
3 | 7 | 8 | -1

Compute the MSE of these predictions.

A. 3.450
B. 1.667
C. 9.000
D. 3.000

Answer: D. 3.000

Working:

- Errors $y - \hat{y}$: 2, 2, -1.
- Squares: 4, 4, 1; sum = 9; MSE = sum / 3 = 3.000.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: That is MAE.
- C: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B8. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 4 | 4 | 0
2 | 5 | 4 | 1
3 | 11 | 11 | 0

Compute the MSE of these predictions.

A. 0.383
B. 0.283

C. 1.000
D. 0.333

Answer: D. 0.333

Working:

- Errors $y - \hat{y}$: 0, 1, 0.
- Squares: 0, 1, 0; sum = 1; MSE = sum / 3 = 0.333.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- C: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B9. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 11 | 12 | -1
2 | 2 | 0 | 2
3 | 8 | 11 | -3

Compute the MSE of these predictions.

A. 1.333
B. 2.000
C. 14.000
D. 4.667

Answer: D. 4.667

Working:

- Errors $y - \hat{y}$: -1, 2, -3.
- Squares: 1, 4, 9; sum = 14; MSE = sum / 3 = 4.667.

Option notes:

- A: Squared the sum instead of summing squares.
- B: That is MAE.
- C: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B10. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 12 | 12 | 0
2 | 11 | 11 | 0
3 | 1 | 4 | -3
4 | 1 | 1 | 0

Compute the MSE of these predictions.

A. 0.750
B. 2.588
C. 9.000
D. 2.250

Answer: D. 2.250

Working:

- Errors y − ŷ: 0, 0, -3, 0.
- Squares: 0, 0, 9, 0; sum = 9; MSE = sum / 4 = 2.250.

Option notes:

- A: That is MAE.
- B: Arithmetic slip: recompute each step carefully.
- C: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B11. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 3 | 2
2 | 3 | 5 | -2
3 | 11 | 14 | -3
4 | 10 | 13 | -3
5 | 4 | 4 | 0

Compute the MSE of these predictions.

A. 5.200
B. 2.000
C. 26.000
D. 7.200

Answer: A. 5.200

Working:

- Errors y − ŷ: 2, -2, -3, -3, 0.
- Squares: 4, 4, 9, 9, 0; sum = 26; MSE = sum / 5 = 5.200.

Option notes:

- B: That is MAE.
- C: Forgot to divide by n.
- D: Squared the sum instead of summing squares.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B12. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | -1 | 2

2 | 1 | 2 | -1
3 | 11 | 11 | 0

Compute the MSE of these predictions.

A. 5.000
B. 1.000
C. 1.667
D. 0.333

Answer: C. 1.667

Working:

- Errors $y - \hat{y}$: 2, -1, 0.
- Squares: 4, 1, 0; sum = 5; MSE = sum / 3 = 1.667.

Option notes:

- A: Forgot to divide by n.
- B: That is MAE.
- D: Squared the sum instead of summing squares.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B13. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 5 | 8 | -3
2 | 1 | -1 | 2
3 | 10 | 8 | 2

Compute the MSE of these predictions.

A. 5.667
B. 17.000
C. 0.333
D. 2.333

Answer: A. 5.667

Working:

- Errors $y - \hat{y}$: -3, 2, 2.
- Squares: 9, 4, 4; sum = 17; MSE = sum / 3 = 5.667.

Option notes:

- B: Forgot to divide by n.
- C: Squared the sum instead of summing squares.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B14. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 10 | 8 | 2

2 | 11 | 11 | 0
3 | 1 | 4 | -3
4 | 1 | -1 | 2

Compute the MSE of these predictions.

A. 0.250
B. 17.000
C. 4.250
D. 1.750

Answer: C. 4.250

Working:

- Errors $y - \hat{y}$: 2, 0, -3, 2.
- Squares: 4, 0, 9, 4; sum = 17; MSE = sum / 4 = 4.250.

Option notes:

- A: Squared the sum instead of summing squares.
- B: Forgot to divide by n.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B15. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 11 | 13 | -2
2 | 9 | 10 | -1
3 | 6 | 3 | 3

Compute the MSE of these predictions.

A. 0.000
B. 14.000
C. 4.667
D. 2.000

Answer: C. 4.667

Working:

- Errors $y - \hat{y}$: -2, -1, 3.
- Squares: 4, 1, 9; sum = 14; MSE = sum / 3 = 4.667.

Option notes:

- A: Squared the sum instead of summing squares.
- B: Forgot to divide by n.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B16. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | 0 | 1
2 | 5 | 6 | -1
3 | 12 | 11 | 1

Compute the MSE of these predictions.

A. 3.000
B. 1.000
C. 1.150
D. 0.333

Answer: B. 1.000

Working:

- Errors y − ŷ: 1, -1, 1.
- Squares: 1, 1, 1; sum = 3; MSE = sum / 3 = 1.000.

Option notes:

- A: Forgot to divide by n.
- C: Arithmetic slip: recompute each step carefully.
- D: Squared the sum instead of summing squares.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B17. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 11 | 12 | -1
2 | 9 | 10 | -1
3 | 9 | 12 | -3

Compute the MAE of these predictions.

A. 1.667
B. 1.917
C. 5.000
D. 3.667

Answer: A. 1.667

Working:

- Errors y − ŷ: -1, -1, -3.
- |errors|: 1, 1, 3; sum = 5; MAE = sum / 3 = 1.667.

Option notes:

- B: Arithmetic slip: recompute each step carefully.
- C: Forgot to divide by n.
- D: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B18. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 9 | 7 | 2
2 | 12 | 14 | -2
3 | 12 | 11 | 1
4 | 1 | 2 | -1

Compute the MAE of these predictions.

A. 0.000
B. 6.000
C. 1.500
D. 2.500

Answer: C. 1.500

Working:

- Errors y − ŷ: 2, -2, 1, -1.
- |errors|: 2, 2, 1, 1; sum = 6; MAE = sum / 4 = 1.500.

Option notes:

- A: Signed errors cancel; take |y − ŷ| first.
- B: Forgot to divide by n.
- D: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B19. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 6 | -2
2 | 9 | 12 | -3
3 | 4 | 4 | 0
4 | 9 | 10 | -1
5 | 12 | 10 | 2

Compute the MAE of these predictions.

A. 1.600
B. 0.800
C. 8.000
D. 3.600

Answer: A. 1.600

Working:

- Errors y − ŷ: -2, -3, 0, -1, 2.
- |errors|: 2, 3, 0, 1, 2; sum = 8; MAE = sum / 5 = 1.600.

Option notes:

- B: Signed errors cancel; take |y − ŷ| first.
- C: Forgot to divide by n.
- D: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B20. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 2 | -1 | 3
2 | 11 | 8 | 3
3 | 11 | 10 | 1
4 | 4 | 4 | 0

Compute the MAE of these predictions.

A. 4.750
B. 1.750
C. 7.000
D. 2.013

Answer: B. 1.750

Working:

- Errors y − ŷ: 3, 3, 1, 0.
- |errors|: 3, 3, 1, 0; sum = 7; MAE = sum / 4 = 1.750.

Option notes:

- A: That is MSE.
- C: Forgot to divide by n.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B21. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 3 | 5 | -2
2 | 9 | 12 | -3
3 | 6 | 6 | 0
4 | 12 | 12 | 0

Compute the MAE of these predictions.

A. 5.000
B. 1.250
C. 1.438
D. 3.250

Answer: B. 1.250

Working:

- Errors y − ŷ: -2, -3, 0, 0.
- |errors|: 2, 3, 0, 0; sum = 5; MAE = sum / 4 = 1.250.

Option notes:

- A: Forgot to divide by n.

- C: Arithmetic slip: recompute each step carefully.
- D: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B22. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | 0 | 1
2 | 5 | 4 | 1
3 | 3 | 1 | 2

Compute the MAE of these predictions.

A. 2.000
B. 1.333
C. 1.533
D. 4.000

Answer: B. 1.333

Working:

- Errors y − ŷ: 1, 1, 2.
- |errors|: 1, 1, 2; sum = 4; MAE = sum / 3 = 1.333.

Option notes:

- A: That is MSE.
- C: Arithmetic slip: recompute each step carefully.
- D: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B23. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 3 | 1 | 2
2 | 4 | 2 | 2
3 | 7 | 8 | -1

Compute the MAE of these predictions.

A. 5.000
B. 3.000
C. 1.000
D. 1.667

Answer: D. 1.667

Working:

- Errors y − ŷ: 2, 2, -1.
- |errors|: 2, 2, 1; sum = 5; MAE = sum / 3 = 1.667.

Option notes:

- A: Forgot to divide by n.

- B: That is MSE.
- C: Signed errors cancel; take |y − ŷ| first.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B24. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 4 | 0
2 | 5 | 4 | 1
3 | 11 | 11 | 0

Compute the MAE of these predictions.

A. 0.383
B. 0.283
C. 1.000
D. 0.333

Answer: D. 0.333

Working:

- Errors y − ŷ: 0, 1, 0.
- |errors|: 0, 1, 0; sum = 1; MAE = sum / 3 = 0.333.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- C: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B25. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 11 | 12 | -1
2 | 2 | 0 | 2
3 | 8 | 11 | -3

Compute the MAE of these predictions.

A. 6.000
B. 4.667
C. 0.667
D. 2.000

Answer: D. 2.000

Working:

- Errors y − ŷ: -1, 2, -3.
- |errors|: 1, 2, 3; sum = 6; MAE = sum / 3 = 2.000.

Option notes:

- A: Forgot to divide by n.

- B: That is MSE.
- C: Signed errors cancel; take |y − ŷ| first.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B26. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 12 | 12 | 0
2 | 11 | 11 | 0
3 | 1 | 4 | -3
4 | 1 | 1 | 0

Compute the MAE of these predictions.

A. 3.000
B. 0.863
C. 2.250
D. 0.750

Answer: D. 0.750

Working:

- Errors y − ŷ: 0, 0, -3, 0.
- |errors|: 0, 0, 3, 0; sum = 3; MAE = sum / 4 = 0.750.

Option notes:

- A: Forgot to divide by n.
- B: Arithmetic slip: recompute each step carefully.
- C: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B27. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 3 | 2
2 | 3 | 5 | -2
3 | 11 | 14 | -3
4 | 10 | 13 | -3
5 | 4 | 4 | 0

Compute the MAE of these predictions.

A. 2.000
B. 5.200
C. 1.200
D. 10.000

Answer: A. 2.000

Working:

- Errors y − ŷ: 2, -2, -3, -3, 0.
- |errors|: 2, 2, 3, 3, 0; sum = 10; MAE = sum / 5 = 2.000.

Option notes:

- B: That is MSE.
- C: Signed errors cancel; take |y − ŷ| first.
- D: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B28. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | -1 | 2
2 | 1 | 2 | -1
3 | 11 | 11 | 0

Compute the MAE of these predictions.

A. 0.333
B. 1.667
C. 1.000
D. 3.000

Answer: C. 1.000

Working:

- Errors y − ŷ: 2, -1, 0.
- |errors|: 2, 1, 0; sum = 3; MAE = sum / 3 = 1.000.

Option notes:

- A: Signed errors cancel; take |y − ŷ| first.
- B: That is MSE.
- D: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B29. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 8 | -3
2 | 1 | -1 | 2
3 | 10 | 8 | 2

Compute the MAE of these predictions.

A. 2.333
B. 0.333
C. 7.000
D. 5.667

Answer: A. 2.333

Working:

- Errors y − ŷ: -3, 2, 2.
- |errors|: 3, 2, 2; sum = 7; MAE = sum / 3 = 2.333.

Option notes:

- B: Signed errors cancel; take |y − ŷ| first.
- C: Forgot to divide by n.
- D: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B30. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 10 | 8 | 2
2 | 11 | 11 | 0
3 | 1 | 4 | -3
4 | 1 | -1 | 2

Compute the MAE of these predictions.

A. 7.000
B. 0.250
C. 1.750
D. 4.250

Answer: C. 1.750

Working:

- Errors y − ŷ: 2, 0, -3, 2.
- |errors|: 2, 0, 3, 2; sum = 7; MAE = sum / 4 = 1.750.

Option notes:

- A: Forgot to divide by n.
- B: Signed errors cancel; take |y − ŷ| first.
- D: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B31. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 11 | 13 | -2
2 | 9 | 10 | -1
3 | 6 | 3 | 3

Compute the MAE of these predictions.

A. 6.000
B. 0.000
C. 2.000
D. 4.667

Answer: C. 2.000

Working:

- Errors y − ŷ: -2, -1, 3.

- |errors|: 2, 1, 3; sum = 6; MAE = sum / 3 = 2.000.

Option notes:

- A: Forgot to divide by n.
- B: Signed errors cancel; take |y − ŷ| first.
- D: That is MSE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B32. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | 0 | 1
2 | 5 | 6 | -1
3 | 12 | 11 | 1

Compute the MAE of these predictions.

A. 0.333
B. 1.000
C. 1.150
D. 3.000

Answer: B. 1.000

Working:

- Errors y − ŷ: 1, -1, 1.
- |errors|: 1, 1, 1; sum = 3; MAE = sum / 3 = 1.000.

Option notes:

- A: Signed errors cancel; take |y − ŷ| first.
- C: Arithmetic slip: recompute each step carefully.
- D: Forgot to divide by n.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B33. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 7 | 9 | -2
2 | 11 | 13 | -2
3 | 9 | 8 | 1
4 | 6 | 4 | 2

Compute the RMSE of these predictions.

A. 1.750
B. 1.803
C. 3.250
D. 3.606

Answer: B. 1.803

Working:

- Errors y − ŷ: -2, -2, 1, 2.
- Squares: 4, 4, 1, 4; sum = 13; MSE = sum / 4 = 3.250.
- RMSE = √3.250 = 1.803.

Option notes:

- A: That is MAE.
- C: That is MSE; RMSE takes the square root.
- D: Forgot to divide by n inside the root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B34. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 6 | 8 | -2
2 | 5 | 8 | -3
3 | 2 | -1 | 3

Compute the RMSE of these predictions.

A. 2.708
B. 2.667
C. 7.333
D. 4.690

Answer: A. 2.708

Working:

- Errors y − ŷ: -2, -3, 3.
- Squares: 4, 9, 9; sum = 22; MSE = sum / 3 = 7.333.
- RMSE = √7.333 = 2.708.

Option notes:

- B: That is MAE.
- C: That is MSE; RMSE takes the square root.
- D: Forgot to divide by n inside the root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B35. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 6 | 7 | -1
2 | 1 | 4 | -3
3 | 10 | 10 | 0
4 | 5 | 4 | 1

Compute the RMSE of these predictions.

A. 1.658
B. 3.317
C. 1.250
D. 2.750

Answer: A. 1.658

Working:

- Errors y − ŷ: -1, -3, 0, 1.
- Squares: 1, 9, 0, 1; sum = 11; MSE = sum / 4 = 2.750.
- RMSE = √2.750 = 1.658.

Option notes:

- B: Forgot to divide by n inside the root.
- C: That is MAE.
- D: That is MSE; RMSE takes the square root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B36. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 7 | 7 | 0
2 | 4 | 1 | 3
3 | 3 | 1 | 2

Compute the RMSE of these predictions.

A. 1.667
B. 2.082
C. 3.606
D. 4.333

Answer: B. 2.082

Working:

- Errors y − ŷ: 0, 3, 2.
- Squares: 0, 9, 4; sum = 13; MSE = sum / 3 = 4.333.
- RMSE = √4.333 = 2.082.

Option notes:

- A: That is MAE.
- C: Forgot to divide by n inside the root.
- D: That is MSE; RMSE takes the square root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B37. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 6 | -1
2 | 8 | 5 | 3
3 | 2 | -1 | 3
4 | 8 | 8 | 0

Compute the RMSE of these predictions.

A. 2.179
B. 1.750
C. 4.359
D. 4.750

Answer: A. 2.179

Working:

- Errors $y - \hat{y}$: -1, 3, 3, 0.
- Squares: 1, 9, 9, 0; sum = 19; MSE = sum / 4 = 4.750.
- RMSE = √4.750 = 2.179.

Option notes:

- B: That is MAE.
- C: Forgot to divide by n inside the root.
- D: That is MSE; RMSE takes the square root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B38. [MCQ · Easy · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 4 | 5 | -1
2 | 6 | 9 | -3
3 | 7 | 9 | -2
4 | 7 | 6 | 1
5 | 3 | 2 | 1

Compute the RMSE of these predictions.

A. 1.600
B. 3.200
C. 4.000
D. 1.789

Answer: D. 1.789

Working:

- Errors $y - \hat{y}$: -1, -3, -2, 1, 1.
- Squares: 1, 9, 4, 1, 1; sum = 16; MSE = sum / 5 = 3.200.
- RMSE = √3.200 = 1.789.

Option notes:

- A: That is MAE.
- B: That is MSE; RMSE takes the square root.
- C: Forgot to divide by n inside the root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B39. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 6 | 5 | 1
2 | 10 | 7 | 3
3 | 6 | 5 | 1

Compute the RMSE of these predictions.

A. 3.317
B. 3.667
C. 1.915
D. 1.667

Answer: C. 1.915

Working:

- Errors y − ŷ: 1, 3, 1.
- Squares: 1, 9, 1; sum = 11; MSE = sum / 3 = 3.667.
- RMSE = √3.667 = 1.915.

Option notes:

- A: Forgot to divide by n inside the root.
- B: That is MSE; RMSE takes the square root.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B40. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 8 | 11 | -3
2 | 1 | 1 | 0
3 | 9 | 7 | 2

Compute the RMSE of these predictions.

A. 2.082
B. 4.333
C. 3.606
D. 1.667

Answer: A. 2.082

Working:

- Errors y − ŷ: -3, 0, 2.
- Squares: 9, 0, 4; sum = 13; MSE = sum / 3 = 4.333.
- RMSE = √4.333 = 2.082.

Option notes:

- B: That is MSE; RMSE takes the square root.
- C: Forgot to divide by n inside the root.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B41. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 1 | 3
2 | 12 | 12 | 0
3 | 2 | 0 | 2

Compute the RMSE of these predictions.

A. 2.082
B. 3.606
C. 1.667
D. 4.333

Answer: A. 2.082

Working:

- Errors y − ŷ: 3, 0, 2.
- Squares: 9, 0, 4; sum = 13; MSE = sum / 3 = 4.333.
- RMSE = √4.333 = 2.082.

Option notes:

- B: Forgot to divide by n inside the root.
- C: That is MAE.
- D: That is MSE; RMSE takes the square root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B42. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 10 | 11 | -1
2 | 8 | 11 | -3
3 | 4 | 4 | 0
4 | 7 | 9 | -2

Compute the RMSE of these predictions.

A. 3.742
B. 1.500
C. 3.500
D. 1.871

Answer: D. 1.871

Working:

- Errors y − ŷ: -1, -3, 0, -2.
- Squares: 1, 9, 0, 4; sum = 14; MSE = sum / 4 = 3.500.
- RMSE = √3.500 = 1.871.

Option notes:

- A: Forgot to divide by n inside the root.
- B: That is MAE.

- C: That is MSE; RMSE takes the square root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B43. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | 3 | -2
2 | 9 | 8 | 1
3 | 9 | 8 | 1

Compute the RMSE of these predictions.

A. 2.000
B. 2.449
C. 1.333
D. 1.414

Answer: D. 1.414

Working:

- Errors y − ŷ: -2, 1, 1.
- Squares: 4, 1, 1; sum = 6; MSE = sum / 3 = 2.000.
- RMSE = √2.000 = 1.414.

Option notes:

- A: That is MSE; RMSE takes the square root.
- B: Forgot to divide by n inside the root.
- C: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B44. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 6 | -2
2 | 6 | 4 | 2
3 | 2 | 4 | -2
4 | 6 | 6 | 0
5 | 12 | 14 | -2

Compute the RMSE of these predictions.

A. 3.200
B. 4.000
C. 1.600
D. 1.789

Answer: D. 1.789

Working:

- Errors y − ŷ: -2, 2, -2, 0, -2.
- Squares: 4, 4, 4, 0, 4; sum = 16; MSE = sum / 5 = 3.200.

- RMSE = √3.200 = 1.789.

Option notes:

- A: That is MSE; RMSE takes the square root.
- B: Forgot to divide by n inside the root.
- C: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B45. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 10 | 13 | -3
2 | 4 | 5 | -1
3 | 2 | 1 | 1
4 | 6 | 4 | 2

Compute the RMSE of these predictions.

A. 1.936
B. 3.873
C. 3.750
D. 1.750

Answer: A. 1.936

Working:

- Errors y − ŷ: -3, -1, 1, 2.
- Squares: 9, 1, 1, 4; sum = 15; MSE = sum / 4 = 3.750.
- RMSE = √3.750 = 1.936.

Option notes:

- B: Forgot to divide by n inside the root.
- C: That is MSE; RMSE takes the square root.
- D: That is MAE.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B46. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 2 | 3
2 | 1 | 3 | -2
3 | 5 | 3 | 2

Compute the RMSE of these predictions.

A. 5.667
B. 2.333
C. 2.380
D. 4.123

Answer: C. 2.380

Working:

- Errors y − ŷ: 3, -2, 2.
- Squares: 9, 4, 4; sum = 17; MSE = sum / 3 = 5.667.
- RMSE = √5.667 = 2.380.

Option notes:

- A: That is MSE; RMSE takes the square root.
- B: That is MAE.
- D: Forgot to divide by n inside the root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B47. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 3 | 4 | -1
2 | 1 | -2 | 3
3 | 8 | 9 | -1
4 | 10 | 11 | -1
5 | 1 | -2 | 3

Compute the RMSE of these predictions.

A. 1.800
B. 4.200
C. 2.049
D. 4.583

Answer: C. 2.049

Working:

- Errors y − ŷ: -1, 3, -1, -1, 3.
- Squares: 1, 9, 1, 1, 9; sum = 21; MSE = sum / 5 = 4.200.
- RMSE = √4.200 = 2.049.

Option notes:

- A: That is MAE.
- B: That is MSE; RMSE takes the square root.
- D: Forgot to divide by n inside the root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B48. [MCQ · Easy · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 9 | 7 | 2
2 | 3 | 5 | -2
3 | 7 | 9 | -2
4 | 12 | 12 | 0
5 | 8 | 8 | 0

Compute the RMSE of these predictions.

A. 3.464
B. 1.200
C. 2.400
D. 1.549

Answer: D. 1.549

Working:

- Errors $y - \hat{y}$: 2, -2, -2, 0, 0.
- Squares: 4, 4, 4, 0, 0; sum = 12; MSE = sum / 5 = 2.400.
- RMSE = $\sqrt{2.400} = 1.549$.

Option notes:

- A: Forgot to divide by n inside the root.
- B: That is MAE.
- C: That is MSE; RMSE takes the square root.

Takeaway: MSE punishes large errors (squares); MAE treats all errors linearly; RMSE is in the target's units.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B49. [BLANK · Hard · calculation]

i	y (actual)	$\hat{y}$ (predicted)	$y - \hat{y}$
1	6	8	-2
2	1	4	-3
3	6	3	3
4	12	11	1
5	7	6	1

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.608

Working:

- $\bar{y}$ = 6+1+6+12+7 / 5 = 6.4000.
- $\Sigma(y - \hat{y})^2 = 24$.
- $\Sigma(y - \bar{y})^2 = (6-6.400)^2 + (1-6.400)^2 + (6-6.400)^2 + (12-6.400)^2 + (7-6.400)^2 = 61.2000$.
- $R^2 = 1 - 24/61.2000 = 0.6078$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B50. [BLANK · Hard · calculation]

i	y (actual)	$\hat{y}$ (predicted)	$y - \hat{y}$
1	3	6	-3
2	4	4	0
3	5	5	0

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: -3.500

Working:

- $\bar{y}$ = 3+4+5 / 3 = 4.0000.
- $\Sigma(y - \hat{y})^2 = 9$.

- $\Sigma(y - \bar{y})^2 = (3-4.000)^2 + (4-4.000)^2 + (5-4.000)^2 = 2.0000$.
- $R^2 = 1 - 9/2.0000 = -3.5000$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B51. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 1 | 3
2 | 9 | 8 | 1
3 | 6 | 4 | 2
4 | 6 | 6 | 0
5 | 9 | 11 | -2

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.043

Working:

- $\bar{y} = 4+9+6+6+9 / 5 = 6.8000$.
- $\Sigma(y - \hat{y})^2 = 18$.
- $\Sigma(y - \bar{y})^2 = (4-6.800)^2 + (9-6.800)^2 + (6-6.800)^2 + (6-6.800)^2 + (9-6.800)^2 = 18.8000$.
- $R^2 = 1 - 18/18.8000 = 0.0426$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B52. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 3 | 2
2 | 7 | 7 | 0
3 | 10 | 8 | 2
4 | 12 | 14 | -2

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.586

Working:

- $\bar{y} = 5+7+10+12 / 4 = 8.5000$.
- $\Sigma(y - \hat{y})^2 = 12$.
- $\Sigma(y - \bar{y})^2 = (5-8.500)^2 + (7-8.500)^2 + (10-8.500)^2 + (12-8.500)^2 = 29.0000$.
- $R^2 = 1 - 12/29.0000 = 0.5862$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B53. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 6 | 9 | -3
2 | 9 | 8 | 1
3 | 10 | 7 | 3

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: -1.192

Working:

- $\bar{y}$ = 6+9+10 / 3 = 8.3333.
- $\Sigma(y - \hat{y})^2$ = 19.
- $\Sigma(y - \bar{y})^2 = (6-8.333)^2 + (9-8.333)^2 + (10-8.333)^2$ = 8.6667.
- $R^2 = 1 - 19/8.6667$ = -1.1923.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B54. [BLANK · Hard · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 4 | 3 | 1
2 | 8 | 8 | 0
3 | 4 | 4 | 0
4 | 5 | 5 | 0

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.907

Working:

- $\bar{y}$ = 4+8+4+5 / 4 = 5.2500.
- $\Sigma(y - \hat{y})^2$ = 1.
- $\Sigma(y - \bar{y})^2 = (4-5.250)^2 + (8-5.250)^2 + (4-5.250)^2 + (5-5.250)^2$ = 10.7500.
- $R^2 = 1 - 1/10.7500$ = 0.9070.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B55. [BLANK · Hard · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 6 | 9 | -3
2 | 8 | 6 | 2
3 | 12 | 14 | -2
4 | 9 | 6 | 3

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: -0.387

Working:

- $\bar{y}$ = 6+8+12+9 / 4 = 8.7500.
- $\Sigma(y - \hat{y})^2$ = 26.
- $\Sigma(y - \bar{y})^2 = (6-8.750)^2 + (8-8.750)^2 + (12-8.750)^2 + (9-8.750)^2$ = 18.7500.
- $R^2 = 1 - 26/18.7500$ = -0.3867.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B56. [BLANK · Hard · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 2 | 4 | -2

2 | 1 | 3 | -2
3 | 10 | 13 | -3
4 | 6 | 6 | 0
5 | 6 | 9 | -3

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.500

Working:

- $\bar{y} = 2+1+10+6+6 / 5 = 5.0000$.
- $\Sigma(y - \hat{y})^2 = 26$.
- $\Sigma(y - \bar{y})^2 = (2-5.000)^2 + (1-5.000)^2 + (10-5.000)^2 + (6-5.000)^2 + (6-5.000)^2 = 52.0000$.
- $R^2 = 1 - 26/52.0000 = 0.5000$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B57. [BLANK · Hard · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 1 | -2 | 3
2 | 9 | 8 | 1
3 | 5 | 7 | -2

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.563

Working:

- $\bar{y} = 1+9+5 / 3 = 5.0000$.
- $\Sigma(y - \hat{y})^2 = 14$.
- $\Sigma(y - \bar{y})^2 = (1-5.000)^2 + (9-5.000)^2 + (5-5.000)^2 = 32.0000$.
- $R^2 = 1 - 14/32.0000 = 0.5625$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B58. [BLANK · Hard · calculation]

i | y (actual) | $\hat{y}$ (predicted) | $y - \hat{y}$
1 | 2 | 3 | -1
2 | 3 | 2 | 1
3 | 8 | 11 | -3
4 | 6 | 3 | 3
5 | 1 | 4 | -3

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.147

Working:

- $\bar{y} = 2+3+8+6+1 / 5 = 4.0000$.
- $\Sigma(y - \hat{y})^2 = 29$.
- $\Sigma(y - \bar{y})^2 = (2-4.000)^2 + (3-4.000)^2 + (8-4.000)^2 + (6-4.000)^2 + (1-4.000)^2 = 34.0000$.
- $R^2 = 1 - 29/34.0000 = 0.1471$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B59. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 9 | 9 | 0
2 | 5 | 2 | 3
3 | 3 | 4 | -1

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.464

Working:

- $\bar{y} = 9+5+3 / 3 = 5.6667$.
- $\Sigma(y - \hat{y})^2 = 10$.
- $\Sigma(y - \bar{y})^2 = (9-5.667)^2 + (5-5.667)^2 + (3-5.667)^2 = 18.6667$.
- $R^2 = 1 - 10/18.6667 = 0.4643$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B60. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 6 | 9 | -3
2 | 3 | 5 | -2
3 | 7 | 4 | 3
4 | 1 | -2 | 3
5 | 9 | 6 | 3

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.020

Working:

- $\bar{y} = 6+3+7+1+9 / 5 = 5.2000$.
- $\Sigma(y - \hat{y})^2 = 40$.
- $\Sigma(y - \bar{y})^2 = (6-5.200)^2 + (3-5.200)^2 + (7-5.200)^2 + (1-5.200)^2 + (9-5.200)^2 = 40.8000$.
- $R^2 = 1 - 40/40.8000 = 0.0196$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B61. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 6 | -1
2 | 6 | 3 | 3
3 | 5 | 5 | 0
4 | 12 | 15 | -3

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.441

Working:

- $\bar{y} = 5+6+5+12 / 4 = 7.0000$.

- $\Sigma(y - \hat{y})^2 = 19$.
- $\Sigma(y - \bar{y})^2 = (5-7.000)^2 + (6-7.000)^2 + (5-7.000)^2 + (12-7.000)^2 = 34.0000$.
- $R^2 = 1 - 19/34.0000 = 0.4412$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B62. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 10 | 8 | 2
2 | 10 | 11 | -1
3 | 8 | 6 | 2

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: -2.375

Working:

- $\bar{y} = 10+10+8 / 3 = 9.3333$.
- $\Sigma(y - \hat{y})^2 = 9$.
- $\Sigma(y - \bar{y})^2 = (10-9.333)^2 + (10-9.333)^2 + (8-9.333)^2 = 2.6667$.
- $R^2 = 1 - 9/2.6667 = -2.3750$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B63. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 6 | 7 | -1
2 | 5 | 2 | 3
3 | 2 | 3 | -1
4 | 4 | 6 | -2
5 | 4 | 2 | 2

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: -1.159

Working:

- $\bar{y} = 6+5+2+4+4 / 5 = 4.2000$.
- $\Sigma(y - \hat{y})^2 = 19$.
- $\Sigma(y - \bar{y})^2 = (6-4.200)^2 + (5-4.200)^2 + (2-4.200)^2 + (4-4.200)^2 + (4-4.200)^2 = 8.8000$.
- $R^2 = 1 - 19/8.8000 = -1.1591$.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B64. [BLANK · Hard · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 2 | -1 | 3
2 | 6 | 7 | -1
3 | 11 | 13 | -2
4 | 7 | 10 | -3

Compute $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$ for these predictions.

Answer: 0.439

Working:

- $\bar{y}$ = 2+6+11+7 / 4 = 6.5000.
- $\Sigma(y - \hat{y})^2$ = 23.
- $\Sigma(y - \bar{y})^2 = (2-6.500)^2 + (6-6.500)^2 + (11-6.500)^2 + (7-6.500)^2$ = 41.0000.
- $R^2 = 1 - 23/41.0000$ = 0.4390.

Takeaway: $R^2 = 1$ is perfect; $R^2 < 0$ means worse than always predicting the mean.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B65. [BLANK · Medium · calculation]

i	y (actual)	$\hat{y}$ (predicted)	$y - \hat{y}$
1	7	9	-2
2	11	13	-2
3	9	8	1
4	6	4	2

Compute the MAPE (%) of these predictions.

Answer: 22.80

Working:

- $|(y - \hat{y})/y|$: 0.2857, 0.1818, 0.1111, 0.3333.
- MAPE = (100/4) × sum = 22.80%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B66. [BLANK · Medium · calculation]

i	y (actual)	$\hat{y}$ (predicted)	$y - \hat{y}$
1	6	8	-2
2	5	8	-3
3	2	-1	3

Compute the MAPE (%) of these predictions.

Answer: 81.11

Working:

- $|(y - \hat{y})/y|$: 0.3333, 0.6000, 1.5000.
- MAPE = (100/3) × sum = 81.11%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B67. [BLANK · Medium · calculation]

i	y (actual)	$\hat{y}$ (predicted)	$y - \hat{y}$
1	6	7	-1
2	1	4	-3
3	10	10	0
4	5	4	1

Compute the MAPE (%) of these predictions.

Answer: 84.17

Working:

- |(y − ŷ)/y|: 0.1667, 3.0000, 0.0000, 0.2000.
- MAPE = (100/4) × sum = 84.17%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B68. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 7 | 7 | 0
2 | 4 | 1 | 3
3 | 3 | 1 | 2

Compute the MAPE (%) of these predictions.

Answer: 47.22

Working:

- |(y − ŷ)/y|: 0.0000, 0.7500, 0.6667.
- MAPE = (100/3) × sum = 47.22%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B69. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 6 | -1
2 | 8 | 5 | 3
3 | 2 | -1 | 3
4 | 8 | 8 | 0

Compute the MAPE (%) of these predictions.

Answer: 51.88

Working:

- |(y − ŷ)/y|: 0.2000, 0.3750, 1.5000, 0.0000.
- MAPE = (100/4) × sum = 51.88%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B70. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 5 | -1
2 | 6 | 9 | -3
3 | 7 | 9 | -2
4 | 7 | 6 | 1
5 | 3 | 2 | 1

Compute the MAPE (%) of these predictions.

Answer: 30.24

Working:

- |(y − ŷ)/y|: 0.2500, 0.5000, 0.2857, 0.1429, 0.3333.
- MAPE = (100/5) × sum = 30.24%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B71. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 6 | 5 | 1
2 | 10 | 7 | 3
3 | 6 | 5 | 1

Compute the MAPE (%) of these predictions.

Answer: 21.11

Working:

- |(y − ŷ)/y|: 0.1667, 0.3000, 0.1667.
- MAPE = (100/3) × sum = 21.11%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B72. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 8 | 11 | -3
2 | 1 | 1 | 0
3 | 9 | 7 | 2

Compute the MAPE (%) of these predictions.

Answer: 19.91

Working:

- |(y − ŷ)/y|: 0.3750, 0.0000, 0.2222.
- MAPE = (100/3) × sum = 19.91%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B73. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 1 | 3
2 | 12 | 12 | 0
3 | 2 | 0 | 2

Compute the MAPE (%) of these predictions.

Answer: 58.33

Working:

- |(y − ŷ)/y|: 0.7500, 0.0000, 1.0000.
- MAPE = (100/3) × sum = 58.33%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B74. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 10 | 11 | -1
2 | 8 | 11 | -3
3 | 4 | 4 | 0
4 | 7 | 9 | -2

Compute the MAPE (%) of these predictions.

Answer: 19.02

Working:

- |(y − ŷ)/y|: 0.1000, 0.3750, 0.0000, 0.2857.
- MAPE = (100/4) × sum = 19.02%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B75. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 1 | 3 | -2
2 | 9 | 8 | 1
3 | 9 | 8 | 1

Compute the MAPE (%) of these predictions.

Answer: 74.07

Working:

- |(y − ŷ)/y|: 2.0000, 0.1111, 0.1111.
- MAPE = (100/3) × sum = 74.07%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B76. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 4 | 6 | -2
2 | 6 | 4 | 2
3 | 2 | 4 | -2
4 | 6 | 6 | 0
5 | 12 | 14 | -2

Compute the MAPE (%) of these predictions.

Answer: 40.00

Working:

- |(y − ŷ)/y|: 0.5000, 0.3333, 1.0000, 0.0000, 0.1667.
- MAPE = (100/5) × sum = 40.00%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B77. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 10 | 13 | -3
2 | 4 | 5 | -1
3 | 2 | 1 | 1
4 | 6 | 4 | 2

Compute the MAPE (%) of these predictions.

Answer: 34.58

Working:

- |(y − ŷ)/y|: 0.3000, 0.2500, 0.5000, 0.3333.
- MAPE = (100/4) × sum = 34.58%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B78. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 5 | 2 | 3
2 | 1 | 3 | -2
3 | 5 | 3 | 2

Compute the MAPE (%) of these predictions.

Answer: 100.00

Working:

- |(y − ŷ)/y|: 0.6000, 2.0000, 0.4000.
- MAPE = (100/3) × sum = 100.00%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B79. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 3 | 4 | -1
2 | 1 | -2 | 3
3 | 8 | 9 | -1
4 | 10 | 11 | -1
5 | 1 | -2 | 3

Compute the MAPE (%) of these predictions.

Answer: 131.17

Working:

- |(y − ŷ)/y|: 0.3333, 3.0000, 0.1250, 0.1000, 3.0000.
- MAPE = (100/5) × sum = 131.17%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B80. [BLANK · Medium · calculation]

i | y (actual) | ŷ (predicted) | y − ŷ
1 | 9 | 7 | 2
2 | 3 | 5 | -2
3 | 7 | 9 | -2
4 | 12 | 12 | 0
5 | 8 | 8 | 0

Compute the MAPE (%) of these predictions.

Answer: 23.49

Working:

- |(y − ŷ)/y|: 0.2222, 0.6667, 0.2857, 0.0000, 0.0000.
- MAPE = (100/5) × sum = 23.49%.

Takeaway: MAPE is scale-independent but breaks when actual values are 0 or near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B81. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.95 for an example whose true label is y = 0. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 0.051
B. 4.322
C. 2.996
D. 0.903

Answer: C. 2.996

Working:

- y = 0 → L = −ln(1 − p) = −ln(0.05) = 2.9957.

Option notes:

- A: Swapped p and 1 − p.
- B: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- D: That is squared error, not cross-entropy.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B82. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.3 for an example whose true label is y = 1. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 1.737
B. 0.490
C. 0.357
D. 1.204

Answer: D. 1.204

Working:

- y = 1 → L = −ln p = −ln(0.3) = 1.2040.

Option notes:

- A: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- B: That is squared error, not cross-entropy.
- C: Swapped p and 1 − p.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B83. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.8 for an example whose true label is y = 1. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 0.223
B. 0.322
C. 1.609
D. 0.040

Answer: A. 0.223

Working:

- y = 1 → L = −ln p = −ln(0.8) = 0.2231.

Option notes:

- B: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- C: Swapped p and 1 − p.
- D: That is squared error, not cross-entropy.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B84. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.9 for an example whose true label is y = 1. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 0.152
B. 2.303
C. 0.010
D. 0.105

Answer: D. 0.105

Working:

- y = 1 → L = −ln p = −ln(0.9) = 0.1054.

Option notes:

- A: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- B: Swapped p and 1 − p.
- C: That is squared error, not cross-entropy.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B85. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.8 for an example whose true label is y = 0. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 0.640
B. 0.223
C. 2.322
D. 1.609

Answer: D. 1.609

Working:

- $y = 0 \rightarrow L = -\ln(1 - p) = -\ln(0.20) = 1.6094$.

Option notes:

- A: That is squared error, not cross-entropy.
- B: Swapped p and 1 − p.
- C: Used $\log_2$; the lecture's cross-entropy uses the natural log here.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B86. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.8 for an example whose true label is y = 1. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 0.040
B. 1.609
C. 0.223
D. 0.322

Answer: C. 0.223

Working:

- $y = 1 \rightarrow L = -\ln p = -\ln(0.8) = 0.2231$.

Option notes:

- A: That is squared error, not cross-entropy.
- B: Swapped p and 1 − p.
- D: Used $\log_2$; the lecture's cross-entropy uses the natural log here.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B87. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.6 for an example whose true label is y = 0. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 1.322
B. 0.916
C. 0.360
D. 0.511

Answer: B. 0.916

Working:

- $y = 0 \rightarrow L = -\ln(1 - p) = -\ln(0.40) = 0.9163$.

Option notes:

- A: Used $\log_2$; the lecture's cross-entropy uses the natural log here.

- C: That is squared error, not cross-entropy.
- D: Swapped p and 1 − p.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B88. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.7 for an example whose true label is y = 0. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 1.204
B. 0.490
C. 0.357
D. 1.737

Answer: A. 1.204

Working:

- y = 0 → L = −ln(1 − p) = −ln(0.30) = 1.2040.

Option notes:

- B: That is squared error, not cross-entropy.
- C: Swapped p and 1 − p.
- D: Used $\log_2$; the lecture's cross-entropy uses the natural log here.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B89. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.7 for an example whose true label is y = 0. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 1.737
B. 0.490
C. 1.204
D. 0.357

Answer: C. 1.204

Working:

- y = 0 → L = −ln(1 − p) = −ln(0.30) = 1.2040.

Option notes:

- A: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- B: That is squared error, not cross-entropy.
- D: Swapped p and 1 − p.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B90. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.7 for an example whose true label is y = 0. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 0.490
B. 0.357
C. 1.204
D. 1.737

Answer: C. 1.204

Working:

- $y = 0 \rightarrow L = -\ln(1 - p) = -\ln(0.30) = 1.2040$.

Option notes:

- A: That is squared error, not cross-entropy.
- B: Swapped p and 1 − p.
- D: Used $\log_2$; the lecture's cross-entropy uses the natural log here.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B91. [MCQ · Medium · calculation]

A logistic regression model outputs $p = P(y=1|x) = 0.3$ for an example whose true label is $y = 1$. Using natural logs, what is the binary cross-entropy loss $L = -[y \cdot \ln p + (1-y) \cdot \ln(1-p)]$?

A. 1.204
B. 1.737
C. 0.357
D. 0.490

Answer: A. 1.204

Working:

- $y = 1 \rightarrow L = -\ln p = -\ln(0.3) = 1.2040$.

Option notes:

- B: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- C: Swapped p and 1 − p.
- D: That is squared error, not cross-entropy.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B92. [MCQ · Medium · calculation]

A logistic regression model outputs $p = P(y=1|x) = 0.1$ for an example whose true label is $y = 1$. Using natural logs, what is the binary cross-entropy loss $L = -[y \cdot \ln p + (1-y) \cdot \ln(1-p)]$?

A. 0.105
B. 2.303
C. 0.810
D. 3.322

Answer: B. 2.303

Working:

- $y = 1 \rightarrow L = -\ln p = -\ln(0.1) = 2.3026$.

Option notes:

- A: Swapped p and 1 − p.

- C: That is squared error, not cross-entropy.
- D: Used $\log_2$; the lecture's cross-entropy uses the natural log here.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B93. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.8 for an example whose true label is y = 1. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 0.223
B. 1.609
C. 0.322
D. 0.040

Answer: A. 0.223

Working:

- y = 1 → L = −ln p = −ln(0.8) = 0.2231.

Option notes:

- B: Swapped p and 1 − p.
- C: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- D: That is squared error, not cross-entropy.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B94. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.3 for an example whose true label is y = 1. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 1.204
B. 0.490
C. 1.737
D. 0.357

Answer: A. 1.204

Working:

- y = 1 → L = −ln p = −ln(0.3) = 1.2040.

Option notes:

- B: That is squared error, not cross-entropy.
- C: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- D: Swapped p and 1 − p.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B95. [MCQ · Medium · calculation]

A logistic regression model outputs p = P(y=1|x) = 0.1 for an example whose true label is y = 1. Using natural logs, what is the binary cross-entropy loss L = −[y·ln p + (1−y)·ln(1−p)]?

A. 3.322
B. 0.810
C. 2.303
D. 0.105

Answer: C. 2.303

Working:

- $y = 1 \rightarrow L = -\ln p = -\ln(0.1) = 2.3026$.

Option notes:

- A: Used $\log_2$; the lecture's cross-entropy uses the natural log here.
- B: That is squared error, not cross-entropy.
- D: Swapped p and 1 − p.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B96. [MCQ · Medium · calculation]

A logistic regression model outputs $p = P(y=1|x) = 0.8$ for an example whose true label is $y = 1$. Using natural logs, what is the binary cross-entropy loss $L = -[y \cdot \ln p + (1-y) \cdot \ln(1-p)]$?

A. 1.609
B. 0.040
C. 0.223
D. 0.322

Answer: C. 0.223

Working:

- $y = 1 \rightarrow L = -\ln p = -\ln(0.8) = 0.2231$.

Option notes:

- A: Swapped p and 1 − p.
- B: That is squared error, not cross-entropy.
- D: Used $\log_2$; the lecture's cross-entropy uses the natural log here.

Takeaway: Confident and wrong → huge loss. Confident and right → loss near 0.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B97. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1 x + w_0$ with $w_0 = 1.5$, $w_1 = -0.5$, learning rate $\alpha = 0.1$. One training example $(x, y) = (4, 3)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x)) \cdot x$. Give h(x), new w_0 and new w_1 (3 dp).

(1) h(x) = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: h(x) = -0.500; new w_0 = 1.850; new w_1 = 0.900

Working:

- $h(x) = -0.5 \times 4 + 1.5 = -0.5$.
- Error $y - h = 3 - -0.5 = 3.5$.
- $w_0 \leftarrow 1.5 + 0.1 \times 3.5 = 1.85$.
- $w_1 \leftarrow -0.5 + 0.1 \times 3.5 \times 4 = 0.9$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B98. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1 x + w_0$ with $w_0 = -1$, $w_1 = -2$, learning rate $\alpha = 0.2$. One training example $(x, y) = (5, 11)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = -11.000$; new $w_0 = 3.400$; new $w_1 = 20.000$

Working:

- $h(x) = -2\times5 + -1 = -11$.
- Error $y - h = 11 - -11 = 22$.
- $w_0 \leftarrow -1 + 0.2\times22 = 3.4$.
- $w_1 \leftarrow -2 + 0.2\times22\times5 = 20$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B99. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1 x + w_0$ with $w_0 = 1$, $w_1 = 2$, learning rate $\alpha = 0.01$. One training example $(x, y) = (4, -1)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = 9.000$; new $w_0 = 0.900$; new $w_1 = 1.600$

Working:

- $h(x) = 2\times4 + 1 = 9$.
- Error $y - h = -1 - 9 = -10$.
- $w_0 \leftarrow 1 + 0.01\times-10 = 0.9$.
- $w_1 \leftarrow 2 + 0.01\times-10\times4 = 1.6$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B100. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1 x + w_0$ with $w_0 = -1.5$, $w_1 = 1$, learning rate $\alpha = 0.01$. One training example $(x, y) = (2, 5)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = 0.500$; new $w_0 = -1.455$; new $w_1 = 1.090$

Working:

- h(x) = 1×2 + -1.5 = 0.5.
- Error y − h = 5 − 0.5 = 4.5.
- $w_0 \leftarrow$ -1.5 + 0.01×4.5 = -1.455.
- $w_1 \leftarrow$ 1 + 0.01×4.5×2 = 1.09.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B101. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with w_0 = -1, w_1 = -1.5, learning rate α = 0.1. One training example (x, y) = (2, -1). Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give h(x), new w_0 and new w_1 (3 dp).

(1) h(x) = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: h(x) = -4.000; new w_0 = -0.700; new w_1 = -0.900

Working:

- h(x) = -1.5×2 + -1 = -4.
- Error y − h = -1 − -4 = 3.
- $w_0 \leftarrow$ -1 + 0.1×3 = -0.7.
- $w_1 \leftarrow$ -1.5 + 0.1×3×2 = -0.9.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B102. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with w_0 = -1, w_1 = 1.5, learning rate α = 0.01. One training example (x, y) = (5, 3). Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give h(x), new w_0 and new w_1 (3 dp).

(1) h(x) = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: h(x) = 6.500; new w_0 = -1.035; new w_1 = 1.325

Working:

- h(x) = 1.5×5 + -1 = 6.5.
- Error y − h = 3 − 6.5 = -3.5.
- $w_0 \leftarrow$ -1 + 0.01×-3.5 = -1.035.
- $w_1 \leftarrow$ 1.5 + 0.01×-3.5×5 = 1.325.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B103. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with w_0 = 1, w_1 = 2, learning rate α = 0.2. One training example (x, y) = (3, 1). Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give h(x), new w_0 and new w_1 (3 dp).

(1) h(x) = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: h(x) = 7.000; new w_0 = -0.200; new w_1 = -1.600

Working:

- h(x) = 2×3 + 1 = 7.
- Error y − h = 1 − 7 = -6.
- $w_0 \leftarrow$ 1 + 0.2×-6 = -0.2.
- $w_1 \leftarrow$ 2 + 0.2×-6×3 = -1.6.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B104. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with w_0 = -1.5, w_1 = -0.5, learning rate α = 0.01. One training example (x, y) = (3, 5). Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give h(x), new w_0 and new w_1 (3 dp).

(1) h(x) = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: h(x) = -3.000; new w_0 = -1.420; new w_1 = -0.260

Working:

- h(x) = -0.5×3 + -1.5 = -3.
- Error y − h = 5 − -3 = 8.
- $w_0 \leftarrow$ -1.5 + 0.01×8 = -1.42.
- $w_1 \leftarrow$ -0.5 + 0.01×8×3 = -0.26.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B105. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with w_0 = -1.5, w_1 = 0.5, learning rate α = 0.01. One training example (x, y) = (3, 9). Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give h(x), new w_0 and new w_1 (3 dp).

(1) h(x) = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: h(x) = 0.000; new w_0 = -1.410; new w_1 = 0.770

Working:

- h(x) = 0.5×3 + -1.5 = 0.
- Error y − h = 9 − 0 = 9.
- $w_0 \leftarrow$ -1.5 + 0.01×9 = -1.41.
- $w_1 \leftarrow$ 0.5 + 0.01×9×3 = 0.77.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B106. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with $w_0 = 1.5$, $w_1 = 1$, learning rate $\alpha = 0.2$. One training example $(x, y) = (3, 11)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = 4.500$; new $w_0 = 2.800$; new $w_1 = 4.900$

Working:

- $h(x) = 1\times3 + 1.5 = 4.5$.
- Error $y - h = 11 - 4.5 = 6.5$.
- $w_0 \leftarrow 1.5 + 0.2\times6.5 = 2.8$.
- $w_1 \leftarrow 1 + 0.2\times6.5\times3 = 4.9$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as $+ \alpha\cdot$error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B107. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with $w_0 = 1$, $w_1 = -2$, learning rate $\alpha = 0.1$. One training example $(x, y) = (3, 10)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = -5.000$; new $w_0 = 2.500$; new $w_1 = 2.500$

Working:

- $h(x) = -2\times3 + 1 = -5$.
- Error $y - h = 10 - -5 = 15$.
- $w_0 \leftarrow 1 + 0.1\times15 = 2.5$.
- $w_1 \leftarrow -2 + 0.1\times15\times3 = 2.5$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as $+ \alpha\cdot$error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B108. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with $w_0 = -1.5$, $w_1 = -2$, learning rate $\alpha = 0.2$. One training example $(x, y) = (3, 11)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x))\cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = -7.500$; new $w_0 = 2.200$; new $w_1 = 9.100$

Working:

- $h(x) = -2\times3 + -1.5 = -7.5$.
- Error $y - h = 11 - -7.5 = 18.5$.
- $w_0 \leftarrow -1.5 + 0.2\times18.5 = 2.2$.

- $w_1 \leftarrow -2 + 0.2 \times 18.5 \times 3 = 9.1$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B109. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1 x + w_0$ with $w_0 = 1.5$, $w_1 = 0$, learning rate $\alpha = 0.01$. One training example $(x, y) = (2, 9)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x)) \cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = 1.500$; new $w_0 = 1.575$; new $w_1 = 0.150$

Working:

- $h(x) = 0 \times 2 + 1.5 = 1.5$.
- Error $y - h = 9 - 1.5 = 7.5$.
- $w_0 \leftarrow 1.5 + 0.01 \times 7.5 = 1.575$.
- $w_1 \leftarrow 0 + 0.01 \times 7.5 \times 2 = 0.15$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B110. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1 x + w_0$ with $w_0 = 1$, $w_1 = 0.5$, learning rate $\alpha = 0.2$. One training example $(x, y) = (3, -1)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x)) \cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = 2.500$; new $w_0 = 0.300$; new $w_1 = -1.600$

Working:

- $h(x) = 0.5 \times 3 + 1 = 2.5$.
- Error $y - h = -1 - 2.5 = -3.5$.
- $w_0 \leftarrow 1 + 0.2 \times -3.5 = 0.3$.
- $w_1 \leftarrow 0.5 + 0.2 \times -3.5 \times 3 = -1.6$.

Takeaway: Gradient descent: step AGAINST the gradient; $w \leftarrow w - \alpha\, \partial L/\partial w$ (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B111. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1 x + w_0$ with $w_0 = 0$, $w_1 = 2$, learning rate $\alpha = 0.05$. One training example $(x, y) = (2, 8)$. Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x)) \cdot x$. Give $h(x)$, new w_0 and new w_1 (3 dp).

(1) $h(x)$ = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: $h(x) = 4.000$; new $w_0 = 0.200$; new $w_1 = 2.400$

Working:

- h(x) = 2×2 + 0 = 4.
- Error y − h = 8 − 4 = 4.
- $w_0 \leftarrow 0 + 0.05 \times 4 = 0.2$.
- $w_1 \leftarrow 2 + 0.05 \times 4 \times 2 = 2.4$.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B112. [BLANKS · Medium · calculation]

Linear regression $h_w(x) = w_1x + w_0$ with $w_0 = 0$, $w_1 = -1.5$, learning rate $\alpha = 0.01$. One training example (x, y) = (1, 4). Apply the per-example update from the lecture: $w_0 \leftarrow w_0 + \alpha(y - h(x))$, $w_1 \leftarrow w_1 + \alpha(y - h(x)) \cdot x$. Give h(x), new w_0 and new w_1 (3 dp).

(1) h(x) = ____
(2) new w_0 = ____
(3) new w_1 = ____

Answer: h(x) = -1.500; new w_0 = 0.055; new w_1 = -1.445

Working:

- h(x) = -1.5×1 + 0 = -1.5.
- Error y − h = 4 − -1.5 = 5.5.
- $w_0 \leftarrow 0 + 0.01 \times 5.5 = 0.055$.
- $w_1 \leftarrow -1.5 + 0.01 \times 5.5 \times 1 = -1.445$.

Takeaway: Gradient descent: step AGAINST the gradient; w ← w − α ∂L/∂w (here written as + α·error).

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B113. [MCQ · Easy · calculation]

A weight w = -0.4 has gradient ∂L/∂w = -0.8. With learning rate α = 0.01, what is w after one gradient-descent step?

A. -0.392
B. -0.008
C. 0.400
D. -0.408

Answer: A. -0.392

Working:

- w_new = w − α·∂L/∂w = -0.4 − 0.01×(-0.8) = -0.392.

Option notes:

- B: That is just the step size, not the new weight.
- C: Forgot the learning rate.
- D: Moved WITH the gradient (gradient ascent). Descent subtracts.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B114. [MCQ · Easy · calculation]

A weight w = -0.6 has gradient ∂L/∂w = 2.6. With learning rate α = 0.01, what is w after one gradient-descent step?

A. -0.574
B. -3.200
C. 0.026
D. -0.626

Answer: D. -0.626

Working:

- w_new = w − α·∂L/∂w = -0.6 − 0.01×(2.6) = -0.626.

Option notes:

- A: Moved WITH the gradient (gradient ascent). Descent subtracts.
- B: Forgot the learning rate.
- C: That is just the step size, not the new weight.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B115. [MCQ · Easy · calculation]

A weight w = 1.8 has gradient ∂L/∂w = -2.2. With learning rate α = 0.5, what is w after one gradient-descent step?

A. 2.900
B. -1.100
C. 0.700
D. 4.000

Answer: A. 2.900

Working:

- w_new = w − α·∂L/∂w = 1.8 − 0.5×(-2.2) = 2.9.

Option notes:

- B: That is just the step size, not the new weight.
- C: Moved WITH the gradient (gradient ascent). Descent subtracts.
- D: Forgot the learning rate.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B116. [MCQ · Easy · calculation]

A weight w = -1.2 has gradient ∂L/∂w = 0.2. With learning rate α = 0.1, what is w after one gradient-descent step?

A. -1.180
B. 0.020
C. -1.400
D. -1.220

Answer: D. -1.220

Working:

- w_new = w − α·∂L/∂w = -1.2 − 0.1×(0.2) = -1.22.

Option notes:

- A: Moved WITH the gradient (gradient ascent). Descent subtracts.

- B: That is just the step size, not the new weight.
- C: Forgot the learning rate.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B117. [MCQ · Easy · calculation]

A weight w = -0.3 has gradient ∂L/∂w = 2.2. With learning rate α = 0.1, what is w after one gradient-descent step?

A. 0.220
B. -0.520
C. -0.080
D. -2.500

Answer: B. -0.520

Working:

- w_new = w − α·∂L/∂w = -0.3 − 0.1×(2.2) = -0.52.

Option notes:

- A: That is just the step size, not the new weight.
- C: Moved WITH the gradient (gradient ascent). Descent subtracts.
- D: Forgot the learning rate.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B118. [MCQ · Easy · calculation]

A weight w = 0 has gradient ∂L/∂w = -1.7. With learning rate α = 0.1, what is w after one gradient-descent step?

A. 0.196
B. 1.700
C. 0.170
D. -0.170

Answer: C. 0.170

Working:

- w_new = w − α·∂L/∂w = 0 − 0.1×(-1.7) = 0.17.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Forgot the learning rate.
- D: Moved WITH the gradient (gradient ascent). Descent subtracts.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B119. [MCQ · Easy · calculation]

A weight w = -1.7 has gradient ∂L/∂w = 0.1. With learning rate α = 0.1, what is w after one gradient-descent step?

A. -1.800
B. -1.690
C. 0.010
D. -1.710

Answer: D. -1.710

Working:

- w_new = w − α·∂L/∂w = -1.7 − 0.1×(0.1) = -1.71.

Option notes:

- A: Forgot the learning rate.
- B: Moved WITH the gradient (gradient ascent). Descent subtracts.
- C: That is just the step size, not the new weight.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B120. [MCQ · Easy · calculation]

A weight w = 1.1 has gradient ∂L/∂w = -2.2. With learning rate α = 0.05, what is w after one gradient-descent step?

A. 0.990
B. 1.210
C. 3.300
D. -0.110

Answer: B. 1.210

Working:

- w_new = w − α·∂L/∂w = 1.1 − 0.05×(-2.2) = 1.21.

Option notes:

- A: Moved WITH the gradient (gradient ascent). Descent subtracts.
- C: Forgot the learning rate.
- D: That is just the step size, not the new weight.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B121. [MCQ · Easy · calculation]

A weight w = -1.7 has gradient ∂L/∂w = -2.7. With learning rate α = 0.1, what is w after one gradient-descent step?

A. -1.430
B. 1.000
C. -1.970
D. -0.270

Answer: A. -1.430

Working:

- w_new = w − α·∂L/∂w = -1.7 − 0.1×(-2.7) = -1.43.

Option notes:

- B: Forgot the learning rate.

- C: Moved WITH the gradient (gradient ascent). Descent subtracts.
- D: That is just the step size, not the new weight.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B122. [MCQ · Easy · calculation]

A weight w = 0.7 has gradient ∂L/∂w = -2. With learning rate α = 0.1, what is w after one gradient-descent step?

A. -0.200
B. 2.700
C. 0.900
D. 0.500

Answer: C. 0.900

Working:

- w_new = w − α·∂L/∂w = 0.7 − 0.1×(-2) = 0.9.

Option notes:

- A: That is just the step size, not the new weight.
- B: Forgot the learning rate.
- D: Moved WITH the gradient (gradient ascent). Descent subtracts.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B123. [MCQ · Easy · calculation]

A weight w = -1.8 has gradient ∂L/∂w = -1.3. With learning rate α = 0.05, what is w after one gradient-descent step?

A. -0.500
B. -1.735
C. -0.065
D. -1.865

Answer: B. -1.735

Working:

- w_new = w − α·∂L/∂w = -1.8 − 0.05×(-1.3) = -1.735.

Option notes:

- A: Forgot the learning rate.
- C: That is just the step size, not the new weight.
- D: Moved WITH the gradient (gradient ascent). Descent subtracts.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B124. [MCQ · Easy · calculation]

A weight w = 0.1 has gradient ∂L/∂w = -2.8. With learning rate α = 0.1, what is w after one gradient-descent step?

A. -0.280
B. -0.180
C. 0.380
D. 2.900

Answer: C. 0.380

Working:

- w_new = w − α·∂L/∂w = 0.1 − 0.1×(-2.8) = 0.38.

Option notes:

- A: That is just the step size, not the new weight.
- B: Moved WITH the gradient (gradient ascent). Descent subtracts.
- D: Forgot the learning rate.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B125. [MCQ · Easy · calculation]

A weight w = -1.9 has gradient ∂L/∂w = -2.9. With learning rate α = 0.05, what is w after one gradient-descent step?

A. -2.045
B. -1.755
C. -0.145
D. 1.000

Answer: B. -1.755

Working:

- w_new = w − α·∂L/∂w = -1.9 − 0.05×(-2.9) = -1.755.

Option notes:

- A: Moved WITH the gradient (gradient ascent). Descent subtracts.
- C: That is just the step size, not the new weight.
- D: Forgot the learning rate.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B126. [MCQ · Easy · calculation]

A weight w = -1.3 has gradient ∂L/∂w = -2.8. With learning rate α = 0.01, what is w after one gradient-descent step?

A. 1.500
B. -0.028
C. -1.328
D. -1.272

Answer: D. -1.272

Working:

- w_new = w − α·∂L/∂w = -1.3 − 0.01×(-2.8) = -1.272.

Option notes:

- A: Forgot the learning rate.

- B: That is just the step size, not the new weight.
- C: Moved WITH the gradient (gradient ascent). Descent subtracts.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B127. [MCQ · Easy · calculation]

A weight w = 0.7 has gradient ∂L/∂w = -3. With learning rate α = 0.5, what is w after one gradient-descent step?

A. -0.800
B. 3.700
C. -1.500
D. 2.200

Answer: D. 2.200

Working:

- w_new = w − α·∂L/∂w = 0.7 − 0.5×(-3) = 2.2.

Option notes:

- A: Moved WITH the gradient (gradient ascent). Descent subtracts.
- B: Forgot the learning rate.
- C: That is just the step size, not the new weight.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B128. [MCQ · Easy · calculation]

A weight w = 0.3 has gradient ∂L/∂w = -1.9. With learning rate α = 0.05, what is w after one gradient-descent step?

A. 0.205
B. 0.395
C. 2.200
D. -0.095

Answer: B. 0.395

Working:

- w_new = w − α·∂L/∂w = 0.3 − 0.05×(-1.9) = 0.395.

Option notes:

- A: Moved WITH the gradient (gradient ascent). Descent subtracts.
- C: Forgot the learning rate.
- D: That is just the step size, not the new weight.

Takeaway: Positive gradient → decrease w; negative gradient → increase w.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B129. [BLANKS · Hard · calculation]

Logistic regression: p = σ(w·x + b) with w = (0, 1), b = 1, x = (3, 3), true y = 1, α = 0.1. Using gradient $\partial L/\partial w_1 = (p - y)\cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 4; p = 0.9820; new w_1 = 0.0054

Working:

- z = 0×3 + 1×3 + 1 = 4.
- p = 1/(1 + e^(−4)) = 0.9820.
- $\partial L/\partial w_1$ = (0.9820 − 1)×3 = -0.0540.
- $w_1 \leftarrow$ 0 − 0.1×-0.0540 = 0.0054.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B130. [BLANKS · Hard · calculation]

Logistic regression: p = σ(w·x + b) with w = (-1.5, 2), b = 0.5, x = (1, 3), true y = 1, α = 0.1. Using gradient $\partial L/\partial w_1 = (p - y)\cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 5; p = 0.9933; new w_1 = -1.4993

Working:

- z = -1.5×1 + 2×3 + 0.5 = 5.
- p = 1/(1 + e^(−5)) = 0.9933.
- $\partial L/\partial w_1$ = (0.9933 − 1)×1 = -0.0067.
- $w_1 \leftarrow$ -1.5 − 0.1×-0.0067 = -1.4993.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B131. [BLANKS · Hard · calculation]

Logistic regression: p = σ(w·x + b) with w = (-2, 0), b = 0, x = (0, 3), true y = 1, α = 0.1. Using gradient $\partial L/\partial w_1 = (p - y)\cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 0; p = 0.5000; new w_1 = -2.0000

Working:

- z = -2×0 + 0×3 + 0 = 0.
- p = 1/(1 + e^(−0)) = 0.5000.
- $\partial L/\partial w_1$ = (0.5000 − 1)×0 = 0.0000.
- $w_1 \leftarrow$ -2 − 0.1×0.0000 = -2.0000.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B132. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (0.5, -1.5)$, $b = 0$, $x = (1, 1)$, true $y = 1$, $\alpha = 0.5$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = -1; p = 0.2689; new w_1 = 0.8655

Working:

- z = 0.5×1 + -1.5×1 + 0 = -1.
- p = 1/(1 + e^(−-1)) = 0.2689.
- $\partial L/\partial w_1$ = (0.2689 − 1)×1 = -0.7311.
- $w_1 \leftarrow$ 0.5 − 0.5×-0.7311 = 0.8655.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B133. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (2, -0.5)$, $b = 0$, $x = (2, 2)$, true $y = 1$, $\alpha = 0.1$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 3; p = 0.9526; new w_1 = 2.0095

Working:

- z = 2×2 + -0.5×2 + 0 = 3.
- p = 1/(1 + e^(−3)) = 0.9526.
- $\partial L/\partial w_1$ = (0.9526 − 1)×2 = -0.0949.
- $w_1 \leftarrow$ 2 − 0.1×-0.0949 = 2.0095.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B134. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (-2, 1.5)$, $b = 0.5$, $x = (0, 2)$, true $y = 1$, $\alpha = 0.5$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 3.5; p = 0.9707; new w_1 = -2.0000

Working:

- z = -2×0 + 1.5×2 + 0.5 = 3.5.
- p = 1/(1 + e^(−3.5)) = 0.9707.
- $\partial L/\partial w_1$ = (0.9707 − 1)×0 = 0.0000.
- $w_1 \leftarrow$ -2 − 0.5×0.0000 = -2.0000.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B135. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (-0.5, -2)$, $b = -0.5$, $x = (0, 2)$, true $y = 0$, $\alpha = 0.5$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = -4.5; p = 0.0110; new w_1 = -0.5000

Working:

- z = -0.5×0 + -2×2 + -0.5 = -4.5.
- p = 1/(1 + e^(−-4.5)) = 0.0110.
- $\partial L/\partial w_1$ = (0.0110 − 0)×0 = 0.0000.
- $w_1 \leftarrow$ -0.5 − 0.5×0.0000 = -0.5000.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B136. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (-1.5, -2)$, $b = 0$, $x = (1, 1)$, true $y = 0$, $\alpha = 0.1$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = -3.5; p = 0.0293; new w_1 = -1.5029

Working:

- z = -1.5×1 + -2×1 + 0 = -3.5.
- p = 1/(1 + e^(−-3.5)) = 0.0293.
- $\partial L/\partial w_1$ = (0.0293 − 0)×1 = 0.0293.
- $w_1 \leftarrow$ -1.5 − 0.1×0.0293 = -1.5029.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B137. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (2, -0.5)$, $b = 0.5$, $x = (0, 0)$, true $y = 0$, $\alpha = 0.1$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 0.5; p = 0.6225; new w_1 = 2.0000

Working:

- z = 2×0 + -0.5×0 + 0.5 = 0.5.
- p = 1/(1 + e^(−0.5)) = 0.6225.
- $\partial L/\partial w_1$ = (0.6225 − 0)×0 = 0.0000.
- $w_1 \leftarrow$ 2 − 0.1×0.0000 = 2.0000.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B138. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (-1, -1)$, $b = -0.5$, $x = (0, 1)$, true $y = 1$, $\alpha = 0.5$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = _____
(2) p = _____
(3) new w_1 = _____

Answer: z = -1.5; p = 0.1824; new w_1 = -1.0000

Working:

- $z = -1 \times 0 + -1 \times 1 + -0.5 = -1.5$.
- $p = 1/(1 + e^{(-(-1.5))}) = 0.1824$.
- $\partial L/\partial w_1 = (0.1824 - 1) \times 0 = 0.0000$.
- $w_1 \leftarrow -1 - 0.5 \times 0.0000 = -1.0000$.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B139. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (2, -0.5)$, $b = 0$, $x = (3, 0)$, true $y = 1$, $\alpha = 0.5$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = _____
(2) p = _____
(3) new w_1 = _____

Answer: z = 6; p = 0.9975; new w_1 = 2.0037

Working:

- $z = 2 \times 3 + -0.5 \times 0 + 0 = 6$.
- $p = 1/(1 + e^{(-6)}) = 0.9975$.
- $\partial L/\partial w_1 = (0.9975 - 1) \times 3 = -0.0074$.
- $w_1 \leftarrow 2 - 0.5 \times -0.0074 = 2.0037$.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B140. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (1, -1.5)$, $b = 0.5$, $x = (1, 2)$, true $y = 0$, $\alpha = 0.1$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = _____
(2) p = _____
(3) new w_1 = _____

Answer: z = -1.5; p = 0.1824; new w_1 = 0.9818

Working:

- $z = 1 \times 1 + -1.5 \times 2 + 0.5 = -1.5$.
- $p = 1/(1 + e^{(-(-1.5))}) = 0.1824$.
- $\partial L/\partial w_1 = (0.1824 - 0) \times 1 = 0.1824$.

- $w_1 \leftarrow 1 - 0.1 \times 0.1824 = 0.9818$.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B141. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (-1, 1.5)$, $b = -1$, $x = (1, 1)$, true $y = 0$, $\alpha = 0.1$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = -0.5; p = 0.3775; new w_1 = -1.0378

Working:

- $z = -1 \times 1 + 1.5 \times 1 + -1 = -0.5$.
- $p = 1/(1 + e^{-(-0.5)}) = 0.3775$.
- $\partial L/\partial w_1 = (0.3775 - 0) \times 1 = 0.3775$.
- $w_1 \leftarrow -1 - 0.1 \times 0.3775 = -1.0378$.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B142. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (-0.5, 0.5)$, $b = 1$, $x = (3, 1)$, true $y = 0$, $\alpha = 0.5$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 0; p = 0.5000; new w_1 = -1.2500

Working:

- $z = -0.5 \times 3 + 0.5 \times 1 + 1 = 0$.
- $p = 1/(1 + e^{-0}) = 0.5000$.
- $\partial L/\partial w_1 = (0.5000 - 0) \times 3 = 1.5000$.
- $w_1 \leftarrow -0.5 - 0.5 \times 1.5000 = -1.2500$.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B143. [BLANKS · Hard · calculation]

Logistic regression: $p = \sigma(w \cdot x + b)$ with $w = (-2, 1.5)$, $b = -1$, $x = (3, 3)$, true $y = 1$, $\alpha = 0.5$. Using gradient $\partial L/\partial w_1 = (p - y) \cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = -2.5; p = 0.0759; new w_1 = -0.6138

Working:

- $z = -2 \times 3 + 1.5 \times 3 + -1 = -2.5$.

- p = 1/(1 + e^(−-2.5)) = 0.0759.
- $\partial L/\partial w_1$ = (0.0759 − 1)×3 = -2.7724.
- $w_1 \leftarrow$ -2 − 0.5×-2.7724 = -0.6138.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B144. [BLANKS · Hard · calculation]

Logistic regression: p = σ(w·x + b) with w = (-1, 1.5), b = 1, x = (1, 0), true y = 1, α = 0.1. Using gradient $\partial L/\partial w_1 = (p - y)\cdot x_1$, give z, p (4 dp) and the updated w_1 (4 dp).

(1) z = ____
(2) p = ____
(3) new w_1 = ____

Answer: z = 0; p = 0.5000; new w_1 = -0.9500

Working:

- z = -1×1 + 1.5×0 + 1 = 0.
- p = 1/(1 + e^(−0)) = 0.5000.
- $\partial L/\partial w_1$ = (0.5000 − 1)×1 = -0.5000.
- $w_1 \leftarrow$ -1 − 0.1×-0.5000 = -0.9500.

Takeaway: Logistic regression = linear score passed through a sigmoid; trained with cross-entropy.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B145. [BLANKS · Hard · calculation]

| Actual + | Actual −
Predicted + | TP = 25 | FP = 23
Predicted − | FN = 6 | TN = 11

Give specificity (TNR) and balanced accuracy = (TPR + TNR)/2 (3 dp).

(1) Specificity = ____
(2) Balanced accuracy = ____

Answer: Specificity = 0.324; Balanced accuracy = 0.565

Working:

- Specificity = TN/(TN+FP) = 11/34 = 0.3235.
- TPR (recall) = 0.8065.
- Balanced accuracy = (0.8065 + 0.3235)/2 = 0.5650.

Takeaway: FPR = 1 − specificity is the x-axis of the ROC curve.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B146. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 57 | FP = 17
Predicted − | FN = 11 | TN = 71

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____

(3) F1 = ____

Answer: Precision = 0.770; Recall = 0.838; F1 = 0.803

Working:

- Precision = TP/(TP+FP) = 57/74 = 0.7703.
- Recall = TP/(TP+FN) = 57/68 = 0.8382.
- F1 = 2PR/(P+R) = 2×0.7703×0.8382/(0.7703+0.8382) = 0.8028.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B147. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 56 | FP = 16
Predicted − | FN = 30 | TN = 14

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____
(3) F1 = ____

Answer: Precision = 0.778; Recall = 0.651; F1 = 0.709

Working:

- Precision = TP/(TP+FP) = 56/72 = 0.7778.
- Recall = TP/(TP+FN) = 56/86 = 0.6512.
- F1 = 2PR/(P+R) = 2×0.7778×0.6512/(0.7778+0.6512) = 0.7089.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B148. [BLANKS · Hard · calculation]

| Actual + | Actual −
Predicted + | TP = 49 | FP = 26
Predicted − | FN = 22 | TN = 44

Give specificity (TNR) and balanced accuracy = (TPR + TNR)/2 (3 dp).

(1) Specificity = ____
(2) Balanced accuracy = ____

Answer: Specificity = 0.629; Balanced accuracy = 0.659

Working:

- Specificity = TN/(TN+FP) = 44/70 = 0.6286.
- TPR (recall) = 0.6901.
- Balanced accuracy = (0.6901 + 0.6286)/2 = 0.6594.

Takeaway: FPR = 1 − specificity is the x-axis of the ROC curve.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B149. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 44 | FP = 29
Predicted − | FN = 25 | TN = 75

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____
(3) F1 = ____

Answer: Precision = 0.603; Recall = 0.638; F1 = 0.620

Working:

- Precision = TP/(TP+FP) = 44/73 = 0.6027.
- Recall = TP/(TP+FN) = 44/69 = 0.6377.
- F1 = 2PR/(P+R) = 2×0.6027×0.6377/(0.6027+0.6377) = 0.6197.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B150. [BLANKS · Hard · calculation]

| Actual + | Actual −
Predicted + | TP = 58 | FP = 22
Predicted − | FN = 12 | TN = 38

Give specificity (TNR) and balanced accuracy = (TPR + TNR)/2 (3 dp).

(1) Specificity = ____
(2) Balanced accuracy = ____

Answer: Specificity = 0.633; Balanced accuracy = 0.731

Working:

- Specificity = TN/(TN+FP) = 38/60 = 0.6333.
- TPR (recall) = 0.8286.
- Balanced accuracy = (0.8286 + 0.6333)/2 = 0.7310.

Takeaway: FPR = 1 − specificity is the x-axis of the ROC curve.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B151. [BLANKS · Hard · calculation]

| Actual + | Actual −
Predicted + | TP = 46 | FP = 5
Predicted − | FN = 15 | TN = 52

Give specificity (TNR) and balanced accuracy = (TPR + TNR)/2 (3 dp).

(1) Specificity = ____
(2) Balanced accuracy = ____

Answer: Specificity = 0.912; Balanced accuracy = 0.833

Working:

- Specificity = TN/(TN+FP) = 52/57 = 0.9123.

- TPR (recall) = 0.7541.
- Balanced accuracy = (0.7541 + 0.9123)/2 = 0.8332.

Takeaway: FPR = 1 − specificity is the x-axis of the ROC curve.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B152. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 18 | FP = 10
Predicted − | FN = 16 | TN = 15

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____
(3) F1 = ____

Answer: Precision = 0.643; Recall = 0.529; F1 = 0.581

Working:

- Precision = TP/(TP+FP) = 18/28 = 0.6429.
- Recall = TP/(TP+FN) = 18/34 = 0.5294.
- F1 = 2PR/(P+R) = 2×0.6429×0.5294/(0.6429+0.5294) = 0.5806.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B153. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 27 | FP = 27
Predicted − | FN = 26 | TN = 25

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____
(3) F1 = ____

Answer: Precision = 0.500; Recall = 0.509; F1 = 0.505

Working:

- Precision = TP/(TP+FP) = 27/54 = 0.5000.
- Recall = TP/(TP+FN) = 27/53 = 0.5094.
- F1 = 2PR/(P+R) = 2×0.5000×0.5094/(0.5000+0.5094) = 0.5047.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B154. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 60 | FP = 13

Predicted − | FN = 28 | TN = 61

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____
(3) F1 = ____

Answer: Precision = 0.822; Recall = 0.682; F1 = 0.745

Working:

- Precision = TP/(TP+FP) = 60/73 = 0.8219.
- Recall = TP/(TP+FN) = 60/88 = 0.6818.
- F1 = 2PR/(P+R) = 2×0.8219×0.6818/(0.8219+0.6818) = 0.7453.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B155. [MCQ · Easy · calculation]

| Actual + | Actual −
Predicted + | TP = 46 | FP = 14
Predicted − | FN = 17 | TN = 42

What is the accuracy of this classifier?

A. 1.467
B. 0.739
C. 0.767
D. 0.730

Answer: B. 0.739

Working:

- Accuracy = (TP+TN)/All = (46+42)/119 = 0.7395.

Option notes:

- A: Wrong denominator: accuracy divides by ALL examples.
- C: That is precision.
- D: That is recall.

Takeaway: Accuracy can mislead with imbalanced classes.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B156. [MCQ · Easy · calculation]

| Actual + | Actual −
Predicted + | TP = 46 | FP = 23
Predicted − | FN = 17 | TN = 83

What is the accuracy of this classifier?

A. 0.730
B. 1.870
C. 0.763
D. 0.667

Answer: C. 0.763

Working:

- Accuracy = (TP+TN)/All = (46+83)/169 = 0.7633.

Option notes:

- A: That is recall.
- B: Wrong denominator: accuracy divides by ALL examples.
- D: That is precision.

Takeaway: Accuracy can mislead with imbalanced classes.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B157. [BLANKS · Hard · calculation]

| Actual + | Actual −
Predicted + | TP = 38 | FP = 17
Predicted − | FN = 18 | TN = 29

Give specificity (TNR) and balanced accuracy = (TPR + TNR)/2 (3 dp).

(1) Specificity = ____
(2) Balanced accuracy = ____

Answer: Specificity = 0.630; Balanced accuracy = 0.655

Working:

- Specificity = TN/(TN+FP) = 29/46 = 0.6304.
- TPR (recall) = 0.6786.
- Balanced accuracy = (0.6786 + 0.6304)/2 = 0.6545.

Takeaway: FPR = 1 − specificity is the x-axis of the ROC curve.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B158. [MCQ · Easy · calculation]

| Actual + | Actual −
Predicted + | TP = 30 | FP = 4
Predicted − | FN = 24 | TN = 80

What is the accuracy of this classifier?

A. 0.556
B. 0.797
C. 3.235
D. 0.882

Answer: B. 0.797

Working:

- Accuracy = (TP+TN)/All = (30+80)/138 = 0.7971.

Option notes:

- A: That is recall.
- C: Wrong denominator: accuracy divides by ALL examples.
- D: That is precision.

Takeaway: Accuracy can mislead with imbalanced classes.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B159. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 51 | FP = 18
Predicted − | FN = 26 | TN = 11

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____
(3) F1 = ____

Answer: Precision = 0.739; Recall = 0.662; F1 = 0.699

Working:

- Precision = TP/(TP+FP) = 51/69 = 0.7391.
- Recall = TP/(TP+FN) = 51/77 = 0.6623.
- F1 = 2PR/(P+R) = 2×0.7391×0.6623/(0.7391+0.6623) = 0.6986.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B160. [BLANKS · Medium · calculation]

| Actual + | Actual −
Predicted + | TP = 56 | FP = 5
Predicted − | FN = 7 | TN = 85

From the confusion matrix (rows = predicted, columns = actual, as in Lecture 4), give precision, recall and F1 (3 dp).

(1) Precision = ____
(2) Recall = ____
(3) F1 = ____

Answer: Precision = 0.918; Recall = 0.889; F1 = 0.903

Working:

- Precision = TP/(TP+FP) = 56/61 = 0.9180.
- Recall = TP/(TP+FN) = 56/63 = 0.8889.
- F1 = 2PR/(P+R) = 2×0.9180×0.8889/(0.9180+0.8889) = 0.9032.

Takeaway: Precision looks along the PREDICTED-positive row; recall looks down the ACTUAL-positive column.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B161. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 10 | 1 | 188
P2 | 6 | 5 | 274
P3 | 4 | 10 | 338
P4 | 9 | 6 | 145
P5 | 7 | 4 | 149

P6 | 5 | 9 | 248
P7 | 2 | 10 | 365

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (5, 8).

Answer: 286.67

Working:

- Euclidean distances to the query (5, 8):
- Point | distance
  P6 | 1.000
  P3 | 2.236
  P2 | 3.162
  P7 | 3.606
  P4 | 4.472
  P5 | 4.472
  P1 | 8.602
- 3 nearest: P6, P3, P2.
- Prediction = average of their targets = (248 + 338 + 274)/3 = 286.67.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B162. [MCQ · Medium · calculation]

Point | x_1 | x_2 | Class
P1 | 1 | 8 | Red
P2 | 2 | 5 | Red
P3 | 8 | 4 | Blue
P4 | 3 | 4 | Blue
P5 | 8 | 7 | Red
P6 | 2 | 7 | Blue
P7 | 0 | 0 | Blue

KNN classification with k = 3 and Euclidean distance. What class is predicted for the query (6, 4)?

A. The class of the single nearest point only
B. Blue
C. Cannot decide (tie)
D. Red

Answer: B. Blue

Working:

- Euclidean distances to the query (6, 4):
- Point | distance
  P3 | 2.000
  P4 | 3.000
  P5 | 3.606
  P2 | 4.123
  P6 | 5.000
  P1 | 6.403
  P7 | 7.211
- 3 nearest: P3, P4, P5.
- Votes: {"Blue":2,"Red":1} → Blue.

Option notes:

- A: That is 1-NN, not 3-NN.
- C: With k = 3 and two classes there is always a majority.
- D: Count the votes among the 3 nearest only.

Takeaway: Scale features before KNN: distances are dominated by large-range features.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B163. [MCQ · Medium · calculation]

Point | x_1 | x_2 | Class
P1 | 10 | 9 | Red
P2 | 7 | 5 | Blue
P3 | 4 | 5 | Red
P4 | 1 | 3 | Blue
P5 | 6 | 1 | Blue
P6 | 6 | 6 | Blue
P7 | 0 | 6 | Red

KNN classification with k = 3 and Euclidean distance. What class is predicted for the query (7, 6)?

A. Cannot decide (tie)
B. Blue
C. Red
D. The class of the single nearest point only

Answer: B. Blue

Working:

- Euclidean distances to the query (7, 6):
- Point | distance
  P2 | 1.000
  P6 | 1.000
  P3 | 3.162
  P1 | 4.243
  P5 | 5.099
  P4 | 6.708
  P7 | 7.000
- 3 nearest: P2, P6, P3.
- Votes: {"Blue":2,"Red":1} → Blue.

Option notes:

- A: With k = 3 and two classes there is always a majority.
- C: Count the votes among the 3 nearest only.
- D: That is 1-NN, not 3-NN.

Takeaway: Scale features before KNN: distances are dominated by large-range features.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B164. [MCQ · Medium · calculation]

Point | x_1 | x_2 | Class
P1 | 0 | 0 | Red
P2 | 0 | 8 | Red

P3 | 7 | 10 | Red
P4 | 1 | 8 | Red
P5 | 6 | 5 | Red
P6 | 2 | 7 | Red
P7 | 7 | 2 | Red

KNN classification with k = 3 and Euclidean distance. What class is predicted for the query (8, 7)?

A. Blue
B. The class of the single nearest point only
C. Red
D. Cannot decide (tie)

Answer: C. Red

Working:

- Euclidean distances to the query (8, 7):
- Point | distance
  P5 | 2.828
  P3 | 3.162
  P7 | 5.099
  P6 | 6.000
  P4 | 7.071
  P2 | 8.062
  P1 | 10.630
- 3 nearest: P5, P3, P7.
- Votes: {"Red":3} → Red.

Option notes:

- A: Count the votes among the 3 nearest only.
- B: That is 1-NN, not 3-NN.
- D: With k = 3 and two classes there is always a majority.

Takeaway: Scale features before KNN: distances are dominated by large-range features.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B165. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 9 | 8 | 185
P2 | 3 | 7 | 323
P3 | 0 | 10 | 101
P4 | 6 | 7 | 187
P5 | 6 | 9 | 304
P6 | 7 | 0 | 185
P7 | 9 | 7 | 373

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (4, 4).

Answer: 231.67

Working:

- Euclidean distances to the query (4, 4):
- Point | distance
  P2 | 3.162
  P4 | 3.606

P6 | 5.000
P5 | 5.385
P7 | 5.831
P1 | 6.403
P3 | 7.211

- 3 nearest: P2, P4, P6.
- Prediction = average of their targets = (323 + 187 + 185)/3 = 231.67.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B166. [MCQ · Medium · calculation]

Point | x_1 | x_2 | Class
P1 | 2 | 3 | Blue
P2 | 0 | 9 | Red
P3 | 5 | 3 | Red
P4 | 5 | 0 | Red
P5 | 6 | 3 | Blue
P6 | 4 | 3 | Red
P7 | 1 | 4 | Blue

KNN classification with k = 3 and Euclidean distance. What class is predicted for the query (6, 7)?

A. The class of the single nearest point only
B. Blue
C. Red
D. Cannot decide (tie)

Answer: C. Red

Working:

- Euclidean distances to the query (6, 7):
- Point | distance
  P5 | 4.000
  P3 | 4.123
  P6 | 4.472
  P1 | 5.657
  P7 | 5.831
  P2 | 6.325
  P4 | 7.071
- 3 nearest: P5, P3, P6.
- Votes: {"Blue":1,"Red":2} → Red.

Option notes:

- A: That is 1-NN, not 3-NN.
- B: Count the votes among the 3 nearest only.
- D: With k = 3 and two classes there is always a majority.

Takeaway: Scale features before KNN: distances are dominated by large-range features.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B167. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 5 | 7 | 128
P2 | 9 | 9 | 312
P3 | 5 | 3 | 378
P4 | 2 | 0 | 358
P5 | 1 | 8 | 134
P6 | 4 | 8 | 192
P7 | 6 | 2 | 166

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (4, 5).

Answer: 232.67

Working:

- Euclidean distances to the query (4, 5):
- Point | distance
  P1 | 2.236
  P3 | 2.236
  P6 | 3.000
  P7 | 3.606
  P5 | 4.243
  P4 | 5.385
  P2 | 6.403
- 3 nearest: P1, P3, P6.
- Prediction = average of their targets = (128 + 378 + 192)/3 = 232.67.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B168. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 2 | 3 | 146
P2 | 10 | 0 | 278
P3 | 5 | 1 | 227
P4 | 1 | 3 | 351
P5 | 10 | 1 | 159
P6 | 8 | 0 | 172
P7 | 4 | 5 | 176

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (6, 5).

Answer: 183.00

Working:

- Euclidean distances to the query (6, 5):
- Point | distance
  P7 | 2.000
  P3 | 4.123
  P1 | 4.472
  P4 | 5.385
  P6 | 5.385

P5 | 5.657
P2 | 6.403

- 3 nearest: P7, P3, P1.
- Prediction = average of their targets = (176 + 227 + 146)/3 = 183.00.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B169. [MCQ · Medium · calculation]

Point | x_1 | x_2 | Class
P1 | 7 | 3 | Red
P2 | 2 | 7 | Red
P3 | 8 | 4 | Red
P4 | 1 | 7 | Blue
P5 | 5 | 10 | Red
P6 | 9 | 6 | Red
P7 | 5 | 5 | Red

KNN classification with k = 3 and Euclidean distance. What class is predicted for the query (2, 6)?

A. The class of the single nearest point only
B. Cannot decide (tie)
C. Red
D. Blue

Answer: C. Red

Working:

- Euclidean distances to the query (2, 6):
- Point | distance
  P2 | 1.000
  P4 | 1.414
  P7 | 3.162
  P5 | 5.000
  P1 | 5.831
  P3 | 6.325
  P6 | 7.000
- 3 nearest: P2, P4, P7.
- Votes: {"Red":2,"Blue":1} → Red.

Option notes:

- A: That is 1-NN, not 3-NN.
- B: With k = 3 and two classes there is always a majority.
- D: Count the votes among the 3 nearest only.

Takeaway: Scale features before KNN: distances are dominated by large-range features.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B170. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 4 | 3 | 156
P2 | 3 | 2 | 126
P3 | 1 | 0 | 240

P4 | 3 | 10 | 337
P5 | 5 | 2 | 345
P6 | 8 | 9 | 358
P7 | 8 | 5 | 364

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (3, 5).

Answer: 209.00

Working:

- Euclidean distances to the query (3, 5):
- Point | distance
  P1 | 2.236
  P2 | 3.000
  P5 | 3.606
  P4 | 5.000
  P7 | 5.000
  P3 | 5.385
  P6 | 6.403
- 3 nearest: P1, P2, P5.
- Prediction = average of their targets = (156 + 126 + 345)/3 = 209.00.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B171. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 10 | 9 | 365
P2 | 6 | 4 | 352
P3 | 7 | 1 | 259
P4 | 2 | 3 | 269
P5 | 3 | 9 | 176
P6 | 10 | 1 | 379
P7 | 3 | 1 | 128

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (7, 4).

Answer: 330.00

Working:

- Euclidean distances to the query (7, 4):
- Point | distance
  P2 | 1.000
  P3 | 3.000
  P6 | 4.243
  P7 | 5.000
  P4 | 5.099
  P1 | 5.831
  P5 | 6.403
- 3 nearest: P2, P3, P6.
- Prediction = average of their targets = (352 + 259 + 379)/3 = 330.00.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B172. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 5 | 3 | 143
P2 | 4 | 5 | 273
P3 | 6 | 0 | 199
P4 | 10 | 9 | 388
P5 | 8 | 6 | 300
P6 | 5 | 2 | 374
P7 | 9 | 10 | 218

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (8, 2).

Answer: 238.67

Working:

- Euclidean distances to the query (8, 2):
- Point | distance
  P3 | 2.828
  P6 | 3.000
  P1 | 3.162
  P5 | 4.000
  P2 | 5.000
  P4 | 7.280
  P7 | 8.062
- 3 nearest: P3, P6, P1.
- Prediction = average of their targets = (199 + 374 + 143)/3 = 238.67.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B173. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 4 | 7 | 220
P2 | 3 | 0 | 303
P3 | 10 | 0 | 162
P4 | 8 | 1 | 239
P5 | 1 | 9 | 364
P6 | 1 | 6 | 371
P7 | 9 | 2 | 361

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (4, 5).

Answer: 318.33

Working:

- Euclidean distances to the query (4, 5):
- Point | distance
  P1 | 2.000
  P6 | 3.162
  P5 | 5.000
  P2 | 5.099
  P4 | 5.657

P7 | 5.831
P3 | 7.810

- 3 nearest: P1, P6, P5.
- Prediction = average of their targets = (220 + 371 + 364)/3 = 318.33.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B174. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 5 | 4 | 299
P2 | 7 | 5 | 192
P3 | 1 | 0 | 297
P4 | 10 | 5 | 215
P5 | 10 | 9 | 186
P6 | 7 | 10 | 279
P7 | 9 | 2 | 209

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (6, 4).

Answer: 233.33

Working:

- Euclidean distances to the query (6, 4):
- Point | distance
  P1 | 1.000
  P2 | 1.414
  P7 | 3.606
  P4 | 4.123
  P6 | 6.083
  P3 | 6.403
  P5 | 6.403
- 3 nearest: P1, P2, P7.
- Prediction = average of their targets = (299 + 192 + 209)/3 = 233.33.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B175. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 7 | 0 | 309
P2 | 0 | 5 | 208
P3 | 6 | 10 | 125
P4 | 2 | 8 | 328
P5 | 10 | 10 | 271
P6 | 1 | 5 | 196
P7 | 2 | 6 | 171

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (6, 2).

Answer: 225.33

Working:

- Euclidean distances to the query (6, 2):

- Point | distance
  P1 | 2.236
  P7 | 5.657
  P6 | 5.831
  P2 | 6.708
  P4 | 7.211
  P3 | 8.000
  P5 | 8.944
- 3 nearest: P1, P7, P6.
- Prediction = average of their targets = (309 + 171 + 196)/3 = 225.33.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B176. [BLANK · Medium · calculation]

Point | x_1 | x_2 | Target value
P1 | 6 | 8 | 374
P2 | 3 | 7 | 222
P3 | 5 | 8 | 224
P4 | 6 | 4 | 226
P5 | 9 | 5 | 171
P6 | 4 | 6 | 391
P7 | 0 | 9 | 206

KNN regression with k = 3 and Euclidean distance. Predict the target for the query point (8, 2).

Answer: 262.67

Working:

- Euclidean distances to the query (8, 2):
- Point | distance
  P4 | 2.828
  P5 | 3.162
  P6 | 5.657
  P1 | 6.325
  P3 | 6.708
  P2 | 7.071
  P7 | 10.630
- 3 nearest: P4, P5, P6.
- Prediction = average of their targets = (226 + 171 + 391)/3 = 262.67.

Takeaway: KNN regressor: average (or distance-weighted average) of the k nearest targets.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B177. [BLANKS · Hard · calculation]

A linear SVM has w = (1, 1.5) and b = 2. For x = (-2, 3) give (1) w·x + b, (2) the predicted class sign(w·x + b) as 1 or -1, (3) the margin width 2/‖w‖ (3 dp).

(1) w·x + b = ____
(2) class = ____
(3) margin 2/‖w‖ = ____

Answer: w·x + b = 4.5; class = 1; margin 2/‖w‖ = 1.109

Working:

- $w \cdot x + b = 1 \times -2 + 1.5 \times 3 + 2 = 4.5$.
- sign → 1.
- $\|w\| = \sqrt{(1^2 + 1.5^2)} = 1.8028$; margin = $2/\|w\| = 1.1094$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B178. [BLANKS · Hard · calculation]

A linear SVM has w = (-0.5, 0.5) and b = -3. For x = (4, -1) give (1) $w \cdot x + b$, (2) the predicted class sign($w \cdot x + b$) as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = -5.5$; class = -1; margin $2/\|w\| = 2.828$

Working:

- $w \cdot x + b = -0.5 \times 4 + 0.5 \times -1 + -3 = -5.5$.
- sign → -1.
- $\|w\| = \sqrt{(-0.5^2 + 0.5^2)} = 0.7071$; margin = $2/\|w\| = 2.8284$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B179. [BLANKS · Hard · calculation]

A linear SVM has w = (-0.5, 1.5) and b = 2. For x = (5, 3) give (1) $w \cdot x + b$, (2) the predicted class sign($w \cdot x + b$) as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = 4$; class = 1; margin $2/\|w\| = 1.265$

Working:

- $w \cdot x + b = -0.5 \times 5 + 1.5 \times 3 + 2 = 4$.
- sign → 1.
- $\|w\| = \sqrt{(-0.5^2 + 1.5^2)} = 1.5811$; margin = $2/\|w\| = 1.2649$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B180. [BLANKS · Hard · calculation]

A linear SVM has w = (2, 1.5) and b = -4. For x = (-2, -2) give (1) $w \cdot x + b$, (2) the predicted class sign($w \cdot x + b$) as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = -11$; class = -1; margin $2/\|w\| = 0.800$

Working:

- $w \cdot x + b = 2 \times -2 + 1.5 \times -2 + -4 = -11$.
- sign → -1.
- $\|w\| = \sqrt{(2^2 + 1.5^2)} = 2.5000$; margin = $2/\|w\| = 0.8000$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B181. [BLANKS · Hard · calculation]

A linear SVM has w = (1, 1) and b = -4. For x = (3, 3) give (1) $w \cdot x + b$, (2) the predicted class sign($w \cdot x + b$) as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = 2$; class = 1; margin $2/\|w\| = 1.414$

Working:

- $w \cdot x + b = 1 \times 3 + 1 \times 3 + -4 = 2$.
- sign → 1.
- $\|w\| = \sqrt{(1^2 + 1^2)} = 1.4142$; margin = $2/\|w\| = 1.4142$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B182. [BLANKS · Hard · calculation]

A linear SVM has w = (-0.5, 1.5) and b = -3. For x = (0, -2) give (1) $w \cdot x + b$, (2) the predicted class sign($w \cdot x + b$) as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = -6$; class = -1; margin $2/\|w\| = 1.265$

Working:

- $w \cdot x + b = -0.5 \times 0 + 1.5 \times -2 + -3 = -6$.
- sign → -1.
- $\|w\| = \sqrt{(-0.5^2 + 1.5^2)} = 1.5811$; margin = $2/\|w\| = 1.2649$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B183. [BLANKS · Hard · calculation]

A linear SVM has w = (0.5, 1) and b = -1. For x = (3, -2) give (1) $w \cdot x + b$, (2) the predicted class sign($w \cdot x + b$) as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = -1.5$; class = -1; margin $2/\|w\| = 1.789$

Working:

- $w \cdot x + b = 0.5 \times 3 + 1 \times -2 + -1 = -1.5$.

- sign → -1.
- ‖w‖ = √(0.5² + 1²) = 1.1180; margin = 2/‖w‖ = 1.7889.

Takeaway: SVM maximises the margin 2/‖w‖ by minimising ½‖w‖². Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B184. [BLANKS · Hard · calculation]

A linear SVM has w = (0.5, 1) and b = -2. For x = (3, 4) give (1) w·x + b, (2) the predicted class sign(w·x + b) as 1 or -1, (3) the margin width 2/‖w‖ (3 dp).

(1) w·x + b = ____
(2) class = ____
(3) margin 2/‖w‖ = ____

Answer: w·x + b = 3.5; class = 1; margin 2/‖w‖ = 1.789

Working:

- w·x + b = 0.5×3 + 1×4 + -2 = 3.5.
- sign → 1.
- ‖w‖ = √(0.5² + 1²) = 1.1180; margin = 2/‖w‖ = 1.7889.

Takeaway: SVM maximises the margin 2/‖w‖ by minimising ½‖w‖². Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B185. [BLANKS · Hard · calculation]

A linear SVM has w = (0.5, 1) and b = 0. For x = (0, -1) give (1) w·x + b, (2) the predicted class sign(w·x + b) as 1 or -1, (3) the margin width 2/‖w‖ (3 dp).

(1) w·x + b = ____
(2) class = ____
(3) margin 2/‖w‖ = ____

Answer: w·x + b = -1; class = -1; margin 2/‖w‖ = 1.789

Working:

- w·x + b = 0.5×0 + 1×-1 + 0 = -1.
- sign → -1.
- ‖w‖ = √(0.5² + 1²) = 1.1180; margin = 2/‖w‖ = 1.7889.

Takeaway: SVM maximises the margin 2/‖w‖ by minimising ½‖w‖². Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B186. [BLANKS · Hard · calculation]

A linear SVM has w = (0.5, 1) and b = -2. For x = (0, 0) give (1) w·x + b, (2) the predicted class sign(w·x + b) as 1 or -1, (3) the margin width 2/‖w‖ (3 dp).

(1) w·x + b = ____
(2) class = ____
(3) margin 2/‖w‖ = ____

Answer: w·x + b = -2; class = -1; margin 2/‖w‖ = 1.789

Working:

- w·x + b = 0.5×0 + 1×0 + -2 = -2.
- sign → -1.

- $\|w\| = \sqrt{(0.5^2 + 1^2)} = 1.1180$; margin $= 2/\|w\| = 1.7889$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B187. [BLANKS · Hard · calculation]

A linear SVM has w = (2, 1) and b = 2. For x = (5, 2) give (1) $w \cdot x + b$, (2) the predicted class $\text{sign}(w \cdot x + b)$ as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = 14$; class = 1; margin $2/\|w\| = 0.894$

Working:

- $w \cdot x + b = 2 \times 5 + 1 \times 2 + 2 = 14$.
- sign $\rightarrow$ 1.
- $\|w\| = \sqrt{(2^2 + 1^2)} = 2.2361$; margin $= 2/\|w\| = 0.8944$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B188. [BLANKS · Hard · calculation]

A linear SVM has w = (2, 0.5) and b = -1. For x = (0, 1) give (1) $w \cdot x + b$, (2) the predicted class $\text{sign}(w \cdot x + b)$ as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = -0.5$; class = -1; margin $2/\|w\| = 0.970$

Working:

- $w \cdot x + b = 2 \times 0 + 0.5 \times 1 + -1 = -0.5$.
- sign $\rightarrow$ -1.
- $\|w\| = \sqrt{(2^2 + 0.5^2)} = 2.0616$; margin $= 2/\|w\| = 0.9701$.

Takeaway: SVM maximises the margin $2/\|w\|$ by minimising $\frac{1}{2}\|w\|^2$. Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B189. [BLANKS · Hard · calculation]

A linear SVM has w = (-0.5, 1.5) and b = 2. For x = (2, 4) give (1) $w \cdot x + b$, (2) the predicted class $\text{sign}(w \cdot x + b)$ as 1 or -1, (3) the margin width $2/\|w\|$ (3 dp).

(1) $w \cdot x + b$ = ____
(2) class = ____
(3) margin $2/\|w\|$ = ____

Answer: $w \cdot x + b = 7$; class = 1; margin $2/\|w\| = 1.265$

Working:

- $w \cdot x + b = -0.5 \times 2 + 1.5 \times 4 + 2 = 7$.
- sign $\rightarrow$ 1.
- $\|w\| = \sqrt{(-0.5^2 + 1.5^2)} = 1.5811$; margin $= 2/\|w\| = 1.2649$.

Takeaway: SVM maximises the margin 2/‖w‖ by minimising ½‖w‖². Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B190. [BLANKS · Hard · calculation]

A linear SVM has w = (-0.5, 0.5) and b = -1. For x = (1, 3) give (1) w·x + b, (2) the predicted class sign(w·x + b) as 1 or -1, (3) the margin width 2/‖w‖ (3 dp).

(1) w·x + b = ____
(2) class = ____
(3) margin 2/‖w‖ = ____

Answer: w·x + b = 0; class = 1; margin 2/‖w‖ = 2.828

Working:

- w·x + b = -0.5×1 + 0.5×3 + -1 = 0.
- sign → 1.
- ‖w‖ = √(-0.5² + 0.5²) = 0.7071; margin = 2/‖w‖ = 2.8284.

Takeaway: SVM maximises the margin 2/‖w‖ by minimising ½‖w‖². Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B191. [BLANKS · Hard · calculation]

A linear SVM has w = (-0.5, 0.5) and b = 0. For x = (-2, -2) give (1) w·x + b, (2) the predicted class sign(w·x + b) as 1 or -1, (3) the margin width 2/‖w‖ (3 dp).

(1) w·x + b = ____
(2) class = ____
(3) margin 2/‖w‖ = ____

Answer: w·x + b = 0; class = 1; margin 2/‖w‖ = 2.828

Working:

- w·x + b = -0.5×-2 + 0.5×-2 + 0 = 0.
- sign → 1.
- ‖w‖ = √(-0.5² + 0.5²) = 0.7071; margin = 2/‖w‖ = 2.8284.

Takeaway: SVM maximises the margin 2/‖w‖ by minimising ½‖w‖². Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

ML B192. [BLANKS · Hard · calculation]

A linear SVM has w = (-0.5, 1.5) and b = -1. For x = (-1, 3) give (1) w·x + b, (2) the predicted class sign(w·x + b) as 1 or -1, (3) the margin width 2/‖w‖ (3 dp).

(1) w·x + b = ____
(2) class = ____
(3) margin 2/‖w‖ = ____

Answer: w·x + b = 4; class = 1; margin 2/‖w‖ = 1.265

Working:

- w·x + b = -0.5×-1 + 1.5×3 + -1 = 4.
- sign → 1.
- ‖w‖ = √(-0.5² + 1.5²) = 1.5811; margin = 2/‖w‖ = 1.2649.

Takeaway: SVM maximises the margin 2/‖w‖ by minimising ½‖w‖². Support vectors sit on the margin.

Source: Lecture 4 (Supervised learning: metrics, regression, gradient descent, KNN, SVM)

🏰 MDPs & Reinforcement Learning (Workshop 6)

Part A: hand-written questions (104)

MDP A1. [MCQ · Easy · calculation]

Route A: 80% U = +10, 20% U = −10. Route B: 100% U = +7. Under MEU the agent chooses…

A. Route B (EU 7 > 6)
B. Route A, because its best outcome is +10
C. Route A (EU 8)
D. Either; they are equal

Answer: A. Route B (EU 7 > 6)

Option notes:

- B: MEU uses expectation, not best case.
- C: EU(A) = 8 − 2 = 6.
- D: 6 ≠ 7.

Takeaway: EU(A) = 6, EU(B) = 7.

Source: Workshop 6, slide 10

MDP A2. [TF · Medium]

The Markov assumption says the current state depends on ALL previous states.

Answer: False

Why: It depends only on a finite fixed number of previous states (first-order: only the last).

Takeaway: Markov = short memory.

Source: Workshop 6, slide 6

MDP A3. [BLANK · Easy]

In decision theory, a _____ function U(s) assigns a single number expressing the desirability of state s.

Answer: utility
(Also accepted: Utility)

Takeaway: Utility = desirability.

Source: Workshop 6, slide 7

MDP A4. [MULTI · Medium]

Select ALL correct statements (Workshop 6 background). (Select ALL that apply.)

A. MEU picks $\text{argmax}_a \Sigma P(s'|a,s) U(s')$
B. In a non-deterministic environment the outcome of an action is a random variable
C. E_t denotes observable variables and X_t unobservable ones
D. The MEU principle chooses the action whose best outcome is highest

Answer: A. MEU picks $\text{argmax}_a \Sigma P(s'|a,s) U(s')$; B. In a non-deterministic environment the outcome of an action is a random variable; C. E_t denotes observable variables and X_t unobservable ones

Option notes:

- A: Correct: Definition.

- B: Correct: Described by P(s′|s,a).
- C: Correct: Slide 5.
- D: Incorrect: Expected, not best.

Takeaway: Uncertain outcomes → expectation.

Source: Workshop 6, slide 5-9

MDP A5. [MCQ · Easy]

An MDP is a sequential decision problem for which kind of environment?

A. Partially observable and deterministic
B. Fully observable, stochastic, Markovian transitions, additive rewards
C. Unknown and non-stationary only
D. Fully observable and deterministic

Answer: B. Fully observable, stochastic, Markovian transitions, additive rewards

Option notes:

- A: MDPs assume full observability (partial → POMDP).
- C: Not the definition.
- D: Deterministic is classical search.

Takeaway: Know the four properties.

Source: Workshop 6, slide 18

MDP A6. [MATCH · Medium]

Match the MDP element to its grid-world instance.

1. Transition model P(s′|s,a)
2. Reward function
3. Actions
4. Terminal states

A. γ = 0.9 always
B. Up, Down, Left, Right
C. (4,3) and (4,2)
D. −0.04 per step, +1/−1 at terminals
E. 0.8 intended, 0.1 left, 0.1 right

Answer: 1 → E, 2 → D, 3 → B, 4 → C

Working:

- Transition model P(s′|s,a) → 0.8 intended, 0.1 left, 0.1 right
- Reward function → −0.04 per step, +1/−1 at terminals
- Actions → Up, Down, Left, Right
- Terminal states → (4,3) and (4,2)

Takeaway: States, actions, transitions, rewards.

Source: Workshop 6, slide 20-21

MDP A7. [TF · Medium]

MDP and RL are the same thing: both learn a policy from trial and error.

Answer: False

Why: MDP DEFINES the task (states, actions, P, R); RL LEARNS how to act, useful when the model or policy is unknown.

Takeaway: MDP = problem; RL = method.

Source: Workshop 6, slide 15

MDP A8. [MULTI · Medium]

Which are key characteristics of sequential decision problems (Workshop 6)? (Select ALL that apply.)

A. Sequential nature
B. An objective/utility to optimise
C. Single one-shot decision
D. Uncertainty
E. State and transition dynamics

Answer: A. Sequential nature; B. An objective/utility to optimise; D. Uncertainty; E. State and transition dynamics

Option notes:

- A: Correct: Order/timing matter.
- B: Correct: Goal.
- C: Incorrect: It's a SERIES of decisions.
- D: Correct: Randomness, incomplete info.
- E: Correct: State evolves.

Takeaway: Five characteristics incl. policy.

Source: Workshop 6, slide 13

MDP A9. [MCQ · Medium]

| | | +1
| ▓ wall | | −1
START | | |

In the 4×3 grid world the agent chooses Up. With what probabilities does it move?

A. 0.8 up, 0.2 down
B. 0.8 up, 0.1 left, 0.1 right
C. 0.7 up, 0.1 each other direction
D. 1.0 up

Answer: B. 0.8 up, 0.1 left, 0.1 right

Option notes:

- A: Slips are perpendicular, not backwards.
- C: Not the Workshop 6 model.
- D: That's the deterministic world.

Takeaway: Perpendicular slips.

Source: Workshop 6, slide 20

MDP A10. [BLANK · Hard · calculation]

| | | +1
| ▓ wall | | −1
START | | |

Grid world, γ = 1, living reward −0.04. From s, action Up reaches a cell with U = 0.9 (prob 0.8), slips left into a wall (stays in s, U(s) = 0.7, prob 0.1) and slips right to U = 0.6 (prob 0.1). Q(s, Up) = _____ (3 dp).

Answer: 0.810
(Also accepted: 0.81)

Working:

- Up: 0.8×(−0.04 + 0.9) = 0.8×0.86 = 0.688
- Left slip (wall, stay): 0.1×(−0.04 + 0.7) = 0.066
- Right slip: 0.1×(−0.04 + 0.6) = 0.056
- Q(s, Up) = 0.688 + 0.066 + 0.056 = 0.810

Takeaway: Each branch: probability × (step reward + next utility).

Source: Workshop 6 (Reinforcement learning)

MDP A11. [MCQ · Easy · calculation]

Rewards 1, 2, 3 (in order) with γ = 0.9. Discounted return?

A. 5.4
B. 4.707
C. 5.23
D. 6

Answer: C. 5.23

Option notes:

- A: Used 0.9 on every reward.
- B: Discounted the first reward too.
- D: No discount.

Takeaway: 1 + 1.8 + 2.43.

Source: Workshop 6 (Reinforcement learning)

MDP A12. [TF · Medium]

With discount factor γ = 1, the discounted reward reduces to the additive reward.

Answer: True

Why: Workshop 6, slide 24.

Takeaway: γ = 1: no discount.

Source: Workshop 6, slide 24

MDP A13. [MATCH · Medium]

Match the MDP search-tree element to its meaning.

1. s (triangle)
2. (s, a) (circle)
3. (s, a, s′)

A. a q-state
B. a state
C. a transition with T = P(s′|s,a) and reward R(s,a,s′)
D. a terminal utility

Answer: 1 → B, 2 → A, 3 → C

Working:

- s (triangle) → a state
- (s, a) (circle) → a q-state
- (s, a, s′) → a transition with T = P(s′|s,a) and reward R(s,a,s′)

Takeaway: State → q-state → transition.

Source: Workshop 6, slide 27

MDP A14. [MCQ · Medium]

What is a policy in an MDP?

A. The transition probabilities
B. A fixed sequence of actions from the start
C. The reward function
D. A mapping that gives an action for every state the agent might reach

Answer: D. A mapping that gives an action for every state the agent might reach

Option notes:

- A: That's the model.
- B: That's a classical-search solution.
- C: Rewards are given by the environment.

Takeaway: π(s) = recommended action in s.

Source: Workshop 6, slide 26

MDP A15. [BLANK · Hard · calculation]

Rewards 0, 0, 10 (in order) with γ = 0.5. The discounted return G = _____.

Answer: 2.5
(Also accepted: 2.50)

Takeaway: 0 + 0.5×0 + 0.25×10.

Source: Workshop 6 (Reinforcement learning)

MDP A16. [MCQ · Medium]

Which equation is the Bellman equation for utilities in Workshop 6?

A. $U(s) = \max_a R(s,a) + \gamma U(s)$
B. $U(s) = \Sigma_a \Sigma_{s'} [R + \gamma U(s')]$
C. $U(s) = \max_a \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$
D. $U(s) = \Sigma_{s'} P(s'|s,a)U(s')$

Answer: C. $U(s) = \max_a \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$

Option notes:

- A: Next-state utilities must be averaged with P.
- B: Missing probabilities and max.
- D: Missing max, reward and γ.

Takeaway: Max over actions of expected (reward + γ next utility).

Source: Workshop 6, slide 29

MDP A17. [TF · Medium]

The optimal policy can be read from the Q-function as $\pi^*(s) = \text{argmax}_a\, Q(s,a)$.

Answer: True

Why: Workshop 6, slide 30.

Takeaway: U = max Q; π* = argmax Q.

Source: Workshop 6, slide 30

MDP A18. [BLANKS · Medium · calculation]

Q(s, Left) = 4.2 and Q(s, Right) = 5.1. U(s) = _____ and π*(s) = _____.

(1) U(s) = ____
(2) π*(s) = ____

Answer: U(s) = 5.1; π*(s) = Right

Takeaway: Take the max value; argmax gives the action.

Source: Workshop 6 (Reinforcement learning)

MDP A19. [MCQ · Hard]

Why are the n Bellman equations of value iteration non-linear?

A. Because of the max operator over actions
B. Because rewards are negative
C. Because of the discount factor
D. Because probabilities sum to 1

Answer: A. Because of the max operator over actions

Option notes:

- B: Irrelevant.
- C: Multiplying by γ is linear.
- D: Linear.

Takeaway: Max → non-linear → solve iteratively.

Source: Workshop 6, slide 33

MDP A20. [MCQ · Medium]

How does policy evaluation differ from a value-iteration update?

A. Policy evaluation has no max: the action in each state is fixed by the policy
B. Policy evaluation ignores rewards
C. Value iteration does not use transition probabilities
D. Policy evaluation uses γ = 1 always

Answer: A. Policy evaluation has no max: the action in each state is fixed by the policy

Option notes:

- B: Rewards are included.
- C: Both use P.
- D: γ is the same.

Takeaway: Fixed π → linear equations.

Source: Workshop 6, slide 37

MDP A21. [TF · Medium]

Policy iteration terminates when the policy improvement step yields no change.

Answer: True

Why: Workshop 6, slide 38.

Takeaway: Then U is a fixed point of the Bellman update → optimal.

Source: Workshop 6, slide 38

MDP A22. [MULTI · Hard]

Select ALL true statements about value and policy iteration. (Select ALL that apply.)

A. Value iteration requires the policy to be known in advance
B. Value iteration repeats the Bellman update until utilities converge
C. Policy iteration alternates evaluation and improvement starting from π_0
D. Policy iteration's evaluation equations are linear because the policy is fixed
E. Value iteration stops when $\delta \leq \varepsilon(1 - \gamma)/\gamma$

Answer: B. Value iteration repeats the Bellman update until utilities converge; C. Policy iteration alternates evaluation and improvement starting from π_0; D. Policy iteration's evaluation equations are linear because the policy is fixed; E. Value iteration stops when $\delta \leq \varepsilon(1 - \gamma)/\gamma$

Option notes:

- A: Incorrect: It finds the policy at the end.
- B: Correct: Slide 34.
- C: Correct: Slide 36.
- D: Correct: No max.
- E: Correct: Figure 17.6.

Takeaway: VI: values then policy; PI: policy evaluation + improvement.

Source: Workshop 6, slide 32-39

MDP A23. [MATCH · Medium]

Match the algorithm to its update.

1. Value iteration
2. Policy evaluation
3. Policy improvement

A. $\pi(s) \leftarrow \text{argmax}_a \Sigma P[R + \gamma U(s')]$
B. $U(s) \leftarrow U(s) + \alpha(R - U(s))$
C. $U_{i+1}(s) \leftarrow \Sigma P(s'|s,\pi_i(s))[R + \gamma U_i(s')]$
D. $U_{i+1}(s) \leftarrow \max_a \Sigma P[R + \gamma U_i(s')]$

Answer: 1 → D, 2 → C, 3 → A

Working:

- Value iteration → $U_{i+1}(s) \leftarrow \max_a \Sigma P[R + \gamma U_i(s')]$
- Policy evaluation → $U_{i+1}(s) \leftarrow \Sigma P(s'|s,\pi_i(s))[R + \gamma U_i(s')]$
- Policy improvement → $\pi(s) \leftarrow \text{argmax}_a \Sigma P[R + \gamma U(s')]$

Takeaway: Max (VI), no max (evaluation), argmax (improvement).

Source: Workshop 6, slide 33-38

MDP A24. [BLANK · Easy]

If an MDP has n states, value iteration solves _____ Bellman equations (one per state).

Answer: n

Takeaway: One equation per state.

Source: Workshop 6, slide 33

MDP A25. [MCQ · Medium]

TD vs ADP: which is correct?

A. ADP makes one adjustment per transition; TD makes many
B. TD learns a transition model; ADP does not
C. TD adjusts a state toward its observed successor; ADP toward all possible successors weighted by probability
D. They converge to different values

Answer: C. TD adjusts a state toward its observed successor; ADP toward all possible successors weighted by probability

Option notes:

- A: Reversed.
- B: Reversed.
- D: TD converges to ADP's values as trials → ∞.

Takeaway: TD = crude but efficient approximation to ADP.

Source: Workshop 6, slide 60

MDP A26. [MATCH · Medium]

Match the RL term to its definition.

1. Exploration
2. Exploitation
3. Passive RL
4. Active RL

A. leveraging known actions that yield high rewards
B. copying an expert's actions exactly
C. decide what to do via exploration and exploitation
D. trying new actions to discover their effects
E. fixed policy, learn state utilities

Answer: 1 → D, 2 → A, 3 → E, 4 → C

Working:

- Exploration → trying new actions to discover their effects
- Exploitation → leveraging known actions that yield high rewards
- Passive RL → fixed policy, learn state utilities
- Active RL → decide what to do via exploration and exploitation

Takeaway: Balance exploration and exploitation.

Source: Workshop 6, slide 43-48

MDP A27. [TF · Medium]

Direct utility estimation updates utilities after every single transition.

Answer: False

Why: It learns nothing until the END of the trial (averages observed reward-to-go). TD updates every transition.

Takeaway: Direct estimation converges very slowly.

Source: Workshop 6, slide 50

MDP A28. [BLANK · Easy · calculation]

First trial (γ = 1): state (0,0) is followed by 4 steps of −0.04 and then +1. Its reward-to-go is _____.

Answer: 0.84

Takeaway: (−0.04)×4 + 1.

Source: Workshop 6, slide 51

MDP A29. [MULTI · Medium]

Which are model-free methods? (Select ALL that apply.)

A. Direct utility estimation
B. Value iteration with a known model
C. Q-learning
D. TD learning
E. ADP

Answer: A. Direct utility estimation; C. Q-learning; D. TD learning

Option notes:

- A: Correct: Averages returns; no P.
- B: Incorrect: Planning with a model.
- C: Correct: Learns Q(s,a) directly.
- D: Correct: No transition model needed.
- E: Incorrect: Learns the transition model.

Takeaway: Model-free = no P(s′|s,a).

Source: Workshop 6, slide 48

MDP A30. [MCQ · Medium]

Sparse rewards means…

A. the reward function is unknown
B. rewards after every transition
C. the agent gets no informative reward signal in most states
D. rewards are always negative

Answer: C. the agent gets no informative reward signal in most states

Option notes:

- A: Unknown ≠ sparse.
- B: That's immediate rewards.
- D: No.

Takeaway: Intermediate rewards make learning easier.

Source: Workshop 6, slide 46

MDP A31. [BLANK · Medium · calculation]

ADP: Right was executed 4 times in (3,3), reaching (4,3) twice and (3,2) twice. P((3,2) | (3,3), Right) = _____.

Answer: 0.5
(Also accepted: 1/2, 0.50)

Takeaway: 2/4.

Source: Workshop 6, slide 56

MDP A32. [TF · Medium]

TD learning needs a transition model P(s′|s,a) to perform its updates.

Answer: False

Why: TD doesn't: the environment supplies the connection through observed transitions.

Takeaway: Model-free.

Source: Workshop 6, slide 58

MDP A33. [MCQ · Easy]

In the TD update $U(s) \leftarrow U(s) + \alpha[R + \gamma U(s') - U(s)]$, what is α?

A. The exploration probability
B. The discount factor
C. The learning rate
D. The reward

Answer: C. The learning rate

Option notes:

- A: Not in this formula.
- B: That's γ.
- D: That's R.

Takeaway: α = step size.

Source: Workshop 6, slide 58

MDP A34. [MCQ · Medium · calculation]

Q(s,a) = 2, α = 0.5, r = 1, γ = 0.9, Q(s′,·) = [3, 1, 0]. The Q-learning update gives…

A. 3.7
B. 2.1
C. 3.0
D. 2.85

Answer: D. 2.85

Option notes:

- A: That's the TD target.
- B: Used the average of Q(s′,·) (1.333) instead of the max.
- C: Forgot γ: 2 + 0.5(1 + 3 − 2).

Takeaway: Target 1 + 0.9×3 = 3.7.

Source: Workshop 6, slide 63

MDP A35. [TF · Medium]

Q-learning needs the transition model both for learning and for choosing actions.

Answer: False

Why: Q-learning is model-free for learning AND action selection (Workshop 6, slide 63).

Takeaway: Q replaces U + model.

Source: Workshop 6, slide 63

MDP A36. [MCQ · Hard]

The exploration function $f(u, n) = R^+$ if $n < N_e$ else u. What does it achieve?

A. It always chooses the greedy action
B. It decays the learning rate
C. It removes rarely tried actions
D. Untried actions look optimistic (R^+) until tried N_e times, encouraging exploration

Answer: D. Untried actions look optimistic (R^+) until tried N_e times, encouraging exploration

Option notes:

- A: Only once $n \geq N_e$.
- B: That's $\alpha(n)$.
- C: It favours them.

Takeaway: Optimism in the face of uncertainty.

Source: Workshop 6, slide 65

MDP A37. [MULTI · Medium]

Select ALL steps of the exploratory Q-learning agent loop. (Select ALL that apply.)

A. Update $Q(s,a)$ with target $r + \gamma \max_{a'} Q(s',a')$
B. Observe the current state s
C. Execute it and observe R and s'
D. Select an action (explore or exploit)
E. Estimate $P(s'|s,a)$ from counts

Answer: A. Update $Q(s,a)$ with target $r + \gamma \max_{a'} Q(s',a')$; B. Observe the current state s; C. Execute it and observe R and s'; D. Select an action (explore or exploit)

Option notes:

- A: Correct: Step 4.
- B: Correct: Step 1.
- C: Correct: Step 3.
- D: Correct: Step 2.
- E: Incorrect: That's ADP.

Takeaway: Observe → act → observe → update → repeat.

Source: Workshop 6, slide 64

MDP A38. [BLANK · Easy]

| | | +1
| ▓ wall | | −1
START | | |

In the 4×3 grid world, the small 'living' reward in every non-terminal state is _____.

Answer: -0.04
(Also accepted: −0.04, -.04)

Takeaway: Workshop 6 slide 20.

Source: Workshop 6 (Reinforcement learning)

MDP A39. [MCQ · Hard]

Which RL method is BOTH active and model-free?

A. Passive TD learning
B. Q-learning
C. Direct utility estimation
D. Adaptive dynamic programming

Answer: B. Q-learning

Option notes:

- A: Passive: fixed policy.
- C: Passive.
- D: Model-based.

Takeaway: Q-learning chooses actions and needs no model.

Source: Workshop 6 (Reinforcement learning)

MDP A40. [BLANK · Easy]

The six key components of RL: agent, environment, state, action, reward and _____.

Answer: policy
(Also accepted: Policy)

Takeaway: Workshop 6 slide 44.

Source: Workshop 6 (Reinforcement learning)

MDP A41. [BLANKS · Easy · calculation]

Route A: 80% reach goal (U = +10), 20% slip (U = −10). Route B: 100% reach goal (U = +7). EU(A) = ___, EU(B) = ___; MEU chooses route ___.

(1) EU(A) = ____
(2) EU(B) = ____
(3) choice = ____

Answer: EU(A) = 6; EU(B) = 7; choice = B

Working:

- 0.8(10) + 0.2(−10) = 6
- 1.0(7) = 7
- 7 > 6 → B

Takeaway: Highest expected utility, not best case.

Source: Book 2 (Exam examples), Q162, p. 25

MDP A42. [BLANKS · Easy · calculation]

Option X: 50% win $100, 50% lose $20. Option Y: 90% win $40, 10% win $0. EU(X) = ___; EU(Y) = ___; choose ___.

(1) EU(X) = ____
(2) EU(Y) = ____
(3) choice = ____

Answer: EU(X) = 40; EU(Y) = 36; choice = X

Working:

- 0.5(100) + 0.5(−20) = 40
- 0.9(40) = 36

Takeaway: Weight by probability.

Source: Book 2 (Exam examples), Q163, p. 25

MDP A43. [TF · Easy]

MEU chooses the action with the best possible outcome.

Answer: False

Why: It chooses the highest expected (probability-weighted) utility.

Takeaway: Expected, not best-case.

Source: Book 2 (Exam examples), Q164, p. 25

MDP A44. [BLANK · Medium · calculation]

Rewards 1, 1, 1, 10 in successive steps, γ = 0.9. G = ___ (1 d.p.).

Answer: 10.0
(Also accepted: 10)

Working:

- 1 + 0.9 + 0.81 = 2.71
- 0.729 × 10 = 7.29
- Total 10.0

Takeaway: 1 + 0.9 + 0.81 + 0.729(10).

Source: Book 2 (Exam examples), Q165, p. 25

MDP A45. [BLANK · Easy · calculation]

Reward 2 every step forever, γ = 0.8. G = ___.

Answer: 10

Takeaway: r/(1 − γ).

Source: Book 2 (Exam examples), Q166, p. 25

MDP A46. [BLANK · Easy]

With γ = 0 the agent cares only about the ___ reward.

Answer: immediate
(Also accepted: next, the immediate, current, immediate (next))

Takeaway: Short-sighted.

Source: Book 2 (Exam examples), Q167, p. 25

MDP A47. [MCQ · Easy]

| | | +1
| ▓ wall | | −1
START | | |

In the 4×3 grid world the living reward is −0.04. Its purpose is to:

A. make the agent prefer shorter paths to the +1 exit
B. make the problem deterministic
C. make the agent avoid the goal
D. represent the wall

Answer: A. make the agent prefer shorter paths to the +1 exit

Option notes:

- B: Transitions stay stochastic.
- C: It pushes toward the goal.
- D: The wall is a blocked square.

Takeaway: Each step costs a little.

Source: Book 2 (Exam examples), Q168, p. 25

MDP A48. [BLANK · Hard · calculation]

| | | +1
| ▓ wall | | −1
START | | |

4×3 grid world, converged utilities (γ = 1, R = −0.04): U(4,3) = 1, U(3,3) = 0.918, U(3,2) = 0.660. Q((3,3), Right) = −0.04 + 0.8U(4,3) + 0.1U(3,3) + 0.1U(3,2) = ___ (3 d.p.).

Answer: 0.918
(Also accepted: 0.917)

Working:

- Up-slip hits the wall: stay at (3,3).
- −0.04 + 0.8 + 0.0918 + 0.066 = 0.918

Takeaway: Equals U(3,3), as Bellman requires.

Source: Book 2 (Exam examples), Q169, p. 26

MDP A49. [BLANKS · Hard · calculation]

| | | +1
| ▓ wall | | −1
START | | |

4×3 grid world (γ = 1, R = −0.04), utilities U(1,1) = 0.705, U(1,2) = 0.762, U(2,1) = 0.655. At (1,1): Q(Up) = ___; Q(Right) = ___ (3 d.p.). Best action ___.

(1) Q(Up) = ____
(2) Q(Right) = ____

(3) best = ____

Answer: Q(Up) = 0.706; Q(Right) = 0.631; best = Up

Working:

- Up: −0.04 + 0.8(0.762) + 0.1(0.705) + 0.1(0.655) = 0.706
- Right: −0.04 + 0.8(0.655) + 0.1(0.762) + 0.1(0.705) = 0.631
- Up wins.

Takeaway: List landing squares with 0.8/0.1/0.1; bumps mean stay.

Source: Book 2 (Exam examples), Q170, p. 26

MDP A50. [BLANK · Medium · calculation]

| | | +1
| ▓ wall | | −1
START | | |

4×3 grid world, value iteration from U0 = 0 everywhere (terminals fixed at +1, −1), γ = 1, R = −0.04. After one update, U1(3,3) = ___.

Answer: 0.76

Working:

- Right: −0.04 + 0.8(1) + 0.1(0) + 0.1(0) = 0.76

Takeaway: Best is Right: 0.8(1).

Source: Book 2 (Exam examples), Q171, p. 26

MDP A51. [BLANK · Medium · calculation]

Same first value-iteration update at (3,3) but with γ = 0.9: U1(3,3) = ___.

Answer: 0.68

Takeaway: −0.04 + 0.9(0.8).

Source: Book 2 (Exam examples), Q172, p. 26

MDP A52. [TF · Medium]

In value iteration the Bellman equations are linear.

Answer: False

Why: The max makes them non-linear; with a fixed policy (policy evaluation) they are linear.

Takeaway: max → non-linear.

Source: Book 2 (Exam examples), Q173, p. 26

MDP A53. [MCQ · Easy]

Policy iteration alternates:

A. exploration and exploitation
B. policy evaluation and policy improvement
C. forward and backward passes
D. sampling and normalising

Answer: B. policy evaluation and policy improvement

Option notes:

- A: That is RL.
- C: That is backprop.
- D: That is approximate inference.

Takeaway: Evaluate, improve, repeat.

Source: Book 2 (Exam examples), Q174, p. 26

MDP A54. [BLANK · Easy]

Policy iteration terminates when the policy-improvement step produces ___.

Answer: no change
(Also accepted: the same policy, no change in the policy, unchanged policy, no policy change, the same policy again)

Takeaway: Stable policy = optimal.

Source: Book 2 (Exam examples), Q175, p. 26

MDP A55. [MCQ · Medium]

Bellman: $U(s) = \max_a \Sigma_{s'} P(s'|s, a)[$ ___ $+ \gamma U(s')]$.

A. $Q(s, a)$
B. $R(s, a, s')$
C. α
D. $U(s)$

Answer: B. $R(s, a, s')$

Option notes:

- A: Q is the whole bracketed sum.
- C: α is a learning rate.
- D: That would be circular.

Takeaway: Reward plus discounted next utility.

Source: Book 2 (Exam examples), Q176, p. 27

MDP A56. [MCQ · Easy]

$U(s) = \max_a$ ___.

A. $\pi(s)$
B. $P(s'|s, a)$
C. $Q(s, a)$
D. $R(s)$

Answer: C. $Q(s, a)$

Option notes:

- A: π is an action, not a value.
- B: A probability.
- D: Only the immediate reward.

Takeaway: Value of state = value of its best action.

Source: Book 2 (Exam examples), Q177, p. 27

MDP A57. [MCQ · Easy]

The optimal policy from Q: $\pi^*(s) =$ ___.

A. max_a Q(s, a)
B. argmax_a Q(s, a)
C. Σ_a Q(s, a)
D. min_a Q(s, a)

Answer: B. argmax_a Q(s, a)

Option notes:

- A: That is U(s), a number.
- C: Not a choice.
- D: Worst action.

Takeaway: argmax gives the action.

Source: Book 2 (Exam examples), Q178, p. 27

MDP A58. [MULTI · Medium]

An MDP requires: (Select ALL that apply.)

A. fully observable environment
B. a set of actions per state
C. stochastic Markovian transition model
D. additive rewards
E. a known optimal policy

Answer: A. fully observable environment; B. a set of actions per state; C. stochastic Markovian transition model; D. additive rewards

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Correct:
- E: Incorrect: the policy is what we solve for

Takeaway: MDP definition.

Source: Book 2 (Exam examples), Q179, p. 27

MDP A59. [TF · Easy]

The solution of an MDP is a fixed sequence of actions.

Answer: False

Why: It is a policy (an action for every state) because outcomes are uncertain.

Takeaway: Policy, not plan.

Source: Book 2 (Exam examples), Q180, p. 27

MDP A60. [BLANK · Easy · calculation]

Direct utility estimation, $\gamma = 1$, step reward -0.04, terminal $+1$. A state visited 4 steps before reaching $+1$ gets the sample $1 - 0.04 \times 4 =$ ___.

Answer: 0.84

Takeaway: Reward-to-go.

Source: Book 2 (Exam examples), Q181, p. 27

MDP A61. [BLANK · Easy · calculation]

A state got reward-to-go samples 0.80 and 0.88 in two trials. Its direct-utility estimate is ___.

Answer: 0.84

Takeaway: Average.

Source: Book 2 (Exam examples), Q182, p. 27

MDP A62. [BLANKS · Medium · calculation]

TD: U(s) = 0.5, U(s′) = 0.8, R = −0.04, γ = 1, α = 0.1. TD target = ___; TD error = ___; new U(s) = ___.

(1) target = _____
(2) error = _____
(3) new U(s) = _____

Answer: target = 0.76; error = 0.26; new U(s) = 0.526

Takeaway: $U \leftarrow U + \alpha(\text{target} - U)$.

Source: Book 2 (Exam examples), Q183, p. 27

MDP A63. [BLANK · Easy · calculation]

ADP: from state s with action a the agent went to s1 30 times and s2 10 times. $\hat{P}(s1 \mid s, a)$ = ___.

Answer: 0.75

Takeaway: 30/40.

Source: Book 2 (Exam examples), Q184, p. 27

MDP A64. [MATCH · Medium]

Match the passive RL method.

1. direct utility estimation
2. ADP
3. TD

A. learns a transition model then solves the Bellman equations
B. adjusts a state toward its observed successor without a model
C. averages observed reward-to-go per state

Answer: 1 → C, 2 → A, 3 → B

Working:

- direct utility estimation → averages observed reward-to-go per state
- ADP → learns a transition model then solves the Bellman equations
- TD → adjusts a state toward its observed successor without a model

Takeaway: Three passive methods.

Source: Book 2 (Exam examples), Q185, p. 27

MDP A65. [TF · Medium]

TD converges to the same values as ADP as the number of training sequences tends to infinity.

Answer: True

Why: TD is a crude but efficient approximation to ADP.

Takeaway: Same limit.

Source: Book 2 (Exam examples), Q186, p. 27

MDP A66. [MCQ · Medium]

Which passive method learns nothing until the end of a trial?

A. ADP
B. TD
C. Q-learning
D. direct utility estimation

Answer: D. direct utility estimation

Option notes:

- A: Updates its model each step.
- B: Updates every transition.
- C: Active and per-step.

Takeaway: Needs the full reward-to-go.

Source: Book 2 (Exam examples), Q187, p. 27

MDP A67. [BLANK · Medium · calculation]

Q-learning: $Q(s, a) = 2$, $R = 1$, $\gamma = 0.9$, $\max_{a'} Q(s', a') = 5$, $\alpha = 0.5$. New $Q(s, a)$ = ___.

Answer: 3.75

Working:

- target $1 + 4.5 = 5.5$
- error 3.5
- $2 + 0.5(3.5) = 3.75$

Takeaway: $Q + \alpha[R + \gamma \max Q' - Q]$.

Source: Book 2 (Exam examples), Q188, p. 28

MDP A68. [BLANK · Medium · calculation]

Q-learning: $Q(s, a) = 0$, $R = -1$, $\gamma = 1$, $Q(s', \cdot) = \{2, -3\}$, $\alpha = 0.2$. New $Q(s, a)$ = ___.

Answer: 0.2

Working:

- target $-1 + 2 = 1$
- $0 + 0.2(1 - 0) = 0.2$

Takeaway: Use the MAX of $Q(s', \cdot) = 2$.

Source: Book 2 (Exam examples), Q189, p. 28

MDP A69. [TF · Easy]

Q-learning needs the transition model $P(s'|s, a)$.

Answer: False

Why: It is model-free.

Takeaway: Model-free.

Source: Book 2 (Exam examples), Q190, p. 28

MDP A70. [BLANKS · Easy]

Trying new actions to learn their effects is ___; using known good actions is ___.

(1) new actions = ____
(2) known actions = ____

Answer: new actions = exploration; known actions = exploitation

Takeaway: Balance both.

Source: Book 2 (Exam examples), Q191, p. 28

MDP A71. [MCQ · Medium]

In the exploration function f(u, n) = R+ if n < Ne, else u, R+ is:

A. a penalty for unvisited states
B. the learning rate
C. the average reward
D. an optimistic estimate of the best possible reward

Answer: D. an optimistic estimate of the best possible reward

Option notes:

- A: It is a bonus.
- B: That is α.
- C: Not optimistic.

Takeaway: Optimism drives exploration.

Source: Book 2 (Exam examples), Q192, p. 28

MDP A72. [MULTI · Medium]

Model-free methods: (Select ALL that apply.)

A. ADP
B. policy learning
C. Q-learning
D. TD

Answer: B. policy learning; C. Q-learning; D. TD

Option notes:

- A: Incorrect: learns a model
- B: Correct:
- C: Correct:
- D: Correct:

Takeaway: ADP is model-based.

Source: Book 2 (Exam examples), Q193, p. 28

MDP A73. [MATCH · Easy]

Match the RL type.

1. passive RL
2. active RL

A. agent decides what to do, must explore
B. fixed policy, learn utilities

Answer: 1 → B, 2 → A

Working:

- passive RL → fixed policy, learn utilities
- active RL → agent decides what to do, must explore

Takeaway: Passive vs active.

Source: Book 2 (Exam examples), Q194, p. 28

MDP A74. [BLANK · Easy · calculation]

Reward 5 at each of three steps, γ = 0.5. Discounted return ___.

Answer: 8.75

Takeaway: 5 + 2.5 + 1.25.

Source: Book 2 (Exam examples), Mixed practice paper Q14, p. 54

MDP A75. [BLANKS · Easy · calculation]

Option A: 60% +20, 40% −5. Option B: certain +8. EU(A) = ___; MEU chooses ___.

(1) EU(A) = ____
(2) choice = ____

Answer: EU(A) = 10; choice = A

Takeaway: 12 − 2 = 10 > 8.

Source: Book 2 (Exam examples), Mixed practice paper Q15, p. 54

MDP A76. [BLANK · Medium · calculation]

TD update: U(s) = 0.2, R = 1, γ = 0.5, U(s′) = 0.6, α = 0.5. New U(s) = ___.

Answer: 0.75

Takeaway: Target 1.3; error 1.1.

Source: Book 2 (Exam examples), Mixed practice paper Q16, p. 54

MDP A77. [BLANK · Medium · calculation]

Q-learning: Q(s, a) = 1, R = 0, γ = 0.9, max_a′ Q(s′, a′) = 4, α = 0.25. New Q(s, a) = ___.

Answer: 1.65

Takeaway: 1 + 0.25(3.6 − 1).

Source: Book 2 (Exam examples), Mixed practice paper Q17, p. 54

MDP A78. [MCQ · Medium]

ADP differs from TD because ADP:

A. needs no model
B. learns P(s′|s, a) from counts and solves the Bellman equations
C. uses Q-values
D. only updates at the end of a trial

Answer: B. learns P(s′|s, a) from counts and solves the Bellman equations

Option notes:

- A: That is TD.
- C: Q-learning.
- D: Direct utility estimation.

Takeaway: Model-based.

Source: Book 2 (Exam examples), Mixed practice paper Q18, p. 54

MDP A79. [BLANKS · Easy · calculation]

Action X gives +20 with probability 0.5 else 0; Y gives 8 for sure. EU(X) = ___; MEU choice ___.

(1) EU(X) = ____
(2) choice = ____

Answer: EU(X) = 10; choice = X

Takeaway: 10 > 8.

Source: Book 3 (AI from zero), Practice 18.1, p. 75

MDP A80. [BLANK · Easy · calculation]

Rewards 0, 0, 10, γ = 0.5. Return = ___.

Answer: 2.5

Takeaway: 0.25 × 10.

Source: Book 3 (AI from zero), Practice 18.2, p. 75

MDP A81. [BLANK · Easy · calculation]

Constant reward −1 forever, γ = 0.9. Return = ___.

Answer: -10

Takeaway: −1/(1 − 0.9).

Source: Book 3 (AI from zero), Practice 18.3, p. 75

MDP A82. [BLANK · Medium · calculation]

From s, a goes to s1 (0.7, U = 10) or s2 (0.3, U = 0); R = −1, γ = 0.9. Q(s, a) = ___.

Answer: 5.3

Takeaway: 0.7(−1 + 9) + 0.3(−1).

Source: Book 3 (AI from zero), Practice 18.4, p. 75

MDP A83. [BLANKS · Hard · calculation]

| | | +1
| ▓ wall | | −1
START | | |

4×3 grid world utilities U(1,1) = 0.705, U(1,2) = 0.762, U(2,1) = 0.655, R = −0.04, γ = 1. Q((1,1), Left) = ___; Q((1,1), Down) = ___ (3 d.p.).

(1) Left = _____
(2) Down = _____

Answer: Left = 0.671; Down = 0.660

Working:

- Left: 0.8 bump (stay 0.705), 0.1 up (0.762), 0.1 down bump (0.705): −0.04 + 0.564 + 0.0762 + 0.0705 = 0.671
- Down: 0.8 bump (0.705), 0.1 left bump (0.705), 0.1 right (0.655): −0.04 + 0.564 + 0.0705 + 0.0655 = 0.660

Takeaway: Wall bumps mean staying at (1,1).

Source: Book 3 (AI from zero), Chapter 18, p. 71

MDP A84. [BLANK · Hard · calculation]

Value iteration at (3,3), γ = 1, R = −0.04. Iteration 1 gives 0.76; with U1(3,3) = 0.76 and U1(3,2) = −0.04, iteration 2 Right = −0.04 + 0.8(1) + 0.1(0.76) + 0.1(−0.04) = ___.

Answer: 0.832

Takeaway: Values spread back from the goal.

Source: Book 3 (AI from zero), Chapter 18, p. 71

MDP A85. [BLANKS · Hard · calculation]

Policy evaluation: from A the policy always goes to B (reward −1). From B: 50% to the terminal (reward +10), 50% back to A (reward −1). γ = 1. U(A) = ___; U(B) = ___.

(1) U(A) = _____
(2) U(B) = _____

Answer: U(A) = 7; U(B) = 8

Working:

- U(B) = 5 + 0.5(−2 + U(B)) = 4 + 0.5U(B)
- U(B) = 8, U(A) = 7

Takeaway: Linear equations: U(A) = −1 + U(B); U(B) = 0.5(10) + 0.5(−1 + U(A)).

Source: Book 3 (AI from zero), Chapter 18, p. 71

MDP A86. [BLANK · Medium · calculation]

Q(s, a) = 4, R = −1, γ = 0.5, max Q(s′, ·) = 6, α = 0.25. New Q = ___.

Answer: 3.5

Takeaway: Target 2.

Source: Book 3 (AI from zero), Practice 19.1, p. 79

MDP A87. [BLANK · Medium · calculation]

TD: U(s) = 1, R = 0, U(s′) = 2, γ = 0.9, α = 0.5. New U(s) = ___.

Answer: 1.4

Takeaway: 1 + 0.5(1.8 − 1).

Source: Book 3 (AI from zero), Practice 19.2, p. 79

MDP A88. [TF · Easy]

Q-learning is model-based.

Answer: False

Why: Model-free.

Takeaway: No P(s′|s,a).

Source: Book 3 (AI from zero), Practice 19.3, p. 79

MDP A89. [BLANK · Easy · calculation]

An action was taken 20 times and landed in s′ 5 times. $\hat{P}(s'|s, a)$ = ___.

Answer: 0.25

Takeaway: 5/20.

Source: Book 3 (AI from zero), Practice 19.4, p. 79

MDP A90. [BLANKS · Hard · calculation]

Tiny Q-table: A → B (reward 0), B → G terminal (reward 10); α = 0.5, γ = 0.9, all Q start at 0. After episode 1: Q(B) = ___. After episode 2: Q(A) = ___, Q(B) = ___.

(1) Q(B) ep1 = ____
(2) Q(A) ep2 = ____
(3) Q(B) ep2 = ____

Answer: Q(B) ep1 = 5; Q(A) ep2 = 2.25; Q(B) ep2 = 7.5

Working:

- Ep1: Q(A) = 0; Q(B) = 0.5(10) = 5
- Ep2: Q(A) = 0.5(0 + 0.9 × 5) = 2.25; Q(B) = 5 + 0.5(10 − 5) = 7.5

Takeaway: Value flows back one step per episode.

Source: Book 3 (AI from zero), Chapter 19, p. 76

MDP A91. [MULTI · Medium]

Deep Q-networks (DQN): (Select ALL that apply.)

A. approximate Q(s, a) with a neural network
B. use experience replay
C. use a slowly updated target network
D. need the transition model

Answer: A. approximate Q(s, a) with a neural network; B. use experience replay; C. use a slowly updated target network

Option notes:

- A: Correct:

- B: Correct: train on random stored mini-batches
- C: Correct: for stability
- D: Incorrect: still model-free

Takeaway: Atari from raw pixels.

Source: Book 3 (AI from zero), Chapter 19, p. 76

MDP A92. [MCQ · Easy]

ε-greedy exploration means:

A. always random
B. always the best action
C. with probability ε pick a random action, otherwise the best-known action
D. decrease γ over time

Answer: C. with probability ε pick a random action, otherwise the best-known action

Option notes:

- A: No exploitation.
- B: No exploration.
- D: γ is the discount.

Takeaway: Common alternative to the exploration function.

Source: Book 3 (AI from zero), Chapter 19, p. 76

MDP A93. [MCQ · Easy]

MDP planning vs reinforcement learning:

A. both need the model
B. they are identical
C. planning knows P and R and computes; RL does not and learns from trials
D. RL needs the model, planning does not

Answer: C. planning knows P and R and computes; RL does not and learns from trials

Option notes:

- A: RL does not.
- B: No.
- D: Reversed.

Takeaway: MDP defines the task; RL learns to act.

Source: Book 3 (AI from zero), Chapter 19, p. 76

MDP A94. [BLANK · Medium · calculation]

With rewards bounded by R_max and discount γ < 1, the utility of any infinite sequence is at most R_max/(1 − γ). For R_max = 1, γ = 0.9 the bound is ___.

Answer: 10

Takeaway: Geometric series.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 17: discounting

MDP A95. [MATCH · Hard]

| | | +1
| ▓ wall | | −1
START | | |

(Extra, not in your slides) 4×3 world: how the optimal policy changes with the living reward r (AIMA).

1. r very negative (life is painful)
2. r moderately negative (−0.04)
3. r slightly negative, near 0
4. r > 0

A. take no risks: avoid the −1 at all costs
B. head for the nearest exit, even −1
C. avoid both exits and live forever
D. take the shortest safe-ish route to +1

Answer: 1 → B, 2 → D, 3 → A, 4 → C

Working:

- r very negative (life is painful) → head for the nearest exit, even −1
- r moderately negative (−0.04) → take the shortest safe-ish route to +1
- r slightly negative, near 0 → take no risks: avoid the −1 at all costs
- r > 0 → avoid both exits and live forever

Takeaway: Rewards shape the policy.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 17: 4×3 world

MDP A96. [MCQ · Hard]

(Extra, not in your slides) Why does value iteration converge (for γ < 1)?

A. the utilities are always 0
B. the Bellman update is a contraction: each step shrinks the max error by a factor γ
C. the max makes it linear
D. it visits every state once

Answer: B. the Bellman update is a contraction: each step shrinks the max error by a factor γ

Option notes:

- A: No.
- C: The max makes it non-linear.
- D: It sweeps repeatedly.

Takeaway: Error shrinks geometrically.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 17: value iteration

MDP A97. [TF · Medium]

(Extra, not in your slides) The policy often becomes optimal long before the utility estimates have converged.

Answer: True

Why: Only the ranking of actions matters for the policy.

Takeaway: Motivates policy iteration.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 17: policy iteration

MDP A98. [BLANKS · Medium]

Decision theory = probability theory + ___ theory; a rational agent maximises ___.

(1) theory = ____
(2) maximises = ____

Answer: theory = utility; maximises = expected utility

Takeaway: MEU principle.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 16: MEU

MDP A99. [BLANK · Medium · calculation]

(Extra, not in your slides) A lottery gives $1,000 with probability 0.3 and $0 otherwise. Its expected monetary value is $___.

Answer: 300

Takeaway: A risk-averse agent may prefer a sure $250.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 16: lotteries

MDP A100. [MATCH · Hard]

(Extra, not in your slides) Match the active RL update.

1. Q-learning (off-policy)
2. SARSA (on-policy)

A. target r only
B. target r + γ max_a′ Q(s′, a′)
C. target r + γ Q(s′, a′) for the action actually taken next

Answer: 1 → B, 2 → C

Working:

- Q-learning (off-policy) → target r + γ max_a′ Q(s′, a′)
- SARSA (on-policy) → target r + γ Q(s′, a′) for the action actually taken next

Takeaway: They differ only when exploring.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 22: Q-learning vs SARSA

MDP A101. [BLANK · Hard · calculation]

(Extra, not in your slides) SARSA: Q(s, a) = 1, r = 0, γ = 0.9, next action a′ with Q(s′, a′) = 2 (best action has 4), α = 0.5. New Q(s, a) = ___.

Answer: 1.4

Takeaway: SARSA uses the taken action's 2, not the max 4 (Q-learning would give 2.3).

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 22: SARSA

MDP A102. [MULTI · Medium]

AIMA's agent designs for RL: (Select ALL that apply.)

A. search agent: needs no reward
B. utility-based agent: learns U, needs a model

C. Q-learning agent: learns Q(s, a), no model
D. reflex agent: learns a policy directly

Answer: B. utility-based agent: learns U, needs a model; C. Q-learning agent: learns Q(s, a), no model; D. reflex agent: learns a policy directly

Option notes:

- A: Incorrect: not an RL design
- B: Correct:
- C: Correct:
- D: Correct:

Takeaway: What is learned determines what is needed.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 22: RL overview

MDP A103. [BLANK · Easy · calculation]

ADP: action Right in (1,3) was tried 3 times, reaching (2,3) twice and (1,2) once. $\hat{P}((2,3) \mid (1,3), \text{Right}) =$ ___.

Answer: 2/3
(Also accepted: 0.667, 0.67)

Takeaway: Count / tries.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 22: ADP

MDP A104. [TF · Easy]

The TD update needs a transition model.

Answer: False

Why: TD adjusts toward the observed successor; no model.

Takeaway: Model-free.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 22: TD

Part B: calculation drills (144)

MDP B1. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = -5, -5, -0.04, -5, 2 (in order). With discount $\gamma = 0.5$, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. -6.520
B. -8.010
C. -4.005
D. -13.040

Answer: B. -8.010

Working:

- 0.5^0×-5 + 0.5^1×-5 + 0.5^2×-0.04 + 0.5^3×-5 + 0.5^4×2 = -8.0100.

Option notes:

- A: Used γ for every reward instead of γ^k.
- C: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.
- D: Ignored the discount (that is the additive reward, $\gamma = 1$).

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B2. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 0, 1, 10, 0, 0 (in order). With discount $\gamma = 0.5$, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 1.500
B. 3.000
C. 5.500
D. 11.000

Answer: B. 3.000

Working:

- 0.5^0×0 + 0.5^1×1 + 0.5^2×10 + 0.5^3×0 + 0.5^4×0 = 3.0000.

Option notes:

- A: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.
- C: Used γ for every reward instead of γ^k.
- D: Ignored the discount (that is the additive reward, γ = 1).

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B3. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 1, 10, -0.04 (in order). With discount $\gamma = 1$, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 10.960
B. 12.604
C. 14.248
D. 9.316

Answer: A. 10.960

Working:

- 1^0×1 + 1^1×10 + 1^2×-0.04 = 10.9600.

Option notes:

- B: Arithmetic slip: recompute each step carefully.
- C: Arithmetic slip: recompute each step carefully.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B4. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = -1, -1, 10, 5, -5 (in order). With discount $\gamma = 0.9$, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 8.000
B. 7.200
C. 5.908
D. 6.565

Answer: D. 6.565

Working:

- 0.9^0×-1 + 0.9^1×-1 + 0.9^2×10 + 0.9^3×5 + 0.9^4×-5 = 6.5645.

Option notes:

- A: Ignored the discount (that is the additive reward, γ = 1).
- B: Used γ for every reward instead of γ^k.
- C: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B5. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 10, 0, 2, 5 (in order). With discount γ = 0.5, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 5.563
B. 8.500
C. 17.000
D. 11.125

Answer: D. 11.125

Working:

- 0.5^0×10 + 0.5^1×0 + 0.5^2×2 + 0.5^3×5 = 11.1250.

Option notes:

- A: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.
- B: Used γ for every reward instead of γ^k.
- C: Ignored the discount (that is the additive reward, γ = 1).

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B6. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = -1, -0.04, 10, -1 (in order). With discount γ = 0.95, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 7.130
B. 7.960
C. 7.562
D. 6.773

Answer: A. 7.130

Working:

- 0.95^0×-1 + 0.95^1×-0.04 + 0.95^2×10 + 0.95^3×-1 = 7.1296.

Option notes:

- B: Ignored the discount (that is the additive reward, γ = 1).
- C: Used γ for every reward instead of γ^k.
- D: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B7. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = -0.04, 10, 1, 1, 2 (in order). With discount $\gamma = 1$, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 16.054
B. 18.148
C. 13.960
D. 11.866

Answer: C. 13.960

Working:

- 1^0×-0.04 + 1^1×10 + 1^2×1 + 1^3×1 + 1^4×2 = 13.9600.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B8. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = -5, -1, -5 (in order). With discount $\gamma = 0.95$, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. -9.939
B. -10.450
C. -11.000
D. -10.463

Answer: D. -10.463

Working:

- 0.95^0×-5 + 0.95^1×-1 + 0.95^2×-5 = -10.4625.

Option notes:

- A: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.
- B: Used γ for every reward instead of γ^k.
- C: Ignored the discount (that is the additive reward, $\gamma = 1$).

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B9. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 0, 0, 10, 5, 10 (in order). With discount $\gamma = 0.95$, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 23.750
B. 25.000
C. 21.457
D. 20.384

Answer: C. 21.457

Working:

- 0.95^0×0 + 0.95^1×0 + 0.95^2×10 + 0.95^3×5 + 0.95^4×10 = 21.4569.

Option notes:

- A: Used γ for every reward instead of γ^k.
- B: Ignored the discount (that is the additive reward, γ = 1).
- D: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B10. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 10, 10, 2 (in order). With discount γ = 0.5, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 22.000
B. 11.000
C. 7.750
D. 15.500

Answer: D. 15.500

Working:

- 0.5^0×10 + 0.5^1×10 + 0.5^2×2 = 15.5000.

Option notes:

- A: Ignored the discount (that is the additive reward, γ = 1).
- B: Used γ for every reward instead of γ^k.
- C: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B11. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = -1, 10, -0.04, 10, -1 (in order). With discount γ = 1, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 23.348
B. 15.266
C. 17.960
D. 20.654

Answer: C. 17.960

Working:

- 1^0×-1 + 1^1×10 + 1^2×-0.04 + 1^3×10 + 1^4×-1 = 17.9600.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B12. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 0, 0, 2, -5 (in order). With discount γ = 1, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. -3.000
B. -3.450
C. -2.100
D. -2.550

Answer: A. -3.000

Working:

- 1^0×0 + 1^1×0 + 1^2×2 + 1^3×-5 = -3.0000.

Option notes:

- B: Arithmetic slip: recompute each step carefully.
- C: Arithmetic slip: recompute each step carefully.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B13. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = -1, -0.04, -5, -5, 10 (in order). With discount γ = 0.9, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. -2.170
B. -1.040
C. -1.953
D. -0.936

Answer: A. -2.170

Working:

- 0.9^0×-1 + 0.9^1×-0.04 + 0.9^2×-5 + 0.9^3×-5 + 0.9^4×10 = -2.1700.

Option notes:

- B: Ignored the discount (that is the additive reward, γ = 1).
- C: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.
- D: Used γ for every reward instead of γ^k.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B14. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 2, -0.04, 1, 2, -5 (in order). With discount γ = 0.95, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. -0.038
B. 0.481
C. -0.040
D. 0.507

Answer: D. 0.507

Working:

- 0.95^0×2 + 0.95^1×-0.04 + 0.95^2×1 + 0.95^3×2 + 0.95^4×-5 = 0.5067.

Option notes:

- A: Used γ for every reward instead of γ^k.
- B: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.
- C: Ignored the discount (that is the additive reward, γ = 1).

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B15. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 1, 2, -0.04 (in order). With discount γ = 0.9, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 2.960
B. 2.664
C. 2.491
D. 2.768

Answer: D. 2.768

Working:

- 0.9^0×1 + 0.9^1×2 + 0.9^2×-0.04 = 2.7676.

Option notes:

- A: Ignored the discount (that is the additive reward, γ = 1).
- B: Used γ for every reward instead of γ^k.
- C: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B16. [MCQ · Easy · calculation]

An agent receives rewards R_1, R_2, … = 1, 2, -5, -5, 10 (in order). With discount γ = 0.5, what is the discounted return $G = R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots$?

A. 1.500
B. 0.375
C. 3.000
D. 0.750

Answer: D. 0.750

Working:

- 0.5^0×1 + 0.5^1×2 + 0.5^2×-5 + 0.5^3×-5 + 0.5^4×10 = 0.7500.

Option notes:

- A: Used γ for every reward instead of γ^k.
- B: Discounted the first reward too; the first reward gets $\gamma^0 = 1$.
- C: Ignored the discount (that is the additive reward, γ = 1).

Takeaway: Discounting: later rewards count less; γ close to 1 = far-sighted.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B17. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.9 | 9 | 0
B | 0.8 | 9 | 4
C | 0.7 | 11 | 4

A rational agent follows the MEU principle. Which action does it choose?

A. Action C, because it has the best possible outcome (11)
B. Action B (EU = 8)
C. Action A (EU = 8.1)
D. Action C (EU = 8.9)

Answer: D. Action C (EU = 8.9)

Working:

- EU(A) = 0.9×9 + 0.1×(0) = 8.1.
- EU(B) = 0.8×9 + 0.2×(4) = 8.
- EU(C) = 0.7×11 + 0.3×(4) = 8.9.
- Choose argmax → C.

Option notes:

- A: MEU uses expected utility, not the best case.
- B: EU(B) = 8 is lower.
- C: EU(A) = 8.1 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B18. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.8 | 12 | 4
B | 0.5 | 3 | 1
C | 0.6 | 9 | -1

A rational agent follows the MEU principle. Which action does it choose?

A. Action C (EU = 5)
B. Action A, because it has the best possible outcome (12)
C. Action A (EU = 10.4)
D. Action B (EU = 2)

Answer: C. Action A (EU = 10.4)

Working:

- EU(A) = 0.8×12 + 0.2×(4) = 10.4.
- EU(B) = 0.5×3 + 0.5×(1) = 2.
- EU(C) = 0.6×9 + 0.4×(-1) = 5.
- Choose argmax → A.

Option notes:

- A: EU(C) = 5 is lower.
- B: MEU uses expected utility, not the best case.
- D: EU(B) = 2 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B19. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.6 | 9 | -6
B | 0.8 | 12 | 2
C | 0.9 | 6 | -2

A rational agent follows the MEU principle. Which action does it choose?

A. Action B, because it has the best possible outcome (12)
B. Action A (EU = 3)
C. Action C (EU = 5.2)
D. Action B (EU = 10)

Answer: D. Action B (EU = 10)

Working:

- EU(A) = 0.6×9 + 0.4×(-6) = 3.
- EU(B) = 0.8×12 + 0.2×(2) = 10.
- EU(C) = 0.9×6 + 0.1×(-2) = 5.2.
- Choose argmax → B.

Option notes:

- A: MEU uses expected utility, not the best case.
- B: EU(A) = 3 is lower.
- C: EU(C) = 5.2 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s)\cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B20. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.5 | 12 | 3
B | 0.6 | 2 | -10
C | 0.7 | 7 | 4

A rational agent follows the MEU principle. Which action does it choose?

A. Action B (EU = -2.8)
B. Action C (EU = 6.1)
C. Action A (EU = 7.5)
D. Action A, because it has the best possible outcome (12)

Answer: C. Action A (EU = 7.5)

Working:

- EU(A) = 0.5×12 + 0.5×(3) = 7.5.
- EU(B) = 0.6×2 + 0.4×(-10) = -2.8.
- EU(C) = 0.7×7 + 0.3×(4) = 6.1.
- Choose argmax → A.

Option notes:

- A: EU(B) = -2.8 is lower.
- B: EU(C) = 6.1 is lower.

- D: MEU uses expected utility, not the best case.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B21. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.5 | 10 | -4
B | 0.9 | 10 | 3
C | 0.7 | 7 | -3

A rational agent follows the MEU principle. Which action does it choose?

A. Action A, because it has the best possible outcome (10)
B. Action A (EU = 3)
C. Action B (EU = 9.3)
D. Action C (EU = 4)

Answer: C. Action B (EU = 9.3)

Working:

- EU(A) = 0.5×10 + 0.5×(-4) = 3.
- EU(B) = 0.9×10 + 0.1×(3) = 9.3.
- EU(C) = 0.7×7 + 0.3×(-3) = 4.
- Choose argmax → B.

Option notes:

- A: MEU uses expected utility, not the best case.
- B: EU(A) = 3 is lower.
- D: EU(C) = 4 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B22. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.5 | 5 | -7
B | 0.6 | 5 | -7
C | 0.8 | 10 | -7

A rational agent follows the MEU principle. Which action does it choose?

A. Action B (EU = 0.2)
B. Action C, because it has the best possible outcome (10)
C. Action C (EU = 6.6)
D. Action A (EU = -1)

Answer: C. Action C (EU = 6.6)

Working:

- EU(A) = 0.5×5 + 0.5×(-7) = -1.
- EU(B) = 0.6×5 + 0.4×(-7) = 0.2.
- EU(C) = 0.8×10 + 0.2×(-7) = 6.6.
- Choose argmax → C.

Option notes:

- A: EU(B) = 0.2 is lower.
- B: MEU uses expected utility, not the best case.
- D: EU(A) = -1 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B23. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.6 | 5 | -3
B | 0.6 | 3 | -1
C | 0.6 | 6 | -2

A rational agent follows the MEU principle. Which action does it choose?

A. Action A (EU = 1.8)
B. Action C, because it has the best possible outcome (6)
C. Action C (EU = 2.8)
D. Action B (EU = 1.4)

Answer: C. Action C (EU = 2.8)

Working:

- EU(A) = 0.6×5 + 0.4×(-3) = 1.8.
- EU(B) = 0.6×3 + 0.4×(-1) = 1.4.
- EU(C) = 0.6×6 + 0.4×(-2) = 2.8.
- Choose argmax → C.

Option notes:

- A: EU(A) = 1.8 is lower.
- B: MEU uses expected utility, not the best case.
- D: EU(B) = 1.4 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B24. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.6 | 6 | 3
B | 0.7 | 5 | -3
C | 0.6 | 8 | -6

A rational agent follows the MEU principle. Which action does it choose?

A. Action A (EU = 4.8)
B. Action B (EU = 2.6)
C. Action C, because it has the best possible outcome (8)
D. Action C (EU = 2.4)

Answer: A. Action A (EU = 4.8)

Working:

- EU(A) = 0.6×6 + 0.4×(3) = 4.8.
- EU(B) = 0.7×5 + 0.3×(-3) = 2.6.
- EU(C) = 0.6×8 + 0.4×(-6) = 2.4.

- Choose argmax → A.

Option notes:

- B: EU(B) = 2.6 is lower.
- C: MEU uses expected utility, not the best case.
- D: EU(C) = 2.4 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B25. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.9 | 3 | -1
B | 0.7 | 3 | 4
C | 0.5 | 8 | 0

A rational agent follows the MEU principle. Which action does it choose?

A. Action B (EU = 3.3)
B. Action C, because it has the best possible outcome (8)
C. Action C (EU = 4)
D. Action A (EU = 2.6)

Answer: C. Action C (EU = 4)

Working:

- EU(A) = 0.9×3 + 0.1×(-1) = 2.6.
- EU(B) = 0.7×3 + 0.3×(4) = 3.3.
- EU(C) = 0.5×8 + 0.5×(0) = 4.
- Choose argmax → C.

Option notes:

- A: EU(B) = 3.3 is lower.
- B: MEU uses expected utility, not the best case.
- D: EU(A) = 2.6 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B26. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.9 | 11 | -10
B | 0.5 | 8 | -3
C | 0.9 | 7 | -7

A rational agent follows the MEU principle. Which action does it choose?

A. Action A, because it has the best possible outcome (11)
B. Action A (EU = 8.9)
C. Action B (EU = 2.5)
D. Action C (EU = 5.6)

Answer: B. Action A (EU = 8.9)

Working:

- EU(A) = 0.9×11 + 0.1×(-10) = 8.9.

- EU(B) = 0.5×8 + 0.5×(-3) = 2.5.
- EU(C) = 0.9×7 + 0.1×(-7) = 5.6.
- Choose argmax → A.

Option notes:

- A: MEU uses expected utility, not the best case.
- C: EU(B) = 2.5 is lower.
- D: EU(C) = 5.6 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B27. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.7 | 4 | 2
B | 0.9 | 5 | -7
C | 0.9 | 12 | 3

A rational agent follows the MEU principle. Which action does it choose?

A. Action C, because it has the best possible outcome (12)
B. Action C (EU = 11.1)
C. Action B (EU = 3.8)
D. Action A (EU = 3.4)

Answer: B. Action C (EU = 11.1)

Working:

- EU(A) = 0.7×4 + 0.3×(2) = 3.4.
- EU(B) = 0.9×5 + 0.1×(-7) = 3.8.
- EU(C) = 0.9×12 + 0.1×(3) = 11.1.
- Choose argmax → C.

Option notes:

- A: MEU uses expected utility, not the best case.
- C: EU(B) = 3.8 is lower.
- D: EU(A) = 3.4 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s) \cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B28. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.5 | 2 | 3
B | 0.6 | 8 | -2
C | 0.9 | 2 | -3

A rational agent follows the MEU principle. Which action does it choose?

A. Action B (EU = 4)
B. Action C (EU = 1.5)
C. Action B, because it has the best possible outcome (8)
D. Action A (EU = 2.5)

Answer: A. Action B (EU = 4)

Working:

- EU(A) = 0.5×2 + 0.5×(3) = 2.5.
- EU(B) = 0.6×8 + 0.4×(-2) = 4.
- EU(C) = 0.9×2 + 0.1×(-3) = 1.5.
- Choose argmax → B.

Option notes:

- B: EU(C) = 1.5 is lower.
- C: MEU uses expected utility, not the best case.
- D: EU(A) = 2.5 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s)\cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B29. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.6 | 2 | 1
B | 0.9 | 4 | -6
C | 0.7 | 10 | 4

A rational agent follows the MEU principle. Which action does it choose?

A. Action C (EU = 8.2)
B. Action A (EU = 1.6)
C. Action C, because it has the best possible outcome (10)
D. Action B (EU = 3)

Answer: A. Action C (EU = 8.2)

Working:

- EU(A) = 0.6×2 + 0.4×(1) = 1.6.
- EU(B) = 0.9×4 + 0.1×(-6) = 3.
- EU(C) = 0.7×10 + 0.3×(4) = 8.2.
- Choose argmax → C.

Option notes:

- B: EU(A) = 1.6 is lower.
- C: MEU uses expected utility, not the best case.
- D: EU(B) = 3 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s)\cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B30. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.8 | 11 | -9
B | 0.5 | 5 | -4
C | 0.9 | 4 | 0

A rational agent follows the MEU principle. Which action does it choose?

A. Action C (EU = 3.6)
B. Action A (EU = 7)

C. Action A, because it has the best possible outcome (11)
D. Action B (EU = 0.5)

Answer: B. Action A (EU = 7)

Working:

- EU(A) = 0.8×11 + 0.2×(-9) = 7.
- EU(B) = 0.5×5 + 0.5×(-4) = 0.5.
- EU(C) = 0.9×4 + 0.1×(0) = 3.6.
- Choose argmax → A.

Option notes:

- A: EU(C) = 3.6 is lower.
- C: MEU uses expected utility, not the best case.
- D: EU(B) = 0.5 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s)\cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B31. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.9 | 9 | -3
B | 0.8 | 9 | -9
C | 0.8 | 2 | 0

A rational agent follows the MEU principle. Which action does it choose?

A. Action B (EU = 5.4)
B. Action C (EU = 1.6)
C. Action A, because it has the best possible outcome (9)
D. Action A (EU = 7.8)

Answer: D. Action A (EU = 7.8)

Working:

- EU(A) = 0.9×9 + 0.1×(-3) = 7.8.
- EU(B) = 0.8×9 + 0.2×(-9) = 5.4.
- EU(C) = 0.8×2 + 0.2×(0) = 1.6.
- Choose argmax → A.

Option notes:

- A: EU(B) = 5.4 is lower.
- B: EU(C) = 1.6 is lower.
- C: MEU uses expected utility, not the best case.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s)\cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B32. [MCQ · Medium · calculation]

Action | P(success) | U(success) | U(failure)
A | 0.5 | 5 | 4
B | 0.6 | 9 | -5
C | 0.7 | 10 | -4

A rational agent follows the MEU principle. Which action does it choose?

A. Action C, because it has the best possible outcome (10)
B. Action A (EU = 4.5)
C. Action B (EU = 3.4)
D. Action C (EU = 5.8)

Answer: D. Action C (EU = 5.8)

Working:

- EU(A) = 0.5×5 + 0.5×(4) = 4.5.
- EU(B) = 0.6×9 + 0.4×(-5) = 3.4.
- EU(C) = 0.7×10 + 0.3×(-4) = 5.8.
- Choose argmax → C.

Option notes:

- A: MEU uses expected utility, not the best case.
- B: EU(A) = 4.5 is lower.
- C: EU(B) = 3.4 is lower.

Takeaway: MEU: action = $\text{argmax}_a \Sigma P(s'|a,s)\cdot U(s')$. Like Route B (EU 7) beating Route A (EU 6).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B33. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s2 | 0.7 | 1 | 9
s7 | 0.2 | 2 | 7
s4 | 0.1 | -0.04 | -2

With γ = 0.5, compute $Q(s, a) = \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$ for the outcomes in the table.

A. 3.750
B. 8.596
C. 9.960
D. 4.846

Answer: D. 4.846

Working:

- Q(s,a) = 0.7×(1 + 0.5×9) + 0.2×(2 + 0.5×7) + 0.1×(-0.04 + 0.5×-2) = 4.8460.

Option notes:

- A: Left out the immediate rewards R(s,a,s′).
- B: Forgot the discount γ.
- C: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B34. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s7 | 0.7 | 5 | 3
s1 | 0.3 | 5 | -3

With γ = 0.95, compute $Q(s, a) = \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$ for the outcomes in the table.

A. 6.200
B. 1.140

C. 6.140
D. 10.000

Answer: C. 6.140

Working:

- Q(s,a) = 0.7×(5 + 0.95×3) + 0.3×(5 + 0.95×-3) = 6.1400.

Option notes:

- A: Forgot the discount γ.
- B: Left out the immediate rewards R(s,a,s′).
- D: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B35. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s6 | 0.7 | 0 | -2
s6 | 0.2 | 1 | 12
s9 | 0.1 | 1 | 20

With γ = 0.8, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 3.300
B. 2.400
C. 2.700
D. 26.000

Answer: C. 2.700

Working:

- Q(s,a) = 0.7×(0 + 0.8×-2) + 0.2×(1 + 0.8×12) + 0.1×(1 + 0.8×20) = 2.7000.

Option notes:

- A: Forgot the discount γ.
- B: Left out the immediate rewards R(s,a,s′).
- D: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B36. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.5 | 2 | 3
s2 | 0.5 | 0 | 10

With γ = 0.5, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 3.250
B. 8.500
C. 7.500
D. 4.250

Answer: D. 4.250

Working:

- Q(s,a) = 0.5×(2 + 0.5×3) + 0.5×(0 + 0.5×10) = 4.2500.

Option notes:

- A: Left out the immediate rewards R(s,a,s′).
- B: Forgot to weight by transition probabilities.
- C: Forgot the discount γ.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B37. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.6 | -0.04 | 15
s1 | 0.2 | 1 | 9
s9 | 0.2 | -0.04 | 13

With γ = 1, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 13.400
B. 13.568
C. 15.603
D. 37.920

Answer: B. 13.568

Working:

- Q(s,a) = 0.6×(-0.04 + 1×15) + 0.2×(1 + 1×9) + 0.2×(-0.04 + 1×13) = 13.5680.

Option notes:

- A: Left out the immediate rewards R(s,a,s′).
- C: Arithmetic slip: recompute each step carefully.
- D: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B38. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s5 | 0.6 | 0 | 19
s6 | 0.2 | 5 | 13
s4 | 0.2 | 0 | 14

With γ = 0.9, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 46.400
B. 16.120
C. 15.120
D. 17.800

Answer: B. 16.120

Working:

- Q(s,a) = 0.6×(0 + 0.9×19) + 0.2×(5 + 0.9×13) + 0.2×(0 + 0.9×14) = 16.1200.

Option notes:

- A: Forgot to weight by transition probabilities.
- C: Left out the immediate rewards R(s,a,s′).

- D: Forgot the discount γ.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B39. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s8 | 0.8 | 5 | 19
s1 | 0.1 | 2 | 10
s8 | 0.1 | 2 | 14

With γ = 0.95, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 16.720
B. 49.850
C. 22.000
D. 21.120

Answer: D. 21.120

Working:

- Q(s,a) = 0.8×(5 + 0.95×19) + 0.1×(2 + 0.95×10) + 0.1×(2 + 0.95×14) = 21.1200.

Option notes:

- A: Left out the immediate rewards R(s,a,s′).
- B: Forgot to weight by transition probabilities.
- C: Forgot the discount γ.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B40. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s7 | 0.8 | -1 | 18
s4 | 0.2 | 1 | 15

With γ = 0.5, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 8.700
B. 16.800
C. 16.500
D. 8.100

Answer: D. 8.100

Working:

- Q(s,a) = 0.8×(-1 + 0.5×18) + 0.2×(1 + 0.5×15) = 8.1000.

Option notes:

- A: Left out the immediate rewards R(s,a,s′).
- B: Forgot the discount γ.
- C: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B41. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.8 | 2 | 4
s9 | 0.1 | 0 | 17
s8 | 0.1 | 0 | -2

With γ = 0.9, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 4.230
B. 6.300
C. 5.830
D. 19.100

Answer: C. 5.830

Working:

- Q(s,a) = 0.8×(2 + 0.9×4) + 0.1×(0 + 0.9×17) + 0.1×(0 + 0.9×-2) = 5.8300.

Option notes:

- A: Left out the immediate rewards R(s,a,s′).
- B: Forgot the discount γ.
- D: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B42. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.8 | 2 | -2
s8 | 0.1 | 2 | 9
s9 | 0.1 | 0 | 19

With γ = 0.5, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 3.000
B. 2.400
C. 0.600
D. 17.000

Answer: B. 2.400

Working:

- Q(s,a) = 0.8×(2 + 0.5×-2) + 0.1×(2 + 0.5×9) + 0.1×(0 + 0.5×19) = 2.4000.

Option notes:

- A: Forgot the discount γ.
- C: Left out the immediate rewards R(s,a,s′).
- D: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B43. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s7 | 0.8 | 0 | -2

s9 | 0.2 | 0 | -2

With γ = 0.5, compute $Q(s, a) = \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$ for the outcomes in the table.

A. -0.850
B. -1.150
C. -1.000
D. -2.000

Answer: C. -1.000

Working:

- $Q(s,a) = 0.8\times(0 + 0.5\times-2) + 0.2\times(0 + 0.5\times-2) = -1.0000$.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- D: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B44. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.7 | 1 | 19
s3 | 0.2 | -0.04 | 14
s5 | 0.1 | 5 | 5

With γ = 0.8, compute $Q(s, a) = \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$ for the outcomes in the table.

A. 13.280
B. 14.472
C. 17.792
D. 36.360

Answer: B. 14.472

Working:

- $Q(s,a) = 0.7\times(1 + 0.8\times19) + 0.2\times(-0.04 + 0.8\times14) + 0.1\times(5 + 0.8\times5) = 14.4720$.

Option notes:

- A: Left out the immediate rewards R(s,a,s′).
- C: Forgot the discount γ.
- D: Forgot to weight by transition probabilities.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B45. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s4 | 0.8 | 2 | 14
s9 | 0.1 | -0.04 | 8
s9 | 0.1 | 2 | 10

With γ = 1, compute $Q(s, a) = \Sigma_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$ for the outcomes in the table.

A. 35.960
B. 13.000

C. 17.015
D. 14.796

Answer: D. 14.796

Working:

- Q(s,a) = 0.8×(2 + 1×14) + 0.1×(-0.04 + 1×8) + 0.1×(2 + 1×10) = 14.7960.

Option notes:

- A: Forgot to weight by transition probabilities.
- B: Left out the immediate rewards R(s,a,s′).
- C: Arithmetic slip: recompute each step carefully.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B46. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.7 | 2 | 7
s6 | 0.3 | 1 | -3

With γ = 0.8, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 4.900
B. 6.200
C. 3.200
D. 5.700

Answer: A. 4.900

Working:

- Q(s,a) = 0.7×(2 + 0.8×7) + 0.3×(1 + 0.8×-3) = 4.9000.

Option notes:

- B: Forgot to weight by transition probabilities.
- C: Left out the immediate rewards R(s,a,s′).
- D: Forgot the discount γ.

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B47. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s3 | 0.9 | 1 | 18
s5 | 0.1 | -1 | 11

With γ = 1, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 29.000
B. 20.815
C. 17.300
D. 18.100

Answer: D. 18.100

Working:

- Q(s,a) = 0.9×(1 + 1×18) + 0.1×(-1 + 1×11) = 18.1000.

Option notes:

- A: Forgot to weight by transition probabilities.
- B: Arithmetic slip: recompute each step carefully.
- C: Left out the immediate rewards R(s,a,s′).

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B48. [MCQ · Medium · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.9 | -1 | 7
s2 | 0.1 | 5 | -2

With γ = 0.9, compute Q(s, a) = $\Sigma_{s'}$ P(s′|s,a)[R(s,a,s′) + γU(s′)] for the outcomes in the table.

A. 8.500
B. 5.700
C. 5.090
D. 5.490

Answer: C. 5.090

Working:

- Q(s,a) = 0.9×(-1 + 0.9×7) + 0.1×(5 + 0.9×-2) = 5.0900.

Option notes:

- A: Forgot to weight by transition probabilities.
- B: Forgot the discount γ.
- D: Left out the immediate rewards R(s,a,s′).

Takeaway: Q(s,a) = expected [reward + discounted value of where you land].

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B49. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s2 | 0.7 | 1 | 9
s7 | 0.2 | 2 | 7
s4 | 0.1 | -0.04 | -2

```
Action b: next state | P(s´|s,a) | R(s,a,s´) | U(s´)
s4 | 0.5 | -1 | 15
s3 | 0.5 | 5 | 17
```

State s has two actions a and b (tables). With γ = 0.5, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 4.846; Q(s,b) = 10.000; U(s) = 10.000; best action = b

Working:

- Q(s,a) = 0.7×(1 + 0.5×9) + 0.2×(2 + 0.5×7) + 0.1×(-0.04 + 0.5×-2) = 4.8460.

- Q(s,b) = 0.5×(-1 + 0.5×15) + 0.5×(5 + 0.5×17) = 10.0000.
- U(s) = max(4.846, 10.000) = 10.000 → π*(s) = argmax = b.

Takeaway: $U(s) = \max_a Q(s,a)$; $\pi^*(s) = \text{argmax}_a Q(s,a)$.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B50. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s7 | 0.7 | 5 | 3
s1 | 0.3 | 5 | -3

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.7 | 1 | 3
s7 | 0.2 | 0 | 2
s6 | 0.1 | 2 | 11
```

State s has two actions a and b (tables). With γ = 0.95, give Q(s,a), Q(s,b), $U(s) = \max_a Q(s,a)$ (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 6.140; Q(s,b) = 4.320; U(s) = 6.140; best action = a

Working:

- Q(s,a) = 0.7×(5 + 0.95×3) + 0.3×(5 + 0.95×-3) = 6.1400.
- Q(s,b) = 0.7×(1 + 0.95×3) + 0.2×(0 + 0.95×2) + 0.1×(2 + 0.95×11) = 4.3200.
- U(s) = max(6.140, 4.320) = 6.140 → π*(s) = argmax = a.

Takeaway: $U(s) = \max_a Q(s,a)$; $\pi^*(s) = \text{argmax}_a Q(s,a)$.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B51. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s6 | 0.7 | 0 | -2
s6 | 0.2 | 1 | 12
s9 | 0.1 | 1 | 20

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s8 | 0.8 | -0.04 | 0
s6 | 0.2 | -1 | 13
```

State s has two actions a and b (tables). With γ = 0.8, give Q(s,a), Q(s,b), $U(s) = \max_a Q(s,a)$ (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 2.700; Q(s,b) = 1.848; U(s) = 2.700; best action = a

Working:

- Q(s,a) = 0.7×(0 + 0.8×-2) + 0.2×(1 + 0.8×12) + 0.1×(1 + 0.8×20) = 2.7000.
- Q(s,b) = 0.8×(-0.04 + 0.8×0) + 0.2×(-1 + 0.8×13) = 1.8480.
- U(s) = max(2.700, 1.848) = 2.700 → π*(s) = argmax = a.

Takeaway: $U(s) = \max_a Q(s,a)$; $\pi^*(s) = \text{argmax}_a Q(s,a)$.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B52. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.5 | 2 | 3
s2 | 0.5 | 0 | 10

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s7 | 0.7 | 0 | 15
s5 | 0.2 | -0.04 | -5
s8 | 0.1 | 2 | 1
```

State s has two actions a and b (tables). With γ = 0.5, give Q(s,a), Q(s,b), $U(s) = \max_a Q(s,a)$ (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 4.250; Q(s,b) = 4.992; U(s) = 4.992; best action = b

Working:

- Q(s,a) = 0.5×(2 + 0.5×3) + 0.5×(0 + 0.5×10) = 4.2500.
- Q(s,b) = 0.7×(0 + 0.5×15) + 0.2×(-0.04 + 0.5×-5) + 0.1×(2 + 0.5×1) = 4.9920.
- U(s) = max(4.250, 4.992) = 4.992 → π*(s) = argmax = b.

Takeaway: $U(s) = \max_a Q(s,a)$; $\pi^*(s) = \text{argmax}_a Q(s,a)$.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B53. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.6 | -0.04 | 15
s1 | 0.2 | 1 | 9
s9 | 0.2 | -0.04 | 13

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s8 | 0.7 | 0 | -5
s8 | 0.2 | -1 | 5
s3 | 0.1 | 2 | 10
```

State s has two actions a and b (tables). With γ = 1, give Q(s,a), Q(s,b), $U(s) = \max_a Q(s,a)$ (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 13.568; Q(s,b) = -1.500; U(s) = 13.568; best action = a

Working:

- Q(s,a) = 0.6×(-0.04 + 1×15) + 0.2×(1 + 1×9) + 0.2×(-0.04 + 1×13) = 13.5680.
- Q(s,b) = 0.7×(0 + 1×-5) + 0.2×(-1 + 1×5) + 0.1×(2 + 1×10) = -1.5000.
- U(s) = max(13.568, -1.500) = 13.568 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B54. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s5 | 0.6 | 0 | 19
s6 | 0.2 | 5 | 13
s4 | 0.2 | 0 | 14

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.9 | 2 | 7
s5 | 0.1 | 5 | 16
```

State s has two actions a and b (tables). With γ = 0.9, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 16.120; Q(s,b) = 9.410; U(s) = 16.120; best action = a

Working:

- Q(s,a) = 0.6×(0 + 0.9×19) + 0.2×(5 + 0.9×13) + 0.2×(0 + 0.9×14) = 16.1200.
- Q(s,b) = 0.9×(2 + 0.9×7) + 0.1×(5 + 0.9×16) = 9.4100.
- U(s) = max(16.120, 9.410) = 16.120 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B55. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s8 | 0.8 | 5 | 19
s1 | 0.1 | 2 | 10
s8 | 0.1 | 2 | 14

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s5 | 0.7 | 5 | 12
s9 | 0.3 | 0 | 13
```

State s has two actions a and b (tables). With γ = 0.95, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 21.120; Q(s,b) = 15.185; U(s) = 21.120; best action = a

Working:

- Q(s,a) = 0.8×(5 + 0.95×19) + 0.1×(2 + 0.95×10) + 0.1×(2 + 0.95×14) = 21.1200.
- Q(s,b) = 0.7×(5 + 0.95×12) + 0.3×(0 + 0.95×13) = 15.1850.
- U(s) = max(21.120, 15.185) = 21.120 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B56. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s7 | 0.8 | -1 | 18
s4 | 0.2 | 1 | 15

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s5 | 0.7 | -1 | -4
s1 | 0.3 | -0.04 | -4
```

State s has two actions a and b (tables). With γ = 0.5, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 8.100; Q(s,b) = -2.712; U(s) = 8.100; best action = a

Working:

- Q(s,a) = 0.8×(-1 + 0.5×18) + 0.2×(1 + 0.5×15) = 8.1000.
- Q(s,b) = 0.7×(-1 + 0.5×-4) + 0.3×(-0.04 + 0.5×-4) = -2.7120.
- U(s) = max(8.100, -2.712) = 8.100 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B57. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.8 | 2 | 4
s9 | 0.1 | 0 | 17
s8 | 0.1 | 0 | -2

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s3 | 0.6 | 5 | 12
s5 | 0.2 | -0.04 | 4
s1 | 0.2 | 2 | 8
```

State s has two actions a and b (tables). With γ = 0.9, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____

(3) U(s) = _____
(4) best action = _____

Answer: Q(s,a) = 5.830; Q(s,b) = 12.032; U(s) = 12.032; best action = b

Working:

- Q(s,a) = 0.8×(2 + 0.9×4) + 0.1×(0 + 0.9×17) + 0.1×(0 + 0.9×-2) = 5.8300.
- Q(s,b) = 0.6×(5 + 0.9×12) + 0.2×(-0.04 + 0.9×4) + 0.2×(2 + 0.9×8) = 12.0320.
- U(s) = max(5.830, 12.032) = 12.032 → π*(s) = argmax = b.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B58. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.8 | 2 | -2
s8 | 0.1 | 2 | 9
s9 | 0.1 | 0 | 19

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.8 | 2 | 20
s3 | 0.1 | 0 | 17
s6 | 0.1 | 1 | 13
```

State s has two actions a and b (tables). With γ = 0.5, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = _____
(2) Q(s,b) = _____
(3) U(s) = _____
(4) best action = _____

Answer: Q(s,a) = 2.400; Q(s,b) = 11.200; U(s) = 11.200; best action = b

Working:

- Q(s,a) = 0.8×(2 + 0.5×-2) + 0.1×(2 + 0.5×9) + 0.1×(0 + 0.5×19) = 2.4000.
- Q(s,b) = 0.8×(2 + 0.5×20) + 0.1×(0 + 0.5×17) + 0.1×(1 + 0.5×13) = 11.2000.
- U(s) = max(2.400, 11.200) = 11.200 → π*(s) = argmax = b.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B59. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s7 | 0.8 | 0 | -2
s9 | 0.2 | 0 | -2

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s5 | 0.6 | 0 | 9
s9 | 0.2 | 1 | -1
s8 | 0.2 | -1 | 20
```

State s has two actions a and b (tables). With γ = 0.5, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = -1.000; Q(s,b) = 4.600; U(s) = 4.600; best action = b

Working:

- Q(s,a) = 0.8×(0 + 0.5×-2) + 0.2×(0 + 0.5×-2) = -1.0000.
- Q(s,b) = 0.6×(0 + 0.5×9) + 0.2×(1 + 0.5×-1) + 0.2×(-1 + 0.5×20) = 4.6000.
- U(s) = max(-1.000, 4.600) = 4.600 → π*(s) = argmax = b.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B60. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.7 | 1 | 19
s3 | 0.2 | -0.04 | 14
s5 | 0.1 | 5 | 5

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s2 | 0.8 | 5 | 12
s4 | 0.2 | 1 | 16
```

State s has two actions a and b (tables). With γ = 0.8, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 14.472; Q(s,b) = 14.440; U(s) = 14.472; best action = a

Working:

- Q(s,a) = 0.7×(1 + 0.8×19) + 0.2×(-0.04 + 0.8×14) + 0.1×(5 + 0.8×5) = 14.4720.
- Q(s,b) = 0.8×(5 + 0.8×12) + 0.2×(1 + 0.8×16) = 14.4400.
- U(s) = max(14.472, 14.440) = 14.472 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B61. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s4 | 0.8 | 2 | 14
s9 | 0.1 | -0.04 | 8
s9 | 0.1 | 2 | 10

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s3 | 0.6 | 5 | 15
s4 | 0.2 | 1 | 6
s6 | 0.2 | 1 | -2
```

State s has two actions a and b (tables). With γ = 1, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 14.796; Q(s,b) = 13.200; U(s) = 14.796; best action = a

Working:

- Q(s,a) = 0.8×(2 + 1×14) + 0.1×(-0.04 + 1×8) + 0.1×(2 + 1×10) = 14.7960.
- Q(s,b) = 0.6×(5 + 1×15) + 0.2×(1 + 1×6) + 0.2×(1 + 1×-2) = 13.2000.
- U(s) = max(14.796, 13.200) = 14.796 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B62. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s1 | 0.7 | 2 | 7
s6 | 0.3 | 1 | -3

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s4 | 0.7 | 0 | 16
s2 | 0.2 | 0 | 8
s8 | 0.1 | 2 | 13
```

State s has two actions a and b (tables). With γ = 0.8, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 4.900; Q(s,b) = 11.480; U(s) = 11.480; best action = b

Working:

- Q(s,a) = 0.7×(2 + 0.8×7) + 0.3×(1 + 0.8×-3) = 4.9000.
- Q(s,b) = 0.7×(0 + 0.8×16) + 0.2×(0 + 0.8×8) + 0.1×(2 + 0.8×13) = 11.4800.
- U(s) = max(4.900, 11.480) = 11.480 → π*(s) = argmax = b.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B63. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s3 | 0.9 | 1 | 18
s5 | 0.1 | -1 | 11

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s8 | 0.7 | 1 | 16
s1 | 0.2 | 0 | 19
s1 | 0.1 | 0 | 10
```

State s has two actions a and b (tables). With γ = 1, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 18.100; Q(s,b) = 16.700; U(s) = 18.100; best action = a

Working:

- Q(s,a) = 0.9×(1 + 1×18) + 0.1×(-1 + 1×11) = 18.1000.
- Q(s,b) = 0.7×(1 + 1×16) + 0.2×(0 + 1×19) + 0.1×(0 + 1×10) = 16.7000.
- U(s) = max(18.100, 16.700) = 18.100 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B64. [BLANKS · Hard · calculation]

Action a: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s9 | 0.9 | -1 | 7
s2 | 0.1 | 5 | -2

```
Action b: next state | P(s′|s,a) | R(s,a,s′) | U(s′)
s3 | 0.5 | 0 | -5
s9 | 0.5 | 5 | -1
```

State s has two actions a and b (tables). With γ = 0.9, give Q(s,a), Q(s,b), U(s) = $\max_a$ Q(s,a) (3 dp) and the optimal action (a or b).

(1) Q(s,a) = ____
(2) Q(s,b) = ____
(3) U(s) = ____
(4) best action = ____

Answer: Q(s,a) = 5.090; Q(s,b) = -0.200; U(s) = 5.090; best action = a

Working:

- Q(s,a) = 0.9×(-1 + 0.9×7) + 0.1×(5 + 0.9×-2) = 5.0900.
- Q(s,b) = 0.5×(0 + 0.9×-5) + 0.5×(5 + 0.9×-1) = -0.2000.
- U(s) = max(5.090, -0.200) = 5.090 → π*(s) = argmax = a.

Takeaway: U(s) = $\max_a$ Q(s,a); π*(s) = argmax_a Q(s,a).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B65. [BLANK · Exam+ · calculation]

| U=0.53 |
U=0.83 | s (U=0.44) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.5110

Working:

- Up: 0.8×(−0.04 + 0.53)
- Left slip: 0.1×(−0.04 + 0.83)
- Right slip: 0.1×(−0.04 + 0.44) (wall: stays in s)
- Q = 0.5110.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B66. [BLANK · Exam+ · calculation]

| U=0.49 |
U=0.41 | s (U=0.75) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.4680

Working:

- Up: 0.8×(−0.04 + 0.49)
- Left slip: 0.1×(−0.04 + 0.41)
- Right slip: 0.1×(−0.04 + 0.75) (wall: stays in s)
- Q = 0.4680.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B67. [BLANK · Exam+ · calculation]

| U=0.46 |
wall | s (U=0.43) | U=0.71

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.4420

Working:

- Up: 0.8×(−0.04 + 0.46)
- Left slip: 0.1×(−0.04 + 0.43) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.71)
- Q = 0.4420.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B68. [BLANK · Exam+ · calculation]

| U=0.51 |
U=0.36 | s (U=0.51) | U=0.25

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.4290

Working:

- Up: 0.8×(−0.04 + 0.51)
- Left slip: 0.1×(−0.04 + 0.36)
- Right slip: 0.1×(−0.04 + 0.25)
- Q = 0.4290.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B69. [BLANK · Exam+ · calculation]

| U=0.37 |
wall | s (U=0.6) | U=0.21

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.3370

Working:

- Up: 0.8×(−0.04 + 0.37)
- Left slip: 0.1×(−0.04 + 0.6) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.21)
- Q = 0.3370.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B70. [BLANK · Exam+ · calculation]

| U=0.3 |
U=0.5 | s (U=0.52) | U=0.5

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.3000

Working:

- Up: 0.8×(−0.04 + 0.3)
- Left slip: 0.1×(−0.04 + 0.5)
- Right slip: 0.1×(−0.04 + 0.5)
- Q = 0.3000.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B71. [BLANK · Exam+ · calculation]

| U=0.45 |
U=0.74 | s (U=0.35) | U=0.47

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.4410

Working:

- Up: 0.8×(−0.04 + 0.45)
- Left slip: 0.1×(−0.04 + 0.74)
- Right slip: 0.1×(−0.04 + 0.47)
- Q = 0.4410.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B72. [BLANK · Exam+ · calculation]

| U=0.57 |
wall | s (U=0.79) | U=0.65

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.5600

Working:

- Up: 0.8×(−0.04 + 0.57)
- Left slip: 0.1×(−0.04 + 0.79) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.65)
- Q = 0.5600.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B73. [BLANK · Exam+ · calculation]

| U=0.3 |
wall | s (U=0.14) | U=0.22

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.2360

Working:

- Up: 0.8×(−0.04 + 0.3)
- Left slip: 0.1×(−0.04 + 0.14) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.22)
- Q = 0.2360.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B74. [BLANK · Exam+ · calculation]

| U=0.73 |
wall | s (U=0.56) | U=0.31

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the

current utilities shown, compute Q(s, Up).

Answer: 0.6310

Working:

- Up: 0.8×(−0.04 + 0.73)
- Left slip: 0.1×(−0.04 + 0.56) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.31)
- Q = 0.6310.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B75. [BLANK · Exam+ · calculation]

| U=0.13 |
U=0.65 | s (U=0.7) | U=0.63

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.1920

Working:

- Up: 0.8×(−0.04 + 0.13)
- Left slip: 0.1×(−0.04 + 0.65)
- Right slip: 0.1×(−0.04 + 0.63)
- Q = 0.1920.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B76. [BLANK · Exam+ · calculation]

| U=0.31 |
U=0.44 | s (U=0.48) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.3000

Working:

- Up: 0.8×(−0.04 + 0.31)
- Left slip: 0.1×(−0.04 + 0.44)
- Right slip: 0.1×(−0.04 + 0.48) (wall: stays in s)
- Q = 0.3000.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B77. [BLANK · Exam+ · calculation]

| U=0.74 |
U=0.36 | s (U=0.46) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.6340

Working:

- Up: 0.8×(−0.04 + 0.74)
- Left slip: 0.1×(−0.04 + 0.36)
- Right slip: 0.1×(−0.04 + 0.46) (wall: stays in s)
- Q = 0.6340.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B78. [BLANK · Exam+ · calculation]

| U=0.41 |
U=0.13 | s (U=0.2) | wall

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.3210

Working:

- Up: 0.8×(−0.04 + 0.41)
- Left slip: 0.1×(−0.04 + 0.13)
- Right slip: 0.1×(−0.04 + 0.2) (wall: stays in s)
- Q = 0.3210.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B79. [BLANK · Exam+ · calculation]

| U=0.27 |
U=0.13 | s (U=0.74) | U=0.62

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.2510

Working:

- Up: 0.8×(−0.04 + 0.27)
- Left slip: 0.1×(−0.04 + 0.13)
- Right slip: 0.1×(−0.04 + 0.62)
- Q = 0.2510.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B80. [BLANK · Exam+ · calculation]

| U=0.67 |
wall | s (U=0.88) | U=0.5

Grid world of Workshop 6: action Up moves up with probability 0.8 and slips left or right with probability 0.1 each; bumping into a wall leaves the agent in s. Every move has reward R = −0.04 and γ = 1. Using the current utilities shown, compute Q(s, Up).

Answer: 0.6340

Working:

- Up: 0.8×(−0.04 + 0.67)
- Left slip: 0.1×(−0.04 + 0.88) (wall: stays in s)
- Right slip: 0.1×(−0.04 + 0.5)
- Q = 0.6340.

Takeaway: Slips go perpendicular to the intended move; bumping a wall means staying put.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B81. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 0.95, α = 0.5, γ = 0.95. The agent takes a, receives r = -1, lands in s′ where Q(s′,·) = [2.3, 0.6, 0.6]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 1.185
B. 1.125
C. 0.529
D. 1.068

Answer: D. 1.068

Working:

- max Q(s′,·) = 2.3.
- TD target = -1 + 0.95×2.3 = 1.1850.
- TD error = 1.1850 − 0.95 = 0.2350.
- New Q = 0.95 + 0.5×0.2350 = 1.0675.

Option notes:

- A: That is the TD target; you still need Q + α(target − Q).
- B: Forgot the discount γ on max Q(s′,a′).
- C: Used the average Q(s′,·); Q-learning uses the MAX.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B82. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 2.14, α = 0.5, γ = 0.8. The agent takes a, receives r = -0.04, lands in s′ where Q(s′,·) = [3.2, 0.3, 3.8]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.950
B. 2.570
C. 3.000
D. 2.023

Answer: B. 2.570

Working:

- max Q(s′,·) = 3.8.
- TD target = -0.04 + 0.8×3.8 = 3.0000.
- TD error = 3.0000 − 2.14 = 0.8600.
- New Q = 2.14 + 0.5×0.8600 = 2.5700.

Option notes:

- A: Forgot the discount γ on max Q(s′,a′).
- C: That is the TD target; you still need Q + α(target − Q).
- D: Used the average Q(s′,·); Q-learning uses the MAX.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B83. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 0.73, α = 0.2, γ = 0.8. The agent takes a, receives r = -1, lands in s′ where Q(s′,·) = [-0.6, 0, 0.7]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. -0.440
B. 0.389
C. 0.496
D. 0.524

Answer: C. 0.496

Working:

- max Q(s′,·) = 0.7.
- TD target = -1 + 0.8×0.7 = -0.4400.
- TD error = -0.4400 − 0.73 = -1.1700.
- New Q = 0.73 + 0.2×-1.1700 = 0.4960.

Option notes:

- A: That is the TD target; you still need Q + α(target − Q).
- B: Used the average Q(s′,·); Q-learning uses the MAX.
- D: Forgot the discount γ on max Q(s′,a′).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B84. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 2.73, α = 0.1, γ = 0.95. The agent takes a, receives r = 2, lands in s′ where Q(s′,·) = [-0.4, 3.8, 3.4]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.872
B. 3.037
C. 5.610
D. 3.018

Answer: D. 3.018

Working:

- max Q(s′,·) = 3.8.
- TD target = 2 + 0.95×3.8 = 5.6100.
- TD error = 5.6100 − 2.73 = 2.8800.

- New Q = 2.73 + 0.1×2.8800 = 3.0180.

Option notes:

- A: Used the average Q(s′,·); Q-learning uses the MAX.
- B: Forgot the discount γ on max Q(s′,a′).
- C: That is the TD target; you still need Q + α(target − Q).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B85. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = -0.75, α = 0.1, γ = 0.9. The agent takes a, receives r = 0, lands in s′ where Q(s′,·) = [0.4, 1, -0.6]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. -0.575
B. -0.585
C. 0.900
D. -0.651

Answer: B. -0.585

Working:

- max Q(s′,·) = 1.
- TD target = 0 + 0.9×1 = 0.9000.
- TD error = 0.9000 − -0.75 = 1.6500.
- New Q = -0.75 + 0.1×1.6500 = -0.5850.

Option notes:

- A: Forgot the discount γ on max Q(s′,a′).
- C: That is the TD target; you still need Q + α(target − Q).
- D: Used the average Q(s′,·); Q-learning uses the MAX.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B86. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 2.91, α = 0.2, γ = 0.95. The agent takes a, receives r = -1, lands in s′ where Q(s′,·) = [3.3, 1.5, 1.1]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.135
B. 2.755
C. 2.788
D. 2.502

Answer: B. 2.755

Working:

- max Q(s′,·) = 3.3.
- TD target = -1 + 0.95×3.3 = 2.1350.
- TD error = 2.1350 − 2.91 = -0.7750.
- New Q = 2.91 + 0.2×-0.7750 = 2.7550.

Option notes:

- A: That is the TD target; you still need Q + α(target − Q).
- C: Forgot the discount γ on max Q(s′,a′).

- D: Used the average Q(s′,·); Q-learning uses the MAX.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B87. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 1.42, α = 0.5, γ = 0.95. The agent takes a, receives r = 5, lands in s′ where Q(s′,·) = [-0.5, 0.9, 1.9]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 3.574
B. 6.805
C. 4.160
D. 4.113

Answer: D. 4.113

Working:

- max Q(s′,·) = 1.9.
- TD target = 5 + 0.95×1.9 = 6.8050.
- TD error = 6.8050 − 1.42 = 5.3850.
- New Q = 1.42 + 0.5×5.3850 = 4.1125.

Option notes:

- A: Used the average Q(s′,·); Q-learning uses the MAX.
- B: That is the TD target; you still need Q + α(target − Q).
- C: Forgot the discount γ on max Q(s′,a′).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B88. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 2.41, α = 0.1, γ = 1. The agent takes a, receives r = -1, lands in s′ where Q(s′,·) = [1.6, 4, 0.3]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 3.000
B. 2.266
C. 2.469
D. 2.839

Answer: C. 2.469

Working:

- max Q(s′,·) = 4.
- TD target = -1 + 1×4 = 3.0000.
- TD error = 3.0000 − 2.41 = 0.5900.
- New Q = 2.41 + 0.1×0.5900 = 2.4690.

Option notes:

- A: That is the TD target; you still need Q + α(target − Q).
- B: Used the average Q(s′,·); Q-learning uses the MAX.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B89. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 2.38, α = 0.1, γ = 1. The agent takes a, receives r = 2, lands in s′ where Q(s′,·) = [1.3, 2.2, 1.3]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 4.200
B. 2.562
C. 2.502
D. 2.946

Answer: B. 2.562

Working:

- max Q(s′,·) = 2.2.
- TD target = 2 + 1×2.2 = 4.2000.
- TD error = 4.2000 − 2.38 = 1.8200.
- New Q = 2.38 + 0.1×1.8200 = 2.5620.

Option notes:

- A: That is the TD target; you still need Q + α(target − Q).
- C: Used the average Q(s′,·); Q-learning uses the MAX.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B90. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 1.78, α = 0.2, γ = 0.95. The agent takes a, receives r = 0, lands in s′ where Q(s′,·) = [-0.4, 3.9, 2]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.204
B. 1.772
C. 2.165
D. 3.705

Answer: C. 2.165

Working:

- max Q(s′,·) = 3.9.
- TD target = 0 + 0.95×3.9 = 3.7050.
- TD error = 3.7050 − 1.78 = 1.9250.
- New Q = 1.78 + 0.2×1.9250 = 2.1650.

Option notes:

- A: Forgot the discount γ on max Q(s′,a′).
- B: Used the average Q(s′,·); Q-learning uses the MAX.
- D: That is the TD target; you still need Q + α(target − Q).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B91. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 0.61, α = 0.5, γ = 1. The agent takes a, receives r = -1, lands in s′ where Q(s′,·) = [2.5, 3.8, 0.2]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.800
B. 1.961
C. 0.888
D. 1.705

Answer: D. 1.705

Working:

- max Q(s′,·) = 3.8.
- TD target = -1 + 1×3.8 = 2.8000.
- TD error = 2.8000 − 0.61 = 2.1900.
- New Q = 0.61 + 0.5×2.1900 = 1.7050.

Option notes:

- A: That is the TD target; you still need Q + α(target − Q).
- B: Arithmetic slip: recompute each step carefully.
- C: Used the average Q(s′,·); Q-learning uses the MAX.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B92. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 2.26, α = 0.5, γ = 0.9. The agent takes a, receives r = 5, lands in s′ where Q(s′,·) = [0, 2, 0.1]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 3.945
B. 4.630
C. 4.530
D. 6.800

Answer: C. 4.530

Working:

- max Q(s′,·) = 2.
- TD target = 5 + 0.9×2 = 6.8000.
- TD error = 6.8000 − 2.26 = 4.5400.
- New Q = 2.26 + 0.5×4.5400 = 4.5300.

Option notes:

- A: Used the average Q(s′,·); Q-learning uses the MAX.
- B: Forgot the discount γ on max Q(s′,a′).
- D: That is the TD target; you still need Q + α(target − Q).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B93. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 1.02, α = 0.5, γ = 0.8. The agent takes a, receives r = 0, lands in s′ where Q(s′,·) = [1, 2.7, 1.5]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.160
B. 1.203
C. 1.860
D. 1.590

Answer: D. 1.590

Working:

- max Q(s′,·) = 2.7.
- TD target = 0 + 0.8×2.7 = 2.1600.
- TD error = 2.1600 − 1.02 = 1.1400.
- New Q = 1.02 + 0.5×1.1400 = 1.5900.

Option notes:

- A: That is the TD target; you still need Q + α(target − Q).
- B: Used the average Q(s′,·); Q-learning uses the MAX.
- C: Forgot the discount γ on max Q(s′,a′).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B94. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 0.85, α = 0.1, γ = 0.95. The agent takes a, receives r = -0.04, lands in s′ where Q(s′,·) = [2.9, 0, 2.9]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 1.051
B. 1.037
C. 2.715
D. 0.945

Answer: B. 1.037

Working:

- max Q(s′,·) = 2.9.
- TD target = -0.04 + 0.95×2.9 = 2.7150.
- TD error = 2.7150 − 0.85 = 1.8650.
- New Q = 0.85 + 0.1×1.8650 = 1.0365.

Option notes:

- A: Forgot the discount γ on max Q(s′,a′).
- C: That is the TD target; you still need Q + α(target − Q).
- D: Used the average Q(s′,·); Q-learning uses the MAX.

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B95. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 2.2, α = 0.1, γ = 0.5. The agent takes a, receives r = 0, lands in s′ where Q(s′,·) = [-0.1, 2.8, -0.5]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.120
B. 2.017
C. 2.260
D. 1.400

Answer: A. 2.120

Working:

- max Q(s′,·) = 2.8.
- TD target = 0 + 0.5×2.8 = 1.4000.

- TD error = 1.4000 − 2.2 = -0.8000.
- New Q = 2.2 + 0.1×-0.8000 = 2.1200.

Option notes:

- B: Used the average Q(s′,·); Q-learning uses the MAX.
- C: Forgot the discount γ on max Q(s′,a′).
- D: That is the TD target; you still need Q + α(target − Q).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B96. [MCQ · Medium · calculation]

Q-learning: Q(s,a) = 0.81, α = 0.2, γ = 0.95. The agent takes a, receives r = 5, lands in s′ where Q(s′,·) = [1.1, 1, 2.2]. New Q(s,a) = Q + α[r + γ $\max_{a'}$ Q(s′,a′) − Q]?

A. 2.066
B. 1.920
C. 2.088
D. 7.090

Answer: A. 2.066

Working:

- max Q(s′,·) = 2.2.
- TD target = 5 + 0.95×2.2 = 7.0900.
- TD error = 7.0900 − 0.81 = 6.2800.
- New Q = 0.81 + 0.2×6.2800 = 2.0660.

Option notes:

- B: Used the average Q(s′,·); Q-learning uses the MAX.
- C: Forgot the discount γ on max Q(s′,a′).
- D: That is the TD target; you still need Q + α(target − Q).

Takeaway: Q-learning is model-free: no P(s′|s,a) needed, just the observed (s, a, r, s′).

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B97. [BLANK · Medium · calculation]

Passive TD learning: Uπ(s) = 0.44, observed transition s → s′ with reward -1, Uπ(s′) = -0.18, α = 0.2, γ = 0.95. Apply Uπ(s) ← Uπ(s) + α[R + γUπ(s′) − Uπ(s)].

Answer: 0.1178

Working:

- TD target = -1 + 0.95×-0.18 = -1.1710.
- TD error = -1.1710 − 0.44 = -1.6110.
- U(s) = 0.44 + 0.2×-1.6110 = 0.1178.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B98. [BLANK · Medium · calculation]

Passive TD learning: Uπ(s) = 0.04, observed transition s → s′ with reward -1, Uπ(s′) = 0.1, α = 0.2, γ = 0.8. Apply Uπ(s) ← Uπ(s) + α[R + γUπ(s′) − Uπ(s)].

Answer: -0.1520

Working:

- TD target = -1 + 0.8×0.1 = -0.9200.
- TD error = -0.9200 − 0.04 = -0.9600.
- U(s) = 0.04 + 0.2×-0.9600 = -0.1520.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B99. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.16$, observed transition $s \to s'$ with reward 0, $U^\pi(s') = 0.82$, $\alpha = 0.2$, $\gamma = 1$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.2920

Working:

- TD target = 0 + 1×0.82 = 0.8200.
- TD error = 0.8200 − 0.16 = 0.6600.
- U(s) = 0.16 + 0.2×0.6600 = 0.2920.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B100. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.32$, observed transition $s \to s'$ with reward -1, $U^\pi(s') = 0.51$, $\alpha = 0.5$, $\gamma = 0.5$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: -0.2125

Working:

- TD target = -1 + 0.5×0.51 = -0.7450.
- TD error = -0.7450 − 0.32 = -1.0650.
- U(s) = 0.32 + 0.5×-1.0650 = -0.2125.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B101. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.38$, observed transition $s \to s'$ with reward -1, $U^\pi(s') = 0.78$, $\alpha = 0.5$, $\gamma = 0.9$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.0410

Working:

- TD target = -1 + 0.9×0.78 = -0.2980.
- TD error = -0.2980 − 0.38 = -0.6780.
- U(s) = 0.38 + 0.5×-0.6780 = 0.0410.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B102. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.13$, observed transition $s \to s'$ with reward 0, $U^\pi(s') = 0.7$, $\alpha = 0.1$, $\gamma = 0.95$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.1835

Working:

- TD target = 0 + 0.95×0.7 = 0.6650.
- TD error = 0.6650 − 0.13 = 0.5350.
- U(s) = 0.13 + 0.1×0.5350 = 0.1835.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B103. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.43$, observed transition $s \to s'$ with reward 1, $U^\pi(s') = 0.64$, $\alpha = 0.5$, $\gamma = 0.5$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.8750

Working:

- TD target = 1 + 0.5×0.64 = 1.3200.
- TD error = 1.3200 − 0.43 = 0.8900.
- U(s) = 0.43 + 0.5×0.8900 = 0.8750.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B104. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = -0.13$, observed transition $s \to s'$ with reward 0, $U^\pi(s') = -0.18$, $\alpha = 0.5$, $\gamma = 1$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: -0.1550

Working:

- TD target = 0 + 1×-0.18 = -0.1800.
- TD error = -0.1800 − -0.13 = -0.0500.
- U(s) = -0.13 + 0.5×-0.0500 = -0.1550.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B105. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = -0.18$, observed transition $s \to s'$ with reward -0.04, $U^\pi(s') = 0.71$, $\alpha = 0.2$, $\gamma = 0.8$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: -0.0384

Working:

- TD target = -0.04 + 0.8×0.71 = 0.5280.
- TD error = 0.5280 − -0.18 = 0.7080.
- U(s) = -0.18 + 0.2×0.7080 = -0.0384.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B106. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = -0.15$, observed transition $s \to s'$ with reward 0, $U^\pi(s') = -0.05$, $\alpha = 0.2$, $\gamma = 1$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: -0.1300

Working:

- TD target = 0 + 1×-0.05 = -0.0500.
- TD error = -0.0500 − -0.15 = 0.1000.
- U(s) = -0.15 + 0.2×0.1000 = -0.1300.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B107. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.82$, observed transition $s \to s'$ with reward 0, $U^\pi(s') = 0.23$, $\alpha = 0.1$, $\gamma = 0.8$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.7564

Working:

- TD target = 0 + 0.8×0.23 = 0.1840.
- TD error = 0.1840 − 0.82 = -0.6360.
- U(s) = 0.82 + 0.1×-0.6360 = 0.7564.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B108. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.35$, observed transition $s \to s'$ with reward -0.04, $U^\pi(s') = -0.04$, $\alpha = 0.2$, $\gamma = 1$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.2640

Working:

- TD target = -0.04 + 1×-0.04 = -0.0800.
- TD error = -0.0800 − 0.35 = -0.4300.
- U(s) = 0.35 + 0.2×-0.4300 = 0.2640.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B109. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.25$, observed transition $s \to s'$ with reward -1, $U^\pi(s') = 0.37$, $\alpha = 0.2$, $\gamma = 0.5$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.0370

Working:

- TD target = -1 + 0.5×0.37 = -0.8150.
- TD error = -0.8150 − 0.25 = -1.0650.
- U(s) = 0.25 + 0.2×-1.0650 = 0.0370.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B110. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.89$, observed transition $s \to s'$ with reward 0, $U^\pi(s') = 0.83$, $\alpha = 0.2$, $\gamma = 0.8$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.8448

Working:

- TD target = 0 + 0.8×0.83 = 0.6640.
- TD error = 0.6640 − 0.89 = -0.2260.
- U(s) = 0.89 + 0.2×-0.2260 = 0.8448.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B111. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = 0.34$, observed transition $s \to s'$ with reward 0, $U^\pi(s') = 0.29$, $\alpha = 0.1$, $\gamma = 1$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.3350

Working:

- TD target = 0 + 1×0.29 = 0.2900.
- TD error = 0.2900 − 0.34 = -0.0500.
- U(s) = 0.34 + 0.1×-0.0500 = 0.3350.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B112. [BLANK · Medium · calculation]

Passive TD learning: $U^\pi(s) = -0.06$, observed transition $s \to s'$ with reward 1, $U^\pi(s') = 0.38$, $\alpha = 0.5$, $\gamma = 0.8$. Apply $U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$.

Answer: 0.6220

Working:

- TD target = 1 + 0.8×0.38 = 1.3040.
- TD error = 1.3040 − -0.06 = 1.3640.
- U(s) = -0.06 + 0.5×1.3640 = 0.6220.

Takeaway: TD adjusts U(s) toward its OBSERVED successor; ADP toward ALL successors weighted by P.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B113. [BLANK · Medium · calculation]

Direct utility estimation, $\gamma = 1$, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 4 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.84

Working:

- Reward-to-go = (−0.04)×4 + (1) = 0.84.

- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B114. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and −1 for entering the terminal state. In a trial, state s is followed by 3 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: -1.12

Working:

- Reward-to-go = (−0.04)×3 + (-1) = -1.12.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B115. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 2 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.92

Working:

- Reward-to-go = (−0.04)×2 + (1) = 0.92.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B116. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 2 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.92

Working:

- Reward-to-go = (−0.04)×2 + (1) = 0.92.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B117. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and −1 for entering the terminal state. In a trial, state s is followed by 4 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: -1.16

Working:

- Reward-to-go = (−0.04)×4 + (-1) = -1.16.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B118. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 4 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.84

Working:

- Reward-to-go = (−0.04)×4 + (1) = 0.84.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B119. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 2 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.92

Working:

- Reward-to-go = (−0.04)×2 + (1) = 0.92.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B120. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and −1 for entering the terminal state. In a trial, state s is followed by 2 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: -1.08

Working:

- Reward-to-go = (−0.04)×2 + (-1) = -1.08.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B121. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 4 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.84

Working:

- Reward-to-go = (−0.04)×4 + (1) = 0.84.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B122. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and −1 for entering the terminal state. In a trial, state s is followed by 2 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: -1.08

Working:

- Reward-to-go = (−0.04)×2 + (-1) = -1.08.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B123. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 5 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.80

Working:

- Reward-to-go = (−0.04)×5 + (1) = 0.80.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B124. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 5 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.80

Working:

- Reward-to-go = (−0.04)×5 + (1) = 0.80.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B125. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 5 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.80

Working:

- Reward-to-go = (−0.04)×5 + (1) = 0.80.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B126. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 4 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.84

Working:

- Reward-to-go = (−0.04)×4 + (1) = 0.84.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B127. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and +1 for entering the terminal state. In a trial, state s is followed by 4 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: 0.84

Working:

- Reward-to-go = (−0.04)×4 + (1) = 0.84.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B128. [BLANK · Medium · calculation]

Direct utility estimation, γ = 1, reward −0.04 for each transition into a non-terminal state and −1 for entering the terminal state. In a trial, state s is followed by 3 transitions into non-terminal states and then 1 transition into the terminal state. What is the observed reward-to-go of s?

Answer: -1.12

Working:

- Reward-to-go = (−0.04)×3 + (-1) = -1.12.
- (Workshop 6 example: U(0,0) = (−0.04)×4 + 1 = 0.84.)

Takeaway: Direct utility estimation averages observed reward-to-go; it learns nothing until the trial ends.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B129. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 6

(3,3) | Right | (3,2) | 5
(3,3) | Right | (2,3) | 3

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.429

Working:

- Right was executed 14 times in (3,3).
- P = 6/14 = 0.429.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B130. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 5
(3,3) | Right | (3,2) | 6
(3,3) | Right | (2,3) | 4

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.333

Working:

- Right was executed 15 times in (3,3).
- P = 5/15 = 0.333.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B131. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 2
(3,3) | Right | (3,2) | 5
(3,3) | Right | (2,3) | 1

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.250

Working:

- Right was executed 8 times in (3,3).
- P = 2/8 = 0.250.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B132. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 1
(3,3) | Right | (3,2) | 6
(3,3) | Right | (2,3) | 4

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.091

Working:

- Right was executed 11 times in (3,3).
- P = 1/11 = 0.091.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B133. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 2
(3,3) | Right | (3,2) | 5
(3,3) | Right | (2,3) | 2

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.222

Working:

- Right was executed 9 times in (3,3).
- P = 2/9 = 0.222.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B134. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 1
(3,3) | Right | (3,2) | 3
(3,3) | Right | (2,3) | 1

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.200

Working:

- Right was executed 5 times in (3,3).
- P = 1/5 = 0.200.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B135. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 2
(3,3) | Right | (3,2) | 2
(3,3) | Right | (2,3) | 2

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.333

Working:

- Right was executed 6 times in (3,3).
- P = 2/6 = 0.333.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B136. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 2
(3,3) | Right | (3,2) | 3
(3,3) | Right | (2,3) | 4

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.222

Working:

- Right was executed 9 times in (3,3).
- P = 2/9 = 0.222.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B137. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 6
(3,3) | Right | (3,2) | 1
(3,3) | Right | (2,3) | 3

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.600

Working:

- Right was executed 10 times in (3,3).
- P = 6/10 = 0.600.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B138. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 6
(3,3) | Right | (3,2) | 6
(3,3) | Right | (2,3) | 0

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.500

Working:

- Right was executed 12 times in (3,3).
- P = 6/12 = 0.500.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B139. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 3
(3,3) | Right | (3,2) | 2
(3,3) | Right | (2,3) | 4

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.333

Working:

- Right was executed 9 times in (3,3).
- P = 3/9 = 0.333.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B140. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 1
(3,3) | Right | (3,2) | 1
(3,3) | Right | (2,3) | 4

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.167

Working:

- Right was executed 6 times in (3,3).
- P = 1/6 = 0.167.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B141. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 3
(3,3) | Right | (3,2) | 1
(3,3) | Right | (2,3) | 3

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.429

Working:

- Right was executed 7 times in (3,3).
- P = 3/7 = 0.429.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B142. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 5

(3,3) | Right | (3,2) | 6
(3,3) | Right | (2,3) | 0

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.455

Working:

- Right was executed 11 times in (3,3).
- P = 5/11 = 0.455.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B143. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 6
(3,3) | Right | (3,2) | 5
(3,3) | Right | (2,3) | 2

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.462

Working:

- Right was executed 13 times in (3,3).
- P = 6/13 = 0.462.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

MDP B144. [BLANK · Easy · calculation]

s | a | s′ | times
(3,3) | Right | (4,3) | 1
(3,3) | Right | (3,2) | 3
(3,3) | Right | (2,3) | 4

An ADP agent counts observed transitions (table). Estimate P((4,3) | (3,3), Right).

Answer: 0.125

Working:

- Right was executed 8 times in (3,3).
- P = 1/8 = 0.125.

Takeaway: ADP: estimate P(s′|s,a) from counts, then solve the (linear, policy-fixed) Bellman equations.

Source: Workshop 6 (MDP, Bellman equation, value/policy iteration, TD, Q-learning)

🎲 Probability & Bayesian Networks (Lecture 11)

Part A: hand-written questions (126)

BAYES A1. [MATCH · Medium]

Match each scenario to the type of uncertainty highlighted in Lecture 11.

1. A drone in fog with noisy GPS
2. A trading agent's same 'buy' order gives different profits
3. A chatbot's password reset sometimes times out
4. A voice assistant can't directly observe user intent

A. non-deterministic
B. fully observable and deterministic
C. partially observable
D. partially observable
E. non-deterministic

Answer: 1 → C, 2 → A, 3 → A, 4 → C

Working:

- A drone in fog with noisy GPS → partially observable
- A trading agent's same 'buy' order gives different profits → non-deterministic
- A chatbot's password reset sometimes times out → non-deterministic
- A voice assistant can't directly observe user intent → partially observable

Takeaway: Can't see the state vs can't predict the outcome.

Source: Lecture 11, slide 5-10

BAYES A2. [TF · Easy]

For sentences that are certainly true, a probabilistic agent's degree of belief is 1.

Answer: True

Why: Lecture 11 slide 14.

Takeaway: 0 = certainly false; strictly between = uncertain.

Source: Lecture 11 (Bayesian network and probabilistic reasoning)

BAYES A3. [MCQ · Medium]

Autonomous driving in city traffic is an example of…

A. only partial observability
B. only non-determinism
C. both partial observability and non-determinism
D. a fully observable deterministic environment

Answer: C. both partial observability and non-determinism

Option notes:

- A: Other drivers' actions are also unpredictable.
- B: Sensors also have blind spots.
- D: No.

Takeaway: Real-world agents often face both.

Source: Lecture 11, slide 11

BAYES A4. [MCQ · Easy · calculation]

A card is known to be a face card. P(King)?

A. 1/13
B. 1/4
C. 1/3
D. 4/52

Answer: C. 1/3

Option notes:

- A: Ignored the condition.
- B: There are 12 face cards, 4 Kings.
- D: That's P(King ∩ any).

Takeaway: Among 12 face cards, 4 are Kings.

Source: Lecture 11, slide 20

BAYES A5. [TF · Medium]

P(A ∪ B) = P(A) + P(B) holds for ANY two events.

Answer: False

Why: Only for MUTUALLY EXCLUSIVE events (otherwise subtract P(A ∩ B)).

Takeaway: OR → add (if exclusive).

Source: Lecture 11, slide 18

BAYES A6. [MATCH · Easy]

Match each keyword to its rule.

1. NOT
2. OR (mutually exclusive)
3. AND (independent)
4. GIVEN

A. 1 − P(A)
B. P(A) + P(B)
C. P(A) × P(B)
D. P(A ∩ B)/P(B)
E. P(A)/P(B)

Answer: 1 → A, 2 → B, 3 → C, 4 → D

Working:

- NOT → 1 − P(A)
- OR (mutually exclusive) → P(A) + P(B)
- AND (independent) → P(A) × P(B)
- GIVEN → P(A ∩ B)/P(B)

Takeaway: Quick memory aid from Lecture 11.

Source: Lecture 11, slide 15

BAYES A7. [MCQ · Easy]

The probability assigned to an event BEFORE new evidence is observed is the…

A. marginal of the evidence
B. posterior probability
C. prior probability
D. likelihood

Answer: C. prior probability

Option notes:

- A: P(evidence).
- B: After evidence.
- D: P(evidence | hypothesis).

Takeaway: Prior → (evidence) → posterior.

Source: Lecture 11, slide 21

BAYES A8. [BLANK · Easy · calculation]

Lec11 quiz: 5000 experiments, event (Val1 = F, Val2 = T, Val3 = T) occurred 88 times. Its probability is _____ % (2 dp).

Answer: 1.76
(Also accepted: 1.760)

Takeaway: 88/5000 = 0.0176.

Source: Lec11 class quiz Q1

BAYES A9. [BLANK · Easy · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

The burglary case has 5 binary variables. The full joint table has _____ rows.

Answer: 32

Takeaway: 2^5.

Source: Lecture 11, slide 23

BAYES A10. [MULTI · Medium]

Why doesn't the full joint table scale (Lecture 11)? (Select ALL that apply.)

A. Rows grow as 2^n
B. It needs many experiments to observe every event
C. It cannot represent conditional probabilities
D. Assigning every probability is unnatural, time-consuming and error-prone

Answer: A. Rows grow as 2^n; B. It needs many experiments to observe every event; D. Assigning every probability is unnatural, time-consuming and error-prone

Option notes:

- A: Correct: Doubles per variable.
- B: Correct: Slide notes.
- C: Incorrect: Conditionals can be computed from it.
- D: Correct: Difficult to specify.

Takeaway: Exponential growth + hard to specify.

Source: Lecture 11, slide 25

BAYES A11. [TF · Medium]

In the spreadsheet, every row's probability = its occurrence count divided by the total number of experiments, and all 32 probabilities sum to 1.

Answer: True

Why: P(event$_j$) = count/N; the table is complete (32 mutually exclusive assignments).

Takeaway: Check: Σ P = 1.000.

Source: Lecture 11, slide 31-32

BAYES A12. [BLANKS · Easy]

Marginalisation quiz: To obtain P(X = x), sum the joint probabilities over _____ and _____.

(1) first = ____
(2) second = ____

Answer: first = Y; second = Z

Takeaway: Sum out every other variable.

Source: Marginalisation fill-in-blanks Q1

BAYES A13. [BLANK · Easy]

Before summing, keep only the rows in which the query variable X has the required _____.

Answer: value
(Also accepted: Value)

Takeaway: Fix the query value.

Source: Marginalisation fill-in-blanks Q2

BAYES A14. [BLANK · Easy]

For X = x_1, the selected probabilities are 0.06, 0.09, 0.14 and _____.

Answer: 0.11

Takeaway: Fourth x_1 row.

Source: Marginalisation fill-in-blanks Q3

BAYES A15. [BLANK · Easy · calculation]

$P(X = x_1) = 0.06 + 0.09 + 0.14 + 0.11 =$ _____.

Answer: 0.40
(Also accepted: 0.4)

Takeaway: Sum the selected rows.

Source: Marginalisation fill-in-blanks Q4

BAYES A16. [BLANK · Easy]

Marginalisation removes variables from a joint distribution by _____ their probabilities.

Answer: summing
(Also accepted: summing out, adding, sum)

Takeaway: Sum out.

Source: Marginalisation fill-in-blanks Q5

BAYES A17. [MATCH · Easy]

Lec11 quiz Q2: match the Bayesian-network component to its explanation.

1. child node
2. an edge
3. arrow of an edge

A. the normalisation constant
B. indicates the link from a parent to a child
C. a node that has a conditional probability distribution given its parents
D. association between two nodes

Answer: 1 → C, 2 → D, 3 → B

Working:

- child node → a node that has a conditional probability distribution given its parents
- an edge → association between two nodes
- arrow of an edge → indicates the link from a parent to a child

Takeaway: Node = variable, arrow = parent → child.

Source: Lec11 class quiz Q2

BAYES A18. [BLANK · Easy · calculation]

B | E | P(A=t|B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

```
A | P(J=t|A) | P(M=t|A)
t | .90 | .70
f | .05 | .01
```

Lec11 quiz Q3: the probability of the alarm sounding when there is an earthquake but no burglary, P(a | ¬b, e) = _____ (3 dp).

Answer: 0.290
(Also accepted: 0.29)

Takeaway: Read the CPT row B = f, E = t.

Source: Lec11 class quiz Q3

BAYES A19. [TF · Medium]

A Bayesian network may contain directed cycles as long as each node has a CPT.

Answer: False

Why: It must be a DAG (directed ACYCLIC graph).

Takeaway: No circular dependencies.

Source: Lecture 11, slide 26

BAYES A20. [MCQ · Medium]

When constructing a Bayesian network, which node ordering gives a more compact network?

A. Causes before effects
B. Largest CPT first
C. Alphabetical order
D. Effects before causes

Answer: A. Causes before effects

Option notes:

- B: Not a criterion.
- C: Any order works but may be dense.
- D: Leads to more arcs.

Takeaway: Order causes first, then choose a minimal parent set.

Source: Lecture 11, slide 34

BAYES A21. [MULTI · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

In the burglary network, which statements are correct? (Select ALL that apply.)

A. MaryCalls depends on JohnCalls
B. JohnCalls depends only on Alarm
C. B and E are root nodes with prior probabilities
D. B and E are assumed independent
E. Alarm depends on Burglary and Earthquake

Answer: B. JohnCalls depends only on Alarm; C. B and E are root nodes with prior probabilities; D. B and E are assumed independent; E. Alarm depends on Burglary and Earthquake

Option notes:

- A: Incorrect: M depends only on A.
- B: Correct: P(J|A).
- C: Correct: P(B), P(E).
- D: Correct: Slide 30 notes.
- E: Correct: P(A|B,E).

Takeaway: Read the arrows.

Source: Lecture 11, slide 30, 35

BAYES A22. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Which factorisation matches the burglary network?

A. P(B)P(E)P(A)P(J)P(M)
B. P(B)P(E)P(A|B,E)P(J|A)P(M|A)

C. P(B,E)P(A|J)P(J|M)
D. P(B|A)P(E|A)P(A|J,M)P(J)P(M)

Answer: B. P(B)P(E)P(A|B,E)P(J|A)P(M|A)

Option notes:

- A: Ignores dependencies.
- C: Not the network.
- D: Arrows are reversed.

Takeaway: Each node given its parents.

Source: Lecture 11, slide 35

BAYES A23. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Using the burglary CPTs, P(j, m, a, ¬b, ¬e) = 0.90 × 0.70 × 0.001 × 0.999 × 0.998 ≈ _____ (scientific notation, 3 s.f., e.g. 1.23e-4).

Answer: 6.28e-4
(Also accepted: 0.000628, 6.28e-04)

Takeaway: ≈ 0.000628 (matches the full-joint row 0.000628111).

Source: Lecture 11, slide 52

BAYES A24. [TF · Medium]

Multiplying the local CPT probabilities of a Bayesian network recovers each row of the full joint table.

Answer: True

Why: Lecture 11 slide 30.

Takeaway: Same distribution, two representations.

Source: Lecture 11, slide 30

BAYES A25. [MCQ · Easy]

A disease D causes symptoms S. Which probability represents reasoning in the CAUSAL direction?

A. P(D ∩ S)
B. P(S | D)
C. P(D | S)
D. P(D)

Answer: B. P(S | D)

Option notes:

- A: A joint, not a direction.
- C: That's diagnostic (effect → cause).
- D: A prior.

Takeaway: Causal: cause → effect.

Source: Causal and Diagnostic Directions MCQ Q1

BAYES A26. [MCQ · Easy]

A patient is observed to have symptoms S. Which probability represents DIAGNOSTIC reasoning?

A. P(D | S)
B. P(S)
C. P(D ∩ S)
D. P(S | D)

Answer: A. P(D | S)

Option notes:

- B: Evidence.
- C: Joint.
- D: Causal.

Takeaway: Bayes computes P(D|S) from P(S|D), P(D), P(S).

Source: Causal and Diagnostic Directions MCQ Q2

BAYES A27. [MATCH · Easy]

Match each direction to its reasoning process.

1. Causal direction
2. Diagnostic direction

A. Cause → Effect | P(effect | cause)
B. Effect → Cause | P(cause | effect)
C. Effect → Effect | P(effect)

Answer: 1 → A, 2 → B

Working:

- Causal direction → Cause → Effect | P(effect | cause)
- Diagnostic direction → Effect → Cause | P(cause | effect)

Takeaway: Reasoning Process Matching Quiz Q1.

Source: Reasoning Process Matching Quiz Q1

BAYES A28. [MATCH · Easy]

Match each probability to the reasoning process.

1. P(S | D)
2. P(D | S)

A. Diagnostic: Symptoms → Disease
B. Joint: Disease and Symptoms
C. Causal: Disease → Symptoms

Answer: 1 → C, 2 → A

Working:

- P(S | D) → Causal: Disease → Symptoms
- P(D | S) → Diagnostic: Symptoms → Disease

Takeaway: Reasoning Process Matching Quiz Q2.

Source: Reasoning Process Matching Quiz Q2

BAYES A29. [TF · Medium]

In P(Y | X) = α P(X | Y) P(Y), α is the normalisation constant that makes the posterior sum to 1.

Answer: True

Why: Lecture 11 slide 29.

Takeaway: Compute unnormalised, then scale.

Source: Lecture 11, slide 29

BAYES A30. [MCQ · Hard]

Conditional independence of X and Y given Z means…

A. P(X, Y) = P(X)P(Y)
B. P(X, Y, Z) = P(X)P(Y)P(Z)
C. P(X | Y, Z) = P(X | Z)
D. P(X | Z) = P(Z | X)

Answer: C. P(X | Y, Z) = P(X | Z)

Option notes:

- A: That's (unconditional) independence.
- B: Full independence.
- D: Not generally true.

Takeaway: Given Z, Y adds nothing about X.

Source: Lecture 11, slide 29

BAYES A31. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

When evaluating a path in an enumeration tree, what operation is performed along that path?

A. Multiply the probabilities appearing on the path
B. Normalise after every branch
C. Select only the largest probability
D. Add all probabilities on the path

Answer: A. Multiply the probabilities appearing on the path

Option notes:

- B: Normalise once at the end.
- C: All branches count.
- D: Add ACROSS alternative branches, multiply ALONG a path.

Takeaway: Enumeration MCQ Q1.

Source: Enumeration Reasoning MCQ Q1

BAYES A32. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

For a fixed value of B, why are the E = true and E = false branches added?

A. They remove the need to consider Alarm
B. They are repeated observations of the same event
C. They convert joint probabilities into conditionals
D. They represent alternative hidden states of Earthquake

Answer: D. They represent alternative hidden states of Earthquake

Option notes:

- A: Alarm is summed too.
- B: They are mutually exclusive alternatives.
- C: Normalisation does that.

Takeaway: Enumeration MCQ Q2.

Source: Enumeration Reasoning MCQ Q2

BAYES A33. [MCQ · Medium]

Which factor must use the Alarm CPT entries for B = false?

A. P(A | ¬B, E)
B. P(j | A)
C. P(E)
D. P(m | A)

Answer: A. P(A | ¬B, E)

Option notes:

- B: Doesn't involve B.
- C: Root prior.
- D: Doesn't involve B.

Takeaway: Enumeration MCQ Q3: the outer prior is also P(¬B).

Source: Enumeration Reasoning MCQ Q3

BAYES A34. [MCQ · Medium]

What does 'evaluate inside out' mean in this calculation?

A. Add the B and ¬B cases before multiplying
B. Normalise before calculating branch values
C. Evaluate evidence likelihoods for each A state, sum over A, weight and sum over E, then apply the selected B prior
D. Apply the B prior and ignore hidden-state branches

Answer: C. Evaluate evidence likelihoods for each A state, sum over A, weight and sum over E, then apply the selected B prior

Option notes:

- A: They are separate scores.
- B: Normalise last.
- D: Hidden branches must be summed.

Takeaway: Enumeration MCQ Q4.

Source: Enumeration Reasoning MCQ Q4

BAYES A35. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

After obtaining s_B and s_¬B, what is the correct next step?

A. Subtract the smaller from the larger
B. Divide each score by s_B + s_¬B
C. Choose the larger score as probability 1
D. Multiply the two scores

Answer: B. Divide each score by s_B + s_¬B

Option notes:

- A: No.
- C: That discards information.
- D: No.

Takeaway: Enumeration MCQ Q5.

Source: Enumeration Reasoning MCQ Q5

BAYES A36. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

P(B | event) = α · P(B) · Σ_e P(E) · Σ_a P(A|B,E)P(j|A)P(m|A). Why can P(B) be moved outside both summations?

A. P(B) does not depend on E or A
B. P(B) is always equal to 1
C. P(B) must be summed over A
D. P(B) is an evidence variable

Answer: A. P(B) does not depend on E or A

Option notes:

- B: P(b) = 0.001.
- C: It doesn't depend on A.
- D: B is the query.

Takeaway: Inference Calculation MCQ Q1.

Source: Inference Calculation MCQs Q1

BAYES A37. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Which variables are summed out to obtain P(B | j, m)?

A. j and m
B. B and A
C. E and A
D. B and event

Answer: C. E and A

Option notes:

- A: Evidence is fixed, not summed.
- B: B is the query.
- D: B is the query.

Takeaway: Inference Calculation MCQ Q2.

Source: Inference Calculation MCQs Q2

BAYES A38. [MCQ · Hard]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Which expression correctly expands Σ_e P(E) Σ_a P(A|¬b,E) P(j|A) P(m|A) over E?

A. P(e)Σ_a P(A|¬b,e)… + P(e)Σ_a P(A|¬b,¬e)…
B. Σ_a P(A|¬b,e)… + Σ_a P(A|¬b,¬e)…
C. P(e)Σ_a P(A|¬b,e)P(j|A)P(m|A) + P(¬e)Σ_a P(A|¬b,¬e)P(j|A)P(m|A)
D. P(e)Σ_a P(A|¬b,e)P(j|A)P(m|A) × P(¬e)Σ_a P(A|¬b,¬e)P(j|A)P(m|A)

Answer: C. P(e)Σ_a P(A|¬b,e)P(j|A)P(m|A) + P(¬e)Σ_a P(A|¬b,¬e)P(j|A)P(m|A)

Option notes:

- A: Second branch needs P(¬e).
- B: Each branch is weighted by its prior.
- D: Branches are ADDED.

Takeaway: No-Burglary Expansion MCQ Q1.

Source: No_Burglary_Expansion MCQ Q1

BAYES A39. [MCQ · Hard]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

For the E = e branch, which expression correctly expands Σ_a P(A | ¬b, e) P(j|A) P(m|A)?

A. P(a|¬b,e)P(j|a) + P(¬a|¬b,e)P(m|¬a)
B. P(a|¬b,e)P(j|a)P(m|a) + P(¬a|¬b,e)P(j|¬a)P(m|¬a)
C. P(a|¬b,e)P(j|a)P(m|a) × P(¬a|¬b,e)P(j|¬a)P(m|¬a)
D. P(a|¬b,e)P(j|a)P(m|a) + P(¬a|¬b,e)P(j|a)P(m|a)

Answer: B. P(a|¬b,e)P(j|a)P(m|a) + P(¬a|¬b,e)P(j|¬a)P(m|¬a)

Option notes:

- A: Each branch needs BOTH evidence terms.
- C: Add, don't multiply, alternatives.
- D: Evidence terms must use ¬a in the second branch.

Takeaway: No-Burglary Expansion MCQ Q2.

Source: No_Burglary_Expansion MCQ Q2

BAYES A40. [MULTI · Medium]

For the target posterior P(X | e), which statements correctly describe the reasoning setup? (Select ALL that apply.)

A. Y contains variables that are neither query nor evidence
B. X represents the query variable(s) whose posterior is required
C. E contains observed variables with their assigned evidence values e
D. The hidden variables Y should be treated as additional observed evidence

Answer: A. Y contains variables that are neither query nor evidence; B. X represents the query variable(s) whose posterior is required; C. E contains observed variables with their assigned evidence values e

Option notes:

- A: Correct: Hidden.
- B: Correct: Query.
- C: Correct: Evidence.
- D: Incorrect: They are marginalised, not observed.

Takeaway: Reasoning Process Multiple Answer Quiz 1, Q1.

Source: Reasoning Process Multiple Answer Quiz 1

BAYES A41. [MULTI · Medium]

Which actions are required to compute the posterior distribution P(X | e) in a Bayesian network? (Select ALL that apply.)

A. Use the Bayesian-network factorisation to express the joint distribution
B. Incorporate the observed evidence assignment e
C. Sum out (marginalise) all hidden variables Y
D. Return the unnormalised result without ensuring probabilities sum to 1

Answer: A. Use the Bayesian-network factorisation to express the joint distribution; B. Incorporate the observed evidence assignment e; C. Sum out (marginalise) all hidden variables Y

Option notes:

- A: Correct: Step 1.
- B: Correct: Step 2.
- C: Correct: Step 3.
- D: Incorrect: Must normalise.

Takeaway: Reasoning Process Multiple Answer Quiz 1, Q2.

Source: Reasoning Process Multiple Answer Quiz 1

BAYES A42. [MULTI · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Which actions are required to compute P(B | j, m) from the full joint table? (Select ALL that apply.)

A. Separate the retained rows into B = true and B = false
B. For each B case, sum over every binary combination of E and A
C. Divide each unnormalised score by s_B + s_¬B
D. Choose only the single row with the largest joint probability
E. Keep only rows in which j = true and m = true

Answer: A. Separate the retained rows into B = true and B = false; B. For each B case, sum over every binary combination of E and A; C. Divide each unnormalised score by s_B + s_¬B; E. Keep only rows in which j = true and m = true

Option notes:

- A: Correct: Group.
- B: Correct: Marginalise.
- C: Correct: Normalise.
- D: Incorrect: Must sum all relevant rows.
- E: Correct: Filter.

Takeaway: Multiple-answer quiz 3, Q1.

Source: Reasoning Process Multiple Answer Quiz 3

BAYES A43. [MULTI · Hard]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls


```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```


What do the unnormalised scores s_B = $\Sigma_e \Sigma_a$ P(B,E,A,j,m) and s_¬B represent? (Select ALL that apply.)

A. s_B combines the four rows for B = true across all assignments of E and A
B. s_¬B combines the four rows for B = false

C. They can be normalised by dividing each by s_B + s_¬B
D. They are already equal to P(B|j,m) and P(¬B|j,m)
E. Both are computed after fixing j = true and m = true

Answer: A. s_B combines the four rows for B = true across all assignments of E and A; B. s_¬B combines the four rows for B = false; C. They can be normalised by dividing each by s_B + s_¬B; E. Both are computed after fixing j = true and m = true

Option notes:

- A: Correct: Four rows.
- B: Correct: Four rows.
- C: Correct: Normalise.
- D: Incorrect: Not until normalised.
- E: Correct: Evidence fixed.

Takeaway: Multiple-answer quiz 3, Q2 (P(B|j,m) ≈ 0.284).

Source: Reasoning Process Multiple Answer Quiz 3

BAYES A44. [MATCH · Medium]

Match each reasoning step with its purpose (exact inference with the factorised BN).

1. Sum over E
2. Sum over A

A. Accounts for both alarm states
B. Accounts for both burglary states
C. Accounts for both earthquake states

Answer: 1 → C, 2 → A

Working:

- Sum over E → Accounts for both earthquake states
- Sum over A → Accounts for both alarm states

Takeaway: BN Reasoning Process Quiz Q1.

Source: Bayesian_Network_Reasoning_Process_Quiz Q1

BAYES A45. [MATCH · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

For the task P(B | J, M), match each component to its role.

1. JohnCalls and MaryCalls
2. Alarm and Earthquake
3. Burglary

A. Hidden variables to be summed out
B. Normalisation constants
C. Query variable
D. Evidence variables

Answer: 1 → D, 2 → A, 3 → C

Working:

- JohnCalls and MaryCalls → Evidence variables
- Alarm and Earthquake → Hidden variables to be summed out
- Burglary → Query variable

Takeaway: BN Reasoning Process Quiz Q2.

Source: Bayesian_Network_Reasoning_Process_Quiz Q2

BAYES A46. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Lecture 11: s_B = 0.00059224 and s_¬B = 0.00149186. P(B | j, m) = _____ (4 dp).

Answer: 0.2842
(Also accepted: 0.284)

Takeaway: 0.00059224/(0.00059224 + 0.00149186).

Source: Lecture 11, slide 56

BAYES A47. [MULTI · Medium]

Select ALL limitations of inference by enumeration named in Lecture 11. (Select ALL that apply.)

A. Poor scalability in large, densely connected networks
B. It gives only approximate answers
C. It cannot use a full joint table
D. Repeated calculations (e.g. P(j|a), P(m|a) computed four times)

Answer: A. Poor scalability in large, densely connected networks; D. Repeated calculations (e.g. P(j|a), P(m|a) computed four times)

Option notes:

- A: Correct: Use approximate inference then.
- B: Incorrect: It is EXACT.
- C: Incorrect: It can (Method 1).
- D: Correct: Fixed by variable elimination / clustering.

Takeaway: Exact but expensive.

Source: Lecture 11, slide 58

BAYES A48. [MATCH · Medium]

Match each approximate-inference sampling method to its description (named in Lecture 11; details out of scope).

1. Direct sampling
2. Rejection sampling
3. Likelihood weighting

A. weight samples according to the evidence
B. generate samples directly from the network
C. discard samples that do not match the evidence
D. sum over all hidden variables exactly

Answer: 1 → B, 2 → C, 3 → A

Working:

- Direct sampling → generate samples directly from the network
- Rejection sampling → discard samples that do not match the evidence
- Likelihood weighting → weight samples according to the evidence

Takeaway: Approximate inference is optional/out of scope, but know the names.

Source: Lecture 11, slide 39

BAYES A49. [TF · Easy]

The topic focus of Lecture 11 is exact inference in static worlds; approximate inference and dynamic worlds are out of scope.

Answer: True

Why: Slide 42.

Takeaway: Static: each variable has one fixed value.

Source: Lecture 11, slide 42

BAYES A50. [MCQ · Medium]

What assumption makes Naive Bayes 'naive'?

A. All classes are equally likely
B. Features are conditionally independent given the class
C. Features are independent of the class
D. Features follow a normal distribution

Answer: B. Features are conditionally independent given the class

Option notes:

- A: Priors can differ.
- C: Then it couldn't classify.

- D: Not required.

Takeaway: $P(x_1 \ldots x_n|c) \approx \Pi\, P(x_i|c)$.

Source: Lecture 11 (Bayesian network and probabilistic reasoning)

BAYES A51. [TF · Medium]

To classify with Naive Bayes, you must normalise the class scores before comparing them.

Answer: False

Why: The class with the larger unnormalised score wins; normalise only to get probabilities.

Takeaway: Lecture 8 poll 3: option D is wrong.

Source: Lecture 8, slide 52

BAYES A52. [BLANK · Easy]

An environment where the agent may never know for certain what state it is in is called partially _____.

Answer: observable
(Also accepted: Observable)

Takeaway: Lecture 11 slide 4.

Source: Lecture 11 (Bayesian network and probabilistic reasoning)

BAYES A53. [MULTI · Hard]

A medical diagnosis assistant (Lecture 11): select ALL correct statements. (Select ALL that apply.)

A. The same treatment can affect patients differently (non-deterministic)
B. The true disease state is hidden (partially observable)
C. Tests can be false positive/negative
D. It is fully observable because tests are run

Answer: A. The same treatment can affect patients differently (non-deterministic); B. The true disease state is hidden (partially observable); C. Tests can be false positive/negative

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Incorrect: Tests are imperfect.

Takeaway: Both kinds of uncertainty.

Source: Lecture 11 (Bayesian network and probabilistic reasoning)

BAYES A54. [TF · Medium]

To compute $P(X = x_1)$ from a full joint table over X, Y, Z, you multiply the probabilities of the rows with $X = x_1$.

Answer: False

Why: You ADD (sum out Y and Z), you don't multiply.

Takeaway: Marginalise = sum.

Source: Lecture 11 (Bayesian network and probabilistic reasoning)

BAYES A55. [MCQ · Medium · calculation]

Joint table rows with X = x_2 have probabilities 0.18, 0.12, 0.16, 0.14. P(X = x_2)?

A. 0.60
B. 0.15
C. 0.40
D. 0.000048

Answer: A. 0.60

Option notes:

- B: That's the average.
- C: That's P(x_1).
- D: Multiplied instead of summing.

Takeaway: Sum the rows.

Source: Lecture 11, slide 44

BAYES A56. [TF · Easy]

Naive Bayes multiplies the class prior by the likelihood of each observed feature given the class.

Answer: True

Why: P(c) Π P(x_i|c).

Takeaway: Unnormalised score.

Source: Lecture 11 (Bayesian network and probabilistic reasoning)

BAYES A57. [BLANKS · Easy · calculation]

Fair die: P(4) = ___; P(1 or 6) = ___.

(1) P(4) = ____
(2) P(1 or 6) = ____

Answer: P(4) = 1/6; P(1 or 6) = 1/3

Takeaway: Chance; OR adds.

Source: Book 2 (Exam examples), Q364, p. 46

BAYES A58. [BLANK · Easy · calculation]

P(rain) = 0.3; P(no rain) = ___.

Answer: 0.7

Takeaway: NOT: 1 − p.

Source: Book 2 (Exam examples), Q365, p. 46

BAYES A59. [BLANK · Easy · calculation]

Coin and die: P(head and 6) = ___.

Answer: 1/12
(Also accepted: 0.0833, 0.083)

Takeaway: AND (independent) multiplies.

Source: Book 2 (Exam examples), Q366, p. 46

BAYES A60. [BLANK · Easy · calculation]

A card is a face card; P(King | face) = ___.

Answer: 1/3
(Also accepted: 4/12, 0.333)

Takeaway: 4 kings among 12 face cards.

Source: Book 2 (Exam examples), Q367, p. 46

BAYES A61. [MATCH · Easy]

Match the probability word to its operation.

1. OR
2. AND
3. NOT
4. GIVEN

A. conditional probability
B. multiply (independent)
C. add (mutually exclusive)
D. 1 − p

Answer: 1 → C, 2 → B, 3 → D, 4 → A

Working:

- OR → add (mutually exclusive)
- AND → multiply (independent)
- NOT → 1 − p
- GIVEN → conditional probability

Takeaway: Lecture 11 memory aid.

Source: Book 2 (Exam examples), Q368, p. 47

BAYES A62. [MATCH · Easy]

Match.

1. prior
2. posterior

A. belief after new evidence
B. belief before new evidence

Answer: 1 → B, 2 → A

Working:

- prior → belief before new evidence
- posterior → belief after new evidence

Takeaway: Before vs after.

Source: Book 2 (Exam examples), Q369, p. 47

BAYES A63. [BLANK · Easy · calculation]

X | Y | Z | P
x1 | y1 | z1 | 0.06
x1 | y1 | z2 | 0.09
x1 | y2 | z1 | 0.14
x1 | y2 | z2 | 0.11
x2 | y1 | z1 | 0.18
x2 | y1 | z2 | 0.12
x2 | y2 | z1 | 0.16
x2 | y2 | z2 | 0.14

Lecture 11 joint table: P(X = x1) = ___.

Answer: 0.40
(Also accepted: 0.4)

Working:

- 0.06 + 0.09 + 0.14 + 0.11 = 0.40

Takeaway: Add the x1 rows.

Source: Book 2 (Exam examples), Q370, p. 47

BAYES A64. [BLANK · Easy · calculation]

X | Y | Z | P
x1 | y1 | z1 | 0.06
x1 | y1 | z2 | 0.09
x1 | y2 | z1 | 0.14
x1 | y2 | z2 | 0.11
x2 | y1 | z1 | 0.18
x2 | y1 | z2 | 0.12
x2 | y2 | z1 | 0.16
x2 | y2 | z2 | 0.14

Same table: P(Y = y1) = ___.

Answer: 0.45

Working:

- 0.06 + 0.09 + 0.18 + 0.12 = 0.45

Takeaway: Rows with y1.

Source: Book 2 (Exam examples), Q371, p. 47

BAYES A65. [BLANK · Medium · calculation]

X | Y | Z | P
x1 | y1 | z1 | 0.06
x1 | y1 | z2 | 0.09
x1 | y2 | z1 | 0.14
x1 | y2 | z2 | 0.11
x2 | y1 | z1 | 0.18
x2 | y1 | z2 | 0.12
x2 | y2 | z1 | 0.16
x2 | y2 | z2 | 0.14

Same table: P(x1 | y1) = ___ (3 d.p.).

Answer: 0.333
(Also accepted: 1/3)

Working:

- (0.06 + 0.09)/0.45 = 0.333

Takeaway: P(x1, y1)/P(y1).

Source: Book 2 (Exam examples), Q372, p. 47

BAYES A66. [BLANK · Easy · calculation]

X | Y | Z | P
x1 | y1 | z1 | 0.06
x1 | y1 | z2 | 0.09
x1 | y2 | z1 | 0.14
x1 | y2 | z2 | 0.11
x2 | y1 | z1 | 0.18
x2 | y1 | z2 | 0.12
x2 | y2 | z1 | 0.16
x2 | y2 | z2 | 0.14

Same table: P(z1) = ___.

Answer: 0.54

Working:

- 0.06 + 0.14 + 0.18 + 0.16 = 0.54

Takeaway: Rows with z1.

Source: Book 2 (Exam examples), Q373, p. 47

BAYES A67. [BLANK · Easy · calculation]

X | Y | Z | P
x1 | y1 | z1 | 0.06
x1 | y1 | z2 | 0.09
x1 | y2 | z1 | 0.14
x1 | y2 | z2 | 0.11
x2 | y1 | z1 | 0.18
x2 | y1 | z2 | 0.12
x2 | y2 | z1 | 0.16
x2 | y2 | z2 | 0.14

Same table: P(x2, z2) = ___.

Answer: 0.26

Working:

- 0.12 + 0.14 = 0.26

Takeaway: Two rows.

Source: Book 2 (Exam examples), Q374, p. 47

BAYES A68. [BLANKS · Easy · calculation]

With 5 binary variables the full joint table has ___ rows; with 20 variables ___ rows.

(1) 5 vars = ____
(2) 20 vars = ____

Answer: 5 vars = 32; 20 vars = 1048576

Takeaway: 2^n.

Source: Book 2 (Exam examples), Q375, p. 47

BAYES A69. [MULTI · Medium]

Why the full joint table does not scale: (Select ALL that apply.)

A. 2^n rows
B. specifying every entry is unnatural and error-prone
C. it cannot represent dependence
D. it needs a huge number of experiments to estimate

Answer: A. 2^n rows; B. specifying every entry is unnatural and error-prone; D. it needs a huge number of experiments to estimate

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: it can represent anything
- D: Correct:

Takeaway: Motivates BNs.

Source: Book 2 (Exam examples), Q376, p. 47

BAYES A70. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(j, m, a, ¬b, ¬e) = ___ (6 d.p.).

Answer: 0.000628
(Also accepted: 6.28e-4)

Working:

- P(j|a)P(m|a)P(a|¬b,¬e)P(¬b)P(¬e)
- 0.90 × 0.70 × 0.001 × 0.999 × 0.998 = 0.000628

Takeaway: One factor per variable.

Source: Book 2 (Exam examples), Q377, p. 47

BAYES A71. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(a | b) (summing over E) = ___ (3 d.p.).

Answer: 0.940
(Also accepted: 0.94)

Working:

- 0.002(0.95) + 0.998(0.94) = 0.94002

Takeaway: Marginalise E.

Source: Book 2 (Exam examples), Q378, p. 47

BAYES A72. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(a) = ___ (4 s.f., e.g. 0.001234).

Answer: 0.002516
(Also accepted: 2.516e-3)

Working:

- 0.001(0.002)(0.95) + 0.001(0.998)(0.94) + 0.999(0.002)(0.29) + 0.999(0.998)(0.001)
- = 0.002516

Takeaway: Sum over B and E.

Source: Book 2 (Exam examples), Q379, p. 47

BAYES A73. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(j) = ___ (4 s.f.).

Answer: 0.05214

Working:

- 0.9(0.002516) + 0.05(0.997484) = 0.05214

Takeaway: P(j|a)P(a) + P(j|¬a)P(¬a).

Source: Book 2 (Exam examples), Q380, p. 48

BAYES A74. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(b | a) = ___ (3 d.p.).

Answer: 0.374

Working:

- P(a|b)P(b)/P(a) = 0.94002(0.001)/0.002516 = 0.374

Takeaway: Bayes' rule.

Source: Book 2 (Exam examples), Q381, p. 48

BAYES A75. [BLANKS · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

P(B | j, m) by enumeration: s_B = __*; s*¬B = ___ (8 d.p.); posterior P(b | j, m) = ___ (4 d.p.).

(1) s_B = _____
(2) s_¬B = _____
(3) P(b|j,m) = _____

Answer: s_B = 0.00059224; s_¬B = 0.00149186; P(b|j,m) = 0.2842

Working:

- s_B = 0.001[0.002{0.95(0.9)(0.7) + 0.05(0.05)(0.01)} + 0.998{0.94(0.9)(0.7) + 0.06(0.05)(0.01)}] = 0.00059224
- s_¬B = 0.999[0.002{0.29(0.9)(0.7) + 0.71(0.05)(0.01)} + 0.998{0.001(0.9)(0.7) + 0.999(0.05)(0.01)}] = 0.00149186
- 0.00059224/0.0020841 = 0.2842

Takeaway: Sum over hidden E, A; normalise.

Source: Book 2 (Exam examples), Q382, p. 48

BAYES A76. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(e | j, m) = ___ (3 d.p.).

Answer: 0.176

Takeaway: Same enumeration with E as the query.

Source: Book 2 (Exam examples), Q383, p. 48

BAYES A77. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(b | j) (only JohnCalls observed) = ___ (3 d.p.).

Answer: 0.016

Takeaway: One call is weak evidence.

Source: Book 2 (Exam examples), Q384, p. 48

BAYES A78. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(b | j, ¬m) (JohnCalls true, MaryCalls false) = ___ (4 d.p.).

Answer: 0.0051

Takeaway: Silence from MaryCalls lowers the belief.

Source: Book 2 (Exam examples), Q385, p. 48

BAYES A79. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Number of independent probabilities in the burglary BN: ___; in the full joint table: ___.

(1) BN = ____
(2) full table = ____

Answer: BN = 10; full table = 31

Takeaway: 1 + 1 + 4 + 2 + 2 vs $2^5 - 1$.

Source: Book 2 (Exam examples), Q386, p. 48

BAYES A80. [BLANK · Easy · calculation]

A Boolean node with 3 Boolean parents needs a CPT with ___ rows.

Answer: 8

Takeaway: 2^3.

Source: Book 2 (Exam examples), Q387, p. 48

BAYES A81. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls


```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```


The burglary factorisation is P(B, E, A, J, M) =

A. P(B)P(E)P(A)P(J)P(M)
B. P(J|A)P(M|A)P(A)
C. P(A|J,M)P(B)P(E)
D. P(B)P(E)P(A|B,E)P(J|A)P(M|A)

Answer: D. P(B)P(E)P(A|B,E)P(J|A)P(M|A)

Option notes:

- A: Ignores dependencies.
- B: Missing B and E.
- C: Arrows reversed.

Takeaway: Product of node | parents.

Source: Book 2 (Exam examples), Q388, p. 48

BAYES A82. [TF · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

In the burglary BN, JohnCalls is conditionally independent of Burglary given Alarm.

Answer: True

Why: J depends only on its parent A.

Takeaway: Given parents, a node ignores other ancestors.

Source: Book 2 (Exam examples), Q389, p. 48

BAYES A83. [TF · Easy]

A Bayesian network may contain a directed cycle.

Answer: False

Why: It must be a DAG.

Takeaway: Acyclic.

Source: Book 2 (Exam examples), Q390, p. 48

BAYES A84. [MCQ · Medium]

When ordering nodes to build a compact BN you should place:

A. any order gives the same network
B. causes before effects
C. effects before causes
D. alphabetically

Answer: B. causes before effects

Option notes:

- A: Order changes compactness.
- C: Gives denser networks.
- D: Arbitrary.

Takeaway: Causal order is compact.

Source: Book 2 (Exam examples), Q391, p. 48

BAYES A85. [MULTI · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

In the query P(B | j, m): (Select ALL that apply.)

A. B is the query variable
B. E, A are hidden variables
C. J, M are evidence
D. E is evidence

Answer: A. B is the query variable; B. E, A are hidden variables; C. J, M are evidence

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Incorrect: E is not observed

Takeaway: Query, evidence, hidden.

Source: Book 2 (Exam examples), Q392, p. 48

BAYES A86. [MULTI · Medium]

Limitations of inference by enumeration: (Select ALL that apply.)

A. it gives approximate answers
B. poor scalability on large multiply connected networks
C. it cannot use a BN
D. repeated calculations, e.g. P(j|a) computed four times

Answer: B. poor scalability on large multiply connected networks; D. repeated calculations, e.g. P(j|a) computed four times

Option notes:

- A: Incorrect: it is exact
- B: Correct:
- C: Incorrect: it uses one
- D: Correct:

Takeaway: Exact but costly.

Source: Book 2 (Exam examples), Q393, p. 48

BAYES A87. [BLANKS · Medium]

Exact methods that avoid repeated work: ___ and ___.

(1) first = ____
(2) second = ____

Answer: first = variable elimination; second = clustering

Takeaway: Reuse intermediate results.

Source: Book 2 (Exam examples), Q394, p. 49

BAYES A88. [MATCH · Medium]

Match the inference family.

1. exact inference
2. approximate inference

A. inference by enumeration
B. likelihood weighting, rejection sampling, MCMC

Answer: 1 → A, 2 → B

Working:

- exact inference → inference by enumeration
- approximate inference → likelihood weighting, rejection sampling, MCMC

Takeaway: Approximate is out of scope.

Source: Book 2 (Exam examples), Q395, p. 49

BAYES A89. [BLANK · Hard · calculation]

A disease has prior 1%. A test detects it 90% of the time and gives false positives 5% of the time. P(disease | positive) = ___ (3 d.p.).

Answer: 0.154

Working:

- P(+) = 0.9(0.01) + 0.05(0.99) = 0.0585
- 0.009/0.0585 = 0.154

Takeaway: Most positives come from the many healthy people.

Source: Book 2 (Exam examples), Q396, p. 49

BAYES A90. [MCQ · Medium]

In medical diagnosis we usually know:

A. both
B. P(symptom|disease) and want P(disease|symptom)
C. P(disease|symptom) and want P(symptom|disease)
D. neither

Answer: B. P(symptom|disease) and want P(disease|symptom)

Option notes:

- A: Then no Bayes needed.

- C: Backwards.
- D: We know the causal direction.

Takeaway: Causal → diagnostic.

Source: Book 2 (Exam examples), Q397, p. 49

BAYES A91. [BLANKS · Easy]

posterior = ___ × ___ ÷ evidence.

(1) first = ____
(2) second = ____

Answer: first = likelihood; second = prior

Takeaway: Bayes' rule in words.

Source: Book 2 (Exam examples), Q398, p. 49

BAYES A92. [MCQ · Medium]

$X \perp Y \mid Z$ means $P(X \mid Y, Z) =$

A. $P(X, Y)$
B. $P(Y \mid Z)$
C. $P(X)$
D. $P(X \mid Z)$

Answer: D. $P(X \mid Z)$

Option notes:

- A: A joint.
- B: Wrong variable.
- C: That is unconditional independence.

Takeaway: Y adds nothing once Z is known.

Source: Book 2 (Exam examples), Q399, p. 49

BAYES A93. [MATCH · Medium]

Classify the environment.

1. drone in fog with noisy GPS
2. trading agent whose identical orders give different results
3. self-driving car in city traffic

A. partially observable
B. non-deterministic
C. both

Answer: 1 → A, 2 → B, 3 → C

Working:

- drone in fog with noisy GPS → partially observable
- trading agent whose identical orders give different results → non-deterministic
- self-driving car in city traffic → both

Takeaway: Two sources of uncertainty.

Source: Book 2 (Exam examples), Q400, p. 49

BAYES A94. [BLANKS · Easy]

An agent's knowledge in an uncertain domain gives only a ___ of belief, handled by ___ theory.

(1) first = ____
(2) second = ____

Answer: first = degree; second = probability

Takeaway: 0 false, 1 true.

Source: Book 2 (Exam examples), Q401, p. 49

BAYES A95. [BLANK · Hard · calculation]

Test: prior 5%, sensitivity 80%, false-positive rate 10%. P(condition | positive) = ___ (3 d.p.).

Answer: 0.296

Working:

- 0.8(0.05) = 0.04
- 0.1(0.95) = 0.095
- 0.04/0.135 = 0.296

Takeaway: 0.04/0.135.

Source: Book 2 (Exam examples), Mixed practice paper Q35, p. 55

BAYES A96. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(¬j, m, a, b, ¬e) = ___ (3 s.f., e.g. 1.23e-5).

Answer: 6.57e-5
(Also accepted: 0.0000657)

Working:

- P(¬j|a)P(m|a)P(a|b,¬e)P(b)P(¬e)
- 0.1(0.7)(0.94)(0.001)(0.998) = 6.57×10^{-5}

Takeaway: Matches the slide table row F,T,T,T,F.

Source: Book 2 (Exam examples), Mixed practice paper Q36, p. 55

BAYES A97. [TF · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

In the burglary BN, Burglary and Earthquake are independent (no evidence observed).

Answer: True

Why: No arrow between them; they become dependent only once Alarm is observed.

Takeaway: Marginal independence.

Source: Book 2 (Exam examples), Mixed practice paper Q37, p. 55

BAYES A98. [BLANK · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(¬j, ¬m, ¬a, ¬b, ¬e) = ___ (3 d.p.).

Answer: 0.936
(Also accepted: 0.937)

Takeaway: 0.999 × 0.998 × 0.999 × 0.95 × 0.99.

Source: Book 3 (AI from zero), Practice 26.1, p. 107

BAYES A99. [BLANK · Easy · calculation]

How many numbers for a Boolean node with 3 Boolean parents?

Answer: 8

Takeaway: 2^3.

Source: Book 3 (AI from zero), Practice 26.2, p. 107

BAYES A100. [BLANK · Easy · calculation]

$s_+ = 0.003$, $s_- = 0.012$. Normalised P(+) = ___.

Answer: 0.2

Takeaway: 0.003/0.015.

Source: Book 3 (AI from zero), Practice 26.3, p. 107

BAYES A101. [TF · Easy]

J is independent of B given A.

Answer: True

Why: Conditional independence from the network structure.

Takeaway: Given parents.

Source: Book 3 (AI from zero), Practice 26.4, p. 107

BAYES A102. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Burglary BN: P(j, m, a, b, ¬e) = ___ (6 d.p.).

Answer: 0.000591
(Also accepted: 5.91e-4)

Takeaway: 0.001 × 0.998 × 0.94 × 0.9 × 0.7.

Source: Book 3 (AI from zero), Chapter 26, p. 104

BAYES A103. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Why is P(b | j, m) only 0.284 even though both neighbours called?

A. the CPTs are wrong
B. John and Mary are unreliable
C. burglaries are rare, so false alarms (earthquakes, random alarms, John's confusion) together are more common
D. enumeration is approximate

Answer: C. burglaries are rare, so false alarms (earthquakes, random alarms, John's confusion) together are more common

Option notes:

- A: No.
- B: Partly, but the prior dominates.
- D: It is exact.

Takeaway: Base rates matter.

Source: Book 3 (AI from zero), Chapter 26, p. 104

BAYES A104. [BLANK · Hard · calculation]

The Lecture 11 slide prints 0.0000405594 for the joint row (¬b, e, ¬a, j, m). The correct value is 0.999 × 0.002 × 0.71 × 0.05 × 0.01 = ___ (3 s.f., e.g. 1.23e-7).

Answer: 7.09e-7
(Also accepted: 0.000000709)

Takeaway: Only this value makes s_¬B = 0.00149186.

Source: Book 1 (Compiled lecture notes), Chapter 11, p. 57

BAYES A105. [MCQ · Medium]

Full joint table vs Bayesian network: main operation for inference?

A. both multiply CPTs
B. joint table: filter matching rows and add; BN: multiply factors and sum hidden states
C. BN: filter rows; joint table: multiply
D. joint table: sampling; BN: exact

Answer: B. joint table: filter matching rows and add; BN: multiply factors and sum hidden states

Option notes:

- A: The table already holds products.
- C: Reversed.
- D: Both exact here.

Takeaway: Same answer 0.2842.

Source: Book 1 (Compiled lecture notes), Chapter 11, p. 57

BAYES A106. [BLANK · Easy · calculation]

| toothache, catch | toothache, ¬catch | ¬toothache, catch | ¬toothache, ¬catch
cavity | .108 | .012 | .072 | .008
¬cavity | .016 | .064 | .144 | .576

AIMA's dentist full joint distribution.

Dentist table (shown): P(cavity) = ___.

Answer: 0.2
(Also accepted: 0.20)

Takeaway: Add the cavity row.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: full joint distribution

BAYES A107. [BLANK · Easy · calculation]

| toothache, catch | toothache, ¬catch | ¬toothache, catch | ¬toothache, ¬catch
cavity | .108 | .012 | .072 | .008
¬cavity | .016 | .064 | .144 | .576

AIMA's dentist full joint distribution.

Dentist table: P(toothache) = ___.

Answer: 0.2
(Also accepted: 0.20)

Takeaway: .108 + .012 + .016 + .064.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: marginalisation

BAYES A108. [BLANK · Medium · calculation]

| toothache, catch | toothache, ¬catch | ¬toothache, catch | ¬toothache, ¬catch
cavity | .108 | .012 | .072 | .008
¬cavity | .016 | .064 | .144 | .576

AIMA's dentist full joint distribution.

Dentist table: P(cavity ∨ toothache) = ___.

Answer: 0.28

Takeaway: Add the six entries where either holds.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: inference by enumeration

BAYES A109. [BLANKS · Medium · calculation]

| toothache, catch | toothache, ¬catch | ¬toothache, catch | ¬toothache, ¬catch
cavity | .108 | .012 | .072 | .008
¬cavity | .016 | .064 | .144 | .576

AIMA's dentist full joint distribution.

Dentist table: P(cavity | toothache) = ___; P(¬cavity | toothache) = ___.

(1) cavity = ____
(2) ¬cavity = ____

Answer: cavity = 0.6; ¬cavity = 0.4

Takeaway: 0.12/0.2 and 0.08/0.2; α normalises.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: conditional probability

BAYES A110. [BLANK · Hard · calculation]

| toothache, catch | toothache, ¬catch | ¬toothache, catch | ¬toothache, ¬catch
cavity | .108 | .012 | .072 | .008
¬cavity | .016 | .064 | .144 | .576

AIMA's dentist full joint distribution.

Dentist table: P(cavity | toothache ∧ catch) = ___ (3 d.p.).

Answer: 0.871

Takeaway: .108/(.108 + .016).

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: conditional probability

BAYES A111. [BLANK · Medium · calculation]

Meningitis: P(stiff neck | meningitis) = 0.7, P(meningitis) = 1/50,000, P(stiff neck) = 0.01. P(meningitis | stiff neck) = ___ (4 d.p.).

Answer: 0.0014

Takeaway: Bayes' rule: still rare, 1 in 700.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: Bayes' rule

BAYES A112. [TF · Medium]

In the dentist domain, Toothache and Catch are conditionally independent given Cavity.

Answer: True

Why: Both are caused by the cavity; neither affects the other directly.

Takeaway: The basis of naive Bayes.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: conditional independence

BAYES A113. [BLANK · Hard]

(Extra, not in your slides) A naive Bayes model with one Boolean cause and n Boolean effects needs how many numbers? (write in terms of n)

Answer: 2n+1
(Also accepted: 2n + 1, 1+2n)

Takeaway: Grows linearly, not as 2^n.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: naive Bayes

BAYES A114. [BLANK · Hard · calculation]

(Extra, not in your slides) Wumpus world: after observing breezes in [1,2] and [2,1] and none in [1,1], with pit prior 0.2, P(pit in [1,3]) ≈ ___ (2 d.p.).

Answer: 0.31

Takeaway: And P(pit in [2,2]) ≈ 0.86: probability tells the agent where it is safest.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 12: Wumpus revisited

BAYES A115. [BLANKS · Medium · calculation]

A Bayesian network with n = 30 Boolean nodes, each with at most k = 5 parents, needs at most ___ numbers ($n \cdot 2^k$), versus about ___ for the full joint (2^{30}).

(1) BN = ____
(2) full joint = ____

Answer: BN = 960; full joint = 1e9

Takeaway: Locally structured = compact.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 13: compactness

BAYES A116. [BLANKS · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

(Extra, not in your slides) Burglary network built with a bad node order. Order MaryCalls, JohnCalls, Alarm, Burglary, Earthquake needs ___ numbers; order MaryCalls, JohnCalls, Earthquake, Burglary, Alarm needs ___ (causal order needs 10).

(1) M, J, A, B, E = ____
(2) M, J, E, B, A = ____

Answer: M, J, A, B, E = 13; M, J, E, B, A = 31

Takeaway: Diagnostic (effect-first) orders give denser networks.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 13: node ordering

BAYES A117. [MCQ · Hard]

(Extra, not in your slides) A node is conditionally independent of all other nodes given its:

A. children only
B. Markov blanket: parents, children and children's other parents
C. ancestors
D. parents only

Answer: B. Markov blanket: parents, children and children's other parents

Option notes:

- A: Not enough.
- C: Not enough.
- D: Parents make it independent of non-descendants only.

Takeaway: Markov blanket.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 13: conditional independence in BNs

BAYES A118. [TF · Medium]

Each BN node is conditionally independent of its non-descendants given its parents.

Answer: True

Why: The local semantics of a Bayesian network.

Takeaway: Topological semantics.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 13: semantics

BAYES A119. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

AIMA burglary example: P(j, m, a, ¬b, ¬e) = 0.90 × 0.70 × 0.001 × 0.999 × 0.998 ≈ ___ (5 d.p.).

Answer: 0.00063
(Also accepted: 0.000628, 6.3e-4)

Takeaway: Same network as Lecture 11.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 13: BN semantics

BAYES A120. [MCQ · Medium]

(Extra, not in your slides) Variable elimination improves on enumeration by:

A. storing intermediate factors so repeated subexpressions are computed once
B. sampling
C. converting to logic
D. ignoring hidden variables

Answer: A. storing intermediate factors so repeated subexpressions are computed once

Option notes:

- B: That is approximate inference.
- C: No.
- D: They are summed out.

Takeaway: Dynamic programming.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 13: variable elimination

BAYES A121. [TF · Hard]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

(Extra, not in your slides) When computing P(JohnCalls | Burglary = true), the variable MaryCalls is irrelevant and can be dropped.

Answer: True

Why: Every variable that is not an ancestor of a query or evidence variable is irrelevant: summing it out gives 1.

Takeaway: Prune irrelevant leaves.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 13: variable elimination

BAYES A122. [BLANK · Medium · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Activity 2: the experiments total N = 99,995,505,198. The event (JohnCalls, MaryCalls, Alarm, Burglary, Earthquake) = (T, F, T, T, F) occurred 25,329,240 times. Its probability = ___ (3 s.f., e.g. 1.23e-4).

Answer: 2.53e-4
(Also accepted: 0.000253)

Takeaway: P(event) = occurrences / total experiments.

Source: Lecture 11 class activity spreadsheets (experiments table, full joint probability table)

BAYES A123. [BLANK · Easy · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

The full joint table for the five binary burglary variables has ___ rows.

Answer: 32

Takeaway: 2^5.

Source: Lecture 11 class activity spreadsheets (experiments table, full joint probability table)

BAYES A124. [BLANKS · Hard · calculation]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Activity: compute P(Burglary | JohnCalls = T, MaryCalls = F) from the full joint table. Summing the rows with J = T, M = F gives 0.000256774 for B = T and 0.049798101 for B = F. P(b | j, ¬m) = ___ (4 d.p.); P(¬b | j, ¬m) = ___ (4 d.p.).

(1) P(b|j,¬m) = _____
(2) P(¬b|j,¬m) = _____

Answer: P(b|j,¬m) = 0.0051; P(¬b|j,¬m) = 0.9949

Working:

- 0.000256774/(0.000256774 + 0.049798101) = 0.00513
- 1 − 0.00513 = 0.99487

Takeaway: Filter, group by B, sum, normalise.

Source: Lecture 11 class activity spreadsheets (experiments table, full joint probability table)

BAYES A125. [MULTI · Medium]

[Diagram as text] Edges (label in brackets): Burglary — Alarm; Earthquake — Alarm; Alarm — JohnCalls; Alarm — MaryCalls

```
P(B) | P(E)
.001 | .002

B | E | P(A=t | B,E)
t | t | .95
t | f | .94
f | t | .29
f | f | .001

A | P(J=t | A) | P(M=t | A)
t | .90 | .70
f | .05 | .01
```

Which steps compute P(Burglary | j, ¬m) from the full joint table? (Select ALL that apply.)

A. add the probabilities in each group (summing over A and E)
B. multiply all 32 rows together
C. keep only rows with JohnCalls = T and MaryCalls = F
D. split the kept rows into B = T and B = F
E. divide each sum by the total of both sums

Answer: A. add the probabilities in each group (summing over A and E); C. keep only rows with JohnCalls = T and MaryCalls = F; D. split the kept rows into B = T and B = F; E. divide each sum by the total of both sums

Option notes:

- A: Correct: marginalise
- B: Incorrect: rows are added, not multiplied
- C: Correct: filter
- D: Correct: group
- E: Correct: normalise

Takeaway: Filter → group → marginalise → normalise.

Source: Lecture 11 class activity spreadsheets (experiments table, full joint probability table)

BAYES A126. [TF · Easy]

The 32 probabilities of the full joint table add up to 1 (apart from rounding).

Answer: True

Why: Every possible event is listed once.

Takeaway: Joint table sums to 1.

Source: Lecture 11 class activity spreadsheets (experiments table, full joint probability table)

Part B: calculation drills (144)

BAYES B1. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 5000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 56 times. What is its probability, as a % with 2 decimal places?

(Lec11 class quiz style.)

Answer: 1.12

Working:

- P(event) = occurrences / total = 56/5000 = 0.01120 = 1.12%.

Takeaway: Full joint table: $P(\text{event}_i) = \text{count}(\text{event}_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B2. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 2000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 345 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 17.25

Working:

- P(event) = occurrences / total = 345/2000 = 0.17250 = 17.25%.

Takeaway: Full joint table: $P(\text{event}_i) = \text{count}(\text{event}_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B3. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 2000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 381 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 19.05

Working:

- P(event) = occurrences / total = 381/2000 = 0.19050 = 19.05%.

Takeaway: Full joint table: $P(\text{event}_i) = \text{count}(\text{event}_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B4. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 2000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 196 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 9.80

Working:

- P(event) = occurrences / total = 196/2000 = 0.09800 = 9.80%.

Takeaway: Full joint table: $P(\text{event}_i) = \text{count}(\text{event}_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B5. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 8000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 178 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 2.23

Working:

- P(event) = occurrences / total = 178/8000 = 0.02225 = 2.23%.

Takeaway: Full joint table: $P(event_i) = count(event_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B6. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 10000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 126 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 1.26

Working:

- P(event) = occurrences / total = 126/10000 = 0.01260 = 1.26%.

Takeaway: Full joint table: $P(event_i) = count(event_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B7. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 8000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 373 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 4.66

Working:

- P(event) = occurrences / total = 373/8000 = 0.04663 = 4.66%.

Takeaway: Full joint table: $P(event_i) = count(event_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B8. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 8000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 186 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 2.33

Working:

- P(event) = occurrences / total = 186/8000 = 0.02325 = 2.33%.

Takeaway: Full joint table: $P(event_i) = count(event_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B9. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 8000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 327 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 4.09

Working:

- P(event) = occurrences / total = 327/8000 = 0.04088 = 4.09%.

Takeaway: Full joint table: P(event$_i$) = count(event$_i$)/N. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B10. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 5000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 17 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 0.34

Working:

- P(event) = occurrences / total = 17/5000 = 0.00340 = 0.34%.

Takeaway: Full joint table: P(event$_i$) = count(event$_i$)/N. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B11. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 10000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 390 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 3.90

Working:

- P(event) = occurrences / total = 390/10000 = 0.03900 = 3.90%.

Takeaway: Full joint table: P(event$_i$) = count(event$_i$)/N. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B12. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 10000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 39 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 0.39

Working:

- P(event) = occurrences / total = 39/10000 = 0.00390 = 0.39%.

Takeaway: Full joint table: P(event$_i$) = count(event$_i$)/N. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B13. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 8000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 258 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 3.23

Working:

- P(event) = occurrences / total = 258/8000 = 0.03225 = 3.23%.

Takeaway: Full joint table: P(event$_i$) = count(event$_i$)/N. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B14. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 5000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 52 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 1.04

Working:

- P(event) = occurrences / total = 52/5000 = 0.01040 = 1.04%.

Takeaway: Full joint table: $P(\text{event}_i) = \text{count}(\text{event}_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B15. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 8000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 195 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 2.44

Working:

- P(event) = occurrences / total = 195/8000 = 0.02438 = 2.44%.

Takeaway: Full joint table: $P(\text{event}_i) = \text{count}(\text{event}_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B16. [BLANK · Easy · calculation]

To build a full joint probability table for three binary variables, 5000 experiments were run. The event Val1 = False, Val2 = True, Val3 = True occurred 81 times. What is its probability, as a % with 2 decimal places? (Lec11 class quiz style.)

Answer: 1.62

Working:

- P(event) = occurrences / total = 81/5000 = 0.01620 = 1.62%.

Takeaway: Full joint table: $P(\text{event}_i) = \text{count}(\text{event}_i)/N$. 3 binary variables → $2^3 = 8$ rows.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B17. [BLANK · Easy · calculation]

P(B) | P(E)
0.02 | 0.05

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.9
f | t | 0.3
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.6
f | 0.1 | 0.02
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and a burglary?

Answer: 0.950

Working:

- Look up P(A = true | B = true, E = true) = 0.95.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B18. [BLANK · Easy · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is no earthquake and no burglary?

Answer: 0.001

Working:

- Look up P(A = true | B = false, E = false) = 0.001.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B19. [BLANK · Easy · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and a burglary?

Answer: 0.950

Working:

- Look up P(A = true | B = true, E = true) = 0.95.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B20. [BLANK · Easy · calculation]

P(B) | P(E)
0.1 | 0.01

```
B | E | P(A=true|B,E)
t | t | 0.9
t | f | 0.94
f | t | 0.4
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.02 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is no earthquake and a burglary?

Answer: 0.940

Working:

- Look up P(A = true | B = true, E = false) = 0.94.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B21. [BLANK · Easy · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and a burglary?

Answer: 0.950

Working:

- Look up P(A = true | B = true, E = true) = 0.95.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B22. [BLANK · Easy · calculation]

P(B) | P(E)
0.1 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.85
f | t | 0.3
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.6
f | 0.1 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and a burglary?

Answer: 0.950

Working:

- Look up P(A = true | B = true, E = true) = 0.95.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B23. [BLANK · Easy · calculation]

P(B) | P(E)
0.05 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.94
f | t | 0.3
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.6
f | 0.05 | 0.05
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is no earthquake and a burglary?

Answer: 0.940

Working:

- Look up P(A = true | B = true, E = false) = 0.94.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B24. [BLANK · Easy · calculation]

P(B) | P(E)
0.1 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.9
f | t | 0.4
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.05 | 0.05
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and no burglary?

Answer: 0.400

Working:

- Look up P(A = true | B = false, E = true) = 0.4.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B25. [BLANK · Easy · calculation]

P(B) | P(E)
0.1 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.9
t | f | 0.9
f | t | 0.2
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.1 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and a burglary?

Answer: 0.900

Working:

- Look up P(A = true | B = true, E = true) = 0.9.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B26. [BLANK · Easy · calculation]

P(B) | P(E)
0.05 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.4
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.6
f | 0.05 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and no burglary?

Answer: 0.400

Working:

- Look up P(A = true | B = false, E = true) = 0.4.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B27. [BLANK · Easy · calculation]

P(B) | P(E)
0.02 | 0.05

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.85
f | t | 0.4
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.1 | 0.05
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and a burglary?

Answer: 0.950

Working:

- Look up P(A = true | B = true, E = true) = 0.95.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B28. [BLANK · Easy · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is no earthquake and no burglary?

Answer: 0.001

Working:

- Look up P(A = true | B = false, E = false) = 0.001.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B29. [BLANK · Easy · calculation]

P(B) | P(E)
0.001 | 0.002


```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is no earthquake and no burglary?

Answer: 0.001

Working:

- Look up P(A = true | B = false, E = false) = 0.001.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B30. [BLANK · Easy · calculation]

P(B) | P(E)
0.02 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.85
f | t | 0.29
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.7
f | 0.02 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and a burglary?

Answer: 0.980

Working:

- Look up P(A = true | B = true, E = true) = 0.98.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B31. [BLANK · Easy · calculation]

P(B) | P(E)
0.1 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.4
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.8
f | 0.02 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is an earthquake and no burglary?

Answer: 0.400

Working:

- Look up P(A = true | B = false, E = true) = 0.4.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B32. [BLANK · Easy · calculation]

P(B) | P(E)
0.02 | 0.01

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.9
f | t | 0.3
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.02 | 0.01
```

Using the burglary network's tables, what is the probability of the alarm sounding when there is no earthquake and no burglary?

Answer: 0.010

Working:

- Look up P(A = true | B = false, E = false) = 0.01.

Takeaway: Each node stores P(node | its parents). Read the row that matches the parents' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B33. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(j, m, ¬a, ¬b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 4.980e-4

Working:

- P(¬b) = 0.999
- P(¬e) = 0.998
- P(¬a|¬b,¬e) = 0.999
- P(j|¬a) = 0.05
- P(m|¬a) = 0.01
- Product = 4.9800e-4.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B34. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(j, m, a, b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 5.910e-4

Working:

- P(b) = 0.001
- P(¬e) = 0.998
- P(a|b,¬e) = 0.94
- P(j|a) = 0.9
- P(m|a) = 0.7
- Product = 5.9102e-4.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B35. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.9
f | t | 0.2
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.1 | 0.01
```

Compute P(j, ¬m, a, b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 2.425e-4

Working:

- P(b) = 0.001
- P(¬e) = 0.998
- P(a|b,¬e) = 0.9
- P(j|a) = 0.9
- P(¬m|a) = 0.3
- Product = 2.4251e-4.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B36. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.9
t | f | 0.9
f | t | 0.4
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.02 | 0.01
```

Compute P(j, m, a, ¬b, e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 4.555e-3

Working:

- P(¬b) = 0.999
- P(e) = 0.02
- P(a|¬b,e) = 0.4
- P(j|a) = 0.95
- P(m|a) = 0.6
- Product = 4.5554e-3.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B37. [BLANK · Medium · calculation]

P(B) | P(E)
0.05 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.85
f | t | 0.2
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.8
f | 0.02 | 0.01
```

Compute P(¬j, ¬m, a, ¬b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 9.481e-5

Working:

- P(¬b) = 0.95
- P(¬e) = 0.998
- P(a|¬b,¬e) = 0.01
- P(¬j|a) = 0.05
- P(¬m|a) = 0.2
- Product = 9.4810e-5.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B38. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(j, ¬m, a, ¬b, e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 1.564e-4

Working:

- P(¬b) = 0.999
- P(e) = 0.002
- P(a|¬b,e) = 0.29
- P(j|a) = 0.9
- P(¬m|a) = 0.3
- Product = 1.5644e-4.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B39. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(j, ¬m, ¬a, b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 2.964e-6

Working:

- P(b) = 0.001
- P(¬e) = 0.998
- P(¬a|b,¬e) = 0.06

- P(j|¬a) = 0.05
- P(¬m|¬a) = 0.99
- Product = 2.9641e-6.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B40. [BLANK · Medium · calculation]

P(B) | P(E)
0.05 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.9
f | t | 0.3
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.6
f | 0.02 | 0.01
```

Compute P(j, m, a, b, e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 4.704e-4

Working:

- P(b) = 0.05
- P(e) = 0.02
- P(a|b,e) = 0.98
- P(j|a) = 0.8
- P(m|a) = 0.6
- Product = 4.7040e-4.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B41. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(j, ¬m, ¬a, ¬b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 4.930e-2

Working:

- P(¬b) = 0.999
- P(¬e) = 0.998
- P(¬a|¬b,¬e) = 0.999
- P(j|¬a) = 0.05
- P(¬m|¬a) = 0.99
- Product = 4.9302e-2.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B42. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(¬j, ¬m, ¬a, ¬b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 9.367e-1

Working:

- P(¬b) = 0.999
- P(¬e) = 0.998
- P(¬a|¬b,¬e) = 0.999
- P(¬j|¬a) = 0.95
- P(¬m|¬a) = 0.99
- Product = 9.3674e-1.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B43. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.01

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.85
f | t | 0.2
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.8
f | 0.02 | 0.02
```

Compute P(¬j, ¬m, a, ¬b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 3.956e-5

Working:

- P(¬b) = 0.999
- P(¬e) = 0.99
- P(a|¬b,¬e) = 0.001
- P(¬j|a) = 0.2
- P(¬m|a) = 0.2
- Product = 3.9560e-5.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B44. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(j, ¬m, ¬a, b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 2.964e-6

Working:

- P(b) = 0.001
- P(¬e) = 0.998
- P(¬a|b,¬e) = 0.06
- P(j|¬a) = 0.05
- P(¬m|¬a) = 0.99
- Product = 2.9641e-6.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B45. [BLANK · Medium · calculation]

P(B) | P(E)
0.05 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.9
f | t | 0.2
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.7
f | 0.1 | 0.02
```

Compute P(j, m, a, b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 2.515e-2

Working:

- P(b) = 0.05
- P(¬e) = 0.998
- P(a|b,¬e) = 0.9
- P(j|a) = 0.8
- P(m|a) = 0.7
- Product = 2.5150e-2.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B46. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(j, ¬m, a, ¬b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 2.692e-4

Working:

- P(¬b) = 0.999
- P(¬e) = 0.998
- P(a|¬b,¬e) = 0.001
- P(j|a) = 0.9
- P(¬m|a) = 0.3
- Product = 2.6919e-4.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B47. [BLANK · Medium · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(¬j, ¬m, ¬a, b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 5.632e-5

Working:

- P(b) = 0.001
- P(¬e) = 0.998
- P(¬a|b,¬e) = 0.06
- P(¬j|¬a) = 0.95
- P(¬m|¬a) = 0.99
- Product = 5.6317e-5.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B48. [BLANK · Medium · calculation]

P(B) | P(E)
0.05 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.4
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.8
f | 0.1 | 0.02
```

Compute P(j, m, ¬a, b, ¬e) = P(B)P(E)P(A|B,E)P(J|A)P(M|A).

Answer: 5.988e-6

Working:

- P(b) = 0.05
- P(¬e) = 0.998
- P(¬a|b,¬e) = 0.06
- P(j|¬a) = 0.1
- P(m|¬a) = 0.02
- Product = 5.9880e-6.

Takeaway: BN joint = product of each node given its parents. Use 1 − p for 'false' values.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B49. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, ¬m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 2.568e-4; s_¬B = 4.980e-2; P(b|evidence) = 0.0051

Working:

- s_B = 0.001×{P(e)[0.95·0.9·0.3 + 0.05·0.05·0.99] + P(¬e)[0.94·0.9·0.3 + 0.06·0.05·0.99]} = 2.56774e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.3 + 0.71·0.05·0.99] + P(¬e)[0.001·0.9·0.3 + 0.999·0.05·0.99]} = 4.97981e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0051.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B50. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.9
t | f | 0.9
f | t | 0.29
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.8
f | 0.02 | 0.05
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 4.090e-3; s_¬B = 4.405e-2; P(b|evidence) = 0.0850

Working:

- s_B = 0.1×{P(e)[0.9·0.05·0.8 + 0.1·0.98·0.05] + P(¬e)[0.9·0.05·0.8 + 0.1·0.98·0.05]} = 4.09000e-3.
- s_¬B = 0.9×{P(e)[0.29·0.05·0.8 + 0.71·0.98·0.05] + P(¬e)[0.005·0.05·0.8 + 0.995·0.98·0.05]} = 4.40549e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0850.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B51. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 6.637e-5; s_¬B = 9.586e-3; P(b|evidence) = 0.0069

Working:

- s_B = 0.001×{P(e)[0.95·0.1·0.7 + 0.05·0.95·0.01] + P(¬e)[0.94·0.1·0.7 + 0.06·0.95·0.01]} = 6.63712e-5.
- s_¬B = 0.999×{P(e)[0.29·0.1·0.7 + 0.71·0.95·0.01] + P(¬e)[0.001·0.1·0.7 + 0.999·0.95·0.01]} = 9.58587e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0069.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B52. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.85
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.7
f | 0.05 | 0.02
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 2.437e-3; s_¬B = 1.881e-2; P(b|evidence) = 0.1147

Working:

- s_B = 0.02×{P(e)[0.95·0.2·0.7 + 0.05·0.95·0.02] + P(¬e)[0.85·0.2·0.7 + 0.15·0.95·0.02]} = 2.43748e-3.
- s_¬B = 0.98×{P(e)[0.29·0.2·0.7 + 0.71·0.95·0.02] + P(¬e)[0.001·0.2·0.7 + 0.999·0.95·0.02]} = 1.88071e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.1147.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B53. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.94
f | t | 0.4
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.02 | 0.01
```

Compute P(B | ¬j, ¬m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 1.727e-3; s_¬B = 9.409e-1; P(b|evidence) = 0.0018

Working:

- s_B = 0.02×{P(e)[0.98·0.1·0.3 + 0.02·0.98·0.99] + P(¬e)[0.94·0.1·0.3 + 0.06·0.98·0.99]} = 1.72674e-3.
- s_¬B = 0.98×{P(e)[0.4·0.1·0.3 + 0.6·0.98·0.99] + P(¬e)[0.01·0.1·0.3 + 0.99·0.98·0.99]} = 9.40863e-1.

- P(b|e) = s_B/(s_B + s_¬B) = 0.0018.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B54. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.01 | 0.05

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.9
f | t | 0.3
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.6
f | 0.1 | 0.02
```

Compute P(B | ¬j, ¬m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 1.582e-3; s_¬B = 8.605e-1; P(b|evidence) = 0.0018

Working:

- s_B = 0.01×{P(e)[0.95·0.2·0.4 + 0.05·0.9·0.98] + P(¬e)[0.9·0.2·0.4 + 0.1·0.9·0.98]} = 1.58195e-3.
- s_¬B = 0.99×{P(e)[0.3·0.2·0.4 + 0.7·0.9·0.98] + P(¬e)[0.001·0.2·0.4 + 0.999·0.9·0.98]} = 8.60516e-1.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0018.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B55. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.6
f | 0.05 | 0.02
```

Compute P(B | j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 5.078e-2; s_¬B = 4.189e-3; P(b|evidence) = 0.9238

Working:

- s_B = 0.1×{P(e)[0.95·0.9·0.6 + 0.05·0.05·0.02] + P(¬e)[0.94·0.9·0.6 + 0.06·0.05·0.02]} = 5.07768e-2.
- s_¬B = 0.9×{P(e)[0.29·0.9·0.6 + 0.71·0.05·0.02] + P(¬e)[0.001·0.9·0.6 + 0.999·0.05·0.02]} = 4.18898e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.9238.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B56. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.01

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.85
f | t | 0.4
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.8
f | 0.02 | 0.05
```

Compute P(B | j, ¬m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 3.120e-3; s_¬B = 2.003e-2; P(b|evidence) = 0.1348

Working:

- s_B = 0.02×{P(e)[0.95·0.9·0.2 + 0.05·0.02·0.95] + P(¬e)[0.85·0.9·0.2 + 0.15·0.02·0.95]} = 3.12022e-3.
- s_¬B = 0.98×{P(e)[0.4·0.9·0.2 + 0.6·0.02·0.95] + P(¬e)[0.005·0.9·0.2 + 0.995·0.02·0.95]} = 2.00321e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.1348.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B57. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.02

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.1 | 0.05
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 3.090e-3; s_¬B = 4.041e-2; P(b|evidence) = 0.0710

Working:

- s_B = 0.1×{P(e)[0.95·0.05·0.6 + 0.05·0.9·0.05] + P(¬e)[0.94·0.05·0.6 + 0.06·0.9·0.05]} = 3.08970e-3.
- s_¬B = 0.9×{P(e)[0.29·0.05·0.6 + 0.71·0.9·0.05] + P(¬e)[0.001·0.05·0.6 + 0.999·0.9·0.05]} = 4.04085e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0710.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B58. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 5.922e-4; s_¬B = 1.492e-3; P(b|evidence) = 0.2842

Working:

- s_B = 0.001×{P(e)[0.95·0.9·0.7 + 0.05·0.05·0.01] + P(¬e)[0.94·0.9·0.7 + 0.06·0.05·0.01]} = 5.92243e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.7 + 0.71·0.05·0.01] + P(¬e)[0.001·0.9·0.7 + 0.999·0.05·0.01]} = 1.49186e-3.

- P(b|e) = s_B/(s_B + s_¬B) = 0.2842.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B59. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.1 | 0.05

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.3
f | f | 0.01

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.7
f | 0.02 | 0.02
```

Compute P(B | j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 5.267e-2; s_¬B = 1.270e-2; P(b|evidence) = 0.8057

Working:

- s_B = 0.1×{P(e)[0.95·0.8·0.7 + 0.05·0.02·0.02] + P(¬e)[0.94·0.8·0.7 + 0.06·0.02·0.02]} = 5.26704e-2.
- s_¬B = 0.9×{P(e)[0.3·0.8·0.7 + 0.7·0.02·0.02] + P(¬e)[0.01·0.8·0.7 + 0.99·0.02·0.02]} = 1.26992e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.8057.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B60. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.9
t | f | 0.85
f | t | 0.29
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.6
f | 0.1 | 0.01
```

Compute P(B | ¬j, ¬m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(¬m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 2.016e-4; s_¬B = 8.856e-1; P(b|evidence) = 0.0002

Working:

- s_B = 0.001×{P(e)[0.9·0.2·0.4 + 0.1·0.9·0.99] + P(¬e)[0.85·0.2·0.4 + 0.15·0.9·0.99]} = 2.01569e-4.
- s_¬B = 0.999×{P(e)[0.29·0.2·0.4 + 0.71·0.9·0.99] + P(¬e)[0.005·0.2·0.4 + 0.995·0.9·0.99]} = 8.85596e-1.
- P(b|e) = s_B/(s_B + s_¬B) = 0.0002.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B61. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.05 | 0.01

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.9
f | t | 0.2
f | f | 0.005

A | P(J=true|A) | P(M=true|A)
t | 0.8 | 0.8
f | 0.02 | 0.05
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 7.448e-3; s_¬B = 4.728e-2; P(b|evidence) = 0.1361

Working:

- s_B = 0.05×{P(e)[0.95·0.2·0.8 + 0.05·0.98·0.05] + P(¬e)[0.9·0.2·0.8 + 0.1·0.98·0.05]} = 7.44778e-3.
- s_¬B = 0.95×{P(e)[0.2·0.2·0.8 + 0.8·0.98·0.05] + P(¬e)[0.005·0.2·0.8 + 0.995·0.98·0.05]} = 4.72829e-2.
- P(b|e) = s_B/(s_B + s_¬B) = 0.1361.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B62. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 5.922e-4; s_¬B = 1.492e-3; P(b|evidence) = 0.2842

Working:

- s_B = 0.001×{P(e)[0.95·0.9·0.7 + 0.05·0.05·0.01] + P(¬e)[0.94·0.9·0.7 + 0.06·0.05·0.01]} = 5.92243e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.7 + 0.71·0.05·0.01] + P(¬e)[0.001·0.9·0.7 + 0.999·0.05·0.01]} = 1.49186e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.2842.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B63. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.02 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.98
t | f | 0.9
f | t | 0.4
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.95 | 0.6
f | 0.1 | 0.02
```

Compute P(B | ¬j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(¬j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 5.760e-4; s_¬B = 1.766e-2; P(b|evidence) = 0.0316

Working:

- s_B = 0.02×{P(e)[0.98·0.05·0.6 + 0.02·0.9·0.02] + P(¬e)[0.9·0.05·0.6 + 0.1·0.9·0.02]} = 5.76038e-4.
- s_¬B = 0.98×{P(e)[0.4·0.05·0.6 + 0.6·0.9·0.02] + P(¬e)[0.001·0.05·0.6 + 0.999·0.9·0.02]} = 1.76611e-2.

- P(b|e) = s_B/(s_B + s_¬B) = 0.0316.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B64. [BLANKS · Exam+ · calculation]

P(B) | P(E)
0.001 | 0.002

```
B | E | P(A=true|B,E)
t | t | 0.95
t | f | 0.94
f | t | 0.29
f | f | 0.001

A | P(J=true|A) | P(M=true|A)
t | 0.9 | 0.7
f | 0.05 | 0.01
```

Compute P(B | j, m) by enumeration: s_B = P(b)Σ_e P(e)Σ_a P(a|b,e)P(j|a)P(m|a), likewise s_¬B, then normalise. Give s_B, s_¬B (scientific notation, 4 significant figures) and P(b | evidence) (4 dp).

(1) s_B = ____
(2) s_¬B = ____
(3) P(b|evidence) = ____

Answer: s_B = 5.922e-4; s_¬B = 1.492e-3; P(b|evidence) = 0.2842

Working:

- s_B = 0.001×{P(e)[0.95·0.9·0.7 + 0.05·0.05·0.01] + P(¬e)[0.94·0.9·0.7 + 0.06·0.05·0.01]} = 5.92243e-4.
- s_¬B = 0.999×{P(e)[0.29·0.9·0.7 + 0.71·0.05·0.01] + P(¬e)[0.001·0.9·0.7 + 0.999·0.05·0.01]} = 1.49186e-3.
- P(b|e) = s_B/(s_B + s_¬B) = 0.2842.
- (Lecture 11 with j, m: s_B = 0.00059224, s_¬B = 0.00149186 → 0.2842.)

Takeaway: Enumeration: multiply along each path, add the hidden-variable branches (E, A), then normalise with α.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B65. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0972
x_1 | y_1 | z_2 | 0.1250
x_1 | y_2 | z_1 | 0.1944
x_1 | y_2 | z_2 | 0.1944
x_2 | y_1 | z_1 | 0.1667
x_2 | y_1 | z_2 | 0.0278
x_2 | y_2 | z_1 | 0.0417
x_2 | y_2 | z_2 | 0.1528

Using the full joint table, compute P(X = x_1) by summing out Y and Z.

Answer: 0.611

Working:

- Keep rows with X = x_1 and add: 0.0972 + 0.1250 + 0.1944 + 0.1944 = 0.6111.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B66. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0323
x_1 | y_1 | z_2 | 0.1129
x_1 | y_2 | z_1 | 0.1935
x_1 | y_2 | z_2 | 0.0968
x_2 | y_1 | z_1 | 0.1935
x_2 | y_1 | z_2 | 0.1290
x_2 | y_2 | z_1 | 0.1774
x_2 | y_2 | z_2 | 0.0645

Using the full joint table, compute P(X = x_1) by summing out Y and Z.

Answer: 0.435

Working:

- Keep rows with X = x_1 and add: 0.0323 + 0.1129 + 0.1935 + 0.0968 = 0.4355.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B67. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1744
x_1 | y_1 | z_2 | 0.0349
x_1 | y_2 | z_1 | 0.0581
x_1 | y_2 | z_2 | 0.1860
x_2 | y_1 | z_1 | 0.1395
x_2 | y_1 | z_2 | 0.1744
x_2 | y_2 | z_1 | 0.1628
x_2 | y_2 | z_2 | 0.0698

Using the full joint table, compute P(X = x_1) by summing out Y and Z.

Answer: 0.453

Working:

- Keep rows with X = x_1 and add: 0.1744 + 0.0349 + 0.0581 + 0.1860 = 0.4535.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B68. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1848
x_1 | y_1 | z_2 | 0.0217
x_1 | y_2 | z_1 | 0.1522
x_1 | y_2 | z_2 | 0.0978
x_2 | y_1 | z_1 | 0.1304
x_2 | y_1 | z_2 | 0.1957
x_2 | y_2 | z_1 | 0.0217
x_2 | y_2 | z_2 | 0.1957

Using the full joint table, compute $P(X = x_1)$ by summing out Y and Z.

Answer: 0.457

Working:

- Keep rows with $X = x_1$ and add: 0.1848 + 0.0217 + 0.1522 + 0.0978 = 0.4565.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B69. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1698
x_1 | y_1 | z_2 | 0.1604
x_1 | y_2 | z_1 | 0.0283
x_1 | y_2 | z_2 | 0.0566
x_2 | y_1 | z_1 | 0.1792
x_2 | y_1 | z_2 | 0.1321
x_2 | y_2 | z_1 | 0.0849
x_2 | y_2 | z_2 | 0.1887

Using the full joint table, compute $P(X = x_1)$ by summing out Y and Z.

Answer: 0.415

Working:

- Keep rows with $X = x_1$ and add: 0.1698 + 0.1604 + 0.0283 + 0.0566 = 0.4151.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B70. [BLANK · Medium · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1290
x_1 | y_1 | z_2 | 0.2581
x_1 | y_2 | z_1 | 0.0323
x_1 | y_2 | z_2 | 0.0323
x_2 | y_1 | z_1 | 0.0484
x_2 | y_1 | z_2 | 0.1774

x_2 | y_2 | z_1 | 0.0484
x_2 | y_2 | z_2 | 0.2742

Using the full joint table, compute $P(X = x_1 \mid Y = y_1)$.

Answer: 0.632

Working:

- $P(x_1, y_1) = 0.1290 + 0.2581 = 0.3871$.
- $P(y_1) = 0.1290 + 0.2581 + 0.0484 + 0.1774 = 0.6129$.
- $P(x_1|y_1) = 0.3871/0.6129 = 0.6316$.

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B71. [BLANK · Medium · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0448
x_1 | y_1 | z_2 | 0.0299
x_1 | y_2 | z_1 | 0.2090
x_1 | y_2 | z_2 | 0.2687
x_2 | y_1 | z_1 | 0.0896
x_2 | y_1 | z_2 | 0.1642
x_2 | y_2 | z_1 | 0.0597
x_2 | y_2 | z_2 | 0.1343

Using the full joint table, compute $P(X = x_1 \mid Y = y_1)$.

Answer: 0.227

Working:

- $P(x_1, y_1) = 0.0448 + 0.0299 = 0.0746$.
- $P(y_1) = 0.0448 + 0.0299 + 0.0896 + 0.1642 = 0.3284$.
- $P(x_1|y_1) = 0.0746/0.3284 = 0.2273$.

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B72. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1717
x_1 | y_1 | z_2 | 0.1515
x_1 | y_2 | z_1 | 0.1515
x_1 | y_2 | z_2 | 0.1717
x_2 | y_1 | z_1 | 0.0808
x_2 | y_1 | z_2 | 0.0707
x_2 | y_2 | z_1 | 0.0909
x_2 | y_2 | z_2 | 0.1111

Using the full joint table, compute $P(X = x_1)$ by summing out Y and Z.

Answer: 0.646

Working:

- Keep rows with $X = x_1$ and add: 0.1717 + 0.1515 + 0.1515 + 0.1717 = 0.6465.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B73. [BLANK · Medium · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0211
x_1 | y_1 | z_2 | 0.1158
x_1 | y_2 | z_1 | 0.1895
x_1 | y_2 | z_2 | 0.1789
x_2 | y_1 | z_1 | 0.1158
x_2 | y_1 | z_2 | 0.1474
x_2 | y_2 | z_1 | 0.0316
x_2 | y_2 | z_2 | 0.2000

Using the full joint table, compute $P(X = x_1 \mid Y = y_1)$.

Answer: 0.342

Working:

- $P(x_1, y_1) = 0.0211 + 0.1158 = 0.1368$.
- $P(y_1) = 0.0211 + 0.1158 + 0.1158 + 0.1474 = 0.4000$.
- $P(x_1|y_1) = 0.1368/0.4000 = 0.3421$.

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B74. [BLANK · Medium · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0538
x_1 | y_1 | z_2 | 0.1075
x_1 | y_2 | z_1 | 0.1183
x_1 | y_2 | z_2 | 0.1828
x_2 | y_1 | z_1 | 0.2043
x_2 | y_1 | z_2 | 0.1290
x_2 | y_2 | z_1 | 0.1290
x_2 | y_2 | z_2 | 0.0753

Using the full joint table, compute $P(X = x_1 \mid Y = y_1)$.

Answer: 0.326

Working:

- $P(x_1, y_1) = 0.0538 + 0.1075 = 0.1613$.
- $P(y_1) = 0.0538 + 0.1075 + 0.2043 + 0.1290 = 0.4946$.
- $P(x_1|y_1) = 0.1613/0.4946 = 0.3261$.

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B75. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1489
x_1 | y_1 | z_2 | 0.0319

x_1 | y_2 | z_1 | 0.1064
x_1 | y_2 | z_2 | 0.2021
x_2 | y_1 | z_1 | 0.1702
x_2 | y_1 | z_2 | 0.1277
x_2 | y_2 | z_1 | 0.0638
x_2 | y_2 | z_2 | 0.1489

Using the full joint table, compute P(X = x_1) by summing out Y and Z.

Answer: 0.489

Working:

- Keep rows with X = x_1 and add: 0.1489 + 0.0319 + 0.1064 + 0.2021 = 0.4894.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B76. [BLANK · Medium · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0319
x_1 | y_1 | z_2 | 0.0638
x_1 | y_2 | z_1 | 0.2021
x_1 | y_2 | z_2 | 0.2021
x_2 | y_1 | z_1 | 0.1064
x_2 | y_1 | z_2 | 0.1596
x_2 | y_2 | z_1 | 0.0745
x_2 | y_2 | z_2 | 0.1596

Using the full joint table, compute P(X = x_1 | Y = y_1).

Answer: 0.265

Working:

- P(x_1, y_1) = 0.0319 + 0.0638 = 0.0957.
- P(y_1) = 0.0319 + 0.0638 + 0.1064 + 0.1596 = 0.3617.
- P(x_1|y_1) = 0.0957/0.3617 = 0.2647.

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B77. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1758
x_1 | y_1 | z_2 | 0.1538
x_1 | y_2 | z_1 | 0.0330
x_1 | y_2 | z_2 | 0.1648
x_2 | y_1 | z_1 | 0.1099
x_2 | y_1 | z_2 | 0.1648
x_2 | y_2 | z_1 | 0.0769
x_2 | y_2 | z_2 | 0.1209

Using the full joint table, compute P(X = x_1) by summing out Y and Z.

Answer: 0.527

Working:

- Keep rows with X = x_1 and add: 0.1758 + 0.1538 + 0.0330 + 0.1648 = 0.5275.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B78. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0959
x_1 | y_1 | z_2 | 0.1370
x_1 | y_2 | z_1 | 0.1781
x_1 | y_2 | z_2 | 0.0685
x_2 | y_1 | z_1 | 0.0548
x_2 | y_1 | z_2 | 0.2329
x_2 | y_2 | z_1 | 0.1096
x_2 | y_2 | z_2 | 0.1233

Using the full joint table, compute P(X = x_1) by summing out Y and Z.

Answer: 0.479

Working:

- Keep rows with X = x_1 and add: 0.0959 + 0.1370 + 0.1781 + 0.0685 = 0.4795.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B79. [BLANK · Easy · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.0206
x_1 | y_1 | z_2 | 0.0309
x_1 | y_2 | z_1 | 0.2062
x_1 | y_2 | z_2 | 0.2062
x_2 | y_1 | z_1 | 0.1649
x_2 | y_1 | z_2 | 0.1546
x_2 | y_2 | z_1 | 0.1856
x_2 | y_2 | z_2 | 0.0309

Using the full joint table, compute P(X = x_1) by summing out Y and Z.

Answer: 0.464

Working:

- Keep rows with X = x_1 and add: 0.0206 + 0.0309 + 0.2062 + 0.2062 = 0.4639.
- (Lecture 11 example: 0.06 + 0.09 + 0.14 + 0.11 = 0.40.)

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B80. [BLANK · Medium · calculation]

X | Y | Z | P(X,Y,Z)
x_1 | y_1 | z_1 | 0.1683
x_1 | y_1 | z_2 | 0.1584
x_1 | y_2 | z_1 | 0.0198
x_1 | y_2 | z_2 | 0.0693
x_2 | y_1 | z_1 | 0.0594
x_2 | y_1 | z_2 | 0.1980
x_2 | y_2 | z_1 | 0.1782
x_2 | y_2 | z_2 | 0.1485

Using the full joint table, compute $P(X = x_1 \mid Y = y_1)$.

Answer: 0.559

Working:

- $P(x_1, y_1) = 0.1683 + 0.1584 = 0.3267$.
- $P(y_1) = 0.1683 + 0.1584 + 0.0594 + 0.1980 = 0.5842$.
- $P(x_1|y_1) = 0.3267/0.5842 = 0.5593$.

Takeaway: Marginalise = sum out. Condition = (joint of both) / (marginal of the evidence).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B81. [MCQ · Hard · calculation]

P(Disease) = 0.001. A test detects the disease with P(Positive | D) = 0.95 and gives false positives P(Positive | ¬D) = 0.01. What is P(D | Positive)?

A. 0.0010
B. 0.0999
C. 0.9500
D. 0.0868

Answer: D. 0.0868

Working:

- P(Pos) = 0.95×0.001 + 0.01×0.999 = 0.01094.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00095/0.01094 = 0.0868.

Option notes:

- A: Forgot to divide by the evidence P(S).
- B: Arithmetic slip: recompute each step carefully.
- C: That is P(S|D), the likelihood; Bayes' rule reverses it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B82. [MCQ · Hard · calculation]

P(Disease) = 0.02. A test detects the disease with P(Positive | D) = 0.8 and gives false positives P(Positive | ¬D) = 0.01. What is P(D | Positive)?

A. 0.8000
B. 0.6202
C. 0.0160
D. 0.0200

Answer: B. 0.6202

Working:

- P(Pos) = 0.8×0.02 + 0.01×0.98 = 0.02580.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.01600/0.02580 = 0.6202.

Option notes:

- A: That is P(S|D), the likelihood; Bayes' rule reverses it.
- C: Forgot to divide by the evidence P(S).
- D: That is the prior; the evidence should update it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B83. [MCQ · Hard · calculation]

P(Disease) = 0.02. A test detects the disease with P(Positive | D) = 0.95 and gives false positives P(Positive | ¬D) = 0.1. What is P(D | Positive)?

A. 0.9500
B. 0.0190
C. 0.0200
D. 0.1624

Answer: D. 0.1624

Working:

- P(Pos) = 0.95×0.02 + 0.1×0.98 = 0.11700.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.01900/0.11700 = 0.1624.

Option notes:

- A: That is P(S|D), the likelihood; Bayes' rule reverses it.
- B: Forgot to divide by the evidence P(S).
- C: That is the prior; the evidence should update it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B84. [MCQ · Hard · calculation]

P(Disease) = 0.001. A test detects the disease with P(Positive | D) = 0.8 and gives false positives P(Positive | ¬D) = 0.1. What is P(D | Positive)?

A. 0.0010
B. 0.8000
C. 0.0079
D. 0.0008

Answer: C. 0.0079

Working:

- P(Pos) = 0.8×0.001 + 0.1×0.999 = 0.10070.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00080/0.10070 = 0.0079.

Option notes:

- A: That is the prior; the evidence should update it.
- B: That is P(S|D), the likelihood; Bayes' rule reverses it.

- D: Forgot to divide by the evidence P(S).

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B85. [MCQ · Hard · calculation]

P(Disease) = 0.01. A test detects the disease with P(Positive | D) = 0.9 and gives false positives P(Positive | ¬D) = 0.1. What is P(D | Positive)?

A. 0.0100
B. 0.0090
C. 0.0833
D. 0.9000

Answer: C. 0.0833

Working:

- P(Pos) = 0.9×0.01 + 0.1×0.99 = 0.10800.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00900/0.10800 = 0.0833.

Option notes:

- A: That is the prior; the evidence should update it.
- B: Forgot to divide by the evidence P(S).
- D: That is P(S|D), the likelihood; Bayes' rule reverses it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B86. [MCQ · Hard · calculation]

P(Disease) = 0.001. A test detects the disease with P(Positive | D) = 0.8 and gives false positives P(Positive | ¬D) = 0.02. What is P(D | Positive)?

A. 0.8000
B. 0.0010
C. 0.0008
D. 0.0385

Answer: D. 0.0385

Working:

- P(Pos) = 0.8×0.001 + 0.02×0.999 = 0.02078.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00080/0.02078 = 0.0385.

Option notes:

- A: That is P(S|D), the likelihood; Bayes' rule reverses it.
- B: That is the prior; the evidence should update it.
- C: Forgot to divide by the evidence P(S).

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B87. [MCQ · Hard · calculation]

P(Disease) = 0.001. A test detects the disease with P(Positive | D) = 0.99 and gives false positives P(Positive | ¬D) = 0.02. What is P(D | Positive)?

A. 0.9900
B. 0.0543
C. 0.0472
D. 0.0010

Answer: C. 0.0472

Working:

- P(Pos) = 0.99×0.001 + 0.02×0.999 = 0.02097.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00099/0.02097 = 0.0472.

Option notes:

- A: That is P(S|D), the likelihood; Bayes' rule reverses it.
- B: Arithmetic slip: recompute each step carefully.
- D: Forgot to divide by the evidence P(S).

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B88. [MCQ · Hard · calculation]

P(Disease) = 0.1. A test detects the disease with P(Positive | D) = 0.8 and gives false positives P(Positive | ¬D) = 0.05. What is P(D | Positive)?

A. 0.1000
B. 0.0800
C. 0.8000
D. 0.6400

Answer: D. 0.6400

Working:

- P(Pos) = 0.8×0.1 + 0.05×0.9 = 0.12500.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.08000/0.12500 = 0.6400.

Option notes:

- A: That is the prior; the evidence should update it.
- B: Forgot to divide by the evidence P(S).
- C: That is P(S|D), the likelihood; Bayes' rule reverses it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B89. [MCQ · Hard · calculation]

P(Disease) = 0.1. A test detects the disease with P(Positive | D) = 0.8 and gives false positives P(Positive | ¬D) = 0.05. What is P(D | Positive)?

A. 0.6400
B. 0.1000
C. 0.8000
D. 0.0800

Answer: A. 0.6400

Working:

- P(Pos) = 0.8×0.1 + 0.05×0.9 = 0.12500.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.08000/0.12500 = 0.6400.

Option notes:

- B: That is the prior; the evidence should update it.
- C: That is P(S|D), the likelihood; Bayes' rule reverses it.
- D: Forgot to divide by the evidence P(S).

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B90. [MCQ · Hard · calculation]

P(Disease) = 0.001. A test detects the disease with P(Positive | D) = 0.99 and gives false positives P(Positive | ¬D) = 0.02. What is P(D | Positive)?

A. 0.0010
B. 0.0543
C. 0.0472
D. 0.9900

Answer: C. 0.0472

Working:

- P(Pos) = 0.99×0.001 + 0.02×0.999 = 0.02097.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00099/0.02097 = 0.0472.

Option notes:

- A: Forgot to divide by the evidence P(S).
- B: Arithmetic slip: recompute each step carefully.
- D: That is P(S|D), the likelihood; Bayes' rule reverses it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B91. [MCQ · Hard · calculation]

P(Disease) = 0.1. A test detects the disease with P(Positive | D) = 0.95 and gives false positives P(Positive | ¬D) = 0.01. What is P(D | Positive)?

A. 0.1000
B. 0.9500
C. 0.0950
D. 0.9135

Answer: D. 0.9135

Working:

- P(Pos) = 0.95×0.1 + 0.01×0.9 = 0.10400.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.09500/0.10400 = 0.9135.

Option notes:

- A: That is the prior; the evidence should update it.
- B: That is P(S|D), the likelihood; Bayes' rule reverses it.
- C: Forgot to divide by the evidence P(S).

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B92. [MCQ · Hard · calculation]

P(Disease) = 0.01. A test detects the disease with P(Positive | D) = 0.8 and gives false positives P(Positive | ¬D) = 0.01. What is P(D | Positive)?

A. 0.0100
B. 0.8000
C. 0.4469
D. 0.0080

Answer: C. 0.4469

Working:

- P(Pos) = 0.8×0.01 + 0.01×0.99 = 0.01790.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00800/0.01790 = 0.4469.

Option notes:

- A: That is the prior; the evidence should update it.
- B: That is P(S|D), the likelihood; Bayes' rule reverses it.
- D: Forgot to divide by the evidence P(S).

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B93. [MCQ · Hard · calculation]

P(Disease) = 0.02. A test detects the disease with P(Positive | D) = 0.99 and gives false positives P(Positive | ¬D) = 0.02. What is P(D | Positive)?

A. 0.5025
B. 0.0198
C. 0.0200
D. 0.9900

Answer: A. 0.5025

Working:

- P(Pos) = 0.99×0.02 + 0.02×0.98 = 0.03940.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.01980/0.03940 = 0.5025.

Option notes:

- B: Forgot to divide by the evidence P(S).
- C: That is the prior; the evidence should update it.
- D: That is P(S|D), the likelihood; Bayes' rule reverses it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B94. [MCQ · Hard · calculation]

P(Disease) = 0.001. A test detects the disease with P(Positive | D) = 0.95 and gives false positives P(Positive | ¬D) = 0.05. What is P(D | Positive)?

A. 0.9500
B. 0.0187
C. 0.0010
D. 0.0215

Answer: B. 0.0187

Working:

- P(Pos) = 0.95×0.001 + 0.05×0.999 = 0.05090.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00095/0.05090 = 0.0187.

Option notes:

- A: That is P(S|D), the likelihood; Bayes' rule reverses it.
- C: Forgot to divide by the evidence P(S).
- D: Arithmetic slip: recompute each step carefully.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B95. [MCQ · Hard · calculation]

P(Disease) = 0.05. A test detects the disease with P(Positive | D) = 0.8 and gives false positives P(Positive | ¬D) = 0.05. What is P(D | Positive)?

A. 0.0400
B. 0.0500
C. 0.4571
D. 0.8000

Answer: C. 0.4571

Working:

- P(Pos) = 0.8×0.05 + 0.05×0.95 = 0.08750.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.04000/0.08750 = 0.4571.

Option notes:

- A: Forgot to divide by the evidence P(S).
- B: That is the prior; the evidence should update it.
- D: That is P(S|D), the likelihood; Bayes' rule reverses it.

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B96. [MCQ · Hard · calculation]

P(Disease) = 0.001. A test detects the disease with P(Positive | D) = 0.95 and gives false positives P(Positive | ¬D) = 0.1. What is P(D | Positive)?

A. 0.0108
B. 0.0094
C. 0.9500
D. 0.0010

Answer: B. 0.0094

Working:

- P(Pos) = 0.95×0.001 + 0.1×0.999 = 0.10085.
- P(D|Pos) = P(Pos|D)P(D)/P(Pos) = 0.00095/0.10085 = 0.0094.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- C: That is P(S|D), the likelihood; Bayes' rule reverses it.

- D: Forgot to divide by the evidence P(S).

Takeaway: posterior = likelihood × prior ÷ evidence. Diagnostic P(cause|effect) from causal P(effect|cause).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B97. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.3 | 0.7
P(Windy = T | class) | 0.6 | 0.4
P(Humid = T | class) | 0.5 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0900; score No = 0.1680; P(Yes|e) = 0.349

Working:

- Yes: 0.3×0.6×0.5 = 0.09000.
- No: 0.7×0.4×0.6 = 0.16800.
- Normalise: 0.09000/(0.09000 + 0.16800) = 0.3488.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B98. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.4 | 0.6
P(Windy = T | class) | 0.7 | 0.5
P(Humid = T | class) | 0.3 | 0.2

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0840; score No = 0.0600; P(Yes|e) = 0.583

Working:

- Yes: 0.4×0.7×0.3 = 0.08400.
- No: 0.6×0.5×0.2 = 0.06000.
- Normalise: 0.08400/(0.08400 + 0.06000) = 0.5833.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B99. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.4 | 0.6

P(Windy = T | class) | 0.7 | 0.1
P(Humid = T | class) | 0.3 | 0.2

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0840; score No = 0.0120; P(Yes|e) = 0.875

Working:

- Yes: 0.4×0.7×0.3 = 0.08400.
- No: 0.6×0.1×0.2 = 0.01200.
- Normalise: 0.08400/(0.08400 + 0.01200) = 0.8750.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B100. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.5 | 0.5
P(Windy = T | class) | 0.2 | 0.4
P(Humid = T | class) | 0.6 | 0.7

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0600; score No = 0.1400; P(Yes|e) = 0.300

Working:

- Yes: 0.5×0.2×0.6 = 0.06000.
- No: 0.5×0.4×0.7 = 0.14000.
- Normalise: 0.06000/(0.06000 + 0.14000) = 0.3000.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B101. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.6 | 0.4
P(Windy = T | class) | 0.2 | 0.1
P(Humid = T | class) | 0.3 | 0.2

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0360; score No = 0.0080; P(Yes|e) = 0.818

Working:

- Yes: 0.6×0.2×0.3 = 0.03600.
- No: 0.4×0.1×0.2 = 0.00800.
- Normalise: 0.03600/(0.03600 + 0.00800) = 0.8182.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B102. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.3 | 0.7
P(Windy = T | class) | 0.3 | 0.9
P(Humid = T | class) | 0.5 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0450; score No = 0.3780; P(Yes|e) = 0.106

Working:

- Yes: 0.3×0.3×0.5 = 0.04500.
- No: 0.7×0.9×0.6 = 0.37800.
- Normalise: 0.04500/(0.04500 + 0.37800) = 0.1064.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B103. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.4 | 0.6
P(Windy = T | class) | 0.8 | 0.9
P(Humid = T | class) | 0.2 | 0.7

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0640; score No = 0.3780; P(Yes|e) = 0.145

Working:

- Yes: 0.4×0.8×0.2 = 0.06400.
- No: 0.6×0.9×0.7 = 0.37800.
- Normalise: 0.06400/(0.06400 + 0.37800) = 0.1448.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B104. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.3 | 0.7
P(Windy = T | class) | 0.2 | 0.9
P(Humid = T | class) | 0.5 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0300; score No = 0.3780; P(Yes|e) = 0.074

Working:

- Yes: 0.3×0.2×0.5 = 0.03000.
- No: 0.7×0.9×0.6 = 0.37800.
- Normalise: 0.03000/(0.03000 + 0.37800) = 0.0735.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B105. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.4 | 0.6
P(Windy = T | class) | 0.3 | 0.4
P(Humid = T | class) | 0.5 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0600; score No = 0.1440; P(Yes|e) = 0.294

Working:

- Yes: 0.4×0.3×0.5 = 0.06000.
- No: 0.6×0.4×0.6 = 0.14400.
- Normalise: 0.06000/(0.06000 + 0.14400) = 0.2941.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B106. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.3 | 0.7
P(Windy = T | class) | 0.2 | 0.5
P(Humid = T | class) | 0.5 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0300; score No = 0.2100; P(Yes|e) = 0.125

Working:

- Yes: 0.3×0.2×0.5 = 0.03000.
- No: 0.7×0.5×0.6 = 0.21000.
- Normalise: 0.03000/(0.03000 + 0.21000) = 0.1250.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B107. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.4 | 0.6
P(Windy = T | class) | 0.6 | 0.5
P(Humid = T | class) | 0.2 | 0.3

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0480; score No = 0.0900; P(Yes|e) = 0.348

Working:

- Yes: 0.4×0.6×0.2 = 0.04800.
- No: 0.6×0.5×0.3 = 0.09000.
- Normalise: 0.04800/(0.04800 + 0.09000) = 0.3478.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B108. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.5 | 0.5
P(Windy = T | class) | 0.3 | 0.4
P(Humid = T | class) | 0.6 | 0.2

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0900; score No = 0.0400; P(Yes|e) = 0.692

Working:

- Yes: 0.5×0.3×0.6 = 0.09000.
- No: 0.5×0.4×0.2 = 0.04000.
- Normalise: 0.09000/(0.09000 + 0.04000) = 0.6923.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B109. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.3 | 0.7
P(Windy = T | class) | 0.6 | 0.9
P(Humid = T | class) | 0.5 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0900; score No = 0.3780; P(Yes|e) = 0.192

Working:

- Yes: 0.3×0.6×0.5 = 0.09000.
- No: 0.7×0.9×0.6 = 0.37800.
- Normalise: 0.09000/(0.09000 + 0.37800) = 0.1923.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B110. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.5 | 0.5
P(Windy = T | class) | 0.3 | 0.4
P(Humid = T | class) | 0.3 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0450; score No = 0.1200; P(Yes|e) = 0.273

Working:

- Yes: 0.5×0.3×0.3 = 0.04500.
- No: 0.5×0.4×0.6 = 0.12000.
- Normalise: 0.04500/(0.04500 + 0.12000) = 0.2727.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B111. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.4 | 0.6
P(Windy = T | class) | 0.6 | 0.5
P(Humid = T | class) | 0.3 | 0.7

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0720; score No = 0.2100; P(Yes|e) = 0.255

Working:

- Yes: 0.4×0.6×0.3 = 0.07200.
- No: 0.6×0.5×0.7 = 0.21000.
- Normalise: 0.07200/(0.07200 + 0.21000) = 0.2553.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B112. [BLANKS · Medium · calculation]

| Class = Yes | Class = No
Prior | 0.6 | 0.4
P(Windy = T | class) | 0.2 | 0.1
P(Humid = T | class) | 0.3 | 0.6

A new day has Windy = T and Humid = T. Using Naive Bayes (features independent given the class), give the unnormalised score for Yes, for No (4 dp) and P(Yes | evidence) (3 dp).

(1) score Yes = ____
(2) score No = ____
(3) P(Yes|e) = ____

Answer: score Yes = 0.0360; score No = 0.0240; P(Yes|e) = 0.600

Working:

- Yes: 0.6×0.2×0.3 = 0.03600.
- No: 0.4×0.1×0.6 = 0.02400.
- Normalise: 0.03600/(0.03600 + 0.02400) = 0.6000.

Takeaway: Compare unnormalised scores to classify; normalise only if you need the probability.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B113. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 11 binary random variables?

A. 2048
B. 22
C. 1024
D. 121

Answer: A. 2048

Working:

- 2^11 = 2048. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- B: Rows double with each variable: 2^n, not 2n.
- C: Off by one in the exponent.
- D: It is exponential, not quadratic.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B114. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 3 binary random variables?

A. 9
B. 8
C. 4
D. 6

Answer: B. 8

Working:

- 2^3 = 8. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: It is exponential, not quadratic.
- C: Off by one in the exponent.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B115. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 13 binary random variables?

A. 169
B. 4096
C. 8192
D. 26

Answer: C. 8192

Working:

- 2^13 = 8192. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: It is exponential, not quadratic.
- B: Off by one in the exponent.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B116. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 5 binary random variables?

A. 10
B. 16
C. 32
D. 25

Answer: C. 32

Working:

- 2^5 = 32. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Rows double with each variable: 2^n, not 2n.
- B: Off by one in the exponent.
- D: It is exponential, not quadratic.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B117. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 9 binary random variables?

A. 81
B. 512
C. 256
D. 18

Answer: B. 512

Working:

- 2^9 = 512. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: It is exponential, not quadratic.
- C: Off by one in the exponent.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B118. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 20 binary random variables?

A. 1048576
B. 400
C. 524288
D. 40

Answer: A. 1048576

Working:

- 2^20 = 1048576. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- B: It is exponential, not quadratic.
- C: Off by one in the exponent.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B119. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 8 binary random variables?

A. 128
B. 256
C. 64
D. 16

Answer: B. 256

Working:

- 2^8 = 256. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Off by one in the exponent.
- C: It is exponential, not quadratic.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B120. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 5 binary random variables?

A. 16
B. 10
C. 32
D. 25

Answer: C. 32

Working:

- 2^5 = 32. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Off by one in the exponent.
- B: Rows double with each variable: 2^n, not 2n.
- D: It is exponential, not quadratic.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B121. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 6 binary random variables?

A. 32
B. 64
C. 36
D. 12

Answer: B. 64

Working:

- 2^6 = 64. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Off by one in the exponent.
- C: It is exponential, not quadratic.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B122. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 12 binary random variables?

A. 144
B. 4096
C. 24
D. 2048

Answer: B. 4096

Working:

- 2^12 = 4096. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: It is exponential, not quadratic.
- C: Rows double with each variable: 2^n, not 2n.
- D: Off by one in the exponent.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B123. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 5 binary random variables?

A. 16
B. 10
C. 25
D. 32

Answer: D. 32

Working:

- 2^5 = 32. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Off by one in the exponent.
- B: Rows double with each variable: 2^n, not 2n.
- C: It is exponential, not quadratic.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B124. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 16 binary random variables?

A. 256
B. 65536
C. 32768
D. 32

Answer: B. 65536

Working:

- 2^16 = 65536. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: It is exponential, not quadratic.
- C: Off by one in the exponent.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B125. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 9 binary random variables?

A. 18
B. 81
C. 512
D. 256

Answer: C. 512

Working:

- 2^9 = 512. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Rows double with each variable: 2^n, not 2n.
- B: It is exponential, not quadratic.
- D: Off by one in the exponent.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B126. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 5 binary random variables?

A. 16
B. 25
C. 32
D. 10

Answer: C. 32

Working:

- 2^5 = 32. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Off by one in the exponent.
- B: It is exponential, not quadratic.
- D: Rows double with each variable: 2^n, not 2n.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B127. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 16 binary random variables?

A. 256
B. 32
C. 32768
D. 65536

Answer: D. 65536

Working:

- 2^16 = 65536. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: It is exponential, not quadratic.
- B: Rows double with each variable: 2^n, not 2n.
- C: Off by one in the exponent.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B128. [MCQ · Easy · calculation]

How many rows does the full joint probability table have for 20 binary random variables?

A. 524288
B. 40
C. 1048576
D. 400

Answer: C. 1048576

Working:

- 2^20 = 1048576. (Lecture 11: 5 → 32, 10 → 1,024, 20 → 1,048,576.)

Option notes:

- A: Off by one in the exponent.
- B: Rows double with each variable: 2^n, not 2n.
- D: It is exponential, not quadratic.

Takeaway: Full joint tables explode as 2^n: Bayesian networks factorise to stay compact.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B129. [BLANK · Easy · calculation]

A coin is tossed and a fair 4-sided die is rolled. P(Head and 4)?

Answer: 0.1250

Working:

- Independent: 1/2 × 1/4 = 1/8.

Takeaway: AND (independent) → multiply.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B130. [BLANK · Easy · calculation]

A fair die is rolled. What is the probability of rolling a number ≤ 3 or a 6 (mutually exclusive)?

Answer: 0.667

Working:

- 3/6 + 1/6 = 4/6 = 0.667.

Takeaway: OR (mutually exclusive) → add.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B131. [BLANK · Easy · calculation]

P(rain) = 0.45. What is P(no rain)?

Answer: 0.55

Working:

- 1 − 0.45 = 0.55.

Takeaway: NOT → 1 − P.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B132. [BLANK · Easy · calculation]

A card is known to be a face card (J, Q, K). What is the probability that it is a King?

Answer: 0.333

Working:

- P(King | face) = P(King ∩ face)/P(face) = (4/52)/(12/52) = 4/12 = 1/3.

Takeaway: GIVEN → P(A|B) = P(A ∩ B)/P(B).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B133. [BLANK · Easy · calculation]

P(rain) = 0.15. What is P(no rain)?

Answer: 0.85

Working:

- 1 − 0.15 = 0.85.

Takeaway: NOT → 1 − P.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B134. [BLANK · Easy · calculation]

A coin is tossed and a fair 6-sided die is rolled. P(Head and 6)?

Answer: 0.0833

Working:

- Independent: 1/2 × 1/6 = 1/12.

Takeaway: AND (independent) → multiply.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B135. [BLANK · Easy · calculation]

A coin is tossed and a fair 4-sided die is rolled. P(Head and 4)?

Answer: 0.1250

Working:

- Independent: 1/2 × 1/4 = 1/8.

Takeaway: AND (independent) → multiply.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B136. [BLANK · Easy · calculation]

A card is known to be a face card (J, Q, K). What is the probability that it is a King?

Answer: 0.333

Working:

- P(King | face) = P(King ∩ face)/P(face) = (4/52)/(12/52) = 4/12 = 1/3.

Takeaway: GIVEN → P(A|B) = P(A ∩ B)/P(B).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B137. [BLANK · Easy · calculation]

A card is known to be a face card (J, Q, K). What is the probability that it is a King?

Answer: 0.333

Working:

- P(King | face) = P(King ∩ face)/P(face) = (4/52)/(12/52) = 4/12 = 1/3.

Takeaway: GIVEN → P(A|B) = P(A ∩ B)/P(B).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B138. [BLANK · Easy · calculation]

A coin is tossed and a fair 6-sided die is rolled. P(Head and 6)?

Answer: 0.0833

Working:

- Independent: 1/2 × 1/6 = 1/12.

Takeaway: AND (independent) → multiply.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B139. [BLANK · Easy · calculation]

A card is known to be a face card (J, Q, K). What is the probability that it is a King?

Answer: 0.333

Working:

- P(King | face) = P(King ∩ face)/P(face) = (4/52)/(12/52) = 4/12 = 1/3.

Takeaway: GIVEN → P(A|B) = P(A ∩ B)/P(B).

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B140. [BLANK · Easy · calculation]

P(rain) = 0.75. What is P(no rain)?

Answer: 0.25

Working:

- 1 − 0.75 = 0.25.

Takeaway: NOT → 1 − P.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B141. [BLANK · Easy · calculation]

P(rain) = 0.5. What is P(no rain)?

Answer: 0.5

Working:

- 1 − 0.5 = 0.5.

Takeaway: NOT → 1 − P.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B142. [BLANK · Easy · calculation]

A coin is tossed and a fair 4-sided die is rolled. P(Head and 4)?

Answer: 0.1250

Working:

- Independent: 1/2 × 1/4 = 1/8.

Takeaway: AND (independent) → multiply.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B143. [BLANK · Easy · calculation]

A coin is tossed and a fair 6-sided die is rolled. P(Head and 6)?

Answer: 0.0833

Working:

- Independent: 1/2 × 1/6 = 1/12.

Takeaway: AND (independent) → multiply.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

BAYES B144. [BLANK · Easy · calculation]

A coin is tossed and a fair 4-sided die is rolled. P(Head and 4)?

Answer: 0.1250

Working:

- Independent: 1/2 × 1/4 = 1/8.

Takeaway: AND (independent) → multiply.

Source: Lecture 11 (Bayesian network & probabilistic reasoning) + Lec11 quiz + class exercises

---

CNNs & Computer Vision (Lecture 7)

Part A: hand-written questions (102)

CNN A1. [MCQ · Easy]

Which task assigns one class label to every pixel?

A. Pooling
B. Object detection
C. Image classification
D. Semantic segmentation

Answer: D. Semantic segmentation

Option notes:

- A: Not a task.
- B: Box + label per object.
- C: One label per image.

Takeaway: Segmentation = per-pixel labels.

Source: Lecture 7, slide 12

CNN A2. [MATCH · Medium]

Match each physical signal to its sensor (Lecture 7).

1. Laser pulses
2. Radio waves
3. Infrared radiation
4. Brightness changes

A. Thermal camera → thermal image
B. LiDAR → point cloud
C. Radar → range-Doppler map
D. Sound waves → spectrogram camera
E. Event camera → event stream

Answer: 1 → B, 2 → C, 3 → A, 4 → E

Working:

- Laser pulses → LiDAR → point cloud
- Radio waves → Radar → range-Doppler map
- Infrared radiation → Thermal camera → thermal image
- Brightness changes → Event camera → event stream

Takeaway: Different sensors, complementary views of one scene.

Source: Lecture 7, slide 5

CNN A3. [BLANK · Easy]

PyTorch image batch shape order: [batch, _____, height, width].

Answer: channels
(Also accepted: channel, C)

Takeaway: NCHW.

Source: Lecture 7, slide 10

CNN A4. [MULTI · Easy]

Which are the three drivers of modern computer vision (Lecture 7)? (Select ALL that apply.)

A. Data (massive datasets)
B. Compute (GPUs)
C. Learning (better architectures and training)
D. Hand-written feature rules

Answer: A. Data (massive datasets); B. Compute (GPUs); C. Learning (better architectures and training)

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Incorrect: That's the old approach.

Takeaway: Data + Compute + Learning.

Source: Lecture 7, slide 3

CNN A5. [TF · Easy]

Image classification tells you where an object is in the image.

Answer: False

Why: Classification says WHAT is present, not where; detection adds boxes.

Takeaway: Different targets → different tasks.

Source: Lecture 7, slide 11

CNN A6. [MCQ · Medium]

Why are fully connected networks a poor default for images?

A. They cannot learn at all
B. Flattening loses neighbourhood structure and the parameter count explodes
C. They need images to be grayscale
D. They have no weights

Answer: B. Flattening loses neighbourhood structure and the parameter count explodes

Option notes:

- A: They can learn, just inefficiently.
- C: No.
- D: They have far too many.

Takeaway: Two problems, one fix: convolution.

Source: Lecture 7, slide 17-19

CNN A7. [BLANKS · Easy · calculation]

224 × 224 × 3 = _____ inputs; connected to 1,000 units gives _____ weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 150528; weights = 150528000

Takeaway: Parameters explode.

Source: Lecture 7, slide 18

CNN A8. [MATCH · Medium]

Match each idea to why convolution works well for images.

1. Locality
2. Weight sharing
3. Hierarchy

A. stack layers so simple features become parts, then objects
B. use the same feature detector at every location
C. make every pixel its own parameter
D. look at a small neighbourhood at a time

Answer: 1 → D, 2 → B, 3 → A

Working:

- Locality → look at a small neighbourhood at a time
- Weight sharing → use the same feature detector at every location
- Hierarchy → stack layers so simple features become parts, then objects

Takeaway: L-W-H.

Source: Lecture 7, slide 23

CNN A9. [MULTI · Medium]

Which variations must hand-engineered features anticipate (Lecture 7)? (Select ALL that apply.)

A. Viewpoint and scale
B. The GPU model used
C. Occlusion and background clutter
D. Illumination
E. Intra-class variation and deformation

Answer: A. Viewpoint and scale; C. Occlusion and background clutter; D. Illumination; E. Intra-class variation and deformation

Option notes:

- A: Correct:
- B: Incorrect: Irrelevant.
- C: Correct:
- D: Correct:
- E: Correct:

Takeaway: A robust rulebook needs special handling for all of them.

Source: Lecture 7, slide 16

CNN A10. [TF · Medium]

Two handwritten '7's that differ pixel-by-pixel can still share local visual clues such as a top bar and a descending diagonal.

Answer: True

Why: Lecture 7 slides 24-25.

Takeaway: Local filters match local clues.

Source: Lecture 7, slide 25

CNN A11. [MCQ · Medium]

1 | 1 | 1 | 1 | 1
0 | 0 | 0 | 1 | 0
0 | 0 | 1 | 0 | 0
0 | 1 | 0 | 0 | 0
0 | 1 | 0 | 0 | 0

```
-1 | -1 | 1
-1 | 1 | -1
1 | -1 | -1
```

In the Lecture 7 '7' example (5×5 input, diagonal kernel), output[1,1] = 3. What does a large positive output mean?

A. The patch is all zeros
B. The patch is the opposite of the pattern
C. The kernel weights were updated
D. The local patch strongly matches the diagonal pattern

Answer: D. The local patch strongly matches the diagonal pattern

Option notes:

- A: That gives 0.
- B: That gives large negatives.
- C: Forward pass only.

Takeaway: Filter = small visual question.

Source: Lecture 7, slide 32

CNN A12. [BLANK · Medium · calculation]

1 | 1 | 1 | 1 | 1
0 | 0 | 0 | 1 | 0
0 | 0 | 1 | 0 | 0
0 | 1 | 0 | 0 | 0
0 | 1 | 0 | 0 | 0

```
-1 | -1 | 1
-1 | 1 | -1
1 | -1 | -1

-2 | -3 | 1
-2 | 3 | -2
1 | -1 | -1
```

Lecture 7 '7' example, step 3: the window's +1-weight matches sum to 3 and −1-weight matches sum to 2. output[0,2] = _____.

Answer: 1

Takeaway: 3 − 2.

Source: Lecture 7, slide 30

CNN A13. [BLANKS · Hard · calculation]

1 | 1 | 1 | 1 | 1
0 | 0 | 0 | 1 | 0
0 | 0 | 1 | 0 | 0
0 | 1 | 0 | 0 | 0
0 | 1 | 0 | 0 | 0

```
-1 | -1 | 1
-1 | 1 | -1
1 | -1 | -1

-2 | -3 | 1
-2 | 3 | -2
1 | -1 | -1
```

Give the bottom row of the Lecture 7 output (output[2,0], output[2,1], output[2,2]).

(1) [2,0] = _____
(2) [2,1] = _____
(3) [2,2] = _____

Answer: [2,0] = 1; [2,1] = -1; [2,2] = -1

Takeaway: Steps 7-9: 2−1, 1−2, 0−1.

Source: Lecture 7, slide 34-36

CNN A14. [TF · Medium]

A convolutional layer adds one bias per output position.

Answer: False

Why: One bias per FILTER, shared across positions: z[p,q] = ΣΣ w x + b.

Takeaway: Shared weights, shared bias.

Source: Lecture 7, slide 39

CNN A15. [MATCH · Medium]

Match each 3×3 kernel (rows top→bottom) to what it detects.

1. [-1 0 1; -1 0 1; -1 0 1]
2. [-1 -1 -1; 0 0 0; 1 1 1]
3. [0 1 2; -1 0 1; -2 -1 0]

A. colour saturation
B. diagonal edges
C. horizontal edges
D. vertical edges

Answer: 1 → D, 2 → C, 3 → B

Working:

- [-1 0 1; -1 0 1; -1 0 1] → vertical edges
- [-1 -1 -1; 0 0 0; 1 1 1] → horizontal edges

- [0 1 2; -1 0 1; -2 -1 0] → diagonal edges

Takeaway: Lecture 7 slide 37 Grogu responses.

Source: Lecture 7, slide 37

CNN A16. [MCQ · Easy · calculation]

7×7 input, 3×3 filter, stride 1, padding 0. Output size?

A. 4×4
B. 3×3
C. 7×7
D. 5×5

Answer: D. 5×5

Option notes:

- A: (7−3)/1 + 1 = 5.
- B: That's for a 5×5 input.
- C: That needs padding 1.

Takeaway: ⌊(7 + 0 − 3)/1⌋ + 1 = 5.

Source: Lecture 7, slide 22

CNN A17. [TF · Medium]

With 'same' padding P = (K − 1)/2 and stride 1, the output has the same width as the input.

Answer: True

Why: N + 2P − K + 1 = N when 2P = K − 1. (Lecture 7 FashionCNN uses padding = 1 with 3×3.)

Takeaway: padding=1 for 3×3 keeps size.

Source: Lecture 7, slide 51

CNN A18. [BLANK · Medium · calculation]

Conv2d(1, 16, 3, stride=2, padding=1) on a 1×28×28 input gives 16××. Give the spatial size.

Answer: 14

Takeaway: ⌊(28 + 2 − 3)/2⌋ + 1 = 13 + 1 = 14.

Source: Lecture 7, slide 53

CNN A19. [MCQ · Easy · calculation]

Two 3×3 filters on an RGB input, one bias per filter. Trainable parameters?

A. 55
B. 20
C. 54
D. 56

Answer: D. 56

Option notes:

- A: One bias per filter, not one in total.
- B: Forgot the 3 input channels: (9+1)×2.
- C: Forgot the biases.

Takeaway: (3×3×3 + 1)×2 = 56.

Source: Key points about convolution filters and parameter counts

CNN A20. [TF · Medium]

A bigger input image means a convolutional layer has more trainable parameters.

Answer: False

Why: Weights are reused across positions; image size doesn't add parameters.

Takeaway: Weight sharing.

Source: Key points about convolution filters and parameter counts

CNN A21. [BLANKS · Medium · calculation]

| | | +1
| ▓ wall | | −1
START | | |

Input 64×64×3, 8 learned filters of 3×3×3, stride 1, no padding. Output volume = _____ × _____ × _____.

(1) H = ____
(2) W = ____
(3) C = ____

Answer: H = 62; W = 62; C = 8

Takeaway: Output depth = number of filters.

Source: Lecture 7, slide 40

CNN A22. [MULTI · Medium]

Select ALL true statements about convolution filters. (Select ALL that apply.)

A. Each standard filter spans all input channels
B. Pooling layers add parameters per channel
C. The same weights are reused across image positions
D. Each filter produces one output feature map
E. Number of filters = number of output channels

Answer: A. Each standard filter spans all input channels; C. The same weights are reused across image positions; D. Each filter produces one output feature map; E. Number of filters = number of output channels

Option notes:

- A: Correct: 3×3 on RGB = 27 weights.
- B: Incorrect: Pooling has none.
- C: Correct: Weight sharing.
- D: Correct:
- E: Correct:

Takeaway: Depth in = filter depth; depth out = #filters.

Source: Key points about convolution filters and parameter counts

CNN A23. [MCQ · Easy]

How many learnable parameters does a 2×2 max-pooling layer have?

A. 4 per channel
B. 0
C. 4
D. 1 bias per channel

Answer: B. 0

Option notes:

- A: No weights.
- C: No weights.
- D: No bias.

Takeaway: Pooling is a fixed operation.

Source: Lecture 7, slide 45

CNN A24. [BLANKS · Easy · calculation]

Max-pool the 4×4 patch [230 135 0 0; 231 146 0 0; 207 136 0 0; 163 120 0 0] with 2×2, stride 2.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 231; [0,1] = 0; [1,0] = 207; [1,1] = 0

Takeaway: Lecture 7 slides 42-45.

Source: Lecture 7, slide 42-45

CNN A25. [MCQ · Medium · calculation]

Leaky ReLU with α = 0.1 applied to −5 gives…

A. −0.5
B. 0.5
C. 0
D. −5

Answer: A. −0.5

Option notes:

- B: Sign is kept.
- C: That's ReLU.
- D: That's identity.

Takeaway: max(0.1×(−5), −5) = −0.5.

Source: Lecture 7, slide 41

CNN A26. [TF · Medium]

Max pooling 2×2 with stride 2 on a 512×512 feature map gives 256×256 and keeps the number of channels.

Answer: True

Why: Lecture 7 slide 42.

Takeaway: Pooling shrinks H, W only.

Source: Lecture 7, slide 42

CNN A27. [MCQ · Medium]

Why do CNNs need a non-linearity like ReLU between layers?

A. To prevent overfitting
B. To reduce the image size
C. To normalise activations
D. Without it, stacked linear layers collapse into a single linear function

Answer: D. Without it, stacked linear layers collapse into a single linear function

Option notes:

- A: That's dropout.
- B: That's pooling.
- C: That's BatchNorm.

Takeaway: Non-linearity lets stacked layers learn complex functions.

Source: Lecture 7, slide 41

CNN A28. [MATCH · Medium]

Block | Output shape
Input | 1×28×28
Conv2d(1,16,3,stride=2,padding=1) | 16×14×14
MaxPool2d(2) | 16×7×7
Conv2d(16,32,3,padding=1) | 32×7×7
MaxPool2d(2) | 32×3×3
Flatten | 288
Linear(288, 10) | 10 logits

Match each block to its role in the custom CNN (Lecture 7 lab readiness).

1. BatchNorm
2. Dropout
3. ReLU
4. MaxPool

A. stabilise activations
B. regularise the classifier
C. add non-linearity
D. reduce spatial size
E. increase the number of channels

Answer: 1 → A, 2 → B, 3 → C, 4 → D

Working:

- BatchNorm → stabilise activations
- Dropout → regularise the classifier
- ReLU → add non-linearity
- MaxPool → reduce spatial size

Takeaway: Know what each layer does.

Source: Lecture 7, slide 53

CNN A29. [MCQ · Medium]

Block | Output shape
Input | 1×28×28
Conv2d(1,16,3,stride=2,padding=1) | 16×14×14
MaxPool2d(2) | 16×7×7
Conv2d(16,32,3,padding=1) | 32×7×7
MaxPool2d(2) | 32×3×3
Flatten | 288
Linear(288, 10) | 10 logits

Why does the FashionCNN return logits and not softmax probabilities?

A. Softmax is not differentiable
B. Softmax changes the predicted class
C. Logits are always positive
D. nn.CrossEntropyLoss expects raw class scores (logits)

Answer: D. nn.CrossEntropyLoss expects raw class scores (logits)

Option notes:

- A: It is.
- B: Argmax is unchanged.
- C: They can be negative.

Takeaway: Don't put Softmax in the model during training.

Source: Lecture 7, slide 51

CNN A30. [MULTI · Easy]

Select ALL steps of each training batch (Lecture 7). (Select ALL that apply.)

A. loss.backward()
B. optimizer.zero_grad()
C. optimizer.step()
D. loss = loss_fn(logits, labels)
E. forward: logits = model(images)
F. model.eval() before every batch

Answer: A. loss.backward(); B. optimizer.zero_grad(); C. optimizer.step(); D. loss = loss_fn(logits, labels); E. forward: logits = model(images)

Option notes:

- A: Correct: Compute gradients.
- B: Correct: Clear old gradients.
- C: Correct: Update weights.
- D: Correct:
- E: Correct:
- F: Incorrect: eval() is for inference.

Takeaway: zero-grad, forward, loss, backward, step.

Source: Lecture 7, slide 52

CNN A31. [BLANK · Medium · calculation]

Block | Output shape
Input | 1×28×28
Conv2d(1,16,3,stride=2,padding=1) | 16×14×14
MaxPool2d(2) | 16×7×7
Conv2d(16,32,3,padding=1) | 32×7×7
MaxPool2d(2) | 32×3×3
Flatten | 288
Linear(288, 10) | 10 logits

Custom CNN: after the second MaxPool the tensor is 32×3×3. Flatten gives a vector of length _____.

Answer: 288

Takeaway: 32 × 3 × 3.

Source: Lecture 7, slide 53

CNN A32. [TF · Medium]

In a trained CNN, deeper layers have smaller receptive fields and detect simpler patterns such as edges.

Answer: False

Why: Deeper layers have LARGER receptive fields and combine simple features into complex ones.

Takeaway: Edges → parts → objects.

Source: Lecture 7, slide 48

CNN A33. [MATCH · Medium]

Match each classic CNN to its design idea.

1. AlexNet (2012)
2. VGG11 (2014)
3. ResNet18 (2015)

A. attention over image patches
B. 3×3 conv blocks + skip (identity) connections
C. repeated 3×3 convs in regular blocks
D. large early kernels + ReLU + dropout; GPU training breakthrough

Answer: 1 → D, 2 → C, 3 → B

Working:

- AlexNet (2012) → large early kernels + ReLU + dropout; GPU training breakthrough
- VGG11 (2014) → repeated 3×3 convs in regular blocks
- ResNet18 (2015) → 3×3 conv blocks + skip (identity) connections

Takeaway: Same recipe, different block design.

Source: Lecture 7, slide 54

CNN A34. [MCQ · Medium]

In a ResNet block y = ReLU(F(x) + x), what happens if F(x) = 0?

A. The block doubles x
B. The output is always 0

C. The block preserves its input x, giving gradients a direct route
D. Training stops

Answer: C. The block preserves its input x, giving gradients a direct route

Option notes:

- A: F(x) = 0 adds nothing.
- B: x is added back.
- D: No.

Takeaway: Skip connections ease very deep training.

Source: Lecture 7, slide 55

CNN A35. [MULTI · Medium]

Which are steps in Lecture 7's fine-tuning recipe? (Select ALL that apply.)

A. Delete all convolutional layers
B. Train with a small learning rate
C. Load pretrained weights
D. Replace the classifier to match your classes
E. Unfreeze selected layers carefully

Answer: B. Train with a small learning rate; C. Load pretrained weights; D. Replace the classifier to match your classes; E. Unfreeze selected layers carefully

Option notes:

- A: Incorrect: Reuse them.
- B: Correct:
- C: Correct: General edges and textures.
- D: Correct:
- E: Correct:

Takeaway: Pretraining is a starting point, not a guarantee.

Source: Lecture 7, slide 56

CNN A36. [BLANK · Hard · calculation]

(Extra) Two stacked 3×3 convolutions with stride 1 give each output unit a receptive field of **×** input pixels (one number).

Answer: 5

Takeaway: 1 + 2 + 2 = 5.

Source: Extra (RF formula), Lecture 7 slide 48 concept

CNN A37. [TF · Medium]

Lecture 7's top-activating patches show early channels with receptive field 7×7 selecting local geometry, while middle channels (RF 99×99) combine parts like eye + face.

Answer: True

Why: Optional slide 47.

Takeaway: Receptive field grows with depth.

Source: Lecture 7, slide 47

CNN A38. [MCQ · Medium]

Which change INCREASES the receptive field of later layers the most?

A. Adding a 1×1 convolution
B. Adding a stride-2 layer early in the network
C. Reducing the number of filters
D. Using padding

Answer: B. Adding a stride-2 layer early in the network

Option notes:

- A: 1×1 adds 0 to RF.
- C: Channels don't change RF.
- D: Padding keeps size, not RF growth.

Takeaway: Strides multiply the growth of all later layers (Extra).

Source: Lecture 7 (Computer vision)

CNN A39. [TF · Easy]

AlexNet, VGG11 and ResNet18 share the same broad recipe: a convolutional feature extractor followed by a classifier head.

Answer: True

Why: Lecture 7 slide 54.

Takeaway: Different block designs.

Source: Lecture 7 (Computer vision)

CNN A40. [MCQ · Hard]

You fine-tune a ResNet pretrained on ImageNet for a 10-class task (Lecture 7 recipe). Which order is correct?

A. Load pretrained weights → replace the classifier head → train with a small learning rate → unfreeze selected layers → evaluate on held-out data
B. Replace the head → train from random weights with a large learning rate → evaluate
C. Unfreeze all layers → large learning rate → skip evaluation
D. Evaluate first, then load pretrained weights

Answer: A. Load pretrained weights → replace the classifier head → train with a small learning rate → unfreeze selected layers → evaluate on held-out data

Option notes:

- B: Fine-tuning starts from the pretrained weights and uses a small LR.
- C: Unfreeze carefully and always evaluate on held-out data.
- D: Evaluation comes last.

Takeaway: Pretraining is a starting point, not a guarantee.

Source: Lecture 7, slide 56

CNN A41. [BLANKS · Hard · calculation]

Binary '7' (rows 11111 / 00010 / 00100 / 01000 / 01000) convolved with the diagonal kernel [[−1,−1,1], [−1,1,−1],[1,−1,−1]], stride 1, no padding. Output size ___ × ___; out[0,0] = ___; out[0,2] = ___; out[1,1] = ___.

(1) rows = _____
(2) cols = _____
(3) out[0,0] = _____
(4) out[0,2] = _____
(5) out[1,1] = _____

Answer: rows = 3; cols = 3; out[0,0] = -2; out[0,2] = 1; out[1,1] = 3

Working:

- (5 − 3)/1 + 1 = 3
- out[0,0]: patch 111/000/001 → 1 − 3 = −2
- out[0,2]: 3 − 2 = 1
- out[1,1]: patch 001/010/100 matches the diagonal → 3

Takeaway: For 0/1 inputs: (1s under +1) − (1s under −1). Full map [[−2,−3,1],[−2,3,−2],[1,−1,−1]].

Source: Book 2 (Exam examples), Q195, p. 29

CNN A42. [BLANKS · Medium · calculation]

1 | 2 | 0 | 1
0 | 1 | 3 | 1
2 | 1 | 0 | 0
1 | 0 | 1 | 2

X = [[1,2,0,1],[0,1,3,1],[2,1,0,0],[1,0,1,2]], K = [[1,0],[0,−1]], stride 1, no padding. Fill the 3 × 3 output (row by row).

(1) out[0,0] = _____
(2) out[0,1] = _____
(3) out[0,2] = _____
(4) out[1,0] = _____
(5) out[1,1] = _____
(6) out[1,2] = _____
(7) out[2,0] = _____
(8) out[2,1] = _____
(9) out[2,2] = _____

Answer: out[0,0] = 0; out[0,1] = -1; out[0,2] = -1; out[1,0] = -1; out[1,1] = 1; out[1,2] = 3; out[2,0] = 2; out[2,1] = 0; out[2,2] = -2

Working:

- out[0,0] = 1 − 1 = 0
- out[1,2] = 3 − 0 = 3
- out[2,2] = 0 − 2 = −2

Takeaway: Each output = top-left − bottom-right of its 2 × 2 patch.

Source: Book 2 (Exam examples), Q196, p. 29

CNN A43. [BLANKS · Hard · calculation]

3 | 1 | 2 | 0 | 1
0 | 2 | 1 | 3 | 2
1 | 0 | 2 | 1 | 0
2 | 1 | 0 | 1 | 3
0 | 3 | 1 | 2 | 1

Convolve the 5 × 5 X shown with the vertical-edge kernel [[1,0,−1],[1,0,−1],[1,0,−1]], stride 1, no padding. Fill the 3 × 3 output.

(1) out[0,0] = ____
(2) out[0,1] = ____
(3) out[0,2] = ____
(4) out[1,0] = ____
(5) out[1,1] = ____
(6) out[1,2] = ____
(7) out[2,0] = ____
(8) out[2,1] = ____
(9) out[2,2] = ____

Answer: out[0,0] = -1; out[0,1] = -1; out[0,2] = 2; out[1,0] = 0; out[1,1] = -2; out[1,2] = -2; out[2,0] = 0; out[2,1] = 0; out[2,2] = -1

Working:

- out[0,0] = (3 + 0 + 1) − (2 + 1 + 2) = −1
- Slide one column right each time, then down one row.

Takeaway: Left column sum minus right column sum.

Source: Book 2 (Exam examples), Q197, p. 30

CNN A44. [BLANKS · Medium · calculation]

3 | 1 | 2 | 0 | 1
0 | 2 | 1 | 3 | 2
1 | 0 | 2 | 1 | 0
2 | 1 | 0 | 1 | 3
0 | 3 | 1 | 2 | 1

Same input and kernel with stride 2. Output size ___; output entries (row by row):

(1) size = ____
(2) out[0,0] = ____
(3) out[0,1] = ____
(4) out[1,0] = ____
(5) out[1,1] = ____

Answer: size = 2; out[0,0] = -1; out[0,1] = 2; out[1,0] = 0; out[1,1] = -1

Working:

- ⌊(5 − 3)/2⌋ + 1 = 2

Takeaway: Stride 2 keeps the corners of the stride-1 map.

Source: Book 2 (Exam examples), Q198, p. 30

CNN A45. [BLANKS · Medium · calculation]

X = [[1,2],[3,4]], zero padding 1, 3 × 3 kernel of all ones, stride 1. Output size ___ × ___; every output value = ___.

(1) rows = ____
(2) cols = ____
(3) value = ____

Answer: rows = 2; cols = 2; value = 10

Working:

- (2 + 2 − 3)/1 + 1 = 2
- 1 + 2 + 3 + 4 = 10

Takeaway: Each 3 × 3 window covers the whole original image.

Source: Book 2 (Exam examples), Q199, p. 30

CNN A46. [TF · Easy]

A convolutional layer applies the same weights at every location.

Answer: True

Why: Weight sharing.

Takeaway: Same filter everywhere.

Source: Book 2 (Exam examples), Q200, p. 30

CNN A47. [MULTI · Medium]

Why convolution suits images (slides): (Select ALL that apply.)

A. locality
B. fewer parameters than dense layers
C. weight sharing
D. hierarchy of features
E. flattening the image

Answer: A. locality; B. fewer parameters than dense layers; C. weight sharing; D. hierarchy of features

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Correct:
- E: Incorrect: flattening destroys structure

Takeaway: Local, shared, hierarchical.

Source: Book 2 (Exam examples), Q201, p. 30

CNN A48. [BLANK · Medium · calculation]

Output size: 32 × 32 input, 5 × 5 filter, P = 0, S = 1 → ___.

Answer: 28

Working:

- (32 − 5)/1 + 1 = 28

Takeaway: N_out = ⌊(N + 2P − K)/S⌋ + 1.

Source: Book 2 (Exam examples), Q202, p. 30

CNN A49. [BLANK · Medium · calculation]

Output size: 32 × 32, 3 × 3, P = 1, S = 1 → ___.

Answer: 32

Working:

- (32 + 2 − 3) + 1 = 32 ('same' padding)

Takeaway: N_out = ⌊(N + 2P − K)/S⌋ + 1.

Source: Book 2 (Exam examples), Q203, p. 30

CNN A50. [BLANK · Medium · calculation]

Output size: 28 × 28, 3 × 3, P = 1, S = 2 → ___.

Answer: 14

Working:

- ⌊27/2⌋ + 1 = 14

Takeaway: N_out = ⌊(N + 2P − K)/S⌋ + 1.

Source: Book 2 (Exam examples), Q204, p. 30

CNN A51. [BLANK · Medium · calculation]

Output size: 224 × 224, 7 × 7, P = 3, S = 2 → ___.

Answer: 112

Working:

- ⌊223/2⌋ + 1 = 112 (ResNet's first layer)

Takeaway: N_out = ⌊(N + 2P − K)/S⌋ + 1.

Source: Book 2 (Exam examples), Q205, p. 30

CNN A52. [BLANK · Medium · calculation]

Output size: 227 × 227, 11 × 11, P = 0, S = 4 → ___.

Answer: 55

Working:

- 216/4 + 1 = 55 (AlexNet's first layer)

Takeaway: N_out = ⌊(N + 2P − K)/S⌋ + 1.

Source: Book 2 (Exam examples), Q206, p. 30

CNN A53. [BLANK · Medium · calculation]

Output size: 7 × 7 input, 3 × 3, S = 1, P = 0 → ___.

Answer: 5

Working:

- (7 − 3) + 1 = 5

Takeaway: N_out = ⌊(N + 2P − K)/S⌋ + 1.

Source: Book 2 (Exam examples), Q207, p. 30

CNN A54. [BLANK · Medium · calculation]

Input 64 × 64 × 3, 8 filters 3 × 3 × 3, S = 1, P = 0 → output volume ___ (H × W × C).

Answer: 62x62x8
(Also accepted: 62 x 62 x 8, 62×62×8, 62 × 62 × 8)

Takeaway: Depth = number of filters.

Source: Book 2 (Exam examples), Q208, p. 30

CNN A55. [BLANKS · Hard · calculation]

Block | Output shape
Input | 1×28×28
Conv2d(1,16,3,stride=2,padding=1) | 16×14×14
MaxPool2d(2) | 16×7×7
Conv2d(16,32,3,padding=1) | 32×7×7
MaxPool2d(2) | 32×3×3
Flatten | 288
Linear(288, 10) | 10 logits

FashionCNN: 1 × 28 × 28 → Conv2d(1,16,3,stride=2,padding=1) → ___ → MaxPool2d(2) → ___ → Conv2d(16,32,3,padding=1) → ___ → MaxPool2d(2) → ___ → flatten ___.

(1) after conv1 = ____
(2) after pool1 = ____
(3) after conv2 = ____
(4) after pool2 = ____
(5) flatten = ____

Answer: after conv1 = 16x14x14; after pool1 = 16x7x7; after conv2 = 32x7x7; after pool2 = 32x3x3; flatten = 288

Working:

- $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$
- pool → 7
- conv P=1 → 7
- pool → 3
- 32 × 3 × 3 = 288

Takeaway: Pool of 7 → $\lfloor 7/2 \rfloor = 3$.

Source: Book 2 (Exam examples), Q209, p. 31

CNN A56. [BLANK · Easy · calculation]

Two 3 × 3 filters on RGB input with one bias each: ___ parameters.

Answer: 56

Working:

- $(3 \cdot 3 \cdot 3 + 1) \times 2 = 56$

Takeaway: (K × K × C_in + 1) × filters.

Source: Book 2 (Exam examples), Q210, p. 31

CNN A57. [BLANK · Medium · calculation]

16 filters of size 5 × 5 on RGB, with biases: ___ parameters.

Answer: 1216

Working:

- (75 + 1) × 16 = 1216

Takeaway: (K × K × C_in + 1) × filters.

Source: Book 2 (Exam examples), Q211, p. 31

CNN A58. [BLANK · Medium · calculation]

32 filters 3 × 3 on a 16-channel input, with biases: ___ parameters.

Answer: 4640

Working:

- (144 + 1) × 32 = 4640

Takeaway: (K × K × C_in + 1) × filters.

Source: Book 2 (Exam examples), Q212, p. 31

CNN A59. [BLANK · Medium · calculation]

32 filters 1 × 1 on a 64-channel input, with biases: ___ parameters.

Answer: 2080

Working:

- (64 + 1) × 32 = 2080

Takeaway: (K × K × C_in + 1) × filters.

Source: Book 2 (Exam examples), Q213, p. 31

CNN A60. [BLANK · Medium · calculation]

10 filters 3 × 3 on RGB, no bias: ___ parameters.

Answer: 270

Working:

- 27 × 10 = 270

Takeaway: (K × K × C_in + 1) × filters.

Source: Book 2 (Exam examples), Q214, p. 31

CNN A61. [BLANK · Medium · calculation]

The 64 × 64 × 3 → 62 × 62 × 8 layer (3 × 3 filters) has ___ parameters.

Answer: 224

Working:

- (27 + 1) × 8 = 224

Takeaway: (K × K × C_in + 1) × filters.

Source: Book 2 (Exam examples), Q215, p. 31

CNN A62. [TF · Medium]

Doubling the image size doubles the number of filter parameters.

Answer: False

Why: Weights are shared across positions; image size never matters.

Takeaway: Params independent of H × W.

Source: Book 2 (Exam examples), Q216, p. 31

CNN A63. [MCQ · Easy]

A max-pooling layer has how many trainable parameters?

A. depends on channels
B. $K^2 + 1$
C. 0
D. K^2

Answer: C. 0

Option notes:

- A: Still zero.
- B: No weights, no bias.
- D: Pooling has no weights.

Takeaway: Pooling is fixed.

Source: Book 2 (Exam examples), Q217, p. 31

CNN A64. [BLANKS · Easy · calculation]

Max pooling 2 × 2, stride 2 on rows (230,135,0,0), (231,146,0,0), (207,136,0,0), (163,120,0,0) gives:

(1) out[0,0] = ____
(2) out[0,1] = ____
(3) out[1,0] = ____
(4) out[1,1] = ____

Answer: out[0,0] = 231; out[0,1] = 0; out[1,0] = 207; out[1,1] = 0

Takeaway: Max of each window.

Source: Book 2 (Exam examples), Q218, p. 31

CNN A65. [BLANKS · Medium · calculation]

1 | 3 | 2 | 4
5 | 6 | 1 | 2
7 | 2 | 9 | 0
1 | 8 | 3 | 4

M = [[1,3,2,4],[5,6,1,2],[7,2,9,0],[1,8,3,4]]: 2 × 2 max pooling, stride 2, then 2 × 2 average pooling, stride 2.

(1) max[0,0] = ____
(2) max[0,1] = ____
(3) max[1,0] = ____
(4) max[1,1] = ____
(5) avg[0,0] = ____
(6) avg[0,1] = ____
(7) avg[1,0] = ____
(8) avg[1,1] = ____

Answer: max[0,0] = 6; max[0,1] = 4; max[1,0] = 8; max[1,1] = 9; avg[0,0] = 3.75; avg[0,1] = 2.25; avg[1,0] = 4.5; avg[1,1] = 4

Working:

- Top-left 1,3,5,6 → max 6, mean 3.75
- Top-right 2,4,1,2 → 4, 2.25
- Bottom-left 7,2,1,8 → 8, 4.5

- Bottom-right 9,0,3,4 → 9, 4

Takeaway: Max keeps the strongest; average takes the mean.

Source: Book 2 (Exam examples), Q219, p. 31

CNN A66. [BLANKS · Medium · calculation]

Receptive field of one output after two stacked 3 × 3 stride-1 convolutions: ___ × ___; after three: ___ × ___.

(1) two (side) = _____
(2) two (side) = _____
(3) three (side) = _____
(4) three (side) = _____

Answer: two (side) = 5; two (side) = 5; three (side) = 7; three (side) = 7

Takeaway: Each 3 × 3 layer adds 2.

Source: Book 2 (Exam examples), Q220, p. 31

CNN A67. [BLANK · Hard · calculation]

Conv 3 × 3 (stride 1) → max-pool 2 × 2 (stride 2) → conv 3 × 3 (stride 1). Receptive field side of the final output: ___.

Answer: 8
(Also accepted: 8x8, 8 x 8)

Working:

- conv: r = 3
- pool: 3 + 1 × 1 = 4
- conv after stride 2: 4 + 2 × 2 = 8

Takeaway: r ← r + (K − 1) × (product of earlier strides).

Source: Book 2 (Exam examples), Q221, p. 32

CNN A68. [TF · Easy]

Deeper CNN layers have larger receptive fields and detect more abstract features.

Answer: True

Why: Edges → parts → objects.

Takeaway: Hierarchy.

Source: Book 2 (Exam examples), Q222, p. 32

CNN A69. [MATCH · Medium]

Match the classic CNN.

1. AlexNet
2. VGG
3. ResNet

A. large early kernels, GPU training breakthrough, 2012
B. repeated 3 × 3 convolutions in regular blocks
C. skip connections, y = ReLU(F(x) + x)

Answer: 1 → A, 2 → B, 3 → C

Working:

- AlexNet → large early kernels, GPU training breakthrough, 2012
- VGG → repeated 3 × 3 convolutions in regular blocks
- ResNet → skip connections, y = ReLU(F(x) + x)

Takeaway: AlexNet, VGG, ResNet.

Source: Book 2 (Exam examples), Q232, p. 33

CNN A70. [MULTI · Medium]

Problems with fully connected layers on images: (Select ALL that apply.)

A. parameters explode
B. they cannot learn at all
C. neighbouring pixels are treated as unrelated
D. spatial structure is lost when flattening

Answer: A. parameters explode; C. neighbouring pixels are treated as unrelated; D. spatial structure is lost when flattening

Option notes:

- A: Correct:
- B: Incorrect: they can, just inefficiently
- C: Correct:
- D: Correct:

Takeaway: Why convolution.

Source: Book 2 (Exam examples), Q233, p. 33

CNN A71. [BLANKS · Easy]

The two major problems of computer vision (Module 4 summary) are ___ and ___.

(1) first = _____
(2) second = _____

Answer: first = reconstruction; second = recognition

Takeaway: Reconstruction and recognition.

Source: Book 2 (Exam examples), Q234, p. 33

CNN A72. [MATCH · Medium]

Match the sensor to its digital representation.

1. LiDAR
2. event camera
3. thermal camera
4. radar

A. range–Doppler map
B. brightness-change event stream
C. depth map
D. point cloud
E. thermal image

Answer: 1 → D, 2 → B, 3 → E, 4 → A

Working:

- LiDAR → point cloud
- event camera → brightness-change event stream
- thermal camera → thermal image
- radar → range–Doppler map

Takeaway: Signal → sensor → data.

Source: Book 2 (Exam examples), Q235, p. 33

CNN A73. [TF · Medium]

In PyTorch the model should apply softmax before CrossEntropyLoss during training.

Answer: False

Why: Return raw logits; CrossEntropyLoss applies log-softmax itself.

Takeaway: Logits in, loss out.

Source: Book 2 (Exam examples), Q236, p. 33

CNN A74. [BLANKS · Medium]

PyTorch's five training steps per batch: zero_grad, ___, loss, ___, step.

(1) 2nd step = ____
(2) 4th step = ____

Answer: 2nd step = forward; 4th step = backward

Takeaway: zero_grad → forward → loss → backward → step.

Source: Book 2 (Exam examples), Q237, p. 33

CNN A75. [MULTI · Medium]

Fine-tuning steps (slides): (Select ALL that apply.)

A. evaluate on held-out data
B. train with a small learning rate
C. unfreeze selected layers
D. load pretrained weights
E. retrain from random weights
F. replace the classifier head

Answer: A. evaluate on held-out data; B. train with a small learning rate; C. unfreeze selected layers; D. load pretrained weights; F. replace the classifier head

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Correct:
- E: Incorrect: that throws away pretraining
- F: Correct:

Takeaway: Pretraining is a starting point, not a guarantee.

Source: Book 2 (Exam examples), Q238, p. 33

CNN A76. [BLANK · Medium · calculation]

Conv output size for a 64 × 64 input, 5 × 5 kernel, padding 2, stride 2: ___.

Answer: 32

Takeaway: ⌊(64 + 4 − 5)/2⌋ + 1.

Source: Book 2 (Exam examples), Mixed practice paper Q19, p. 55

CNN A77. [BLANK · Medium · calculation]

64 filters of 3 × 3 on a 32-channel input, with biases: ___ parameters.

Answer: 18496
(Also accepted: 18,496)

Takeaway: (288 + 1) × 64.

Source: Book 2 (Exam examples), Mixed practice paper Q20, p. 55

CNN A78. [BLANK · Medium · calculation]

Receptive field side after four stacked 3 × 3 stride-1 convolutions: ___.

Answer: 9
(Also accepted: 9x9, 9 x 9)

Takeaway: 1 + 4 × 2.

Source: Book 2 (Exam examples), Mixed practice paper Q21, p. 55

CNN A79. [BLANK · Easy · calculation]

Output size: N = 64, K = 5, P = 2, S = 1 → ___.

Answer: 64

Takeaway: 'Same' padding.

Source: Book 3 (AI from zero), Practice 20.1, p. 83

CNN A80. [BLANK · Medium · calculation]

Output size: N = 13, K = 3, P = 1, S = 2 → ___.

Answer: 7

Takeaway: ⌊12/2⌋ + 1.

Source: Book 3 (AI from zero), Practice 20.2, p. 84

CNN A81. [BLANK · Easy · calculation]

Parameters: 3 × 3 filters, 3 in, 10 out, no bias = ___.

Answer: 270

Takeaway: 27 × 10.

Source: Book 3 (AI from zero), Practice 20.3, p. 84

CNN A82. [BLANK · Easy · calculation]

Patch [[1,0],[2,3]], filter [[1,−1],[0,2]]. Output = ___.

Answer: 7

Takeaway: 1 + 0 + 0 + 6.

Source: Book 3 (AI from zero), Practice 20.4, p. 84

CNN A83. [BLANK · Easy · calculation]

Max pool of [[2,7],[5,1]] = ___.

Answer: 7

Takeaway: Largest value.

Source: Book 3 (AI from zero), Practice 20.5, p. 84

CNN A84. [BLANK · Medium · calculation]

Output size: N = 10, K = 3, P = 0, S = 2 → ___.

Answer: 4

Takeaway: ⌊7/2⌋ + 1.

Source: Book 3 (AI from zero), Chapter 20, p. 80

CNN A85. [BLANKS · Medium · calculation]

M = [[1,3,2,4],[6,4,1,0],[2,8,3,9],[5,3,2,2]], 2 × 2 average pooling, stride 2:

(1) avg[0,0] = ____
(2) avg[0,1] = ____
(3) avg[1,0] = ____
(4) avg[1,1] = ____

Answer: avg[0,0] = 3.5; avg[0,1] = 1.75; avg[1,0] = 4.5; avg[1,1] = 4

Takeaway: Mean of each window. (Max pooling gives [[6,4],[8,9]].)

Source: Book 3 (AI from zero), Chapter 20, p. 80

CNN A86. [BLANKS · Medium · calculation]

softmax of scores (2, 1, 0) = (___, ___, ___) (3 d.p.).

(1) 1st = ____
(2) 2nd = ____
(3) 3rd = ____

Answer: 1st = 0.665; 2nd = 0.245; 3rd = 0.090

Takeaway: e^z/Σe^z.

Source: Book 3 (AI from zero), Chapter 20, p. 80

CNN A87. [TF · Hard]

Deep-learning 'convolution' flips the filter before sliding it.

Answer: False

Why: It does not flip: technically it is cross-correlation.

Takeaway: No flip in CNNs.

Source: Book 3 (AI from zero), Chapter 20, p. 80

CNN A88. [BLANK · Hard · calculation]

A dense layer from 64 × 64 × 3 inputs to a 62 × 62 × 8 output would need about 12,288 × 30,752 ≈ ___ million weights (nearest million), versus 224 for the conv layer.

Answer: 378

Takeaway: Why convolution.

Source: Book 3 (AI from zero), Chapter 20, p. 80

CNN A89. [BLANK · Easy]

Leaky ReLU in the slides is f(x) = max(αx, x) with α = ___.

Answer: 0.1

Takeaway: Small slope for negatives.

Source: Book 1 (Compiled lecture notes), Chapter 7, p. 39

CNN A90. [MCQ · Hard · calculation]

Block | Output shape
Input | 1×28×28
Conv2d(1,16,3,stride=2,padding=1) | 16×14×14
MaxPool2d(2) | 16×7×7
Conv2d(16,32,3,padding=1) | 32×7×7
MaxPool2d(2) | 32×3×3
Flatten | 288
Linear(288, 10) | 10 logits

Lab custom CNN: 1 × 28 × 28 → BatchNorm → Conv16 stride 2 → ReLU → MaxPool → Conv32 → MaxPool → Dropout → Flatten → Linear → 10. What is the flattened length?

A. 288
B. 1568
C. 3136
D. 512

Answer: A. 288

Option notes:

- B: That is the stride-1 version (32 × 7 × 7).
- C: 64 × 7 × 7.
- D: No.

Takeaway: 32 × 3 × 3 = 288.

Source: Book 1 (Compiled lecture notes), Chapter 7, p. 39

CNN A91. [BLANKS · Hard · calculation]

AIMA's 1-D convolution: input x = (5, 6, 6, 2, 5, 6, 5), kernel k = (+1, −1, +1), stride 2, no padding. Output z = (___, ___, ___).

(1) z1 = ____
(2) z2 = ____
(3) z3 = ____

Answer: z1 = 5; z2 = 9; z3 = 4

Working:

- (5, 6, 6): 5 − 6 + 6 = 5
- (6, 2, 5): 6 − 2 + 5 = 9
- (5, 6, 5): 5 − 6 + 5 = 4

Takeaway: Windows start at positions 1, 3, 5.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: convolutional networks

CNN A92. [MCQ · Medium]

In AIMA's 1-D example the kernel (+1, −1, +1) responds most strongly to:

A. a steady increase
B. a dark point between two bright points (a local dip pattern)
C. constant brightness
D. a single bright point

Answer: B. a dark point between two bright points (a local dip pattern)

Option notes:

- A: That would be (−1, 0, +1).
- C: Gives just the value.
- D: That would be (−1, +1, −1).

Takeaway: A kernel is a feature detector.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: convolution

CNN A93. [TF · Easy]

A pooling layer with stride s downsamples its input by a factor of about s.

Answer: True

Why: Like convolution with stride s, but with a fixed max or average.

Takeaway: No learned weights.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: pooling

CNN A94. [BLANK · Hard · calculation]

(Extra, not in your slides) Receptive field: two 3-wide 1-D convolution layers, the first with stride 2, the second with stride 1. Each final unit sees ___ input positions.

Answer: 7

Takeaway: r = 3 + (3 − 1) × 2.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: receptive fields

CNN A95. [BLANK · Easy · calculation]

Sheet rule: a 3 by 3 filter on RGB input contains ___ weights.

Answer: 27

Takeaway: 3 × 3 × 3: each filter spans all input channels.

Source: Key points about convolution filters and parameter counts

CNN A96. [BLANK · Medium · calculation]

Four 5 by 5 filters on RGB input, one bias per filter. Trainable parameters = ___.

Answer: 304

Working:

- 5 × 5 × 3 = 75
- ◦ 1 bias = 76
- × 4 filters = 304

Takeaway: (5 × 5 × 3 + 1) × 4.

Source: Key points about convolution filters and parameter counts

CNN A97. [BLANK · Medium · calculation]

Eight 3 by 3 filters on an input with 16 channels, no bias. Trainable parameters = ___.

Answer: 1152
(Also accepted: 1,152)

Working:

- 3 × 3 × 16 = 144
- × 8 = 1152

Takeaway: No bias stated → no +1.

Source: Key points about convolution filters and parameter counts

CNN A98. [BLANK · Medium · calculation]

Two 3 by 3 filters on a greyscale (1-channel) input, one bias per filter. Trainable parameters = ___.

Answer: 20

Takeaway: (9 + 1) × 2.

Source: Key points about convolution filters and parameter counts

CNN A99. [BLANK · Easy]

Sheet rule: a layer with 6 filters produces ___ output feature maps (output channels).

Answer: 6

Takeaway: One map per filter.

Source: Key points about convolution filters and parameter counts

CNN A100. [TF · Medium]

Moving from a 32 × 32 to a 224 × 224 RGB image changes the parameter count of a layer of two 3 × 3 filters (with biases).

Answer: False

Why: Still (27 + 1) × 2 = 56: weights are reused across positions.

Takeaway: Image size never adds parameters.

Source: Key points about convolution filters and parameter counts

CNN A101. [MCQ · Medium]

A student computes two 3 by 3 filters on RGB (with biases) as (3 × 3 + 1) × 2 = 20. The mistake is:

A. multiplying by the number of filters
B. adding the bias
C. using the image size
D. forgetting that each filter spans all 3 input channels

Answer: D. forgetting that each filter spans all 3 input channels

Option notes:

- A: That step is correct.
- B: The bias was specified.
- C: They did not.

Takeaway: Correct: (27 + 1) × 2 = 56.

Source: Key points about convolution filters and parameter counts

CNN A102. [MULTI · Easy]

The sheet's recipe for counting parameters (select all steps): (Select ALL that apply.)

A. multiply by the number of image positions
B. add one bias per filter if specified
C. multiply by the number of filters
D. multiply filter height × width × input channels

Answer: B. add one bias per filter if specified; C. multiply by the number of filters; D. multiply filter height × width × input channels

Option notes:

- A: Incorrect: weights are shared across positions
- B: Correct:
- C: Correct:
- D: Correct:

Takeaway: Three steps.

Source: Key points about convolution filters and parameter counts

Part B: calculation drills (128)

CNN B1. [MCQ · Easy · calculation]

Input 64×64, kernel 11×11, stride 4, padding 0. What is the output width?

A. 14
B. 21
C. 13
D. 54

Answer: A. 14

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (64 + 0 - 11)/4 \rfloor + 1 = \lfloor 13.25 \rfloor + 1 = 14$.

Option notes:

- B: Arithmetic slip: recompute each step carefully.

- C: Forgot the + 1.
- D: Ignored stride/padding.

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B2. [MCQ · Easy · calculation]

Input 227×227, kernel 1×1, stride 2, padding 2. What is the output width?

A. 115
B. 114
C. 116
D. 227

Answer: C. 116

Working:

- N_out = ⌊(N + 2P − K)/S⌋ + 1 = ⌊(227 + 4 − 1)/2⌋ + 1 = ⌊115⌋ + 1 = 116.

Option notes:

- A: Forgot the + 1.
- B: Forgot the padding (2P).
- D: Ignored stride/padding.

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B3. [MCQ · Easy · calculation]

Input 32×32, kernel 5×5, stride 2, padding 1. What is the output width?

A. 28
B. 22
C. 15
D. 14

Answer: C. 15

Working:

- N_out = ⌊(N + 2P − K)/S⌋ + 1 = ⌊(32 + 2 − 5)/2⌋ + 1 = ⌊14.5⌋ + 1 = 15.

Option notes:

- A: Ignored stride/padding.
- B: Arithmetic slip: recompute each step carefully.
- D: Forgot the padding (2P).

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B4. [MCQ · Easy · calculation]

Input 64×64, kernel 3×3, stride 4, padding 0. What is the output width?

A. 23
B. 16
C. 15
D. 62

Answer: B. 16

Working:

- N_out = ⌊(N + 2P − K)/S⌋ + 1 = ⌊(64 + 0 − 3)/4⌋ + 1 = ⌊15.25⌋ + 1 = 16.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- C: Forgot the + 1.
- D: Ignored stride/padding.

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B5. [MCQ · Easy · calculation]

Input 227×227, kernel 7×7, stride 2, padding 3. What is the output width?

A. 113
B. 114
C. 111
D. 112

Answer: B. 114

Working:

- N_out = ⌊(N + 2P − K)/S⌋ + 1 = ⌊(227 + 6 − 7)/2⌋ + 1 = ⌊113⌋ + 1 = 114.

Option notes:

- A: Forgot the + 1.
- C: Forgot the padding (2P).
- D: Padding is added on both sides: 2P.

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B6. [MCQ · Easy · calculation]

Input 5×5, kernel 5×5, stride 2, padding 1. What is the output width?

A. -12
B. 1
C. 9
D. 2

Answer: D. 2

Working:

- N_out = ⌊(N + 2P − K)/S⌋ + 1 = ⌊(5 + 2 − 5)/2⌋ + 1 = ⌊1⌋ + 1 = 2.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Forgot the + 1.
- C: Arithmetic slip: recompute each step carefully.

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B7. [MCQ · Easy · calculation]

Input 224×224, kernel 5×5, stride 1, padding 3. What is the output width?

A. 225
B. 226
C. 220
D. 223

Answer: B. 226

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (224 + 6 - 5)/1 \rfloor + 1 = \lfloor 225 \rfloor + 1 = 226$.

Option notes:

- A: Forgot the + 1.
- C: Forgot the padding (2P).
- D: Padding is added on both sides: 2P.

Takeaway: Output size = $\lfloor (N + 2P - K)/S \rfloor + 1$. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B8. [MCQ · Easy · calculation]

Input 64×64, kernel 1×1, stride 1, padding 2. What is the output width?

A. 67
B. 66
C. 64
D. 68

Answer: D. 68

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (64 + 4 - 1)/1 \rfloor + 1 = \lfloor 67 \rfloor + 1 = 68$.

Option notes:

- A: Forgot the + 1.
- B: Padding is added on both sides: 2P.
- C: Forgot the padding (2P).

Takeaway: Output size = $\lfloor (N + 2P - K)/S \rfloor + 1$. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B9. [MCQ · Easy · calculation]

Input 28×28, kernel 7×7, stride 2, padding 3. What is the output width?

A. 13
B. 14
C. 22
D. 11

Answer: B. 14

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (28 + 6 - 7)/2 \rfloor + 1 = \lfloor 13.5 \rfloor + 1 = 14$.

Option notes:

- A: Padding is added on both sides: 2P.
- C: Ignored stride/padding.
- D: Forgot the padding (2P).

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B10. [MCQ · Easy · calculation]

Input 64×64, kernel 7×7, stride 1, padding 3. What is the output width?

A. 61
B. 63
C. 64
D. 58

Answer: C. 64

Working:

- N_out = ⌊(N + 2P − K)/S⌋ + 1 = ⌊(64 + 6 − 7)/1⌋ + 1 = ⌊63⌋ + 1 = 64.

Option notes:

- A: Padding is added on both sides: 2P.
- B: Forgot the + 1.
- D: Forgot the padding (2P).

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B11. [MCQ · Easy · calculation]

Input 64×64, kernel 3×3, stride 1, padding 2. What is the output width?

A. 62
B. 64
C. 66
D. 65

Answer: C. 66

Working:

- N_out = ⌊(N + 2P − K)/S⌋ + 1 = ⌊(64 + 4 − 3)/1⌋ + 1 = ⌊65⌋ + 1 = 66.

Option notes:

- A: Forgot the padding (2P).
- B: Padding is added on both sides: 2P.
- D: Forgot the + 1.

Takeaway: Output size = ⌊(N + 2P − K)/S⌋ + 1. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B12. [MCQ · Easy · calculation]

Input 28×28, kernel 7×7, stride 4, padding 0. What is the output width?

A. 5
B. 6

C. 13
D. 22

Answer: B. 6

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (28 + 0 - 7)/4 \rfloor + 1 = \lfloor 5.25 \rfloor + 1 = 6$.

Option notes:

- A: Forgot the + 1.
- C: Arithmetic slip: recompute each step carefully.
- D: Ignored stride/padding.

Takeaway: Output size = $\lfloor (N + 2P - K)/S \rfloor + 1$. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B13. [MCQ · Easy · calculation]

Input 227×227, kernel 7×7, stride 1, padding 0. What is the output width?

A. 221
B. 220
C. 188
D. 254

Answer: A. 221

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (227 + 0 - 7)/1 \rfloor + 1 = \lfloor 220 \rfloor + 1 = 221$.

Option notes:

- B: Forgot the + 1.
- C: Arithmetic slip: recompute each step carefully.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Output size = $\lfloor (N + 2P - K)/S \rfloor + 1$. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B14. [MCQ · Easy · calculation]

Input 7×7, kernel 5×5, stride 1, padding 0. What is the output width?

A. 3
B. -11
C. 2
D. 10

Answer: A. 3

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (7 + 0 - 5)/1 \rfloor + 1 = \lfloor 2 \rfloor + 1 = 3$.

Option notes:

- B: Arithmetic slip: recompute each step carefully.
- C: Forgot the + 1.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Output size = $\lfloor (N + 2P - K)/S \rfloor + 1$. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B15. [MCQ · Easy · calculation]

Input 227×227, kernel 5×5, stride 2, padding 0. What is the output width?

A. 112
B. 111
C. 129
D. 223

Answer: A. 112

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (227 + 0 - 5)/2 \rfloor + 1 = \lfloor 111 \rfloor + 1 = 112$.

Option notes:

- B: Forgot the + 1.
- C: Arithmetic slip: recompute each step carefully.
- D: Ignored stride/padding.

Takeaway: Output size = $\lfloor (N + 2P - K)/S \rfloor + 1$. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B16. [MCQ · Easy · calculation]

Input 5×5, kernel 1×1, stride 2, padding 3. What is the output width?

A. 3
B. 4
C. 5
D. 6

Answer: D. 6

Working:

- $N_{out} = \lfloor (N + 2P - K)/S \rfloor + 1 = \lfloor (5 + 6 - 1)/2 \rfloor + 1 = \lfloor 5 \rfloor + 1 = 6$.

Option notes:

- A: Forgot the padding (2P).
- B: Padding is added on both sides: 2P.
- C: Forgot the + 1.

Takeaway: Output size = $\lfloor (N + 2P - K)/S \rfloor + 1$. Floor BEFORE adding 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B17. [MCQ · Medium · calculation]

A convolutional layer applies 128 filters of size 5×5 to an input with 64 channel(s), with no bias. How many trainable parameters? (Image size does not matter.)

A. 235520
B. 3200
C. 204928
D. 204800

Answer: D. 204800

Working:

- Weights per filter = 5×5×64 = 1600.
- No bias.

- × 128 filters = 204800.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Forgot that each filter spans ALL input channels (× Cin).
- C: No bias was specified, so don't add one.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B18. [MCQ · Medium · calculation]

A convolutional layer applies 16 filters of size 3×3 to an input with 64 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 160
B. 9217
C. 9232
D. 9216

Answer: C. 9232

Working:

- Weights per filter = 3×3×64 = 576.
- +1 bias → 577 per filter.
- × 16 filters = 9232.

Option notes:

- A: Forgot that each filter spans ALL input channels (× Cin).
- B: One bias per FILTER, not one in total.
- D: Forgot the biases.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B19. [MCQ · Medium · calculation]

A convolutional layer applies 16 filters of size 7×7 to an input with 1 channel(s), with no bias. How many trainable parameters? (Image size does not matter.)

A. 666
B. 902
C. 800
D. 784

Answer: D. 784

Working:

- Weights per filter = 7×7×1 = 49.
- No bias.
- × 16 filters = 784.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- C: No bias was specified, so don't add one.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B20. [MCQ · Medium · calculation]

A convolutional layer applies 2 filters of size 1×1 to an input with 64 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 130
B. 129
C. 4
D. 128

Answer: A. 130

Working:

- Weights per filter = 1×1×64 = 64.
- +1 bias → 65 per filter.
- × 2 filters = 130.

Option notes:

- B: One bias per FILTER, not one in total.
- C: Forgot that each filter spans ALL input channels (× Cin).
- D: Forgot the biases.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B21. [MCQ · Medium · calculation]

A convolutional layer applies 128 filters of size 3×3 to an input with 16 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 18432
B. 18560
C. 1280
D. 18433

Answer: B. 18560

Working:

- Weights per filter = 3×3×16 = 144.
- +1 bias → 145 per filter.
- × 128 filters = 18560.

Option notes:

- A: Forgot the biases.
- C: Forgot that each filter spans ALL input channels (× Cin).
- D: One bias per FILTER, not one in total.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B22. [MCQ · Medium · calculation]

A convolutional layer applies 2 filters of size 5×5 to an input with 16 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 801
B. 52
C. 802
D. 800

Answer: C. 802

Working:

- Weights per filter = 5×5×16 = 400.
- +1 bias → 401 per filter.
- × 2 filters = 802.

Option notes:

- A: One bias per FILTER, not one in total.
- B: Forgot that each filter spans ALL input channels (× Cin).
- D: Forgot the biases.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B23. [MCQ · Medium · calculation]

A convolutional layer applies 8 filters of size 7×7 to an input with 64 channel(s), with no bias. How many trainable parameters? (Image size does not matter.)

A. 25088
B. 25096
C. 28851
D. 392

Answer: A. 25088

Working:

- Weights per filter = 7×7×64 = 3136.
- No bias.
- × 8 filters = 25088.

Option notes:

- B: No bias was specified, so don't add one.
- C: Arithmetic slip: recompute each step carefully.
- D: Forgot that each filter spans ALL input channels (× Cin).

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B24. [MCQ · Medium · calculation]

A convolutional layer applies 128 filters of size 5×5 to an input with 1 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 3200
B. 3201
C. 3328
D. 3827

Answer: C. 3328

Working:

- Weights per filter = 5×5×1 = 25.
- +1 bias → 26 per filter.
- × 128 filters = 3328.

Option notes:

- A: Forgot the biases.
- B: One bias per FILTER, not one in total.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B25. [MCQ · Medium · calculation]

A convolutional layer applies 16 filters of size 5×5 to an input with 64 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 25616
B. 25601
C. 416
D. 25600

Answer: A. 25616

Working:

- Weights per filter = 5×5×64 = 1600.
- +1 bias → 1601 per filter.
- × 16 filters = 25616.

Option notes:

- B: One bias per FILTER, not one in total.
- C: Forgot that each filter spans ALL input channels (× Cin).
- D: Forgot the biases.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B26. [MCQ · Medium · calculation]

A convolutional layer applies 128 filters of size 3×3 to an input with 1 channel(s), with no bias. How many trainable parameters? (Image size does not matter.)

A. 979
B. 1280
C. 1152
D. 1325

Answer: C. 1152

Working:

- Weights per filter = 3×3×1 = 9.
- No bias.
- × 128 filters = 1152.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: No bias was specified, so don't add one.

- D: Arithmetic slip: recompute each step carefully.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B27. [MCQ · Medium · calculation]

A convolutional layer applies 2 filters of size 7×7 to an input with 64 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 100
B. 6272
C. 6273
D. 6274

Answer: D. 6274

Working:

- Weights per filter = 7×7×64 = 3136.
- +1 bias → 3137 per filter.
- × 2 filters = 6274.

Option notes:

- A: Forgot that each filter spans ALL input channels (× Cin).
- B: Forgot the biases.
- C: One bias per FILTER, not one in total.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B28. [MCQ · Medium · calculation]

A convolutional layer applies 8 filters of size 7×7 to an input with 16 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 400
B. 6280
C. 6272
D. 6273

Answer: B. 6280

Working:

- Weights per filter = 7×7×16 = 784.
- +1 bias → 785 per filter.
- × 8 filters = 6280.

Option notes:

- A: Forgot that each filter spans ALL input channels (× Cin).
- C: Forgot the biases.
- D: One bias per FILTER, not one in total.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B29. [MCQ · Medium · calculation]

A convolutional layer applies 2 filters of size 1×1 to an input with 32 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 66
B. 4
C. 65
D. 64

Answer: A. 66

Working:

- Weights per filter = 1×1×32 = 32.
- +1 bias → 33 per filter.
- × 2 filters = 66.

Option notes:

- B: Forgot that each filter spans ALL input channels (× Cin).
- C: One bias per FILTER, not one in total.
- D: Forgot the biases.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B30. [MCQ · Medium · calculation]

A convolutional layer applies 32 filters of size 5×5 to an input with 32 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 25632
B. 25601
C. 25600
D. 832

Answer: A. 25632

Working:

- Weights per filter = 5×5×32 = 800.
- +1 bias → 801 per filter.
- × 32 filters = 25632.

Option notes:

- B: One bias per FILTER, not one in total.
- C: Forgot the biases.
- D: Forgot that each filter spans ALL input channels (× Cin).

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B31. [MCQ · Medium · calculation]

A convolutional layer applies 16 filters of size 1×1 to an input with 1 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 16
B. 17

C. 39
D. 32

Answer: D. 32

Working:

- Weights per filter = 1×1×1 = 1.
- +1 bias → 2 per filter.
- × 16 filters = 32.

Option notes:

- A: Forgot the biases.
- B: One bias per FILTER, not one in total.
- C: Arithmetic slip: recompute each step carefully.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B32. [MCQ · Medium · calculation]

A convolutional layer applies 16 filters of size 5×5 to an input with 64 channel(s), one bias per filter. How many trainable parameters? (Image size does not matter.)

A. 25616
B. 416
C. 25601
D. 25600

Answer: A. 25616

Working:

- Weights per filter = 5×5×64 = 1600.
- +1 bias → 1601 per filter.
- × 16 filters = 25616.

Option notes:

- B: Forgot that each filter spans ALL input channels (× Cin).
- C: One bias per FILTER, not one in total.
- D: Forgot the biases.

Takeaway: Params = (K·K·C_in + 1)·C_out. Weight sharing: image size never appears.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B33. [BLANKS · Medium · calculation]

Input 64×64×1. Conv layer: 8 filters, 5×5, stride 1, padding 2, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 64; parameters = 208; channels = 8

Working:

- Size = ⌊(64 + 4 − 5)/1⌋ + 1 = 64.
- Params = (5×5×1 + 1)×8 = 208.
- Channels = number of filters = 8. Output volume 64×64×8.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B34. [BLANKS · Medium · calculation]

Input 64×64×3. Conv layer: 8 filters, 3×3, stride 2, padding 0, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 31; parameters = 224; channels = 8

Working:

- Size = $\lfloor (64 + 0 - 3)/2 \rfloor + 1 = 31$.
- Params = $(3 \times 3 \times 3 + 1) \times 8 = 224$.
- Channels = number of filters = 8. Output volume 31×31×8.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B35. [BLANKS · Medium · calculation]

Input 64×64×1. Conv layer: 32 filters, 3×3, stride 2, padding 0, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 31; parameters = 320; channels = 32

Working:

- Size = $\lfloor (64 + 0 - 3)/2 \rfloor + 1 = 31$.
- Params = $(3 \times 3 \times 1 + 1) \times 32 = 320$.
- Channels = number of filters = 32. Output volume 31×31×32.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B36. [BLANKS · Medium · calculation]

Input 28×28×1. Conv layer: 16 filters, 5×5, stride 2, padding 1, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 13; parameters = 416; channels = 16

Working:

- Size = $\lfloor (28 + 2 - 5)/2 \rfloor + 1 = 13$.
- Params = $(5 \times 5 \times 1 + 1) \times 16 = 416$.
- Channels = number of filters = 16. Output volume 13×13×16.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B37. [BLANKS · Medium · calculation]

Input 28×28×1. Conv layer: 8 filters, 3×3, stride 2, padding 2, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 15; parameters = 80; channels = 8

Working:

- Size = $\lfloor (28 + 4 - 3)/2 \rfloor + 1 = 15$.
- Params = $(3\times3\times1 + 1)\times8 = 80$.
- Channels = number of filters = 8. Output volume 15×15×8.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B38. [BLANKS · Medium · calculation]

Input 28×28×3. Conv layer: 16 filters, 5×5, stride 1, padding 2, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 28; parameters = 1216; channels = 16

Working:

- Size = $\lfloor (28 + 4 - 5)/1 \rfloor + 1 = 28$.
- Params = $(5\times5\times3 + 1)\times16 = 1216$.
- Channels = number of filters = 16. Output volume 28×28×16.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B39. [BLANKS · Medium · calculation]

Input 224×224×3. Conv layer: 8 filters, 3×3, stride 2, padding 0, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 111; parameters = 224; channels = 8

Working:

- Size = $\lfloor (224 + 0 - 3)/2 \rfloor + 1 = 111$.
- Params = $(3\times3\times3 + 1)\times8 = 224$.
- Channels = number of filters = 8. Output volume 111×111×8.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B40. [BLANKS · Medium · calculation]

Input 28×28×3. Conv layer: 32 filters, 5×5, stride 1, padding 2, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 28; parameters = 2432; channels = 32

Working:

- Size = $\lfloor (28 + 4 - 5)/1 \rfloor + 1 = 28$.
- Params = $(5\times5\times3 + 1)\times32 = 2432$.
- Channels = number of filters = 32. Output volume 28×28×32.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B41. [BLANKS · Medium · calculation]

Input 32×32×1. Conv layer: 32 filters, 5×5, stride 1, padding 0, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 28; parameters = 832; channels = 32

Working:

- Size = $\lfloor (32 + 0 - 5)/1 \rfloor + 1 = 28$.
- Params = $(5\times5\times1 + 1)\times32 = 832$.
- Channels = number of filters = 32. Output volume 28×28×32.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B42. [BLANKS · Medium · calculation]

Input 28×28×3. Conv layer: 8 filters, 5×5, stride 1, padding 2, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 28; parameters = 608; channels = 8

Working:

- Size = $\lfloor (28 + 4 - 5)/1 \rfloor + 1 = 28$.
- Params = $(5\times5\times3 + 1)\times8 = 608$.
- Channels = number of filters = 8. Output volume 28×28×8.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B43. [BLANKS · Medium · calculation]

Input 32×32×3. Conv layer: 16 filters, 3×3, stride 1, padding 0, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 30; parameters = 448; channels = 16

Working:

- Size = $\lfloor (32 + 0 - 3)/1 \rfloor + 1 = 30$.
- Params = $(3\times3\times3 + 1)\times16 = 448$.
- Channels = number of filters = 16. Output volume 30×30×16.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B44. [BLANKS · Medium · calculation]

Input 32×32×1. Conv layer: 32 filters, 5×5, stride 2, padding 1, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 15; parameters = 832; channels = 32

Working:

- Size = $\lfloor (32 + 2 - 5)/2 \rfloor + 1 = 15$.
- Params = $(5\times5\times1 + 1)\times32 = 832$.
- Channels = number of filters = 32. Output volume 15×15×32.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B45. [BLANKS · Medium · calculation]

Input 32×32×3. Conv layer: 16 filters, 5×5, stride 1, padding 2, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 32; parameters = 1216; channels = 16

Working:

- Size = $\lfloor (32 + 4 - 5)/1 \rfloor + 1 = 32$.
- Params = $(5\times5\times3 + 1)\times16 = 1216$.
- Channels = number of filters = 16. Output volume 32×32×16.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B46. [BLANKS · Medium · calculation]

Input 32×32×1. Conv layer: 32 filters, 3×3, stride 1, padding 1, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 32; parameters = 320; channels = 32

Working:

- Size = $\lfloor (32 + 2 - 3)/1 \rfloor + 1 = 32$.
- Params = $(3\times3\times1 + 1)\times32 = 320$.
- Channels = number of filters = 32. Output volume 32×32×32.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B47. [BLANKS · Medium · calculation]

Input 64×64×3. Conv layer: 8 filters, 3×3, stride 2, padding 0, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 31; parameters = 224; channels = 8

Working:

- Size = $\lfloor (64 + 0 - 3)/2 \rfloor + 1 = 31$.
- Params = $(3\times3\times3 + 1)\times8 = 224$.
- Channels = number of filters = 8. Output volume 31×31×8.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B48. [BLANKS · Medium · calculation]

Input 28×28×1. Conv layer: 16 filters, 3×3, stride 1, padding 1, one bias per filter. Give output height/width, number of parameters, number of output channels.

(1) output size = ____
(2) parameters = ____
(3) channels = ____

Answer: output size = 28; parameters = 160; channels = 16

Working:

- Size = $\lfloor (28 + 2 - 3)/1 \rfloor + 1 = 28$.
- Params = $(3\times3\times1 + 1)\times16 = 160$.
- Channels = number of filters = 16. Output volume 28×28×16.

Takeaway: Output depth = number of filters (e.g. 64×64×3 with 8 filters of 3×3 → 62×62×8).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B49. [BLANKS · Hard · calculation]

2 | 3 | 2 | 1
3 | 3 | 1 | 0
3 | 1 | 0 | 1
0 | 1 | 1 | 0

```
0 | 1 | -1
-1 | 1 | -1
-1 | -1 | -1
```

Convolve the input with the 3×3 kernel (stride 1, no padding, no kernel flip, as in Lecture 7). Give the 2×2 output in row order.

(1) out[0,0] = ____
(2) out[0,1] = ____
(3) out[1,0] = ____
(4) out[1,1] = ____

Answer: out[0,0] = -4; out[0,1] = -3; out[1,0] = -2; out[1,1] = -3

Working:

- out[0,0] = 2×0 + 3×1 + 2×-1 + 3×-1 + 3×1 + 1×-1 + 3×-1 + 1×-1 + 0×-1 = -4
- out[0,1] = 3×0 + 2×1 + 1×-1 + 3×-1 + 1×1 + 0×-1 + 1×-1 + 0×-1 + 1×-1 = -3
- out[1,0] = 3×0 + 3×1 + 1×-1 + 3×-1 + 1×1 + 0×-1 + 0×-1 + 1×-1 + 1×-1 = -2
- out[1,1] = 3×0 + 1×1 + 0×-1 + 1×-1 + 0×1 + 1×-1 + 1×-1 + 1×-1 + 0×-1 = -3

Takeaway: Slide, multiply element-wise, sum. Output size = (N − K)/S + 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B50. [BLANKS · Hard · calculation]

1 | 1 | 0 | 3
3 | 0 | 3 | 1
3 | 2 | 0 | 0
3 | 1 | 0 | 2

```
-1 | 0 | 1
1 | -1 | 0
-1 | -1 | 0
```

Convolve the input with the 3×3 kernel (stride 1, no padding, no kernel flip, as in Lecture 7). Give the 2×2 output in row order.

(1) out[0,0] = ____
(2) out[0,1] = ____
(3) out[1,0] = ____
(4) out[1,1] = ____

Answer: out[0,0] = -3; out[0,1] = -3; out[1,0] = -3; out[1,1] = 2

Working:

- out[0,0] = 1×-1 + 1×0 + 0×1 + 3×1 + 0×-1 + 3×0 + 3×-1 + 2×-1 + 0×0 = -3
- out[0,1] = 1×-1 + 0×0 + 3×1 + 0×1 + 3×-1 + 1×0 + 2×-1 + 0×-1 + 0×0 = -3
- out[1,0] = 3×-1 + 0×0 + 3×1 + 3×1 + 2×-1 + 0×0 + 3×-1 + 1×-1 + 0×0 = -3
- out[1,1] = 0×-1 + 3×0 + 1×1 + 2×1 + 0×-1 + 0×0 + 1×-1 + 0×-1 + 2×0 = 2

Takeaway: Slide, multiply element-wise, sum. Output size = (N − K)/S + 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B51. [BLANK · Medium · calculation]

1 | 0 | 3 | 1 | 2
3 | 2 | 1 | 1 | 3
2 | 3 | 3 | 3 | 2
0 | 3 | 1 | 0 | 3
1 | 1 | 0 | 1 | 0

```
1 | 1 | 0
0 | 1 | 1
-1 | 0 | -1
```

Convolve the 5×5 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[2,2] (0-indexed; the highlighted window)?

Answer: 9

Working:

- Window starts at input row 2, col 2.
- Sum = 3×1 + 3×1 + 2×0 + 1×0 + 0×1 + 3×1 + 0×-1 + 1×0 + 0×-1 = 9.
- (Full output is 3×3.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B52. [BLANK · Medium · calculation]

2 | 1 | 0 | 2
0 | 1 | 1 | 2
1 | 3 | 3 | 2
2 | 1 | 2 | 2

```
-1 | -1 | -1
0 | -1 | 0
0 | -1 | 1
```

Convolve the 4×4 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[1,0] (0-indexed; the highlighted window)?

Answer: -4

Working:

- Window starts at input row 1, col 0.
- Sum = 0×-1 + 1×-1 + 1×-1 + 1×0 + 3×-1 + 3×0 + 2×0 + 1×-1 + 2×1 = -4.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B53. [BLANK · Medium · calculation]

1 | 2 | 0 | 2
2 | 0 | 0 | 1
1 | 1 | 3 | 2
3 | 0 | 3 | 1

```
1 | 0 | -1
-1 | 0 | 0
-1 | 0 | 1
```

Convolve the 4×4 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[1,1] (0-indexed; the highlighted window)?

Answer: -1

Working:

- Window starts at input row 1, col 1.
- Sum = 0×1 + 0×0 + 1×-1 + 1×-1 + 3×0 + 2×0 + 0×-1 + 3×0 + 1×1 = -1.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B54. [BLANK · Medium · calculation]

1 | 1 | 2 | 2 | 0
2 | 3 | 3 | 1 | 1
0 | 0 | 2 | 3 | 0
1 | 3 | 0 | 0 | 0
0 | 0 | 2 | 0 | 3

```
1 | 0 | -1
0 | 1 | -1
-1 | -1 | 0
```

Convolve the 5×5 input with the 3×3 kernel (stride 2, no padding, no flip). What is output[1,1] (0-indexed; the highlighted window)?

Answer: 0

Working:

- Window starts at input row 2, col 2.
- Sum = 2×1 + 3×0 + 0×-1 + 0×0 + 0×1 + 0×-1 + 2×-1 + 0×-1 + 3×0 = 0.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B55. [BLANK · Medium · calculation]

1 | 3 | 1 | 1
0 | 1 | 2 | 0
2 | 3 | 3 | 3
1 | 1 | 3 | 0

```
1 | -1 | 0
0 | -1 | -1
0 | -1 | 0
```

Convolve the 4×4 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[0,1] (0-indexed; the highlighted window)?

Answer: -3

Working:

- Window starts at input row 0, col 1.
- Sum = 3×1 + 1×-1 + 1×0 + 1×0 + 2×-1 + 0×-1 + 3×0 + 3×-1 + 3×0 = -3.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B56. [BLANK · Medium · calculation]

2 | 0 | 2 | 3
2 | 0 | 2 | 3
3 | 2 | 2 | 3
1 | 0 | 0 | 3

```
1 | 0 | 1
-1 | -1 | -1
0 | -1 | 0
```

Convolve the 4×4 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[1,0] (0-indexed; the highlighted window)?

Answer: -3

Working:

- Window starts at input row 1, col 0.
- Sum = 2×1 + 0×0 + 2×1 + 3×-1 + 2×-1 + 2×-1 + 1×0 + 0×-1 + 0×0 = -3.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B57. [BLANK · Medium · calculation]

1 | 3 | 0 | 0
2 | 0 | 1 | 3
2 | 3 | 2 | 0
0 | 0 | 3 | 0

```
0 | 0 | 1
0 | 1 | 1
0 | -1 | 0
```

Convolve the 4×4 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[1,0] (0-indexed; the highlighted window)?

Answer: 6

Working:

- Window starts at input row 1, col 0.
- Sum = 2×0 + 0×0 + 1×1 + 2×0 + 3×1 + 2×1 + 0×0 + 0×-1 + 3×0 = 6.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B58. [BLANK · Medium · calculation]

3 | 2 | 1 | 2 | 2
3 | 1 | 3 | 0 | 1
3 | 3 | 2 | 3 | 2
2 | 2 | 1 | 1 | 3
1 | 3 | 0 | 1 | 3

```
-1 | -1 | -1
1 | 0 | 0
1 | -1 | 0
```

Convolve the 5×5 input with the 3×3 kernel (stride 2, no padding, no flip). What is output[0,0] (0-indexed; the highlighted window)?

Answer: -3

Working:

- Window starts at input row 0, col 0.
- Sum = 3×-1 + 2×-1 + 1×-1 + 3×1 + 1×0 + 3×0 + 3×1 + 3×-1 + 2×0 = -3.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B59. [BLANKS · Hard · calculation]

0 | 2 | 2 | 2
1 | 1 | 0 | 2
0 | 2 | 0 | 0
3 | 2 | 0 | 3

```
0 | -1 | 1
1 | -1 | -1
0 | 0 | 1
```

Convolve the input with the 3×3 kernel (stride 1, no padding, no kernel flip, as in Lecture 7). Give the 2×2 output in row order.

(1) out[0,0] = ____
(2) out[0,1] = ____
(3) out[1,0] = ____
(4) out[1,1] = ____

Answer: out[0,0] = 0; out[0,1] = -1; out[1,0] = -3; out[1,1] = 7

Working:

- out[0,0] = 0×0 + 2×-1 + 2×1 + 1×1 + 1×-1 + 0×-1 + 0×0 + 2×0 + 0×1 = 0

- out[0,1] = 2×0 + 2×-1 + 2×1 + 1×1 + 0×-1 + 2×-1 + 2×0 + 0×0 + 0×1 = -1
- out[1,0] = 1×0 + 1×-1 + 0×1 + 0×1 + 2×-1 + 0×-1 + 3×0 + 2×0 + 0×1 = -3
- out[1,1] = 1×0 + 0×-1 + 2×1 + 2×1 + 0×-1 + 0×-1 + 2×0 + 0×0 + 3×1 = 7

Takeaway: Slide, multiply element-wise, sum. Output size = (N − K)/S + 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B60. [BLANK · Medium · calculation]

1 | 1 | 0 | 1 | 3
3 | 0 | 3 | 2 | 2
0 | 3 | 1 | 2 | 0
0 | 3 | 1 | 1 | 0
0 | 2 | 1 | 0 | 3

```
0 | -1 | 0
0 | 0 | 0
-1 | -1 | 1
```

Convolve the 5×5 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[2,1] (0-indexed; the highlighted window)?

Answer: -4

Working:

- Window starts at input row 2, col 1.
- Sum = 3×0 + 1×-1 + 2×0 + 3×0 + 1×0 + 1×0 + 2×-1 + 1×-1 + 0×1 = -4.
- (Full output is 3×3.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B61. [BLANK · Medium · calculation]

3 | 1 | 0 | 1
3 | 2 | 1 | 0
2 | 1 | 2 | 2
3 | 0 | 3 | 2

```
-1 | 1 | 0
0 | 1 | -1
-1 | -1 | 0
```

Convolve the 4×4 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[1,1] (0-indexed; the highlighted window)?

Answer: -4

Working:

- Window starts at input row 1, col 1.
- Sum = 2×-1 + 1×1 + 0×0 + 1×0 + 2×1 + 2×-1 + 0×-1 + 3×-1 + 2×0 = -4.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B62. [BLANK · Medium · calculation]

1 | 0 | 1 | 0
3 | 0 | 3 | 0
1 | 0 | 3 | 2
3 | 0 | 2 | 3

```
1 | -1 | -1
1 | -1 | 1
-1 | 0 | 0
```

Convolve the 4×4 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[1,0] (0-indexed; the highlighted window)?

Answer: 1

Working:

- Window starts at input row 1, col 0.
- Sum = 3×1 + 0×-1 + 3×-1 + 1×1 + 0×-1 + 3×1 + 3×-1 + 0×0 + 2×0 = 1.
- (Full output is 2×2.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B63. [BLANKS · Hard · calculation]

0 | 0 | 2 | 3 | 0
2 | 0 | 2 | 2 | 0
3 | 0 | 3 | 2 | 3
2 | 3 | 3 | 2 | 0
2 | 2 | 0 | 2 | 1

```
0 | -1 | 1
-1 | 0 | -1
0 | 0 | 1
```

Convolve the input with the 3×3 kernel (stride 2, no padding, no kernel flip, as in Lecture 7). Give the 2×2 output in row order.

(1) out[0,0] = ____
(2) out[0,1] = ____
(3) out[1,0] = ____
(4) out[1,1] = ____

Answer: out[0,0] = 1; out[0,1] = -2; out[1,0] = -2; out[1,1] = -1

Working:

- out[0,0] = 0×0 + 0×-1 + 2×1 + 2×-1 + 0×0 + 2×-1 + 3×0 + 0×0 + 3×1 = 1
- out[0,1] = 2×0 + 3×-1 + 0×1 + 2×-1 + 2×0 + 0×-1 + 3×0 + 2×0 + 3×1 = -2
- out[1,0] = 3×0 + 0×-1 + 3×1 + 2×-1 + 3×0 + 3×-1 + 2×0 + 2×0 + 0×1 = -2
- out[1,1] = 3×0 + 2×-1 + 3×1 + 3×-1 + 2×0 + 0×-1 + 0×0 + 2×0 + 1×1 = -1

Takeaway: Slide, multiply element-wise, sum. Output size = (N − K)/S + 1.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B64. [BLANK · Medium · calculation]

2 | 0 | 2 | 3 | 2
0 | 3 | 3 | 2 | 2
0 | 2 | 2 | 2 | 1
2 | 3 | 3 | 0 | 2
2 | 1 | 0 | 0 | 0

```
0 | -1 | -1
0 | 1 | -1
1 | -1 | 1
```

Convolve the 5×5 input with the 3×3 kernel (stride 1, no padding, no flip). What is output[2,0] (0-indexed; the highlighted window)?

Answer: -3

Working:

- Window starts at input row 2, col 0.
- Sum = 0×0 + 2×-1 + 2×-1 + 2×0 + 3×1 + 3×-1 + 2×1 + 1×-1 + 0×1 = -3.
- (Full output is 3×3.)

Takeaway: Each output value = sum of (window ⊙ kernel), plus bias if given.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B65. [BLANKS · Easy · calculation]

4 | 5 | 9 | 7
4 | 7 | 8 | 3
1 | 8 | 3 | 0
4 | 2 | 4 | 2

Apply 2×2 average pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 5; [0,1] = 6.75; [1,0] = 3.75; [1,1] = 2.25

Working:

- mean(4, 5, 4, 7) = 5
- mean(9, 7, 8, 3) = 6.75
- mean(1, 8, 4, 2) = 3.75
- mean(3, 0, 4, 2) = 2.25

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B66. [BLANKS · Easy · calculation]

0 | 4 | 3 | 1
8 | 8 | 0 | 9

4 | 7 | 6 | 1
1 | 8 | 3 | 1

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 8; [0,1] = 9; [1,0] = 8; [1,1] = 6

Working:

- max(0, 4, 8, 8) = 8
- max(3, 1, 0, 9) = 9
- max(4, 7, 1, 8) = 8
- max(6, 1, 3, 1) = 6

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B67. [BLANKS · Easy · calculation]

6 | 4 | 0 | 7
4 | 6 | 9 | 5
3 | 2 | 7 | 6
9 | 9 | 9 | 5

Apply 2×2 average pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 5; [0,1] = 5.25; [1,0] = 5.75; [1,1] = 6.75

Working:

- mean(6, 4, 4, 6) = 5
- mean(0, 7, 9, 5) = 5.25
- mean(3, 2, 9, 9) = 5.75
- mean(7, 6, 9, 5) = 6.75

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B68. [BLANKS · Easy · calculation]

1 | 5 | 3 | 1
5 | 0 | 2 | 2
6 | 4 | 9 | 8
6 | 5 | 3 | 6

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 5; [0,1] = 3; [1,0] = 6; [1,1] = 9

Working:

- max(1, 5, 5, 0) = 5
- max(3, 1, 2, 2) = 3
- max(6, 4, 6, 5) = 6
- max(9, 8, 3, 6) = 9

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B69. [BLANKS · Easy · calculation]

3 | 3 | 6 | 1
6 | 6 | 1 | 0
4 | 4 | 4 | 7
5 | 9 | 2 | 8

Apply 2×2 average pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 4.5; [0,1] = 2; [1,0] = 5.5; [1,1] = 5.25

Working:

- mean(3, 3, 6, 6) = 4.5
- mean(6, 1, 1, 0) = 2
- mean(4, 4, 5, 9) = 5.5
- mean(4, 7, 2, 8) = 5.25

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B70. [BLANKS · Easy · calculation]

9 | 2 | 4 | 5
5 | 2 | 5 | 9
7 | 4 | 3 | 1
0 | 5 | 8 | 0

Apply 2×2 average pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 4.5; [0,1] = 5.75; [1,0] = 4; [1,1] = 3

Working:

- mean(9, 2, 5, 2) = 4.5
- mean(4, 5, 5, 9) = 5.75
- mean(7, 4, 0, 5) = 4
- mean(3, 1, 8, 0) = 3

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B71. [BLANKS · Easy · calculation]

2 | 4 | 7 | 4
3 | 0 | 3 | 6
1 | 6 | 8 | 7
9 | 3 | 2 | 7

Apply 2×2 average pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 2.25; [0,1] = 5; [1,0] = 4.75; [1,1] = 6

Working:

- mean(2, 4, 3, 0) = 2.25
- mean(7, 4, 3, 6) = 5
- mean(1, 6, 9, 3) = 4.75
- mean(8, 7, 2, 7) = 6

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B72. [BLANKS · Easy · calculation]

1 | 5 | 0 | 6
8 | 5 | 2 | 7
8 | 9 | 6 | 6
8 | 4 | 0 | 1

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 8; [0,1] = 7; [1,0] = 9; [1,1] = 6

Working:

- max(1, 5, 8, 5) = 8
- max(0, 6, 2, 7) = 7
- max(8, 9, 8, 4) = 9
- max(6, 6, 0, 1) = 6

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B73. [BLANKS · Easy · calculation]

1 | 2 | 9 | 1
0 | 5 | 2 | 4
9 | 7 | 7 | 6
1 | 1 | 0 | 9

Apply 2×2 average pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 2; [0,1] = 4; [1,0] = 4.5; [1,1] = 5.5

Working:

- mean(1, 2, 0, 5) = 2
- mean(9, 1, 2, 4) = 4
- mean(9, 7, 1, 1) = 4.5
- mean(7, 6, 0, 9) = 5.5

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B74. [BLANKS · Easy · calculation]

5 | 7 | 5 | 2
5 | 5 | 9 | 4
8 | 0 | 4 | 8
8 | 7 | 9 | 5

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 7; [0,1] = 9; [1,0] = 8; [1,1] = 9

Working:

- max(5, 7, 5, 5) = 7
- max(5, 2, 9, 4) = 9
- max(8, 0, 8, 7) = 8
- max(4, 8, 9, 5) = 9

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B75. [BLANKS · Easy · calculation]

1 | 0 | 6 | 6
7 | 2 | 3 | 0
6 | 1 | 6 | 0
0 | 9 | 5 | 1

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 7; [0,1] = 6; [1,0] = 9; [1,1] = 6

Working:

- max(1, 0, 7, 2) = 7
- max(6, 6, 3, 0) = 6
- max(6, 1, 0, 9) = 9
- max(6, 0, 5, 1) = 6

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B76. [BLANKS · Easy · calculation]

7 | 2 | 4 | 0
4 | 9 | 7 | 1
7 | 5 | 7 | 1
8 | 4 | 6 | 2

Apply 2×2 average pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 5.5; [0,1] = 3; [1,0] = 6; [1,1] = 4

Working:

- mean(7, 2, 4, 9) = 5.5
- mean(4, 0, 7, 1) = 3
- mean(7, 5, 8, 4) = 6
- mean(7, 1, 6, 2) = 4

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B77. [BLANKS · Easy · calculation]

3 | 8 | 3 | 0
4 | 9 | 6 | 3

1 | 5 | 4 | 5
6 | 8 | 1 | 9

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 9; [0,1] = 6; [1,0] = 8; [1,1] = 9

Working:

- max(3, 8, 4, 9) = 9
- max(3, 0, 6, 3) = 6
- max(1, 5, 6, 8) = 8
- max(4, 5, 1, 9) = 9

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B78. [BLANKS · Easy · calculation]

1 | 3 | 0 | 4
1 | 8 | 1 | 8
0 | 4 | 0 | 9
5 | 9 | 0 | 6

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 8; [0,1] = 8; [1,0] = 9; [1,1] = 9

Working:

- max(1, 3, 1, 8) = 8
- max(0, 4, 1, 8) = 8
- max(0, 4, 5, 9) = 9
- max(0, 9, 0, 6) = 9

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B79. [BLANKS · Easy · calculation]

7 | 2 | 0 | 6
7 | 0 | 6 | 1
6 | 7 | 0 | 7
0 | 9 | 6 | 9

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 7; [0,1] = 6; [1,0] = 9; [1,1] = 9

Working:

- max(7, 2, 7, 0) = 7
- max(0, 6, 6, 1) = 6
- max(6, 7, 0, 9) = 9
- max(0, 7, 6, 9) = 9

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B80. [BLANKS · Easy · calculation]

9 | 7 | 1 | 5
9 | 6 | 1 | 7
8 | 5 | 5 | 0
5 | 6 | 6 | 2

Apply 2×2 max pooling with stride 2. Give the 2×2 output in row order.

(1) [0,0] = ____
(2) [0,1] = ____
(3) [1,0] = ____
(4) [1,1] = ____

Answer: [0,0] = 9; [0,1] = 7; [1,0] = 8; [1,1] = 6

Working:

- max(9, 7, 9, 6) = 9
- max(1, 5, 1, 7) = 7
- max(8, 5, 5, 6) = 8
- max(5, 0, 6, 2) = 6

Takeaway: Pooling has NO learnable parameters and keeps the number of channels; it halves H and W here.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B81. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 7; flatten length = 288; total params = 5306

Working:

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$ → 8×14×14.

- Pool: ⌊14/2⌋ = 7.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B82. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 14; flatten length = 1568; total params = 18106

Working:

- Conv1: ⌊(28 + 2 − 3)/1⌋ + 1 = 28 → 8×28×28.
- Pool: ⌊28/2⌋ = 14.
- Conv2 (padding 1, stride 1): 14; Pool: 7 → 32×7×7.
- Flatten: 32×7×7 = 1568.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (1568+1)×10 = 15690; total 18106.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B83. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 16; flatten length = 1024; total params = 11498

Working:

- Conv1: ⌊(64 + 2 − 3)/2⌋ + 1 = 32 → 8×32×32.
- Pool: ⌊32/2⌋ = 16.
- Conv2 (padding 1, stride 1): 16; Pool: 8 → 16×8×8.
- Flatten: 16×8×8 = 1024.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×16 = 1168; linear (1024+1)×10 = 10250; total 11498.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B84. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 8, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 32; flatten length = 8192; total params = 84346

Working:

- Conv1: $\lfloor (64 + 2 - 3)/1 \rfloor + 1 = 64$ → 8×64×64.
- Pool: $\lfloor 64/2 \rfloor = 32$.
- Conv2 (padding 1, stride 1): 32; Pool: 16 → 32×16×16.
- Flatten: 32×16×16 = 8192.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (8192+1)×10 = 81930; total 84346.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B85. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 32; flatten length = 4096; total params = 43450

Working:

- Conv1: $\lfloor (64 + 2 - 3)/1 \rfloor + 1 = 64$ → 16×64×64.
- Pool: $\lfloor 64/2 \rfloor = 32$.
- Conv2 (padding 1, stride 1): 32; Pool: 16 → 16×16×16.
- Flatten: 16×16×16 = 4096.
- Params: conv1 (9×1+1)×16 = 160; conv2 (9×16+1)×16 = 2320; linear (4096+1)×10 = 40970; total 43450.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B86. [BLANKS · Exam+ · calculation]

Input 1×32×32. Conv2d(1, 16, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 16; flatten length = 1024; total params = 12730

Working:

- Conv1: ⌊(32 + 2 − 3)/1⌋ + 1 = 32 → 16×32×32.
- Pool: ⌊32/2⌋ = 16.
- Conv2 (padding 1, stride 1): 16; Pool: 8 → 16×8×8.
- Flatten: 16×8×8 = 1024.
- Params: conv1 (9×1+1)×16 = 160; conv2 (9×16+1)×16 = 2320; linear (1024+1)×10 = 10250; total 12730.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B87. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 7; flatten length = 288; total params = 5306

Working:

- Conv1: ⌊(28 + 2 − 3)/2⌋ + 1 = 14 → 8×14×14.
- Pool: ⌊14/2⌋ = 7.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B88. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 16; flatten length = 2048; total params = 25290

Working:

- Conv1: ⌊(64 + 2 − 3)/2⌋ + 1 = 32 → 16×32×32.
- Pool: ⌊32/2⌋ = 16.
- Conv2 (padding 1, stride 1): 16; Pool: 8 → 32×8×8.
- Flatten: 32×8×8 = 2048.
- Params: conv1 (9×1+1)×16 = 160; conv2 (9×16+1)×32 = 4640; linear (2048+1)×10 = 20490; total 25290.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B89. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 7; flatten length = 288; total params = 5306

Working:

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$ → 8×14×14.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B90. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 7; flatten length = 288; total params = 5306

Working:

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$ → 8×14×14.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B91. [BLANKS · Exam+ · calculation]

Input 1×32×32. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 8; flatten length = 256; total params = 3818

Working:

- Conv1: $\lfloor (32 + 2 - 3)/2 \rfloor + 1 = 16 \rightarrow 8 \times 16 \times 16$.
- Pool: $\lfloor 16/2 \rfloor = 8$.
- Conv2 (padding 1, stride 1): 8; Pool: $4 \rightarrow 16 \times 4 \times 4$.
- Flatten: $16 \times 4 \times 4 = 256$.
- Params: conv1 $(9 \times 1 + 1) \times 8 = 80$; conv2 $(9 \times 8 + 1) \times 16 = 1168$; linear $(256 + 1) \times 10 = 2570$; total 3818.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B92. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 7; flatten length = 288; total params = 5306

Working:

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14 \rightarrow 8 \times 14 \times 14$.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: $3 \rightarrow 32 \times 3 \times 3$.
- Flatten: $32 \times 3 \times 3 = 288$.
- Params: conv1 $(9 \times 1 + 1) \times 8 = 80$; conv2 $(9 \times 8 + 1) \times 32 = 2336$; linear $(288 + 1) \times 10 = 2890$; total 5306.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B93. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 16; flatten length = 1024; total params = 12730

Working:

- Conv1: $\lfloor (64 + 2 - 3)/2 \rfloor + 1 = 32 \rightarrow 16 \times 32 \times 32$.
- Pool: $\lfloor 32/2 \rfloor = 16$.
- Conv2 (padding 1, stride 1): 16; Pool: $8 \rightarrow 16 \times 8 \times 8$.
- Flatten: $16 \times 8 \times 8 = 1024$.
- Params: conv1 $(9 \times 1 + 1) \times 16 = 160$; conv2 $(9 \times 16 + 1) \times 16 = 2320$; linear $(1024 + 1) \times 10 = 10250$; total 12730.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B94. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 14; flatten length = 1568; total params = 18106

Working:

- Conv1: $\lfloor (28 + 2 - 3)/1 \rfloor + 1 = 28$ → 8×28×28.
- Pool: $\lfloor 28/2 \rfloor = 14$.
- Conv2 (padding 1, stride 1): 14; Pool: 7 → 32×7×7.
- Flatten: 32×7×7 = 1568.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (1568+1)×10 = 15690; total 18106.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B95. [BLANKS · Exam+ · calculation]

Input 1×28×28. Conv2d(1, 8, 3, stride=2, padding=1) → ReLU → MaxPool2d(2) → Conv2d(8, 32, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 7; flatten length = 288; total params = 5306

Working:

- Conv1: $\lfloor (28 + 2 - 3)/2 \rfloor + 1 = 14$ → 8×14×14.
- Pool: $\lfloor 14/2 \rfloor = 7$.
- Conv2 (padding 1, stride 1): 7; Pool: 3 → 32×3×3.
- Flatten: 32×3×3 = 288.
- Params: conv1 (9×1+1)×8 = 80; conv2 (9×8+1)×32 = 2336; linear (288+1)×10 = 2890; total 5306.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. 1×28×28 → 16×14×14 → … → 288 → 10).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B96. [BLANKS · Exam+ · calculation]

Input 1×64×64. Conv2d(1, 16, 3, stride=1, padding=1) → ReLU → MaxPool2d(2) → Conv2d(16, 16, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(·, 10). Give the spatial size after the first pool, the flattened length, and the total number of trainable parameters.

(1) size after pool 1 = ____
(2) flatten length = ____
(3) total params = ____

Answer: size after pool 1 = 32; flatten length = 4096; total params = 43450

Working:

- Conv1: $\lfloor (64 + 2 - 3)/1 \rfloor + 1 = 64 \rightarrow 16 \times 64 \times 64$.
- Pool: $\lfloor 64/2 \rfloor = 32$.
- Conv2 (padding 1, stride 1): 32; Pool: $16 \rightarrow 16 \times 16 \times 16$.
- Flatten: $16 \times 16 \times 16 = 4096$.
- Params: conv1 $(9 \times 1 + 1) \times 16 = 160$; conv2 $(9 \times 16 + 1) \times 16 = 2320$; linear $(4096 + 1) \times 10 = 40970$; total 43450.

Takeaway: Lab habit from Lecture 7: print x.shape after every block (e.g. $1 \times 28 \times 28 \rightarrow 16 \times 14 \times 14 \rightarrow \ldots \rightarrow 288 \rightarrow 10$).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B97. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 3×3, stride 2; L3: 5×5, stride 2; L4: 2×2, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use $RF_l = RF_{(l-1)} + (k_l - 1) \times$ (product of earlier strides), $RF_0 = 1$.

Answer: 17

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=3, s=2): RF = prev + (3 − 1)×1 = 5
- Layer 3 (k=5, s=2): RF = prev + (5 − 1)×2 = 13
- Layer 4 (k=2, s=1): RF = prev + (2 − 1)×4 = 17
- Receptive field = 17×17.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B98. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 3×3, stride 2. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use $RF_l = RF_{(l-1)} + (k_l - 1) \times$ (product of earlier strides), $RF_0 = 1$.

Answer: 5

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=3, s=2): RF = prev + (3 − 1)×1 = 5
- Receptive field = 5×5.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B99. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 2; L2: 3×3, stride 1; L3: 5×5, stride 1; L4: 3×3, stride 1. What is the receptive field (in

input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 19

Working:

- Layer 1 (k=3, s=2): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=3, s=1): RF = prev + (3 − 1)×2 = 7
- Layer 3 (k=5, s=1): RF = prev + (5 − 1)×2 = 15
- Layer 4 (k=3, s=1): RF = prev + (3 − 1)×2 = 19
- Receptive field = 19×19.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B100. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 2×2, stride 2; L3: 3×3, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 8

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=2, s=2): RF = prev + (2 − 1)×1 = 4
- Layer 3 (k=3, s=1): RF = prev + (3 − 1)×2 = 8
- Receptive field = 8×8.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B101. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 2; L2: 2×2, stride 2. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 5

Working:

- Layer 1 (k=3, s=2): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=2, s=2): RF = prev + (2 − 1)×2 = 5
- Receptive field = 5×5.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B102. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 3×3, stride 1; L3: 3×3, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 7

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=3, s=1): RF = prev + (3 − 1)×1 = 5
- Layer 3 (k=3, s=1): RF = prev + (3 − 1)×1 = 7
- Receptive field = 7×7.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B103. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 2×2, stride 2; L3: 2×2, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 6

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=2, s=2): RF = prev + (2 − 1)×1 = 4
- Layer 3 (k=2, s=1): RF = prev + (2 − 1)×2 = 6
- Receptive field = 6×6.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B104. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 2×2, stride 1; L3: 3×3, stride 2; L4: 2×2, stride 2. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 8

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=2, s=1): RF = prev + (2 − 1)×1 = 4
- Layer 3 (k=3, s=2): RF = prev + (3 − 1)×1 = 6
- Layer 4 (k=2, s=2): RF = prev + (2 − 1)×2 = 8
- Receptive field = 8×8.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B105. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 2; L2: 3×3, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 7

Working:

- Layer 1 (k=3, s=2): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=3, s=1): RF = prev + (3 − 1)×2 = 7
- Receptive field = 7×7.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B106. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 5×5, stride 1; L3: 3×3, stride 1; L4: 3×3, stride 2. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 11

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=5, s=1): RF = prev + (5 − 1)×1 = 7
- Layer 3 (k=3, s=1): RF = prev + (3 − 1)×1 = 9
- Layer 4 (k=3, s=2): RF = prev + (3 − 1)×1 = 11
- Receptive field = 11×11.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B107. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 5×5, stride 1; L2: 3×3, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 7

Working:

- Layer 1 (k=5, s=1): RF = prev + (5 − 1)×1 = 5
- Layer 2 (k=3, s=1): RF = prev + (3 − 1)×1 = 7
- Receptive field = 7×7.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B108. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 5×5, stride 1; L3: 5×5, stride 2; L4: 2×2, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 13

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=5, s=1): RF = prev + (5 − 1)×1 = 7
- Layer 3 (k=5, s=2): RF = prev + (5 − 1)×1 = 11
- Layer 4 (k=2, s=1): RF = prev + (2 − 1)×2 = 13
- Receptive field = 13×13.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B109. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 3×3, stride 2; L3: 5×5, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 13

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=3, s=2): RF = prev + (3 − 1)×1 = 5
- Layer 3 (k=5, s=1): RF = prev + (5 − 1)×2 = 13
- Receptive field = 13×13.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B110. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 2×2, stride 2; L2: 5×5, stride 1. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 10

Working:

- Layer 1 (k=2, s=2): RF = prev + (2 − 1)×1 = 2
- Layer 2 (k=5, s=1): RF = prev + (5 − 1)×2 = 10
- Receptive field = 10×10.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B111. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 3×3, stride 1; L3: 3×3, stride 2; L4: 3×3, stride 2. What is the receptive field (in input pixels, one side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 11

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=3, s=1): RF = prev + (3 − 1)×1 = 5
- Layer 3 (k=3, s=2): RF = prev + (3 − 1)×1 = 7
- Layer 4 (k=3, s=2): RF = prev + (3 − 1)×2 = 11
- Receptive field = 11×11.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B112. [BLANK · Hard · calculation]

(Extra, not in your slides: Lecture 7 only states that deeper layers have larger receptive fields.) Layers stacked: L1: 3×3, stride 1; L2: 2×2, stride 1; L3: 3×3, stride 1. What is the receptive field (in input pixels, one

side) of one unit in the last layer? Use RF_l = RF_(l−1) + (k_l − 1) × (product of earlier strides), RF_0 = 1.

Answer: 6

Working:

- Layer 1 (k=3, s=1): RF = prev + (3 − 1)×1 = 3
- Layer 2 (k=2, s=1): RF = prev + (2 − 1)×1 = 4
- Layer 3 (k=3, s=1): RF = prev + (3 − 1)×1 = 6
- Receptive field = 6×6.

Takeaway: Stacking small kernels grows the receptive field; strides multiply the growth of later layers.

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B113. [BLANKS · Easy · calculation]

An image 64×64×1 is flattened and fed to a dense layer of 100 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 4096; weights = 409600

Working:

- Inputs = 64×64×1 = 4096.
- Weights = 4096×100 = 409600.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B114. [BLANKS · Easy · calculation]

An image 32×32×1 is flattened and fed to a dense layer of 100 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 1024; weights = 102400

Working:

- Inputs = 32×32×1 = 1024.
- Weights = 1024×100 = 102400.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B115. [BLANKS · Easy · calculation]

An image 224×224×1 is flattened and fed to a dense layer of 1000 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 50176; weights = 50176000

Working:

- Inputs = 224×224×1 = 50176.
- Weights = 50176×1000 = 50176000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B116. [BLANKS · Easy · calculation]

An image 32×32×1 is flattened and fed to a dense layer of 500 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 1024; weights = 512000

Working:

- Inputs = 32×32×1 = 1024.
- Weights = 1024×500 = 512000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B117. [BLANKS · Easy · calculation]

An image 28×28×1 is flattened and fed to a dense layer of 1000 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 784; weights = 784000

Working:

- Inputs = 28×28×1 = 784.
- Weights = 784×1000 = 784000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B118. [BLANKS · Easy · calculation]

An image 64×64×1 is flattened and fed to a dense layer of 500 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 4096; weights = 2048000

Working:

- Inputs = 64×64×1 = 4096.
- Weights = 4096×500 = 2048000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B119. [BLANKS · Easy · calculation]

An image 64×64×1 is flattened and fed to a dense layer of 500 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 4096; weights = 2048000

Working:

- Inputs = 64×64×1 = 4096.
- Weights = 4096×500 = 2048000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B120. [BLANKS · Easy · calculation]

An image 224×224×1 is flattened and fed to a dense layer of 100 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 50176; weights = 5017600

Working:

- Inputs = 224×224×1 = 50176.
- Weights = 50176×100 = 5017600.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B121. [BLANKS · Easy · calculation]

An image 32×32×1 is flattened and fed to a dense layer of 1000 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 1024; weights = 1024000

Working:

- Inputs = 32×32×1 = 1024.
- Weights = 1024×1000 = 1024000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B122. [BLANKS · Easy · calculation]

An image 224×224×1 is flattened and fed to a dense layer of 100 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 50176; weights = 5017600

Working:

- Inputs = 224×224×1 = 50176.
- Weights = 50176×100 = 5017600.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B123. [BLANKS · Easy · calculation]

An image 32×32×3 is flattened and fed to a dense layer of 500 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 3072; weights = 1536000

Working:

- Inputs = 32×32×3 = 3072.
- Weights = 3072×500 = 1536000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B124. [BLANKS · Easy · calculation]

An image 224×224×1 is flattened and fed to a dense layer of 100 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 50176; weights = 5017600

Working:

- Inputs = 224×224×1 = 50176.

- Weights = 50176×100 = 5017600.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B125. [BLANKS · Easy · calculation]

An image 64×64×1 is flattened and fed to a dense layer of 500 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 4096; weights = 2048000

Working:

- Inputs = 64×64×1 = 4096.
- Weights = 4096×500 = 2048000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B126. [BLANKS · Easy · calculation]

An image 28×28×3 is flattened and fed to a dense layer of 1000 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 2352; weights = 2352000

Working:

- Inputs = 28×28×3 = 2352.
- Weights = 2352×1000 = 2352000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B127. [BLANKS · Easy · calculation]

An image 224×224×1 is flattened and fed to a dense layer of 500 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 50176; weights = 25088000

Working:

- Inputs = 224×224×1 = 50176.
- Weights = 50176×500 = 25088000.

- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

CNN B128. [BLANKS · Easy · calculation]

An image 32×32×1 is flattened and fed to a dense layer of 500 units (ignore biases). Give the number of inputs and the number of weights.

(1) inputs = ____
(2) weights = ____

Answer: inputs = 1024; weights = 512000

Working:

- Inputs = 32×32×1 = 1024.
- Weights = 1024×500 = 512000.
- (Lecture 7: 224×224×3 = 150,528 inputs; ×1,000 = 150,528,000 weights.)

Takeaway: Dense layers explode parameters and destroy 2D neighbourhoods; convolution fixes both (local connections + shared weights).

Source: Lecture 7 (Computer vision) + Key points about convolution filters and parameter counts

---

📦 Object Detection (IoU & NMS) (Lecture 7)

Part A: hand-written questions (55)

DETECT A1. [MCQ · Easy]

Which three things does each object-detection record contain?

A. Class, pixel mask and depth
B. Box and loss value
C. Only a class probability vector
D. Class, bounding box and confidence

Answer: D. Class, bounding box and confidence

Option notes:

- A: Masks are segmentation.
- B: Loss is for training.
- C: That's classification.

Takeaway: What, where, how sure.

Source: Lecture 7, slide 58

DETECT A2. [TF · Medium]

Object detection outputs a fixed number of boxes for every image.

Answer: False

Why: One image → a VARIABLE-length set of object records.

Takeaway: Count of objects varies.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A3. [MULTI · Medium]

Why is classifying many crops a poor detection strategy? (Select ALL that apply.)

A. Nearby crops overlap, so the CNN repeats the same computation
B. The number of candidate crops explodes (positions × scales × shapes)
C. It needs one CNN pass per crop
D. Crops can't be classified by a CNN

Answer: A. Nearby crops overlap, so the CNN repeats the same computation; B. The number of candidate crops explodes (positions × scales × shapes); C. It needs one CNN pass per crop

Option notes:

- A: Correct:
- B: Correct: 57,600 for 80×80×3×3.
- C: Correct: Very slow.
- D: Incorrect: They can.

Takeaway: Better: run the CNN once and share the feature map.

Source: Lecture 7, slides 59-60

DETECT A4. [BLANK · Easy · calculation]

80 × 80 locations × 3 window scales × 3 aspect ratios = _____ candidate crops.

Answer: 57600
(Also accepted: 57,600)

Takeaway: Lecture 7 slide 60.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A5. [MCQ · Medium]

In YOLO (Lecture 7), what does each spatial feature vector of the 20×20×256 map get mapped to?

A. A single class label for the whole image
B. A new crop to classify
C. Box (x, y, w, h) + objectness + class scores
D. A per-pixel mask

Answer: C. Box (x, y, w, h) + objectness + class scores

Option notes:

- A: That's classification.
- B: No crops in YOLO.
- D: That's segmentation.

Takeaway: Dense predictions in one pass.

Source: Lecture 7, slide 61

DETECT A6. [TF · Medium]

YOLO processes all 400 spatial locations of a 20×20 feature map together in one pass.

Answer: True

Why: Lecture 7 slide 61.

Takeaway: One look.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A7. [BLANK · Medium · calculation]

A YOLO head with C = 3 classes on a 20×20 grid predicts 20×20×_____ values.

Answer: 8

Takeaway: 5 + C = 8.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A8. [MCQ · Easy · calculation]

A has corners (0,0),(6,4); B has (2,0),(8,4). IoU?

A. 0.67
B. 0.25
C. 0.50
D. 0.33

Answer: C. 0.50

Option notes:

- A: 16/24 divides by one box only.
- B: Arithmetic slip.
- D: Union must subtract the intersection: 16/48 is wrong.

Takeaway: 16/(24 + 24 − 16) = 0.5.

Source: Key points about IoU and non-max suppression

DETECT A9. [TF · Easy]

If two boxes do not overlap at all, their IoU is 0.

Answer: True

Why: Intersection 0 → IoU 0.

Takeaway: IoU ∈ [0, 1].

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A10. [TF · Medium]

Union area = area of A + area of B.

Answer: False

Why: Union = A + B − intersection; otherwise the overlap is counted twice.

Takeaway: Classic IoU trap.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A11. [BLANKS · Medium · calculation]

Boxes (1,1)-(5,5) and (3,3)-(7,7). Intersection = _____, union = _____, IoU = _____ (3 dp).

(1) I = ____
(2) U = ____
(3) IoU = ____

Answer: I = 4; U = 28; IoU = 0.143

Takeaway: Overlap 2×2 = 4; 16 + 16 − 4 = 28.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A12. [MCQ · Medium · calculation]

Box A is completely inside box B (A = 4, B = 16). IoU?

A. 0.33
B. 0.2
C. 0.25
D. 1.0

Answer: C. 0.25

Option notes:

- A: Divided by B − A.
- B: 4/(16 + 4) forgets to subtract I.
- D: Only identical boxes give 1.

Takeaway: I = 4, U = 16 → 0.25.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A13. [MCQ · Medium]

NMS rule: suppress if IoU > 0.5. The kept box and a lower-scoring same-class box have IoU exactly 0.5. What happens?

A. Both are removed
B. It is suppressed
C. The lower-scoring box is kept (equality does not trigger suppression)
D. The higher-scoring box is removed

Answer: C. The lower-scoring box is kept (equality does not trigger suppression)

Option notes:

- A: The top box is always kept.
- B: Only strictly greater suppresses.
- D: NMS keeps the highest score.

Takeaway: Read > vs ≥ carefully.

Source: Key points about IoU and non-max suppression

DETECT A14. [TF · Medium]

NMS suppresses overlapping boxes even when they belong to different classes.

Answer: False

Why: Suppression is applied to boxes of the SAME class.

Takeaway: Per-class NMS.

Source: Key points about IoU and non-max suppression

DETECT A15. [MULTI · Medium]

Select ALL steps of non-max suppression (Lecture 7). (Select ALL that apply.)

A. Suppress highly overlapping boxes
B. Repeat with the next remaining box
C. Delete the highest-score box if it overlaps others
D. Average the overlapping boxes into one
E. Keep the highest-score box

Answer: A. Suppress highly overlapping boxes; B. Repeat with the next remaining box; E. Keep the highest-score box

Option notes:

- A: Correct: Step 2.
- B: Correct: Step 3.
- C: Incorrect: It is always kept.
- D: Incorrect: That's not standard NMS.
- E: Correct: Step 1.

Takeaway: Keep, suppress, repeat.

Source: Lecture 7, slide 62

DETECT A16. [MCQ · Hard]

Raising the NMS IoU threshold (e.g. 0.3 → 0.7) tends to…

A. keep more boxes, so more duplicate detections may survive
B. merge classes
C. change the confidence scores
D. remove more boxes

Answer: A. keep more boxes, so more duplicate detections may survive

Option notes:

- B: No.
- C: Scores are unchanged.
- D: Higher threshold suppresses fewer.

Takeaway: Thresholds trade missed objects against duplicates.

Source: Lecture 7, slide 62

DETECT A17. [MATCH · Medium]

Match each detection term to its meaning.

1. IoU
2. NMS
3. Objectness
4. Class score

A. overlap area ÷ union area
B. pixel-level class map
C. how likely a box contains any object
D. removes duplicate boxes
E. how likely the object is a given class

Answer: 1 → A, 2 → D, 3 → C, 4 → E

Working:

- IoU → overlap area ÷ union area
- NMS → removes duplicate boxes
- Objectness → how likely a box contains any object
- Class score → how likely the object is a given class

Takeaway: Know the vocabulary.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A18. [MCQ · Medium]

In the segmentation model, what does the decoder do?

A. Draws bounding boxes
B. Restores spatial resolution with upsampling + convolution
C. Applies NMS
D. Reduces resolution with downsampling

Answer: B. Restores spatial resolution with upsampling + convolution

Option notes:

- A: Detection.
- C: Not used here.
- D: That's the encoder.

Takeaway: Encoder ↓, decoder ↑.

Source: Lecture 7, slide 63

DETECT A19. [BLANKS · Medium]

SceneSegmenter(num_classes = 4): logits shape [N, _____, H, W]; pred_mask = logits.argmax(dim = 1) has shape [N, _____, _____] (use H and W).

(1) classes = ____
(2) dim 2 = ____
(3) dim 3 = ____

Answer: classes = 4; dim 2 = H; dim 3 = W

Takeaway: One class ID per pixel.

Source: Lecture 7, slide 63

DETECT A20. [TF · Easy]

Semantic segmentation predicts one class for every pixel.

Answer: True

Why: Lecture 7 slide 63.

Takeaway: Per-pixel classification.

Source: Lecture 7 (Object detection) + Key points about IoU and NMS

DETECT A21. [MATCH · Medium]

Match the vision task to its output.

1. Classification
2. Object detection
3. Semantic segmentation

A. a caption sentence
B. one label for the whole image
C. a box, label and confidence for every object
D. a class label for every pixel

Answer: 1 → B, 2 → C, 3 → D

Working:

- Classification → one label for the whole image
- Object detection → a box, label and confidence for every object
- Semantic segmentation → a class label for every pixel

Takeaway: Different training targets, different questions.

Source: Lecture 7, slide 12

DETECT A22. [BLANK · Easy]

Object detection asks 'what?' and '_____?' for every object.

Answer: where
(Also accepted: Where)

Takeaway: Class + box.

Source: Lecture 7 (Object detection)

DETECT A23. [TF · Hard]

A detection box in Lecture 7 is described by (x, y, width, height).

Answer: True

Why: Slide 58.

Takeaway: Box = where.

Source: Lecture 7 (Object detection)

DETECT A24. [TF · Medium]

Running the CNN separately on many heavily overlapping crops repeats much of the same computation.

Answer: True

Why: Slide 60.

Takeaway: Share one feature map instead.

Source: Lecture 7 (Object detection)

DETECT A25. [MCQ · Hard · calculation]

A YOLO-style head on a 13×13 grid with 20 classes and one box per cell outputs how many values?

A. 325
B. 845
C. 4,225
D. 3,380

Answer: C. 4,225

Option notes:

- A: Missing the 13×13 grid.
- B: Used 5 per cell (forgot the classes).
- D: Used 20 per cell (forgot the 5 box/objectness values).

Takeaway: 13×13×(5 + 20) = 169 × 25.

Source: Lecture 7 (Object detection)

DETECT A26. [MCQ · Hard]

pred_mask = logits.argmax(dim=1) on logits of shape [N, 4, H, W] returns…

A. a probability per pixel
B. one class ID (0-3) per pixel, shape [N, H, W]
C. one box per image
D. the 4 class scores per pixel

Answer: B. one class ID (0-3) per pixel, shape [N, H, W]

Option notes:

- A: argmax returns indices.
- C: That's detection.
- D: argmax picks the index.

Takeaway: Lecture 7 slide 63 code.

Source: Lecture 7 (Object detection)

DETECT A27. [BLANKS · Medium · calculation]

A (0, 0)–(6, 4), B (2, 0)–(8, 4). Area A ___; overlap ___; union ___; IoU ___. If A scores higher and suppression requires IoU strictly greater than 0.40, is B suppressed (yes/no)? ___

(1) area A = ____
(2) overlap = ____
(3) union = ____
(4) IoU = ____
(5) suppressed? = ____

Answer: area A = 24; overlap = 16; union = 32; IoU = 0.50; suppressed? = yes

Working:

- overlap width min(6,8) − max(0,2) = 4; height 4
- union 24 + 24 − 16 = 32
- IoU 0.50 > 0.40 → suppress B

Takeaway: Overlap 4 × 4.

Source: Book 2 (Exam examples), Q223, p. 32

DETECT A28. [BLANK · Medium · calculation]

A (1, 1)–(5, 5), B (3, 2)–(7, 6). IoU = ___ (3 d.p.).

Answer: 0.231

Working:

- overlap (5 − 3) × (5 − 2) = 6
- union 16 + 16 − 6 = 26
- 6/26 = 0.231

Takeaway: IoU = I/(A + B − I).

Source: Book 2 (Exam examples), Q224, p. 32

DETECT A29. [BLANK · Medium · calculation]

A (0, 0)–(4, 4), B (2, 2)–(6, 6). IoU = ___ (3 d.p.).

Answer: 0.143

Working:

- overlap 2 × 2 = 4
- union 28
- 4/28 = 0.143

Takeaway: IoU = I/(A + B − I).

Source: Book 2 (Exam examples), Q225, p. 32

DETECT A30. [BLANK · Medium · calculation]

A (0, 0)–(10, 10), B (5, 0)–(15, 10). IoU = ___ (3 d.p.).

Answer: 0.333

Working:

- overlap 50

- union 150
- 0.333

Takeaway: IoU = I/(A + B − I).

Source: Book 2 (Exam examples), Q226, p. 32

DETECT A31. [TF · Medium]

Two boxes have IoU exactly 0.50 and the rule is 'suppress if IoU > 0.5'. The lower-scored box is suppressed.

Answer: False

Why: Equality does not trigger 'strictly greater'.

Takeaway: Read > vs ≥.

Source: Book 2 (Exam examples), Q227, p. 32

DETECT A32. [MULTI · Hard · calculation]

NMS, rule 'IoU > 0.5 suppresses'. Boxes: A car 0.90 (0,0)–(4,4); B car 0.80 (1,1)–(5,5); C car 0.75 (0,0)–(4,3); D person 0.70 (1,0)–(5,4). Which boxes remain? (Select ALL that apply.)

A. D
B. C
C. B
D. A

Answer: A. D; C. B; D. A

Working:

- Keep A.
- IoU(A,B) = 9/(16 + 16 − 9) = 0.39 → keep B.
- IoU(A,C) = 12/16 = 0.75 → suppress C.
- D is a person: separate class.

Option notes:

- A: Correct: different class, never compared with cars
- B: Incorrect: IoU(A,C) = 12/16 = 0.75 → suppressed
- C: Correct: IoU(A,B) = 9/23 = 0.39 ≤ 0.5
- D: Correct: highest score, kept

Takeaway: NMS is per class.

Source: Book 2 (Exam examples), Q228, p. 32

DETECT A33. [MCQ · Easy]

YOLO's key idea is to:

A. use only fully connected layers
B. segment each pixel
C. run the CNN once and predict boxes everywhere on a shared feature map
D. classify 57,600 crops separately

Answer: C. run the CNN once and predict boxes everywhere on a shared feature map

Option notes:

- A: It is convolutional.
- B: That is segmentation.

- D: That is the naive approach.

Takeaway: You Only Look Once.

Source: Book 2 (Exam examples), Q229, p. 32

DETECT A34. [BLANK · Medium · calculation]

A YOLO head on a 20 × 20 feature map with C classes outputs a tensor of shape 20 × 20 × (___ + C).

Answer: 5

Takeaway: Box 4 + objectness 1 + C class scores.

Source: Book 2 (Exam examples), Q230, p. 32

DETECT A35. [MATCH · Easy]

Match the vision task to its output.

1. classification
2. object detection
3. semantic segmentation

A. a class label for every pixel
B. one label for the whole image
C. boxes, labels and confidences for every object

Answer: 1 → B, 2 → C, 3 → A

Working:

- classification → one label for the whole image
- object detection → boxes, labels and confidences for every object
- semantic segmentation → a class label for every pixel

Takeaway: What vs what + where vs per pixel.

Source: Book 2 (Exam examples), Q231, p. 32

DETECT A36. [BLANK · Medium · calculation]

IoU of (0, 0)–(3, 3) and (1, 1)–(4, 4): ___ (3 d.p.).

Answer: 0.286

Takeaway: 4/14.

Source: Book 2 (Exam examples), Mixed practice paper Q22, p. 55

DETECT A37. [TF · Easy]

NMS compares boxes of different classes against each other.

Answer: False

Why: Suppression is within the same class.

Takeaway: Per-class NMS.

Source: Book 2 (Exam examples), Mixed practice paper Q23, p. 55

DETECT A38. [BLANK · Easy · calculation]

IoU of (0, 0)–(2, 2) and (1, 1)–(3, 3) = ___ (3 d.p.).

Answer: 0.143

Takeaway: 1/7.

Source: Book 3 (AI from zero), Practice 21.1, p. 87

DETECT A39. [BLANK · Easy · calculation]

Two boxes don't touch. IoU = ___.

Answer: 0

Takeaway: max(0, ·) handles it.

Source: Book 3 (AI from zero), Practice 21.2, p. 87

DETECT A40. [TF · Medium]

After NMS with threshold 0.5, two kept boxes can still overlap with IoU 0.4.

Answer: True

Why: Only overlaps above 0.5 are suppressed.

Takeaway: Threshold, not zero overlap.

Source: Book 3 (AI from zero), Practice 21.3, p. 87

DETECT A41. [BLANKS · Medium]

Segmentation of one 256 × 256 image with 5 classes: logits shape ___; mask shape ___.

(1) logits = ____
(2) mask = ____

Answer: logits = 5x256x256; mask = 256x256

Takeaway: argmax over classes.

Source: Book 3 (AI from zero), Practice 21.4, p. 87

DETECT A42. [MULTI · Medium · calculation]

NMS (threshold 0.5): boxes P 0.95, Q 0.90, R 0.80, S 0.60; IoUs P–Q 0.70, P–R 0.10, P–S 0.20, R–S 0.65. Which are kept? (Select ALL that apply.)

A. Q
B. R
C. P
D. S

Answer: B. R; C. P

Option notes:

- A: Incorrect: 0.70 > 0.5 with P
- B: Correct: 0.10 with P
- C: Correct: highest
- D: Incorrect: 0.65 > 0.5 with R

Takeaway: Keep best, delete its duplicates, repeat.

Source: Book 3 (AI from zero), Chapter 21, p. 85

DETECT A43. [MCQ · Medium · calculation]

A box is given as centre (x, y) = (5, 4), w = 4, h = 2. Its corners are:

A. (4, 3)–(6, 5)
B. (5, 4)–(9, 6)
C. (3, 3)–(7, 5)
D. (1, 2)–(9, 6)

Answer: C. (3, 3)–(7, 5)

Option notes:

- A: Halved twice.
- B: That treats the centre as a corner.
- D: Used full w and h on each side.

Takeaway: $x_1 = x - w/2$.

Source: Book 3 (AI from zero), Chapter 21, p. 85

DETECT A44. [MATCH · Medium]

NMS threshold trade-off.

1. threshold too low
2. threshold too high

A. duplicates survive (false detections)
B. neighbouring real objects get suppressed (missed detections)

Answer: 1 → B, 2 → A

Working:

- threshold too low → neighbouring real objects get suppressed (missed detections)
- threshold too high → duplicates survive (false detections)

Takeaway: Tune it.

Source: Book 3 (AI from zero), Chapter 21, p. 85

DETECT A45. [MULTI · Medium]

AIMA's object detection pipeline: (Select ALL that apply.)

A. suppress duplicate boxes (non-maximum suppression)
B. require one box per image
C. classify each box with a CNN
D. propose many candidate boxes (regions)

Answer: A. suppress duplicate boxes (non-maximum suppression); C. classify each box with a CNN; D. propose many candidate boxes (regions)

Option notes:

- A: Correct:
- B: Incorrect: detection allows many objects
- C: Correct:
- D: Correct:

Takeaway: Region proposals → classify → NMS.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 25: object detection

DETECT A46. [MCQ · Easy]

Detection quality on a test image is usually measured by:

A. R^2
B. intersection over union (IoU) between predicted and ground-truth boxes
C. perplexity
D. pixel accuracy only

Answer: B. intersection over union (IoU) between predicted and ground-truth boxes

Option notes:

- A: Regression.
- C: Language models.
- D: That is for segmentation.

Takeaway: IoU ≥ threshold = hit.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 25: evaluation

DETECT A47. [BLANKS · Medium · calculation]

Sheet method. A has corners (1, 2) and (7, 6); B has corners (4, 3) and (9, 8). Fill in: overlap width, overlap height, intersection, union, IoU (3 d.p.).

(1) width = ____
(2) height = ____
(3) intersection = ____
(4) union = ____
(5) IoU = ____

Answer: width = 3; height = 3; intersection = 9; union = 40; IoU = 0.225

Working:

- width = min(7, 9) − max(1, 4) = 3
- height = min(6, 8) − max(2, 3) = 3
- intersection 9
- areas 6 × 4 = 24 and 5 × 5 = 25 → union 24 + 25 − 9 = 40
- IoU = 9/40 = 0.225

Takeaway: Shared width = $\min(x_2) - \max(x_1)$; shared height = $\min(y_2) - \max(y_1)$.

Source: Key points about IoU and non-max suppression

DETECT A48. [BLANKS · Medium · calculation]

A (0, 0)–(5, 5), B (1, 1)–(6, 6). Intersection ___, union ___, IoU ___ (3 d.p.).

(1) intersection = ____
(2) union = ____
(3) IoU = ____

Answer: intersection = 16; union = 34; IoU = 0.471

Working:

- overlap 4 × 4 = 16
- 25 + 25 − 16 = 34
- 16/34 = 0.471

Takeaway: 4 × 4 overlap.

Source: Key points about IoU and non-max suppression

DETECT A49. [MCQ · Easy]

According to the key-points sheet, union area =

A. area of A × area of B
B. area of A + area of B − intersection area
C. area of A + area of B
D. the larger box's area

Answer: B. area of A + area of B − intersection area

Option notes:

- A: Areas add, not multiply.
- C: That counts the overlap twice.
- D: Only if one box contains the other.

Takeaway: Subtract the overlap once.

Source: Key points about IoU and non-max suppression

DETECT A50. [TF · Easy]

For corner coordinates, a box's width is the larger x-coordinate minus the smaller x-coordinate.

Answer: True

Why: Height likewise uses y; area = width × height.

Takeaway: Width from x, height from y.

Source: Key points about IoU and non-max suppression

DETECT A51. [MULTI · Hard · calculation]

Threshold rule: 'suppress if IoU is strictly greater than 0.40'. Car boxes: P 0.92, Q 0.85, R 0.70. IoU(P,Q) = 0.40, IoU(P,R) = 0.55, IoU(Q,R) = 0.10. Which remain? (Select ALL that apply.)

A. R
B. Q
C. P

Answer: B. Q; C. P

Working:

- Keep P.
- Q: 0.40 > 0.40? No → keep.
- R: 0.55 > 0.40 → suppress (P is kept, so R goes).

Option notes:

- A: Incorrect: IoU(P, R) = 0.55 > 0.40 → suppressed
- B: Correct: IoU 0.40 is not strictly greater than 0.40
- C: Correct: highest score, kept first

Takeaway: Equality does not trigger 'strictly greater'.

Source: Key points about IoU and non-max suppression

DETECT A52. [MULTI · Hard · calculation]

Same boxes, but the rule is now 'suppress if IoU ≥ 0.40'. Which remain? (Select ALL that apply.)

A. P
B. Q
C. R

Answer: A. P

Option notes:

- A: Correct: kept first
- B: Incorrect: 0.40 ≥ 0.40 → suppressed
- C: Incorrect: 0.55 ≥ 0.40 → suppressed

Takeaway: Read > versus ≥ carefully.

Source: Key points about IoU and non-max suppression

DETECT A53. [MCQ · Medium]

NMS: a person box (0.70) overlaps the kept car box (0.95) with IoU 0.8, threshold 0.5. What happens to the person box?

A. it is merged with the car box
B. the car box is suppressed
C. it is kept: NMS only suppresses boxes of the same class
D. it is suppressed because 0.8 > 0.5

Answer: C. it is kept: NMS only suppresses boxes of the same class

Option notes:

- A: NMS deletes, it does not merge.
- B: The higher score is kept.
- D: Different classes are not compared.

Takeaway: Same class only.

Source: Key points about IoU and non-max suppression

DETECT A54. [MCQ · Easy]

NMS starts by:

A. deleting the lowest-confidence box
B. keeping the highest-confidence detection
C. averaging all boxes
D. computing IoU between every pair of classes

Answer: B. keeping the highest-confidence detection

Option notes:

- A: It keeps the best first.
- C: No averaging.
- D: Same class only.

Takeaway: Then suppress its same-class overlaps, and repeat.

Source: Key points about IoU and non-max suppression

DETECT A55. [BLANK · Easy · calculation]

Sheet example: A (0,0)–(6,4), B (2,0)–(8,4). IoU = ___ (2 d.p.).

Answer: 0.50
(Also accepted: 0.5, 1/2)

Working:

- Each area 24; overlap 4 × 4 = 16; union 32; IoU 0.50

Takeaway: 16/32.

Source: Key points about IoU and non-max suppression

Part B: calculation drills (80)

DETECT B1. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (2,6)–(6,10); box B: (4,4)–(9,7). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 2; Union = 29; IoU = 0.069

Working:

- Areas: A = 16, B = 15.
- Overlap = 2 × 1 = 2.
- Union = 16 + 15 − 2 = 29.
- IoU = 0.0690.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B2. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (4,2)–(6,7); box B: (1,3)–(6,6). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 6; Union = 19; IoU = 0.316

Working:

- Areas: A = 10, B = 15.
- Overlap = 2 × 3 = 6.
- Union = 10 + 15 − 6 = 19.
- IoU = 0.3158.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B3. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (3,1)–(6,5); box B: (1,4)–(5,6). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 2; Union = 18; IoU = 0.111

Working:

- Areas: A = 12, B = 8.
- Overlap = 2 × 1 = 2.
- Union = 12 + 8 − 2 = 18.
- IoU = 0.1111.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B4. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (5,4)–(9,6); box B: (5,1)–(9,5). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 4; Union = 20; IoU = 0.200

Working:

- Areas: A = 8, B = 16.
- Overlap = 4 × 1 = 4.
- Union = 8 + 16 − 4 = 20.
- IoU = 0.2000.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B5. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (3,4)–(5,8); box B: (4,3)–(9,7). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 3; Union = 25; IoU = 0.120

Working:

- Areas: A = 8, B = 20.
- Overlap = 1 × 3 = 3.
- Union = 8 + 20 − 3 = 25.
- IoU = 0.1200.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B6. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (2,5)–(6,10); box B: (5,1)–(7,5). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 0; Union = 28; IoU = 0.000

Working:

- Areas: A = 20, B = 8.
- Overlap = 1 × 0 = 0.
- Union = 20 + 8 − 0 = 28.
- IoU = 0.0000.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B7. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (1,1)–(4,5); box B: (1,1)–(5,3). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 6; Union = 14; IoU = 0.429

Working:

- Areas: A = 12, B = 8.
- Overlap = 3 × 2 = 6.
- Union = 12 + 8 − 6 = 14.
- IoU = 0.4286.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B8. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (2,5)–(7,10); box B: (3,2)–(7,7). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 8; Union = 37; IoU = 0.216

Working:

- Areas: A = 25, B = 20.
- Overlap = 4 × 2 = 8.
- Union = 25 + 20 − 8 = 37.
- IoU = 0.2162.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B9. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (4,2)–(6,6); box B: (5,2)–(7,6). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 4; Union = 12; IoU = 0.333

Working:

- Areas: A = 8, B = 8.
- Overlap = 1 × 4 = 4.
- Union = 8 + 8 − 4 = 12.
- IoU = 0.3333.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B10. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (2,6)–(7,8); box B: (0,3)–(4,8). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 4; Union = 26; IoU = 0.154

Working:

- Areas: A = 10, B = 20.
- Overlap = 2 × 2 = 4.
- Union = 10 + 20 − 4 = 26.
- IoU = 0.1538.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B11. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (6,2)–(8,7); box B: (5,2)–(7,7). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 5; Union = 15; IoU = 0.333

Working:

- Areas: A = 10, B = 10.
- Overlap = 1 × 5 = 5.
- Union = 10 + 10 − 5 = 15.

- IoU = 0.3333.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B12. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (0,0)–(2,5); box B: (1,4)–(5,9). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 1; Union = 29; IoU = 0.034

Working:

- Areas: A = 10, B = 20.
- Overlap = 1 × 1 = 1.
- Union = 10 + 20 − 1 = 29.
- IoU = 0.0345.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B13. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (6,3)–(11,8); box B: (2,3)–(7,6). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 3; Union = 37; IoU = 0.081

Working:

- Areas: A = 25, B = 15.
- Overlap = 1 × 3 = 3.
- Union = 25 + 15 − 3 = 37.
- IoU = 0.0811.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B14. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (4,2)–(8,5); box B: (4,3)–(6,5). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 4; Union = 12; IoU = 0.333

Working:

- Areas: A = 12, B = 4.

- Overlap = 2 × 2 = 4.
- Union = 12 + 4 − 4 = 12.
- IoU = 0.3333.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B15. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (2,5)–(4,7); box B: (1,5)–(6,7). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 4; Union = 10; IoU = 0.400

Working:

- Areas: A = 4, B = 10.
- Overlap = 2 × 2 = 4.
- Union = 4 + 10 − 4 = 10.
- IoU = 0.4000.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B16. [BLANKS · Medium · calculation]

[Diagram as text] Labels: A · B

Box A: (1,0)–(4,5); box B: (2,4)–(5,8). Give intersection area, union area and IoU (3 dp).

(1) Intersection = ____
(2) Union = ____
(3) IoU = ____

Answer: Intersection = 2; Union = 25; IoU = 0.080

Working:

- Areas: A = 15, B = 12.
- Overlap = 2 × 1 = 2.
- Union = 15 + 12 − 2 = 25.
- IoU = 0.0800.

Takeaway: Use the SHARED width and height for the intersection.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B17. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (3,6) and (8,8); box B has corners (3,3) and (7,7). What is IoU(A, B)?

A. 0.182
B. 0.400
C. 0.250
D. 0.154

Answer: A. 0.182

Working:

- Area A = 5×2 = 10; Area B = 4×4 = 16.
- Overlap width = min(8,7) − max(3,3) = 4; height = 1; I = 4.
- Union = 10 + 16 − 4 = 22.
- IoU = 4/22 = 0.1818.

Option notes:

- B: Divided by the smaller box, not the union.
- C: Divided by the larger box, not the union.
- D: Union must subtract the intersection: A + B − I.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B18. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (5,0) and (8,4); box B has corners (6,2) and (8,7). What is IoU(A, B)?

A. 0.333
B. 0.182
C. 0.400
D. 0.222

Answer: D. 0.222

Working:

- Area A = 3×4 = 12; Area B = 2×5 = 10.
- Overlap width = min(8,8) − max(5,6) = 2; height = 2; I = 4.
- Union = 12 + 10 − 4 = 18.
- IoU = 4/18 = 0.2222.

Option notes:

- A: Divided by the larger box, not the union.
- B: Union must subtract the intersection: A + B − I.
- C: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B19. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (2,3) and (7,7); box B has corners (2,0) and (6,4). What is IoU(A, B)?

A. 0.111
B. 0.125
C. 0.200
D. 0.250

Answer: B. 0.125

Working:

- Area A = 5×4 = 20; Area B = 4×4 = 16.

- Overlap width = min(7,6) − max(2,2) = 4; height = 1; I = 4.
- Union = 20 + 16 − 4 = 32.
- IoU = 4/32 = 0.1250.

Option notes:

- A: Union must subtract the intersection: A + B − I.
- C: Divided by the larger box, not the union.
- D: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B20. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (3,2) and (8,4); box B has corners (4,2) and (6,5). What is IoU(A, B)?

A. 0.333
B. 0.250
C. 0.400
D. 0.667

Answer: A. 0.333

Working:

- Area A = 5×2 = 10; Area B = 2×3 = 6.
- Overlap width = min(8,6) − max(3,4) = 2; height = 2; I = 4.
- Union = 10 + 6 − 4 = 12.
- IoU = 4/12 = 0.3333.

Option notes:

- B: Union must subtract the intersection: A + B − I.
- C: Divided by the larger box, not the union.
- D: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B21. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (5,4) and (8,9); box B has corners (5,5) and (7,9). What is IoU(A, B)?

A. 0.533
B. 0.348
C. 0.613
D. 1.000

Answer: A. 0.533

Working:

- Area A = 3×5 = 15; Area B = 2×4 = 8.
- Overlap width = min(8,7) − max(5,5) = 2; height = 4; I = 8.
- Union = 15 + 8 − 8 = 15.
- IoU = 8/15 = 0.5333.

Option notes:

- B: Union must subtract the intersection: A + B − I.
- C: Arithmetic slip: recompute each step carefully.
- D: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B22. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (0,4) and (4,7); box B has corners (1,6) and (5,11). What is IoU(A, B)?

A. 0.150
B. 0.103
C. 0.094
D. 0.250

Answer: B. 0.103

Working:

- Area A = 4×3 = 12; Area B = 4×5 = 20.
- Overlap width = min(4,5) − max(0,1) = 3; height = 1; I = 3.
- Union = 12 + 20 − 3 = 29.
- IoU = 3/29 = 0.1034.

Option notes:

- A: Divided by the larger box, not the union.
- C: Union must subtract the intersection: A + B − I.
- D: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B23. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (5,3) and (7,8); box B has corners (6,6) and (8,10). What is IoU(A, B)?

A. 0.125
B. 0.111
C. 0.200
D. 0.250

Answer: A. 0.125

Working:

- Area A = 2×5 = 10; Area B = 2×4 = 8.
- Overlap width = min(7,8) − max(5,6) = 1; height = 2; I = 2.
- Union = 10 + 8 − 2 = 16.
- IoU = 2/16 = 0.1250.

Option notes:

- B: Union must subtract the intersection: A + B − I.
- C: Divided by the larger box, not the union.

- D: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B24. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (1,1) and (4,3); box B has corners (0,0) and (5,2). What is IoU(A, B)?

A. 0.188
B. 0.300
C. 0.500
D. 0.231

Answer: D. 0.231

Working:

- Area A = 3×2 = 6; Area B = 5×2 = 10.
- Overlap width = min(4,5) − max(1,0) = 3; height = 1; I = 3.
- Union = 6 + 10 − 3 = 13.
- IoU = 3/13 = 0.2308.

Option notes:

- A: Union must subtract the intersection: A + B − I.
- B: Divided by the larger box, not the union.
- C: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B25. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (0,4) and (2,6); box B has corners (4,2) and (9,5). What is IoU(A, B)?

A. 0.007
B. -0.014
C. 0.000
D. 0.021

Answer: C. 0.000

Working:

- Area A = 2×2 = 4; Area B = 5×3 = 15.
- Overlap width = min(2,9) − max(0,4) = 0; height = 1; I = 0.
- Union = 4 + 15 − 0 = 19.
- IoU = 0/19 = 0.0000.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Arithmetic slip: recompute each step carefully.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B26. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (2,5) and (6,9); box B has corners (5,3) and (9,6). What is IoU(A, B)?

A. 0.036
B. 0.063
C. 0.083
D. 0.037

Answer: D. 0.037

Working:

- Area A = 4×4 = 16; Area B = 4×3 = 12.
- Overlap width = min(6,9) − max(2,5) = 1; height = 1; I = 1.
- Union = 16 + 12 − 1 = 27.
- IoU = 1/27 = 0.0370.

Option notes:

- A: Union must subtract the intersection: A + B − I.
- B: Divided by the larger box, not the union.
- C: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B27. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (3,2) and (5,6); box B has corners (2,0) and (7,3). What is IoU(A, B)?

A. 0.087
B. 0.133
C. 0.250
D. 0.095

Answer: D. 0.095

Working:

- Area A = 2×4 = 8; Area B = 5×3 = 15.
- Overlap width = min(5,7) − max(3,2) = 2; height = 1; I = 2.
- Union = 8 + 15 − 2 = 21.
- IoU = 2/21 = 0.0952.

Option notes:

- A: Union must subtract the intersection: A + B − I.
- B: Divided by the larger box, not the union.
- C: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B28. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (2,5) and (7,7); box B has corners (3,6) and (5,10). What is IoU(A, B)?

A. 0.125
B. 0.200
C. 0.111
D. 0.250

Answer: A. 0.125

Working:

- Area A = 5×2 = 10; Area B = 2×4 = 8.
- Overlap width = min(7,5) − max(2,3) = 2; height = 1; I = 2.
- Union = 10 + 8 − 2 = 16.
- IoU = 2/16 = 0.1250.

Option notes:

- B: Divided by the larger box, not the union.
- C: Union must subtract the intersection: A + B − I.
- D: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B29. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (5,4) and (8,7); box B has corners (4,5) and (9,9). What is IoU(A, B)?

A. 0.261
B. 0.207
C. 0.300
D. 0.667

Answer: A. 0.261

Working:

- Area A = 3×3 = 9; Area B = 5×4 = 20.
- Overlap width = min(8,9) − max(5,4) = 3; height = 2; I = 6.
- Union = 9 + 20 − 6 = 23.
- IoU = 6/23 = 0.2609.

Option notes:

- B: Union must subtract the intersection: A + B − I.
- C: Divided by the larger box, not the union.
- D: Divided by the smaller box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B30. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (5,6) and (8,9); box B has corners (5,1) and (8,5). What is IoU(A, B)?

A. 0.000
B. 0.007
C. 0.021
D. -0.014

Answer: A. 0.000

Working:

- Area A = 3×3 = 9; Area B = 3×4 = 12.
- Overlap width = min(8,8) − max(5,5) = 3; height = 0; I = 0.
- Union = 9 + 12 − 0 = 21.
- IoU = 0/21 = 0.0000.

Option notes:

- B: Arithmetic slip: recompute each step carefully.
- C: Arithmetic slip: recompute each step carefully.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B31. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (2,4) and (4,7); box B has corners (0,4) and (5,8). What is IoU(A, B)?

A. 0.231
B. 0.300
C. 1.000
D. 0.345

Answer: B. 0.300

Working:

- Area A = 2×3 = 6; Area B = 5×4 = 20.
- Overlap width = min(4,5) − max(2,0) = 2; height = 3; I = 6.
- Union = 6 + 20 − 6 = 20.
- IoU = 6/20 = 0.3000.

Option notes:

- A: Union must subtract the intersection: A + B − I.
- C: Divided by the smaller box, not the union.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B32. [MCQ · Easy · calculation]

[Diagram as text] Labels: A · B

Box A has corners (0,1) and (4,6); box B has corners (0,3) and (2,8). What is IoU(A, B)?

A. 0.600
B. 0.200
C. 0.300
D. 0.250

Answer: D. 0.250

Working:

- Area A = 4×5 = 20; Area B = 2×5 = 10.

- Overlap width = min(4,2) − max(0,0) = 2; height = 3; I = 6.
- Union = 20 + 10 − 6 = 24.
- IoU = 6/24 = 0.2500.

Option notes:

- A: Divided by the smaller box, not the union.
- B: Union must subtract the intersection: A + B − I.
- C: Divided by the larger box, not the union.

Takeaway: IoU = I / (A + B − I). No overlap → IoU = 0; identical boxes → 1.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B33. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (7,6)–(11,11) | 0.58
B2 | (8,2)–(12,9) | 0.74
B3 | (6,6)–(13,9) | 0.95
B4 | (5,2)–(8,7) | 0.82
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

Answer: B2,B3,B4
(Also accepted: B2, B3, B4, B3,B4,B2)

Working:

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B4(0.82), B2(0.74), B1(0.58).
- Keep B3.
- IoU(B3, B4) = 0.059 ≤ thr → keep for now
- IoU(B3, B2) = 0.324 ≤ thr → keep for now
- IoU(B3, B1) = 0.414 > thr → suppress B1
- Keep B4.
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- Keep B2.
- Remaining: B2,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B34. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (3,7)–(10,10) | 0.74
B2 | (6,5)–(9,11) | 0.82
B3 | (4,6)–(13,8) | 0.66
B4 | (2,2)–(7,6) | 0.9
B5 | (5,4)–(7,8) | 0.58
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

Answer: B1,B2,B3,B4,B5
(Also accepted: B1, B2, B3, B4, B5, B4,B2,B1,B3,B5)

Working:

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.9), B2(0.82), B1(0.74), B3(0.66), B5(0.58).
- Keep B4.
- IoU(B4, B2) = 0.027 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- IoU(B4, B3) = 0.000 ≤ thr → keep for now
- IoU(B4, B5) = 0.167 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B1) = 0.300 ≤ thr → keep for now
- IoU(B2, B3) = 0.200 ≤ thr → keep for now
- IoU(B2, B5) = 0.130 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B3) = 0.182 ≤ thr → keep for now
- IoU(B1, B5) = 0.074 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B5) = 0.182 ≤ thr → keep for now
- Keep B5.
- Remaining: B1,B2,B3,B4,B5.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B35. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (4,6)–(8,9) | 0.58
B2 | (2,2)–(7,10) | 0.74
B3 | (4,6)–(7,9) | 0.66
B4 | (2,3)–(4,6) | 0.9
B5 | (4,1)–(8,5) | 0.82
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

Answer: B2,B3,B4,B5
(Also accepted: B2, B3, B4, B5, B4,B5,B2,B3)

Working:

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.9), B5(0.82), B2(0.74), B3(0.66), B1(0.58).
- Keep B4.
- IoU(B4, B5) = 0.000 ≤ thr → keep for now
- IoU(B4, B2) = 0.150 ≤ thr → keep for now

- IoU(B4, B3) = 0.000 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- Keep B5.
- IoU(B5, B2) = 0.191 ≤ thr → keep for now
- IoU(B5, B3) = 0.000 ≤ thr → keep for now
- IoU(B5, B1) = 0.000 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B3) = 0.225 ≤ thr → keep for now
- IoU(B2, B1) = 0.209 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B1) = 0.750 > thr → suppress B1
- Remaining: B2,B3,B4,B5.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B36. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (0,4)–(6,10) | 0.82
B2 | (2,8)–(4,10) | 0.95
B3 | (0,8)–(7,12) | 0.74
B4 | (0,5)–(4,9) | 0.58
B5 | (0,5)–(2,9) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

Answer: B1,B2,B3,B5
(Also accepted: B1, B2, B3, B5, B2,B5,B1,B3)

Working:

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B5(0.9), B1(0.82), B3(0.74), B4(0.58).
- Keep B2.
- IoU(B2, B5) = 0.000 ≤ thr → keep for now
- IoU(B2, B1) = 0.111 ≤ thr → keep for now
- IoU(B2, B3) = 0.143 ≤ thr → keep for now
- IoU(B2, B4) = 0.111 ≤ thr → keep for now
- Keep B5.
- IoU(B5, B1) = 0.222 ≤ thr → keep for now
- IoU(B5, B3) = 0.059 ≤ thr → keep for now
- IoU(B5, B4) = 0.500 > thr → suppress B4
- Keep B1.
- IoU(B1, B3) = 0.231 ≤ thr → keep for now
- Keep B3.
- Remaining: B1,B2,B3,B5.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B37. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (2,7)–(5,11) | 0.74
B2 | (1,6)–(3,12) | 0.82
B3 | (2,5)–(6,11) | 0.95
B4 | (3,6)–(5,9) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

Answer: B2,B3,B4
(Also accepted: B2, B3, B4, B3,B4,B2)

Working:

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B4(0.9), B2(0.82), B1(0.74).
- Keep B3.
- IoU(B3, B4) = 0.250 ≤ thr → keep for now
- IoU(B3, B2) = 0.161 ≤ thr → keep for now
- IoU(B3, B1) = 0.500 > thr → suppress B1
- Keep B4.
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- Keep B2.
- Remaining: B2,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B38. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (0,1)–(4,7) | 0.66
B2 | (0,1)–(1,8) | 0.82
B3 | (0,1)–(3,4) | 0.95
B4 | (3,2)–(6,5) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

Answer: B2,B3,B4
(Also accepted: B2, B3, B4, B3,B2,B4)

Working:

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B2(0.82), B4(0.74), B1(0.66).
- Keep B3.
- IoU(B3, B2) = 0.231 ≤ thr → keep for now

- IoU(B3, B4) = 0.000 ≤ thr → keep for now
- IoU(B3, B1) = 0.375 > thr → suppress B1
- Keep B2.
- IoU(B2, B4) = 0.000 ≤ thr → keep for now
- Keep B4.
- Remaining: B2,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B39. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (0,2)–(5,3) | 0.9
B2 | (1,2)–(4,4) | 0.82
B3 | (3,2)–(4,4) | 0.95
B4 | (2,2)–(4,4) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

Answer: B1,B2,B3
(Also accepted: B1, B2, B3, B3,B1,B2)

Working:

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B1(0.9), B2(0.82), B4(0.74).
- Keep B3.
- IoU(B3, B1) = 0.167 ≤ thr → keep for now
- IoU(B3, B2) = 0.333 ≤ thr → keep for now
- IoU(B3, B4) = 0.500 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B2) = 0.375 ≤ thr → keep for now
- IoU(B1, B4) = 0.286 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B4) = 0.667 > thr → suppress B4
- Remaining: B1,B2,B3.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B40. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (0,2)–(6,8) | 0.74
B2 | (0,3)–(9,9) | 0.95
B3 | (1,1)–(8,9) | 0.58
B4 | (5,5)–(7,8) | 0.66
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

Answer: B2,B4
(Also accepted: B2, B4)

Working:

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B1(0.74), B4(0.66), B3(0.58).
- Keep B2.
- IoU(B2, B1) = 0.500 > thr → suppress B1
- IoU(B2, B4) = 0.111 ≤ thr → keep for now
- IoU(B2, B3) = 0.618 > thr → suppress B3
- Keep B4.
- Remaining: B2,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B41. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (3,3)–(8,6) | 0.74
B2 | (6,0)–(9,7) | 0.95
B3 | (6,1)–(8,7) | 0.66
B4 | (3,6)–(8,10) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

Answer: B1,B2,B4
(Also accepted: B1, B2, B4, B2,B4,B1)

Working:

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B4(0.9), B1(0.74), B3(0.66).
- Keep B2.
- IoU(B2, B4) = 0.051 ≤ thr → keep for now
- IoU(B2, B1) = 0.200 ≤ thr → keep for now
- IoU(B2, B3) = 0.571 > thr → suppress B3
- Keep B4.
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- Keep B1.
- Remaining: B1,B2,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B42. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (6,6)–(11,9) | 0.82
B2 | (5,6)–(9,8) | 0.95
B3 | (5,4)–(7,7) | 0.9
B4 | (1,6)–(4,8) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.3 (suppress when IoU is strictly greater than 0.3). Which boxes remain?

Answer: B2,B3,B4
(Also accepted: B2, B3, B4)

Working:

- Threshold: suppress if IoU > 0.3 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.95), B3(0.9), B1(0.82), B4(0.74).
- Keep B2.
- IoU(B2, B3) = 0.167 ≤ thr → keep for now
- IoU(B2, B1) = 0.353 > thr → suppress B1
- IoU(B2, B4) = 0.000 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B4) = 0.000 ≤ thr → keep for now
- Keep B4.
- Remaining: B2,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B43. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (1,0)–(10,9) | 0.74
B2 | (4,1)–(9,6) | 0.58
B3 | (4,3)–(8,6) | 0.82
B4 | (4,1)–(6,4) | 0.66
B5 | (1,6)–(6,8) | 0.95
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

Answer: B1,B2,B3,B4,B5
(Also accepted: B1, B2, B3, B4, B5, B5,B3,B1,B4,B2)

Working:

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B5(0.95), B3(0.82), B1(0.74), B4(0.66), B2(0.58).
- Keep B5.
- IoU(B5, B3) = 0.000 ≤ thr → keep for now
- IoU(B5, B1) = 0.123 ≤ thr → keep for now

- IoU(B5, B4) = 0.000 ≤ thr → keep for now
- IoU(B5, B2) = 0.000 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B1) = 0.148 ≤ thr → keep for now
- IoU(B3, B4) = 0.125 ≤ thr → keep for now
- IoU(B3, B2) = 0.480 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B4) = 0.074 ≤ thr → keep for now
- IoU(B1, B2) = 0.309 ≤ thr → keep for now
- Keep B4.
- IoU(B4, B2) = 0.240 ≤ thr → keep for now
- Keep B2.
- Remaining: B1,B2,B3,B4,B5.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B44. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (1,0)–(8,1) | 0.82
B2 | (0,0)–(6,3) | 0.9
B3 | (1,0)–(8,5) | 0.74
B4 | (5,4)–(9,6) | 0.95
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

Answer: B1,B2,B3,B4
(Also accepted: B1, B2, B3, B4, B4,B2,B1,B3)

Working:

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.95), B2(0.9), B1(0.82), B3(0.74).
- Keep B4.
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- IoU(B4, B3) = 0.075 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B1) = 0.250 ≤ thr → keep for now
- IoU(B2, B3) = 0.395 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B3) = 0.200 ≤ thr → keep for now
- Keep B3.
- Remaining: B1,B2,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B45. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (1,0)–(8,7) | 0.82
B2 | (4,0)–(10,8) | 0.9
B3 | (1,0)–(10,5) | 0.95
B4 | (5,1)–(10,6) | 0.66
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

Answer: B3,B4
(Also accepted: B3, B4)

Working:

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B2(0.9), B1(0.82), B4(0.66).
- Keep B3.
- IoU(B3, B2) = 0.476 > thr → suppress B2
- IoU(B3, B1) = 0.593 > thr → suppress B1
- IoU(B3, B4) = 0.400 ≤ thr → keep for now
- Keep B4.
- Remaining: B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B46. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4 · B5

```
Box | Corners | Score
B1 | (4,6)–(10,8) | 0.82
B2 | (6,5)–(8,9) | 0.9
B3 | (6,6)–(8,7) | 0.58
B4 | (6,3)–(11,8) | 0.66
B5 | (5,6)–(10,8) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

Answer: B1,B2,B3,B4
(Also accepted: B1, B2, B3, B4, B2,B1,B4,B3)

Working:

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B2(0.9), B1(0.82), B5(0.74), B4(0.66), B3(0.58).
- Keep B2.
- IoU(B2, B1) = 0.250 ≤ thr → keep for now
- IoU(B2, B5) = 0.286 ≤ thr → keep for now
- IoU(B2, B4) = 0.222 ≤ thr → keep for now
- IoU(B2, B3) = 0.250 ≤ thr → keep for now

- Keep B1.
- IoU(B1, B5) = 0.833 > thr → suppress B5
- IoU(B1, B4) = 0.276 ≤ thr → keep for now
- IoU(B1, B3) = 0.167 ≤ thr → keep for now
- Keep B4.
- IoU(B4, B3) = 0.080 ≤ thr → keep for now
- Keep B3.
- Remaining: B1,B2,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B47. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (7,2)–(11,8) | 0.58
B2 | (7,2)–(8,10) | 0.66
B3 | (6,6)–(8,10) | 0.74
B4 | (1,5)–(6,10) | 0.9
```

Apply non-max suppression to these same-class detections with IoU threshold 0.5 (suppress when IoU is strictly greater than 0.5). Which boxes remain?

Answer: B1,B2,B3,B4
(Also accepted: B1, B2, B3, B4, B4,B3,B2,B1)

Working:

- Threshold: suppress if IoU > 0.5 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B4(0.9), B3(0.74), B2(0.66), B1(0.58).
- Keep B4.
- IoU(B4, B3) = 0.000 ≤ thr → keep for now
- IoU(B4, B2) = 0.000 ≤ thr → keep for now
- IoU(B4, B1) = 0.000 ≤ thr → keep for now
- Keep B3.
- IoU(B3, B2) = 0.333 ≤ thr → keep for now
- IoU(B3, B1) = 0.067 ≤ thr → keep for now
- Keep B2.
- IoU(B2, B1) = 0.231 ≤ thr → keep for now
- Keep B1.
- Remaining: B1,B2,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B48. [BLANK · Exam+ · calculation]

[Diagram as text] Labels: B1 · B2 · B3 · B4

```
Box | Corners | Score
B1 | (1,1)–(6,7) | 0.9
B2 | (0,0)–(7,8) | 0.66
B3 | (2,3)–(4,4) | 0.95
B4 | (0,0)–(5,2) | 0.74
```

Apply non-max suppression to these same-class detections with IoU threshold 0.4 (suppress when IoU is strictly greater than 0.4). Which boxes remain?

Answer: B1,B3,B4
(Also accepted: B1, B3, B4, B3,B1,B4)

Working:

- Threshold: suppress if IoU > 0.4 (strictly greater) with a kept box (all boxes same class).
- Sort by score: B3(0.95), B1(0.9), B4(0.74), B2(0.66).
- Keep B3.
- IoU(B3, B1) = 0.067 ≤ thr → keep for now
- IoU(B3, B4) = 0.000 ≤ thr → keep for now
- IoU(B3, B2) = 0.036 ≤ thr → keep for now
- Keep B1.
- IoU(B1, B4) = 0.111 ≤ thr → keep for now
- IoU(B1, B2) = 0.536 > thr → suppress B2
- Keep B4.
- Remaining: B1,B3,B4.

Takeaway: NMS: keep the top-scoring box, delete its heavy overlaps, repeat. Read whether the rule is > or ≥.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B49. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 13×13 feature map with one box per location, each with x, y, w, h, objectness and 20 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 25; total = 4225

Working:

- Per location = 4 box + 1 objectness + 20 classes = 25.
- Total = 13×13×25 = 4225. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B50. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 7×7 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 735

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 7×7×15 = 735. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B51. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 13×13 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 2535

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 13×13×15 = 2535. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B52. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 7×7 feature map with one box per location, each with x, y, w, h, objectness and 20 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 25; total = 1225

Working:

- Per location = 4 box + 1 objectness + 20 classes = 25.
- Total = 7×7×25 = 1225. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B53. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 13×13 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 2535

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 13×13×15 = 2535. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B54. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 20×20 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 6000

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 20×20×15 = 6000. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B55. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 7×7 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 735

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 7×7×15 = 735. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B56. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 7×7 feature map with one box per location, each with x, y, w, h, objectness and 20 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 25; total = 1225

Working:

- Per location = 4 box + 1 objectness + 20 classes = 25.
- Total = 7×7×25 = 1225. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B57. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 7×7 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 735

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 7×7×15 = 735. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B58. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 13×13 feature map with one box per location, each with x, y, w, h, objectness and 80 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 85; total = 14365

Working:

- Per location = 4 box + 1 objectness + 80 classes = 85.
- Total = 13×13×85 = 14365. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B59. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 7×7 feature map with one box per location, each with x, y, w, h, objectness and 3 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 8; total = 392

Working:

- Per location = 4 box + 1 objectness + 3 classes = 8.
- Total = 7×7×8 = 392. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B60. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 20×20 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 6000

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 20×20×15 = 6000. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B61. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 13×13 feature map with one box per location, each with x, y, w, h, objectness and 80 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 85; total = 14365

Working:

- Per location = 4 box + 1 objectness + 80 classes = 85.
- Total = 13×13×85 = 14365. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B62. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 7×7 feature map with one box per location, each with x, y, w, h, objectness and 10 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 15; total = 735

Working:

- Per location = 4 box + 1 objectness + 10 classes = 15.
- Total = 7×7×15 = 735. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B63. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 20×20 feature map with one box per location, each with x, y, w, h, objectness and 3 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 8; total = 3200

Working:

- Per location = 4 box + 1 objectness + 3 classes = 8.
- Total = 20×20×8 = 3200. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B64. [BLANKS · Medium · calculation]

A YOLO-style head (Lecture 7) predicts on a 20×20 feature map with one box per location, each with x, y, w, h, objectness and 20 class scores. Give the number of values per location and the total number of predicted values.

(1) per location = ____
(2) total = ____

Answer: per location = 25; total = 10000

Working:

- Per location = 4 box + 1 objectness + 20 classes = 25.
- Total = 20×20×25 = 10000. (Lecture 7: 20×20×(5 + C), 400 locations in one pass.)

Takeaway: YOLO runs the CNN once and predicts densely on the shared feature map.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B65. [BLANK · Easy · calculation]

Naive detection classifies crops at 80×80 locations × 2 window scales × 4 aspect ratios. How many candidate crops per image?

Answer: 51200

Working:

- 80×80×2×4 = 51200. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B66. [BLANK · Easy · calculation]

Naive detection classifies crops at 80×80 locations × 4 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 76800

Working:

- 80×80×4×3 = 76800. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B67. [BLANK · Easy · calculation]

Naive detection classifies crops at 80×80 locations × 2 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 38400

Working:

- 80×80×2×3 = 38400. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B68. [BLANK · Easy · calculation]

Naive detection classifies crops at 40×40 locations × 3 window scales × 4 aspect ratios. How many candidate crops per image?

Answer: 19200

Working:

- 40×40×3×4 = 19200. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B69. [BLANK · Easy · calculation]

Naive detection classifies crops at 40×40 locations × 2 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 9600

Working:

- 40×40×2×3 = 9600. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B70. [BLANK · Easy · calculation]

Naive detection classifies crops at 40×40 locations × 4 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 19200

Working:

- 40×40×4×3 = 19200. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B71. [BLANK · Easy · calculation]

Naive detection classifies crops at 100×100 locations × 4 window scales × 2 aspect ratios. How many candidate crops per image?

Answer: 80000

Working:

- 100×100×4×2 = 80000. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B72. [BLANK · Easy · calculation]

Naive detection classifies crops at 40×40 locations × 4 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 19200

Working:

- 40×40×4×3 = 19200. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B73. [BLANK · Easy · calculation]

Naive detection classifies crops at 60×60 locations × 2 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 21600

Working:

- 60×60×2×3 = 21600. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B74. [BLANK · Easy · calculation]

Naive detection classifies crops at 40×40 locations × 3 window scales × 4 aspect ratios. How many candidate crops per image?

Answer: 19200

Working:

- 40×40×3×4 = 19200. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B75. [BLANK · Easy · calculation]

Naive detection classifies crops at 60×60 locations × 4 window scales × 2 aspect ratios. How many candidate crops per image?

Answer: 28800

Working:

- 60×60×4×2 = 28800. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B76. [BLANK · Easy · calculation]

Naive detection classifies crops at 60×60 locations × 2 window scales × 4 aspect ratios. How many candidate crops per image?

Answer: 28800

Working:

- 60×60×2×4 = 28800. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B77. [BLANK · Easy · calculation]

Naive detection classifies crops at 60×60 locations × 4 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 43200

Working:

- 60×60×4×3 = 43200. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B78. [BLANK · Easy · calculation]

Naive detection classifies crops at 60×60 locations × 2 window scales × 2 aspect ratios. How many candidate crops per image?

Answer: 14400

Working:

- 60×60×2×2 = 14400. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B79. [BLANK · Easy · calculation]

Naive detection classifies crops at 80×80 locations × 4 window scales × 2 aspect ratios. How many candidate crops per image?

Answer: 51200

Working:

- 80×80×4×2 = 51200. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

DETECT B80. [BLANK · Easy · calculation]

Naive detection classifies crops at 40×40 locations × 2 window scales × 3 aspect ratios. How many candidate crops per image?

Answer: 9600

Working:

- 40×40×2×3 = 9600. (Lecture 7: 80×80×3×3 = 57,600.)

Takeaway: The crop search space explodes, and nearby crops repeat computation: share one feature map instead.

Source: Lecture 7 (Object detection) + Key points about IoU and non-max suppression

---

🔁 Sequence Models & NLP (Lecture 8)

Part A: hand-written questions (99)

SEQ A1. [MCQ · Easy]

Why can't a plain feed-forward network model sequences well?

A. It cannot use non-linear activations
B. It requires one-hot inputs
C. It has no state: its output depends only on the current fixed-size input
D. It has too few parameters

Answer: C. It has no state: its output depends only on the current fixed-size input

Option notes:

- A: It can.
- B: No.
- D: Not the issue.

Takeaway: No memory → no context.

Source: Lecture 8, slide 8

SEQ A2. [MULTI · Medium]

Which are requirements for a sequence model (Lecture 8)? (Select ALL that apply.)

A. Carry context forward
B. Preserve order
C. Use a separate weight matrix for every time step
D. Handle variable-length sequences
E. Share parameters across time

Answer: A. Carry context forward; B. Preserve order; D. Handle variable-length sequences; E. Share parameters across time

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: Opposite of sharing.
- D: Correct:
- E: Correct:

Takeaway: Four design criteria.

Source: Lecture 8, slide 7

SEQ A3. [MATCH · Medium]

Poll 1: match each task to its sequence structure.

1. ECG signal → diagnosis
2. Initial pose → motion sequence
3. Speech → text
4. Style/seed → music sequence
5. Sensor history → failure

A. many-to-one
B. many-to-many
C. one-to-one
D. many-to-one
E. one-to-many
F. one-to-many

Answer: 1 → A, 2 → E, 3 → B, 4 → E, 5 → A

Working:

- ECG signal → diagnosis → many-to-one
- Initial pose → motion sequence → one-to-many
- Speech → text → many-to-many
- Style/seed → music sequence → one-to-many
- Sensor history → failure → many-to-one

Takeaway: Count inputs and outputs over time.

Source: Lecture 8, slide 18

SEQ A4. [MCQ · Easy]

Image captioning ('A runner crosses the field') is which structure?

A. one-to-many
B. many-to-many
C. one-to-one
D. many-to-one

Answer: A. one-to-many

Option notes:

- B: Translation.
- C: Single output.
- D: That's sentiment.

Takeaway: One image in, many words out.

Source: Lecture 8, slide 16

SEQ A5. [TF · Medium]

In many-to-many machine translation, the input and output sequences must have the same length.

Answer: False

Why: Lecture 8: lengths may differ ('Mando protects the child' → 'Mando protège l'enfant').

Takeaway: Seq2seq lengths can differ.

Source: Lecture 8, slide 17

SEQ A6. [MCQ · Easy · calculation]

Key points: $x_t = 2$, $w_x = 0.5$, $h_{t-1} = 0.4$, $w_h = 0.25$, no bias, tanh(1.1) = 0.800. h_t = ?

A. 0.800
B. 1.1
C. 0.4
D. 1.0

Answer: A. 0.800

Option notes:

- B: That's the sum before activation.
- C: That's the old state.
- D: Ignored the previous state.

Takeaway: Sum then squash.

Source: Key points about a simple RNN hidden-state update

SEQ A7. [TF · Medium]

Unrolling an RNN creates a new set of weights for every time step.

Answer: False

Why: Unrolling copies the computation, not the parameters: the same cell is reused.

Takeaway: Shared W_xh, W_hh, W_hy.

Source: Lecture 8, slide 13

SEQ A8. [BLANK · Easy]

The initial hidden state h_0 is usually set to _____ (or a learned state).

Answer: zeros
(Also accepted: zero, 0)

Takeaway: Lecture 8 slide 11.

Source: Lecture 8, slide 11

SEQ A9. [MATCH · Medium]

Match the RNN weight matrix to its role (MyRNNCell(3, 4, 2)).

1. W_xh
2. W_hh
3. W_hy

A. maps output back to input
B. maps previous hidden h_{t-1} (4) to hidden (4)
C. maps input x_t (3) to hidden (4)
D. maps hidden h_t (4) to output $\hat{y}_t$ (2)

Answer: 1 → C, 2 → B, 3 → D

Working:

- W_xh → maps input x_t (3) to hidden (4)
- W_hh → maps previous hidden h_{t-1} (4) to hidden (4)
- W_hy → maps hidden h_t (4) to output $\hat{y}_t$ (2)

Takeaway: x → h, h → h, h → y.

Source: Lecture 8, slide 28

SEQ A10. [MCQ · Medium]

In the PyTorch RNN loop over 3 time steps, what changes from step to step?

A. The activation function
B. The weight matrices

C. The hidden size
D. Only x_t, h_t and $\hat{y}_t$; the parameters stay fixed

Answer: D. Only x_t, h_t and $\hat{y}_t$; the parameters stay fixed

Option notes:

- A: Always tanh.
- B: Same cell object.
- C: Fixed at 4.

Takeaway: Parameter sharing.

Source: Lecture 8, slide 29

SEQ A11. [BLANK · Medium · calculation]

$h_t = \tanh(0.3\times1 + 0.5\times0.2 + 0.1)$. The value inside tanh is _____.

Answer: 0.5
(Also accepted: 0.50)

Takeaway: 0.3 + 0.1 + 0.1.

Source: Lecture 8 (Sequence modelling)

SEQ A12. [MCQ · Medium · calculation]

Poll 2: a dependency crosses 30 steps; each step multiplies the gradient by 0.8. Which diagnosis and response fit?

A. ≈ 0.24; clip the gradients
B. ≈ 0.001; use an LSTM or GRU
C. ≈ 24; normalise the inputs
D. ≈ 0.8; lower the learning rate

Answer: B. ≈ 0.001; use an LSTM or GRU

Option notes:

- A: Clipping is for exploding gradients.
- C: That's 0.8 × 30, not 0.8^{30}.
- D: 0.8^{30} is tiny, not 0.8.

Takeaway: Repeated products shrink the signal.

Source: Lecture 8, slide 43

SEQ A13. [TF · Medium]

Gradient clipping is the standard fix for vanishing gradients.

Answer: False

Why: Clipping scales down EXPLODING gradients. Vanishing → better activation, initialisation or architecture (LSTM/GRU).

Takeaway: Know which fix goes with which problem.

Source: Lecture 8, slide 40-41

SEQ A14. [MULTI · Medium]

Select ALL correct statements about BPTT in Lecture 8. (Select ALL that apply.)

A. Losses over time steps are summed into a total loss
B. backward() computes gradients through time
C. step() updates the shared parameters
D. Each time step's weights are updated separately
E. Vanishing gradients bias the model toward short-term dependencies

Answer: A. Losses over time steps are summed into a total loss; B. backward() computes gradients through time; C. step() updates the shared parameters; E. Vanishing gradients bias the model toward short-term dependencies

Option notes:

- A: Correct: $L = L_1 + L_2 + L_3$.
- B: Correct:
- C: Correct:
- D: Incorrect: Weights are shared.
- E: Correct: Slide 42.

Takeaway: Sum → backprop once → update.

Source: Lecture 8, slide 35-42

SEQ A15. [MCQ · Hard]

Why do long-term gradients vanish in a simple RNN?

A. $\partial L/\partial h_1$ involves repeated multiplication by W_hh and activation derivatives that are often < 1
B. Because tanh outputs are always 0
C. Because $h_0 = 0$
D. Because the loss is summed

Answer: A. $\partial L/\partial h_1$ involves repeated multiplication by W_hh and activation derivatives that are often < 1

Option notes:

- B: They're in (−1, 1).
- C: Initial state isn't the cause.
- D: Summing doesn't shrink gradients.

Takeaway: Product of many small factors.

Source: Lecture 8, slide 37-39

SEQ A16. [BLANKS · Easy · calculation]

Gate g = [1.0, 0.1, 0.7], candidate ñ = [0.8, 0.5, −0.6]. $h_t = g \odot ñ$ = [___, ___, ___].

(1) h_1 = _____
(2) h_2 = _____
(3) h_3 = _____

Answer: $h_1 = 0.8$; $h_2 = 0.05$; $h_3 = -0.42$

Takeaway: Element-wise products.

Source: Lecture 8, slide 44

SEQ A17. [TF · Medium]

A gate value comes from a sigmoid, so it lies between 0 and 1 and controls how much of the candidate passes through.

Answer: True

Why: Lecture 8 slide 44.

Takeaway: Gating idea used in LSTMs and GRUs.

Source: Lecture 8 (Sequence modelling)

SEQ A18. [MCQ · Medium]

What does a gate value near 0 do to that component of the candidate hidden state?

A. Passes it unchanged
B. Blocks it from passing to the next time step
C. Doubles it
D. Negates it

Answer: B. Blocks it from passing to the next time step

Option notes:

- A: That's a gate value near 1.
- C: Gates are in [0, 1].
- D: No.

Takeaway: 0 = closed, 1 = open.

Source: Lecture 8 (Sequence modelling)

SEQ A19. [MCQ · Easy]

Why can't a neural network take the raw word 'with' as input?

A. Neural networks operate on numerical vectors
B. Words have no order
C. Words are too long
D. The vocabulary is unknown

Answer: A. Neural networks operate on numerical vectors

Option notes:

- B: Irrelevant.
- C: Length isn't the issue.
- D: Even known words must be numbers.

Takeaway: Tokenise → index → vector.

Source: Lecture 8, slide 47

SEQ A20. [TF · Medium]

Token IDs carry numerical meaning: 'Leia' (ID 5) is 'larger' than 'Jedi' (ID 0).

Answer: False

Why: IDs identify words, not magnitude.

Takeaway: Use one-hot or embeddings.

Source: Lecture 8, slide 48

SEQ A21. [MCQ · Medium]

onehot(droid) × E selects…

A. row 2 of the embedding matrix E (droid's ID)
B. the sum of all rows

C. a random row
D. column 2 of E

Answer: A. row 2 of the embedding matrix E (droid's ID)

Option notes:

- B: Only one 1 in the one-hot.
- C: Deterministic lookup.
- D: One-hot row vector × E picks a row.

Takeaway: Embedding lookup = one-hot × E.

Source: Lecture 8, slide 48

SEQ A22. [MULTI · Medium]

Select ALL true statements about one-hot vectors vs learned embeddings. (Select ALL that apply.)

A. Learned embeddings place similar words near each other
B. New words shrink |V|
C. All one-hot vectors are equally distant
D. One-hot vectors have length |V|

Answer: A. Learned embeddings place similar words near each other; C. All one-hot vectors are equally distant; D. One-hot vectors have length |V|

Option notes:

- A: Correct:
- B: Incorrect: They expand it.
- C: Correct: No similarity.
- D: Correct:

Takeaway: Embeddings capture meaning.

Source: Lecture 8, slide 48

SEQ A23. [MCQ · Medium · calculation]

word | count in + ($T_+ = 20$) | count in − ($T_- = 18$) | P_s(w|+) | P_s(w|−)
heartwarming | 6 | 2 | 7/26 ≈ 0.27 | 3/24 ≈ 0.13
delightful | 5 | 2 | 6/26 ≈ 0.23 | 3/24 ≈ 0.13
slow | 2 | 6 | 3/26 ≈ 0.12 | 7/24 ≈ 0.29
repetitive | 1 | 6 | 2/26 ≈ 0.08 | 7/24 ≈ 0.29
charming | 6 | 1 | 7/26 ≈ 0.27 | 2/24 ≈ 0.08
dull | 0 | 1 | 1/26 ≈ 0.04 | 2/24 ≈ 0.08

Lecture 8: 11 positive and 9 negative reviews → P(+) = 0.55, P(−) = 0.45; |V| = 6.

Poll 3: 'heartwarming and delightful, but slow'. P(+) = 0.55, P(−) = 0.45; likelihoods (+, −): heartwarming 0.27/0.13, delightful 0.23/0.13, slow 0.12/0.29. Which conclusion is correct?

A. Negative: slow has the largest negative likelihood
B. Positive: $S_+ \approx 0.00410 > S_- \approx 0.00221$
C. Tie: two positive words vs one negative
D. Cannot decide until the scores are normalised

Answer: B. Positive: $S_+ \approx 0.00410 > S_- \approx 0.00221$

Option notes:

- A: Must multiply all terms.

- C: Counting words isn't Naive Bayes.
- D: Compare unnormalised scores directly.

Takeaway: Multiply prior and all matched likelihoods per class.

Source: Lecture 8, slide 52

SEQ A24. [BLANK · Easy · calculation]

word | count in + (T_+ = 20) | count in − (T_- = 18) | P_s(w|+) | P_s(w|−)
heartwarming | 6 | 2 | 7/26 ≈ 0.27 | 3/24 ≈ 0.13
delightful | 5 | 2 | 6/26 ≈ 0.23 | 3/24 ≈ 0.13
slow | 2 | 6 | 3/26 ≈ 0.12 | 7/24 ≈ 0.29
repetitive | 1 | 6 | 2/26 ≈ 0.08 | 7/24 ≈ 0.29
charming | 6 | 1 | 7/26 ≈ 0.27 | 2/24 ≈ 0.08
dull | 0 | 1 | 1/26 ≈ 0.04 | 2/24 ≈ 0.08

Lecture 8: 11 positive and 9 negative reviews → P(+) = 0.55, P(−) = 0.45; |V| = 6.

Add-one smoothing: heartwarming appears 6 times in T_+ = 20 positive occurrences, |V| = 6.
P_s(heartwarming|+) = 7/_____.

Answer: 26

Takeaway: (6 + 1)/(20 + 6).

Source: Lecture 8, slide 50

SEQ A25. [TF · Medium]

Without smoothing, a word never seen in positive reviews makes the whole positive score 0.

Answer: True

Why: Product with P = 0 → 0 ('dull' has 0 positive counts).

Takeaway: Smoothing prevents zeros.

Source: Lecture 8, slide 50

SEQ A26. [BLANKS · Hard · calculation]

Lecture 8: S_+ = 0.000328 and S_- = 0.000640. Give P(+|text) (2 dp) and the prediction.

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.34; class = negative

Takeaway: 0.000328/0.000968.

Source: Lecture 8, slide 51

SEQ A27. [BLANK · Easy · calculation]

Trigram corpus: 'Mando protects' is followed by Grogu 3 times, villagers once, civilians once. P(Grogu | 'Mando protects') = _____.

Answer: 3/5
(Also accepted: 0.6, 0.60)

Takeaway: C(h, w)/C(h).

Source: Lecture 8, slide 53

SEQ A28. [MULTI · Medium]

What are the limits of n-gram models (Lecture 8)? (Select ALL that apply.)

A. Fixed context: only the last n − 1 words
B. They cannot be estimated from counts
C. They memorise local patterns rather than learning representations
D. No semantic similarity between words

Answer: A. Fixed context: only the last n − 1 words; C. They memorise local patterns rather than learning representations; D. No semantic similarity between words

Option notes:

- A: Correct:
- B: Incorrect: Counting is how they work.
- C: Correct:
- D: Correct: Grogu ≈ foundling unknown.

Takeaway: Fixed context + no meaning.

Source: Lecture 8, slide 54

SEQ A29. [MATCH · Medium]

Match each Word2Vec variant to its training task.

1. CBOW
2. Skip-gram

A. the centre word predicts each nearby context word
B. context words predict the centre word
C. the centre word predicts the next sentence

Answer: 1 → B, 2 → A

Working:

- CBOW → context words predict the centre word
- Skip-gram → the centre word predicts each nearby context word

Takeaway: Window radius 2 around 'Grogu'.

Source: Lecture 8, slide 55

SEQ A30. [MCQ · Medium]

In CBOW with window radius 2, the hidden vector h is…

A. a one-hot vector
B. the sum of all vocabulary embeddings
C. the average of the four context embeddings
D. the centre word's embedding

Answer: C. the average of the four context embeddings

Option notes:

- A: Embeddings are dense.
- B: Only context rows.
- D: That's skip-gram's input.

Takeaway: h = ¼Σ v_w.

Source: Lecture 8, slide 56

SEQ A31. [TF · Easy]

Word2Vec training examples are created by a sliding window, so no manual labels are needed.

Answer: True

Why: Lecture 8 slide 55.

Takeaway: Self-supervised.

Source: Lecture 8 (Sequence modelling)

SEQ A32. [BLANK · Medium]

Vector arithmetic: v('Paris') − v('France') + v('Italy') ≈ v('_____').

Answer: Rome
(Also accepted: rome)

Takeaway: Analogies via offsets.

Source: Lecture 8, slide 59

SEQ A33. [BLANK · Easy]

Sequence = observations + meaningful _____.

Answer: order
(Also accepted: Order)

Takeaway: Lecture 8 slide 5.

Source: Lecture 8 (Sequence modelling)

SEQ A34. [MATCH · Medium]

Match each sequence example to what is ordered (Lecture 8).

1. Markets
2. Music
3. DNA
4. Text

A. bases in order
B. price over time
C. notes over time
D. tokens in order
E. pixels in a single photo

Answer: 1 → B, 2 → C, 3 → A, 4 → D

Working:

- Markets → price over time
- Music → notes over time
- DNA → bases in order
- Text → tokens in order

Takeaway: Sequences in the wild.

Source: Lecture 8 (Sequence modelling)

SEQ A35. [MCQ · Hard]

Applying the same network independently at each time step ($\hat{y}_t = f(x_t)$) fails because…

A. it cannot handle numbers
B. no information flows between time steps
C. each step uses different weights
D. the network has too many parameters

Answer: B. no information flows between time steps

Option notes:

- A: No.
- C: Weights are shared here.
- D: Sharing reduces them.

Takeaway: Sharing a rule ≠ sharing context.

Source: Lecture 8, slide 9

SEQ A36. [TF · Easy]

Sentiment classification of a review is a many-to-one task.

Answer: True

Why: Process each word, one prediction after the last.

Takeaway: Many in, one out.

Source: Lecture 8 (Sequence modelling)

SEQ A37. [BLANK · Hard]

Poll 1: 'Video frame → action label' (processing frames over time to one label) is **-to-** (answer like many-to-one).

Answer: many-to-one
(Also accepted: many to one)

Takeaway: Many frames, one label.

Source: Lecture 8, slide 18

SEQ A38. [TF · Medium]

A trigram model predicts the next word using the previous two words.

Answer: True

Why: History = first 2 words of each 3-word window.

Takeaway: n − 1 words of context.

Source: Lecture 8 (Sequence modelling)

SEQ A39. [TF · Hard]

In skip-gram, backpropagation updates the centre embedding v_c and the output embeddings W_out.

Answer: True

Why: Lecture 8 slide 57.

Takeaway: CBOW updates all context embeddings jointly.

Source: Lecture 8 (Sequence modelling)

SEQ A40. [BLANKS · Easy · calculation]

Key points sheet: x = 2, w_x = 0.5, h_prev = 0.4, w_h = 0.25, no bias, tanh with tanh(1.1) = 0.800 supplied. Sum ___; new state ___.

(1) sum = ____
(2) h = ____

Answer: sum = 1.1; h = 0.800

Working:

- 0.5 × 2 + 0.25 × 0.4 = 1.1
- tanh(1.1) = 0.800

Takeaway: The sum is not the answer: apply tanh.

Source: Book 2 (Exam examples), Q239, p. 34

SEQ A41. [BLANKS · Medium · calculation]

x = 1, w_x = 0.8, h_prev = 0.5, w_h = −0.4, bias 0.1. Sum ___; h = tanh(sum) = ___ (given tanh(0.7) = 0.604).

(1) sum = ____
(2) h = ____

Answer: sum = 0.7; h = 0.604

Working:

- 0.8 − 0.2 + 0.1 = 0.7

Takeaway: Bias only if given.

Source: Book 2 (Exam examples), Q240, p. 34

SEQ A42. [BLANKS · Hard · calculation]

Two steps: x1 = 1, x2 = 0.5, w_x = w_h = 0.5, h0 = 0, tanh. h1 = ___, h2 = ___ (3 d.p.; tanh 0.5 = 0.462, tanh 0.481 = 0.447).

(1) h1 = ____
(2) h2 = ____

Answer: h1 = 0.462; h2 = 0.447

Working:

- s1 = 0.5 → h1 = 0.462
- s2 = 0.25 + 0.231 = 0.481 → h2 = 0.447

Takeaway: Feed h1 into step 2.

Source: Book 2 (Exam examples), Q241, p. 34

SEQ A43. [MCQ · Easy]

A student reports h = 1.1 for the lecture example (x = 2, w_x = 0.5, h_prev = 0.4, w_h = 0.25). The mistake is:

A. forgot to apply the activation
B. added a bias
C. used ReLU
D. wrong weights

Answer: A. forgot to apply the activation

Option notes:

- B: No bias was added.
- C: ReLU(1.1) is also 1.1, but the task says tanh.
- D: The weights give 1.1 correctly.

Takeaway: Pre-activation ≠ hidden state.

Source: Book 2 (Exam examples), Q242, p. 34

SEQ A44. [MCQ · Medium · calculation]

(Poll 2) A dependency crosses 30 steps; each multiplies the gradient by 0.8.

A. ≈ 0.8; lower the learning rate
B. ≈ 0.24; clip gradients
C. ≈ 24; normalise inputs
D. ≈ 0.001; use an LSTM/GRU

Answer: D. ≈ 0.001; use an LSTM/GRU

Option notes:

- A: 0.8 is one step only.
- B: 0.8 × 30 is not how it compounds; clipping fixes exploding.
- C: That is 0.8 × 30.

Takeaway: $0.8^{30} = 0.0012$: vanishing.

Source: Book 2 (Exam examples), Q253, p. 35

SEQ A45. [BLANKS · Medium · calculation]

If each step multiplies by 1.2 over 30 steps the factor is ≈ ___ (integer), a case of ___ gradients, fixed by ___.

(1) factor = ____
(2) type = ____
(3) fix = ____

Answer: factor = 237; type = exploding; fix = gradient clipping

Takeaway: $1.2^{30} \approx 237$.

Source: Book 2 (Exam examples), Q254, p. 35

SEQ A46. [MATCH · Medium]

(Poll 1) Match each task to its sequence structure.

1. ECG → diagnosis
2. initial pose → motion
3. speech → text

A. one-to-one
B. many-to-one
C. many-to-many
D. one-to-many

Answer: 1 → B, 2 → D, 3 → C

Working:

- ECG → diagnosis → many-to-one
- initial pose → motion → one-to-many
- speech → text → many-to-many

Takeaway: Also: sensor history → failure and video → action label are many-to-one; seed → music is one-to-many.

Source: Book 2 (Exam examples), Q255, p. 35

SEQ A47. [MATCH · Easy]

Match the task to its structure.

1. image captioning
2. sentiment classification
3. machine translation

A. many-to-one
B. one-to-many
C. one-to-one
D. many-to-many

Answer: 1 → B, 2 → A, 3 → D

Working:

- image captioning → one-to-many
- sentiment classification → many-to-one
- machine translation → many-to-many

Takeaway: Caption, sentiment, translate.

Source: Book 2 (Exam examples), Q256, p. 35

SEQ A48. [MCQ · Medium]

Order the RNN training steps.

A. initialise h0 → update → process → BPTT
B. BPTT → initialise h0 → process → update
C. initialise h0 → process and accumulate loss → BPTT → update parameters
D. process → update → BPTT → initialise h0

Answer: C. initialise h0 → process and accumulate loss → BPTT → update parameters

Option notes:

- A: Update comes last.
- B: You need a forward pass before backprop.
- D: h0 comes first.

Takeaway: Sum losses, backprop once, update shared parameters.

Source: Book 2 (Exam examples), Q257, p. 35

SEQ A49. [BLANKS · Easy · calculation]

Gate example: candidate [0.8, 0.5, −0.6], gate [1.0, 0.1, 0.7]. Output (elementwise product):

(1) 1st = ____
(2) 2nd = ____
(3) 3rd = ____

Answer: 1st = 0.8; 2nd = 0.05; 3rd = -0.42

Takeaway: Elementwise ⊙.

Source: Book 2 (Exam examples), Q258, p. 36

SEQ A50. [MULTI · Medium]

Limits of RNNs listed on the slides: (Select ALL that apply.)

A. cannot handle variable length
B. slow sequential computation
C. encoding bottleneck
D. weak long-range memory

Answer: B. slow sequential computation; C. encoding bottleneck; D. weak long-range memory

Option notes:

- A: Incorrect: RNNs handle variable length
- B: Correct:
- C: Correct:
- D: Correct:

Takeaway: Motivates attention.

Source: Book 2 (Exam examples), Q259, p. 36

SEQ A51. [BLANKS · Hard · calculation]

word | P(w|+) | P(w|−)
heartwarming | 0.27 | 0.13
delightful | 0.23 | 0.13
slow | 0.12 | 0.29
repetitive | 0.08 | 0.29
charming | 0.27 | 0.08
dull | 0.04 | 0.08

Priors P(+) = 0.55, P(−) = 0.45 (smoothed likelihoods).

'... heartwarming and delightful, but ... slow and repetitive.' S+ = ___; S− = ___ (6 d.p.); P(+|text) = ___ (2 d.p.); class ___.

(1) S+ = ____
(2) S− = ____
(3) P(+|text) = ____
(4) class = ____

Answer: S+ = 0.000328; S− = 0.000640; P(+|text) = 0.34; class = negative

Working:

- S+ = 0.55 × 0.27 × 0.23 × 0.12 × 0.08 = 0.000328
- S− = 0.45 × 0.13 × 0.13 × 0.29 × 0.29 = 0.000640
- 0.000328/0.000968 = 0.34 → negative

Takeaway: Multiply prior by each word likelihood.

Source: Book 2 (Exam examples), Q260, p. 36

SEQ A52. [MCQ · Medium · calculation]

word | P(w|+) | P(w|−)
heartwarming | 0.27 | 0.13
delightful | 0.23 | 0.13
slow | 0.12 | 0.29
repetitive | 0.08 | 0.29
charming | 0.27 | 0.08
dull | 0.04 | 0.08

Priors P(+) = 0.55, P(−) = 0.45 (smoothed likelihoods).

(Poll 3) 'heartwarming and delightful, but slow.'

A. Negative: slow has the largest negative likelihood
B. Cannot decide until normalised
C. Tie
D. Positive: S+ ≈ 0.00410 > S− ≈ 0.00221

Answer: D. Positive: S+ ≈ 0.00410 > S− ≈ 0.00221

Option notes:

- A: Compare whole products, not one word.
- B: Normalising divides both by the same Z.
- C: The products differ.

Takeaway: Compare S+ and S−.

Source: Book 2 (Exam examples), Q261, p. 36

SEQ A53. [BLANKS · Hard · calculation]

word | P(w|+) | P(w|−)
heartwarming | 0.27 | 0.13
delightful | 0.23 | 0.13
slow | 0.12 | 0.29
repetitive | 0.08 | 0.29
charming | 0.27 | 0.08
dull | 0.04 | 0.08

Priors P(+) = 0.55, P(−) = 0.45 (smoothed likelihoods).

'dull at times, but ... heartwarming, delightful, and charming.' S+ = ___; S− = ___ (3 s.f., e.g. 1.23e-4); P(+) = ___ (2 d.p.).

(1) S+ = ____
(2) S− = ____
(3) P(+) = ____

Answer: S+ = 3.69e-4; S− = 4.87e-5; P(+) = 0.88

Working:

- S+ = $0.55(0.04)(0.27)(0.23)(0.27) = 3.69 \times 10^{-4}$
- S− = $0.45(0.08)(0.13)(0.13)(0.08) = 4.87 \times 10^{-5}$
- P(+) = 3.69/(3.69 + 0.487) = 0.88

Takeaway: Positive.

Source: Book 2 (Exam examples), Q262, p. 36

SEQ A54. [BLANKS · Medium · calculation]

word | count in + ($T_+ = 20$) | count in − ($T_- = 18$) | P_s(w|+) | P_s(w|−)
heartwarming | 6 | 2 | 7/26 ≈ 0.27 | 3/24 ≈ 0.13
delightful | 5 | 2 | 6/26 ≈ 0.23 | 3/24 ≈ 0.13
slow | 2 | 6 | 3/26 ≈ 0.12 | 7/24 ≈ 0.29
repetitive | 1 | 6 | 2/26 ≈ 0.08 | 7/24 ≈ 0.29
charming | 6 | 1 | 7/26 ≈ 0.27 | 2/24 ≈ 0.08
dull | 0 | 1 | 1/26 ≈ 0.04 | 2/24 ≈ 0.08

Lecture 8: 11 positive and 9 negative reviews → P(+) = 0.55, P(−) = 0.45; |V| = 6.

'heartwarming' occurs 6 times in positive reviews, T+ = 20, |V| = 6. Unsmoothed P = ___; add-one smoothed P = ___ (2 d.p.).

(1) unsmoothed = ____
(2) smoothed = ____

Answer: unsmoothed = 0.30; smoothed = 0.27

Working:

- 6/20 = 0.30
- 7/26 = 0.27

Takeaway: (count + 1)/(T + |V|).

Source: Book 2 (Exam examples), Q263, p. 36

SEQ A55. [BLANKS · Medium · calculation]

'dull' never occurs in positive reviews (T+ = 20, |V| = 6). Smoothed P(dull|+) = ___ (2 d.p.). Without smoothing one unseen word makes the whole product ___.

(1) P(dull|+) = ____
(2) product = ____

Answer: P(dull|+) = 0.04; product = 0

Takeaway: Laplace avoids zeros.

Source: Book 2 (Exam examples), Q264, p. 36

SEQ A56. [BLANKS · Hard · calculation]

Spam filter: P(spam) = 0.4; P(free|spam) = 0.3, P(free|ham) = 0.05; P(meeting|spam) = 0.02, P(meeting|ham) = 0.2. Email 'free meeting': S_spam = ___, S_ham = ___, P(spam|email) = ___ (2 d.p.).

(1) S_spam = ____
(2) S_ham = ____
(3) P(spam|email) = ____

Answer: S_spam = 0.0024; S_ham = 0.006; P(spam|email) = 0.29

Working:

- 0.4(0.3)(0.02) = 0.0024
- 0.6(0.05)(0.2) = 0.006
- 0.0024/0.0084 = 0.29

Takeaway: Ham wins.

Source: Book 2 (Exam examples), Q265, p. 36

SEQ A57. [BLANK · Easy]

The 'naive' assumption is that words are ___ given the class.

Answer: conditionally independent
(Also accepted: independent, conditionally independent of each other)

Takeaway: Naive = conditional independence.

Source: Book 2 (Exam examples), Q266, p. 36

SEQ A58. [TF · Medium]

To decide the Naive Bayes class you must first divide by P(text).

Answer: False

Why: P(text) is the same for both classes.

Takeaway: Compare unnormalised scores.

Source: Book 2 (Exam examples), Q267, p. 36

SEQ A59. [BLANK · Easy · calculation]

Corpus of five 'Mando protects ...' sentences: Grogu 3, villagers 1, civilians 1. Trigram P(Grogu | Mando protects) = ___.

Answer: 3/5
(Also accepted: 0.6)

Takeaway: C(h, w)/C(h).

Source: Book 2 (Exam examples), Q268, p. 36

SEQ A60. [BLANKS · Medium · calculation]

Corpus 'I like AI', 'I like cats', 'you like AI'. Bigram P(AI|like) = ___; P(like|I) = ___.

(1) P(AI|like) = ____
(2) P(like|I) = ____

Answer: P(AI|like) = 2/3; P(like|I) = 1

Takeaway: Count, divide.

Source: Book 2 (Exam examples), Q269, p. 37

SEQ A61. [BLANK · Easy]

An n-gram model is a Markov chain of order ___.

Answer: n-1
(Also accepted: n − 1, n minus 1)

Takeaway: Previous n − 1 items.

Source: Book 2 (Exam examples), Q270, p. 37

SEQ A62. [MULTI · Medium]

Limits of n-gram models: (Select ALL that apply.)

A. no notion of semantic similarity
B. they memorise local patterns

C. they cannot be trained from counts
D. fixed context window

Answer: A. no notion of semantic similarity; B. they memorise local patterns; D. fixed context window

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: counting is exactly how they are trained
- D: Correct:

Takeaway: Motivates embeddings.

Source: Book 2 (Exam examples), Q271, p. 37

SEQ A63. [BLANK · Easy]

Vocabulary {he, is, singing, she, dancing, stage}: the one-hot vector for 'she' is ___.

Answer: [0, 0, 0, 1, 0, 0]
(Also accepted: 0,0,0,1,0,0, 000100, (0,0,0,1,0,0))

Takeaway: Position 4.

Source: Book 2 (Exam examples), Q272, p. 37

SEQ A64. [MCQ · Easy]

one-hot(droid), with droid → index 2, multiplied by the embedding matrix E returns:

A. row 2 of E (droid's embedding)
B. the whole matrix E
C. column 2 of E
D. the scalar 2

Answer: A. row 2 of E (droid's embedding)

Option notes:

- B: Only one row survives.
- C: Row vector × matrix selects a row.
- D: IDs are names, not sizes.

Takeaway: Embedding lookup = row selection.

Source: Book 2 (Exam examples), Q273, p. 37

SEQ A65. [MULTI · Medium]

Drawbacks of one-hot vectors: (Select ALL that apply.)

A. sparse/inefficient for large vocabularies
B. no similarity between words
C. they need training
D. all pairs equally distant

Answer: A. sparse/inefficient for large vocabularies; B. no similarity between words; D. all pairs equally distant

Option notes:

- A: Correct:
- B: Correct:

- C: Incorrect: one-hot needs no training
- D: Correct:

Takeaway: Embeddings fix these.

Source: Book 2 (Exam examples), Q274, p. 37

SEQ A66. [MATCH · Easy]

Match the word2vec model.

1. CBOW
2. Skip-gram

A. centre word predicts context words
B. context words predict the centre word

Answer: 1 → B, 2 → A

Working:

- CBOW → context words predict the centre word
- Skip-gram → centre word predicts context words

Takeaway: CBOW in, skip-gram out.

Source: Book 2 (Exam examples), Q275, p. 37

SEQ A67. [BLANKS · Medium]

CBOW averages the context embeddings into h, scores s = W_out h, applies ___, and uses loss ___.

(1) function = ____
(2) loss = ____

Answer: function = softmax; loss = -log p_c

Takeaway: Softmax + negative log.

Source: Book 2 (Exam examples), Q276, p. 37

SEQ A68. [BLANK · Easy]

v(Paris) − v(France) + v(Italy) ≈ v(___).

Answer: Rome

Takeaway: Offsets encode relations.

Source: Book 2 (Exam examples), Q277, p. 37

SEQ A69. [BLANK · Easy]

Similarity between word vectors is measured with ___ similarity.

Answer: cosine

Takeaway: Angle, not length.

Source: Book 2 (Exam examples), Q278, p. 37

SEQ A70. [MCQ · Easy]

'My grandson loved it! So much fun!' as a bag of words (after cleaning):

A. {loved:1, fun:1}
B. {my:1, grandson:1, loved:1, it:1, so:1, much:1, fun:1}
C. [my, grandson, loved, it, so, much, fun] in order
D. {my grandson:1, loved it:1, so much fun:1}

Answer: B. {my:1, grandson:1, loved:1, it:1, so:1, much:1, fun:1}

Option notes:

- A: No words removed in the slide example.
- C: Bag of words ignores order.
- D: Those are phrases, not words.

Takeaway: Counts only.

Source: Book 2 (Exam examples), Q279, p. 37

SEQ A71. [TF · Easy]

Bag-of-words keeps word order.

Answer: False

Why: It keeps counts only.

Takeaway: Order lost.

Source: Book 2 (Exam examples), Q280, p. 37

SEQ A72. [BLANK · Easy]

Unknown words in an n-gram model are handled by the token ___.

Answer:
(Also accepted: UNK, , unk)

Takeaway: Out-of-vocabulary token.

Source: Book 2 (Exam examples), Q281, p. 37

SEQ A73. [BLANK · Easy]

Firth: 'You shall know a word by the ___ it keeps.'

Answer: company

Takeaway: Distributional hypothesis.

Source: Book 2 (Exam examples), Q282, p. 37

SEQ A74. [BLANK · Easy · calculation]

RNN: x = 3, w_x = 0.2, h_prev = −0.5, w_h = 0.6, no bias, tanh(0.3) = 0.291 supplied. New hidden state ___.

Answer: 0.291

Takeaway: Sum 0.6 − 0.3 = 0.3.

Source: Book 2 (Exam examples), Mixed practice paper Q24, p. 55

SEQ A75. [BLANKS · Medium · calculation]

Naive Bayes: P(+) = P(−) = 0.5; P(great|+) = 0.2, P(great|−) = 0.05; P(boring|+) = 0.02, P(boring|−) = 0.1. 'great but boring': predicted class ___; P(+|text) = ___ (3 d.p.).

(1) class = ____
(2) P(+|text) = ____

Answer: class = negative; P(+|text) = 0.444

Takeaway: S+ = 0.002, S− = 0.0025.

Source: Book 2 (Exam examples), Mixed practice paper Q26, p. 55

SEQ A76. [MATCH · Easy]

Match the text representation.

1. CBOW
2. skip-gram
3. bag-of-words
4. one-hot

A. context predicts centre
B. centre predicts context
C. word counts, order ignored
D. single 1 in a |V|-length vector

Answer: 1 → A, 2 → B, 3 → C, 4 → D

Working:

- CBOW → context predicts centre
- skip-gram → centre predicts context
- bag-of-words → word counts, order ignored
- one-hot → single 1 in a |V|-length vector

Takeaway: Four representations.

Source: Book 2 (Exam examples), Mixed practice paper Q27, p. 55

SEQ A77. [BLANK · Easy · calculation]

Corpus 'I like AI. I like cats. You like AI.' P(AI | like) = ___.

Answer: 2/3
(Also accepted: 0.667, 0.67)

Takeaway: C(like AI)/C(like).

Source: Book 3 (AI from zero), Practice 22.1, p. 90

SEQ A78. [BLANKS · Medium · calculation]

A word is seen 3 times in a class with T = 10, |V| = 5. Laplace P = ___; an unseen word gets ___ (3 d.p.).

(1) seen = ____
(2) unseen = ____

Answer: seen = 0.267; unseen = 0.067

Takeaway: 4/15; 1/15.

Source: Book 3 (AI from zero), Practice 22.2, p. 91

SEQ A79. [BLANK · Medium · calculation]

Priors 0.4 spam / 0.6 ham; P(free|spam) = 0.3, P(free|ham) = 0.05. Email 'free'. P(spam) = ___.

Answer: 0.8

Takeaway: 0.12/(0.12 + 0.03).

Source: Book 3 (AI from zero), Practice 22.3, p. 91

SEQ A80. [BLANK · Easy]

A trigram model is a Markov chain of order ___.

Answer: 2

Takeaway: n − 1.

Source: Book 3 (AI from zero), Practice 22.4, p. 91

SEQ A81. [MCQ · Easy]

Speech-to-text is which structure?

A. one-to-one
B. many-to-many
C. many-to-one
D. one-to-many

Answer: B. many-to-many

Option notes:

- A: No.
- C: Output is a sequence.
- D: Input is a sequence.

Takeaway: Sequence in, sequence out.

Source: Book 3 (AI from zero), Practice 22.5, p. 91

SEQ A82. [BLANK · Medium · calculation]

Character-trigram probability of 'excellence' is a product of how many factors?

Answer: 10

Takeaway: One per character.

Source: Book 3 (AI from zero), Chapter 22, p. 88

SEQ A83. [MULTI · Medium]

Ways to avoid zero n-gram probabilities: (Select ALL that apply.)

A. back-off to a shorter n-gram
B. delete unseen words from the test text
C. add-one smoothing
D. linear interpolation $\lambda_3 P_tri + \lambda_2 P_bi + \lambda_1 P_uni$ (λ's sum to 1)

Answer: A. back-off to a shorter n-gram; C. add-one smoothing; D. linear interpolation $\lambda_3 P_tri + \lambda_2 P_bi + \lambda_1 P_uni$ (λ's sum to 1)

Option notes:

- A: Correct:
- B: Incorrect: use instead
- C: Correct:
- D: Correct:

Takeaway: Smoothing.

Source: Book 3 (AI from zero), Chapter 22, p. 88

SEQ A84. [MCQ · Easy]

Indexing gives Jedi→0, Sith→1, droid→2. Which is true?

A. IDs measure similarity
B. Sith is half of droid
C. IDs are learned embeddings
D. IDs name words; Sith is not 'less than' droid

Answer: D. IDs name words; Sith is not 'less than' droid

Option notes:

- A: No.
- B: IDs are not magnitudes.
- C: Embeddings are learned vectors.

Takeaway: Names, not sizes.

Source: Book 3 (AI from zero), Chapter 22, p. 88

SEQ A85. [BLANKS · Medium]

'Luke and Leia board the ship with a droid. ... a Jedi warns them about the Sith.' gives ___ tokens, ___ unique.

(1) tokens = _____
(2) unique = _____

Answer: tokens = 16; unique = 14

Takeaway: 'a' and 'the' repeat.

Source: Book 1 (Compiled lecture notes), Chapter 8, p. 43

SEQ A86. [BLANK · Medium]

v(woman) − v(man) + v(uncle) ≈ v(___).

Answer: aunt

Takeaway: Cosine 0.824.

Source: Book 1 (Compiled lecture notes), Chapter 8, p. 43

SEQ A87. [MCQ · Hard]

CBOW forward pass, in order:

A. softmax → average → look up → loss
B. one-hot → loss → softmax
C. centre embedding → s = W_out v_c → softmax → loss −log p_o
D. look up context embeddings → average into h → s = W_out h → softmax → loss −log p_c

Answer: D. look up context embeddings → average into h → s = W_out h → softmax → loss −log p_c

Option notes:

- A: Order wrong.
- B: No.
- C: That is skip-gram.

Takeaway: Context in, centre out.

Source: Book 1 (Compiled lecture notes), Chapter 8, p. 43

SEQ A88. [BLANK · Medium · calculation]

Corpus counts: C(the) = 1,000, C(the cat) = 20. Bigram P(cat | the) = ___.

Answer: 0.02

Takeaway: C(h, w)/C(h).

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 23: n-gram models

SEQ A89. [BLANK · Medium · calculation]

Add-one smoothing: C(the cat) = 0, C(the) = 1,000, vocabulary 9,000 words. P(cat | the) = ___ (write as a fraction).

Answer: 1/10000
(Also accepted: 0.0001, 1e-4)

Takeaway: (0 + 1)/(1,000 + 9,000).

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 23: smoothing

SEQ A90. [MCQ · Hard]

(Extra, not in your slides) Perplexity of a language model on test text is:

A. higher is better
B. the inverse probability of the text, normalised by length: lower is better
C. the accuracy of next-word prediction
D. the number of words

Answer: B. the inverse probability of the text, normalised by length: lower is better

Option notes:

- A: Lower is better.
- C: Related but not the definition.
- D: No.

Takeaway: Perplexity = 2^(cross-entropy).

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 23: evaluation

SEQ A91. [MULTI · Medium]

AIMA on word embeddings: (Select ALL that apply.)

A. they are one-hot vectors
B. similar words get similar vectors
C. they are learned low-dimensional dense vectors
D. vector offsets capture relations (e.g. Paris − France + Italy ≈ Rome)

Answer: B. similar words get similar vectors; C. they are learned low-dimensional dense vectors; D. vector offsets capture relations (e.g. Paris − France + Italy ≈ Rome)

Option notes:

- A: Incorrect: one-hot is the sparse alternative
- B: Correct:
- C: Correct:

- D: Correct:

Takeaway: Distributional semantics.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 24: word embeddings

SEQ A92. [MCQ · Hard]

(Extra, not in your slides) A bidirectional RNN is useful for tagging because:

A. it generates text faster
B. it removes vanishing gradients
C. each word's label can depend on both left and right context
D. it halves the parameters

Answer: C. each word's label can depend on both left and right context

Option notes:

- A: It needs the whole input.
- B: Not by itself.
- D: It doubles them.

Takeaway: Two directions, concatenated states.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 24: RNNs for NLP

SEQ A93. [BLANKS · Medium · calculation]

Current input = 3, input weight = 0.2, previous state = 0.5, recurrent weight = 0.4, no bias. Sum = ___; with tanh(0.8) supplied as 0.664, new state = ___.

(1) sum = ____
(2) new state = ____

Answer: sum = 0.8; new state = 0.664

Working:

- 0.6 + 0.2 = 0.8
- tanh(0.8) = 0.664 (supplied)

Takeaway: 0.2 × 3 + 0.4 × 0.5.

Source: Key points about a simple RNN hidden-state update

SEQ A94. [BLANKS · Medium · calculation]

Current input = 1, input weight = 0.6, previous state = −0.5, recurrent weight = 0.4, bias 0.1. Sum = ___; with tanh(0.5) supplied as 0.462, new state = ___.

(1) sum = ____
(2) new state = ____

Answer: sum = 0.5; new state = 0.462

Working:

- 0.6 − 0.2 + 0.1 = 0.5
- tanh(0.5) = 0.462

Takeaway: Bias is included because it is specified.

Source: Key points about a simple RNN hidden-state update

SEQ A95. [MCQ · Hard]

Current input = 2 (input weight 0.5), previous state = 0.4 (recurrent weight 0.25). A student computes 0.5 × 0.4 + 0.25 × 2 = 0.7. The mistake is:

A. mixing up the input and the previous state (each must use its own weight)
B. forgetting the bias
C. nothing: 0.7 is right
D. applying tanh

Answer: A. mixing up the input and the previous state (each must use its own weight)

Option notes:

- B: No bias was given.
- C: Correct sum is 1.1.
- D: tanh is required.

Takeaway: Keep current input and previous state distinct.

Source: Key points about a simple RNN hidden-state update

SEQ A96. [TF · Easy]

The sum before the activation is the new hidden state.

Answer: False

Why: Apply the activation (use the supplied tanh lookup).

Takeaway: Sum ≠ state.

Source: Key points about a simple RNN hidden-state update

SEQ A97. [TF · Easy]

The previous hidden state carries information from earlier steps of the sequence.

Answer: True

Why: That is how an RNN remembers.

Takeaway: Memory = previous state.

Source: Key points about a simple RNN hidden-state update

SEQ A98. [MCQ · Medium]

When should a bias be added in the simple RNN update?

A. only for tanh
B. never
C. only if a bias is specified in the question
D. always add 1

Answer: C. only if a bias is specified in the question

Option notes:

- A: Activation does not decide it.
- B: Add it when given.
- D: Never assume a bias.

Takeaway: Bias only if specified.

Source: Key points about a simple RNN hidden-state update

SEQ A99. [BLANK · Medium · calculation]

Previous state 0, input 1, input weight 1, recurrent weight 0.9, no bias; tanh(1) supplied as 0.76159. Give the new state to 2 decimal places.

Answer: 0.76

Takeaway: Round only the final result.

Source: Key points about a simple RNN hidden-state update

Part B: calculation drills (128)

SEQ B1. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input $x_t = 2$, input weight $w_x = 0.1$, previous state $h_{t-1} = 0.4$, recurrent weight $w_h = 0.8$, no bias. Lookup: tanh(0.520) = 0.478. What is h_t?

A. 0.520
B. 0.478
C. 0.927
D. 0.197

Answer: B. 0.478

Working:

- Sum = 0.1×2 + 0.8×0.4 = 0.520.
- h_t = tanh(0.520) = 0.478 (from the lookup).

Option notes:

- A: That is the pre-activation sum; apply tanh (use the supplied lookup).
- C: Swapped input and previous state (multiplied by the wrong weights).
- D: Ignored the previous hidden state.

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B2. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input $x_t = 3$, input weight $w_x = -0.3$, previous state $h_{t-1} = 0.8$, recurrent weight $w_h = -0.5$, bias $b = 0.1$. Lookup: tanh(-1.200) = -0.834. What is h_t?

A. -0.716
B. -1.200
C. -0.834
D. -0.927

Answer: C. -0.834

Working:

- Sum = -0.3×3 + -0.5×0.8 + 0.1 = -1.200.
- h_t = tanh(-1.200) = -0.834 (from the lookup).

Option notes:

- A: Ignored the previous hidden state.
- B: That is the pre-activation sum; apply tanh (use the supplied lookup).
- D: Swapped input and previous state (multiplied by the wrong weights).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B3. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input $x_t = 0.5$, input weight $w_x = 0.2$, previous state $h_{t-1} = 0.4$, recurrent weight $w_h = 0.5$, no bias. Lookup: tanh(0.300) = 0.291. What is h_t?

A. 0.291
B. 0.300
C. 0.100
D. 0.319

Answer: A. 0.291

Working:

- Sum = 0.2×0.5 + 0.5×0.4 = 0.300.
- h_t = tanh(0.300) = 0.291 (from the lookup).

Option notes:

- B: That is the pre-activation sum; apply tanh (use the supplied lookup).
- C: Ignored the previous hidden state.
- D: Swapped input and previous state (multiplied by the wrong weights).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B4. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input $x_t = -1$, input weight $w_x = 0.5$, previous state $h_{t-1} = 0.7$, recurrent weight $w_h = -0.5$, bias $b = 0.1$. Lookup: tanh(-0.750) = -0.635. What is h_t?

A. -0.462
B. 0.740
C. -0.750
D. -0.635

Answer: D. -0.635

Working:

- Sum = 0.5×-1 + -0.5×0.7 + 0.1 = -0.750.
- h_t = tanh(-0.750) = -0.635 (from the lookup).

Option notes:

- A: Ignored the previous hidden state.
- B: Swapped input and previous state (multiplied by the wrong weights).
- C: That is the pre-activation sum; apply tanh (use the supplied lookup).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B5. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input $x_t = 3$, input weight $w_x = 0.5$, previous state $h_{t-1} = 0.5$, recurrent weight $w_h = 0.5$, no bias. Lookup: tanh(1.750) = 0.941. What is h_t?

A. 0.941
B. 1.750

C. 0.905
D. 1.083

Answer: A. 0.941

Working:

- Sum = 0.5×3 + 0.5×0.5 = 1.750.
- h_t = tanh(1.750) = 0.941 (from the lookup).

Option notes:

- B: That is the pre-activation sum; apply tanh (use the supplied lookup).
- C: Ignored the previous hidden state.
- D: Arithmetic slip: recompute each step carefully.

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B6. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input x_t = 1, input weight w_x = 0.5, previous state h_{t-1} = 0, recurrent weight w_h = 0.25, bias b = -0.2. Lookup: tanh(0.300) = 0.291. What is h_t?

A. 0.291
B. 0.462
C. 0.050
D. 0.300

Answer: A. 0.291

Working:

- Sum = 0.5×1 + 0.25×0 + -0.2 = 0.300.
- h_t = tanh(0.300) = 0.291 (from the lookup).

Option notes:

- B: Ignored the previous hidden state.
- C: Swapped input and previous state (multiplied by the wrong weights).
- D: That is the pre-activation sum; apply tanh (use the supplied lookup).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B7. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input x_t = 1, input weight w_x = 0.2, previous state h_{t-1} = -0.1, recurrent weight w_h = 0.8, bias b = 0.1. Lookup: tanh(0.220) = 0.217. What is h_t?

A. 0.220
B. 0.706
C. 0.217
D. 0.197

Answer: C. 0.217

Working:

- Sum = 0.2×1 + 0.8×-0.1 + 0.1 = 0.220.
- h_t = tanh(0.220) = 0.217 (from the lookup).

Option notes:

- A: That is the pre-activation sum; apply tanh (use the supplied lookup).
- B: Swapped input and previous state (multiplied by the wrong weights).
- D: Ignored the previous hidden state.

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B8. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input $x_t = 1$, input weight $w_x = 0.2$, previous state $h_{t-1} = 0.1$, recurrent weight $w_h = -0.5$, no bias. Lookup: tanh(0.150) = 0.149. What is h_t?

A. 0.197
B. 0.149
C. -0.446
D. 0.150

Answer: B. 0.149

Working:

- Sum = 0.2×1 + -0.5×0.1 = 0.150.
- h_t = tanh(0.150) = 0.149 (from the lookup).

Option notes:

- A: Ignored the previous hidden state.
- C: Swapped input and previous state (multiplied by the wrong weights).
- D: That is the pre-activation sum; apply tanh (use the supplied lookup).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B9. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input $x_t = 1$, input weight $w_x = 0.1$, previous state $h_{t-1} = -0.2$, recurrent weight $w_h = 0.8$, no bias. Lookup: tanh(-0.060) = -0.060. What is h_t?

A. 0.100
B. -0.051
C. 0.653
D. -0.060

Answer: D. -0.060

Working:

- Sum = 0.1×1 + 0.8×-0.2 = -0.060.
- h_t = tanh(-0.060) = -0.060 (from the lookup).

Option notes:

- A: Ignored the previous hidden state.
- B: Arithmetic slip: recompute each step carefully.
- C: Swapped input and previous state (multiplied by the wrong weights).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B10. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input $x_t = 2$, input weight $w_x = 0.1$, previous state $h_{t-1} = 0.7$, recurrent weight $w_h = 0.25$, bias $b = -0.2$. Lookup: $\tanh(0.175) = 0.173$. What is h_t?

A. 0.175
B. 0.173
C. 0.354
D. 0.197

Answer: B. 0.173

Working:

- Sum = 0.1×2 + 0.25×0.7 + -0.2 = 0.175.
- $h_t = \tanh(0.175) = 0.173$ (from the lookup).

Option notes:

- A: That is the pre-activation sum; apply tanh (use the supplied lookup).
- C: Swapped input and previous state (multiplied by the wrong weights).
- D: Ignored the previous hidden state.

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B11. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input $x_t = 0.5$, input weight $w_x = 0.4$, previous state $h_{t-1} = -0.1$, recurrent weight $w_h = -0.5$, no bias. Lookup: $\tanh(0.250) = 0.245$. What is h_t?

A. 0.197
B. -0.282
C. 0.245
D. 0.250

Answer: C. 0.245

Working:

- Sum = 0.4×0.5 + -0.5×-0.1 = 0.250.
- $h_t = \tanh(0.250) = 0.245$ (from the lookup).

Option notes:

- A: Ignored the previous hidden state.
- B: Swapped input and previous state (multiplied by the wrong weights).
- D: That is the pre-activation sum; apply tanh (use the supplied lookup).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B12. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input $x_t = 1$, input weight $w_x = 0.5$, previous state $h_{t-1} = -0.4$, recurrent weight $w_h = -0.5$, bias $b = -0.2$. Lookup: $\tanh(0.500) = 0.462$. What is h_t?

A. 0.500
B. 0.462
C. 0.531
D. -0.716

Answer: B. 0.462

Working:

- Sum = 0.5×1 + -0.5×-0.4 + -0.2 = 0.500.
- h_t = tanh(0.500) = 0.462 (from the lookup).

Option notes:

- A: That is the pre-activation sum; apply tanh (use the supplied lookup).
- C: Arithmetic slip: recompute each step carefully.
- D: Swapped input and previous state (multiplied by the wrong weights).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B13. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input x_t = 1, input weight w_x = 0.2, previous state h_{t-1} = -0.3, recurrent weight w_h = -0.5, no bias. Lookup: tanh(0.350) = 0.336. What is h_t?

A. 0.197
B. 0.350
C. -0.508
D. 0.336

Answer: D. 0.336

Working:

- Sum = 0.2×1 + -0.5×-0.3 = 0.350.
- h_t = tanh(0.350) = 0.336 (from the lookup).

Option notes:

- A: Ignored the previous hidden state.
- B: That is the pre-activation sum; apply tanh (use the supplied lookup).
- C: Swapped input and previous state (multiplied by the wrong weights).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B14. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input x_t = -1, input weight w_x = -0.3, previous state h_{t-1} = 0.7, recurrent weight w_h = 0.25, bias b = 0.1. Lookup: tanh(0.575) = 0.519. What is h_t?

A. -0.345
B. 0.519
C. 0.291
D. 0.575

Answer: B. 0.519

Working:

- Sum = -0.3×-1 + 0.25×0.7 + 0.1 = 0.575.
- h_t = tanh(0.575) = 0.519 (from the lookup).

Option notes:

- A: Swapped input and previous state (multiplied by the wrong weights).
- C: Ignored the previous hidden state.

- D: That is the pre-activation sum; apply tanh (use the supplied lookup).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B15. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1})$. Current input $x_t = 2$, input weight $w_x = 0.1$, previous state $h_{t-1} = 0.4$, recurrent weight $w_h = 0.5$, no bias. Lookup: tanh(0.400) = 0.380. What is h_t?

A. 0.778
B. 0.400
C. 0.380
D. 0.197

Answer: C. 0.380

Working:

- Sum = 0.1×2 + 0.5×0.4 = 0.400.
- h_t = tanh(0.400) = 0.380 (from the lookup).

Option notes:

- A: Swapped input and previous state (multiplied by the wrong weights).
- B: That is the pre-activation sum; apply tanh (use the supplied lookup).
- D: Ignored the previous hidden state.

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B16. [MCQ · Medium · calculation]

Simple RNN: $h_t = \tanh(w_x \cdot x_t + w_h \cdot h_{t-1} + b)$. Current input $x_t = 2$, input weight $w_x = 0.5$, previous state $h_{t-1} = 0$, recurrent weight $w_h = -0.5$, bias $b = 0.3$. Lookup: tanh(1.300) = 0.862. What is h_t?

A. 0.862
B. 0.762
C. -0.604
D. 1.300

Answer: A. 0.862

Working:

- Sum = 0.5×2 + -0.5×0 + 0.3 = 1.300.
- h_t = tanh(1.300) = 0.862 (from the lookup).

Option notes:

- B: Ignored the previous hidden state.
- C: Swapped input and previous state (multiplied by the wrong weights).
- D: That is the pre-activation sum; apply tanh (use the supplied lookup).

Takeaway: The sum before activation is NOT the hidden state. Keep x_t and h_{t-1} (and their weights) distinct.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B17. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.3$, $w_h = -0.4$. Inputs $x_1 = -1$, $x_2 = 2$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = -0.291$; $h_2 = 0.615$

Working:

- $h_1 = \tanh(0.3\times-1 + -0.4\times0) = \tanh(-0.3) = -0.2913$.
- $h_2 = \tanh(0.3\times2 + -0.4\times-0.2913) = \tanh(0.7165) = 0.6148$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B18. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.5$, $w_h = -0.4$. Inputs $x_1 = 2$, $x_2 = 0$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = 0.762$; $h_2 = -0.296$

Working:

- $h_1 = \tanh(0.5\times2 + -0.4\times0) = \tanh(1) = 0.7616$.
- $h_2 = \tanh(0.5\times0 + -0.4\times0.7616) = \tanh(-0.3046) = -0.2956$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B19. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.3$, $w_h = -0.4$. Inputs $x_1 = 1$, $x_2 = 2$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = 0.291$; $h_2 = 0.449$

Working:

- $h_1 = \tanh(0.3\times1 + -0.4\times0) = \tanh(0.3) = 0.2913$.
- $h_2 = \tanh(0.3\times2 + -0.4\times0.2913) = \tanh(0.4835) = 0.4490$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B20. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.5$, $w_h = -0.4$. Inputs $x_1 = 1$, $x_2 = 0$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = 0.462$; $h_2 = -0.183$

Working:

- $h_1 = \tanh(0.5\times1 + -0.4\times0) = \tanh(0.5) = 0.4621$.
- $h_2 = \tanh(0.5\times0 + -0.4\times0.4621) = \tanh(-0.1848) = -0.1828$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B21. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.3$, $w_h = -0.4$. Inputs $x_1 = 2$, $x_2 = 0$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = 0.537$; $h_2 = -0.212$

Working:

- $h_1 = \tanh(0.3\times2 + -0.4\times0) = \tanh(0.6) = 0.5370$.
- $h_2 = \tanh(0.3\times0 + -0.4\times0.5370) = \tanh(-0.2148) = -0.2116$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B22. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.8$, $w_h = 0.5$. Inputs $x_1 = 2$, $x_2 = 1$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = 0.922$; $h_2 = 0.851$

Working:

- $h_1 = \tanh(0.8\times2 + 0.5\times0) = \tanh(1.6) = 0.9217$.
- $h_2 = \tanh(0.8\times1 + 0.5\times0.9217) = \tanh(1.2608) = 0.8513$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B23. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.5$, $w_h = -0.4$. Inputs $x_1 = -1$, $x_2 = 1$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = -0.462$; $h_2 = 0.595$

Working:

- $h_1 = \tanh(0.5\times-1 + -0.4\times0) = \tanh(-0.5) = -0.4621$.
- $h_2 = \tanh(0.5\times1 + -0.4\times-0.4621) = \tanh(0.6848) = 0.5947$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B24. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.5$, $w_h = -0.4$. Inputs $x_1 = 1$, $x_2 = 2$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = 0.462$; $h_2 = 0.672$

Working:

- $h_1 = \tanh(0.5\times1 + -0.4\times0) = \tanh(0.5) = 0.4621$.
- $h_2 = \tanh(0.5\times2 + -0.4\times0.4621) = \tanh(0.8152) = 0.6724$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B25. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.5$, $w_h = 0.5$. Inputs $x_1 = -1$, $x_2 = 0$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = -0.462$; $h_2 = -0.227$

Working:

- $h_1 = \tanh(0.5\times-1 + 0.5\times0) = \tanh(-0.5) = -0.4621$.
- $h_2 = \tanh(0.5\times0 + 0.5\times-0.4621) = \tanh(-0.2311) = -0.2270$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B26. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.3$, $w_h = 0.6$. Inputs $x_1 = 2$, $x_2 = 0$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = 0.537$; $h_2 = 0.312$

Working:

- $h_1 = \tanh(0.3\times2 + 0.6\times0) = \tanh(0.6) = 0.5370$.
- $h_2 = \tanh(0.3\times0 + 0.6\times0.5370) = \tanh(0.3222) = 0.3115$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B27. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.5$, $w_h = 0.5$. Inputs $x_1 = -1$, $x_2 = 2$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = _____
(2) h_2 = _____

Answer: $h_1 = -0.462$; $h_2 = 0.646$

Working:

- $h_1 = \tanh(0.5\times-1 + 0.5\times0) = \tanh(-0.5) = -0.4621$.
- $h_2 = \tanh(0.5\times2 + 0.5\times-0.4621) = \tanh(0.7689) = 0.6463$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B28. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.8$, $w_h = 0.5$. Inputs $x_1 = 2$, $x_2 = 0$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = ____
(2) h_2 = ____

Answer: $h_1 = 0.922$; $h_2 = 0.431$

Working:

- $h_1 = \tanh(0.8\times2 + 0.5\times0) = \tanh(1.6) = 0.9217$.
- $h_2 = \tanh(0.8\times0 + 0.5\times0.9217) = \tanh(0.4608) = 0.4308$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B29. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.3$, $w_h = 0.6$. Inputs $x_1 = 1$, $x_2 = 0$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = ____
(2) h_2 = ____

Answer: $h_1 = 0.291$; $h_2 = 0.173$

Working:

- $h_1 = \tanh(0.3\times1 + 0.6\times0) = \tanh(0.3) = 0.2913$.
- $h_2 = \tanh(0.3\times0 + 0.6\times0.2913) = \tanh(0.1748) = 0.1730$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B30. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.5$, $w_h = -0.4$. Inputs $x_1 = 1$, $x_2 = 1$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = ____
(2) h_2 = ____

Answer: $h_1 = 0.462$; $h_2 = 0.305$

Working:

- $h_1 = \tanh(0.5\times1 + -0.4\times0) = \tanh(0.5) = 0.4621$.
- $h_2 = \tanh(0.5\times1 + -0.4\times0.4621) = \tanh(0.3152) = 0.3051$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B31. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.8$, $w_h = 0.5$. Inputs $x_1 = 1$, $x_2 = 1$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = ____
(2) h_2 = ____

Answer: $h_1 = 0.664$; $h_2 = 0.812$

Working:

- $h_1 = \tanh(0.8\times1 + 0.5\times0) = \tanh(0.8) = 0.6640$.
- $h_2 = \tanh(0.8\times1 + 0.5\times0.6640) = \tanh(1.1320) = 0.8117$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B32. [BLANKS · Hard · calculation]

Scalar RNN $h_t = \tanh(w_x x_t + w_h h_{t-1})$, $h_0 = 0$, $w_x = 0.8$, $w_h = 0.6$. Inputs $x_1 = 1$, $x_2 = 1$. Give h_1 and h_2 (3 dp; use a calculator for tanh).

(1) h_1 = ____
(2) h_2 = ____

Answer: $h_1 = 0.664$; $h_2 = 0.833$

Working:

- $h_1 = \tanh(0.8\times1 + 0.6\times0) = \tanh(0.8) = 0.6640$.
- $h_2 = \tanh(0.8\times1 + 0.6\times0.6640) = \tanh(1.1984) = 0.8332$.

Takeaway: The SAME weights are reused at every step; only x_t and h_t change.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B33. [BLANKS · Medium · calculation]

A dependency crosses 50 recurrent steps and the gradient is multiplied by 1.1 at each step. Give 1.1^50 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 117; behaviour = exploding

Working:

- 1.1^50 = 117.4.
- Factors > 1 → gradient grows → exploding. Fix: gradient clipping.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B34. [BLANKS · Medium · calculation]

A dependency crosses 50 recurrent steps and the gradient is multiplied by 0.9 at each step. Give 0.9^50 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 0.00515; behaviour = vanishing

Working:

- 0.9^50 = 0.005154.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B35. [BLANKS · Medium · calculation]

A dependency crosses 10 recurrent steps and the gradient is multiplied by 0.7 at each step. Give 0.7^10 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 0.0282; behaviour = vanishing

Working:

- 0.7^10 = 0.02825.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B36. [BLANKS · Medium · calculation]

A dependency crosses 50 recurrent steps and the gradient is multiplied by 0.5 at each step. Give 0.5^50 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 8.88e-16; behaviour = vanishing

Working:

- 0.5^50 = 8.882e-16.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B37. [BLANKS · Medium · calculation]

A dependency crosses 30 recurrent steps and the gradient is multiplied by 0.9 at each step. Give 0.9^30 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 0.0424; behaviour = vanishing

Working:

- 0.9^30 = 0.04239.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B38. [BLANKS · Medium · calculation]

A dependency crosses 20 recurrent steps and the gradient is multiplied by 1.1 at each step. Give 1.1^20 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 6.73; behaviour = exploding

Working:

- 1.1^20 = 6.727.
- Factors > 1 → gradient grows → exploding. Fix: gradient clipping.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B39. [BLANKS · Medium · calculation]

A dependency crosses 50 recurrent steps and the gradient is multiplied by 0.7 at each step. Give 0.7^50 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 1.8e-8; behaviour = vanishing

Working:

- 0.7^50 = 1.798e-8.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B40. [BLANKS · Medium · calculation]

A dependency crosses 10 recurrent steps and the gradient is multiplied by 1.2 at each step. Give 1.2^10 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 6.19; behaviour = exploding

Working:

- 1.2^10 = 6.192.
- Factors > 1 → gradient grows → exploding. Fix: gradient clipping.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B41. [BLANKS · Medium · calculation]

A dependency crosses 50 recurrent steps and the gradient is multiplied by 1.1 at each step. Give 1.1^50 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 117; behaviour = exploding

Working:

- 1.1^50 = 117.4.
- Factors > 1 → gradient grows → exploding. Fix: gradient clipping.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B42. [BLANKS · Medium · calculation]

A dependency crosses 10 recurrent steps and the gradient is multiplied by 0.9 at each step. Give 0.9^10 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 0.349; behaviour = vanishing

Working:

- 0.9^10 = 0.3487.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B43. [BLANKS · Medium · calculation]

A dependency crosses 50 recurrent steps and the gradient is multiplied by 0.7 at each step. Give 0.7^50 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 1.8e-8; behaviour = vanishing

Working:

- 0.7^50 = 1.798e-8.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B44. [BLANKS · Medium · calculation]

A dependency crosses 20 recurrent steps and the gradient is multiplied by 0.7 at each step. Give 0.7^20 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 0.000798; behaviour = vanishing

Working:

- 0.7^20 = 0.0007979.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B45. [BLANKS · Medium · calculation]

A dependency crosses 30 recurrent steps and the gradient is multiplied by 0.8 at each step. Give 0.8^30 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 0.00124; behaviour = vanishing

Working:

- 0.8^30 = 0.001238.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B46. [BLANKS · Medium · calculation]

A dependency crosses 30 recurrent steps and the gradient is multiplied by 1.2 at each step. Give 1.2^30 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 237; behaviour = exploding

Working:

- 1.2^30 = 237.4.
- Factors > 1 → gradient grows → exploding. Fix: gradient clipping.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B47. [BLANKS · Medium · calculation]

A dependency crosses 10 recurrent steps and the gradient is multiplied by 0.8 at each step. Give 0.8^10 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 0.107; behaviour = vanishing

Working:

- 0.8^10 = 0.1074.
- Repeated factors < 1 → gradient shrinks → vanishing. Fix: LSTM/GRU, better activation, initialisation.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B48. [BLANKS · Medium · calculation]

A dependency crosses 50 recurrent steps and the gradient is multiplied by 1.1 at each step. Give 1.1^50 (scientific notation or decimal, 3 significant figures) and the behaviour (vanishing or exploding).

(1) factor = ____
(2) behaviour = ____

Answer: factor = 117; behaviour = exploding

Working:

- 1.1^50 = 117.4.
- Factors > 1 → gradient grows → exploding. Fix: gradient clipping.

Takeaway: Lecture 8 poll: $0.8^{30} \approx 0.001$ → vanishing → use an LSTM or GRU.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B49. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [0, 0.1, 0.8], gate g_t = [0.7, 0.4, 0.8]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = 0; h[2] = 0.04; h[3] = 0.64

Working:

- 0 × 0.7 = 0
- 0.1 × 0.4 = 0.04
- 0.8 × 0.8 = 0.64
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B50. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.9, 0, -0.2], gate g_t = [0.1, 0.8, 0.9]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.09; h[2] = 0; h[3] = -0.18

Working:

- -0.9 × 0.1 = -0.09
- 0 × 0.8 = 0
- -0.2 × 0.9 = -0.18
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B51. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [0.2, -0.1, -1], gate g_t = [0.8, 0.4, 0.7]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = 0.16; h[2] = -0.04; h[3] = -0.7

Working:

- 0.2 × 0.8 = 0.16
- -0.1 × 0.4 = -0.04
- -1 × 0.7 = -0.7
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B52. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.8, 0, -0.3], gate g_t = [0.2, 0.5, 0]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.16; h[2] = 0; h[3] = 0

Working:

- -0.8 × 0.2 = -0.16
- 0 × 0.5 = 0
- -0.3 × 0 = 0
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B53. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.2, -0.3, 0.2], gate g_t = [0.1, 0.6, 0.6]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.02; h[2] = -0.18; h[3] = 0.12

Working:

- -0.2 × 0.1 = -0.02
- -0.3 × 0.6 = -0.18
- 0.2 × 0.6 = 0.12
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B54. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [0.9, -0.5, 0], gate g_t = [0.5, 0.5, 0.2]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = 0.45; h[2] = -0.25; h[3] = 0

Working:

- 0.9 × 0.5 = 0.45
- -0.5 × 0.5 = -0.25
- 0 × 0.2 = 0
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B55. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.4, -0.1, 0.6], gate g_t = [0.5, 0.3, 0]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.2; h[2] = -0.03; h[3] = 0

Working:

- -0.4 × 0.5 = -0.2
- -0.1 × 0.3 = -0.03
- 0.6 × 0 = 0
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B56. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.7, 0.2, -0.9], gate g_t = [0.7, 0.9, 0.5]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.49; h[2] = 0.18; h[3] = -0.45

Working:

- -0.7 × 0.7 = -0.49
- 0.2 × 0.9 = 0.18
- -0.9 × 0.5 = -0.45
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B57. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.6, -0.5, 0.9], gate g_t = [0.1, 0.1, 0.6]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.06; h[2] = -0.05; h[3] = 0.54

Working:

- -0.6 × 0.1 = -0.06
- -0.5 × 0.1 = -0.05
- 0.9 × 0.6 = 0.54

- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B58. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [0, 0.6, 0.2], gate g_t = [0.3, 0.6, 0.6]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = 0; h[2] = 0.36; h[3] = 0.12

Working:

- 0 × 0.3 = 0
- 0.6 × 0.6 = 0.36
- 0.2 × 0.6 = 0.12
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B59. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.7, -0.9, 0.4], gate g_t = [0.7, 0.7, 0.3]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.49; h[2] = -0.63; h[3] = 0.12

Working:

- -0.7 × 0.7 = -0.49
- -0.9 × 0.7 = -0.63
- 0.4 × 0.3 = 0.12
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B60. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [0.5, -0.5, -0.2], gate g_t = [0.1, 0.5, 1]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = 0.05; h[2] = -0.25; h[3] = -0.2

Working:

- 0.5 × 0.1 = 0.05

- -0.5 × 0.5 = -0.25
- -0.2 × 1 = -0.2
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B61. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.3, 0.6, -0.3], gate g_t = [0.1, 0.4, 1]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.03; h[2] = 0.24; h[3] = -0.3

Working:

- -0.3 × 0.1 = -0.03
- 0.6 × 0.4 = 0.24
- -0.3 × 1 = -0.3
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B62. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [-0.8, -0.2, -0.9], gate g_t = [0.4, 0.1, 0.8]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = -0.32; h[2] = -0.02; h[3] = -0.72

Working:

- -0.8 × 0.4 = -0.32
- -0.2 × 0.1 = -0.02
- -0.9 × 0.8 = -0.72
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B63. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [0.5, -0.6, -0.9], gate g_t = [0.6, 0.8, 0]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = 0.3; h[2] = -0.48; h[3] = 0

Working:

- 0.5 × 0.6 = 0.3
- -0.6 × 0.8 = -0.48
- -0.9 × 0 = 0
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B64. [BLANKS · Easy · calculation]

A learned gate scales the candidate hidden state element-wise: $h_t = g_t \odot \tilde{h}_t$. Candidate $\tilde{h}_t$ = [0.9, 0.4, -0.6], gate g_t = [0.5, 1, 0.6]. Give h_t.

(1) h[1] = ____
(2) h[2] = ____
(3) h[3] = ____

Answer: h[1] = 0.45; h[2] = 0.4; h[3] = -0.36

Working:

- 0.9 × 0.5 = 0.45
- 0.4 × 1 = 0.4
- -0.6 × 0.6 = -0.36
- (Lecture 8 example: [0.8, 0.5, −0.6] ⊙ [1.0, 0.1, 0.7] = [0.8, 0.05, −0.42].)

Takeaway: Gate values lie in [0, 1] (sigmoid): 1 lets information through, 0 blocks it.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B65. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 6 | 7
delightful | 4 | 8
slow | 0 | 0
repetitive | 3 | 3
charming | 8 | 3
dull | 5 | 3

Training data: 5 positive and 6 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences T_+ = 26, T_- = 24. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, repetitive, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.458; class = negative

Working:

- Priors: P(+) = 5/11 = 0.4545, P(−) = 0.5455.
- heartwarming: P(w|+) = (6+1)/(26+6) = 0.2188; P(w|−) = (7+1)/(24+6) = 0.2667
- slow: P(w|+) = (0+1)/(26+6) = 0.0313; P(w|−) = (0+1)/(24+6) = 0.0333
- repetitive: P(w|+) = (3+1)/(26+6) = 0.1250; P(w|−) = (3+1)/(24+6) = 0.1333
- dull: P(w|+) = (5+1)/(26+6) = 0.1875; P(w|−) = (3+1)/(24+6) = 0.1333
- S_+ = 0.4545 × 0.2188 × 0.0313 × 0.1250 × 0.1875 = 7.2826e-5.
- S_- = 0.5455 × 0.2667 × 0.0333 × 0.1333 × 0.1333 = 8.6195e-5.

- $P(+|text) = S_+/(S_+ + S_-) = 0.4580 \rightarrow$ negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B66. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 4
delightful | 2 | 1
slow | 2 | 2
repetitive | 3 | 6
charming | 7 | 5
dull | 4 | 6

Training data: 12 positive and 6 negative reviews (priors from these counts); vocabulary $|V| = 6$; total word occurrences $T_+ = 20$, $T_- = 24$. Using add-one smoothing $P(w|c) = (count + 1)/(T_c + |V|)$, classify a review containing: heartwarming, slow, repetitive, charming. Give $P(+|text)$ (3 dp) and the predicted class (positive/negative).

(1) $P(+|text)$ = ____
(2) class = ____

Answer: $P(+|text) = 0.618$; class = positive

Working:

- Priors: $P(+) = 12/18 = 0.6667$, $P(-) = 0.3333$.
- heartwarming: $P(w|+) = (2+1)/(20+6) = 0.1154$; $P(w|-) = (4+1)/(24+6) = 0.1667$
- slow: $P(w|+) = (2+1)/(20+6) = 0.1154$; $P(w|-) = (2+1)/(24+6) = 0.1000$
- repetitive: $P(w|+) = (3+1)/(20+6) = 0.1538$; $P(w|-) = (6+1)/(24+6) = 0.2333$
- charming: $P(w|+) = (7+1)/(20+6) = 0.3077$; $P(w|-) = (5+1)/(24+6) = 0.2000$
- $S_+ = 0.6667 \times 0.1154 \times 0.1154 \times 0.1538 \times 0.3077 = 4.2015e\text{-}4$.
- $S_- = 0.3333 \times 0.1667 \times 0.1000 \times 0.2333 \times 0.2000 = 2.5926e\text{-}4$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.6184 \rightarrow$ positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B67. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 8 | 1
delightful | 2 | 3
slow | 6 | 1
repetitive | 4 | 4
charming | 4 | 7
dull | 6 | 3

Training data: 10 positive and 13 negative reviews (priors from these counts); vocabulary $|V| = 6$; total word occurrences $T_+ = 30$, $T_- = 19$. Using add-one smoothing $P(w|c) = (count + 1)/(T_c + |V|)$, classify a review containing: heartwarming, delightful, slow, dull. Give $P(+|text)$ (3 dp) and the predicted class (positive/negative).

(1) $P(+|text)$ = ____
(2) class = ____

Answer: P(+|text) = 0.787; class = positive

Working:

- Priors: P(+) = 10/23 = 0.4348, P(−) = 0.5652.
- heartwarming: P(w|+) = (8+1)/(30+6) = 0.2500; P(w|−) = (1+1)/(19+6) = 0.0800
- delightful: P(w|+) = (2+1)/(30+6) = 0.0833; P(w|−) = (3+1)/(19+6) = 0.1600
- slow: P(w|+) = (6+1)/(30+6) = 0.1944; P(w|−) = (1+1)/(19+6) = 0.0800
- dull: P(w|+) = (6+1)/(30+6) = 0.1944; P(w|−) = (3+1)/(19+6) = 0.1600
- S_+ = 0.4348 × 0.2500 × 0.0833 × 0.1944 × 0.1944 = 3.4247e-4.
- S_- = 0.5652 × 0.0800 × 0.1600 × 0.0800 × 0.1600 = 9.2605e-5.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.7872 → positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B68. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 3 | 4
delightful | 3 | 2
slow | 4 | 7
repetitive | 7 | 2
charming | 8 | 4
dull | 1 | 6

Training data: 10 positive and 12 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences T_+ = 26, T_- = 25. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: slow, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.694; class = positive

Working:

- Priors: P(+) = 10/22 = 0.4545, P(−) = 0.5455.
- slow: P(w|+) = (4+1)/(26+6) = 0.1563; P(w|−) = (7+1)/(25+6) = 0.2581
- repetitive: P(w|+) = (7+1)/(26+6) = 0.2500; P(w|−) = (2+1)/(25+6) = 0.0968
- charming: P(w|+) = (8+1)/(26+6) = 0.2813; P(w|−) = (4+1)/(25+6) = 0.1613
- S_+ = 0.4545 × 0.1563 × 0.2500 × 0.2813 = 4.9938e-3.
- S_- = 0.5455 × 0.2581 × 0.0968 × 0.1613 = 2.1971e-3.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.6945 → positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B69. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 0 | 0
delightful | 4 | 1
slow | 6 | 6
repetitive | 7 | 0

charming | 8 | 8
dull | 2 | 4

Training data: 6 positive and 10 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 27$, $T_- = 19$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, repetitive, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.487; class = negative

Working:

- Priors: P(+) = 6/16 = 0.3750, P(−) = 0.6250.
- heartwarming: P(w|+) = (0+1)/(27+6) = 0.0303; P(w|−) = (0+1)/(19+6) = 0.0400
- repetitive: P(w|+) = (7+1)/(27+6) = 0.2424; P(w|−) = (0+1)/(19+6) = 0.0400
- charming: P(w|+) = (8+1)/(27+6) = 0.2727; P(w|−) = (8+1)/(19+6) = 0.3600
- dull: P(w|+) = (2+1)/(27+6) = 0.0909; P(w|−) = (4+1)/(19+6) = 0.2000
- $S_+ = 0.3750 \times 0.0303 \times 0.2424 \times 0.2727 \times 0.0909 = 6.8301e\text{-}5$.
- $S_- = 0.6250 \times 0.0400 \times 0.0400 \times 0.3600 \times 0.2000 = 7.2000e\text{-}5$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.4868 \rightarrow$ negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B70. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 5 | 4
delightful | 2 | 5
slow | 5 | 4
repetitive | 2 | 1
charming | 3 | 7
dull | 3 | 8

Training data: 11 positive and 15 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 20$, $T_- = 29$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: delightful, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.401; class = negative

Working:

- Priors: P(+) = 11/26 = 0.4231, P(−) = 0.5769.
- delightful: P(w|+) = (2+1)/(20+6) = 0.1154; P(w|−) = (5+1)/(29+6) = 0.1714
- repetitive: P(w|+) = (2+1)/(20+6) = 0.1154; P(w|−) = (1+1)/(29+6) = 0.0571
- charming: P(w|+) = (3+1)/(20+6) = 0.1538; P(w|−) = (7+1)/(29+6) = 0.2286
- $S_+ = 0.4231 \times 0.1154 \times 0.1154 \times 0.1538 = 8.6657e\text{-}4$.
- $S_- = 0.5769 \times 0.1714 \times 0.0571 \times 0.2286 = 1.2918e\text{-}3$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.4015 \rightarrow$ negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B71. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 4 | 2
delightful | 4 | 6
slow | 5 | 0
repetitive | 8 | 8
charming | 6 | 6
dull | 8 | 8

Training data: 15 positive and 12 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 35$, $T_- = 30$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: repetitive, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.491; class = negative

Working:

- Priors: P(+) = 15/27 = 0.5556, P(−) = 0.4444.
- repetitive: P(w|+) = (8+1)/(35+6) = 0.2195; P(w|−) = (8+1)/(30+6) = 0.2500
- dull: P(w|+) = (8+1)/(35+6) = 0.2195; P(w|−) = (8+1)/(30+6) = 0.2500
- $S_+ = 0.5556 \times 0.2195 \times 0.2195 = 2.6770e\text{-}2$.
- $S_- = 0.4444 \times 0.2500 \times 0.2500 = 2.7778e\text{-}2$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.4908 \rightarrow$ negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B72. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 8 | 7
delightful | 0 | 7
slow | 0 | 8
repetitive | 7 | 4
charming | 3 | 8
dull | 4 | 5

Training data: 5 positive and 5 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 22$, $T_- = 39$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.236; class = negative

Working:

- Priors: P(+) = 5/10 = 0.5000, P(−) = 0.5000.
- heartwarming: P(w|+) = (8+1)/(22+6) = 0.3214; P(w|−) = (7+1)/(39+6) = 0.1778
- slow: P(w|+) = (0+1)/(22+6) = 0.0357; P(w|−) = (8+1)/(39+6) = 0.2000
- charming: P(w|+) = (3+1)/(22+6) = 0.1429; P(w|−) = (8+1)/(39+6) = 0.2000
- dull: P(w|+) = (4+1)/(22+6) = 0.1786; P(w|−) = (5+1)/(39+6) = 0.1333

- S_+ = 0.5000 × 0.3214 × 0.0357 × 0.1429 × 0.1786 = 1.4642e-4.
- S_- = 0.5000 × 0.1778 × 0.2000 × 0.2000 × 0.1333 = 4.7407e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.2360 → negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B73. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 6
delightful | 0 | 2
slow | 6 | 8
repetitive | 3 | 3
charming | 0 | 7
dull | 3 | 1

Training data: 14 positive and 9 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences T_+ = 14, T_- = 27. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: delightful, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.226; class = negative

Working:

- Priors: P(+) = 14/23 = 0.6087, P(−) = 0.3913.
- delightful: P(w|+) = (0+1)/(14+6) = 0.0500; P(w|−) = (2+1)/(27+6) = 0.0909
- repetitive: P(w|+) = (3+1)/(14+6) = 0.2000; P(w|−) = (3+1)/(27+6) = 0.1212
- charming: P(w|+) = (0+1)/(14+6) = 0.0500; P(w|−) = (7+1)/(27+6) = 0.2424
- S_+ = 0.6087 × 0.0500 × 0.2000 × 0.0500 = 3.0435e-4.
- S_- = 0.3913 × 0.0909 × 0.1212 × 0.2424 = 1.0453e-3.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.2255 → negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B74. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 7 | 5
delightful | 0 | 2
slow | 1 | 8
repetitive | 5 | 0
charming | 4 | 8
dull | 0 | 3

Training data: 9 positive and 5 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences T_+ = 17, T_- = 26. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.869; class = positive

Working:

- Priors: P(+) = 9/14 = 0.6429, P(−) = 0.3571.
- heartwarming: P(w|+) = (7+1)/(17+6) = 0.3478; P(w|−) = (5+1)/(26+6) = 0.1875
- slow: P(w|+) = (1+1)/(17+6) = 0.0870; P(w|−) = (8+1)/(26+6) = 0.2813
- repetitive: P(w|+) = (5+1)/(17+6) = 0.2609; P(w|−) = (0+1)/(26+6) = 0.0313
- charming: P(w|+) = (4+1)/(17+6) = 0.2174; P(w|−) = (8+1)/(26+6) = 0.2813
- S_+ = 0.6429 × 0.3478 × 0.0870 × 0.2609 × 0.2174 = 1.1027e-3.
- S_- = 0.3571 × 0.1875 × 0.2813 × 0.0313 × 0.2813 = 1.6553e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.8695 → positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B75. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 6
delightful | 6 | 0
slow | 3 | 4
repetitive | 1 | 0
charming | 3 | 5
dull | 0 | 1

Training data: 13 positive and 15 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences T_+ = 15, T_- = 16. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.214; class = negative

Working:

- Priors: P(+) = 13/28 = 0.4643, P(−) = 0.5357.
- heartwarming: P(w|+) = (2+1)/(15+6) = 0.1429; P(w|−) = (6+1)/(16+6) = 0.3182
- charming: P(w|+) = (3+1)/(15+6) = 0.1905; P(w|−) = (5+1)/(16+6) = 0.2727
- S_+ = 0.4643 × 0.1429 × 0.1905 = 1.2634e-2.
- S_- = 0.5357 × 0.3182 × 0.2727 = 4.6488e-2.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.2137 → negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B76. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 7 | 8
delightful | 3 | 7
slow | 1 | 0
repetitive | 4 | 0
charming | 0 | 1
dull | 6 | 6

Training data: 15 positive and 14 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 21$, $T_- = 22$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: slow, repetitive, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.857; class = positive

Working:

- Priors: P(+) = 15/29 = 0.5172, P(−) = 0.4828.
- slow: P(w|+) = (1+1)/(21+6) = 0.0741; P(w|−) = (0+1)/(22+6) = 0.0357
- repetitive: P(w|+) = (4+1)/(21+6) = 0.1852; P(w|−) = (0+1)/(22+6) = 0.0357
- charming: P(w|+) = (0+1)/(21+6) = 0.0370; P(w|−) = (1+1)/(22+6) = 0.0714
- $S_+ = 0.5172 \times 0.0741 \times 0.1852 \times 0.0370 = 2.6279e\text{-}4$.
- $S_- = 0.4828 \times 0.0357 \times 0.0357 \times 0.0714 = 4.3983e\text{-}5$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.8566 \rightarrow$ positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B77. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 4 | 0
delightful | 3 | 4
slow | 3 | 8
repetitive | 3 | 6
charming | 8 | 5
dull | 5 | 3

Training data: 15 positive and 9 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 26$, $T_- = 26$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, slow, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.893; class = positive

Working:

- Priors: P(+) = 15/24 = 0.6250, P(−) = 0.3750.
- heartwarming: P(w|+) = (4+1)/(26+6) = 0.1563; P(w|−) = (0+1)/(26+6) = 0.0313
- slow: P(w|+) = (3+1)/(26+6) = 0.1250; P(w|−) = (8+1)/(26+6) = 0.2813
- charming: P(w|+) = (8+1)/(26+6) = 0.2813; P(w|−) = (5+1)/(26+6) = 0.1875
- dull: P(w|+) = (5+1)/(26+6) = 0.1875; P(w|−) = (3+1)/(26+6) = 0.1250
- $S_+ = 0.6250 \times 0.1563 \times 0.1250 \times 0.2813 \times 0.1875 = 6.4373e\text{-}4$.
- $S_- = 0.3750 \times 0.0313 \times 0.2813 \times 0.1875 \times 0.1250 = 7.7248e\text{-}5$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.8929 \rightarrow$ positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B78. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 2 | 2
delightful | 7 | 1
slow | 6 | 4
repetitive | 5 | 2
charming | 2 | 2
dull | 6 | 4

Training data: 8 positive and 6 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 28$, $T_- = 15$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, delightful, charming, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.521; class = positive

Working:

- Priors: P(+) = 8/14 = 0.5714, P(−) = 0.4286.
- heartwarming: P(w|+) = (2+1)/(28+6) = 0.0882; P(w|−) = (2+1)/(15+6) = 0.1429
- delightful: P(w|+) = (7+1)/(28+6) = 0.2353; P(w|−) = (1+1)/(15+6) = 0.0952
- charming: P(w|+) = (2+1)/(28+6) = 0.0882; P(w|−) = (2+1)/(15+6) = 0.1429
- dull: P(w|+) = (6+1)/(28+6) = 0.2059; P(w|−) = (4+1)/(15+6) = 0.2381
- $S_+ = 0.5714 \times 0.0882 \times 0.2353 \times 0.0882 \times 0.2059 = 2.1551e\text{-}4$.
- $S_- = 0.4286 \times 0.1429 \times 0.0952 \times 0.1429 \times 0.2381 = 1.9833e\text{-}4$.
- $P(+|text) = S_+/(S_+ + S_-) = 0.5208 \rightarrow$ positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c) \cdot \Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B79. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 7 | 6
delightful | 3 | 0
slow | 3 | 6
repetitive | 1 | 6
charming | 2 | 1
dull | 2 | 0

Training data: 6 positive and 7 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences $T_+ = 18$, $T_- = 19$. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: heartwarming, delightful, slow, charming. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.798; class = positive

Working:

- Priors: P(+) = 6/13 = 0.4615, P(−) = 0.5385.
- heartwarming: P(w|+) = (7+1)/(18+6) = 0.3333; P(w|−) = (6+1)/(19+6) = 0.2800

- delightful: P(w|+) = (3+1)/(18+6) = 0.1667; P(w|−) = (0+1)/(19+6) = 0.0400
- slow: P(w|+) = (3+1)/(18+6) = 0.1667; P(w|−) = (6+1)/(19+6) = 0.2800
- charming: P(w|+) = (2+1)/(18+6) = 0.1250; P(w|−) = (1+1)/(19+6) = 0.0800
- S_+ = 0.4615 × 0.3333 × 0.1667 × 0.1667 × 0.1250 = 5.3419e-4.
- S_- = 0.5385 × 0.2800 × 0.0400 × 0.2800 × 0.0800 = 1.3509e-4.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.7982 → positive.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B80. [BLANKS · Exam+ · calculation]

word | count in + reviews | count in − reviews
heartwarming | 3 | 5
delightful | 1 | 7
slow | 4 | 8
repetitive | 7 | 2
charming | 5 | 0
dull | 1 | 8

Training data: 15 positive and 10 negative reviews (priors from these counts); vocabulary |V| = 6; total word occurrences T_+ = 21, T_- = 30. Using add-one smoothing P(w|c) = (count + 1)/(T_c + |V|), classify a review containing: delightful, repetitive, dull. Give P(+|text) (3 dp) and the predicted class (positive/negative).

(1) P(+|text) = ____
(2) class = ____

Answer: P(+|text) = 0.345; class = negative

Working:

- Priors: P(+) = 15/25 = 0.6000, P(−) = 0.4000.
- delightful: P(w|+) = (1+1)/(21+6) = 0.0741; P(w|−) = (7+1)/(30+6) = 0.2222
- repetitive: P(w|+) = (7+1)/(21+6) = 0.2963; P(w|−) = (2+1)/(30+6) = 0.0833
- dull: P(w|+) = (1+1)/(21+6) = 0.0741; P(w|−) = (8+1)/(30+6) = 0.2500
- S_+ = 0.6000 × 0.0741 × 0.2963 × 0.0741 = 9.7546e-4.
- S_- = 0.4000 × 0.2222 × 0.0833 × 0.2500 = 1.8519e-3.
- P(+|text) = $S_+/(S_+ + S_-)$ = 0.3450 → negative.

Takeaway: Naive Bayes: $P(c|text) \propto P(c)\cdot\Pi_i P(x_i|c)$. Smoothing stops a zero count wiping out the product.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B81. [BLANKS · Easy · calculation]

A word appears 5 times among T = 38 word occurrences of the positive class; |V| = 12. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.132; add-one = 0.120

Working:

- Unsmoothed = 5/38 = 0.1316.
- Add-one = (5+1)/(38+12) = 0.1200. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B82. [BLANKS · Easy · calculation]

A word appears 4 times among T = 40 word occurrences of the positive class; |V| = 7. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.100; add-one = 0.106

Working:

- Unsmoothed = 4/40 = 0.1000.
- Add-one = (4+1)/(40+7) = 0.1064. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B83. [BLANKS · Easy · calculation]

A word appears 8 times among T = 19 word occurrences of the positive class; |V| = 8. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.421; add-one = 0.333

Working:

- Unsmoothed = 8/19 = 0.4211.
- Add-one = (8+1)/(19+8) = 0.3333. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B84. [BLANKS · Easy · calculation]

A word appears 0 times among T = 39 word occurrences of the positive class; |V| = 5. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.000; add-one = 0.023

Working:

- Unsmoothed = 0/39 = 0.0000.
- Add-one = (0+1)/(39+5) = 0.0227. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B85. [BLANKS · Easy · calculation]

A word appears 4 times among T = 28 word occurrences of the positive class; |V| = 11. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.143; add-one = 0.128

Working:

- Unsmoothed = 4/28 = 0.1429.
- Add-one = (4+1)/(28+11) = 0.1282. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B86. [BLANKS · Easy · calculation]

A word appears 6 times among T = 26 word occurrences of the positive class; |V| = 5. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.231; add-one = 0.226

Working:

- Unsmoothed = 6/26 = 0.2308.
- Add-one = (6+1)/(26+5) = 0.2258. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B87. [BLANKS · Easy · calculation]

A word appears 9 times among T = 35 word occurrences of the positive class; |V| = 6. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.257; add-one = 0.244

Working:

- Unsmoothed = 9/35 = 0.2571.
- Add-one = (9+1)/(35+6) = 0.2439. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B88. [BLANKS · Easy · calculation]

A word appears 7 times among T = 18 word occurrences of the positive class; |V| = 12. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.389; add-one = 0.267

Working:

- Unsmoothed = 7/18 = 0.3889.
- Add-one = (7+1)/(18+12) = 0.2667. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B89. [BLANKS · Easy · calculation]

A word appears 6 times among T = 36 word occurrences of the positive class; |V| = 7. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.167; add-one = 0.163

Working:

- Unsmoothed = 6/36 = 0.1667.
- Add-one = (6+1)/(36+7) = 0.1628. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B90. [BLANKS · Easy · calculation]

A word appears 4 times among T = 17 word occurrences of the positive class; |V| = 11. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.235; add-one = 0.179

Working:

- Unsmoothed = 4/17 = 0.2353.
- Add-one = (4+1)/(17+11) = 0.1786. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B91. [BLANKS · Easy · calculation]

A word appears 8 times among T = 40 word occurrences of the positive class; |V| = 5. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.200; add-one = 0.200

Working:

- Unsmoothed = 8/40 = 0.2000.
- Add-one = (8+1)/(40+5) = 0.2000. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B92. [BLANKS · Easy · calculation]

A word appears 8 times among T = 26 word occurrences of the positive class; |V| = 7. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.308; add-one = 0.273

Working:

- Unsmoothed = 8/26 = 0.3077.
- Add-one = (8+1)/(26+7) = 0.2727. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B93. [BLANKS · Easy · calculation]

A word appears 1 times among T = 33 word occurrences of the positive class; |V| = 5. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.030; add-one = 0.053

Working:

- Unsmoothed = 1/33 = 0.0303.
- Add-one = (1+1)/(33+5) = 0.0526. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B94. [BLANKS · Easy · calculation]

A word appears 7 times among T = 33 word occurrences of the positive class; |V| = 9. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.212; add-one = 0.190

Working:

- Unsmoothed = 7/33 = 0.2121.
- Add-one = (7+1)/(33+9) = 0.1905. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B95. [BLANKS · Easy · calculation]

A word appears 1 times among T = 18 word occurrences of the positive class; |V| = 8. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.056; add-one = 0.077

Working:

- Unsmoothed = 1/18 = 0.0556.
- Add-one = (1+1)/(18+8) = 0.0769. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B96. [BLANKS · Easy · calculation]

A word appears 5 times among T = 37 word occurrences of the positive class; |V| = 8. Give P(word|+) without smoothing and with add-one smoothing (3 dp).

(1) unsmoothed = ____
(2) add-one = ____

Answer: unsmoothed = 0.135; add-one = 0.133

Working:

- Unsmoothed = 5/37 = 0.1351.
- Add-one = (5+1)/(37+8) = 0.1333. (Lecture 8: heartwarming 6/20 = 0.30 vs 7/26 ≈ 0.27.)

Takeaway: Add-one adds 1 to the numerator and |V| to the denominator.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B97. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 3
villagers | 1
civilians | 4
the | 1
Din | 5

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(villagers | "Mando protects")?

Answer: 0.071

Working:

- C(h) = 3 + 1 + 4 + 1 + 5 = 14.
- P = 1/14 = 0.0714.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B98. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 4
villagers | 4
civilians | 2
the | 4
Din | 1

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(villagers | "Mando protects")?

Answer: 0.267

Working:

- C(h) = 4 + 4 + 2 + 4 + 1 = 15.
- P = 4/15 = 0.2667.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B99. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 3
villagers | 0
civilians | 2
the | 5
Din | 1

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(Din | "Mando protects")?

Answer: 0.091

Working:

- C(h) = 3 + 0 + 2 + 5 + 1 = 11.
- P = 1/11 = 0.0909.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B100. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 2
villagers | 2
civilians | 5
the | 3
Din | 2

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(the | "Mando protects")?

Answer: 0.214

Working:

- C(h) = 2 + 2 + 5 + 3 + 2 = 14.
- P = 3/14 = 0.2143.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B101. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 2
villagers | 0
civilians | 2
the | 5
Din | 4

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(Grogu | "Mando protects")?

Answer: 0.154

Working:

- C(h) = 2 + 0 + 2 + 5 + 4 = 13.

- P = 2/13 = 0.1538.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B102. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 1
villagers | 4
civilians | 3
the | 2
Din | 4

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(civilians | "Mando protects")?

Answer: 0.214

Working:

- C(h) = 1 + 4 + 3 + 2 + 4 = 14.
- P = 3/14 = 0.2143.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B103. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 5
villagers | 5
civilians | 1
the | 3
Din | 0

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(villagers | "Mando protects")?

Answer: 0.357

Working:

- C(h) = 5 + 5 + 1 + 3 + 0 = 14.
- P = 5/14 = 0.3571.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B104. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 1
villagers | 5
civilians | 3
the | 2
Din | 5

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(Din | "Mando protects")?

Answer: 0.313

Working:

- C(h) = 1 + 5 + 3 + 2 + 5 = 16.
- P = 5/16 = 0.3125.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B105. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 2
villagers | 1
civilians | 3
the | 2
Din | 0

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(the | "Mando protects")?

Answer: 0.250

Working:

- C(h) = 2 + 1 + 3 + 2 + 0 = 8.
- P = 2/8 = 0.2500.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B106. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 2
villagers | 3
civilians | 4
the | 2
Din | 4

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(Grogu | "Mando protects")?

Answer: 0.133

Working:

- C(h) = 2 + 3 + 4 + 2 + 4 = 15.
- P = 2/15 = 0.1333.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B107. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 2

villagers | 4
civilians | 0
the | 0
Din | 0

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(villagers | "Mando protects")?

Answer: 0.667

Working:

- C(h) = 2 + 4 + 0 + 0 + 0 = 6.
- P = 4/6 = 0.6667.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B108. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 1
villagers | 1
civilians | 5
the | 4
Din | 2

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(Din | "Mando protects")?

Answer: 0.154

Working:

- C(h) = 1 + 1 + 5 + 4 + 2 = 13.
- P = 2/13 = 0.1538.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B109. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 2
villagers | 4
civilians | 3
the | 2
Din | 5

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(civilians | "Mando protects")?

Answer: 0.188

Working:

- C(h) = 2 + 4 + 3 + 2 + 5 = 16.
- P = 3/16 = 0.1875.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B110. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 1
villagers | 1
civilians | 1
the | 2
Din | 2

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(Din | "Mando protects")?

Answer: 0.286

Working:

- C(h) = 1 + 1 + 1 + 2 + 2 = 7.
- P = 2/7 = 0.2857.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B111. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 3
villagers | 4
civilians | 1
the | 3
Din | 1

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(villagers | "Mando protects")?

Answer: 0.333

Working:

- C(h) = 3 + 4 + 1 + 3 + 1 = 12.
- P = 4/12 = 0.3333.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B112. [BLANK · Easy · calculation]

next word after 'Mando protects' | count
Grogu | 1
villagers | 1
civilians | 2
the | 2
Din | 3

In a training corpus, the history h = "Mando protects" is followed by the words in the table. Using P(w|h) = C(h, w)/C(h), what is P(villagers | "Mando protects")?

Answer: 0.111

Working:

- C(h) = 1 + 1 + 2 + 2 + 3 = 9.

- P = 1/9 = 0.1111.

Takeaway: N-grams only see a fixed short context and treat words as unrelated symbols.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B113. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | 0.44 | 0.82 | -0.8
1 | Sith | -0.39 | -0.87 | -0.21
2 | droid | -0.17 | -0.34 | 0.38
3 | ship | -0.32 | -0.69 | 0.88
4 | Luke | -0.26 | -0.58 | 0.3
5 | Leia | -0.22 | 0.55 | 0.58

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Leia" (6 digits separated by commas) and the result of onehot(Leia) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,0,0,0,1; embedding = -0.22,0.55,0.58

Working:

- ID of Leia = 5 → 1 at position 5.
- onehot × E selects row 5 of E: [-0.22, 0.55, 0.58].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B114. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.19 | 0.35 | -0.06
1 | Sith | 0.95 | -0.71 | 0.2
2 | droid | -0.06 | -0.13 | 0.03
3 | ship | -0.63 | -0.83 | 0.82
4 | Luke | 0.97 | 0.79 | 0.77
5 | Leia | 0.13 | -0.61 | 0.67

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Sith" (6 digits separated by commas) and the result of onehot(Sith) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,1,0,0,0,0; embedding = 0.95,-0.71,0.2

Working:

- ID of Sith = 1 → 1 at position 1.
- onehot × E selects row 1 of E: [0.95, -0.71, 0.2].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B115. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | 0.81 | -0.87 | -0.82
1 | Sith | 0.9 | 0.55 | 0.21
2 | droid | 0.75 | 0.01 | -0.62
3 | ship | -0.34 | -0.41 | 0.68
4 | Luke | 0.88 | 0.3 | 0.36
5 | Leia | -0.74 | -0.08 | -0.33

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Luke" (6 digits separated by commas) and the result of onehot(Luke) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,0,0,1,0; embedding = 0.88,0.3,0.36

Working:

- ID of Luke = 4 → 1 at position 4.
- onehot × E selects row 4 of E: [0.88, 0.3, 0.36].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B116. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | 0.6 | -0.69 | -0.85
1 | Sith | -0.4 | 0.73 | -0.29
2 | droid | -0.85 | 0.06 | -0.55
3 | ship | 0.66 | 0.11 | -0.72
4 | Luke | 0.49 | 0.32 | -0.49
5 | Leia | -0.01 | -0.67 | -0.92

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "droid" (6 digits separated by commas) and the result of onehot(droid) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,1,0,0,0; embedding = -0.85,0.06,-0.55

Working:

- ID of droid = 2 → 1 at position 2.
- onehot × E selects row 2 of E: [-0.85, 0.06, -0.55].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B117. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.08 | 0.79 | -0.73
1 | Sith | -0.86 | -0.24 | -0.29
2 | droid | 0.79 | 0.31 | 0.92
3 | ship | -0.51 | -0.75 | -0.62

4 | Luke | -0.28 | -0.83 | 0.88
5 | Leia | 0.73 | 0.16 | 0.1

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "droid" (6 digits separated by commas) and the result of onehot(droid) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,1,0,0,0; embedding = 0.79,0.31,0.92

Working:

- ID of droid = 2 → 1 at position 2.
- onehot × E selects row 2 of E: [0.79, 0.31, 0.92].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B118. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.96 | -0.98 | -0.39
1 | Sith | -0.96 | 0.41 | -0.76
2 | droid | 0.59 | -0.54 | 0.73
3 | ship | 0.2 | 0.83 | -0.26
4 | Luke | -0.91 | 0.6 | 0.03
5 | Leia | -0.87 | 0.39 | -0.88

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Jedi" (6 digits separated by commas) and the result of onehot(Jedi) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 1,0,0,0,0,0; embedding = -0.96,-0.98,-0.39

Working:

- ID of Jedi = 0 → 1 at position 0.
- onehot × E selects row 0 of E: [-0.96, -0.98, -0.39].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B119. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | 0.81 | -0.99 | -0.58
1 | Sith | 0.31 | -0.39 | -0.67
2 | droid | -0.94 | 0.05 | -0.62
3 | ship | -0.93 | -0.96 | 0.52
4 | Luke | 0.86 | 0.93 | -0.13
5 | Leia | -0.57 | -0.35 | -0.54

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "droid" (6 digits separated by commas) and the result of onehot(droid) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,1,0,0,0; embedding = -0.94,0.05,-0.62

Working:

- ID of droid = 2 → 1 at position 2.
- onehot × E selects row 2 of E: [-0.94, 0.05, -0.62].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B120. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.55 | -0.41 | 0.77
1 | Sith | -0.39 | -0.4 | 0.23
2 | droid | 0.04 | -0.97 | -0.33
3 | ship | 0.23 | 0.05 | 0.02
4 | Luke | -0.18 | 0.82 | 0.76
5 | Leia | -0.41 | 0.87 | -0.55

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Sith" (6 digits separated by commas) and the result of onehot(Sith) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,1,0,0,0,0; embedding = -0.39,-0.4,0.23

Working:

- ID of Sith = 1 → 1 at position 1.
- onehot × E selects row 1 of E: [-0.39, -0.4, 0.23].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B121. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.64 | 0.73 | 0.42
1 | Sith | 0.93 | 0.97 | 0.05
2 | droid | -0.4 | -0.81 | 0.17
3 | ship | 0.36 | 0.08 | 0.53
4 | Luke | -0.67 | -0.04 | 0.58
5 | Leia | 0.12 | -0.14 | 0.25

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Leia" (6 digits separated by commas) and the result of onehot(Leia) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,0,0,0,1; embedding = 0.12,-0.14,0.25

Working:

- ID of Leia = 5 → 1 at position 5.
- onehot × E selects row 5 of E: [0.12, -0.14, 0.25].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B122. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.5 | 0.72 | 0.09
1 | Sith | 0.63 | -0.67 | 0.64
2 | droid | 0.01 | 0.72 | 0.35
3 | ship | 0.9 | -0.12 | 0.75
4 | Luke | -0.94 | 0.43 | 0.98
5 | Leia | -0.64 | 0.81 | -0.1

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "droid" (6 digits separated by commas) and the result of onehot(droid) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,1,0,0,0; embedding = 0.01,0.72,0.35

Working:

- ID of droid = 2 → 1 at position 2.
- onehot × E selects row 2 of E: [0.01, 0.72, 0.35].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B123. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.01 | 0.8 | 0.67
1 | Sith | -0.71 | 0.59 | 0.97
2 | droid | -0.43 | 0.64 | 0.73
3 | ship | 0.54 | -0.98 | -0.91
4 | Luke | -0.13 | 0.69 | 0.24
5 | Leia | 0.87 | -0.33 | -0.06

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Leia" (6 digits separated by commas) and the result of onehot(Leia) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,0,0,0,1; embedding = 0.87,-0.33,-0.06

Working:

- ID of Leia = 5 → 1 at position 5.
- onehot × E selects row 5 of E: [0.87, -0.33, -0.06].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B124. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.96 | -0.46 | -0.98
1 | Sith | 0.74 | -0.98 | -0.55
2 | droid | 0.2 | -0.79 | -0.78
3 | ship | 0.75 | -0.03 | 0.84

4 | Luke | -0.15 | 0.46 | 0.47
5 | Leia | 0.83 | 0.98 | -0.55

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Leia" (6 digits separated by commas) and the result of onehot(Leia) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,0,0,0,1; embedding = 0.83,0.98,-0.55

Working:

- ID of Leia = 5 → 1 at position 5.
- onehot × E selects row 5 of E: [0.83, 0.98, -0.55].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B125. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | 0.82 | 0.76 | -0.42
1 | Sith | 0.16 | 0.08 | -0.75
2 | droid | -0.04 | -0.09 | 1
3 | ship | -0.67 | 0.21 | 0.91
4 | Luke | 0.1 | 0.38 | -0.5
5 | Leia | 0.49 | -0.94 | 0.39

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "ship" (6 digits separated by commas) and the result of onehot(ship) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,0,1,0,0; embedding = -0.67,0.21,0.91

Working:

- ID of ship = 3 → 1 at position 3.
- onehot × E selects row 3 of E: [-0.67, 0.21, 0.91].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B126. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | 0.3 | -0.33 | 0.78
1 | Sith | -0.56 | 0.53 | -0.44
2 | droid | 0.43 | 0.77 | 0.34
3 | ship | 0.6 | 0.27 | 0.56
4 | Luke | -0.13 | -0.79 | 0.47
5 | Leia | -0.91 | 0.79 | -0.53

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Jedi" (6 digits separated by commas) and the result of onehot(Jedi) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 1,0,0,0,0,0; embedding = 0.3,-0.33,0.78

Working:

- ID of Jedi = 0 → 1 at position 0.
- onehot × E selects row 0 of E: [0.3, -0.33, 0.78].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B127. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.87 | 0.23 | -0.46
1 | Sith | -0.55 | 0.52 | -0.77
2 | droid | -0.84 | -0.03 | 0.89
3 | ship | -0.3 | -0.27 | 0.45
4 | Luke | -0.16 | 0.5 | -0.75
5 | Leia | 0.71 | 0.81 | 0.74

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "Sith" (6 digits separated by commas) and the result of onehot(Sith) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,1,0,0,0,0; embedding = -0.55,0.52,-0.77

Working:

- ID of Sith = 1 → 1 at position 1.
- onehot × E selects row 1 of E: [-0.55, 0.52, -0.77].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

SEQ B128. [BLANKS · Easy · calculation]

ID | token | e_1 | e_2 | e_3
0 | Jedi | -0.73 | 0.02 | -0.51
1 | Sith | -0.59 | -0.95 | 0.94
2 | droid | 0.45 | 0.13 | -0.69
3 | ship | 0.26 | -0.85 | -0.13
4 | Luke | 0.98 | 0.78 | -0.89
5 | Leia | -0.07 | 0.38 | -0.87

Token IDs are 0–5 as shown, |V| = 6. Give the one-hot vector for "ship" (6 digits separated by commas) and the result of onehot(ship) × E (3 values separated by commas).

(1) one-hot = ____
(2) embedding = ____

Answer: one-hot = 0,0,0,1,0,0; embedding = 0.26,-0.85,-0.13

Working:

- ID of ship = 3 → 1 at position 3.
- onehot × E selects row 3 of E: [0.26, -0.85, -0.13].

Takeaway: One-hot vectors are all equally distant; learned embeddings put similar words close together.

Source: Lecture 8 (Sequence modelling) + Key points about a simple RNN hidden-state update

🎯 Attention & Transformers (Lecture 8)

Part A: hand-written questions (63)

ATTN A1. [MCQ · Medium]

Poll 4: 'What was Mando searching for?' attends to mission-log tokens. Which Q/K/V assignment is correct?

A. Q = log-token projections; K = question projection; V = question projection repeated
B. Q = question projection; K = log-token keys; V = attention scores over log tokens
C. Q = question projection; K = log-token key projections for matching; V = separate log-token value projections for aggregation
D. Q = question projection; K = log-token value projections; V = log-token key projections

Answer: C. Q = question projection; K = log-token key projections for matching; V = separate log-token value projections for aggregation

Option notes:

- A: Direction of retrieval reversed.
- B: Scores are not values.
- D: Keys and values swapped.

Takeaway: The question asks; the log tokens answer.

Source: Lecture 8, slide 70

ATTN A2. [MULTI · Medium]

Which are limitations of RNNs named in Lecture 8? (Select ALL that apply.)

A. Encoding bottleneck
B. They cannot use non-linear activations
C. Weak long-range memory
D. Slow sequential computation

Answer: A. Encoding bottleneck; C. Weak long-range memory; D. Slow sequential computation

Option notes:

- A: Correct:
- B: Incorrect: They use tanh.
- C: Correct:
- D: Correct:

Takeaway: Three limits motivate attention.

Source: Lecture 8, slide 62

ATTN A3. [MATCH · Easy]

Match each attention component to its role.

1. Query
2. Key
3. Value

A. compact description used for matching
B. the loss function

C. the information need
D. full information returned after a match

Answer: 1 → C, 2 → A, 3 → D

Working:

- Query → the information need
- Key → compact description used for matching
- Value → full information returned after a match

Takeaway: Attention as a search process.

Source: Lecture 8, slide 65-67

ATTN A4. [TF · Medium]

Feeding every position into one fixed dense block is a good alternative to recurrence because it is scalable and keeps order.

Answer: False

Why: It has no flexible input, is not scalable and has no explicit sequence order.

Takeaway: Next idea: attend to what matters.

Source: Lecture 8, slide 63

ATTN A5. [MCQ · Easy · calculation]

Key points: weights [0.3, 0.7], values [2, 0] and [0, 2]. Output?

A. [1.4, 0.6]
B. [2, 2]
C. [0.6, 1.4]
D. [0, 2]

Answer: C. [0.6, 1.4]

Option notes:

- A: Components swapped.
- B: Added unweighted values.
- D: Picked only the largest-weight value.

Takeaway: Weight each value, then add per component.

Source: Key points about Attention

ATTN A6. [TF · Easy]

Normalised attention weights for one query sum to 1.

Answer: True

Why: Softmax output.

Takeaway: Row of A sums to 1.

Source: Lecture 8 (Self-attention)

ATTN A7. [TF · Medium]

In scaled dot-product attention, scores are divided by d_k.

Answer: False

Why: Divided by √d_k.

Takeaway: √d_k keeps softmax from saturating.

Source: Lecture 8 (Self-attention)

ATTN A8. [MCQ · Medium · calculation]

Scores [2, 1, 0.1] → softmax weights?

A. [0.645, 0.323, 0.032]
B. [0.333, 0.333, 0.333]
C. [7.389, 2.718, 1.105]
D. [0.659, 0.242, 0.099]

Answer: D. [0.659, 0.242, 0.099]

Option notes:

- A: Divided raw scores by their sum.
- B: Uniform only if scores are equal.
- C: Forgot to normalise.

Takeaway: Python warm-up 2 expected output.

Source: Python additional warm-up 2 (softmax)

ATTN A9. [BLANKS · Medium · calculation]

q = [1, 2], k_1 = [1, 0], k_2 = [0, 1]. Raw scores $q \cdot k_1$ = ___, $q \cdot k_2$ = ___.

(1) $q \cdot k_1$ = ____
(2) $q \cdot k_2$ = ____

Answer: $q \cdot k_1 = 1$; $q \cdot k_2 = 2$

Takeaway: Dot products.

Source: Lecture 8 (Self-attention)

ATTN A10. [BLANK · Hard · calculation]

Scores 1 and 2 (already scaled). Softmax weight on the second key (3 dp)?

Answer: 0.731
(Also accepted: 0.73)

Takeaway: $e^2/(e^1 + e^2)$ = 7.389/10.107.

Source: Lecture 8 (Self-attention)

ATTN A11. [MCQ · Medium]

Dot product measures query-key compatibility. More aligned vectors give…

A. a larger score
B. a score of exactly 1
C. a smaller score
D. a negative score

Answer: A. a larger score

Option notes:

- B: Not normalised.
- C: Opposite.

- D: Aligned → positive.

Takeaway: Lecture 8 slide 74.

Source: Lecture 8, slide 74

ATTN A12. [MCQ · Medium · calculation]

With 8 tokens, the score matrix $S = QK^T/\sqrt{d_k}$ has shape…

A. 8×8
B. 8×4
C. 8×3
D. 3×3

Answer: A. 8×8

Option notes:

- B: That's X.
- C: That's Q, K, V or Z.
- D: $d_k \times d_k$ is not the score matrix.

Takeaway: One score per query-key pair.

Source: Lecture 8, slide 75

ATTN A13. [MATCH · Medium]

Match each matrix in the Lecture 8 example to its shape (8 tokens, d = 4, d_k = 3).

1. X
2. W_Q
3. $S = QK^T/\sqrt{d_k}$
4. $Z = AV$

A. 3×8 output
B. 4×3
C. 8×4
D. 8×8
E. 8×3

Answer: 1 → C, 2 → B, 3 → D, 4 → E

Working:

- X → 8×4
- W_Q → 4×3
- $S = QK^T/\sqrt{d_k}$ → 8×8
- $Z = AV$ → 8×3

Takeaway: Track the shapes.

Source: Lecture 8, slide 73-77

ATTN A14. [TF · Medium]

Each COLUMN of the attention matrix A = row-softmax(S) sums to 1.

Answer: False

Why: Each ROW (one query's distribution over keys) sums to 1.

Takeaway: Row-softmax.

Source: Lecture 8, slide 76

ATTN A15. [MULTI · Medium]

Select ALL steps of one self-attention head (Lecture 8). (Select ALL that apply.)

A. Project tokens into query, key and value spaces
B. Pass tokens through an RNN first
C. Compute and normalise query-key scores
D. Add position information
E. Combine value vectors using attention weights

Answer: A. Project tokens into query, key and value spaces; C. Compute and normalise query-key scores; D. Add position information; E. Combine value vectors using attention weights

Option notes:

- A: Correct:
- B: Incorrect: No recurrence needed.
- C: Correct:
- D: Correct:
- E: Correct:

Takeaway: Four steps.

Source: Lecture 8, slide 78

ATTN A16. [MCQ · Medium]

Why is positional encoding needed in self-attention?

A. To reduce the vocabulary size
B. To normalise the attention weights
C. Self-attention by itself has no notion of token order
D. To replace the value vectors

Answer: C. Self-attention by itself has no notion of token order

Option notes:

- A: No.
- B: Softmax does that.
- D: It's added to embeddings.

Takeaway: Order must be injected.

Source: Lecture 8, slide 72

ATTN A17. [BLANK · Medium]

PE(pos, 2i) uses _____ and PE(pos, 2i + 1) uses cos.

Answer: sin
(Also accepted: sine)

Takeaway: Even dims sin, odd dims cos.

Source: Lecture 8, slide 72

ATTN A18. [BLANK · Hard · calculation]

PE(1, 0) with any d_model (pos = 1, i = 0) = sin(1) = _____ (3 dp, radians).

Answer: 0.841
(Also accepted: 0.84)

Takeaway: $10000^0 = 1$, sin(1 rad).

Source: Lecture 8 (Self-attention)

ATTN A19. [TF · Easy]

Positional encodings are added to the token embeddings to give position-aware encodings.

Answer: True

Why: Lecture 8 slide 72.

Takeaway: Embedding + p_i.

Source: Lecture 8 (Self-attention)

ATTN A20. [MCQ · Medium]

In multi-head attention, why use several heads?

A. Different heads can learn different attention patterns and relationships
B. To reduce the number of parameters to zero
C. To replace positional encoding
D. To make attention sequential

Answer: A. Different heads can learn different attention patterns and relationships

Option notes:

- B: Heads add parameters.
- C: Unrelated.
- D: Heads run in parallel.

Takeaway: Face / cookie / cookie box heads.

Source: Lecture 8, slide 79

ATTN A21. [MATCH · Medium]

Match each Lecture 8 application to its tokens.

1. Llama 3
2. AlphaFold 3
3. SAM 2
4. RT-2

A. text
B. observations and actions
C. molecular entities
D. audio waveforms
E. image patches

Answer: 1 → A, 2 → C, 3 → E, 4 → B

Working:

- Llama 3 → text

- AlphaFold 3 → molecular entities
- SAM 2 → image patches
- RT-2 → observations and actions

Takeaway: Attention connects many modalities.

Source: Lecture 8, slide 80

ATTN A22. [MULTI · Hard]

Select ALL parts of the transformer encoder block in Lecture 8. (Select ALL that apply.)

A. Add & Norm with residual paths
B. Feed-forward layer
C. A recurrent hidden state
D. Multi-head self-attention
E. Repeated N times

Answer: A. Add & Norm with residual paths; B. Feed-forward layer; D. Multi-head self-attention; E. Repeated N times

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: No recurrence.
- D: Correct:
- E: Correct:

Takeaway: MHSA → Add & Norm → FFN → Add & Norm.

Source: Lecture 8, slide 79

ATTN A23. [BLANK · Medium]

MultiHead(Q, K, V) = Concat($head_1$, …, head_h)·_____.

Answer: W^O
(Also accepted: WO, W_O, Wo)

Takeaway: Output projection.

Source: Lecture 8, slide 79

ATTN A24. [MATCH · Hard]

Compare models: match each to its key property.

1. CNN
2. RNN
3. Self-attention
4. N-gram

A. random forest of trees
B. counts of a fixed short history
C. local filters with shared weights over space
D. hidden state carried step by step through time
E. every token attends to every token in parallel

Answer: 1 → C, 2 → D, 3 → E, 4 → B

Working:

- CNN → local filters with shared weights over space
- RNN → hidden state carried step by step through time
- Self-attention → every token attends to every token in parallel
- N-gram → counts of a fixed short history

Takeaway: Cross-topic comparison.

Source: Lecture 8 (Self-attention)

ATTN A25. [TF · Easy]

Self-attention lets every token build a context-aware representation from the same sequence.

Answer: True

Why: Lecture 8 slide 64.

Takeaway: Each token retrieves what it needs.

Source: Lecture 8 (Self-attention)

ATTN A26. [MCQ · Hard]

word | count in + ($T_+ = 20$) | count in − ($T_- = 18$) | P_s(w|+) | P_s(w|−)
heartwarming | 6 | 2 | 7/26 ≈ 0.27 | 3/24 ≈ 0.13
delightful | 5 | 2 | 6/26 ≈ 0.23 | 3/24 ≈ 0.13
slow | 2 | 6 | 3/26 ≈ 0.12 | 7/24 ≈ 0.29
repetitive | 1 | 6 | 2/26 ≈ 0.08 | 7/24 ≈ 0.29
charming | 6 | 1 | 7/26 ≈ 0.27 | 2/24 ≈ 0.08
dull | 0 | 1 | 1/26 ≈ 0.04 | 2/24 ≈ 0.08

Lecture 8: 11 positive and 9 negative reviews → P(+) = 0.55, P(−) = 0.45; |V| = 6.

A [CLS] sentiment query over the review "The episode was dark and tense, but Mando protecting Grogu was genuinely heartwarming" gives attention weights: dark 8%, tense 12%, but 15%, protecting 8%, genuinely 7%, heartwarming 38%. Which token's value vector contributes most to z_[CLS], and why?

A. heartwarming, because the output is Σ α·v and it has the largest weight α = 0.38
B. but, because contrast words always dominate attention
C. All tokens contribute equally
D. The first token, because attention reads left to right

Answer: A. heartwarming, because the output is Σ α·v and it has the largest weight α = 0.38

Option notes:

- B: Attention follows the learned weights, not grammar rules.
- C: Weights differ; the output is a weighted sum.
- D: Self-attention compares the query with every token in parallel.

Takeaway: Largest weight = largest contribution.

Source: Lecture 8, slide 64

ATTN A27. [TF · Medium]

Values are the compact descriptions used for matching against the query.

Answer: False

Why: That describes KEYS. Values are the full information returned after a match.

Takeaway: K matches, V delivers.

Source: Lecture 8 (Self-attention)

ATTN A28. [BLANK · Easy]

In attention, the vector representing 'the information need' is the _____.

Answer: query
(Also accepted: Query, Q)

Takeaway: Lecture 8 slide 65.

Source: Lecture 8 (Self-attention)

ATTN A29. [MCQ · Hard]

In self-attention, where do Q, K and V come from?

A. Q from X, K and V from a separate memory
B. All three are linear projections of the same position-aware token encodings X
C. K and V are the attention weights
D. They are the raw one-hot vectors

Answer: B. All three are linear projections of the same position-aware token encodings X

Option notes:

- A: That's cross-attention.
- C: No.
- D: They're learned projections.

Takeaway: Q = XW_Q, K = XW_K, V = XW_V.

Source: Lecture 8 (Self-attention)

ATTN A30. [TF · Medium]

The attention output vector has the same number of components as each value vector.

Answer: True

Why: Key points about Attention.

Takeaway: Weighted sum of value vectors.

Source: Lecture 8 (Self-attention)

ATTN A31. [MATCH · Hard]

Match each mistake to the wrong output it produces (weights [0.3, 0.7], values [2, 0], [0, 2]).

1. Pick only the largest-weight value
2. Add the unweighted values
3. Correct weighted sum

A. [0, 2]
B. [0.6, 1.4]
C. [2, 2]
D. [1, 1]

Answer: 1 → A, 2 → C, 3 → B

Working:

- Pick only the largest-weight value → [0, 2]
- Add the unweighted values → [2, 2]
- Correct weighted sum → [0.6, 1.4]

Takeaway: Use the weights.

Source: Lecture 8 (Self-attention)

ATTN A32. [TF · Hard]

softmax($QK^T/\sqrt{d_k}$)V computes, for every query, a weighted average of the value vectors.

Answer: True

Why: Lecture 8 slide 78.

Takeaway: Full attention in one line.

Source: Lecture 8 (Self-attention)

ATTN A33. [MCQ · Medium]

Order the steps for ONE query: (a) softmax, (b) dot products with keys, (c) weighted sum of values, (d) divide by $\sqrt{d_k}$.

A. b, a, d, c
B. b, d, a, c
C. a, b, c, d
D. d, b, c, a

Answer: B. b, d, a, c

Option notes:

- A: Scale before softmax.
- C: Softmax needs scores first.
- D: Can't scale before computing scores.

Takeaway: Dot → scale → softmax → sum.

Source: Lecture 8 (Self-attention)

ATTN A34. [TF · Easy]

Z = AV produces one contextual output row for every token.

Answer: True

Why: 8×8 × 8×3 = 8×3.

Takeaway: Rows = tokens.

Source: Lecture 8 (Self-attention)

ATTN A35. [BLANKS · Easy · calculation]

Attention weights 0.3 and 0.7; values [2, 0] and [0, 2]. Output = [___, ___].

(1) 1st = _____
(2) 2nd = _____

Answer: 1st = 0.6; 2nd = 1.4

Takeaway: Weight every component, then add.

Source: Book 2 (Exam examples), Q243, p. 34

ATTN A36. [BLANKS · Medium · calculation]

Weights (0.2, 0.5, 0.3); values v1 = [1, 0, 2], v2 = [0, 1, 1], v3 = [2, 2, 0]. Output = [___, ___, ___].

(1) 1st = ____
(2) 2nd = ____
(3) 3rd = ____

Answer: 1st = 0.8; 2nd = 1.1; 3rd = 0.9

Working:

- [0.2 + 0 + 0.6, 0 + 0.5 + 0.6, 0.4 + 0.5 + 0]

Takeaway: Same length as each value vector.

Source: Book 2 (Exam examples), Q244, p. 34

ATTN A37. [BLANKS · Hard · calculation]

Query q = [1, 0]; keys k1 = [1, 0], k2 = [0, 1], k3 = [1, 1]; d_k = 2. Scaled scores (2 d.p.) and softmax weights (2 d.p.).

(1) s1 = ____
(2) s2 = ____
(3) s3 = ____
(4) α1 = ____
(5) α2 = ____
(6) α3 = ____

Answer: s1 = 0.71; s2 = 0; s3 = 0.71; α1 = 0.40; α2 = 0.20; α3 = 0.40

Working:

- dot products 1, 0, 1; ÷ √2 → 0.71, 0, 0.71
- e^0.71 = 2.03, e^0 = 1; total 5.06
- weights 0.40, 0.20, 0.40

Takeaway: Scores q·k/√d_k, then softmax.

Source: Book 2 (Exam examples), Q245, p. 34

ATTN A38. [BLANKS · Hard · calculation]

Same query (weights 0.40, 0.20, 0.40) with values [1, 2], [3, 0], [0, 4]. Output (2 d.p.) = [___, ___].

(1) 1st = ____
(2) 2nd = ____

Answer: 1st = 0.99; 2nd = 2.41

Working:

- 0.401(1) + 0.198(3) + 0 = 0.99
- 0.401(2) + 0 + 0.401(4) = 2.41

Takeaway: Use unrounded weights 0.401/0.198/0.401.

Source: Book 2 (Exam examples), Q246, p. 35

ATTN A39. [BLANKS · Medium · calculation]

softmax([2, 1, 0]) = ___ (2 d.p. each).

(1) 1st = ____
(2) 2nd = ____

(3) 3rd = ____

Answer: 1st = 0.67; 2nd = 0.24; 3rd = 0.09

Working:

- $e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$; total 11.107

Takeaway: e^z / Σe^z.

Source: Book 2 (Exam examples), Q247, p. 35

ATTN A40. [MCQ · Medium]

(Poll 4) 'What was Mando searching for?' over a mission log. Correct Q/K/V roles:

A. Q = log projections, K = question, V = log values
B. Q = question; K = log-token keys for matching; V = separate log-token values for aggregation
C. Q = question, K = log keys, V = attention scores
D. Q = question, K = log value projections, V = question repeated

Answer: B. Q = question; K = log-token keys for matching; V = separate log-token values for aggregation

Option notes:

- A: The question is the query.
- C: Scores are weights, not values.
- D: V comes from the log tokens.

Takeaway: Query asks, keys match, values answer.

Source: Book 2 (Exam examples), Q248, p. 35

ATTN A41. [BLANK · Easy]

Scaled dot-product attention = softmax(___/√d_k)V.

Answer: QK^T
(Also accepted: QK⊤, QKᵀ, Q K^T, QK', Q*K^T, QKT)

Takeaway: Query-key scores.

Source: Book 2 (Exam examples), Q249, p. 35

ATTN A42. [TF · Easy]

Each row of the attention matrix A sums to 1.

Answer: True

Why: Row-wise softmax.

Takeaway: Weights add to 1.

Source: Book 2 (Exam examples), Q250, p. 35

ATTN A43. [MATCH · Easy]

Match the attention role.

1. Query
2. Key
3. Value

A. the full information returned after a match
B. a compact description used for matching

C. the information need

Answer: 1 → C, 2 → B, 3 → A

Working:

- Query → the information need
- Key → a compact description used for matching
- Value → the full information returned after a match

Takeaway: Search engine analogy.

Source: Book 2 (Exam examples), Q251, p. 35

ATTN A44. [BLANK · Easy]

Self-attention has no built-in notion of order, so ___ is added to the token embeddings.

Answer: positional encoding
(Also accepted: positional encodings, position encoding, positional embedding)

Takeaway: sin/cos encodings.

Source: Book 2 (Exam examples), Q252, p. 35

ATTN A45. [BLANKS · Easy · calculation]

Attention weights (0.1, 0.6, 0.3) on values [1, 1], [2, 0], [0, 3]. Output = [___, ___].

(1) 1st = _____
(2) 2nd = _____

Answer: 1st = 1.3; 2nd = 1.0

Takeaway: [0.1 + 1.2 + 0, 0.1 + 0 + 0.9].

Source: Book 2 (Exam examples), Mixed practice paper Q25, p. 55

ATTN A46. [BLANKS · Easy · calculation]

Weights (0.5, 0.5), values (4, 0) and (0, 2). Output = (___, ___).

(1) 1st = _____
(2) 2nd = _____

Answer: 1st = 2; 2nd = 1

Takeaway: Weighted sum.

Source: Book 3 (AI from zero), Practice 23.1, p. 94

ATTN A47. [BLANK · Easy]

Q is 10 × 64, K is 10 × 64. Shape of QK^T = ___.

Answer: 10x10
(Also accepted: 10 x 10, 10×10)

Takeaway: Every query with every key.

Source: Book 3 (AI from zero), Practice 23.3, p. 94

ATTN A48. [MCQ · Medium]

In 'What was Mando searching for?', which vectors are matched against the query?

A. the query itself
B. the positional encodings
C. the values
D. the keys of the log tokens (values are aggregated)

Answer: D. the keys of the log tokens (values are aggregated)

Option notes:

- A: No.
- B: They are added to embeddings.
- C: Values are mixed, not matched.

Takeaway: Keys match, values answer.

Source: Book 3 (AI from zero), Practice 23.4, p. 94

ATTN A49. [BLANKS · Hard · calculation]

q = (1, 0); keys (1, 1), (0, 1), (1, 0); values (2, 0), (0, 2), (1, 1); d_k = 2. Weights (3 d.p.) and output (3 d.p.).

(1) α1 = ____
(2) α2 = ____
(3) α3 = ____
(4) out1 = ____
(5) out2 = ____

Answer: α1 = 0.401; α2 = 0.198; α3 = 0.401; out1 = 1.203; out2 = 0.797

Takeaway: Scores 0.707, 0, 0.707.

Source: Book 3 (AI from zero), Chapter 23, p. 92

ATTN A50. [BLANKS · Medium · calculation]

Positional encoding, dimension pair i = 0: PE at position 1 = (sin 1, cos 1) = (___, ___) (3 d.p.).

(1) sin 1 = ____
(2) cos 1 = ____

Answer: sin 1 = 0.841; cos 1 = 0.540

Takeaway: Position 0 gives (0, 1).

Source: Book 3 (AI from zero), Chapter 23, p. 92

ATTN A51. [MCQ · Hard]

Why divide scores by √d_k?

A. to make weights sum to 1
B. dot products of d_k random terms have variance d_k; scaling keeps softmax from becoming too peaky
C. because keys are unit vectors
D. to reduce parameters

Answer: B. dot products of d_k random terms have variance d_k; scaling keeps softmax from becoming too peaky

Option notes:

- A: Softmax does that.
- C: Not assumed.
- D: No parameters change.

Takeaway: Healthy gradients.

Source: Book 3 (AI from zero), Chapter 23, p. 92

ATTN A52. [MULTI · Medium]

A transformer block contains: (Select ALL that apply.)

A. multi-head attention
B. a feed-forward network
C. a recurrent hidden state
D. residual connections and layer normalisation

Answer: A. multi-head attention; B. a feed-forward network; D. residual connections and layer normalisation

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: no recurrence
- D: Correct:

Takeaway: GPT adds a causal mask.

Source: Book 3 (AI from zero), Chapter 23, p. 92

ATTN A53. [MCQ · Medium]

Multi-head attention:

A. shares one W_Q across heads
B. averages the heads' inputs
C. runs one head several times in sequence
D. runs several heads with their own W_Q, W_K, W_V in parallel, then concatenates

Answer: D. runs several heads with their own W_Q, W_K, W_V in parallel, then concatenates

Option notes:

- A: Each head has its own.
- B: Concatenates outputs.
- C: Parallel, separate weights.

Takeaway: One head for grammar, another for meaning.

Source: Book 3 (AI from zero), Chapter 23, p. 92

ATTN A54. [MCQ · Medium]

The sequence-to-sequence bottleneck that attention fixes is:

A. the decoder has no RNN
B. too many parameters in the softmax
C. the whole source sentence must be squeezed into one fixed-length vector
D. words are one-hot

Answer: C. the whole source sentence must be squeezed into one fixed-length vector

Option notes:

- A: It does.
- B: Not the bottleneck.
- D: Embeddings fix that.

Takeaway: Attention lets the decoder look back at every source state.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 24: attention

ATTN A55. [TF · Medium]

Transformers process all positions of a sequence in parallel during training, unlike RNNs.

Answer: True

Why: Self-attention has no sequential dependency between positions.

Takeaway: Parallel = fast on GPUs.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 24: transformers

ATTN A56. [MCQ · Medium]

In a transformer decoder used for generation, masked self-attention:

A. removes positional encoding
B. drops random words
C. hides the encoder
D. prevents each position attending to later (future) positions

Answer: D. prevents each position attending to later (future) positions

Option notes:

- A: No.
- B: That is masked-LM pretraining (BERT).
- C: No.

Takeaway: Causal mask.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 24: transformer decoder

ATTN A57. [BLANKS · Easy · calculation]

Sheet method. Weights 0.4 and 0.6; values [5, 1] and [0, 3]. Output = [___, ___].

(1) 1st = _____
(2) 2nd = _____

Answer: 1st = 2; 2nd = 2.2

Working:

- 1st: 2 + 0 = 2
- 2nd: 0.4 + 1.8 = 2.2

Takeaway: [0.4·5 + 0.6·0, 0.4·1 + 0.6·3].

Source: Key points about Attention: weighted value aggregation

ATTN A58. [BLANKS · Medium · calculation]

Weights 0.5, 0.25, 0.25; values [4, 0, 2], [0, 8, 4], [2, 2, 2]. Output = [___, ___, ___].

(1) 1st = _____
(2) 2nd = _____
(3) 3rd = _____

Answer: 1st = 2.5; 2nd = 2.5; 3rd = 2.5

Working:

- 1st: 2 + 0 + 0.5 = 2.5

- 2nd: 0 + 2 + 0.5 = 2.5
- 3rd: 1 + 1 + 0.5 = 2.5

Takeaway: Component by component.

Source: Key points about Attention: weighted value aggregation

ATTN A59. [MCQ · Medium]

Weights 0.3 and 0.7; values [2, 0] and [0, 2]. A student answers [0, 2] (the value with the largest weight). The mistake is:

A. using the wrong order
B. selecting only the largest-weight vector instead of the weighted sum
C. adding the unweighted vectors
D. nothing: [0, 2] is correct

Answer: B. selecting only the largest-weight vector instead of the weighted sum

Option notes:

- A: Order is fine.
- C: That would give [2, 2].
- D: Correct output is [0.6, 1.4].

Takeaway: Every value vector contributes in proportion to its weight.

Source: Key points about Attention: weighted value aggregation

ATTN A60. [MCQ · Medium]

Same weights and values. Another student answers [2, 2]. The mistake is:

A. adding the value vectors without multiplying by their weights
B. picking the largest weight
C. normalising twice
D. nothing

Answer: A. adding the value vectors without multiplying by their weights

Option notes:

- B: That gives [0, 2].
- C: No normalising here.
- D: Correct is [0.6, 1.4].

Takeaway: Weight first, then add.

Source: Key points about Attention: weighted value aggregation

ATTN A61. [TF · Easy]

The attention output has the same number of components as each value vector.

Answer: True

Why: Corresponding components are added.

Takeaway: Output length = value length.

Source: Key points about Attention: weighted value aggregation

ATTN A62. [TF · Easy]

Normalised attention weights sum to one.

Answer: True

Why: They come from a softmax.

Takeaway: Weights add to 1.

Source: Key points about Attention: weighted value aggregation

ATTN A63. [BLANK · Medium · calculation]

Normalised weights for three values are 0.2, 0.5 and ___.

Answer: 0.3

Takeaway: They must sum to 1.

Source: Key points about Attention: weighted value aggregation

Part B: calculation drills (112)

ATTN B1. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.9, 0.1] and the value vectors are [1, 3], [3, 4]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 1.2; out[2] = 3.1

Working:

- Component 1: 0.9×1 + 0.1×3 = 1.2
- Component 2: 0.9×3 + 0.1×4 = 3.1
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B2. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.9, 0.1] and the value vectors are [-1, 4], [2, 1]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = -0.7; out[2] = 3.7

Working:

- Component 1: 0.9×-1 + 0.1×2 = -0.7
- Component 2: 0.9×4 + 0.1×1 = 3.7
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B3. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.5, 0.25, 0.25] and the value vectors are [-1, -1], [-1, 2], [1, -1]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = -0.5; out[2] = -0.25

Working:

- Component 1: 0.5×-1 + 0.25×-1 + 0.25×1 = -0.5
- Component 2: 0.5×-1 + 0.25×2 + 0.25×-1 = -0.25
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B4. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.1, 0.6, 0.3] and the value vectors are [2, 0], [2, 0], [2, 1]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 2; out[2] = 0.3

Working:

- Component 1: 0.1×2 + 0.6×2 + 0.3×2 = 2
- Component 2: 0.1×0 + 0.6×0 + 0.3×1 = 0.3
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B5. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.5, 0.25, 0.25] and the value vectors are [4, -1], [4, 1], [2, 2]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 3.5; out[2] = 0.25

Working:

- Component 1: 0.5×4 + 0.25×4 + 0.25×2 = 3.5
- Component 2: 0.5×-1 + 0.25×1 + 0.25×2 = 0.25
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B6. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.25, 0.75] and the value vectors are [3, 1], [-1, 3]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 0; out[2] = 2.5

Working:

- Component 1: 0.25×3 + 0.75×-1 = 0
- Component 2: 0.25×1 + 0.75×3 = 2.5
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B7. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.25, 0.75] and the value vectors are [1, -1], [1, -1]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 1; out[2] = -1

Working:

- Component 1: 0.25×1 + 0.75×1 = 1
- Component 2: 0.25×-1 + 0.75×-1 = -1
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B8. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.1, 0.6, 0.3] and the value vectors are [1, 0], [-1, -1], [1, 1]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = -0.2; out[2] = -0.3

Working:

- Component 1: 0.1×1 + 0.6×-1 + 0.3×1 = -0.2
- Component 2: 0.1×0 + 0.6×-1 + 0.3×1 = -0.3
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B9. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.3, 0.7] and the value vectors are [4, 1], [2, 0]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 2.6; out[2] = 0.3

Working:

- Component 1: 0.3×4 + 0.7×2 = 2.6
- Component 2: 0.3×1 + 0.7×0 = 0.3
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B10. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.6, 0.4] and the value vectors are [0, 0], [0, -1]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 0; out[2] = -0.4

Working:

- Component 1: 0.6×0 + 0.4×0 = 0
- Component 2: 0.6×0 + 0.4×-1 = -0.4
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B11. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.1, 0.6, 0.3] and the value vectors are [4, 1], [2, 4], [-1, 4]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 1.3; out[2] = 3.7

Working:

- Component 1: 0.1×4 + 0.6×2 + 0.3×-1 = 1.3
- Component 2: 0.1×1 + 0.6×4 + 0.3×4 = 3.7
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B12. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.25, 0.75] and the value vectors are [3, 0], [3, 0]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 3; out[2] = 0

Working:

- Component 1: 0.25×3 + 0.75×3 = 3
- Component 2: 0.25×0 + 0.75×0 = 0
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B13. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.2, 0.5, 0.3] and the value vectors are [-1, 1], [0, 4], [-1, 1]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = -0.5; out[2] = 2.5

Working:

- Component 1: 0.2×-1 + 0.5×0 + 0.3×-1 = -0.5
- Component 2: 0.2×1 + 0.5×4 + 0.3×1 = 2.5
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B14. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.6, 0.4] and the value vectors are [1, 2], [4, 4]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = 2.2; out[2] = 2.8

Working:

- Component 1: 0.6×1 + 0.4×4 = 2.2
- Component 2: 0.6×2 + 0.4×4 = 2.8
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B15. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.9, 0.1] and the value vectors are [-1, 4], [-1, 3]. Give the output vector (2 components).

(1) out[1] = ____
(2) out[2] = ____

Answer: out[1] = -1; out[2] = 3.9

Working:

- Component 1: 0.9×-1 + 0.1×-1 = -1
- Component 2: 0.9×4 + 0.1×3 = 3.9
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B16. [BLANKS · Easy · calculation]

For one query, the attention weights are [0.2, 0.5, 0.3] and the value vectors are [4, 1], [0, 4], [4, 1]. Give the output vector (2 components).

(1) out[1] = _____
(2) out[2] = _____

Answer: out[1] = 2; out[2] = 2.5

Working:

- Component 1: 0.2×4 + 0.5×0 + 0.3×4 = 2
- Component 2: 0.2×1 + 0.5×4 + 0.3×1 = 2.5
- (Key points example: [0.3, 0.7] with [2,0],[0,2] → [0.6, 1.4].)

Takeaway: Output = Σ $\text{weight}_i \cdot \text{value}_i$, component by component. Don't just pick the biggest-weight value.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B17. [BLANKS · Easy · calculation]

Query q = [3, 3]. Keys: k1 = [3, -2], k2 = [3, 1], k3 = [0, 3]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = _____
(2) q·k2 = _____
(3) q·k3 = _____

Answer: q·k1 = 3; q·k2 = 12; q·k3 = 9

Working:

- q·k1 = 3×3 + 3×-2 = 3
- q·k2 = 3×3 + 3×1 = 12
- q·k3 = 3×0 + 3×3 = 9

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B18. [BLANKS · Easy · calculation]

Query q = [3, 0]. Keys: k1 = [0, 3], k2 = [-1, -1]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = _____
(2) q·k2 = _____

Answer: q·k1 = 0; q·k2 = -3

Working:

- q·k1 = 3×0 + 0×3 = 0
- q·k2 = 3×-1 + 0×-1 = -3

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B19. [BLANKS · Easy · calculation]

Query q = [-1, 0]. Keys: k1 = [3, -2], k2 = [1, 2], k3 = [3, 2]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = _____
(2) q·k2 = _____
(3) q·k3 = _____

Answer: q·k1 = -3; q·k2 = -1; q·k3 = -3

Working:

- q·k1 = -1×3 + 0×-2 = -3
- q·k2 = -1×1 + 0×2 = -1

- q·k3 = -1×3 + 0×2 = -3

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B20. [BLANKS · Easy · calculation]

Query q = [3, -2]. Keys: k1 = [-2, 3], k2 = [1, 3]. Give the raw dot-product scores q·k_i.

(1) q·k1 = ____
(2) q·k2 = ____

Answer: q·k1 = -12; q·k2 = -3

Working:

- q·k1 = 3×-2 + -2×3 = -12
- q·k2 = 3×1 + -2×3 = -3

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B21. [BLANKS · Easy · calculation]

Query q = [1, 3]. Keys: k1 = [0, 1], k2 = [1, -1]. Give the raw dot-product scores q·k_i.

(1) q·k1 = ____
(2) q·k2 = ____

Answer: q·k1 = 3; q·k2 = -2

Working:

- q·k1 = 1×0 + 3×1 = 3
- q·k2 = 1×1 + 3×-1 = -2

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B22. [BLANKS · Easy · calculation]

Query q = [0, -2]. Keys: k1 = [-1, 3], k2 = [-2, 1], k3 = [2, 1]. Give the raw dot-product scores q·k_i.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = -6; q·k2 = -2; q·k3 = -2

Working:

- q·k1 = 0×-1 + -2×3 = -6
- q·k2 = 0×-2 + -2×1 = -2
- q·k3 = 0×2 + -2×1 = -2

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B23. [BLANKS · Easy · calculation]

Query q = [2, -1]. Keys: k1 = [2, 0], k2 = [0, 1], k3 = [1, -2]. Give the raw dot-product scores q·k_i.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = 4; q·k2 = -1; q·k3 = 4

Working:

- q·k1 = 2×2 + -1×0 = 4
- q·k2 = 2×0 + -1×1 = -1
- q·k3 = 2×1 + -1×-2 = 4

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B24. [BLANKS · Easy · calculation]

Query q = [-2, 3]. Keys: k1 = [-2, 3], k2 = [-2, 1], k3 = [-2, 2]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = 13; q·k2 = 7; q·k3 = 10

Working:

- q·k1 = -2×-2 + 3×3 = 13
- q·k2 = -2×-2 + 3×1 = 7
- q·k3 = -2×-2 + 3×2 = 10

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B25. [BLANKS · Easy · calculation]

Query q = [2, 0]. Keys: k1 = [0, 2], k2 = [1, 2], k3 = [1, -2]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = 0; q·k2 = 2; q·k3 = 2

Working:

- q·k1 = 2×0 + 0×2 = 0
- q·k2 = 2×1 + 0×2 = 2
- q·k3 = 2×1 + 0×-2 = 2

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B26. [BLANKS · Easy · calculation]

Query q = [-2, 3]. Keys: k1 = [2, 1], k2 = [-2, 2]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = ____
(2) q·k2 = ____

Answer: q·k1 = -1; q·k2 = 10

Working:

- q·k1 = -2×2 + 3×1 = -1
- q·k2 = -2×-2 + 3×2 = 10

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B27. [BLANKS · Easy · calculation]

Query q = [3, -2]. Keys: k1 = [2, -2], k2 = [2, -2], k3 = [0, -2]. Give the raw dot-product scores q·k_i.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = 10; q·k2 = 10; q·k3 = 4

Working:

- q·k1 = 3×2 + -2×-2 = 10
- q·k2 = 3×2 + -2×-2 = 10
- q·k3 = 3×0 + -2×-2 = 4

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B28. [BLANKS · Easy · calculation]

Query q = [0, -1]. Keys: k1 = [-1, 1], k2 = [3, 0], k3 = [0, 1]. Give the raw dot-product scores q·k_i.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = -1; q·k2 = 0; q·k3 = -1

Working:

- q·k1 = 0×-1 + -1×1 = -1
- q·k2 = 0×3 + -1×0 = 0
- q·k3 = 0×0 + -1×1 = -1

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B29. [BLANKS · Easy · calculation]

Query q = [2, -2]. Keys: k1 = [-1, 3], k2 = [3, 3]. Give the raw dot-product scores q·k_i.

(1) q·k1 = ____
(2) q·k2 = ____

Answer: q·k1 = -8; q·k2 = 0

Working:

- q·k1 = 2×-1 + -2×3 = -8
- q·k2 = 2×3 + -2×3 = 0

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B30. [BLANKS · Easy · calculation]

Query q = [2, 1]. Keys: k1 = [-1, 0], k2 = [1, 3], k3 = [-2, 1]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = -2; q·k2 = 5; q·k3 = -3

Working:

- q·k1 = 2×-1 + 1×0 = -2
- q·k2 = 2×1 + 1×3 = 5
- q·k3 = 2×-2 + 1×1 = -3

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B31. [BLANKS · Easy · calculation]

Query q = [-2, 0]. Keys: k1 = [1, -1], k2 = [-2, 3]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = ____
(2) q·k2 = ____

Answer: q·k1 = -2; q·k2 = 4

Working:

- q·k1 = -2×1 + 0×-1 = -2
- q·k2 = -2×-2 + 0×3 = 4

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B32. [BLANKS · Easy · calculation]

Query q = [3, 0]. Keys: k1 = [1, 3], k2 = [3, 2], k3 = [-2, 1]. Give the raw dot-product scores $q \cdot k_i$.

(1) q·k1 = ____
(2) q·k2 = ____
(3) q·k3 = ____

Answer: q·k1 = 3; q·k2 = 9; q·k3 = -6

Working:

- q·k1 = 3×1 + 0×3 = 3
- q·k2 = 3×3 + 0×2 = 9
- q·k3 = 3×-2 + 0×1 = -6

Takeaway: More aligned query and key → larger score.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B33. [MCQ · Medium · calculation]

Scaled scores are [1.9, 0.5]. What is the softmax weight of score #2?

A. 1.649
B. 0.208

C. 0.198
D. 0.500

Answer: C. 0.198

Working:

- exp: 6.6859, 1.6487; sum = 8.3346.
- weight = 1.6487/8.3346 = 0.1978.

Option notes:

- A: Forgot to normalise by Σ exp.
- B: Divided raw scores by their sum; softmax exponentiates first.
- D: Softmax is not uniform unless all scores are equal.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B34. [MCQ · Medium · calculation]

Scaled scores are [0.3, 1.3]. What is the softmax weight of score #2?

A. 0.500
B. 0.731
C. 0.813
D. 3.669

Answer: B. 0.731

Working:

- exp: 1.3499, 3.6693; sum = 5.0192.
- weight = 3.6693/5.0192 = 0.7311.

Option notes:

- A: Softmax is not uniform unless all scores are equal.
- C: Divided raw scores by their sum; softmax exponentiates first.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B35. [MCQ · Medium · calculation]

Scaled scores are [-0.3, 0.1, 2.1, 1.9]. What is the softmax weight of score #1?

A. 0.250
B. -0.079
C. 0.044
D. 0.741

Answer: C. 0.044

Working:

- exp: 0.7408, 1.1052, 8.1662, 6.6859; sum = 16.6981.
- weight = 0.7408/16.6981 = 0.0444.

Option notes:

- A: Softmax is not uniform unless all scores are equal.
- B: Divided raw scores by their sum; softmax exponentiates first.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B36. [MCQ · Medium · calculation]

Scaled scores are [0.8, 1.9, 1, 1.7]. What is the softmax weight of score #1?

A. 2.226
B. 0.130
C. 0.148
D. 0.250

Answer: B. 0.130

Working:

- exp: 2.2255, 6.6859, 2.7183, 5.4739; sum = 17.1037.
- weight = 2.2255/17.1037 = 0.1301.

Option notes:

- A: Forgot to normalise by Σ exp.
- C: Divided raw scores by their sum; softmax exponentiates first.
- D: Softmax is not uniform unless all scores are equal.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B37. [MCQ · Medium · calculation]

Scaled scores are [-0.5, 2.1]. What is the softmax weight of score #1?

A. 0.607
B. 0.500
C. 0.069
D. -0.313

Answer: C. 0.069

Working:

- exp: 0.6065, 8.1662; sum = 8.7727.
- weight = 0.6065/8.7727 = 0.0691.

Option notes:

- A: Forgot to normalise by Σ exp.
- B: Softmax is not uniform unless all scores are equal.
- D: Divided raw scores by their sum; softmax exponentiates first.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B38. [MCQ · Medium · calculation]

Scaled scores are [-0.2, 1.4, -0.3, 1.5]. What is the softmax weight of score #2?

A. 0.402
B. 0.583
C. 0.250
D. 4.055

Answer: A. 0.402

Working:

- exp: 0.8187, 4.0552, 0.7408, 4.4817; sum = 10.0964.
- weight = 4.0552/10.0964 = 0.4016.

Option notes:

- B: Divided raw scores by their sum; softmax exponentiates first.
- C: Softmax is not uniform unless all scores are equal.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B39. [MCQ · Medium · calculation]

Scaled scores are [1.5, 2.5, 0.5, 1.7]. What is the softmax weight of score #1?

A. 0.242
B. 4.482
C. 0.250
D. 0.188

Answer: D. 0.188

Working:

- exp: 4.4817, 12.1825, 1.6487, 5.4739; sum = 23.7869.
- weight = 4.4817/23.7869 = 0.1884.

Option notes:

- A: Divided raw scores by their sum; softmax exponentiates first.
- B: Forgot to normalise by Σ exp.
- C: Softmax is not uniform unless all scores are equal.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B40. [MCQ · Medium · calculation]

Scaled scores are [-0.3, -0.1, 1.9, 0.7]. What is the softmax weight of score #4?

A. 0.195
B. 2.014
C. 0.318
D. 0.250

Answer: A. 0.195

Working:

- exp: 0.7408, 0.9048, 6.6859, 2.0138; sum = 10.3453.
- weight = 2.0138/10.3453 = 0.1947.

Option notes:

- B: Forgot to normalise by Σ exp.
- C: Divided raw scores by their sum; softmax exponentiates first.
- D: Softmax is not uniform unless all scores are equal.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B41. [MCQ · Medium · calculation]

Scaled scores are [2.1, 1.2]. What is the softmax weight of score #1?

A. 0.636
B. 0.711
C. 8.166
D. 0.500

Answer: B. 0.711

Working:

- exp: 8.1662, 3.3201; sum = 11.4863.
- weight = 8.1662/11.4863 = 0.7109.

Option notes:

- A: Divided raw scores by their sum; softmax exponentiates first.
- C: Forgot to normalise by Σ exp.
- D: Softmax is not uniform unless all scores are equal.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B42. [MCQ · Medium · calculation]

Scaled scores are [2.5, 0.4, 0.7, 2]. What is the softmax weight of score #3?

A. 0.087
B. 0.125
C. 0.250
D. 2.014

Answer: A. 0.087

Working:

- exp: 12.1825, 1.4918, 2.0138, 7.3891; sum = 23.0771.
- weight = 2.0138/23.0771 = 0.0873.

Option notes:

- B: Divided raw scores by their sum; softmax exponentiates first.
- C: Softmax is not uniform unless all scores are equal.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B43. [MCQ · Medium · calculation]

Scaled scores are [0.1, 2, -0.2]. What is the softmax weight of score #3?

A. 0.819
B. 0.333
C. 0.088
D. -0.105

Answer: C. 0.088

Working:

- exp: 1.1052, 7.3891, 0.8187; sum = 9.3130.
- weight = 0.8187/9.3130 = 0.0879.

Option notes:

- A: Forgot to normalise by Σ exp.
- B: Softmax is not uniform unless all scores are equal.
- D: Divided raw scores by their sum; softmax exponentiates first.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B44. [MCQ · Medium · calculation]

Scaled scores are [0.9, -0.2, -0.1]. What is the softmax weight of score #3?

A. 0.216
B. 0.333
C. -0.167
D. 0.905

Answer: A. 0.216

Working:

- exp: 2.4596, 0.8187, 0.9048; sum = 4.1832.
- weight = 0.9048/4.1832 = 0.2163.

Option notes:

- B: Softmax is not uniform unless all scores are equal.
- C: Divided raw scores by their sum; softmax exponentiates first.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B45. [MCQ · Medium · calculation]

Scaled scores are [1.8, 0.7, 0.8, 0.9]. What is the softmax weight of score #3?

A. 0.190
B. 0.250
C. 0.175
D. 2.226

Answer: C. 0.175

Working:

- exp: 6.0496, 2.0138, 2.2255, 2.4596; sum = 12.7485.
- weight = 2.2255/12.7485 = 0.1746.

Option notes:

- A: Divided raw scores by their sum; softmax exponentiates first.
- B: Softmax is not uniform unless all scores are equal.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B46. [MCQ · Medium · calculation]

Scaled scores are [0.4, 2, 0.1]. What is the softmax weight of score #1?

A. 0.149
B. 0.160
C. 0.333
D. 1.492

Answer: A. 0.149

Working:

- exp: 1.4918, 7.3891, 1.1052; sum = 9.9861.
- weight = 1.4918/9.9861 = 0.1494.

Option notes:

- B: Divided raw scores by their sum; softmax exponentiates first.
- C: Softmax is not uniform unless all scores are equal.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B47. [MCQ · Medium · calculation]

Scaled scores are [2.3, -0.2]. What is the softmax weight of score #1?

A. 1.095
B. 0.500
C. 0.924
D. 9.974

Answer: C. 0.924

Working:

- exp: 9.9742, 0.8187; sum = 10.7929.
- weight = 9.9742/10.7929 = 0.9241.

Option notes:

- A: Divided raw scores by their sum; softmax exponentiates first.
- B: Softmax is not uniform unless all scores are equal.
- D: Forgot to normalise by Σ exp.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B48. [MCQ · Medium · calculation]

Scaled scores are [0.2, -0.4]. What is the softmax weight of score #2?

A. 2.000
B. 0.670
C. 0.354
D. 0.500

Answer: C. 0.354

Working:

- exp: 1.2214, 0.6703; sum = 1.8917.
- weight = 0.6703/1.8917 = 0.3543.

Option notes:

- A: Divided raw scores by their sum; softmax exponentiates first.
- B: Forgot to normalise by Σ exp.
- D: Softmax is not uniform unless all scores are equal.

Takeaway: softmax(s_i) = e^{s_i}/Σ_je^{s_j}; weights are positive and sum to 1. (Warm-up: [2, 1, 0.1] → [0.659, 0.242, 0.099].)

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B49. [BLANKS · Exam+ · calculation]

Query q = [1, 3]; keys k1 = [2, 0], k2 = [2, 3]; values v1 = [1, 0], v2 = [4, 1]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.002; out[1] = 3.995; out[2] = 0.998

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [2, 0] | [1, 0] | 2 | 1.414 | 4.113 | 0.002
  k2 | [2, 3] | [4, 1] | 11 | 7.778 | 2387.912 | 0.998
- Output = Σ $w_i v_i$ = [3.9948, 0.9983].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B50. [BLANKS · Exam+ · calculation]

Query q = [0, 0]; keys k1 = [-2, 2], k2 = [3, -2]; values v1 = [4, 1], v2 = [3, 2]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) weight_1 = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: weight_1 = 0.500; out[1] = 3.500; out[2] = 1.500

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-2, 2] | [4, 1] | 0 | 0.000 | 1.000 | 0.500
  k2 | [3, -2] | [3, 2] | 0 | 0.000 | 1.000 | 0.500
- Output = $\Sigma\, w_i v_i$ = [3.5000, 1.5000].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B51. [BLANKS · Exam+ · calculation]

Query q = [0, -2]; keys k1 = [2, 0], k2 = [1, 3], k3 = [1, 0]; values v1 = [0, 3], v2 = [2, 4], v3 = [4, 4]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) weight_1 = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: weight_1 = 0.496; out[1] = 2.000; out[2] = 3.504

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [2, 0] | [0, 3] | 0 | 0.000 | 1.000 | 0.496
  k2 | [1, 3] | [2, 4] | -6 | -4.243 | 0.014 | 0.007
  k3 | [1, 0] | [4, 4] | 0 | 0.000 | 1.000 | 0.496
- Output = $\Sigma\, w_i v_i$ = [2.0000, 3.5036].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B52. [BLANKS · Exam+ · calculation]

Query q = [1, -1]; keys k1 = [-1, 1], k2 = [-2, -1]; values v1 = [0, 2], v2 = [1, 4]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) weight_1 = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: weight_1 = 0.330; out[1] = 0.670; out[2] = 3.340

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-1, 1] | [0, 2] | -2 | -1.414 | 0.243 | 0.330
  k2 | [-2, -1] | [1, 4] | -1 | -0.707 | 0.493 | 0.670
- Output = $\Sigma\, w_i v_i$ = [0.6698, 3.3395].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B53. [BLANKS · Exam+ · calculation]

Query q = [0, 1]; keys k1 = [-2, 1], k2 = [1, -2]; values v1 = [-1, 1], v2 = [1, 1]; d_k = 2. Compute softmax(q·K^T/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.893; out[1] = -0.786; out[2] = 1.000

Working:

- Key | k_i | v_i | score q·k_i | scaled /√2 | exp | weight
  k1 | [-2, 1] | [-1, 1] | 1 | 0.707 | 2.028 | 0.893
  k2 | [1, -2] | [1, 1] | -2 | -1.414 | 0.243 | 0.107
- Output = Σ $w_i v_i$ = [-0.7859, 1.0000].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B54. [BLANKS · Exam+ · calculation]

Query q = [-1, 0]; keys k1 = [1, 1], k2 = [-1, 1], k3 = [3, 2]; values v1 = [1, 1], v2 = [0, -1], v3 = [2, 4]; d_k = 2. Compute softmax(q·K^T/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.187; out[1] = 0.277; out[2] = -0.400

Working:

- Key | k_i | v_i | score q·k_i | scaled /√2 | exp | weight
  k1 | [1, 1] | [1, 1] | -1 | -0.707 | 0.493 | 0.187
  k2 | [-1, 1] | [0, -1] | 1 | 0.707 | 2.028 | 0.768
  k3 | [3, 2] | [2, 4] | -3 | -2.121 | 0.120 | 0.045
- Output = Σ $w_i v_i$ = [0.2775, -0.3997].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B55. [BLANKS · Exam+ · calculation]

Query q = [0, 2]; keys k1 = [0, -1], k2 = [-2, 0]; values v1 = [3, -1], v2 = [3, 4]; d_k = 2. Compute softmax(q·K^T/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.196; out[1] = 3.000; out[2] = 3.022

Working:

- Key | k_i | v_i | score q·k_i | scaled /√2 | exp | weight
  k1 | [0, -1] | [3, -1] | -2 | -1.414 | 0.243 | 0.196
  k2 | [-2, 0] | [3, 4] | 0 | 0.000 | 1.000 | 0.804
- Output = Σ $w_i v_i$ = [3.0000, 3.0221].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B56. [BLANKS · Exam+ · calculation]

Query q = [1, -2]; keys k1 = [2, 3], k2 = [1, -1]; values v1 = [3, 4], v2 = [4, 2]; d_k = 2. Compute softmax($q \cdot K^T / \sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.007; out[1] = 3.993; out[2] = 2.014

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [2, 3] | [3, 4] | -4 | -2.828 | 0.059 | 0.007
  k2 | [1, -1] | [4, 2] | 3 | 2.121 | 8.342 | 0.993
- Output = Σ $w_i v_i$ = [3.9930, 2.0141].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B57. [BLANKS · Exam+ · calculation]

Query q = [-1, 3]; keys k1 = [-2, -2], k2 = [1, -1]; values v1 = [1, 4], v2 = [3, 3]; d_k = 2. Compute softmax($q \cdot K^T / \sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.500; out[1] = 2.000; out[2] = 3.500

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-2, -2] | [1, 4] | -4 | -2.828 | 0.059 | 0.500
  k2 | [1, -1] | [3, 3] | -4 | -2.828 | 0.059 | 0.500
- Output = Σ $w_i v_i$ = [2.0000, 3.5000].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B58. [BLANKS · Exam+ · calculation]

Query q = [2, 1]; keys k1 = [-1, 1], k2 = [1, 3], k3 = [0, 3]; values v1 = [-1, 1], v2 = [4, 4], v3 = [3, 4]; d_k = 2. Compute softmax($q \cdot K^T / \sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.011; out[1] = 3.750; out[2] = 3.966

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled /√2 | exp | weight
  k1 | [-1, 1] | [-1, 1] | -1 | -0.707 | 0.493 | 0.011
  k2 | [1, 3] | [4, 4] | 5 | 3.536 | 34.313 | 0.795
  k3 | [0, 3] | [3, 4] | 3 | 2.121 | 8.342 | 0.193

- Output = Σ $w_i v_i$ = [3.7495, 3.9657].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B59. [BLANKS · Exam+ · calculation]

Query q = [-2, 2]; keys k1 = [2, 2], k2 = [-1, -1]; values v1 = [-1, 3], v2 = [0, 3]; d_k = 2. Compute softmax(q·K^T/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.500; out[1] = -0.500; out[2] = 3.000

Working:

- Key | k_i | v_i | score q·k_i | scaled /√2 | exp | weight
  k1 | [2, 2] | [-1, 3] | 0 | 0.000 | 1.000 | 0.500
  k2 | [-1, -1] | [0, 3] | 0 | 0.000 | 1.000 | 0.500
- Output = Σ $w_i v_i$ = [-0.5000, 3.0000].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B60. [BLANKS · Exam+ · calculation]

Query q = [-1, 0]; keys k1 = [-2, 0], k2 = [3, 2], k3 = [-2, 2]; values v1 = [2, 3], v2 = [-1, 4], v3 = [1, 2]; d_k = 2. Compute softmax(q·K^T/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.493; out[1] = 1.464; out[2] = 2.522

Working:

- Key | k_i | v_i | score q·k_i | scaled /√2 | exp | weight
  k1 | [-2, 0] | [2, 3] | 2 | 1.414 | 4.113 | 0.493
  k2 | [3, 2] | [-1, 4] | -3 | -2.121 | 0.120 | 0.014
  k3 | [-2, 2] | [1, 2] | 2 | 1.414 | 4.113 | 0.493
- Output = Σ $w_i v_i$ = [1.4641, 2.5215].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B61. [BLANKS · Exam+ · calculation]

Query q = [2, -1]; keys k1 = [-2, 0], k2 = [3, 1]; values v1 = [1, 0], v2 = [2, 1]; d_k = 2. Compute softmax(q·K^T/√d_k)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) $weight_1$ = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $weight_1$ = 0.002; out[1] = 1.998; out[2] = 0.998

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled $/\sqrt{2}$ | exp | weight
  k1 | [-2, 0] | [1, 0] | -4 | -2.828 | 0.059 | 0.002
  k2 | [3, 1] | [2, 1] | 5 | 3.536 | 34.313 | 0.998
- Output = $\Sigma\, w_i v_i$ = [1.9983, 0.9983].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B62. [BLANKS · Exam+ · calculation]

Query q = [0, -2]; keys k1 = [0, -2], k2 = [2, -1]; values v1 = [4, -1], v2 = [1, -1]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) weight_1 = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: weight_1 = 0.804; out[1] = 3.413; out[2] = -1.000

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled $/\sqrt{2}$ | exp | weight
  k1 | [0, -2] | [4, -1] | 4 | 2.828 | 16.919 | 0.804
  k2 | [2, -1] | [1, -1] | 2 | 1.414 | 4.113 | 0.196
- Output = $\Sigma\, w_i v_i$ = [3.4133, -1.0000].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B63. [BLANKS · Exam+ · calculation]

Query q = [-1, -2]; keys k1 = [1, 2], k2 = [-2, 2], k3 = [-2, 1]; values v1 = [3, -1], v2 = [3, -1], v3 = [4, 2]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) weight_1 = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: weight_1 = 0.023; out[1] = 3.786; out[2] = 1.358

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled $/\sqrt{2}$ | exp | weight
  k1 | [1, 2] | [3, -1] | -5 | -3.536 | 0.029 | 0.023
  k2 | [-2, 2] | [3, -1] | -2 | -1.414 | 0.243 | 0.191
  k3 | [-2, 1] | [4, 2] | 0 | 0.000 | 1.000 | 0.786
- Output = $\Sigma\, w_i v_i$ = [3.7860, 1.3580].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B64. [BLANKS · Exam+ · calculation]

Query q = [2, -1]; keys k1 = [1, 3], k2 = [1, -2], k3 = [2, 3]; values v1 = [2, 2], v2 = [-1, 2], v3 = [2, 2]; d_k = 2. Compute softmax($q \cdot K^T/\sqrt{d_k}$)·V. Give the weight on key 1 (3 dp) and the 2 output components (3 dp).

(1) weight_1 = ____
(2) out[1] = ____
(3) out[2] = ____

Answer: $\text{weight}_1 = 0.025$; out[1] = -0.611; out[2] = 2.000

Working:

- Key | k_i | v_i | score $q \cdot k_i$ | scaled $/\sqrt{2}$ | exp | weight
  k1 | [1, 3] | [2, 2] | -1 | -0.707 | 0.493 | 0.025
  k2 | [1, -2] | [-1, 2] | 4 | 2.828 | 16.919 | 0.870
  k3 | [2, 3] | [2, 2] | 1 | 0.707 | 2.028 | 0.104
- Output = $\Sigma w_i v_i$ = [-0.6109, 2.0000].

Takeaway: Scores → scale by √d_k → row softmax → weighted sum of values.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B65. [MCQ · Easy · calculation]

A raw dot-product score is 17 with key dimension d_k = 16. What is the scaled score used in scaled dot-product attention?

A. 4.250
B. 1.063
C. 68.000
D. 17.000

Answer: A. 4.250

Working:

- $17/\sqrt{16} = 17/4 = 4.250$.

Option notes:

- B: Divide by √d_k, not d_k.
- C: Divide, don't multiply.
- D: Forgot the scaling.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B66. [MCQ · Easy · calculation]

A raw dot-product score is 8 with key dimension d_k = 4. What is the scaled score used in scaled dot-product attention?

A. 8.000
B. 16.000
C. 2.000
D. 4.000

Answer: D. 4.000

Working:

- $8/\sqrt{4} = 8/2 = 4.000$.

Option notes:

- A: Forgot the scaling.
- B: Divide, don't multiply.
- C: Divide by √d_k, not d_k.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B67. [MCQ · Easy · calculation]

A raw dot-product score is 24 with key dimension d_k = 64. What is the scaled score used in scaled dot-product attention?

A. 24.000
B. 3.000
C. 192.000
D. 0.375

Answer: B. 3.000

Working:

- 24/√64 = 24/8 = 3.000.

Option notes:

- A: Forgot the scaling.
- C: Divide, don't multiply.
- D: Divide by √d_k, not d_k.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B68. [MCQ · Easy · calculation]

A raw dot-product score is 7 with key dimension d_k = 64. What is the scaled score used in scaled dot-product attention?

A. 0.875
B. 56.000
C. 0.109
D. 7.000

Answer: A. 0.875

Working:

- 7/√64 = 7/8 = 0.875.

Option notes:

- B: Divide, don't multiply.
- C: Divide by √d_k, not d_k.
- D: Forgot the scaling.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B69. [MCQ · Easy · calculation]

A raw dot-product score is 14 with key dimension d_k = 4. What is the scaled score used in scaled dot-product attention?

A. 3.500
B. 7.000
C. 14.000
D. 28.000

Answer: B. 7.000

Working:

- 14/√4 = 14/2 = 7.000.

Option notes:

- A: Divide by √d_k, not d_k.
- C: Forgot the scaling.
- D: Divide, don't multiply.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B70. [MCQ · Easy · calculation]

A raw dot-product score is 6 with key dimension d_k = 4. What is the scaled score used in scaled dot-product attention?

A. 1.500
B. 6.000
C. 12.000
D. 3.000

Answer: D. 3.000

Working:

- 6/√4 = 6/2 = 3.000.

Option notes:

- A: Divide by √d_k, not d_k.
- B: Forgot the scaling.
- C: Divide, don't multiply.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B71. [MCQ · Easy · calculation]

A raw dot-product score is 4 with key dimension d_k = 64. What is the scaled score used in scaled dot-product attention?

A. 0.063
B. 32.000
C. 0.500
D. 4.000

Answer: C. 0.500

Working:

- 4/√64 = 4/8 = 0.500.

Option notes:

- A: Divide by √d_k, not d_k.
- B: Divide, don't multiply.
- D: Forgot the scaling.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B72. [MCQ · Easy · calculation]

A raw dot-product score is 20 with key dimension d_k = 16. What is the scaled score used in scaled dot-product attention?

A. 1.250
B. 5.000
C. 20.000
D. 80.000

Answer: B. 5.000

Working:

- 20/√16 = 20/4 = 5.000.

Option notes:

- A: Divide by √d_k, not d_k.
- C: Forgot the scaling.
- D: Divide, don't multiply.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B73. [MCQ · Easy · calculation]

A raw dot-product score is 4 with key dimension d_k = 16. What is the scaled score used in scaled dot-product attention?

A. 0.250
B. 4.000
C. 16.000
D. 1.000

Answer: D. 1.000

Working:

- 4/√16 = 4/4 = 1.000.

Option notes:

- A: Divide by √d_k, not d_k.
- B: Forgot the scaling.
- C: Divide, don't multiply.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B74. [MCQ · Easy · calculation]

A raw dot-product score is 17 with key dimension d_k = 16. What is the scaled score used in scaled dot-product attention?

A. 68.000
B. 17.000
C. 4.250
D. 1.063

Answer: C. 4.250

Working:

- 17/√16 = 17/4 = 4.250.

Option notes:

- A: Divide, don't multiply.
- B: Forgot the scaling.
- D: Divide by √d_k, not d_k.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B75. [MCQ · Easy · calculation]

A raw dot-product score is 20 with key dimension d_k = 4. What is the scaled score used in scaled dot-product attention?

A. 10.000
B. 20.000
C. 40.000
D. 5.000

Answer: A. 10.000

Working:

- 20/√4 = 20/2 = 10.000.

Option notes:

- B: Forgot the scaling.
- C: Divide, don't multiply.
- D: Divide by √d_k, not d_k.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B76. [MCQ · Easy · calculation]

A raw dot-product score is 8 with key dimension d_k = 64. What is the scaled score used in scaled dot-product attention?

A. 1.000
B. 8.000
C. 0.125
D. 64.000

Answer: A. 1.000

Working:

- 8/√64 = 8/8 = 1.000.

Option notes:

- B: Forgot the scaling.
- C: Divide by √d_k, not d_k.
- D: Divide, don't multiply.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B77. [MCQ · Easy · calculation]

A raw dot-product score is 10 with key dimension d_k = 16. What is the scaled score used in scaled dot-product attention?

A. 0.625
B. 2.500
C. 40.000
D. 10.000

Answer: B. 2.500

Working:

- 10/√16 = 10/4 = 2.500.

Option notes:

- A: Divide by √d_k, not d_k.
- C: Divide, don't multiply.
- D: Forgot the scaling.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B78. [MCQ · Easy · calculation]

A raw dot-product score is 14 with key dimension d_k = 4. What is the scaled score used in scaled dot-product attention?

A. 7.000
B. 14.000
C. 3.500
D. 28.000

Answer: A. 7.000

Working:

- 14/√4 = 14/2 = 7.000.

Option notes:

- B: Forgot the scaling.
- C: Divide by √d_k, not d_k.
- D: Divide, don't multiply.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B79. [MCQ · Easy · calculation]

A raw dot-product score is 17 with key dimension d_k = 4. What is the scaled score used in scaled dot-product attention?

A. 4.250
B. 34.000
C. 8.500
D. 17.000

Answer: C. 8.500

Working:

- 17/√4 = 17/2 = 8.500.

Option notes:

- A: Divide by √d_k, not d_k.
- B: Divide, don't multiply.
- D: Forgot the scaling.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B80. [MCQ · Easy · calculation]

A raw dot-product score is 12 with key dimension d_k = 16. What is the scaled score used in scaled dot-product attention?

A. 3.000
B. 0.750
C. 48.000
D. 12.000

Answer: A. 3.000

Working:

- 12/√16 = 12/4 = 3.000.

Option notes:

- B: Divide by √d_k, not d_k.
- C: Divide, don't multiply.
- D: Forgot the scaling.

Takeaway: Scaling keeps large-dimension dot products from saturating the softmax.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B81. [BLANKS · Medium · calculation]

X has 10 tokens of dimension 64 (X is 10×64); W_Q, W_K, W_V are 64×64. With 8 heads concatenated, give: the shape of Q, the shape of S = $QK^T/\sqrt{d_k}$, and the width of Concat($head_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 10x64; S shape = 10x10; concat width = 512

Working:

- Q = XW_Q: (10×64)(64×64) = 10×64.
- S = QK^T: (10×64)(64×10) = 10×10 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 64 = 512.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B82. [BLANKS · Medium · calculation]

X has 5 tokens of dimension 16 (X is 5×16); W_Q, W_K, W_V are 16×3. With 4 heads concatenated, give: the shape of Q, the shape of S = $QK^T/\sqrt{d_k}$, and the width of Concat($head_1$…head_4) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 5x3; S shape = 5x5; concat width = 12

Working:

- Q = XW_Q: (5×16)(16×3) = 5×3.
- S = QK^T: (5×3)(3×5) = 5×5 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 4 heads of width 3 = 12.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B83. [BLANKS · Medium · calculation]

X has 12 tokens of dimension 512 (X is 12×512); W_Q, W_K, W_V are 512×3. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 12x3; S shape = 12x12; concat width = 24

Working:

- Q = XW_Q: (12×512)(512×3) = 12×3.
- S = QK^T: (12×3)(3×12) = 12×12 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 3 = 24.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B84. [BLANKS · Medium · calculation]

X has 12 tokens of dimension 4 (X is 12×4); W_Q, W_K, W_V are 4×64. With 4 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_4) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 12x64; S shape = 12x12; concat width = 256

Working:

- Q = XW_Q: (12×4)(4×64) = 12×64.
- S = QK^T: (12×64)(64×12) = 12×12 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 4 heads of width 64 = 256.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B85. [BLANKS · Medium · calculation]

X has 5 tokens of dimension 64 (X is 5×64); W_Q, W_K, W_V are 64×8. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 5x8; S shape = 5x5; concat width = 64

Working:

- Q = XW_Q: (5×64)(64×8) = 5×8.
- S = QK^T: (5×8)(8×5) = 5×5 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 8 = 64.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B86. [BLANKS · Medium · calculation]

X has 5 tokens of dimension 4 (X is 5×4); W_Q, W_K, W_V are 4×3. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 5x3; S shape = 5x5; concat width = 24

Working:

- Q = XW_Q: (5×4)(4×3) = 5×3.
- S = QK^T: (5×3)(3×5) = 5×5 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 3 = 24.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B87. [BLANKS · Medium · calculation]

X has 12 tokens of dimension 4 (X is 12×4); W_Q, W_K, W_V are 4×64. With 2 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_2) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 12x64; S shape = 12x12; concat width = 128

Working:

- Q = XW_Q: (12×4)(4×64) = 12×64.
- S = QK^T: (12×64)(64×12) = 12×12 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 2 heads of width 64 = 128.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B88. [BLANKS · Medium · calculation]

X has 10 tokens of dimension 512 (X is 10×512); W_Q, W_K, W_V are 512×8. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 10x8; S shape = 10x10; concat width = 64

Working:

- Q = XW_Q: (10×512)(512×8) = 10×8.
- S = QK^T: (10×8)(8×10) = 10×10 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 8 = 64.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B89. [BLANKS · Medium · calculation]

X has 10 tokens of dimension 4 (X is 10×4); W_Q, W_K, W_V are 4×3. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 10x3; S shape = 10x10; concat width = 24

Working:

- Q = XW_Q: (10×4)(4×3) = 10×3.
- S = QK^T: (10×3)(3×10) = 10×10 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 3 = 24.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B90. [BLANKS · Medium · calculation]

X has 10 tokens of dimension 64 (X is 10×64); W_Q, W_K, W_V are 64×3. With 2 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_2) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 10x3; S shape = 10x10; concat width = 6

Working:

- Q = XW_Q: (10×64)(64×3) = 10×3.
- S = QK^T: (10×3)(3×10) = 10×10 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 2 heads of width 3 = 6.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B91. [BLANKS · Medium · calculation]

X has 5 tokens of dimension 512 (X is 5×512); W_Q, W_K, W_V are 512×8. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 5x8; S shape = 5x5; concat width = 64

Working:

- Q = XW_Q: (5×512)(512×8) = 5×8.
- S = QK^T: (5×8)(8×5) = 5×5 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 8 = 64.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B92. [BLANKS · Medium · calculation]

X has 12 tokens of dimension 16 (X is 12×16); W_Q, W_K, W_V are 16×8. With 4 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_4) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 12x8; S shape = 12x12; concat width = 32

Working:

- Q = XW_Q: (12×16)(16×8) = 12×8.
- S = QK^T: (12×8)(8×12) = 12×12 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 4 heads of width 8 = 32.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B93. [BLANKS · Medium · calculation]

X has 10 tokens of dimension 16 (X is 10×16); W_Q, W_K, W_V are 16×64. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 10x64; S shape = 10x10; concat width = 512

Working:

- Q = XW_Q: (10×16)(16×64) = 10×64.
- S = QK^T: (10×64)(64×10) = 10×10 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 64 = 512.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B94. [BLANKS · Medium · calculation]

X has 5 tokens of dimension 64 (X is 5×64); W_Q, W_K, W_V are 64×64. With 4 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_4) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 5x64; S shape = 5x5; concat width = 256

Working:

- Q = XW_Q: (5×64)(64×64) = 5×64.
- S = QK^T: (5×64)(64×5) = 5×5 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 4 heads of width 64 = 256.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B95. [BLANKS · Medium · calculation]

X has 5 tokens of dimension 64 (X is 5×64); W_Q, W_K, W_V are 64×64. With 2 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_2) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 5x64; S shape = 5x5; concat width = 128

Working:

- Q = XW_Q: (5×64)(64×64) = 5×64.
- S = QK^T: (5×64)(64×5) = 5×5 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 2 heads of width 64 = 128.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B96. [BLANKS · Medium · calculation]

X has 10 tokens of dimension 16 (X is 10×16); W_Q, W_K, W_V are 16×64. With 8 heads concatenated, give: the shape of Q, the shape of S = QK^T/√d_k, and the width of Concat(head$_1$…head_8) before W^O.

(1) Q shape = ____
(2) S shape = ____
(3) concat width = ____

Answer: Q shape = 10x64; S shape = 10x10; concat width = 512

Working:

- Q = XW_Q: (10×16)(16×64) = 10×64.
- S = QK^T: (10×64)(64×10) = 10×10 (one score per query–key pair; Lecture 8: 8×3 × 3×8 = 8×8).
- Concat of 8 heads of width 64 = 512.

Takeaway: Each ROW of A = row-softmax(S) sums to 1; Z = AV has one row per token.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B97. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 16, compute PE(3, 3).

Answer: 0.5828

Working:

- i = 1; exponent 2i/d_model = 0.125; angle = 3/10000^0.125 = 0.94868.
- cos(0.94868) = 0.5828.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B98. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 4, compute PE(0, 1).

Answer: 1.0000

Working:

- i = 0; exponent 2i/d_model = 0; angle = 0/10000^0 = 0.00000.
- cos(0.00000) = 1.0000.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B99. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 4, compute PE(5, 3).

Answer: 0.9988

Working:

- i = 1; exponent 2i/d_model = 0.5; angle = 5/10000^0.5 = 0.05000.
- cos(0.05000) = 0.9988.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B100. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 16, compute PE(5, 0).

Answer: -0.9589

Working:

- i = 0; exponent 2i/d_model = 0; angle = 5/10000^0 = 5.00000.
- sin(5.00000) = -0.9589.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B101. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 8, compute PE(0, 3).

Answer: 1.0000

Working:

- i = 1; exponent 2i/d_model = 0.25; angle = 0/10000^0.25 = 0.00000.

- cos(0.00000) = 1.0000.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B102. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 4, compute PE(1, 1).

Answer: 0.5403

Working:

- i = 0; exponent 2i/d_model = 0; angle = 1/10000^0 = 1.00000.
- cos(1.00000) = 0.5403.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B103. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 16, compute PE(5, 0).

Answer: -0.9589

Working:

- i = 0; exponent 2i/d_model = 0; angle = 5/10000^0 = 5.00000.
- sin(5.00000) = -0.9589.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B104. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 8, compute PE(5, 3).

Answer: 0.8776

Working:

- i = 1; exponent 2i/d_model = 0.25; angle = 5/10000^0.25 = 0.50000.
- cos(0.50000) = 0.8776.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B105. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 4, compute PE(5, 1).

Answer: 0.2837

Working:

- i = 0; exponent 2i/d_model = 0; angle = 5/10000^0 = 5.00000.
- cos(5.00000) = 0.2837.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B106. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 4, compute PE(4, 2).

Answer: 0.0400

Working:

- i = 1; exponent 2i/d_model = 0.5; angle = 4/10000^0.5 = 0.04000.
- sin(0.04000) = 0.0400.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B107. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 8, compute PE(0, 3).

Answer: 1.0000

Working:

- i = 1; exponent 2i/d_model = 0.25; angle = 0/10000^0.25 = 0.00000.
- cos(0.00000) = 1.0000.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B108. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 8, compute PE(6, 1).

Answer: 0.9602

Working:

- i = 0; exponent 2i/d_model = 0; angle = 6/10000^0 = 6.00000.
- cos(6.00000) = 0.9602.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B109. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 16, compute PE(3, 1).

Answer: -0.9900

Working:

- i = 0; exponent 2i/d_model = 0; angle = 3/10000^0 = 3.00000.
- cos(3.00000) = -0.9900.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B110. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 16, compute PE(0, 2).

Answer: 0.0000

Working:

- i = 1; exponent 2i/d_model = 0.125; angle = 0/10000^0.125 = 0.00000.
- sin(0.00000) = 0.0000.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B111. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 16, compute PE(1, 2).

Answer: 0.3110

Working:

- i = 1; exponent 2i/d_model = 0.125; angle = 1/10000^0.125 = 0.31623.
- sin(0.31623) = 0.3110.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

ATTN B112. [BLANK · Hard · calculation]

Using PE(pos, 2i) = sin(pos/10000^{2i/d_model}) and PE(pos, 2i+1) = cos(pos/10000^{2i/d_model}) with d_model = 16, compute PE(4, 1).

Answer: -0.6536

Working:

- i = 0; exponent 2i/d_model = 0; angle = 4/10000^0 = 4.00000.
- cos(4.00000) = -0.6536.

Takeaway: Positional encodings are ADDED to token embeddings so attention knows word order.

Source: Lecture 8 (Self-attention) + Key points about Attention

---

🧠 Neural Networks (Workshop 5)

Part A: hand-written questions (79)

NN A1. [MCQ · Easy]

Each unit j first computes $in_j = \Sigma\, w_{ij}a_i$. What does it do next?

A. Multiplies by the learning rate
B. Sends in_j unchanged to the next layer
C. Applies an activation function g to get $out_j = g(in_j)$
D. Normalises by the number of inputs

Answer: C. Applies an activation function g to get $out_j = g(in_j)$

Option notes:

- A: Only in training updates.
- B: Activation is applied.
- D: Not part of the unit.

Takeaway: Weighted sum, then activation.

Source: Workshop 5, slide 7

NN A2. [MATCH · Medium]

Match each activation to its typical use (Workshop 5).

1. ReLU
2. Sigmoid
3. Tanh
4. Softplus

A. probability-style output
B. efficient hidden layers
C. smooth ReLU alternative
D. counting parameters
E. centred output scale

Answer: 1 → B, 2 → A, 3 → E, 4 → C

Working:

- ReLU → efficient hidden layers
- Sigmoid → probability-style output
- Tanh → centred output scale
- Softplus → smooth ReLU alternative

Takeaway: Choose by layer purpose.

Source: Workshop 5, slide 9

NN A3. [TF · Easy]

The universal approximation theorem says a network with one non-linear layer followed by a linear layer can approximate any continuous function to arbitrary accuracy.

Answer: True

Why: Workshop 5 slide 22.

Takeaway: Width, not depth, in the theorem.

Source: Workshop 5, slide 22

NN A4. [BLANK · Medium · calculation]

A dense layer with 4 inputs and 3 units (each with a bias) has _____ parameters.

Answer: 15

Takeaway: (4 + 1) × 3.

Source: Workshop 5 (Neural network models)

NN A5. [MULTI · Medium]

Why do we need (different) activation functions? (Select ALL that apply.)

A. To shape responses to negative, small or large inputs
B. To add non-linearity so networks can learn complex patterns
C. To match the layer's purpose (hidden vs output)
D. To improve learning speed, gradient flow and stability
E. To reduce the number of weights

Answer: A. To shape responses to negative, small or large inputs; B. To add non-linearity so networks can learn complex patterns; C. To match the layer's purpose (hidden vs output); D. To improve learning speed, gradient flow and stability

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Correct:
- E: Incorrect: Activations add no weights.

Takeaway: Workshop 5 slide 9.

Source: Workshop 5, slide 9

NN A6. [MCQ · Medium]

Which equals tanh(x) (Python warm-up 2)?

A. 2/(1 + e^(−2x)) − 1
B. 1/(1 + e^(−x))
C. log(1 + e^x)
D. max(0, x)

Answer: A. 2/(1 + e^(−2x)) − 1

Option notes:

- B: That's sigmoid.
- C: Softplus.
- D: ReLU.

Takeaway: tanh = 2σ(2x) − 1.

Source: Python additional warm-up 2

NN A7. [TF · Easy]

A McCulloch-Pitts style unit fires when a linear combination of its inputs exceeds a (hard or soft) threshold.

Answer: True

Why: Workshop 5 slide 6.

Takeaway: Neuron model (1943).

Source: Workshop 5, slide 6

NN A8. [MCQ · Medium]

What does the bias w_0 do in a perceptron?

A. Shifts the activation function curve left or right
B. Sets the learning rate
C. Scales the inputs to [0, 1]
D. Counts the number of inputs

Answer: A. Shifts the activation function curve left or right

Option notes:

- B: Separate hyperparameter.
- C: That's normalisation.
- D: No.

Takeaway: Positive bias shifts sigmoid left (centre at $x = -b$).

Source: Workshop 5, slide 12-14

NN A9. [TF · Medium]

If the perceptron already classifies an example correctly, the perceptron learning rule leaves its weights unchanged.

Answer: True

Why: $y - h = 0 \rightarrow$ no change.

Takeaway: Learning only from mistakes.

Source: Workshop 5 (Neural network models)

NN A10. [BLANKS · Medium · calculation]

Perceptron: $w_0 = -1$, $w_1 = 1$, $w_2 = 1$, $x = (1, 0)$, hard threshold (1 if $ws \geq 0$). Give ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 0; output = 1

Takeaway: $-1 + 1 + 0 = 0 \rightarrow ws \geq 0 \rightarrow 1$.

Source: Workshop 5 (Neural network models)

NN A11. [MCQ · Hard · calculation]

Perceptron with $w = (w_0, w_1, w_2) = (-1, 1, 1)$, $x = (1, 0)$, $y = 0$, $\alpha = 0.5$. After one update?

A. (−0.5, 1.5, 1)
B. (−1, 1, 1)

C. (−1.5, 0.5, 0.5)
D. (−1.5, 0.5, 1)

Answer: D. (−1.5, 0.5, 1)

Option notes:

- A: Wrong sign: $y - h = -1$.
- B: It misclassified, so weights change.
- C: $x_2 = 0$, so w_2 doesn't change.

Takeaway: $h = 1$, error -1: $w \leftarrow w - 0.5 \cdot (1, 1, 0)$.

Source: Workshop 5 (Neural network models)

NN A12. [MATCH · Medium]

Match the learning rule to its activation (Workshop 5 slide 17).

1. Hard threshold
2. Sigmoid

A. $w_i \leftarrow w_i + \alpha(y - h)x_i$
B. $w_i \leftarrow w_i - \alpha x_i$ always
C. $w_i \leftarrow w_i + \alpha(y - h)h(1 - h)x_i$

Answer: 1 → A, 2 → C

Working:

- Hard threshold → $w_i \leftarrow w_i + \alpha(y - h)x_i$
- Sigmoid → $w_i \leftarrow w_i + \alpha(y - h)h(1 - h)x_i$

Takeaway: Sigmoid adds the derivative $h(1 - h)$.

Source: Workshop 5, slide 17

NN A13. [MCQ · Medium]

What is the derivative of the sigmoid $\sigma(z)$?

A. $\sigma(z)^2$
B. e^(−z)
C. $\sigma(z)(1 - \sigma(z))$
D. $1 - \sigma(z)$

Answer: C. $\sigma(z)(1 - \sigma(z))$

Option notes:

- A: Missing $(1 - \sigma)$.
- B: Not the derivative of σ.
- D: Missing $\sigma(z)$.

Takeaway: Max value 0.25 at $z = 0$.

Source: Workshop 5, slide 27

NN A14. [TF · Medium]

Modern neural networks are almost always trained with some variant of stochastic gradient descent, such as Adam.

Answer: True

Why: Workshop 5 slide 35.

Takeaway: Optimiser chosen at compile time in keras.

Source: Workshop 5 (Neural network models)

NN A15. [BLANK · Hard · calculation]

Sigmoid unit, h = 0.5, y = 1, α = 1, $x_1 = 2$. Rule $w_i \leftarrow w_i + \alpha(y - h)h(1 - h)x_i$: the change in w_1 is _____.

Answer: 0.25
(Also accepted: +0.25)

Takeaway: 0.5 × 0.25 × 2 = 0.25.

Source: Workshop 5 (Neural network models)

NN A16. [MCQ · Medium]

In backpropagation, the gradient of the loss with respect to weight $w_{i,j}$ equals…

A. $a_i \times \Delta_j$ (incoming activation × perceived error at unit j)
B. $a_j \times \Delta_i$
C. $\Delta_i \times \Delta_j$
D. $w_{i,j} \times \Delta_j$

Answer: A. $a_i \times \Delta_j$ (incoming activation × perceived error at unit j)

Option notes:

- B: Indices reversed.
- C: Only one Δ.
- D: That's used to pass the error back, not the gradient.

Takeaway: Workshop 5 slide 32.

Source: Workshop 5, slide 32

NN A17. [MULTI · Medium]

Select ALL correct statements about the learning process of a feedforward NN. (Select ALL that apply.)

A. Forward phase passes activations from input to output
B. Backpropagation needs no derivatives
C. The perceived error at a hidden unit uses the errors of the units it feeds
D. Backward phase propagates errors from output to input
E. Weights are updated using gradient descent

Answer: A. Forward phase passes activations from input to output; C. The perceived error at a hidden unit uses the errors of the units it feeds; D. Backward phase propagates errors from output to input; E. Weights are updated using gradient descent

Option notes:

- A: Correct:
- B: Incorrect: It uses the chain rule.
- C: Correct: $\Delta_i = g'(in_i)w_{ij}\Delta_j$.
- D: Correct:
- E: Correct:

Takeaway: Forward then backward.

Source: Workshop 5, slide 23-32

NN A18. [BLANK · Hard · calculation]

Output unit: $\hat{y} = 0.8$, $y = 1$, sigmoid. $\Delta_5 = 2(\hat{y} - y)\hat{y}(1 - \hat{y})$ = _____ (3 dp).

Answer: -0.064
(Also accepted: −0.064)

Takeaway: $2 \times (-0.2) \times 0.8 \times 0.2$.

Source: Workshop 5 (Neural network models)

NN A19. [TF · Medium]

The vanishing gradient problem means error signals grow larger as they are propagated back through the network.

Answer: False

Why: They are EXTINGUISHED (shrink) as they propagate back.

Takeaway: Growing = exploding.

Source: Workshop 5, slide 33

NN A20. [MCQ · Medium]

In keras, model.compile(loss='mean_squared_error', optimizer='adam', metrics=['mean_absolute_error']). What is MAE here?

A. The loss being minimised
B. A layer
C. A metric reported during training, not the loss being minimised
D. The optimiser

Answer: C. A metric reported during training, not the loss being minimised

Option notes:

- A: MSE is the loss.
- B: No.
- D: Adam is the optimiser.

Takeaway: Loss vs metric.

Source: Workshop 5, slide 34

NN A21. [MCQ · Medium]

Which LSTM gate decides whether each element of the memory cell is kept or reset to zero?

A. Reset gate
B. Input gate
C. Forget gate
D. Output gate

Answer: C. Forget gate

Option notes:

- A: That's a GRU term.
- B: Adds new information.
- D: Exposes memory to z_t.

Takeaway: f ≈ 1 keep, f ≈ 0 forget.

Source: Workshop 5, slide 42

NN A22. [TF · Medium]

In an LSTM, new information enters the memory cell by multiplication, which is why gradients vanish less.

Answer: False

Why: New information enters by ADDING updates, so gradients don't accumulate multiplicatively.

Takeaway: Additive cell path = nearly linear gradient flow.

Source: Workshop 5, slide 41

NN A23. [MCQ · Hard]

For a simple RNN with recurrent weight w_zz and activation derivatives $g' \le 1$, which is true?

A. Exploding happens when w_zz < 1
B. w_zz has no effect on gradients
C. Gradients never vanish with tanh
D. If w_zz 1 they may explode

Answer: D. If w_zz 1 they may explode

Option notes:

- A: Reversed.
- B: Gradients ∝ w_zz^T Π g′.
- C: $g' \le 1$ for tanh.

Takeaway: Workshop 5 slide 40.

Source: Workshop 5, slide 40

NN A24. [BLANKS · Medium · calculation]

$c_{t-1} = 0.5$, $f_t = 0.8$, $i_t = 0.4$, candidate $\tilde{c}_t = 0.5$. $c_t = c_{t-1} \cdot f_t + i_t \cdot \tilde{c}_t$ = _____.

(1) c_t = ____

Answer: $c_t = 0.6$

Takeaway: 0.4 + 0.2.

Source: Workshop 5 (Neural network models)

NN A25. [MATCH · Medium]

Match each LSTM gate to its role.

1. Forget gate f
2. Input gate i
3. Output gate o

A. compute the loss
B. add new information from the input
C. remember or reset memory elements
D. transfer memory to the short-term state z

Answer: 1 → C, 2 → B, 3 → D

Working:

- Forget gate f → remember or reset memory elements
- Input gate i → add new information from the input
- Output gate o → transfer memory to the short-term state z

Takeaway: F-I-O.

Source: Workshop 5, slide 42

NN A26. [MCQ · Medium]

The RNN Markov assumption in Workshop 5 states that…

A. the hidden state z_t suffices to capture the information from all previous inputs
B. outputs depend only on the first input
C. each input is independent of all others
D. the network forgets everything each step

Answer: A. the hidden state z_t suffices to capture the information from all previous inputs

Option notes:

- B: No.
- C: That's i.i.d.
- D: No: z_t carries context.

Takeaway: $z_t = f_w(z_{t-1}, x_t)$.

Source: Workshop 5, slide 37

NN A27. [BLANKS · Easy · calculation]

A neuron has inputs x = (2, −1), weights (0.5, 1.5), bias 0.25. z = ___; with ReLU, a = ___.

(1) z = _____
(2) ReLU(z) = _____

Answer: z = -0.25; ReLU(z) = 0

Working:

- 0.5(2) + 1.5(−1) + 0.25 = −0.25
- ReLU(−0.25) = 0

Takeaway: Weighted sum, then activation.

Source: Book 2 (Exam examples), Q135, p. 22

NN A28. [BLANK · Medium · calculation]

Same neuron (z = −0.25) with a sigmoid: a = ___ (3 d.p.).

Answer: 0.438

Takeaway: 1/(1 + e^0.25).

Source: Book 2 (Exam examples), Q136, p. 22

NN A29. [BLANK · Easy · calculation]

Same neuron (z = −0.25) with a step function (1 if z ≥ 0): a = ___.

Answer: 0

Takeaway: z < 0.

Source: Book 2 (Exam examples), Q137, p. 22

NN A30. [BLANKS · Medium · calculation]

tanh(0) = ___, σ(0) = ___, ReLU(0) = ___, softplus(0) = ___ (3 d.p.).

(1) tanh(0) = ____
(2) σ(0) = ____
(3) ReLU(0) = ____
(4) softplus(0) = ____

Answer: tanh(0) = 0; σ(0) = 0.5; ReLU(0) = 0; softplus(0) = 0.693

Takeaway: softplus(0) = ln 2.

Source: Book 2 (Exam examples), Q138, p. 22

NN A31. [MATCH · Easy]

Match the activation to its output range.

1. sigmoid
2. tanh
3. ReLU
4. step

A. (0, 1)
B. {0, 1}
C. [0, ∞)
D. (−1, 1)

Answer: 1 → A, 2 → D, 3 → C, 4 → B

Working:

- sigmoid → (0, 1)
- tanh → (−1, 1)
- ReLU → [0, ∞)
- step → {0, 1}

Takeaway: Know the ranges.

Source: Book 2 (Exam examples), Q139, p. 22

NN A32. [MCQ · Easy]

Which activation is called 'a smooth version of ReLU'?

A. sigmoid
B. tanh
C. step
D. softplus

Answer: D. softplus

Option notes:

- A: S-curve in (0, 1).
- B: Zero-centred S-curve.
- C: Not smooth.

Takeaway: softplus = ln(1 + e^z).

Source: Book 2 (Exam examples), Q140, p. 22

NN A33. [BLANKS · Medium · calculation]

Perceptron (hard threshold), w = (w0, w1, w2) = (−0.5, 0.2, 0.4), input x = (1, 0, 1) with x0 = 1, target y = 0, α = 0.1. Weighted sum ___; output ___; new w0, w1, w2 = ___, ___, ___.

(1) sum = _____
(2) output = _____
(3) w0 = _____
(4) w1 = _____
(5) w2 = _____

Answer: sum = -0.1; output = 0; w0 = -0.5; w1 = 0.2; w2 = 0.4

Working:

- −0.5 + 0 + 0.4 = −0.1 < 0 → output 0
- y − h = 0 → weights unchanged

Takeaway: Correct output → no change.

Source: Book 2 (Exam examples), Q141, p. 22

NN A34. [BLANKS · Hard · calculation]

Same perceptron (w = (−0.5, 0.2, 0.4), α = 0.1), input x = (1, 1, 1), target y = 0. Output ___; new w0, w1, w2 = ___, ___, ___.

(1) output = _____
(2) w0 = _____
(3) w1 = _____
(4) w2 = _____

Answer: output = 1; w0 = -0.6; w1 = 0.1; w2 = 0.3

Working:

- sum = −0.5 + 0.2 + 0.4 = 0.1 ≥ 0 → output 1 (wrong)
- y − h = −1 → subtract 0.1·x
- (−0.6, 0.1, 0.3)

Takeaway: w ← w + α(y − h)x.

Source: Book 2 (Exam examples), Q142, p. 22

NN A35. [TF · Easy]

If the perceptron output equals the target, the weights do not change.

Answer: True

Why: y − h = 0.

Takeaway: No error, no update.

Source: Book 2 (Exam examples), Q143, p. 22

NN A36. [BLANKS · Medium · calculation]

Sigmoid neuron: w = (0.1, 0.4, −0.3) (first is bias), x = (1, 2, 1), target y = 1, squared loss. z = ___; h = σ(z) = ___ (3 d.p.); loss = ___ (3 d.p.).

(1) z = _____
(2) h = _____
(3) loss = _____

Answer: z = 0.6; h = 0.646; loss = 0.126

Working:

- z = 0.1 + 0.8 − 0.3 = 0.6
- h = σ(0.6) = 0.646
- loss = (1 − 0.646)² = 0.126
- ∂L/∂w1 = −2(0.354)(0.646)(0.354)(2) = −0.324
- w1 = 0.4 − 0.5(−0.324) = 0.562

Takeaway: Forward pass first.

Source: Book 2 (Exam examples), Q144, p. 23

NN A37. [BLANK · Hard · calculation]

Same neuron (h = 0.646, y = 1, x1 = 2). ∂L/∂w1 = −2(y − h)h(1 − h)x1 = ___ (3 d.p.).

Answer: -0.324

Working:

- z = 0.1 + 0.8 − 0.3 = 0.6
- h = σ(0.6) = 0.646
- loss = (1 − 0.646)² = 0.126
- ∂L/∂w1 = −2(0.354)(0.646)(0.354)(2) = −0.324
- w1 = 0.4 − 0.5(−0.324) = 0.562

Takeaway: Chain rule.

Source: Book 2 (Exam examples), Q145, p. 23

NN A38. [BLANK · Hard · calculation]

Same neuron, α = 0.5, w1 = 0.4, ∂L/∂w1 = −0.324. New w1 = ___ (3 d.p.).

Answer: 0.562

Working:

- z = 0.1 + 0.8 − 0.3 = 0.6
- h = σ(0.6) = 0.646
- loss = (1 − 0.646)² = 0.126
- ∂L/∂w1 = −2(0.354)(0.646)(0.354)(2) = −0.324
- w1 = 0.4 − 0.5(−0.324) = 0.562

Takeaway: w ← w − α·gradient.

Source: Book 2 (Exam examples), Q146, p. 23

NN A39. [MCQ · Medium]

Why is h(1 − h) in the sigmoid-neuron gradient?

A. it is the learning rate
B. it is the derivative of the sigmoid
C. it is the bias
D. it is the loss

Answer: B. it is the derivative of the sigmoid

Option notes:

- A: That is α.

- C: The bias is w0.
- D: The loss is $(y - h)^2$.

Takeaway: $\sigma' = \sigma(1 - \sigma)$.

Source: Book 2 (Exam examples), Q147, p. 23

NN A40. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): x_1 — 3 ($w_{1,3}$); x_1 — 4 ($w_{1,4}$); x_2 — 3 ($w_{2,3}$); x_2 — 4 ($w_{2,4}$); 3 — 5 ($w_{3,5}$); 4 — 5 ($w_{4,5}$)

2-2-1 network, x = (1, 2). Hidden ReLU: w1,3 = 0.5, w2,3 = −0.25, b3 = 0.1; w1,4 = −1, w2,4 = 0.5, b4 = 0. in3 = ___, a3 = ___; in4 = ___, a4 = ___.

(1) in3 = _____
(2) a3 = _____
(3) in4 = _____
(4) a4 = _____

Answer: in3 = 0.1; a3 = 0.1; in4 = 0; a4 = 0

Working:

- in3 = 0.5 − 0.5 + 0.1 = 0.1
- in4 = −1 + 1 + 0 = 0

Takeaway: ReLU keeps positives.

Source: Book 2 (Exam examples), Q148, p. 23

NN A41. [BLANKS · Medium · calculation]

[Diagram as text] Edges (label in brackets): x_1 — 3 ($w_{1,3}$); x_1 — 4 ($w_{1,4}$); x_2 — 3 ($w_{2,3}$); x_2 — 4 ($w_{2,4}$); 3 — 5 ($w_{3,5}$); 4 — 5 ($w_{4,5}$)

Same network: a3 = 0.1, a4 = 0. Output (sigmoid): w3,5 = 1, w4,5 = 2, b5 = −0.5. in5 = ___; $\hat{y}$ = ___ (3 d.p.).

(1) in5 = _____
(2) $\hat{y}$ = _____

Answer: in5 = -0.4; $\hat{y}$ = 0.401

Working:

- in5 = 0.1 + 0 − 0.5 = −0.4
- $\sigma(-0.4) = 0.401$

Takeaway: $\sigma(-0.4)$.

Source: Book 2 (Exam examples), Q149, p. 23

NN A42. [BLANK · Medium · calculation]

A fully connected network 4–5–3–2 (inputs–hidden–hidden–outputs) with biases has ___ parameters.

Answer: 51

Working:

- (4 + 1)5 = 25
- (5 + 1)3 = 18
- (3 + 1)2 = 8
- Total 51

Takeaway: (in + 1) × out per layer.

Source: Book 2 (Exam examples), Q150, p. 23

NN A43. [BLANK · Medium · calculation]

A dense layer connecting a 224 × 224 × 3 image to 1,000 units has ___ weights (ignore biases).

Answer: 150528000
(Also accepted: 150,528,000)

Working:

- 224 × 224 × 3 = 150,528 inputs
- × 1,000 = 150,528,000

Takeaway: Why dense layers on images explode.

Source: Book 2 (Exam examples), Q151, p. 23

NN A44. [BLANKS · Medium]

Universal approximation theorem: a network with two layers of computational units, the first ___ and the second ___, can approximate any continuous function.

(1) first layer = ____
(2) second layer = ____

Answer: first layer = non-linear; second layer = linear

Takeaway: Non-linear hidden, linear output.

Source: Book 2 (Exam examples), Q152, p. 24

NN A45. [BLANKS · Easy]

The first mathematical neuron model was devised by ___ and ___ in 1943.

(1) first = ____
(2) second = ____

Answer: first = McCulloch; second = Pitts

Takeaway: McCulloch and Pitts.

Source: Book 2 (Exam examples), Q153, p. 24

NN A46. [TF · Easy]

Error signals shrinking as they propagate back through many layers is called the vanishing gradient problem.

Answer: True

Why: Sigmoid slopes ≤ 0.25 multiply together.

Takeaway: Vanishing gradient.

Source: Book 2 (Exam examples), Q154, p. 24

NN A47. [MCQ · Medium]

In Keras, the optimiser (e.g. Adam) is specified in the:

A. Dense layer
B. Input layer
C. compile step
D. fit step

Answer: C. compile step

Option notes:

- A: Layers set units and activation.
- B: Sets input shape.
- D: fit sets epochs and batch size.

Takeaway: model.compile(loss, optimizer, metrics).

Source: Book 2 (Exam examples), Q155, p. 24

NN A48. [MULTI · Medium]

To define an FNN's hypothesis space you specify: (Select ALL that apply.)

A. the test set
B. number of outputs
C. number of inputs
D. number of hidden layers
E. neurons and activation per hidden layer

Answer: B. number of outputs; C. number of inputs; D. number of hidden layers; E. neurons and activation per hidden layer

Option notes:

- A: Incorrect: data, not the hypothesis space
- B: Correct:
- C: Correct:
- D: Correct:
- E: Correct:

Takeaway: Architecture = hypothesis space.

Source: Book 2 (Exam examples), Q156, p. 24

NN A49. [BLANK · Easy]

RNN hidden-state update: z_t = g_z(W_zz z_(t−1) + W_xz ___).

Answer: x_t
(Also accepted: xt, x(t))

Takeaway: The current input.

Source: Book 2 (Exam examples), Q157, p. 24

NN A50. [TF · Medium]

In a simple RNN with w_zz > 1 we may get exploding gradients.

Answer: True

Why: Repeated multiplication by w_zz > 1 grows.

Takeaway: w_zz 1 explodes.

Source: Book 2 (Exam examples), Q158, p. 24

NN A51. [MATCH · Medium]

Match the LSTM gate to its job.

1. forget gate
2. input gate
3. output gate

A. whether each cell element is kept or reset to zero
B. whether the cell is transferred to the hidden state z
C. whether the cell is updated additively with new information

Answer: 1 → A, 2 → C, 3 → B

Working:

- forget gate → whether each cell element is kept or reset to zero
- input gate → whether the cell is updated additively with new information
- output gate → whether the cell is transferred to the hidden state z

Takeaway: f keep/reset, i write, o expose.

Source: Book 2 (Exam examples), Q159, p. 24

NN A52. [BLANK · Easy]

The algorithm used to compute RNN gradients is ___.

Answer: back-propagation through time
(Also accepted: BPTT, backpropagation through time, back propagation through time)

Takeaway: Unroll, then backprop.

Source: Book 2 (Exam examples), Q160, p. 24

NN A53. [MCQ · Medium]

Why do LSTMs suffer less from vanishing gradients?

A. they have fewer weights
B. the memory cell is updated additively, so gradients don't multiply repeatedly
C. they have no activation functions
D. they use ReLU only

Answer: B. the memory cell is updated additively, so gradients don't multiply repeatedly

Option notes:

- A: They have more.
- C: They use sigmoid and tanh.
- D: No.

Takeaway: Constant error flow through c_t.

Source: Book 2 (Exam examples), Q161, p. 24

NN A54. [BLANKS · Medium · calculation]

Perceptron, w = (0, 1, −1) (bias first), x = (1, 0.5, 1), y = 1, α = 0.5, step activation. New w0, w1, w2 = ___.

(1) w0 = ____
(2) w1 = ____
(3) w2 = ____

Answer: w0 = 0.5; w1 = 1.25; w2 = -0.5

Takeaway: Sum −0.5 → output 0 (wrong) → w + 0.5(1 − 0)x.

Source: Book 2 (Exam examples), Mixed practice paper Q12, p. 54

NN A55. [TF · Medium]

Sigmoid derivatives are at most 0.25, which contributes to vanishing gradients in deep sigmoid networks.

Answer: True

Why: $\sigma' = \sigma(1 - \sigma) \leq 0.25$.

Takeaway: $0.25^{10} \approx 10^{-6}$.

Source: Book 2 (Exam examples), Mixed practice paper Q13, p. 54

NN A56. [BLANK · Easy · calculation]

Inputs (2, −1), weights (0.5, 1), bias 0.5, ReLU. Output = ___.

Answer: 0.5

Takeaway: 1 − 1 + 0.5.

Source: Book 3 (AI from zero), Practice 16.1, p. 66

NN A57. [BLANK · Easy · calculation]

Same neuron with sigmoid: output = ___ (3 d.p.).

Answer: 0.622

Takeaway: σ(0.5).

Source: Book 3 (AI from zero), Practice 16.2, p. 66

NN A58. [BLANK · Medium · calculation]

Parameters in a 10–20–1 fully connected network (with biases) = ___.

Answer: 241

Takeaway: 220 + 21.

Source: Book 3 (AI from zero), Practice 16.3, p. 66

NN A59. [MCQ · Medium]

Why can't a single perceptron learn XOR?

A. it has no bias
B. the classes are not separable by one straight line
C. XOR has three inputs
D. the learning rate is too small

Answer: B. the classes are not separable by one straight line

Option notes:

- A: Bias does not fix it.
- C: Two inputs.
- D: No α helps.

Takeaway: Needs a hidden layer.

Source: Book 3 (AI from zero), Practice 16.4, p. 67

NN A60. [BLANK · Hard · calculation]

Backprop: $\Delta 5 = 0.2$, $w_{3,5} = -1$, $a_3 = 0.5$ (sigmoid). $\Delta 3 = g'(in3)\, w_{3,5}\, \Delta 5$ = ___.

Answer: -0.05

Takeaway: g′ = 0.5 × 0.5 = 0.25.

Source: Book 3 (AI from zero), Practice 16.5, p. 67

NN A61. [BLANKS · Medium · calculation]

AND neuron: w0 = −1.5, w1 = w2 = 1, step activation. Outputs for (1,1), (1,0), (0,0):

(1) (1,1) = ____
(2) (1,0) = ____
(3) (0,0) = ____

Answer: (1,1) = 1; (1,0) = 0; (0,0) = 0

Takeaway: 0.5 ≥ 0; −0.5; −1.5.

Source: Book 3 (AI from zero), Chapter 16, p. 62

NN A62. [BLANKS · Medium · calculation]

Perceptron w = (−0.5, 0.2, 0.4), x = (1, 0, 1), α = 0.1, output 0, but target y = 1. New weights?

(1) w0 = ____
(2) w1 = ____
(3) w2 = ____

Answer: w0 = -0.4; w1 = 0.2; w2 = 0.5

Takeaway: w + 0.1(1)x.

Source: Book 3 (AI from zero), Chapter 16, p. 62

NN A63. [BLANKS · Hard · calculation]

Weight | Value
w1,3 / w2,3 / b3 | 0.4 / −0.2 / 0.1
w1,4 / w2,4 / b4 | 0.3 / 0.6 / −0.1
w3,5 / w4,5 / b5 | 0.7 / −0.5 / 0.2

x = (1, 0.5), y = 1, α = 0.5, sigmoid everywhere, squared loss.

Full backprop example (table shown). Forward pass: a3 = ___, a4 = ___, ŷ = ___ (4 d.p.).

(1) a3 = ____
(2) a4 = ____
(3) ŷ = ____

Answer: a3 = 0.5987; a4 = 0.6225; ŷ = 0.5764

Takeaway: in3 = 0.4, in4 = 0.5, in5 = 0.3079.

Source: Book 3 (AI from zero), Chapter 16, p. 62

NN A64. [BLANKS · Hard · calculation]

Weight | Value
w1,3 / w2,3 / b3 | 0.4 / −0.2 / 0.1
w1,4 / w2,4 / b4 | 0.3 / 0.6 / −0.1
w3,5 / w4,5 / b5 | 0.7 / −0.5 / 0.2

x = (1, 0.5), y = 1, α = 0.5, sigmoid everywhere, squared loss.

Same example. Δ5 = 2(y − ŷ)ŷ(1 − ŷ) = ___; Δ3 = ___; Δ4 = ___ (4 d.p.).

(1) Δ5 = ____
(2) Δ3 = ____
(3) Δ4 = ____

Answer: Δ5 = 0.2069; Δ3 = 0.0348; Δ4 = -0.0243

Takeaway: Δj = g′(inj) wj,5 Δ5.

Source: Book 3 (AI from zero), Chapter 16, p. 62

NN A65. [BLANKS · Hard · calculation]

Weight | Value
w1,3 / w2,3 / b3 | 0.4 / −0.2 / 0.1
w1,4 / w2,4 / b4 | 0.3 / 0.6 / −0.1
w3,5 / w4,5 / b5 | 0.7 / −0.5 / 0.2

x = (1, 0.5), y = 1, α = 0.5, sigmoid everywhere, squared loss.

Same example, updates wi,j ← wi,j + α ai Δj: new w3,5 = ___; new w1,3 = ___ (3 d.p.).

(1) w3,5 = ____
(2) w1,3 = ____

Answer: w3,5 = 0.762; w1,3 = 0.417

Takeaway: 0.7 + 0.5(0.5987)(0.2069); 0.4 + 0.5(1)(0.0348).

Source: Book 3 (AI from zero), Chapter 16, p. 62

NN A66. [TF · Medium]

Without activation functions, a deep network of weighted sums collapses into one weighted sum (a linear model).

Answer: True

Why: Non-linearity lets networks learn curves.

Takeaway: Activations add non-linearity.

Source: Book 3 (AI from zero), Chapter 16, p. 62

NN A67. [MATCH · Easy]

Match the training term.

1. epoch
2. batch
3. Adam

A. a hidden layer
B. a variant of stochastic gradient descent
C. one pass through all training data
D. the chunk used for one gradient step

Answer: 1 → C, 2 → D, 3 → B

Working:

- epoch → one pass through all training data
- batch → the chunk used for one gradient step
- Adam → a variant of stochastic gradient descent

Takeaway: Lab 4 vocabulary.

Source: Book 3 (AI from zero), Chapter 16, p. 62

NN A68. [BLANK · Medium · calculation]

x_t = 2, w_x = 0.5, h_(t−1) = −1, w_h = 0.5, b = 0, tanh. h_t = ___ (3 d.p.).

Answer: 0.462

Takeaway: tanh(1 − 0.5).

Source: Book 3 (AI from zero), Practice 17.1, p. 70

NN A69. [MCQ · Easy]

Which LSTM gate erases memory?

A. forget gate
B. output gate
C. candidate
D. input gate

Answer: A. forget gate

Option notes:

- B: Exposes the cell.
- C: Proposes new content.
- D: Writes new info.

Takeaway: f near 0 wipes.

Source: Book 3 (AI from zero), Practice 17.2, p. 70

NN A70. [BLANK · Medium · calculation]

c_(t−1) = 2, f = 0.5, i = 1, candidate −0.4. c_t = ___.

Answer: 0.6

Takeaway: 2(0.5) + 1(−0.4).

Source: Book 3 (AI from zero), Practice 17.3, p. 70

NN A71. [MCQ · Easy]

|w_zz| > 1 in a simple RNN risks:

A. exploding gradients
B. no effect
C. overfitting only
D. vanishing gradients

Answer: A. exploding gradients

Option notes:

- B: It matters.
- C: Not the issue.
- D: That is < 1.

Takeaway: Repeated multiplication.

Source: Book 3 (AI from zero), Practice 17.4, p. 70

NN A72. [BLANKS · Hard · calculation]

LSTM step: c_(t−1) = 0.8, f = 0.9, i = 0.5, o = 0.7, candidate 0.6. c_t = ___; z_t = tanh(c_t) × o = ___ (3 d.p.).

(1) c_t = ____
(2) z_t = ____

Answer: c_t = 1.02; z_t = 0.539

Takeaway: 0.72 + 0.30; tanh(1.02) = 0.770.

Source: Book 3 (AI from zero), Chapter 17, p. 68

NN A73. [MCQ · Medium]

(Extra, not in your slides) A GRU differs from an LSTM by:

A. merging forget and input gates into one update gate, with no separate cell state
B. using ReLU only
C. having no gates
D. needing labels per step

Answer: A. merging forget and input gates into one update gate, with no separate cell state

Option notes:

- B: No.
- C: It has gates.
- D: No.

Takeaway: Fewer parameters, similar performance.

Source: Book 3 (AI from zero), Chapter 17, p. 68

NN A74. [TF · Easy]

A single-layer perceptron can represent XOR.

Answer: False

Why: XOR is not linearly separable; a hidden layer is needed.

Takeaway: Minsky & Papert's famous point.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: perceptrons

NN A75. [BLANKS · Hard · calculation]

XOR with step units (1 if z ≥ 0): hidden h_1 = OR ($w_0 = -0.5$, $w_1 = w_2 = 1$), h_2 = AND ($w_0 = -1.5$, $w_1 = w_2 = 1$), output = h_1 AND NOT h_2 ($w_0 = -0.5$, w_h1 = 1, w_h2 = −1). Outputs for inputs (0,0), (0,1), (1,1):

(1) (0,0) = ____
(2) (0,1) = ____
(3) (1,1) = ____

Answer: (0,0) = 0; (0,1) = 1; (1,1) = 0

Takeaway: Two lines carve out the XOR region.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: networks

NN A76. [MCQ · Medium]

Why does ReLU help deep networks train compared with sigmoid?

A. it outputs probabilities
B. it is bounded above
C. it has zero derivative everywhere
D. its derivative is 1 for positive inputs, so gradients do not shrink layer after layer

Answer: D. its derivative is 1 for positive inputs, so gradients do not shrink layer after layer

Option notes:

- A: Sigmoid does.
- B: ReLU is unbounded.
- C: Only for negative inputs.

Takeaway: Vanishing gradients.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: activation functions

NN A77. [MCQ · Medium]

AIMA's residual networks compute $z^{(i)} = g(z^{(i-1)} + f(z^{(i-1)}))$. The point is that:

A. they use fewer parameters than any CNN
B. they remove all non-linearity
C. they replace convolution
D. a layer can default to passing its input through, so very deep networks stay trainable

Answer: D. a layer can default to passing its input through, so very deep networks stay trainable

Option notes:

- A: Not the point.
- B: g is still applied.
- C: They are built from convolutions.

Takeaway: ResNet skip connections.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: residual networks

NN A78. [MULTI · Medium]

Ways AIMA lists to improve generalisation of deep networks: (Select ALL that apply.)

A. weight decay (L2 regularisation)
B. training until the training loss is exactly zero
C. choosing a suitable architecture / network size
D. dropout

Answer: A. weight decay (L2 regularisation); C. choosing a suitable architecture / network size; D. dropout

Option notes:

- A: Correct:
- B: Incorrect: that overfits
- C: Correct:
- D: Correct: randomly switch off units during training

Takeaway: Generalisation tools.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: generalisation

NN A79. [BLANK · Medium · calculation]

A fully connected layer with 100 inputs and 50 units has ___ parameters including biases.

Answer: 5050
(Also accepted: 5,050)

Takeaway: (100 + 1) × 50.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: parameters

Part B: calculation drills (112)

NN B1. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 1$ (input $x_0 = 1$), $w_1 = 1$, $w_2 = 0$ and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (0, 1) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 1; output = 1

Working:

- ws = 1×1 + 1×0 + 0×1 = 1.
- 1 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B2. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 2$ (input $x_0 = 1$), $w_1 = -1$, $w_2 = 1.5$ and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (3, 3) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 3.5; output = 1

Working:

- ws = 1×2 + -1×3 + 1.5×3 = 3.5.
- 3.5 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B3. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 0$ (input $x_0 = 1$), $w_1 = -1$, $w_2 = -0.5$ and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (0, -2) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 1; output = 1

Working:

- ws = 1×0 + -1×0 + -0.5×-2 = 1.
- 1 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B4. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = -1$ (input $x_0 = 1$), $w_1 = 1.5$, $w_2 = 2$ and a hard threshold $g(ws) = 1$ if $ws \geq 0$ else 0. For x = (2, -2) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = -2; output = 0

Working:

- ws = 1×-1 + 1.5×2 + 2×-2 = -2.
- -2 < 0 → output 0.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with $g(ws \geq 0) = 1$.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B5. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = -1$ (input $x_0 = 1$), $w_1 = -1.5$, $w_2 = -2$ and a hard threshold $g(ws) = 1$ if $ws \geq 0$ else 0. For x = (1, -1) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = -0.5; output = 0

Working:

- ws = 1×-1 + -1.5×1 + -2×-1 = -0.5.
- -0.5 < 0 → output 0.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with $g(ws \geq 0) = 1$.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B6. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 0.5$ (input $x_0 = 1$), $w_1 = 0.5$, $w_2 = 2$ and a hard threshold $g(ws) = 1$ if $ws \geq 0$ else 0. For x = (0, 3) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 6.5; output = 1

Working:

- ws = 1×0.5 + 0.5×0 + 2×3 = 6.5.
- 6.5 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with $g(ws \geq 0) = 1$.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B7. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 1$ (input $x_0 = 1$), $w_1 = 1$, $w_2 = 0.5$ and a hard threshold $g(ws) = 1$ if $ws \geq 0$ else 0. For x = (2, -2) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 2; output = 1

Working:

- ws = 1×1 + 1×2 + 0.5×-2 = 2.
- 2 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B8. [BLANKS · Easy · calculation]

A perceptron has bias weight w_0 = -1.5 (input x_0 = 1), w_1 = 2, w_2 = 1.5 and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (0, 1) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 0; output = 1

Working:

- ws = 1×-1.5 + 2×0 + 1.5×1 = 0.
- 0 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B9. [BLANKS · Easy · calculation]

A perceptron has bias weight w_0 = -2 (input x_0 = 1), w_1 = 1.5, w_2 = 1.5 and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (2, 0) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 1; output = 1

Working:

- ws = 1×-2 + 1.5×2 + 1.5×0 = 1.
- 1 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B10. [BLANKS · Easy · calculation]

A perceptron has bias weight w_0 = 0.5 (input x_0 = 1), w_1 = 1, w_2 = 1 and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (1, -2) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = -0.5; output = 0

Working:

- ws = 1×0.5 + 1×1 + 1×-2 = -0.5.
- -0.5 < 0 → output 0.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B11. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 2$ (input $x_0 = 1$), $w_1 = 2$, $w_2 = -0.5$ and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (0, 2) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 1; output = 1

Working:

- ws = 1×2 + 2×0 + -0.5×2 = 1.
- 1 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B12. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 1.5$ (input $x_0 = 1$), $w_1 = -2$, $w_2 = 1.5$ and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (2, -1) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = -4; output = 0

Working:

- ws = 1×1.5 + -2×2 + 1.5×-1 = -4.
- -4 < 0 → output 0.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B13. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = 1$ (input $x_0 = 1$), $w_1 = -1.5$, $w_2 = 0$ and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (1, 0) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = -0.5; output = 0

Working:

- ws = 1×1 + -1.5×1 + 0×0 = -0.5.
- -0.5 < 0 → output 0.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B14. [BLANKS · Easy · calculation]

A perceptron has bias weight $w_0 = -1.5$ (input $x_0 = 1$), $w_1 = 0.5$, $w_2 = 0$ and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (3, 2) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 0; output = 1

Working:

- ws = 1×-1.5 + 0.5×3 + 0×2 = 0.
- 0 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B15. [BLANKS · Easy · calculation]

A perceptron has bias weight w_0 = -2 (input x_0 = 1), w_1 = 0.5, w_2 = 1.5 and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (1, -1) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = -3; output = 0

Working:

- ws = 1×-2 + 0.5×1 + 1.5×-1 = -3.
- -3 < 0 → output 0.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B16. [BLANKS · Easy · calculation]

A perceptron has bias weight w_0 = -0.5 (input x_0 = 1), w_1 = 0.5, w_2 = 0 and a hard threshold g(ws) = 1 if ws ≥ 0 else 0. For x = (2, 0) give the weighted sum ws and the output.

(1) ws = ____
(2) output = ____

Answer: ws = 0.5; output = 1

Working:

- ws = 1×-0.5 + 0.5×2 + 0×0 = 0.5.
- 0.5 ≥ 0 → output 1.

Takeaway: The bias w_0 shifts the threshold. ws = 0 gives output 1 with g(ws ≥ 0) = 1.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B17. [BLANKS · Medium · calculation]

Perceptron with w_0 = 1, w_1 = 0, w_2 = 0.5, hard threshold (1 if ws ≥ 0). Example x = (3, 2) has true label y = 0; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ (x_0 = 1). Give new w_0, w_1, w_2.

(1) w_0 = ____
(2) w_1 = ____
(3) w_2 = ____

Answer: w_0 = 0.5; w_1 = -1.5; w_2 = -0.5

Working:

- ws = 2 → h = 1. Error y − h = -1.
- $w_0 \leftarrow$ 1 + 0.5×-1×1 = 0.5.
- $w_1 \leftarrow$ 0 + 0.5×-1×3 = -1.5.
- $w_2 \leftarrow$ 0.5 + 0.5×-1×2 = -0.5.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B18. [BLANKS · Medium · calculation]

Perceptron with $w_0 = -2$, $w_1 = -2$, $w_2 = -0.5$, hard threshold (1 if ws ≥ 0). Example x = (3, -2) has true label y = 1; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = -1.5$; $w_1 = -0.5$; $w_2 = -1.5$

Working:

- ws = -7 → h = 0. Error y − h = 1.
- $w_0 \leftarrow -2 + 0.5 \times 1 \times 1 = -1.5$.
- $w_1 \leftarrow -2 + 0.5 \times 1 \times 3 = -0.5$.
- $w_2 \leftarrow -0.5 + 0.5 \times 1 \times -2 = -1.5$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B19. [BLANKS · Medium · calculation]

Perceptron with $w_0 = 1.5$, $w_1 = -1.5$, $w_2 = -1$, hard threshold (1 if ws ≥ 0). Example x = (0, 2) has true label y = 1; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = 2$; $w_1 = -1.5$; $w_2 = 0$

Working:

- ws = -0.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow 1.5 + 0.5 \times 1 \times 1 = 2$.
- $w_1 \leftarrow -1.5 + 0.5 \times 1 \times 0 = -1.5$.
- $w_2 \leftarrow -1 + 0.5 \times 1 \times 2 = 0$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B20. [BLANKS · Medium · calculation]

Perceptron with $w_0 = 1$, $w_1 = -1$, $w_2 = -1$, hard threshold (1 if ws ≥ 0). Example x = (2, 1) has true label y = 1; α = 1. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = 2$; $w_1 = 1$; $w_2 = 0$

Working:

- ws = -2 → h = 0. Error y − h = 1.
- $w_0 \leftarrow 1 + 1 \times 1 \times 1 = 2$.
- $w_1 \leftarrow -1 + 1 \times 1 \times 2 = 1$.
- $w_2 \leftarrow -1 + 1 \times 1 \times 1 = 0$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B21. [BLANKS · Medium · calculation]

Perceptron with $w_0 = -2$, $w_1 = 0$, $w_2 = 1$, hard threshold (1 if ws ≥ 0). Example x = (2, -2) has true label y = 1; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = -1.5$; $w_1 = 1$; $w_2 = 0$

Working:

- ws = -4 → h = 0. Error y − h = 1.
- $w_0 \leftarrow -2 + 0.5 \times 1 \times 1 = -1.5$.
- $w_1 \leftarrow 0 + 0.5 \times 1 \times 2 = 1$.
- $w_2 \leftarrow 1 + 0.5 \times 1 \times -2 = 0$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B22. [BLANKS · Medium · calculation]

Perceptron with $w_0 = 1$, $w_1 = -1.5$, $w_2 = 0$, hard threshold (1 if ws ≥ 0). Example x = (3, 0) has true label y = 1; α = 0.1. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = 1.1$; $w_1 = -1.2$; $w_2 = 0$

Working:

- ws = -3.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow 1 + 0.1 \times 1 \times 1 = 1.1$.
- $w_1 \leftarrow -1.5 + 0.1 \times 1 \times 3 = -1.2$.
- $w_2 \leftarrow 0 + 0.1 \times 1 \times 0 = 0$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B23. [BLANKS · Medium · calculation]

Perceptron with $w_0 = -1.5$, $w_1 = 0.5$, $w_2 = 1.5$, hard threshold (1 if ws ≥ 0). Example x = (3, 2) has true label y = 0; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = -2$; $w_1 = -1$; $w_2 = 0.5$

Working:

- ws = 3 → h = 1. Error y − h = -1.
- $w_0 \leftarrow -1.5 + 0.5 \times -1 \times 1 = -2$.
- $w_1 \leftarrow 0.5 + 0.5 \times -1 \times 3 = -1$.

- $w_2 \leftarrow 1.5 + 0.5\times\text{-}1\times2 = 0.5$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B24. [BLANKS · Medium · calculation]

Perceptron with $w_0 = -1.5$, $w_1 = -1.5$, $w_2 = 0$, hard threshold (1 if ws ≥ 0). Example x = (0, 2) has true label y = 1; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = -1$; $w_1 = -1.5$; $w_2 = 1$

Working:

- ws = -1.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow -1.5 + 0.5\times1\times1 = -1$.
- $w_1 \leftarrow -1.5 + 0.5\times1\times0 = -1.5$.
- $w_2 \leftarrow 0 + 0.5\times1\times2 = 1$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B25. [BLANKS · Medium · calculation]

Perceptron with $w_0 = -0.5$, $w_1 = 0.5$, $w_2 = 1$, hard threshold (1 if ws ≥ 0). Example x = (2, -2) has true label y = 1; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = 0$; $w_1 = 1.5$; $w_2 = 0$

Working:

- ws = -1.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow -0.5 + 0.5\times1\times1 = 0$.
- $w_1 \leftarrow 0.5 + 0.5\times1\times2 = 1.5$.
- $w_2 \leftarrow 1 + 0.5\times1\times\text{-}2 = 0$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B26. [BLANKS · Medium · calculation]

Perceptron with $w_0 = -1.5$, $w_1 = 2$, $w_2 = 0.5$, hard threshold (1 if ws ≥ 0). Example x = (2, 2) has true label y = 0; α = 1. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = -2.5$; $w_1 = 0$; $w_2 = -1.5$

Working:

- ws = 3.5 → h = 1. Error y − h = -1.

- $w_0 \leftarrow -1.5 + 1\times-1\times1 = -2.5$.
- $w_1 \leftarrow 2 + 1\times-1\times2 = 0$.
- $w_2 \leftarrow 0.5 + 1\times-1\times2 = -1.5$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B27. [BLANKS · Medium · calculation]

Perceptron with $w_0 = 1.5$, $w_1 = -1$, $w_2 = -0.5$, hard threshold (1 if ws ≥ 0). Example x = (1, 2) has true label y = 1; α = 0.1. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = 1.6$; $w_1 = -0.9$; $w_2 = -0.3$

Working:

- ws = -0.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow 1.5 + 0.1\times1\times1 = 1.6$.
- $w_1 \leftarrow -1 + 0.1\times1\times1 = -0.9$.
- $w_2 \leftarrow -0.5 + 0.1\times1\times2 = -0.3$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B28. [BLANKS · Medium · calculation]

Perceptron with $w_0 = 0.5$, $w_1 = 0.5$, $w_2 = 2$, hard threshold (1 if ws ≥ 0). Example x = (2, -2) has true label y = 1; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = 1$; $w_1 = 1.5$; $w_2 = 1$

Working:

- ws = -2.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow 0.5 + 0.5\times1\times1 = 1$.
- $w_1 \leftarrow 0.5 + 0.5\times1\times2 = 1.5$.
- $w_2 \leftarrow 2 + 0.5\times1\times-2 = 1$.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B29. [BLANKS · Medium · calculation]

Perceptron with $w_0 = -1$, $w_1 = -1$, $w_2 = 0.5$, hard threshold (1 if ws ≥ 0). Example x = (3, 3) has true label y = 1; α = 0.1. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ ($x_0 = 1$). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: $w_0 = -0.9$; $w_1 = -0.7$; $w_2 = 0.8$

Working:

- ws = -2.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow$ -1 + 0.1×1×1 = -0.9.
- $w_1 \leftarrow$ -1 + 0.1×1×3 = -0.7.
- $w_2 \leftarrow$ 0.5 + 0.1×1×3 = 0.8.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B30. [BLANKS · Medium · calculation]

Perceptron with w_0 = -1.5, w_1 = -0.5, w_2 = 0.5, hard threshold (1 if ws ≥ 0). Example x = (0, 1) has true label y = 1; α = 0.5. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ (x_0 = 1). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: w_0 = -1; w_1 = -0.5; w_2 = 1

Working:

- ws = -1 → h = 0. Error y − h = 1.
- $w_0 \leftarrow$ -1.5 + 0.5×1×1 = -1.
- $w_1 \leftarrow$ -0.5 + 0.5×1×0 = -0.5.
- $w_2 \leftarrow$ 0.5 + 0.5×1×1 = 1.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B31. [BLANKS · Medium · calculation]

Perceptron with w_0 = 0, w_1 = -2, w_2 = 1, hard threshold (1 if ws ≥ 0). Example x = (1, 2) has true label y = 0; α = 1. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ (x_0 = 1). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: w_0 = -1; w_1 = -3; w_2 = -1

Working:

- ws = 0 → h = 1. Error y − h = -1.
- $w_0 \leftarrow$ 0 + 1×-1×1 = -1.
- $w_1 \leftarrow$ -2 + 1×-1×1 = -3.
- $w_2 \leftarrow$ 1 + 1×-1×2 = -1.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B32. [BLANKS · Medium · calculation]

Perceptron with w_0 = -1, w_1 = -1.5, w_2 = 0.5, hard threshold (1 if ws ≥ 0). Example x = (2, -1) has true label y = 1; α = 1. Apply $w_i \leftarrow w_i + \alpha(y - h)x_i$ (x_0 = 1). Give new w_0, w_1, w_2.

(1) w_0 = _____
(2) w_1 = _____
(3) w_2 = _____

Answer: w_0 = 0; w_1 = 0.5; w_2 = -0.5

Working:

- ws = -4.5 → h = 0. Error y − h = 1.
- $w_0 \leftarrow$ -1 + 1×1×1 = 0.
- $w_1 \leftarrow$ -1.5 + 1×1×2 = 0.5.
- $w_2 \leftarrow$ 0.5 + 1×1×-1 = -0.5.

Takeaway: If the perceptron is already right (y = h), the weights do not change.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B33. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = 2$, $w_1 = 0$, $w_2 = 1.5$. For x = (0, 1), y = 1, α = 1, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = _____
(2) new w_1 = _____

Answer: h = 0.9707; new w_1 = 0.0000

Working:

- in = 3.5; h = 1/(1 + e^(−3.5)) = 0.9707.
- (y − h)·h(1 − h) = (1 − 0.9707)×0.9707×0.0293 = 0.00083.
- $w_1 \leftarrow$ 0 + 1×0.00083×0 = 0.0000.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B34. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = -0.5$, $w_1 = 1.5$, $w_2 = -2$. For x = (2, 3), y = 0, α = 0.1, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = _____
(2) new w_1 = _____

Answer: h = 0.0293; new w_1 = 1.4998

Working:

- in = -3.5; h = 1/(1 + e^(−-3.5)) = 0.0293.
- (y − h)·h(1 − h) = (0 − 0.0293)×0.0293×0.9707 = -0.00083.
- $w_1 \leftarrow$ 1.5 + 0.1×-0.00083×2 = 1.4998.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B35. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = 1$, $w_1 = -0.5$, $w_2 = 0$. For x = (2, 0), y = 0, α = 0.5, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = _____
(2) new w_1 = _____

Answer: h = 0.5000; new w_1 = -0.6250

Working:

- in = 0; h = 1/(1 + e^(−0)) = 0.5000.
- (y − h)·h(1 − h) = (0 − 0.5000)×0.5000×0.5000 = -0.12500.

- $w_1 \leftarrow$ -0.5 + 0.5×-0.12500×2 = -0.6250.

Takeaway: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B36. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = 1.5$, $w_1 = 0$, $w_2 = -0.5$. For $x = (0, 1)$, $y = 0$, $\alpha = 0.1$, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = _____
(2) new w_1 = _____

Answer: h = 0.7311; new w_1 = 0.0000

Working:

- in = 1; h = 1/(1 + e^(−1)) = 0.7311.
- $(y - h)\cdot h(1 - h)$ = (0 − 0.7311)×0.7311×0.2689 = -0.14373.
- $w_1 \leftarrow$ 0 + 0.1×-0.14373×0 = 0.0000.

Takeaway: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B37. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = -0.5$, $w_1 = 1.5$, $w_2 = 1$. For $x = (3, 2)$, $y = 1$, $\alpha = 0.1$, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = _____
(2) new w_1 = _____

Answer: h = 0.9975; new w_1 = 1.5000

Working:

- in = 6; h = 1/(1 + e^(−6)) = 0.9975.
- $(y - h)\cdot h(1 - h)$ = (1 − 0.9975)×0.9975×0.0025 = 0.00001.
- $w_1 \leftarrow$ 1.5 + 0.1×0.00001×3 = 1.5000.

Takeaway: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B38. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = 0.5$, $w_1 = -2$, $w_2 = 0.5$. For $x = (1, -2)$, $y = 1$, $\alpha = 0.5$, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = _____
(2) new w_1 = _____

Answer: h = 0.0759; new w_1 = -1.9676

Working:

- in = -2.5; h = 1/(1 + e^(−-2.5)) = 0.0759.
- $(y - h)\cdot h(1 - h)$ = (1 − 0.0759)×0.0759×0.9241 = 0.06479.
- $w_1 \leftarrow$ -2 + 0.5×0.06479×1 = -1.9676.

Takeaway: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B39. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = -1$, $w_1 = 1$, $w_2 = 0$. For $x = (3, 3)$, $y = 1$, $\alpha = 0.1$, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.8808; new w_1 = 1.0038

Working:

- in = 2; h = 1/(1 + e^(−2)) = 0.8808.
- $(y - h)\cdot h(1 - h) = (1 - 0.8808)\times 0.8808\times 0.1192 = 0.01252$.
- $w_1 \leftarrow 1 + 0.1\times 0.01252\times 3 = 1.0038$.

Takeaway: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B40. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = -1.5$, $w_1 = 0$, $w_2 = -1.5$. For $x = (3, -2)$, $y = 1$, $\alpha = 0.5$, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.8176; new w_1 = 0.0408

Working:

- in = 1.5; h = 1/(1 + e^(−1.5)) = 0.8176.
- $(y - h)\cdot h(1 - h) = (1 - 0.8176)\times 0.8176\times 0.1824 = 0.02721$.
- $w_1 \leftarrow 0 + 0.5\times 0.02721\times 3 = 0.0408$.

Takeaway: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B41. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = -2$, $w_1 = -2$, $w_2 = 0.5$. For $x = (0, 1)$, $y = 0$, $\alpha = 1$, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.1824; new w_1 = -2.0000

Working:

- in = -1.5; h = 1/(1 + e^(−-1.5)) = 0.1824.
- $(y - h)\cdot h(1 - h) = (0 - 0.1824)\times 0.1824\times 0.8176 = -0.02721$.
- $w_1 \leftarrow -2 + 1\times -0.02721\times 0 = -2.0000$.

Takeaway: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B42. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = -1$, $w_1 = 0$, $w_2 = 1$. For $x = (3, 1)$, $y = 0$, $\alpha = 1$, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.5000; new w_1 = -0.3750

Working:

- in = 0; h = 1/(1 + e^(−0)) = 0.5000.
- (y − h)·h(1 − h) = (0 − 0.5000)×0.5000×0.5000 = -0.12500.
- $w_1 \leftarrow$ 0 + 1×-0.12500×3 = -0.3750.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B43. [BLANKS · Hard · calculation]

A sigmoid unit h = σ($w_0 + w_1x_1 + w_2x_2$) has w_0 = -2, w_1 = 0, w_2 = -1. For x = (2, 0), y = 0, α = 1, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.1192; new w_1 = -0.0250

Working:

- in = -2; h = 1/(1 + e^(−-2)) = 0.1192.
- (y − h)·h(1 − h) = (0 − 0.1192)×0.1192×0.8808 = -0.01252.
- $w_1 \leftarrow$ 0 + 1×-0.01252×2 = -0.0250.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B44. [BLANKS · Hard · calculation]

A sigmoid unit h = σ($w_0 + w_1x_1 + w_2x_2$) has w_0 = 1.5, w_1 = -1, w_2 = 1. For x = (0, 1), y = 1, α = 0.1, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.9241; new w_1 = -1.0000

Working:

- in = 2.5; h = 1/(1 + e^(−2.5)) = 0.9241.
- (y − h)·h(1 − h) = (1 − 0.9241)×0.9241×0.0759 = 0.00532.
- $w_1 \leftarrow$ -1 + 0.1×0.00532×0 = -1.0000.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B45. [BLANKS · Hard · calculation]

A sigmoid unit h = σ($w_0 + w_1x_1 + w_2x_2$) has w_0 = -1, w_1 = 1.5, w_2 = 1. For x = (1, 1), y = 1, α = 1, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.8176; new w_1 = 1.5272

Working:

- in = 1.5; h = 1/(1 + e^(−1.5)) = 0.8176.
- (y − h)·h(1 − h) = (1 − 0.8176)×0.8176×0.1824 = 0.02721.
- $w_1 \leftarrow 1.5 + 1 \times 0.02721 \times 1 = 1.5272$.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B46. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = -0.5$, $w_1 = -1.5$, $w_2 = 0$. For x = (0, 1), y = 0, α = 0.5, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.3775; new w_1 = -1.5000

Working:

- in = -0.5; h = 1/(1 + e^(−-0.5)) = 0.3775.
- (y − h)·h(1 − h) = (0 − 0.3775)×0.3775×0.6225 = -0.08872.
- $w_1 \leftarrow -1.5 + 0.5 \times -0.08872 \times 0 = -1.5000$.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B47. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = 0.5$, $w_1 = 1.5$, $w_2 = 0$. For x = (1, 1), y = 1, α = 0.5, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.8808; new w_1 = 1.5063

Working:

- in = 2; h = 1/(1 + e^(−2)) = 0.8808.
- (y − h)·h(1 − h) = (1 − 0.8808)×0.8808×0.1192 = 0.01252.
- $w_1 \leftarrow 1.5 + 0.5 \times 0.01252 \times 1 = 1.5063$.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B48. [BLANKS · Hard · calculation]

A sigmoid unit $h = \sigma(w_0 + w_1x_1 + w_2x_2)$ has $w_0 = 0.5$, $w_1 = -2$, $w_2 = -1.5$. For x = (3, -2), y = 0, α = 0.1, use the Workshop 5 rule $w_i \leftarrow w_i + \alpha(y - h)\cdot h(1 - h)\cdot x_i$. Give h (4 dp) and new w_1 (4 dp).

(1) h = ____
(2) new w_1 = ____

Answer: h = 0.0759; new w_1 = -2.0016

Working:

- in = -2.5; h = 1/(1 + e^(−-2.5)) = 0.0759.
- (y − h)·h(1 − h) = (0 − 0.0759)×0.0759×0.9241 = -0.00532.
- $w_1 \leftarrow -2 + 0.1 \times -0.00532 \times 3 = -2.0016$.

Takeaway: σ′(z) = σ(z)(1 − σ(z)), at most 0.25: one reason gradients vanish in deep sigmoid nets.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B49. [BLANKS · Easy · calculation]

For input z = -0.5, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.05; Step = 0; Sigmoid = 0.378

Working:

- ReLU = max(0, -0.5) = 0.
- Leaky = max(0.1×-0.5, -0.5) = -0.05.
- Step = 0.
- σ(-0.5) = 1/(1 + e^(0.5)) = 0.3775.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B50. [BLANKS · Easy · calculation]

For input z = -3, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.3; Step = 0; Sigmoid = 0.047

Working:

- ReLU = max(0, -3) = 0.
- Leaky = max(0.1×-3, -3) = -0.3.
- Step = 0.
- σ(-3) = 1/(1 + e^(3)) = 0.0474.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B51. [BLANKS · Easy · calculation]

For input z = 0.5, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0.5; Leaky ReLU = 0.5; Step = 1; Sigmoid = 0.622

Working:

- ReLU = max(0, 0.5) = 0.5.

- Leaky = max(0.1×0.5, 0.5) = 0.5.
- Step = 1.
- σ(0.5) = 1/(1 + e^(-0.5)) = 0.6225.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B52. [BLANKS · Easy · calculation]

For input z = -2, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.2; Step = 0; Sigmoid = 0.119

Working:

- ReLU = max(0, -2) = 0.
- Leaky = max(0.1×-2, -2) = -0.2.
- Step = 0.
- σ(-2) = 1/(1 + e^(2)) = 0.1192.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B53. [BLANKS · Easy · calculation]

For input z = 3, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 3; Leaky ReLU = 3; Step = 1; Sigmoid = 0.953

Working:

- ReLU = max(0, 3) = 3.
- Leaky = max(0.1×3, 3) = 3.
- Step = 1.
- σ(3) = 1/(1 + e^(-3)) = 0.9526.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B54. [BLANKS · Easy · calculation]

For input z = -1.5, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____

(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.15; Step = 0; Sigmoid = 0.182

Working:

- ReLU = max(0, -1.5) = 0.
- Leaky = max(0.1×-1.5, -1.5) = -0.15.
- Step = 0.
- σ(-1.5) = 1/(1 + e^(1.5)) = 0.1824.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B55. [BLANKS · Easy · calculation]

For input z = -3, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.3; Step = 0; Sigmoid = 0.047

Working:

- ReLU = max(0, -3) = 0.
- Leaky = max(0.1×-3, -3) = -0.3.
- Step = 0.
- σ(-3) = 1/(1 + e^(3)) = 0.0474.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B56. [BLANKS · Easy · calculation]

For input z = 3, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 3; Leaky ReLU = 3; Step = 1; Sigmoid = 0.953

Working:

- ReLU = max(0, 3) = 3.
- Leaky = max(0.1×3, 3) = 3.
- Step = 1.
- σ(3) = 1/(1 + e^(-3)) = 0.9526.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B57. [BLANKS · Easy · calculation]

For input z = -2, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.2; Step = 0; Sigmoid = 0.119

Working:

- ReLU = max(0, -2) = 0.
- Leaky = max(0.1×-2, -2) = -0.2.
- Step = 0.
- σ(-2) = 1/(1 + e^(2)) = 0.1192.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B58. [BLANKS · Easy · calculation]

For input z = 0.5, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0.5; Leaky ReLU = 0.5; Step = 1; Sigmoid = 0.622

Working:

- ReLU = max(0, 0.5) = 0.5.
- Leaky = max(0.1×0.5, 0.5) = 0.5.
- Step = 1.
- σ(0.5) = 1/(1 + e^(-0.5)) = 0.6225.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B59. [BLANKS · Easy · calculation]

For input z = -1.5, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.15; Step = 0; Sigmoid = 0.182

Working:

- ReLU = max(0, -1.5) = 0.
- Leaky = max(0.1×-1.5, -1.5) = -0.15.
- Step = 0.

- σ(-1.5) = 1/(1 + e^(1.5)) = 0.1824.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B60. [BLANKS · Easy · calculation]

For input z = -2, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.2; Step = 0; Sigmoid = 0.119

Working:

- ReLU = max(0, -2) = 0.
- Leaky = max(0.1×-2, -2) = -0.2.
- Step = 0.
- σ(-2) = 1/(1 + e^(2)) = 0.1192.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B61. [BLANKS · Easy · calculation]

For input z = 0.5, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0.5; Leaky ReLU = 0.5; Step = 1; Sigmoid = 0.622

Working:

- ReLU = max(0, 0.5) = 0.5.
- Leaky = max(0.1×0.5, 0.5) = 0.5.
- Step = 1.
- σ(0.5) = 1/(1 + e^(-0.5)) = 0.6225.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B62. [BLANKS · Easy · calculation]

For input z = -3, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 0; Leaky ReLU = -0.3; Step = 0; Sigmoid = 0.047

Working:

- ReLU = max(0, -3) = 0.
- Leaky = max(0.1×-3, -3) = -0.3.
- Step = 0.
- σ(-3) = 1/(1 + e^(3)) = 0.0474.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B63. [BLANKS · Easy · calculation]

For input z = 3, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 3; Leaky ReLU = 3; Step = 1; Sigmoid = 0.953

Working:

- ReLU = max(0, 3) = 3.
- Leaky = max(0.1×3, 3) = 3.
- Step = 1.
- σ(3) = 1/(1 + e^(-3)) = 0.9526.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B64. [BLANKS · Easy · calculation]

For input z = 3, give ReLU(z), Leaky ReLU(z) with α = 0.1 (f = max(αz, z)), the step output (1 if z ≥ 0 else 0), and σ(z) (3 dp).

(1) ReLU = ____
(2) Leaky ReLU = ____
(3) Step = ____
(4) Sigmoid = ____

Answer: ReLU = 3; Leaky ReLU = 3; Step = 1; Sigmoid = 0.953

Working:

- ReLU = max(0, 3) = 3.
- Leaky = max(0.1×3, 3) = 3.
- Step = 1.
- σ(3) = 1/(1 + e^(-3)) = 0.9526.

Takeaway: ReLU kills negatives; Leaky ReLU keeps a small slope α for negatives.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B65. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.21$, $a_4 = 0.24$; $w_{3,5} = 0.5$, $w_{4,5} = 1$, $w_{0,5} = 1$; target $y = 1$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y}$ = 0.7933; Δ_5 = -0.0678; $\partial L/\partial w_{3,5}$ = -0.0142

Working:

- in_5 = 0.5×0.21 + 1×0.24 + 1 = 1.3450; $\hat{y} = \sigma(in_5)$ = 0.7933.
- $\Delta_5 = 2(0.7933 - 1)\times 0.7933\times(1 - 0.7933)$ = -0.0678.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = -0.0678×0.21 = -0.0142.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = -0.0056, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B66. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations a_3 = 0.4, a_4 = 0.5; $w_{3,5}$ = -1.5, $w_{4,5}$ = 2, $w_{0,5}$ = 0.5; target y = 0. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y}$ = 0.7109; Δ_5 = 0.2922; $\partial L/\partial w_{3,5}$ = 0.1169

Working:

- in_5 = -1.5×0.4 + 2×0.5 + 0.5 = 0.9000; $\hat{y} = \sigma(in_5)$ = 0.7109.
- $\Delta_5 = 2(0.7109 - 0)\times 0.7109\times(1 - 0.7109)$ = 0.2922.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = 0.2922×0.4 = 0.1169.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = -0.1052, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B67. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations a_3 = 0.64, a_4 = 0.61; $w_{3,5}$ = -1, $w_{4,5}$ = 0.5, $w_{0,5}$ = -1; target y = 0. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y}$ = 0.2083; Δ_5 = 0.0687; $\partial L/\partial w_{3,5}$ = 0.0440

Working:

- in_5 = -1×0.64 + 0.5×0.61 + -1 = -1.3350; $\hat{y} = \sigma(in_5)$ = 0.2083.
- $\Delta_5 = 2(0.2083 - 0)\times 0.2083\times(1 - 0.2083)$ = 0.0687.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = 0.0687×0.64 = 0.0440.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = -0.0158, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B68. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.73$, $a_4 = 0.23$; $w_{3,5} = 1.5$, $w_{4,5} = 1.5$, $w_{0,5} = 0$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.8085$; $\Delta_5 = 0.2504$; $\partial L/\partial w_{3,5} = 0.1828$

Working:

- $in_5 = 1.5\times0.73 + 1.5\times0.23 + 0 = 1.4400$; $\hat{y} = \sigma(in_5) = 0.8085$.
- $\Delta_5 = 2(0.8085 - 0)\times0.8085\times(1 - 0.8085) = 0.2504$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2504\times0.73 = 0.1828$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0740$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B69. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.59$, $a_4 = 0.43$; $w_{3,5} = 2$, $w_{4,5} = 1$, $w_{0,5} = 1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.9315$; $\Delta_5 = 0.1189$; $\partial L/\partial w_{3,5} = 0.0701$

Working:

- $in_5 = 2\times0.59 + 1\times0.43 + 1 = 2.6100$; $\hat{y} = \sigma(in_5) = 0.9315$.
- $\Delta_5 = 2(0.9315 - 0)\times0.9315\times(1 - 0.9315) = 0.1189$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.1189\times0.59 = 0.0701$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0575$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B70. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.51$, $a_4 = 0.25$; $w_{3,5} = 1.5$, $w_{4,5} = -0.5$, $w_{0,5} = -1$; target $y = 1$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.4110$; $\Delta_5 = -0.2852$; $\partial L/\partial w_{3,5} = -0.1454$

Working:

- $in_5 = 1.5\times0.51 + -0.5\times0.25 + -1 = -0.3600$; $\hat{y} = \sigma(in_5) = 0.4110$.
- $\Delta_5 = 2(0.4110 - 1)\times0.4110\times(1 - 0.4110) = -0.2852$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = -0.2852\times0.51 = -0.1454$.

- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = -0.1069$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B71. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.51$, $a_4 = 0.27$; $w_{3,5} = -0.5$, $w_{4,5} = -2$, $w_{0,5} = -0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.2150$; $\Delta_5 = 0.0726$; $\partial L/\partial w_{3,5} = 0.0370$

Working:

- $in_5 = -0.5 \times 0.51 + -2 \times 0.27 + -0.5 = -1.2950$; $\hat{y} = \sigma(in_5) = 0.2150$.
- $\Delta_5 = 2(0.2150 - 0) \times 0.2150 \times (1 - 0.2150) = 0.0726$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.0726 \times 0.51 = 0.0370$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = -0.0091$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B72. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.38$, $a_4 = 0.45$; $w_{3,5} = 0.5$, $w_{4,5} = 1$, $w_{0,5} = 0$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.6548$; $\Delta_5 = 0.2960$; $\partial L/\partial w_{3,5} = 0.1125$

Working:

- $in_5 = 0.5 \times 0.38 + 1 \times 0.45 + 0 = 0.6400$; $\hat{y} = \sigma(in_5) = 0.6548$.
- $\Delta_5 = 2(0.6548 - 0) \times 0.6548 \times (1 - 0.6548) = 0.2960$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2960 \times 0.38 = 0.1125$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0349$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B73. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.6$, $a_4 = 0.26$; $w_{3,5} = 2$, $w_{4,5} = -0.5$, $w_{0,5} = 0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y) \cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.8278$; $\Delta_5 = 0.2360$; $\partial L/\partial w_{3,5} = 0.1416$

Working:

- $in_5 = 2\times0.6 + -0.5\times0.26 + 0.5 = 1.5700$; $\hat{y} = \sigma(in_5) = 0.8278$.
- $\Delta_5 = 2(0.8278 - 0)\times0.8278\times(1 - 0.8278) = 0.2360$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2360\times0.6 = 0.1416$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.1133$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B74. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.53$, $a_4 = 0.53$; $w_{3,5} = -1$, $w_{4,5} = -1$, $w_{0,5} = -0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.1736$; $\Delta_5 = 0.0498$; $\partial L/\partial w_{3,5} = 0.0264$

Working:

- $in_5 = -1\times0.53 + -1\times0.53 + -0.5 = -1.5600$; $\hat{y} = \sigma(in_5) = 0.1736$.
- $\Delta_5 = 2(0.1736 - 0)\times0.1736\times(1 - 0.1736) = 0.0498$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.0498\times0.53 = 0.0264$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = -0.0124$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B75. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.53$, $a_4 = 0.35$; $w_{3,5} = 0.5$, $w_{4,5} = 2$, $w_{0,5} = 1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.8771$; $\Delta_5 = 0.1891$; $\partial L/\partial w_{3,5} = 0.1002$

Working:

- $in_5 = 0.5\times0.53 + 2\times0.35 + 1 = 1.9650$; $\hat{y} = \sigma(in_5) = 0.8771$.
- $\Delta_5 = 2(0.8771 - 0)\times0.8771\times(1 - 0.8771) = 0.1891$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.1891\times0.53 = 0.1002$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0236$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B76. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.63$, $a_4 = 0.38$; $w_{3,5} = 1$, $w_{4,5} = -1.5$, $w_{0,5} = 0.5$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y}$ = 0.6365; Δ_5 = 0.2945; $\partial L/\partial w_{3,5}$ = 0.1856

Working:

- in_5 = 1×0.63 + -1.5×0.38 + 0.5 = 0.5600; $\hat{y} = \sigma(in_5)$ = 0.6365.
- $\Delta_5 = 2(0.6365 - 0)\times 0.6365\times(1 - 0.6365)$ = 0.2945.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = 0.2945×0.63 = 0.1856.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = 0.0687, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B77. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations a_3 = 0.63, a_4 = 0.36; $w_{3,5}$ = 0.5, $w_{4,5}$ = 0.5, $w_{0,5}$ = -1; target y = 0. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y}$ = 0.3764; Δ_5 = 0.1767; $\partial L/\partial w_{3,5}$ = 0.1113

Working:

- in_5 = 0.5×0.63 + 0.5×0.36 + -1 = -0.5050; $\hat{y} = \sigma(in_5)$ = 0.3764.
- $\Delta_5 = 2(0.3764 - 0)\times 0.3764\times(1 - 0.3764)$ = 0.1767.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = 0.1767×0.63 = 0.1113.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = 0.0206, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B78. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations a_3 = 0.69, a_4 = 0.4; $w_{3,5}$ = -0.5, $w_{4,5}$ = 0.5, $w_{0,5}$ = 1; target y = 1. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y}$ = 0.7016; Δ_5 = -0.1249; $\partial L/\partial w_{3,5}$ = -0.0862

Working:

- in_5 = -0.5×0.69 + 0.5×0.4 + 1 = 0.8550; $\hat{y} = \sigma(in_5)$ = 0.7016.
- $\Delta_5 = 2(0.7016 - 1)\times 0.7016\times(1 - 0.7016)$ = -0.1249.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ = -0.1249×0.69 = -0.0862.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3)$ = 0.0134, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B79. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.62$, $a_4 = 0.71$; $w_{3,5} = -2$, $w_{4,5} = 1.5$, $w_{0,5} = -1$; target $y = 1$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.2360$; $\Delta_5 = -0.2755$; $\partial L/\partial w_{3,5} = -0.1708$

Working:

- $in_5 = -2\times0.62 + 1.5\times0.71 + -1 = -1.1750$; $\hat{y} = \sigma(in_5) = 0.2360$.
- $\Delta_5 = 2(0.2360 - 1)\times0.2360\times(1 - 0.2360) = -0.2755$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = -0.2755\times0.62 = -0.1708$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.1298$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B80. [BLANKS · Exam+ · calculation]

Network of Workshop 5 (inputs 1,2 → sigmoid hidden 3,4 → sigmoid output 5), loss $(y - \hat{y})^2$. Hidden activations $a_3 = 0.78$, $a_4 = 0.71$; $w_{3,5} = 1$, $w_{4,5} = -0.5$, $w_{0,5} = 1$; target $y = 0$. Using $\Delta_5 = 2(\hat{y} - y)\cdot g'(in_5)$ and $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot g'(in_3)$ with $g' = a(1 - a)$, give $\hat{y}$, Δ_5 and $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3$ (4 dp each).

(1) $\hat{y}$ = _____
(2) Δ_5 = _____
(3) $\partial L/\partial w_{3,5}$ = _____

Answer: $\hat{y} = 0.8061$; $\Delta_5 = 0.2520$; $\partial L/\partial w_{3,5} = 0.1965$

Working:

- $in_5 = 1\times0.78 + -0.5\times0.71 + 1 = 1.4250$; $\hat{y} = \sigma(in_5) = 0.8061$.
- $\Delta_5 = 2(0.8061 - 0)\times0.8061\times(1 - 0.8061) = 0.2520$.
- $\partial L/\partial w_{3,5} = \Delta_5 \cdot a_3 = 0.2520\times0.78 = 0.1965$.
- (Further back: $\Delta_3 = \Delta_5 \cdot w_{3,5} \cdot a_3(1-a_3) = 0.0432$, and $\partial L/\partial w_{1,3} = \Delta_3 \cdot x_1$.)

Takeaway: Gradient for weight $w_{i,j}$ = (activation coming in, a_i) × (perceived error at the receiving unit, Δ_j).

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B81. [MCQ · Medium · calculation]

A fully connected network has 10 inputs, one hidden layer of 16 units and 10 output unit(s); every unit has a bias. How many trainable parameters?

A. 320
B. 346
C. 176
D. 1600

Answer: B. 346

Working:

- Hidden layer: (10 + 1)×16 = 176.
- Output layer: (16 + 1)×10 = 170.
- Total = 346.

Option notes:

- A: Forgot the bias of each neuron (+1 per unit).
- C: Counted only the first layer.
- D: Weights are added layer by layer, not multiplied.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B82. [MCQ · Medium · calculation]

A fully connected network has 64 inputs, one hidden layer of 32 units and 2 output unit(s); every unit has a bias. How many trainable parameters?

A. 2080
B. 4096
C. 2112
D. 2146

Answer: D. 2146

Working:

- Hidden layer: (64 + 1)×32 = 2080.
- Output layer: (32 + 1)×2 = 66.
- Total = 2146.

Option notes:

- A: Counted only the first layer.
- B: Weights are added layer by layer, not multiplied.
- C: Forgot the bias of each neuron (+1 per unit).

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B83. [MCQ · Medium · calculation]

A fully connected network has 3 inputs, one hidden layer of 32 units and 1 output unit(s); every unit has a bias. How many trainable parameters?

A. 96
B. 128
C. 185
D. 161

Answer: D. 161

Working:

- Hidden layer: (3 + 1)×32 = 128.
- Output layer: (32 + 1)×1 = 33.
- Total = 161.

Option notes:

- A: Weights are added layer by layer, not multiplied.
- B: Forgot the bias of each neuron (+1 per unit).
- C: Arithmetic slip: recompute each step carefully.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B84. [MCQ · Medium · calculation]

A fully connected network has 10 inputs, one hidden layer of 16 units and 1 output unit(s); every unit has a bias. How many trainable parameters?

A. 222
B. 193
C. 176
D. 160

Answer: B. 193

Working:

- Hidden layer: (10 + 1)×16 = 176.
- Output layer: (16 + 1)×1 = 17.
- Total = 193.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- C: Forgot the bias of each neuron (+1 per unit).
- D: Weights are added layer by layer, not multiplied.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B85. [MCQ · Medium · calculation]

A fully connected network has 64 inputs, one hidden layer of 64 units and 10 output unit(s); every unit has a bias. How many trainable parameters?

A. 40960
B. 4160
C. 4810
D. 4736

Answer: C. 4810

Working:

- Hidden layer: (64 + 1)×64 = 4160.
- Output layer: (64 + 1)×10 = 650.
- Total = 4810.

Option notes:

- A: Weights are added layer by layer, not multiplied.
- B: Counted only the first layer.
- D: Forgot the bias of each neuron (+1 per unit).

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B86. [MCQ · Medium · calculation]

A fully connected network has 4 inputs, one hidden layer of 8 units and 3 output unit(s); every unit has a bias. How many trainable parameters?

A. 56
B. 96

C. 40
D. 67

Answer: D. 67

Working:

- Hidden layer: (4 + 1)×8 = 40.
- Output layer: (8 + 1)×3 = 27.
- Total = 67.

Option notes:

- A: Forgot the bias of each neuron (+1 per unit).
- B: Weights are added layer by layer, not multiplied.
- C: Counted only the first layer.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B87. [MCQ · Medium · calculation]

A fully connected network has 64 inputs, one hidden layer of 8 units and 2 output unit(s); every unit has a bias. How many trainable parameters?

A. 538
B. 528
C. 1024
D. 520

Answer: A. 538

Working:

- Hidden layer: (64 + 1)×8 = 520.
- Output layer: (8 + 1)×2 = 18.
- Total = 538.

Option notes:

- B: Forgot the bias of each neuron (+1 per unit).
- C: Weights are added layer by layer, not multiplied.
- D: Counted only the first layer.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B88. [MCQ · Medium · calculation]

A fully connected network has 128 inputs, one hidden layer of 4 units and 1 output unit(s); every unit has a bias. How many trainable parameters?

A. 599
B. 516
C. 521
D. 512

Answer: C. 521

Working:

- Hidden layer: (128 + 1)×4 = 516.
- Output layer: (4 + 1)×1 = 5.

- Total = 521.

Option notes:

- A: Arithmetic slip: recompute each step carefully.
- B: Forgot the bias of each neuron (+1 per unit).
- D: Weights are added layer by layer, not multiplied.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B89. [MCQ · Medium · calculation]

A fully connected network has 10 inputs, one hidden layer of 64 units and 10 output unit(s); every unit has a bias. How many trainable parameters?

A. 6400
B. 1354
C. 1280
D. 704

Answer: B. 1354

Working:

- Hidden layer: (10 + 1)×64 = 704.
- Output layer: (64 + 1)×10 = 650.
- Total = 1354.

Option notes:

- A: Weights are added layer by layer, not multiplied.
- C: Forgot the bias of each neuron (+1 per unit).
- D: Counted only the first layer.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B90. [MCQ · Medium · calculation]

A fully connected network has 128 inputs, one hidden layer of 32 units and 10 output unit(s); every unit has a bias. How many trainable parameters?

A. 4458
B. 4128
C. 40960
D. 4416

Answer: A. 4458

Working:

- Hidden layer: (128 + 1)×32 = 4128.
- Output layer: (32 + 1)×10 = 330.
- Total = 4458.

Option notes:

- B: Counted only the first layer.
- C: Weights are added layer by layer, not multiplied.
- D: Forgot the bias of each neuron (+1 per unit).

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B91. [MCQ · Medium · calculation]

A fully connected network has 784 inputs, one hidden layer of 16 units and 10 output unit(s); every unit has a bias. How many trainable parameters?

A. 12560
B. 12730
C. 125440
D. 12704

Answer: B. 12730

Working:

- Hidden layer: (784 + 1)×16 = 12560.
- Output layer: (16 + 1)×10 = 170.
- Total = 12730.

Option notes:

- A: Counted only the first layer.
- C: Weights are added layer by layer, not multiplied.
- D: Forgot the bias of each neuron (+1 per unit).

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B92. [MCQ · Medium · calculation]

A fully connected network has 4 inputs, one hidden layer of 4 units and 10 output unit(s); every unit has a bias. How many trainable parameters?

A. 20
B. 56
C. 70
D. 160

Answer: C. 70

Working:

- Hidden layer: (4 + 1)×4 = 20.
- Output layer: (4 + 1)×10 = 50.
- Total = 70.

Option notes:

- A: Counted only the first layer.
- B: Forgot the bias of each neuron (+1 per unit).
- D: Weights are added layer by layer, not multiplied.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B93. [MCQ · Medium · calculation]

A fully connected network has 10 inputs, one hidden layer of 4 units and 3 output unit(s); every unit has a bias. How many trainable parameters?

A. 120
B. 52
C. 59
D. 44

Answer: C. 59

Working:

- Hidden layer: (10 + 1)×4 = 44.
- Output layer: (4 + 1)×3 = 15.
- Total = 59.

Option notes:

- A: Weights are added layer by layer, not multiplied.
- B: Forgot the bias of each neuron (+1 per unit).
- D: Counted only the first layer.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B94. [MCQ · Medium · calculation]

A fully connected network has 64 inputs, one hidden layer of 4 units and 1 output unit(s); every unit has a bias. How many trainable parameters?

A. 265
B. 305
C. 256
D. 260

Answer: A. 265

Working:

- Hidden layer: (64 + 1)×4 = 260.
- Output layer: (4 + 1)×1 = 5.
- Total = 265.

Option notes:

- B: Arithmetic slip: recompute each step carefully.
- C: Weights are added layer by layer, not multiplied.
- D: Forgot the bias of each neuron (+1 per unit).

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B95. [MCQ · Medium · calculation]

A fully connected network has 4 inputs, one hidden layer of 64 units and 3 output unit(s); every unit has a bias. How many trainable parameters?

A. 768
B. 320
C. 515
D. 448

Answer: C. 515

Working:

- Hidden layer: (4 + 1)×64 = 320.
- Output layer: (64 + 1)×3 = 195.
- Total = 515.

Option notes:

- A: Weights are added layer by layer, not multiplied.
- B: Counted only the first layer.
- D: Forgot the bias of each neuron (+1 per unit).

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B96. [MCQ · Medium · calculation]

A fully connected network has 3 inputs, one hidden layer of 32 units and 3 output unit(s); every unit has a bias. How many trainable parameters?

A. 227
B. 288
C. 192
D. 128

Answer: A. 227

Working:

- Hidden layer: (3 + 1)×32 = 128.
- Output layer: (32 + 1)×3 = 99.
- Total = 227.

Option notes:

- B: Weights are added layer by layer, not multiplied.
- C: Forgot the bias of each neuron (+1 per unit).
- D: Counted only the first layer.

Takeaway: Dense layer params = (inputs + 1) × units.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B97. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [0.6,0.7], f_t = [0.4,0.2], i_t = [0.9,0.4], candidate $\tilde{c}_t$ = tanh(…) = [-0.9,-0.1]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = -0.57; $c_t[2]$ = 0.10

Working:

- $c_t[1]$ = 0.6×0.4 + 0.9×-0.9 = -0.570.
- $c_t[2]$ = 0.7×0.2 + 0.4×-0.1 = 0.100.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B98. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [0.8,-0.9], f_t = [0.9,0.5], i_t = [0.8,0.7], candidate $\tilde{c}_t$ = tanh(…) = [-0.7,-0.7]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = 0.16; $c_t[2]$ = -0.94

Working:

- $c_t[1]$ = 0.8×0.9 + 0.8×-0.7 = 0.160.
- $c_t[2]$ = -0.9×0.5 + 0.7×-0.7 = -0.940.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B99. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [0.3,1], f_t = [0.5,0.3], i_t = [0.3,0.8], candidate $\tilde{c}_t$ = tanh(…) = [0.2,0.8]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = 0.21; $c_t[2]$ = 0.94

Working:

- $c_t[1]$ = 0.3×0.5 + 0.3×0.2 = 0.210.
- $c_t[2]$ = 1×0.3 + 0.8×0.8 = 0.940.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B100. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [-1,-0.5], f_t = [0.3,0.6], i_t = [0.5,1], candidate $\tilde{c}_t$ = tanh(…) = [0.6,0.4]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = 0.00; $c_t[2]$ = 0.10

Working:

- $c_t[1]$ = -1×0.3 + 0.5×0.6 = 0.000.
- $c_t[2]$ = -0.5×0.6 + 1×0.4 = 0.100.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B101. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [0.2,-0.7], f_t = [0.1,0.5], i_t = [0.5,0.4], candidate $\tilde{c}_t$ = tanh(…) = [0.6,0]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = _____
(2) $c_t[2]$ = _____

Answer: $c_t[1] = 0.32$; $c_t[2] = -0.35$

Working:

- $c_t[1] = 0.2\times0.1 + 0.5\times0.6 = 0.320$.
- $c_t[2] = -0.7\times0.5 + 0.4\times0 = -0.350$.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B102. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). $c_{t-1} = [-0.6,0.2]$, $f_t = [1,0.8]$, $i_t = [0.4,0.4]$, candidate $\tilde{c}_t = \tanh(\ldots) = [-0.6,-0.8]$. Give both components of c_t (2 dp).

(1) $c_t[1]$ = _____
(2) $c_t[2]$ = _____

Answer: $c_t[1] = -0.84$; $c_t[2] = -0.16$

Working:

- $c_t[1] = -0.6\times1 + 0.4\times-0.6 = -0.840$.
- $c_t[2] = 0.2\times0.8 + 0.4\times-0.8 = -0.160$.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B103. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). $c_{t-1} = [-1,-0.2]$, $f_t = [0.7,0.1]$, $i_t = [0.7,0.8]$, candidate $\tilde{c}_t = \tanh(\ldots) = [0.6,0.8]$. Give both components of c_t (2 dp).

(1) $c_t[1]$ = _____
(2) $c_t[2]$ = _____

Answer: $c_t[1] = -0.28$; $c_t[2] = 0.62$

Working:

- $c_t[1] = -1\times0.7 + 0.7\times0.6 = -0.280$.
- $c_t[2] = -0.2\times0.1 + 0.8\times0.8 = 0.620$.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B104. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). $c_{t-1} = [0,-0.5]$, $f_t = [0.7,0.9]$, $i_t = [0.9,0.6]$, candidate $\tilde{c}_t = \tanh(\ldots) = [0.2,0.6]$. Give both components of c_t (2 dp).

(1) $c_t[1]$ = _____
(2) $c_t[2]$ = _____

Answer: $c_t[1] = 0.18$; $c_t[2] = -0.09$

Working:

- $c_t[1]$ = 0×0.7 + 0.9×0.2 = 0.180.
- $c_t[2]$ = -0.5×0.9 + 0.6×0.6 = -0.090.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B105. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ (⊙ = elementwise). c_{t-1} = [0.1,-0.5], f_t = [0.5,0.9], i_t = [0.7,0.8], candidate $\tilde{c}_t$ = tanh(…) = [0.2,-0.7]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = 0.19; $c_t[2]$ = -1.01

Working:

- $c_t[1]$ = 0.1×0.5 + 0.7×0.2 = 0.190.
- $c_t[2]$ = -0.5×0.9 + 0.8×-0.7 = -1.010.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B106. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ (⊙ = elementwise). c_{t-1} = [0.2,0.8], f_t = [0.5,0.8], i_t = [0.1,0.4], candidate $\tilde{c}_t$ = tanh(…) = [0.8,0.8]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = 0.18; $c_t[2]$ = 0.96

Working:

- $c_t[1]$ = 0.2×0.5 + 0.1×0.8 = 0.180.
- $c_t[2]$ = 0.8×0.8 + 0.4×0.8 = 0.960.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B107. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ (⊙ = elementwise). c_{t-1} = [-0.4,-0.4], f_t = [0,0.7], i_t = [0.2,0.7], candidate $\tilde{c}_t$ = tanh(…) = [-0.9,-0.8]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = -0.18; $c_t[2]$ = -0.84

Working:

- $c_t[1]$ = -0.4×0 + 0.2×-0.9 = -0.180.
- $c_t[2]$ = -0.4×0.7 + 0.7×-0.8 = -0.840.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B108. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [1,0.6], f_t = [0.2,0.8], i_t = [0.5,0.7], candidate $\tilde{c}_t$ = tanh(…) = [-0.8,0.8]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = -0.20; $c_t[2]$ = 1.04

Working:

- $c_t[1]$ = 1×0.2 + 0.5×-0.8 = -0.200.
- $c_t[2]$ = 0.6×0.8 + 0.7×0.8 = 1.040.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B109. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [1,0.2], f_t = [0.4,0.2], i_t = [0.6,0.5], candidate $\tilde{c}_t$ = tanh(…) = [0.1,0.3]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = 0.46; $c_t[2]$ = 0.19

Working:

- $c_t[1]$ = 1×0.4 + 0.6×0.1 = 0.460.
- $c_t[2]$ = 0.2×0.2 + 0.5×0.3 = 0.190.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B110. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [0.6,-0.6], f_t = [0.8,0.1], i_t = [0.4,0.1], candidate $\tilde{c}_t$ = tanh(…) = [0.9,0]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1]$ = 0.84; $c_t[2]$ = -0.06

Working:

- $c_t[1]$ = 0.6×0.8 + 0.4×0.9 = 0.840.
- $c_t[2]$ = -0.6×0.1 + 0.1×0 = -0.060.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B111. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [-1,0.3], f_t = [0.1,0.6], i_t = [0.7,0.1], candidate $\tilde{c}_t$ = tanh(…) = [0.5,-0.9]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1] = 0.25$; $c_t[2] = 0.09$

Working:

- $c_t[1]$ = -1×0.1 + 0.7×0.5 = 0.250.
- $c_t[2]$ = 0.3×0.6 + 0.1×-0.9 = 0.090.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

NN B112. [BLANKS · Hard · calculation]

LSTM cell update $c_t = c_{t-1} \odot f_t + i_t \odot \tilde{c}_t$ ($\odot$ = elementwise). c_{t-1} = [0.3,-0.7], f_t = [0.8,0.8], i_t = [0.5,0.5], candidate $\tilde{c}_t$ = tanh(…) = [-0.8,0.1]. Give both components of c_t (2 dp).

(1) $c_t[1]$ = ____
(2) $c_t[2]$ = ____

Answer: $c_t[1] = -0.16$; $c_t[2] = -0.51$

Working:

- $c_t[1]$ = 0.3×0.8 + 0.5×-0.8 = -0.160.
- $c_t[2]$ = -0.7×0.8 + 0.5×0.1 = -0.510.

Takeaway: Forget gate f keeps (≈1) or erases (≈0) memory; input gate i adds new info ADDITIVELY, so gradients don't shrink multiplicatively.

Source: Workshop 5 (Neural network models: perceptron, backpropagation, LSTM)

⛰️ Local Search & GA (Lecture 3)

Part A: hand-written questions (81)

LOCAL A1. [MCQ · Easy]

What distinguishes a local search (optimisation) problem from classical path search?

A. Only the final state matters, not the path to reach it
B. It always uses a heuristic h(n)
C. It requires a known goal state
D. It needs a FIFO frontier

Answer: A. Only the final state matters, not the path to reach it

Option notes:

- B: Not required.
- C: Often only an objective to optimise.
- D: No frontier needed.

Takeaway: Destination, not journey.

Source: Lecture 3, slide 5

LOCAL A2. [BLANK · Medium · calculation]

In 8-queens (one queen per column, moves within its column) each state has 8 × 7 = _____ successors.

Answer: 56

Takeaway: 8 queens × 7 other rows.

Source: Lecture 3, slide 10

LOCAL A3. [TF · Medium]

Minimising $y = x^2 - 4x + 7$ subject to $-2 \le x \le 6$ is a discrete optimisation problem.

Answer: False

Why: x is drawn from a continuous range → continuous optimisation.

Takeaway: Integers → discrete; real ranges → continuous.

Source: Lecture 3, slide 7

LOCAL A4. [MATCH · Medium]

Delivery route optimisation (Lecture 3): match each concept to its meaning.

1. STATE
2. NEIGHBOUR
3. OBJECTIVE
4. CONSTRAINT

A. route
B. minimise distance
C. visit each customer once

D. maximise the number of stops
E. swap two stops

Answer: 1 → A, 2 → E, 3 → B, 4 → C

Working:

- STATE → route
- NEIGHBOUR → swap two stops
- OBJECTIVE → minimise distance
- CONSTRAINT → visit each customer once

Takeaway: Formulate before solving.

Source: Lecture 3, slide 12

LOCAL A5. [BLANK · Easy]

The 8-queens problem has _____ goal states (12 fundamental solutions).

Answer: 92

Takeaway: Lecture 3 slide 10.

Source: Lecture 3, slide 10

LOCAL A6. [MCQ · Medium]

In the 8-queens formulation, what is the cost function?

A. The number of pairs of queens attacking each other
B. The number of queens on the board
C. The number of empty rows
D. The distance moved by the last queen

Answer: A. The number of pairs of queens attacking each other

Option notes:

- B: Always 8.
- C: Not the lecture's cost.
- D: Path cost is irrelevant.

Takeaway: cost(2,4,6,8,3,1,7,5) = 0.

Source: Lecture 3, slide 11

LOCAL A7. [TF · Medium]

Steepest-ascent hill climbing always finds the global maximum.

Answer: False

Why: It stops at the first peak where no neighbour is better: possibly a local maximum.

Takeaway: Restart or use SA/GA to escape.

Source: Lecture 3, slide 14-16

LOCAL A8. [MCQ · Easy]

In hill climbing, what happens if the best neighbour's value equals the current value?

A. The algorithm returns the current state
B. It picks a random neighbour

C. It moves to that neighbour
D. It restarts

Answer: A. The algorithm returns the current state

Option notes:

- B: That's not steepest ascent.
- C: Moves only if strictly better (VALUE(neighbor) ≤ VALUE(current) → return).
- D: No restart in the basic algorithm.

Takeaway: Pseudocode Fig 4.2.

Source: Lecture 3, slide 16

LOCAL A9. [MULTI · Medium]

Select ALL true statements about hill climbing (Lecture 3). (Select ALL that apply.)

A. It keeps track of one current state
B. It needs a population of candidates
C. For 8-queens it maximises f = −cost
D. It accepts worse moves with probability e^(−Δ/T)
E. It moves to the neighbour with the highest value (steepest ascent)

Answer: A. It keeps track of one current state; C. For 8-queens it maximises f = −cost; E. It moves to the neighbour with the highest value (steepest ascent)

Option notes:

- A: Correct:
- B: Incorrect: That's GA.
- C: Correct: Slide 15.
- D: Incorrect: That's SA.
- E: Correct:

Takeaway: One state, greedy moves.

Source: Lecture 3, slide 14-16

LOCAL A10. [MCQ · Medium · calculation]

In simulated annealing, a neighbour is worse by Δ = 2 at T = 5. Acceptance probability?

A. e^(−10) ≈ 0.000045
B. 0.4
C. e^(0.4) ≈ 1.49
D. e^(−0.4) ≈ 0.670

Answer: D. e^(−0.4) ≈ 0.670

Option notes:

- A: Used Δ × T.
- B: Forgot the exponential.
- C: Sign error; probabilities ≤ 1.

Takeaway: p = exp(−Δ/T).

Source: Lecture 3, slide 23

LOCAL A11. [TF · Medium]

As the temperature T decreases, simulated annealing becomes MORE likely to accept worse moves.

Answer: False

Why: Lower T → exp(−Δ/T) smaller → fewer worse moves accepted.

Takeaway: High T explores; low T is selective.

Source: Lecture 3, slide 23

LOCAL A12. [MATCH · Medium]

Match each SA limitation to its consequence.

1. Cooling too fast
2. Cooling too slow
3. Parameter sensitivity
4. Randomness

A. different runs may return different solutions
B. may require many iterations before convergence
C. guaranteed global optimum in one run
D. initial temperature and schedule must be chosen carefully
E. may behave like hill climbing and get trapped early

Answer: 1 → E, 2 → B, 3 → D, 4 → A

Working:

- Cooling too fast → may behave like hill climbing and get trapped early
- Cooling too slow → may require many iterations before convergence
- Parameter sensitivity → initial temperature and schedule must be chosen carefully
- Randomness → different runs may return different solutions

Takeaway: SA can escape local optima but guarantees nothing in one run.

Source: Lecture 3, slide 28

LOCAL A13. [MCQ · Easy]

Simulated annealing is described in Lecture 3 as a version of…

A. breadth-first search with a temperature
B. A* with a cooling heuristic
C. a genetic algorithm with one individual
D. stochastic hill climbing where some downhill moves are allowed

Answer: D. stochastic hill climbing where some downhill moves are allowed

Option notes:

- A: Not a tree search.
- B: No.
- C: No crossover.

Takeaway: Fig 4.5 caption.

Source: Lecture 3, slide 24

LOCAL A14. [MULTI · Medium]

Which are stages of the metallurgical annealing analogy? (Select ALL that apply.)

A. Heating
B. Slow cooling

C. Rapid quenching in ice water
D. Soaking (hold temperature)

Answer: A. Heating; B. Slow cooling; D. Soaking (hold temperature)

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: That's the opposite of annealing.
- D: Correct:

Takeaway: Slow cooling → low-energy, stable state.

Source: Lecture 3, slide 20

LOCAL A15. [BLANK · Easy · calculation]

Individual | Fitness | P = f/78 | Wheel segment (x4, x3, x2, x1 order)
x1 | 24 | 0.31 | [0.69, 1.0]
x2 | 23 | 0.29 | [0.40, 0.69)
x3 | 20 | 0.26 | [0.14, 0.40)
x4 | 11 | 0.14 | [0, 0.14)

Roulette wheel: fitness 24, 23, 20, 11 (total 78). P(x1) = 24/78 = _____ (2 dp).

Answer: 0.31
(Also accepted: 0.308, 0.3077)

Takeaway: Lecture 3 slide 39.

Source: Lecture 3, slide 39

LOCAL A16. [MCQ · Medium · calculation]

Individual | Fitness | P = f/78 | Wheel segment (x4, x3, x2, x1 order)
x1 | 24 | 0.31 | [0.69, 1.0]
x2 | 23 | 0.29 | [0.40, 0.69)
x3 | 20 | 0.26 | [0.14, 0.40)
x4 | 11 | 0.14 | [0, 0.14)

Lecture 3 wheel order x4 [0, 0.14), x3 [0.14, 0.40), x2 [0.40, 0.69), x1 [0.69, 1.0]. Random number 0.416 selects…

A. x2
B. x1
C. x3
D. x4

Answer: A. x2

Option notes:

- B: 0.416 < 0.69.
- C: 0.416 > 0.40.
- D: 0.416 > 0.14.

Takeaway: 0.40 ≤ 0.416 < 0.69.

Source: Lecture 3, slide 40

LOCAL A17. [MATCH · Easy]

Match each GA operator to what it does.

1. Selection
2. Crossover
3. Mutation
4. Elitism

A. combine parts of two parents
B. choose parents; fitter individuals more likely
C. copy a few best candidates directly
D. sort the population alphabetically
E. randomly alter a small part to keep diversity

Answer: 1 → B, 2 → A, 3 → E, 4 → C

Working:

- Selection → choose parents; fitter individuals more likely
- Crossover → combine parts of two parents
- Mutation → randomly alter a small part to keep diversity
- Elitism → copy a few best candidates directly

Takeaway: Lecture 3 slide 35.

Source: Lecture 3, slide 35

LOCAL A18. [TF · Medium]

In tournament selection, a larger tournament size k increases diversity in the population.

Answer: False

Why: Larger k → higher selection pressure → LESS diversity, faster convergence. Small k (2) keeps more diversity.

Takeaway: k controls selection pressure.

Source: Lecture 3, slide 36

LOCAL A19. [MULTI · Medium]

Which factors affect an evolutionary algorithm (Lecture 3)? (Select ALL that apply.)

A. Population size
B. Representation of each individual
C. Selection and recombination procedures
D. Mutation rate
E. The learning rate α

Answer: A. Population size; B. Representation of each individual; C. Selection and recombination procedures; D. Mutation rate

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Correct:
- E: Incorrect: That's gradient descent.

Takeaway: Also mixing number and makeup of the next generation.

Source: Lecture 3, slide 32

LOCAL A20. [MCQ · Medium · calculation]

Tournament (k = 3) with population A70, B55, C40, D30, E20, F10 draws {C, D, E}. Winner?

A. C
B. A
C. D
D. E

Answer: A. C

Option notes:

- B: A wasn't drawn.
- C: 40 > 30.
- D: Lowest fitness.

Takeaway: Best fitness among the k drawn.

Source: Lecture 3, slide 36

LOCAL A21. [TF · Easy]

The genetic algorithm was introduced by Holland in 1975 and uses reproduction, crossover and mutation operators.

Answer: True

Why: Lecture 3 slide 33.

Takeaway: Inspired by natural evolution.

Source: Lecture 3 (Local search)

LOCAL A22. [MATCH · Hard]

Which local search method fits each scenario?

1. Smooth landscape, need a quick decent answer
2. Many local optima, single candidate, can tune a cooling schedule
3. Huge search space with many good alternatives to explore in parallel

A. simulated annealing
B. A* search
C. hill climbing
D. genetic algorithm

Answer: 1 → C, 2 → A, 3 → D

Working:

- Smooth landscape, need a quick decent answer → hill climbing
- Many local optima, single candidate, can tune a cooling schedule → simulated annealing
- Huge search space with many good alternatives to explore in parallel → genetic algorithm

Takeaway: HC fast, SA escapes optima, GA explores populations.

Source: Lecture 3, slide 14-41

LOCAL A23. [MATCH · Easy]

Delivery-route optimisation (Lecture 3). Match each part of the formulation.

1. STATE
2. NEIGHBOUR
3. OBJECTIVE
4. CONSTRAINT

A. swap two stops
B. maximise the number of customers
C. visit each location once and return to base
D. a route (ordering of the stops)
E. minimise distance

Answer: 1 → D, 2 → A, 3 → E, 4 → C

Working:

- STATE → a route (ordering of the stops)
- NEIGHBOUR → swap two stops
- OBJECTIVE → minimise distance
- CONSTRAINT → visit each location once and return to base

Takeaway: State, neighbour, objective, constraint.

Source: Book 2 (Exam examples), Q41, p. 12

LOCAL A24. [BLANK · Medium · calculation]

A driver must visit 20 customers. The number of possible orders is 20! ≈ ___ (write e.g. 1.2e18).

Answer: 2.4e18
(Also accepted: 2.4 × 10^18, 2.4x10^18, 2.43e18, 2.4×10^18)

Takeaway: Too many to check exhaustively, so use local search.

Source: Book 2 (Exam examples), Q42, p. 12

LOCAL A25. [BLANKS · Medium · calculation]

Minimise $y = x^2 - 4x + 7$ subject to $-2 \le x \le 6$. This is a ___ optimisation problem; the minimum is at x = ___ with y = ___.

(1) type = ____
(2) x = ____
(3) y = ____

Answer: type = continuous; x = 2; y = 3

Takeaway: Complete the square: $y = (x - 2)^2 + 3$.

Source: Book 2 (Exam examples), Q43, p. 12

LOCAL A26. [BLANK · Easy · calculation]

In 8-queens with one queen per column, each state has ___ successors.

Answer: 56

Takeaway: 8 queens × 7 other rows.

Source: Book 2 (Exam examples), Q44, p. 12

LOCAL A27. [BLANKS · Easy]

8-queens has ___ solutions, of which ___ are fundamental.

(1) solutions = ____
(2) fundamental = ____

Answer: solutions = 92; fundamental = 12

Takeaway: The rest are reflections and rotations.

Source: Book 2 (Exam examples), Q45, p. 12

LOCAL A28. [BLANK · Hard · calculation]

cost(2, 4, 7, 4, 8, 5, 5, 2) = ___ (attacking pairs).

Answer: 4

Working:

- Same row: (1,8) row 2, (2,4) row 4, (6,7) row 5.
- Diagonal: cols 3 & 8, |7 − 2| = 5 = 8 − 3.
- Total 4.

Takeaway: Rows 2 (cols 1 & 8), 4 (cols 2 & 4), 5 (cols 6 & 7) + one diagonal (cols 3 & 8).

Source: Book 2 (Exam examples), Q46, p. 12

LOCAL A29. [BLANK · Medium · calculation]

cost(2, 4, 6, 8, 3, 1, 7, 5) = ___.

Answer: 0

Working:

- Same row: x_i = x_j.
- Same diagonal: |x_i − x_j| = j − i.
- Check all 28 pairs.

Takeaway: It is a solution.

Source: Book 2 (Exam examples), Q47, p. 12

LOCAL A30. [BLANK · Medium · calculation]

cost(1, 2, 3, 4, 5, 6, 7, 8) = ___.

Answer: 28

Working:

- Same row: x_i = x_j.
- Same diagonal: |x_i − x_j| = j − i.
- Check all 28 pairs.

Takeaway: All 8 queens on one diagonal: C(8,2) = 28.

Source: Book 2 (Exam examples), Q48, p. 12

LOCAL A31. [BLANK · Easy · calculation]

cost(1, 1, 1, 1, 1, 1, 1, 1) = ___.

Answer: 28

Working:

- Same row: x_i = x_j.
- Same diagonal: |x_i − x_j| = j − i.
- Check all 28 pairs.

Takeaway: All in one row: every pair attacks.

Source: Book 2 (Exam examples), Q49, p. 13

LOCAL A32. [BLANK · Hard · calculation]

cost(1, 3, 5, 7, 2, 4, 6, 8) = ___.

Answer: 1

Working:

- Same row: x_i = x_j.
- Same diagonal: |x_i − x_j| = j − i.
- Check all 28 pairs.

Takeaway: Only columns 1 and 8: |1 − 8| = 7 = 8 − 1.

Source: Book 2 (Exam examples), Q50, p. 13

LOCAL A33. [MCQ · Medium]

For hill climbing on 8-queens the objective is written f = max(___).

A. 1/cost(X)
B. −cost(X)
C. Σ x_i
D. cost(X)

Answer: B. −cost(X)

Option notes:

- A: Undefined at a solution (cost 0).
- C: Row numbers are not quality.
- D: That maximises attacks.

Takeaway: Maximising −cost = minimising cost.

Source: Book 2 (Exam examples), Q51, p. 13

LOCAL A34. [TF · Easy]

Local search keeps the path from the start state to the solution.

Answer: False

Why: Only the final state matters.

Takeaway: Classic search = path; local search = state.

Source: Book 2 (Exam examples), Q52, p. 13

LOCAL A35. [MCQ · Easy]

A local search problem is best described as:

A. an optimisation problem
B. a planning problem with a fixed action sequence

C. a classification problem
D. a navigation problem

Answer: A. an optimisation problem

Option notes:

- B: That is classic search.
- C: That is supervised learning.
- D: Navigation needs a path (classic search).

Takeaway: Find the best state.

Source: Book 2 (Exam examples), Q53, p. 13

LOCAL A36. [MCQ · Easy]

Hill climbing stops when:

A. no neighbour is better than the current state
B. the temperature reaches 0
C. the frontier is empty
D. the population converges

Answer: A. no neighbour is better than the current state

Option notes:

- B: That is simulated annealing.
- C: That is classic search.
- D: That is a GA.

Takeaway: Stop at a peak.

Source: Book 2 (Exam examples), Q54, p. 13

LOCAL A37. [MULTI · Medium]

Hill climbing can get stuck on: (Select ALL that apply.)

A. ridges
B. shoulders
C. the global maximum
D. plateaus
E. local maxima

Answer: A. ridges; B. shoulders; D. plateaus; E. local maxima

Option notes:

- A: Correct: sequence of local maxima
- B: Correct: a flat area that later rises
- C: Incorrect: stopping there is success, not being stuck
- D: Correct: flat: no improving move
- E: Correct: no neighbour higher

Takeaway: Local maxima, ridges, plateaus, shoulders.

Source: Book 2 (Exam examples), Q55, p. 13

LOCAL A38. [BLANK · Easy · calculation]

Current state value 10; neighbours have values 8, 12, 15, 9. Steepest-ascent hill climbing moves to the neighbour with value ___.

Answer: 15

Takeaway: Best neighbour, not first improving one.

Source: Book 2 (Exam examples), Q56, p. 13

LOCAL A39. [TF · Easy]

Hill climbing is also called steepest ascent because it moves to the best neighbour.

Answer: True

Why: It heads in the direction of steepest ascent.

Takeaway: Steepest ascent = best neighbour.

Source: Book 2 (Exam examples), Q57, p. 13

LOCAL A40. [MCQ · Easy]

Simulated annealing accepts a worse move (worse by $\Delta > 0$) with probability p = ___.

A. e^(Δ/T)
B. 1 − e^(−Δ/T)
C. e^(−Δ/T)
D. Δ/T

Answer: C. e^(−Δ/T)

Option notes:

- A: That is > 1 for $\Delta > 0$.
- B: That inverts the behaviour.
- D: Not a probability rule from the slides.

Takeaway: Bigger Δ or smaller T → smaller p.

Source: Book 2 (Exam examples), Q58, p. 13

LOCAL A41. [BLANK · Medium · calculation]

SA: a move is worse by $\Delta = 10$ at $T = 20$. p = ___ (2 d.p.).

Answer: 0.61

Working:

- $\Delta/T = 0.5$
- $e^{(-0.5)} = 0.6065 \approx 0.61$

Takeaway: $e^{(-0.5)} = 0.607$.

Source: Book 2 (Exam examples), Q59, p. 13

LOCAL A42. [BLANK · Medium · calculation]

SA: same move ($\Delta = 10$) at $T = 5$. p = ___ (2 d.p.).

Answer: 0.14

Working:

- $\Delta/T = 2$
- $e^{(-2)} = 0.1353 \approx 0.14$

Takeaway: Lower temperature ⇒ more selective.

Source: Book 2 (Exam examples), Q60, p. 13

LOCAL A43. [BLANK · Medium · calculation]

SA: Δ = 3, T = 1. p = ___ (3 d.p.).

Answer: 0.050

Takeaway: e^(−3) = 0.0498.

Source: Book 2 (Exam examples), Q61, p. 13

LOCAL A44. [BLANK · Medium · calculation]

SA: Δ = 1, T = 10. p = ___ (2 d.p.).

Answer: 0.90

Takeaway: e^(−0.1) = 0.905.

Source: Book 2 (Exam examples), Q62, p. 14

LOCAL A45. [MCQ · Easy]

A move improves the cost by 5. SA accepts it with probability:

A. e^(−5/T)
B. 0.5
C. e^(5/T)
D. 1

Answer: D. 1

Option notes:

- A: That is for worse moves.
- B: No coin toss for better moves.
- C: Not a probability.

Takeaway: Better moves are always accepted.

Source: Book 2 (Exam examples), Q63, p. 14

LOCAL A46. [MULTI · Medium]

Which are true of simulated annealing? (Select ALL that apply.)

A. different runs may give different results
B. performance depends on the initial temperature and schedule
C. it guarantees the global optimum in one run
D. high T explores widely
E. cooling too fast makes it behave like hill climbing

Answer: A. different runs may give different results; B. performance depends on the initial temperature and schedule; D. high T explores widely; E. cooling too fast makes it behave like hill climbing

Option notes:

- A: Correct: it is random
- B: Correct: sensitive to tuning
- C: Incorrect: it does not
- D: Correct: accepts many bad moves
- E: Correct: trapped early

Takeaway: SA escapes local optima but guarantees nothing in one run.

Source: Book 2 (Exam examples), Q64, p. 14

LOCAL A47. [BLANKS · Easy]

SA is named after the ___ process in metallurgy; its objective is to minimise ___.

(1) process = ____
(2) quantity = ____

Answer: process = annealing; quantity = energy

Takeaway: Heat, then cool slowly.

Source: Book 2 (Exam examples), Q65, p. 14

LOCAL A48. [BLANKS · Easy]

Tournament selection, k = 3. Fitness A 70, B 55, C 40, D 30, E 20, F 10. Tournaments {A, C, E}, {B, D, F}, {C, D, E} select ___, ___, ___.

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = A; T2 = B; T3 = C

Takeaway: Highest fitness in each group wins.

Source: Book 2 (Exam examples), Q66, p. 14

LOCAL A49. [BLANKS · Easy]

Same population (A 70, B 55, C 40, D 30, E 20, F 10). Tournaments {D, E, F} and {B, C, F} select ___ and ___.

(1) T1 = ____
(2) T2 = ____

Answer: T1 = D; T2 = B

Takeaway: D 30 is best of D, E, F.

Source: Book 2 (Exam examples), Q67, p. 14

LOCAL A50. [MCQ · Medium]

Increasing the tournament size k:

A. increases selection pressure, faster convergence
B. decreases selection pressure
C. has no effect
D. increases diversity

Answer: A. increases selection pressure, faster convergence

Option notes:

- B: Opposite.
- C: k sets the pressure.
- D: Small k keeps diversity.

Takeaway: Small k diverse, large k greedy.

Source: Book 2 (Exam examples), Q68, p. 14

LOCAL A51. [BLANKS · Medium · calculation]

Roulette: f1 = 24, f2 = 23, f3 = 20, f4 = 11. Total ___; P1 = ___, P4 = ___ (2 d.p.).

(1) total = ____
(2) P1 = ____
(3) P4 = ____

Answer: total = 78; P1 = 0.31; P4 = 0.14

Takeaway: P_i = f_i / Σf.

Source: Book 2 (Exam examples), Q69, p. 14

LOCAL A52. [BLANK · Medium · calculation]

Roulette with sorted cumulative intervals x4 [0,11), x3 [11,31), x2 [31,54), x1 [54,78]. Random numbers 15, 42, 51, 69 select (comma separated) ___.

Answer: x3, x2, x2, x1
(Also accepted: x3,x2,x2,x1, x3 x2 x2 x1)

Takeaway: Find the interval each number lands in.

Source: Book 2 (Exam examples), Q70, p. 14

LOCAL A53. [BLANK · Medium · calculation]

Using probabilities instead (cumulative 0.14, 0.40, 0.69, 1.0 for x4, x3, x2, x1), random numbers 0.1637, 0.4162, 0.6071, 0.8940 select ___.

Answer: x3, x2, x2, x1
(Also accepted: x3,x2,x2,x1, x3 x2 x2 x1)

Takeaway: Same as with raw fitness.

Source: Book 2 (Exam examples), Q71, p. 14

LOCAL A54. [BLANKS · Medium · calculation]

Fitness 10, 20, 30, 40 for a, b, c, d (in that order). Cumulative probabilities: ___. A random number 0.45 selects ___.

(1) cumulative = ____
(2) selected = ____

Answer: cumulative = 0.1, 0.3, 0.6, 1.0; selected = c

Takeaway: 0.45 lies in [0.3, 0.6).

Source: Book 2 (Exam examples), Q72, p. 14

LOCAL A55. [BLANK · Easy · calculation]

Roulette: A 50, B 30, C 15, D 5. Probability of selecting C = ___.

Answer: 15%
(Also accepted: 0.15, 15)

Takeaway: 15/100.

Source: Book 2 (Exam examples), Q73, p. 14

LOCAL A56. [MATCH · Easy]

Match the GA operator to its purpose.

1. selection
2. crossover
3. mutation
4. elitism

A. choose parents, fitter more likely
B. copy the best directly to the next generation
C. randomly alter a small part to keep diversity
D. combine parts of two parents

Answer: 1 → A, 2 → D, 3 → C, 4 → B

Working:

- selection → choose parents, fitter more likely
- crossover → combine parts of two parents
- mutation → randomly alter a small part to keep diversity
- elitism → copy the best directly to the next generation

Takeaway: Four operators.

Source: Book 2 (Exam examples), Q74, p. 14

LOCAL A57. [MULTI · Medium]

Factors affecting an evolutionary algorithm (slides): (Select ALL that apply.)

A. makeup of next generation
B. representation
C. learning rate α
D. mixing number
E. mutation rate
F. population size

Answer: A. makeup of next generation; B. representation; D. mixing number; E. mutation rate; F. population size

Option notes:

- A: Correct: listed
- B: Correct: listed
- C: Incorrect: gradient descent, not EAs
- D: Correct: parents per child
- E: Correct: listed
- F: Correct: listed

Takeaway: Also selection and recombination procedures.

Source: Book 2 (Exam examples), Q75, p. 15

LOCAL A58. [BLANK · Easy]

In a traditional GA an individual is represented as a ___, a string of 0s and 1s.

Answer: chromosome

Takeaway: Bit-string chromosome.

Source: Book 2 (Exam examples), Q76, p. 15

LOCAL A59. [BLANK · Easy]

The genetic algorithm was introduced by ___ in 1975.

Answer: Holland
(Also accepted: John Holland, J. Holland)

Takeaway: John Holland, 1975.

Source: Book 2 (Exam examples), Q77, p. 15

LOCAL A60. [TF · Medium]

Tournament selection requires fitness values to be normalised into probabilities.

Answer: False

Why: It only compares fitness inside each group.

Takeaway: No normalisation needed.

Source: Book 2 (Exam examples), Q78, p. 15

LOCAL A61. [TF · Medium]

In roulette selection an individual with fitness 0 can never be selected.

Answer: True

Why: Its slice of the wheel is empty.

Takeaway: f = 0 → P = 0.

Source: Book 2 (Exam examples), Q79, p. 15

LOCAL A62. [BLANK · Medium · calculation]

SA: a move is worse by Δ = 4 at temperature T = 2. Acceptance probability ___ (3 d.p.).

Answer: 0.135

Takeaway: e^(−2).

Source: Book 2 (Exam examples), Mixed practice paper Q4, p. 54

LOCAL A63. [MATCH · Easy]

Match the local search algorithm.

1. hill climbing
2. simulated annealing
3. genetic algorithm

A. single state, always move to best neighbour
B. single state, may accept worse moves with probability e^(−Δ/T)
C. population, crossover, mutation

Answer: 1 → A, 2 → B, 3 → C

Working:

- hill climbing → single state, always move to best neighbour
- simulated annealing → single state, may accept worse moves with probability e^(−Δ/T)

- genetic algorithm → population, crossover, mutation

Takeaway: Three algorithms.

Source: Book 2 (Exam examples), Mixed practice paper Q5, p. 54

LOCAL A64. [BLANKS · Easy · calculation]

Fitness 10, 30, 60. Roulette probabilities?

(1) P1 = ____
(2) P2 = ____
(3) P3 = ____

Answer: P1 = 0.1; P2 = 0.3; P3 = 0.6

Takeaway: $f_i/\Sigma f$.

Source: Book 3 (AI from zero), Practice 10.1, p. 42

LOCAL A65. [BLANK · Easy · calculation]

SA, minimising: Δ = 3, T = 3. Acceptance probability = ___ (3 d.p.).

Answer: 0.368

Takeaway: e^{-1}.

Source: Book 3 (AI from zero), Practice 10.2, p. 42

LOCAL A66. [MATCH · Medium]

What does SA behave like at extreme temperatures?

1. T always huge
2. T always near 0

A. hill climbing
B. a random walk
C. a genetic algorithm

Answer: 1 → B, 2 → A

Working:

- T always huge → a random walk
- T always near 0 → hill climbing

Takeaway: Hot explores, cold exploits.

Source: Book 3 (AI from zero), Practice 10.3, p. 42

LOCAL A67. [BLANK · Easy · calculation]

How many neighbours does an 8-queens state have?

Answer: 56

Takeaway: 8 × 7.

Source: Book 3 (AI from zero), Practice 10.4, p. 42

LOCAL A68. [TF · Easy]

Local search returns a path.

Answer: False

Why: It returns a state.

Takeaway: State, not path.

Source: Book 3 (AI from zero), Practice 10.5, p. 42

LOCAL A69. [BLANKS · Medium · calculation]

SA, a move 2 worse (Δ = 2). p at T = 10 = ___; at T = 1 = ___ (3 d.p.).

(1) T = 10 = _____
(2) T = 1 = _____

Answer: T = 10 = 0.819; T = 1 = 0.135

Takeaway: Cooling makes SA picky.

Source: Book 3 (AI from zero), Chapter 10, p. 37

LOCAL A70. [BLANK · Medium · calculation]

Hill climbing on $y = -x^2 + 4x$ over integers, start x = 0, neighbours x ± 1. It stops at x = ___.

Answer: 2

Working:

- y(0) = 0 → y(1) = 3
- y(1) = 3 → y(2) = 4
- neighbours 3, 3 → stop

Takeaway: y(2) = 4 beats y(1) = y(3) = 3.

Source: Book 3 (AI from zero), Chapter 10, p. 37

LOCAL A71. [BLANKS · Medium]

One-point crossover after position 3: parents 11001 and 00111. Children ___ and ___.

(1) child 1 = _____
(2) child 2 = _____

Answer: child 1 = 11011; child 2 = 00101

Working:

- 110|01 + 001|11 → 110|11 and 001|01

Takeaway: Swap the tails.

Source: Book 3 (AI from zero), Chapter 10, p. 37

LOCAL A72. [MCQ · Hard]

Two SA conventions: maximising value uses e^(ΔE/T) with ΔE 0. Which is true?

A. both give the same probability between 0 and 1
B. they give opposite decisions
C. the second can exceed 1
D. only the first is a probability

Answer: A. both give the same probability between 0 and 1

Option notes:

- B: Same decision.

- C: Δ > 0 keeps it below 1.
- D: Both are.

Takeaway: Same rule, different sign convention.

Source: Book 3 (AI from zero), Chapter 10, p. 37

LOCAL A73. [BLANK · Medium]

8-queens: how many of the 92 solutions are fundamental (e.g. 24683175, 17468253, ...)?

Answer: 12

Takeaway: The rest are reflections/rotations.

Source: Book 1 (Compiled lecture notes), Chapter 3, p. 17

LOCAL A74. [BLANKS · Medium]

(Extra, not in your slides) Steepest-ascent hill climbing on random 8-queens states gets stuck ___ % of the time and solves only ___ % (it takes about 4 steps when it succeeds).

(1) stuck % = ____
(2) solved % = ____

Answer: stuck % = 86; solved % = 14

Takeaway: Fast, but trapped by local maxima.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: hill climbing on 8-queens

LOCAL A75. [BLANK · Medium]

(Extra, not in your slides) If each hill-climbing run succeeds with probability p, random-restart hill climbing needs about ___ restarts on average (write in terms of p).

Answer: 1/p
(Also accepted: 1 / p)

Takeaway: With p ≈ 0.14 for 8-queens, about 7 runs.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: random restarts

LOCAL A76. [TF · Medium]

(Extra, not in your slides) Allowing up to 100 sideways moves raises hill climbing's 8-queens success rate from 14% to 94%, at the cost of more steps.

Answer: True

Why: Sideways moves cross plateaus (shoulders).

Takeaway: Plateaus are escapable by sideways moves.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: sideways moves

LOCAL A77. [BLANK · Easy · calculation]

Complete-state 8-queens (one queen per column, any row) has 8^8 = ___ states.

Answer: 16777216
(Also accepted: 16,777,216, 1.7e7)

Takeaway: About 17 million.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: 8-queens state space

LOCAL A78. [BLANKS · Medium · calculation]

Individual | Fitness | P = f/78 | Wheel segment (x4, x3, x2, x1 order)
x1 | 24 | 0.31 | [0.69, 1.0]
x2 | 23 | 0.29 | [0.40, 0.69)
x3 | 20 | 0.26 | [0.14, 0.40)
x4 | 11 | 0.14 | [0, 0.14)

AIMA genetic algorithm for 8-queens: fitness = number of NON-attacking pairs. A solution has fitness ___. Four states with fitness 24, 23, 20, 11 are selected with probabilities ___ %, ___ %, ___ %, ___ %.

(1) solution fitness = ____
(2) 24 = ____
(3) 23 = ____
(4) 20 = ____
(5) 11 = ____

Answer: solution fitness = 28; 24 = 31; 23 = 29; 20 = 26; 11 = 14

Takeaway: Same numbers as the Lecture 3 roulette example.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: genetic algorithm

LOCAL A79. [MCQ · Medium]

(Extra, not in your slides) Local beam search with k states differs from k independent random restarts because:

A. it uses crossover
B. the k states share information: the best successors of ALL states are kept
C. it accepts worse moves with probability e^(−Δ/T)
D. it keeps only one state

Answer: B. the k states share information: the best successors of ALL states are kept

Option notes:

- A: That is a GA.
- C: That is SA.
- D: That is hill climbing.

Takeaway: Useful information passes among the k threads.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: local beam search

LOCAL A80. [TF · Hard]

(Extra, not in your slides) If the temperature schedule lowers T slowly enough, simulated annealing finds a global optimum with probability approaching 1.

Answer: True

Why: AIMA's theoretical guarantee; in practice schedules are too fast for it.

Takeaway: Slow cooling in theory, heuristic in practice.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: simulated annealing

LOCAL A81. [MCQ · Medium]

Why are ridges hard for hill climbing?

A. they are flat
B. they have no neighbours
C. they are the global maximum
D. they are a sequence of local maxima not directly connected by single moves

Answer: D. they are a sequence of local maxima not directly connected by single moves

Option notes:

- A: That is a plateau.
- B: Every state has neighbours.
- C: No.

Takeaway: Each step along the ridge looks downhill.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 4: landscape

Part B: calculation drills (80)

LOCAL B1. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 12
x2 | 14
x3 | 22
x4 | 22

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.5695.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.171; Selected individual = x3

Working:

- Σf = 12 + 14 + 22 + 22 = 70.
- P(x1) = 12/70 = 0.1714.
- x4: [0.0000, 0.3143)
- x3: [0.3143, 0.6286)
- x2: [0.6286, 0.8286)
- x1: [0.8286, 1.0000)
- r = 0.5695 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B2. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 5
x2 | 12
x3 | 18
x4 | 11

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.5493.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.109; Selected individual = x3

Working:

- Σf = 5 + 12 + 18 + 11 = 46.
- P(x1) = 5/46 = 0.1087.
- x4: [0.0000, 0.2391)
- x3: [0.2391, 0.6304)
- x2: [0.6304, 0.8913)
- x1: [0.8913, 1.0000)
- r = 0.5493 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B3. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 22
x2 | 7
x3 | 9
x4 | 24

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.548.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.355; Selected individual = x2

Working:

- Σf = 22 + 7 + 9 + 24 = 62.
- P(x1) = 22/62 = 0.3548.
- x4: [0.0000, 0.3871)
- x3: [0.3871, 0.5323)
- x2: [0.5323, 0.6452)
- x1: [0.6452, 1.0000)
- r = 0.548 falls in x2's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B4. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 26
x2 | 6
x3 | 21
x4 | 15

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.5275.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.382; Selected individual = x3

Working:

- Σf = 26 + 6 + 21 + 15 = 68.
- P(x1) = 26/68 = 0.3824.
- x4: [0.0000, 0.2206)
- x3: [0.2206, 0.5294)
- x2: [0.5294, 0.6176)
- x1: [0.6176, 1.0000)
- r = 0.5275 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B5. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 27
x2 | 25
x3 | 6
x4 | 10

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.8967.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.397; Selected individual = x1

Working:

- Σf = 27 + 25 + 6 + 10 = 68.
- P(x1) = 27/68 = 0.3971.
- x4: [0.0000, 0.1471)
- x3: [0.1471, 0.2353)
- x2: [0.2353, 0.6029)
- x1: [0.6029, 1.0000)
- r = 0.8967 falls in x1's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B6. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 14
x2 | 25
x3 | 6
x4 | 5

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.1002.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.280; Selected individual = x3

Working:

- Σf = 14 + 25 + 6 + 5 = 50.
- P(x1) = 14/50 = 0.2800.
- x4: [0.0000, 0.1000)
- x3: [0.1000, 0.2200)
- x2: [0.2200, 0.7200)
- x1: [0.7200, 1.0000)
- r = 0.1002 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B7. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 7
x2 | 5
x3 | 22
x4 | 28

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.259.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.113; Selected individual = x4

Working:

- Σf = 7 + 5 + 22 + 28 = 62.
- P(x1) = 7/62 = 0.1129.
- x4: [0.0000, 0.4516)
- x3: [0.4516, 0.8065)
- x2: [0.8065, 0.8871)
- x1: [0.8871, 1.0000)
- r = 0.259 falls in x4's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B8. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 26
x2 | 22
x3 | 24
x4 | 26

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.3617.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.265; Selected individual = x3

Working:

- Σf = 26 + 22 + 24 + 26 = 98.
- P(x1) = 26/98 = 0.2653.
- x4: [0.0000, 0.2653)
- x3: [0.2653, 0.5102)
- x2: [0.5102, 0.7347)
- x1: [0.7347, 1.0000)
- r = 0.3617 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B9. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 5
x2 | 17
x3 | 27
x4 | 26

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.5081.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.067; Selected individual = x3

Working:

- Σf = 5 + 17 + 27 + 26 = 75.
- P(x1) = 5/75 = 0.0667.
- x4: [0.0000, 0.3467)
- x3: [0.3467, 0.7067)
- x2: [0.7067, 0.9333)
- x1: [0.9333, 1.0000)
- r = 0.5081 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B10. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 9
x2 | 17
x3 | 18
x4 | 26

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.9076.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.129; Selected individual = x1

Working:

- Σf = 9 + 17 + 18 + 26 = 70.
- P(x1) = 9/70 = 0.1286.
- x4: [0.0000, 0.3714)
- x3: [0.3714, 0.6286)
- x2: [0.6286, 0.8714)
- x1: [0.8714, 1.0000)
- r = 0.9076 falls in x1's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B11. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 21
x2 | 7
x3 | 16
x4 | 29

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.736.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.288; Selected individual = x1

Working:

- Σf = 21 + 7 + 16 + 29 = 73.
- P(x1) = 21/73 = 0.2877.
- x4: [0.0000, 0.3973)
- x3: [0.3973, 0.6164)
- x2: [0.6164, 0.7123)
- x1: [0.7123, 1.0000)
- r = 0.736 falls in x1's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B12. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 7
x2 | 11
x3 | 28
x4 | 29

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.4255.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.093; Selected individual = x3

Working:

- Σf = 7 + 11 + 28 + 29 = 75.
- P(x1) = 7/75 = 0.0933.
- x4: [0.0000, 0.3867)
- x3: [0.3867, 0.7600)
- x2: [0.7600, 0.9067)
- x1: [0.9067, 1.0000)
- r = 0.4255 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B13. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 25
x2 | 21
x3 | 7
x4 | 22

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.4653.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.333; Selected individual = x2

Working:

- Σf = 25 + 21 + 7 + 22 = 75.
- P(x1) = 25/75 = 0.3333.
- x4: [0.0000, 0.2933)
- x3: [0.2933, 0.3867)
- x2: [0.3867, 0.6667)
- x1: [0.6667, 1.0000)
- r = 0.4653 falls in x2's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B14. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 12
x2 | 15
x3 | 21
x4 | 10

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.1201.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.207; Selected individual = x4

Working:

- Σf = 12 + 15 + 21 + 10 = 58.
- P(x1) = 12/58 = 0.2069.
- x4: [0.0000, 0.1724)
- x3: [0.1724, 0.5345)
- x2: [0.5345, 0.7931)
- x1: [0.7931, 1.0000)
- r = 0.1201 falls in x4's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B15. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 5
x2 | 7
x3 | 29
x4 | 30

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.7583.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.070; Selected individual = x3

Working:

- Σf = 5 + 7 + 29 + 30 = 71.
- P(x1) = 5/71 = 0.0704.
- x4: [0.0000, 0.4225)
- x3: [0.4225, 0.8310)
- x2: [0.8310, 0.9296)
- x1: [0.9296, 1.0000)
- r = 0.7583 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B16. [BLANKS · Medium · calculation]

Individual | Fitness
x1 | 26
x2 | 24
x3 | 5
x4 | 12

Roulette-wheel selection (Lecture 3 layout: segments placed in the order x4, x3, x2, x1 starting at 0). Give (1) P(x1) = f1/Σf (3 dp) and (2) which individual is selected by the random number r = 0.2448.

(1) P(x1) = ____
(2) Selected individual = ____

Answer: P(x1) = 0.388; Selected individual = x3

Working:

- Σf = 26 + 24 + 5 + 12 = 67.
- P(x1) = 26/67 = 0.3881.
- x4: [0.0000, 0.1791)
- x3: [0.1791, 0.2537)
- x2: [0.2537, 0.6119)
- x1: [0.6119, 1.0000)
- r = 0.2448 falls in x3's segment.

Takeaway: Roulette: P(i) = f_i / Σf. Fitter individuals get bigger slices, but anyone with f > 0 can be picked.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B17. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 9 at temperature T = 5. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.079 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.1653; Accepted? = Yes

Working:

- p = e^(−9/5) = e^(-1.8000) = 0.1653.
- r = 0.079 < p → accept the worse move.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B18. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 4 at temperature T = 2. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.267 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.1353; Accepted? = No

Working:

- p = e^(−4/2) = e^(-2.0000) = 0.1353.
- r = 0.267 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B19. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 11 at temperature T = 2. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.745 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.0041; Accepted? = No

Working:

- p = e^(−11/2) = e^(-5.5000) = 0.0041.
- r = 0.745 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B20. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 6 at temperature T = 5. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.541 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.3012; Accepted? = No

Working:

- p = e^(−6/5) = e^(-1.2000) = 0.3012.
- r = 0.541 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B21. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 1 at temperature T = 5. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.722 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.8187; Accepted? = Yes

Working:

- p = e^(−1/5) = e^(-0.2000) = 0.8187.
- r = 0.722 < p → accept the worse move.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B22. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 8 at temperature T = 2. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.665 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.0183; Accepted? = No

Working:

- p = e^(−8/2) = e^(-4.0000) = 0.0183.
- r = 0.665 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B23. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 7 at temperature T = 5. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.626 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.2466; Accepted? = No

Working:

- p = e^(−7/5) = e^(-1.4000) = 0.2466.
- r = 0.626 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B24. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 11 at temperature T = 1. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.083 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.0000; Accepted? = No

Working:

- p = e^(−11/1) = e^(-11.0000) = 0.0000.
- r = 0.083 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B25. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 4 at temperature T = 1. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.672 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.0183; Accepted? = No

Working:

- p = e^(−4/1) = e^(-4.0000) = 0.0183.
- r = 0.672 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B26. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 11 at temperature T = 1. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.168 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.0000; Accepted? = No

Working:

- p = e^(−11/1) = e^(-11.0000) = 0.0000.
- r = 0.168 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B27. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 4 at temperature T = 20. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.351 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.8187; Accepted? = Yes

Working:

- p = e^(−4/20) = e^(-0.2000) = 0.8187.
- r = 0.351 < p → accept the worse move.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B28. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 11 at temperature T = 5. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.174 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.1108; Accepted? = No

Working:

- p = e^(−11/5) = e^(-2.2000) = 0.1108.
- r = 0.174 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B29. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 7 at temperature T = 5. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.444 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.2466; Accepted? = No

Working:

- p = e^(−7/5) = e^(-1.4000) = 0.2466.
- r = 0.444 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B30. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 3 at temperature T = 20. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.752 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.8607; Accepted? = Yes

Working:

- p = e^(−3/20) = e^(-0.1500) = 0.8607.
- r = 0.752 < p → accept the worse move.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B31. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 11 at temperature T = 5. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.396 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.1108; Accepted? = No

Working:

- p = e^(−11/5) = e^(-2.2000) = 0.1108.
- r = 0.396 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B32. [BLANKS · Medium · calculation]

Simulated annealing (minimising energy). A neighbour is worse by Δ = 5 at temperature T = 1. Give (1) the acceptance probability p = exp(−Δ/T) (4 dp) and (2) whether the move is accepted if the random number is r = 0.456 (accept if r < p): answer Yes or No.

(1) p = ____
(2) Accepted? = ____

Answer: p = 0.0067; Accepted? = No

Working:

- p = e^(−5/1) = e^(-5.0000) = 0.0067.
- r = 0.456 ≥ p → reject.

Takeaway: High T → p near 1 (explore). Low T → p near 0 (behaves like hill climbing). Better moves are always accepted.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B33. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (8, 8, 8, 2, 8, 5, 3, 8) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 12

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,2) same row; (1,3) same row; (1,5) same row; (1,8) same row; (2,3) same row; (2,5) same row; (2,7) diagonal; (2,8) same row; (3,5) same row; (3,6) diagonal; (3,8) same row; (5,8) same row.
- Cost = 12.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B34. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (8, 3, 4, 7, 3, 3, 4, 4) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 10

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,6) diagonal; (2,3) diagonal; (2,5) same row; (2,6) same row; (3,7) same row; (3,8) same row; (4,7) diagonal; (5,6) same row; (6,7) diagonal; (7,8) same row.
- Cost = 10.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B35. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (1, 2, 4, 1) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 2

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,2) diagonal; (1,4) same row.
- Cost = 2.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B36. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (8, 1, 1, 7, 6, 8, 2, 6) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 7

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,6) same row; (1,7) diagonal; (2,3) same row; (3,8) diagonal; (4,5) diagonal; (5,8) same row; (6,8) diagonal.
- Cost = 7.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B37. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (5, 7, 3, 5, 6, 2, 4, 2) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 6

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,3) diagonal; (1,4) same row; (2,4) diagonal; (4,5) diagonal; (5,7) diagonal; (6,8) same row.
- Cost = 6.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B38. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (2, 1, 1, 4) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 2

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,2) diagonal; (2,3) same row.
- Cost = 2.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B39. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (4, 1, 4, 2) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 1

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,3) same row.
- Cost = 1.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B40. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (1, 4, 1, 4) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 3

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,3) same row; (1,4) diagonal; (2,4) same row.
- Cost = 3.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B41. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (4, 2, 2, 4) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 4

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,3) diagonal; (1,4) same row; (2,3) same row; (2,4) diagonal.
- Cost = 4.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B42. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (1, 7, 7, 5, 1, 6, 2, 5) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 7

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,5) same row; (1,6) diagonal; (2,3) same row; (2,4) diagonal; (2,7) diagonal; (4,7) diagonal; (4,8) same row.
- Cost = 7.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B43. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (4, 1, 4, 1) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 3

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,3) same row; (1,4) diagonal; (2,4) same row.
- Cost = 3.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B44. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (2, 2, 2, 3) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 4

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,2) same row; (1,3) same row; (2,3) same row; (3,4) diagonal.
- Cost = 4.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B45. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (6, 1, 2, 8, 8, 7, 8, 7) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 10

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (2,3) diagonal; (2,8) diagonal; (3,8) diagonal; (4,5) same row; (4,7) same row; (5,6) diagonal; (5,7) same row; (6,7) diagonal; (6,8) same row; (7,8) diagonal.
- Cost = 10.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B46. [BLANK · Hard · calculation]

In the 4-queens formulation (Lecture 3), state x = (3, 3, 1, 2) gives the row of the queen in each column 1..4. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 3

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,2) same row; (1,3) diagonal; (3,4) diagonal.
- Cost = 3.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B47. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (2, 4, 5, 2, 1, 8, 4, 7) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 8

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,4) same row; (2,3) diagonal; (2,4) diagonal; (2,5) diagonal; (2,6) diagonal; (2,7) same row; (3,6) diagonal; (4,5) diagonal.
- Cost = 8.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B48. [BLANK · Hard · calculation]

In the 8-queens formulation (Lecture 3), state x = (7, 4, 5, 8, 8, 7, 1, 5) gives the row of the queen in each column 1..8. The cost is the number of pairs of queens attacking each other (same row or same diagonal). What is the cost?

Answer: 10

Working:

- Check every pair of columns (i, j): attack if $x_i = x_j$ or $|x_i - x_j| = j - i$.
- Attacking pairs: (1,3) diagonal; (1,6) same row; (1,7) diagonal; (2,3) diagonal; (3,7) diagonal; (3,8) same row; (4,5) same row; (5,6) diagonal; (5,8) diagonal; (6,8) diagonal.
- Cost = 10.

Takeaway: Each 8-queens state has 8×7 = 56 successors (move one queen within its column).

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B49. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 70 | 49 | 75 | 18 | 64 | 34

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {E,C,D}, {A,F,D}, {B,F,A}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = C; T2 = A; T3 = A

Working:

- T1: E=64, C=75, D=18 → C.
- T2: A=70, F=34, D=18 → A.
- T3: B=49, F=34, A=70 → A.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B50. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 30 | 76 | 95 | 88 | 81 | 28

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {C,A,E}, {F,D,C}, {E,C,B}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = C; T2 = C; T3 = C

Working:

- T1: C=95, A=30, E=81 → C.
- T2: F=28, D=88, C=95 → C.
- T3: E=81, C=95, B=76 → C.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B51. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 26 | 44 | 48 | 8 | 11 | 23

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {D,F,B}, {C,E,B}, {C,B,D}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = B; T2 = C; T3 = C

Working:

- T1: D=8, F=23, B=44 → B.
- T2: C=48, E=11, B=44 → C.
- T3: C=48, B=44, D=8 → C.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B52. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 76 | 89 | 27 | 59 | 16 | 91

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {B,C,E}, {E,F,D}, {B,E,D}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = B; T2 = F; T3 = B

Working:

- T1: B=89, C=27, E=16 → B.
- T2: E=16, F=91, D=59 → F.
- T3: B=89, E=16, D=59 → B.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B53. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 19 | 10 | 32 | 33 | 31 | 42

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {E,B,F}, {B,F,D}, {B,E,F}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = F; T2 = F; T3 = F

Working:

- T1: E=31, B=10, F=42 → F.
- T2: B=10, F=42, D=33 → F.
- T3: B=10, E=31, F=42 → F.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B54. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 60 | 94 | 56 | 6 | 82 | 49

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {D,A,B}, {C,E,D}, {B,C,F}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = B; T2 = E; T3 = B

Working:

- T1: D=6, A=60, B=94 → B.
- T2: C=56, E=82, D=6 → E.
- T3: B=94, C=56, F=49 → B.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B55. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 66 | 60 | 71 | 69 | 14 | 38

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {F,C,B}, {B,E,D}, {F,A,B}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = C; T2 = D; T3 = A

Working:

- T1: F=38, C=71, B=60 → C.
- T2: B=60, E=14, D=69 → D.
- T3: F=38, A=66, B=60 → A.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B56. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 91 | 82 | 21 | 8 | 51 | 95

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {D,C,F}, {F,B,C}, {E,D,B}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = F; T2 = F; T3 = B

Working:

- T1: D=8, C=21, F=95 → F.
- T2: F=95, B=82, C=21 → F.
- T3: E=51, D=8, B=82 → B.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B57. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 85 | 81 | 10 | 61 | 47 | 62

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {D,A,B}, {D,B,C}, {C,A,E}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = A; T2 = B; T3 = A

Working:

- T1: D=61, A=85, B=81 → A.
- T2: D=61, B=81, C=10 → B.
- T3: C=10, A=85, E=47 → A.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B58. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 67 | 68 | 63 | 29 | 15 | 94

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {C,E,B}, {C,D,B}, {B,A,F}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = B; T2 = B; T3 = F

Working:

- T1: C=63, E=15, B=68 → B.
- T2: C=63, D=29, B=68 → B.
- T3: B=68, A=67, F=94 → F.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B59. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 93 | 41 | 86 | 6 | 68 | 93

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {D,F,E}, {C,F,E}, {A,E,B}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = F; T2 = F; T3 = A

Working:

- T1: D=6, F=93, E=68 → F.
- T2: C=86, F=93, E=68 → F.
- T3: A=93, E=68, B=41 → A.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B60. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 5 | 79 | 77 | 70 | 22 | 58

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {E,F,D}, {B,E,C}, {F,D,A}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = D; T2 = B; T3 = D

Working:

- T1: E=22, F=58, D=70 → D.
- T2: B=79, E=22, C=77 → B.
- T3: F=58, D=70, A=5 → D.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B61. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 23 | 50 | 70 | 31 | 41 | 73

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {F,B,C}, {E,A,D}, {D,F,E}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = F; T2 = E; T3 = F

Working:

- T1: F=73, B=50, C=70 → F.

- T2: E=41, A=23, D=31 → E.
- T3: D=31, F=73, E=41 → F.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B62. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 62 | 47 | 19 | 90 | 75 | 23

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {B,F,A}, {D,F,E}, {B,A,D}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = A; T2 = D; T3 = D

Working:

- T1: B=47, F=23, A=62 → A.
- T2: D=90, F=23, E=75 → D.
- T3: B=47, A=62, D=90 → D.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B63. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 56 | 77 | 6 | 31 | 20 | 74

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {B,F,D}, {F,C,E}, {F,A,D}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = B; T2 = F; T3 = F

Working:

- T1: B=77, F=74, D=31 → B.
- T2: F=74, C=6, E=20 → F.
- T3: F=74, A=56, D=31 → F.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B64. [BLANKS · Easy · calculation]

Individual | A | B | C | D | E | F
Fitness | 63 | 46 | 37 | 69 | 42 | 41

Tournament selection with k = 3 (higher fitness is better). The three tournaments draw {C,A,B}, {B,C,A}, {D,A,F}. Which individual wins each tournament?

(1) T1 = ____
(2) T2 = ____
(3) T3 = ____

Answer: T1 = A; T2 = A; T3 = D

Working:

- T1: C=37, A=63, B=46 → A.
- T2: B=46, C=37, A=63 → A.
- T3: D=69, A=63, F=41 → D.

Takeaway: Tournament selection needs no probabilities; larger k = more selection pressure, less diversity.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B65. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 12. Neighbour values: N1 = 11, N2 = 6, N3 = 10, N4 = 16. What happens next?

A. Move to N4 (value 16)
B. Stop and return the current state (value 12)
C. Move to N2 (value 6)
D. Move to a random neighbour

Answer: A. Move to N4 (value 16)

Working:

- Highest-valued neighbour = 16.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 16 > 12.

Option notes:

- B: A better neighbour exists, so it keeps climbing.
- C: That is the worst neighbour (maximising objective).
- D: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B66. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 8. Neighbour values: N1 = 4, N2 = 4, N3 = 6, N4 = 11. What happens next?

A. Move to N1 (value 4)
B. Move to N4 (value 11)
C. Stop and return the current state (value 8)
D. Move to a random neighbour

Answer: B. Move to N4 (value 11)

Working:

- Highest-valued neighbour = 11.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 11 > 8.

Option notes:

- A: That is the worst neighbour (maximising objective).
- C: A better neighbour exists, so it keeps climbing.
- D: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B67. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 14. Neighbour values: N1 = 11, N2 = 16, N3 = 13, N4 = 13. What happens next?

A. Stop and return the current state (value 14)
B. Move to N1 (value 11)
C. Move to N2 (value 16)
D. Move to a random neighbour

Answer: C. Move to N2 (value 16)

Working:

- Highest-valued neighbour = 16.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 16 > 14.

Option notes:

- A: A better neighbour exists, so it keeps climbing.
- B: That is the worst neighbour (maximising objective).
- D: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B68. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 9. Neighbour values: N1 = 7, N2 = 8, N3 = 11, N4 = 13. What happens next?

A. Move to a random neighbour
B. Move to N1 (value 7)
C. Stop and return the current state (value 9)
D. Move to N4 (value 13)

Answer: D. Move to N4 (value 13)

Working:

- Highest-valued neighbour = 13.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 13 > 9.

Option notes:

- A: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- B: That is the worst neighbour (maximising objective).
- C: A better neighbour exists, so it keeps climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B69. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 5. Neighbour values: N1 = 3, N2 = 6, N3 = 7, N4 = 9. What happens next?

A. Move to N1 (value 3)
B. Move to a random neighbour
C. Move to N4 (value 9)
D. Stop and return the current state (value 5)

Answer: C. Move to N4 (value 9)

Working:

- Highest-valued neighbour = 9.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 9 > 5.

Option notes:

- A: That is the worst neighbour (maximising objective).
- B: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- D: A better neighbour exists, so it keeps climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B70. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 11. Neighbour values: N1 = 8, N2 = 12, N3 = 7, N4 = 9. What happens next?

A. Move to N2 (value 12)
B. Move to a random neighbour
C. Move to N3 (value 7)
D. Stop and return the current state (value 11)

Answer: A. Move to N2 (value 12)

Working:

- Highest-valued neighbour = 12.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 12 > 11.

Option notes:

- B: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- C: That is the worst neighbour (maximising objective).
- D: A better neighbour exists, so it keeps climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B71. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 10. Neighbour values: N1 = 9, N2 = 10, N3 = 14, N4 = 12. What happens next?

A. Move to a random neighbour
B. Stop and return the current state (value 10)
C. Move to N3 (value 14)
D. Move to N1 (value 9)

Answer: C. Move to N3 (value 14)

Working:

- Highest-valued neighbour = 14.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 14 > 10.

Option notes:

- A: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- B: A better neighbour exists, so it keeps climbing.
- D: That is the worst neighbour (maximising objective).

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B72. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 14. Neighbour values: N1 = 9, N2 = 8, N3 = 16, N4 = 12. What happens next?

A. Move to N3 (value 16)
B. Move to N2 (value 8)
C. Move to a random neighbour
D. Stop and return the current state (value 14)

Answer: A. Move to N3 (value 16)

Working:

- Highest-valued neighbour = 16.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 16 > 14.

Option notes:

- B: That is the worst neighbour (maximising objective).
- C: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- D: A better neighbour exists, so it keeps climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B73. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 8. Neighbour values: N1 = 2, N2 = 9, N3 = 5, N4 = 2. What happens next?

A. Move to N1 (value 2)
B. Move to N2 (value 9)
C. Move to a random neighbour
D. Stop and return the current state (value 8)

Answer: B. Move to N2 (value 9)

Working:

- Highest-valued neighbour = 9.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 9 > 8.

Option notes:

- A: That is the worst neighbour (maximising objective).
- C: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- D: A better neighbour exists, so it keeps climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B74. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 14. Neighbour values: N1 = 9, N2 = 9, N3 = 14, N4 = 13. What happens next?

A. Move to a random neighbour
B. Move to N1 (value 9)
C. Move to N3 (value 14)
D. Stop and return the current state (value 14)

Answer: D. Stop and return the current state (value 14)

Working:

- Highest-valued neighbour = 14.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 14 ≤ 14.

Option notes:

- A: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- B: That is the worst neighbour (maximising objective).
- C: Hill climbing only moves if the best neighbour is strictly better.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B75. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 7. Neighbour values: N1 = 9, N2 = 4, N3 = 3, N4 = 5. What happens next?

A. Stop and return the current state (value 7)
B. Move to N3 (value 3)
C. Move to a random neighbour
D. Move to N1 (value 9)

Answer: D. Move to N1 (value 9)

Working:

- Highest-valued neighbour = 9.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 9 > 7.

Option notes:

- A: A better neighbour exists, so it keeps climbing.
- B: That is the worst neighbour (maximising objective).
- C: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B76. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 14. Neighbour values: N1 = 12, N2 = 9, N3 = 14, N4 = 8. What happens next?

A. Stop and return the current state (value 14)
B. Move to N3 (value 14)
C. Move to N4 (value 8)
D. Move to a random neighbour

Answer: A. Stop and return the current state (value 14)

Working:

- Highest-valued neighbour = 14.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 14 ≤ 14.

Option notes:

- B: Hill climbing only moves if the best neighbour is strictly better.
- C: That is the worst neighbour (maximising objective).
- D: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B77. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 10. Neighbour values: N1 = 8, N2 = 8, N3 = 7, N4 = 14. What happens next?

A. Stop and return the current state (value 10)
B. Move to N3 (value 7)
C. Move to N4 (value 14)
D. Move to a random neighbour

Answer: C. Move to N4 (value 14)

Working:

- Highest-valued neighbour = 14.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 14 > 10.

Option notes:

- A: A better neighbour exists, so it keeps climbing.
- B: That is the worst neighbour (maximising objective).
- D: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B78. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 7. Neighbour values: N1 = 9, N2 = 9, N3 = 7, N4 = 3. What happens next?

A. Stop and return the current state (value 7)
B. Move to N4 (value 3)
C. Move to N1 (value 9)
D. Move to a random neighbour

Answer: C. Move to N1 (value 9)

Working:

- Highest-valued neighbour = 9.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 9 > 7.

Option notes:

- A: A better neighbour exists, so it keeps climbing.
- B: That is the worst neighbour (maximising objective).

- D: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B79. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 14. Neighbour values: N1 = 12, N2 = 12, N3 = 9, N4 = 10. What happens next?

A. Move to N3 (value 9)
B. Stop and return the current state (value 14)
C. Move to a random neighbour
D. Move to N1 (value 12)

Answer: B. Stop and return the current state (value 14)

Working:

- Highest-valued neighbour = 12.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 12 ≤ 14.

Option notes:

- A: That is the worst neighbour (maximising objective).
- C: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- D: Hill climbing only moves if the best neighbour is strictly better.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

LOCAL B80. [MCQ · Easy · calculation]

Hill climbing (maximise VALUE). Current state value = 9. Neighbour values: N1 = 4, N2 = 8, N3 = 12, N4 = 9. What happens next?

A. Move to N1 (value 4)
B. Move to a random neighbour
C. Stop and return the current state (value 9)
D. Move to N3 (value 12)

Answer: D. Move to N3 (value 12)

Working:

- Highest-valued neighbour = 12.
- If VALUE(neighbor) ≤ VALUE(current) return current; else move. 12 > 9.

Option notes:

- A: That is the worst neighbour (maximising objective).
- B: That is stochastic behaviour (like SA), not steepest-ascent hill climbing.
- C: A better neighbour exists, so it keeps climbing.

Takeaway: Hill climbing stops at a peak, which may only be a LOCAL maximum.

Source: Lecture 3 (Local search: hill climbing, simulated annealing, genetic algorithm)

✨ Generative AI (Lecture 9)

Part A: hand-written questions (78)

GENAI A1. [MATCH · Easy]

Match each learning type to its goal (Lecture 9).

1. Supervised
2. Unsupervised
3. Generative modelling

A. learn the hidden structure of unlabelled data
B. maximise cumulative reward
C. learn a function mapping x → y
D. learn a model whose samples look like the training distribution

Answer: 1 → C, 2 → A, 3 → D

Working:

- Supervised → learn a function mapping x → y
- Unsupervised → learn the hidden structure of unlabelled data
- Generative modelling → learn a model whose samples look like the training distribution

Takeaway: Generative: P_model ≈ P_data.

Source: Lecture 9, slide 2-4

GENAI A2. [MULTI · Medium]

Why are generative models useful (Lecture 9)? (Select ALL that apply.)

A. Sample generation (language, images, molecules)
B. Guaranteeing perfect classification accuracy
C. Debiasing datasets by uncovering underlying features
D. Outlier detection (e.g. manufacturing defects)

Answer: A. Sample generation (language, images, molecules); C. Debiasing datasets by uncovering underlying features; D. Outlier detection (e.g. manufacturing defects)

Option notes:

- A: Correct:
- B: Incorrect: Not a claim made.
- C: Correct:
- D: Correct:

Takeaway: Three motivations.

Source: Lecture 9, slide 5-7

GENAI A3. [MCQ · Medium]

In the shadow example, what are the latent variables?

A. The observed shadow
B. The camera resolution

C. Object shape, orientation, light direction and light distance
D. The pixel values

Answer: C. Object shape, orientation, light direction and light distance

Option notes:

- A: That's x.
- B: Not a hidden factor here.
- D: Observed.

Takeaway: Hidden factors = z; observed result = x.

Source: Lecture 9, slide 8

GENAI A4. [TF · Easy]

Density estimation and sample generation are the two goals of generative modelling in Lecture 9.

Answer: True

Why: Slide 4.

Takeaway: Learn P_model(x) similar to P_data(x).

Source: Lecture 9 (Generative AI)

GENAI A5. [MCQ · Easy]

What is the autoencoder's training loss?

A. The discriminator loss
B. KL divergence only
C. Cross-entropy with class labels
D. $\|x - \hat{x}\|^2$, comparing the input with its reconstruction

Answer: D. $\|x - \hat{x}\|^2$, comparing the input with its reconstruction

Option notes:

- A: GANs.
- B: That's the VAE regulariser.
- C: No labels used.

Takeaway: Input = target.

Source: Lecture 9, slide 14

GENAI A6. [TF · Medium]

A smaller latent space (e.g. 2D instead of 5D) gives a tighter bottleneck and usually blurrier reconstructions.

Answer: True

Why: Slide 15: autoencoding is compression.

Takeaway: Bottleneck size trades compression vs quality.

Source: Lecture 9, slide 15

GENAI A7. [MCQ · Medium]

In a denoising autoencoder, the reconstruction is compared with…

A. the clean image x
B. random noise

C. the noisy input $\tilde{x}$
D. the latent z

Answer: A. the clean image x

Option notes:

- B: No.
- C: That would learn to reproduce noise.
- D: Different spaces.

Takeaway: Noisy in, clean target.

Source: Lecture 9, slide 16

GENAI A8. [MULTI · Medium]

Select ALL applications/limitations of standard autoencoders (Lecture 9). (Select ALL that apply.)

A. Compact representations for visualisation
B. Denoising
C. Excellent generation from randomly sampled codes
D. Anomaly detection via high reconstruction error

Answer: A. Compact representations for visualisation; B. Denoising; D. Anomaly detection via high reconstruction error

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: That's their LIMITATION.
- D: Correct:

Takeaway: AEs aren't good generators.

Source: Lecture 9, slide 17

GENAI A9. [MCQ · Medium]

What is the key difference between a VAE and a traditional autoencoder?

A. The VAE has no decoder
B. The VAE uses class labels
C. The VAE uses a discriminator
D. The encoder outputs a mean μ and standard deviation σ, and z is sampled from them

Answer: D. The encoder outputs a mean μ and standard deviation σ, and z is sampled from them

Option notes:

- A: It has one.
- B: Still unsupervised.
- C: That's a GAN.

Takeaway: Probabilistic twist.

Source: Lecture 9, slide 20-21

GENAI A10. [BLANK · Hard · calculation]

KL term for one latent dimension with $\mu = 0.5$, $\sigma = 1$: $\frac{1}{2}(\mu^2 + \sigma^2 - 1 - \ln \sigma^2) =$ _____.

Answer: 0.125

Takeaway: ½(0.25 + 1 − 1 − 0).

Source: Lecture 9 (Generative AI)

GENAI A11. [TF · Medium]

Regularising with the normal prior encourages means toward 0 and variances toward 1, which gives continuity and completeness.

Answer: True

Why: Slides 28-30.

Takeaway: Close points decode similarly; samples decode meaningfully.

Source: Lecture 9 (Generative AI)

GENAI A12. [MCQ · Medium]

Why is the reparameterisation trick needed?

A. To remove the KL term
B. Gradients cannot be backpropagated through a random sampling node
C. To speed up the forward pass
D. To make the decoder deterministic

Answer: B. Gradients cannot be backpropagated through a random sampling node

Option notes:

- A: KL stays.
- C: Not the purpose.
- D: Decoder is already deterministic.

Takeaway: $z = \mu + \sigma \odot \varepsilon$ moves randomness to ε.

Source: Lecture 9, slide 31-34

GENAI A13. [MCQ · Hard]

In a β-VAE with $\beta > 1$, what happens?

A. The model becomes a GAN
B. Latents become more entangled
C. Reconstruction is weighted more heavily
D. The latent bottleneck is constrained more, encouraging disentangled factors

Answer: D. The latent bottleneck is constrained more, encouraging disentangled factors

Option notes:

- A: No.
- B: Opposite.
- C: β weights the KL term.

Takeaway: β = 250: head rotation without smile change.

Source: Lecture 9, slide 36

GENAI A14. [MATCH · Medium]

Match each VAE loss term to its role.

1. Reconstruction loss

2. KL regulariser

A. keep q(z|x) close to the prior N(0, I)
B. make D output 0.5
C. make $\hat{x}$ close to x (e.g. $\|x - \hat{x}\|^2$)

Answer: 1 → C, 2 → A

Working:

- Reconstruction loss → make $\hat{x}$ close to x (e.g. $\|x - \hat{x}\|^2$)
- KL regulariser → keep q(z|x) close to the prior N(0, I)

Takeaway: Two terms.

Source: Lecture 9, slide 22-27

GENAI A15. [MCQ · Easy]

In a GAN, what does the discriminator learn?

A. To distinguish real training samples from generated samples
B. To encode images into μ and σ
C. To predict the next token
D. To generate new samples from noise

Answer: A. To distinguish real training samples from generated samples

Option notes:

- B: That's a VAE encoder.
- C: Autoregressive.
- D: That's the generator.

Takeaway: D outputs a realness score.

Source: Lecture 9, slide 40

GENAI A16. [TF · Medium]

At the ideal GAN equilibrium, the discriminator outputs D(x) ≈ 0.5 because generated and real distributions match.

Answer: True

Why: Slide 45.

Takeaway: Neither source is more likely.

Source: Lecture 9 (Generative AI)

GENAI A17. [MCQ · Medium]

The generator learns one output that fools D very well and ignores the other modes of the data. This is…

A. posterior collapse
B. vanishing gradient
C. mode collapse
D. overfitting

Answer: C. mode collapse

Option notes:

- A: VAE term.
- B: Different issue.

- D: Different concept.

Takeaway: Matches one region, misses diversity.

Source: Lecture 9, slide 46

GENAI A18. [MULTI · Hard]

Select ALL true statements about GAN training (Lecture 9). (Select ALL that apply.)

A. D maximises E[log D(x)] + E[log(1 − D(G(z)))]
B. Training alternates between D and G
C. After training, the generator alone creates new data
D. G minimises the same expression; the real term is constant w.r.t. G
E. G is trained on real images directly

Answer: A. D maximises E[log D(x)] + E[log(1 − D(G(z)))]; B. Training alternates between D and G; C. After training, the generator alone creates new data; D. G minimises the same expression; the real term is constant w.r.t. G

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Correct:
- E: Incorrect: It learns via D.

Takeaway: Adversarial min-max.

Source: Lecture 9, slide 44-51

GENAI A19. [BLANK · Medium · calculation]

D(x) = 0.5 and D(G(z)) = 0.5: log D(x) + log(1 − D(G(z))) = 2 ln 0.5 = _____ (3 dp).

Answer: -1.386
(Also accepted: −1.386)

Takeaway: ln 0.5 = −0.693.

Source: Lecture 9 (Generative AI)

GENAI A20. [MCQ · Easy]

How does an autoregressive model generate a sentence?

A. Denoise random noise
B. Classify the whole sentence
C. Predict a distribution for the next token, sample it, append it, repeat
D. Decode a single latent vector in one step

Answer: C. Predict a distribution for the next token, sample it, append it, repeat

Option notes:

- A: Diffusion.
- B: Not generation.
- D: That's VAE/GAN style.

Takeaway: Predict, sample, append, repeat.

Source: Lecture 9, slide 58

GENAI A21. [TF · Medium]

With causal masking, autoregressive models generate all tokens of a new sentence in one parallel step.

Answer: False

Why: TRAINING is parallel across positions; GENERATION is sequential, token by token.

Takeaway: Mask = no peeking at the future.

Source: Lecture 9, slide 60

GENAI A22. [MATCH · Medium]

Match each domain to its autoregressive model (Lecture 9 table).

1. Language
2. Images
3. Audio
4. DNA

A. Evo (nucleotides)
B. PixelCNN (pixels)
C. GPT-style models (tokens)
D. YOLO (boxes)
E. WaveNet (audio samples)

Answer: 1 → C, 2 → B, 3 → E, 4 → A

Working:

- Language → GPT-style models (tokens)
- Images → PixelCNN (pixels)
- Audio → WaveNet (audio samples)
- DNA → Evo (nucleotides)

Takeaway: Same idea, different elements.

Source: Lecture 9, slide 62

GENAI A23. [MCQ · Medium]

In FVSBN, which inputs does the output unit $\hat{x}_4$ receive?

A. Only the earlier visible variables x_1, x_2, x_3
B. Only x_3
C. A latent vector z
D. All four variables including x_4

Answer: A. Only the earlier visible variables x_1, x_2, x_3

Option notes:

- B: All earlier ones.
- C: Fully visible: no latents.
- D: It can't see itself.

Takeaway: $\hat{x}_4 = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$.

Source: Lecture 9, slide 59

GENAI A24. [BLANK · Medium · calculation]

FVSBN: $b_4 = -1$, $w_{41} = 2$, $w_{42} = 0.5$, $w_{43} = -1$, x = (1, 0, 1). The input to the sigmoid is _____.

Answer: 0
(Also accepted: 0.0)

Takeaway: −1 + 2 + 0 − 1.

Source: Lecture 9 (Generative AI)

GENAI A25. [MCQ · Medium]

What does the diffusion denoising network $\varepsilon_\theta(x_t, t)$ learn to predict during training?

A. The noise ε that was added to produce x_t
B. Whether x_t is real or fake
C. The class label
D. The latent μ and σ

Answer: A. The noise ε that was added to produce x_t

Option notes:

- B: GAN discriminator.
- C: No labels.
- D: VAE.

Takeaway: MSE loss $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9, slide 65

GENAI A26. [TF · Medium]

In the forward process, as t approaches T, x_t approaches pure Gaussian noise N(0, I).

Answer: True

Why: Slide 64.

Takeaway: Clear image → nearly pure noise.

Source: Lecture 9 (Generative AI)

GENAI A27. [MULTI · Medium]

Select ALL true statements about text-guided latent diffusion. (Select ALL that apply.)

A. It decodes to an image once at the end
B. It needs no noise at all
C. A text encoder conditions each denoising step
D. It denoises in a compact latent space

Answer: A. It decodes to an image once at the end; C. A text encoder conditions each denoising step; D. It denoises in a compact latent space

Option notes:

- A: Correct:
- B: Incorrect: Starts from random latent noise.
- C: Correct:
- D: Correct:

Takeaway: Prompt steers the denoising.

Source: Lecture 9, slide 68

GENAI A28. [MATCH · Medium]

Lecture 9 'Four ways to generate new data': match each model to its process.

1. VAE
2. GAN
3. Autoregressive
4. Diffusion

A. noise z → generator → image
B. latent z → decoder → image
C. image → classifier → label
D. earlier elements → next element → repeat
E. Gaussian noise → denoise → repeat → image

Answer: 1 → B, 2 → A, 3 → D, 4 → E

Working:

- VAE → latent z → decoder → image
- GAN → noise z → generator → image
- Autoregressive → earlier elements → next element → repeat
- Diffusion → Gaussian noise → denoise → repeat → image

Takeaway: Slide 70.

Source: Lecture 9, slide 70

GENAI A29. [MCQ · Medium]

In ControlNet (Lecture 9), what does the pose map control?

A. The noise schedule
B. The model's training data
C. The colour palette only
D. Where things appear (spatial structure), while text describes the content

Answer: D. Where things appear (spatial structure), while text describes the content

Option notes:

- A: No.
- B: No.
- C: No.

Takeaway: Same spatial guide, varied prompts.

Source: Lecture 9, slide 69

GENAI A30. [BLANK · Hard · calculation]

$x_t = \sqrt{\bar{\alpha}}\, x_0 + \sqrt{(1 - \bar{\alpha})}\, \varepsilon$ with $\bar{\alpha} = 0.64$, $x_0 = 1$, $\varepsilon = 0.5$. x_t = _____.

Answer: 1.1
(Also accepted: 1.10)

Takeaway: 0.8×1 + 0.6×0.5.

Source: Lecture 9 (Generative AI)

GENAI A31. [BLANK · Medium]

Generative modelling aims to learn P_model(x) similar to P_____(x).

Answer: data
(Also accepted: Data)

Takeaway: Lecture 9 slide 4.

Source: Lecture 9 (Generative AI)

GENAI A32. [MATCH · Easy]

[Diagram as text] Edges (label in brackets): Arad — Sibiu (140); Arad — Timișoara (118); Arad — Zerind (75); Zerind — Oradea (71); Oradea — Sibiu (151); Sibiu — Făgăraș (99); Sibiu — Râmnicu V. (80); Râmnicu V. — Pitești (97); Râmnicu V. — Craiova (146); Pitești — Craiova (138); Pitești — Bucharest (101); Făgăraș — Bucharest (211); Timișoara — Lugoj (111); Lugoj — Mehadia (70); Mehadia — Drobeta (75); Drobeta — Craiova (120); Bucharest — Giurgiu (90); Bucharest — Urziceni (85); Urziceni — Hirsova (98); Hirsova — Eforie (86); Urziceni — Vaslui (142); Vaslui — Iasi (92); Iasi — Neamt (87)
Heuristic values: Arad h=366, Bucharest h=0, Craiova h=160, Drobeta h=242, Eforie h=161, Făgăraș h=176, Giurgiu h=77, Hirsova h=151, Iasi h=226, Lugoj h=244, Mehadia h=241, Neamt h=234, Oradea h=380, Pitești h=100, Râmnicu V. h=193, Sibiu h=253, Timișoara h=329, Urziceni h=80, Vaslui h=199, Zerind h=374
Romania (Lecture 2). Blue numbers = road costs; h = straight-line distance to Bucharest.

Match the learning paradigm.

1. supervised
2. unsupervised
3. generative modelling

A. learn hidden structure from unlabelled data
B. learn a model of the data distribution to generate samples
C. learn x → y from labelled pairs

Answer: 1 → C, 2 → A, 3 → B

Working:

- supervised → learn x → y from labelled pairs
- unsupervised → learn hidden structure from unlabelled data
- generative modelling → learn a model of the data distribution to generate samples

Takeaway: Three paradigms.

Source: Book 2 (Exam examples), Q283, p. 38

GENAI A33. [MULTI · Easy]

Uses of generative models (slides): (Select ALL that apply.)

A. debiasing datasets
B. computing gradients exactly
C. sample generation
D. outlier detection

Answer: A. debiasing datasets; C. sample generation; D. outlier detection

Option notes:

- A: Correct:
- B: Incorrect: not a use of generative models

- C: Correct:
- D: Correct:

Takeaway: Generate, detect outliers, debias.

Source: Book 2 (Exam examples), Q284, p. 38

GENAI A34. [BLANKS · Easy]

Hidden factors that explain an observation are called ___ variables, usually written ___.

(1) name = ____
(2) symbol = ____

Answer: name = latent; symbol = z

Takeaway: Latent z.

Source: Book 2 (Exam examples), Q285, p. 38

GENAI A35. [MCQ · Easy]

An autoencoder's encoder maps x to ___, its decoder maps it back to ___, and the loss is ___.

A. noise ε; a fake sample; the GAN loss
B. a low-dimensional latent z; a reconstruction $\hat{x}$; $\|x - \hat{x}\|^2$
C. a bigger vector; the class; accuracy
D. a label y; the input x; cross-entropy

Answer: B. a low-dimensional latent z; a reconstruction $\hat{x}$; $\|x - \hat{x}\|^2$

Option notes:

- A: That is a GAN.
- C: The code is smaller, not bigger.
- D: Autoencoders need no labels.

Takeaway: Compress and reconstruct.

Source: Book 2 (Exam examples), Q286, p. 38

GENAI A36. [TF · Easy]

Autoencoder training needs labels.

Answer: False

Why: The input is its own target.

Takeaway: Unsupervised.

Source: Book 2 (Exam examples), Q287, p. 38

GENAI A37. [TF · Medium]

A smaller latent space gives a tighter bottleneck and usually blurrier reconstructions.

Answer: True

Why: More compression, less detail.

Takeaway: Latent size sets quality.

Source: Book 2 (Exam examples), Q288, p. 38

GENAI A38. [BLANKS · Easy]

A denoising autoencoder takes a ___ input and compares its output with the ___ image.

(1) input = ____
(2) target = ____

Answer: input = noisy; target = clean

Takeaway: Noise in, clean out.

Source: Book 2 (Exam examples), Q289, p. 38

GENAI A39. [MCQ · Medium]

The main limitation of a standard autoencoder for generation is that:

A. it has no decoder
B. it needs labels
C. it cannot compress
D. it is not suited to generating new samples from random latent codes

Answer: D. it is not suited to generating new samples from random latent codes

Option notes:

- A: It has one.
- B: It does not.
- C: Compression is its job.

Takeaway: Latent space has gaps → VAE.

Source: Book 2 (Exam examples), Q290, p. 38

GENAI A40. [BLANKS · Easy]

A VAE encoder outputs a ___ vector and a ___ vector.

(1) first = ____
(2) second = ____

Answer: first = mean; second = standard deviation

Takeaway: μ and σ.

Source: Book 2 (Exam examples), Q291, p. 38

GENAI A41. [MCQ · Medium]

VAE loss = ___ loss + ___ term $D_{KL}(q(z|x) \| p(z))$, with prior $p(z)$ = ___.

A. classification; dropout; uniform
B. reconstruction; regularisation; N(0, I)
C. adversarial; reconstruction; $N(\mu, \sigma^2)$
D. regularisation; reconstruction; N(1, 0)

Answer: B. reconstruction; regularisation; N(0, I)

Option notes:

- A: Wrong terms.
- C: That mixes GAN ideas; the prior is standard normal.
- D: Order and prior wrong.

Takeaway: Reconstruct + stay near N(0, I).

Source: Book 2 (Exam examples), Q292, p. 39

GENAI A42. [MATCH · Medium]

Match the latent-space property.

1. continuity
2. completeness

A. nearby latent points decode to similar content
B. sampling from latent space gives meaningful content

Answer: 1 → A, 2 → B

Working:

- continuity → nearby latent points decode to similar content
- completeness → sampling from latent space gives meaningful content

Takeaway: What regularisation buys.

Source: Book 2 (Exam examples), Q293, p. 39

GENAI A43. [MCQ · Easy]

The reparameterisation trick writes z as:

A. $\sigma + \mu\varepsilon$
B. a deterministic μ only
C. $\mu \cdot \sigma$
D. $\mu + \sigma \odot \varepsilon$, $\varepsilon \sim N(0, I)$

Answer: D. $\mu + \sigma \odot \varepsilon$, $\varepsilon \sim N(0, I)$

Option notes:

- A: Swapped roles.
- B: Then it is not a sample.
- C: No noise, wrong form.

Takeaway: So gradients flow through sampling.

Source: Book 2 (Exam examples), Q294, p. 39

GENAI A44. [BLANK · Medium]

In a β-VAE, $\beta > 1$ encourages ___ of the latent factors.

Answer: disentanglement
(Also accepted: disentangling)

Takeaway: One factor per dimension.

Source: Book 2 (Exam examples), Q295, p. 39

GENAI A45. [MATCH · Easy]

Match the GAN part.

1. generator
2. discriminator

A. transforms noise into samples
B. outputs a realness score for real vs fake

Answer: 1 → A, 2 → B

Working:

- generator → transforms noise into samples
- discriminator → outputs a realness score for real vs fake

Takeaway: Forger vs detective.

Source: Book 2 (Exam examples), Q296, p. 39

GENAI A46. [BLANK · Medium]

At the ideal GAN equilibrium p_G = p_data and D(x) = ___ everywhere.

Answer: 0.5
(Also accepted: 1/2)

Takeaway: Cannot tell real from fake.

Source: Book 2 (Exam examples), Q297, p. 39

GENAI A47. [BLANK · Easy]

When a generator produces only one kind of output that fools D it is called ___.

Answer: mode collapse

Takeaway: Misses diversity.

Source: Book 2 (Exam examples), Q298, p. 39

GENAI A48. [MCQ · Medium]

In min_G max_D E[log D(x)] + E[log(1 − D(G(z)))], the discriminator tries to:

A. minimise it
B. keep it at 0
C. ignore the second term
D. maximise it

Answer: D. maximise it

Option notes:

- A: That is the generator.
- B: No.
- C: D uses both terms.

Takeaway: D max, G min.

Source: Book 2 (Exam examples), Q299, p. 39

GENAI A49. [TF · Easy]

After training, both G and D are used to generate images.

Answer: False

Why: Only G generates.

Takeaway: Keep G, drop D.

Source: Book 2 (Exam examples), Q300, p. 39

GENAI A50. [MCQ · Medium]

Autoregressive models factorise p(x1, ..., xn) = Π_i p(x_i | ___).

A. z
B. x1, ..., x_(i−1)
C. all other x_j
D. x_(i+1), ..., xn

Answer: B. x1, ..., x_(i−1)

Option notes:

- A: That is a latent-variable model.
- C: Only earlier elements.
- D: Future elements are not seen.

Takeaway: Chain rule.

Source: Book 2 (Exam examples), Q301, p. 39

GENAI A51. [TF · Medium]

With causal masking, training predictions for all positions can be computed in parallel, but generation is still sequential.

Answer: True

Why: Mask lets one forward pass train every position.

Takeaway: Parallel training, sequential sampling.

Source: Book 2 (Exam examples), Q302, p. 39

GENAI A52. [MATCH · Medium]

Match the autoregressive model to what it generates.

1. GPT-style
2. PixelCNN
3. WaveNet
4. Evo

A. tokens
B. pixels
C. nucleotides
D. audio samples

Answer: 1 → A, 2 → B, 3 → D, 4 → C

Working:

- GPT-style → tokens
- PixelCNN → pixels
- WaveNet → audio samples
- Evo → nucleotides

Takeaway: Same idea, different modalities.

Source: Book 2 (Exam examples), Q303, p. 39

GENAI A53. [BLANKS · Medium]

Diffusion training: the network ε_θ(x_t, t) predicts the ***, with loss*** ‖ − ε‖².

(1) predicts = ____
(2) loss term = ____

Answer: predicts = noise; loss term = ε̂

Takeaway: Noise prediction.

Source: Book 2 (Exam examples), Q304, p. 39

GENAI A54. [MCQ · Medium]

Latent diffusion (Stable Diffusion) is cheaper because:

A. it skips text conditioning
B. it denoises a compact latent representation and decodes once
C. it uses one step
D. it uses no neural network

Answer: B. it denoises a compact latent representation and decodes once

Option notes:

- A: It uses text conditioning.
- C: Still many steps.
- D: It uses a U-Net.

Takeaway: Work small, decode once.

Source: Book 2 (Exam examples), Q305, p. 39

GENAI A55. [MATCH · Medium]

Match the generative model.

1. VAE
2. GAN
3. autoregressive
4. diffusion

A. generator vs discriminator game
B. encoder outputs μ, σ; KL regulariser
C. predict, sample, append, one element at a time
D. repeatedly denoise from pure noise

Answer: 1 → B, 2 → A, 3 → C, 4 → D

Working:

- VAE → encoder outputs μ, σ; KL regulariser
- GAN → generator vs discriminator game
- autoregressive → predict, sample, append, one element at a time
- diffusion → repeatedly denoise from pure noise

Takeaway: Four families.

Source: Book 2 (Exam examples), Q306, p. 40

GENAI A56. [BLANK · Easy · calculation]

Reparameterisation: $\mu = 2$, $\sigma = 0.5$, sampled $\varepsilon = -1.2$. z = ___.

Answer: 1.4

Takeaway: $\mu + \sigma\varepsilon$.

Source: Book 2 (Exam examples), Q307, p. 40

GENAI A57. [BLANK · Medium · calculation]

KL term for one latent dimension with $\mu = 1$, $\sigma = 1$: $\frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$ = ___.

Answer: 0.5

Takeaway: $\frac{1}{2}(1 + 1 - 1 - 0)$.

Source: Book 2 (Exam examples), Q308, p. 40

GENAI A58. [BLANK · Easy · calculation]

KL term with $\mu = 0$, $\sigma = 1$: ___.

Answer: 0

Takeaway: Identical to the prior.

Source: Book 2 (Exam examples), Q309, p. 40

GENAI A59. [BLANK · Hard · calculation]

KL term with $\mu = 0$, $\sigma = 2$ (natural log): ___ (3 d.p.).

Answer: 0.807

Working:

- $\frac{1}{2}(0 + 4 - 1 - 1.386) = 0.807$

Takeaway: $\frac{1}{2}(4 - 1 - \ln 4)$.

Source: Book 2 (Exam examples), Q310, p. 40

GENAI A60. [BLANK · Medium · calculation]

Discriminator outputs $D(x) = 0.8$ on a real image and $D(G(z)) = 0.3$ on a fake. $\log D(x) + \log(1 - D(G(z)))$ = ___ (natural log, 3 d.p.).

Answer: -0.580

Working:

- $-0.223 - 0.357 = -0.580$

Takeaway: $\ln 0.8 + \ln 0.7$.

Source: Book 2 (Exam examples), Q311, p. 40

GENAI A61. [BLANK · Medium · calculation]

At equilibrium ($D = 0.5$ for both), the same quantity is ___ (3 d.p.).

Answer: -1.386

Takeaway: $2 \ln 0.5$.

Source: Book 2 (Exam examples), Q312, p. 40

GENAI A62. [BLANK · Medium · calculation]

Diffusion forward step: $\bar{\alpha}_t = 0.36$, $x_0 = 2$, $\varepsilon = 1$. $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1 - \bar{\alpha}_t)}\, \varepsilon$ = ___.

Answer: 2.0
(Also accepted: 2)

Takeaway: 0.6(2) + 0.8(1).

Source: Book 2 (Exam examples), Q313, p. 40

GENAI A63. [BLANK · Easy · calculation]

Autoencoder input $x = (1, 0, 2)$, reconstruction $\hat{x} = (0.5, 0, 1.5)$. Loss $\|x - \hat{x}\|^2$ = ___.

Answer: 0.5

Takeaway: 0.25 + 0 + 0.25.

Source: Book 2 (Exam examples), Q314, p. 40

GENAI A64. [MCQ · Medium]

The reparameterisation trick is needed in VAEs because:

A. KL is not differentiable
B. labels are missing
C. the decoder has no gradient
D. we cannot backpropagate through random sampling

Answer: D. we cannot backpropagate through random sampling

Option notes:

- A: It is.
- B: VAEs need no labels.
- C: It does.

Takeaway: $z = \mu + \sigma\varepsilon$.

Source: Book 2 (Exam examples), Mixed practice paper Q28, p. 55

GENAI A65. [BLANK · Medium · calculation]

KL term for one latent dimension with $\mu = 0.5$, $\sigma = 1$: ___.

Answer: 0.125

Takeaway: ½(0.25 + 1 − 1 − 0).

Source: Book 2 (Exam examples), Mixed practice paper Q29, p. 55

GENAI A66. [TF · Medium]

In a GAN, the discriminator is trained to maximise $\log D(x) + \log(1 - D(G(z)))$.

Answer: True

Why: D maximises, G minimises.

Takeaway: Minimax.

Source: Book 2 (Exam examples), Mixed practice paper Q30, p. 55

GENAI A67. [BLANK · Easy · calculation]

KL for μ = 0, σ = 1 = ___.

Answer: 0

Takeaway: Matches the prior.

Source: Book 3 (AI from zero), Practice 24.1, p. 98

GENAI A68. [BLANK · Easy · calculation]

Reparameterise: μ = −1, σ = 2, ε = 0.5. z = ___.

Answer: 0

Takeaway: −1 + 1.

Source: Book 3 (AI from zero), Practice 24.2, p. 98

GENAI A69. [BLANK · Medium · calculation]

D(real) = 0.9, D(fake) = 0.1. Objective log D(x) + log(1 − D(G(z))) = ___ (3 d.p.).

Answer: -0.211

Takeaway: 2 ln 0.9.

Source: Book 3 (AI from zero), Practice 24.3, p. 98

GENAI A70. [BLANK · Medium · calculation]

ᾱ_t = 0.64, x0 = 1, ε = −1. x_t = ___.

Answer: 0.2

Takeaway: 0.8 − 0.6.

Source: Book 3 (AI from zero), Practice 24.4, p. 98

GENAI A71. [MCQ · Easy]

Which model can't easily generate from random codes?

A. VAE
B. diffusion
C. GAN
D. the plain autoencoder

Answer: D. the plain autoencoder

Option notes:

- A: Regularised latent.
- B: Starts from noise.
- C: Samples noise.

Takeaway: Gaps in the latent space.

Source: Book 3 (AI from zero), Practice 24.5, p. 98

GENAI A72. [BLANK · Easy · calculation]

x = (1, 0, 1), x̂ = (0.9, 0.2, 0.7): loss ‖x − x̂‖² = ___.

Answer: 0.14

Takeaway: 0.01 + 0.04 + 0.09.

Source: Book 3 (AI from zero), Chapter 24, p. 95

GENAI A73. [BLANK · Medium · calculation]

KL for two dimensions μ = (0.5, −0.5), σ = (1, 1) = ___.

Answer: 0.25

Takeaway: ½(0.25 + 0.25).

Source: Book 3 (AI from zero), Chapter 24, p. 95

GENAI A74. [BLANK · Medium · calculation]

p(May) = 0.1, p(the|May) = 0.5, p(Force|May the) = 0.4. p(May the Force) = ___.

Answer: 0.02

Takeaway: Chain rule product.

Source: Book 3 (AI from zero), Chapter 24, p. 95

GENAI A75. [MATCH · Hard]

Match the model to its weakness.

1. VAE
2. GAN
3. autoregressive
4. diffusion

A. slow sequential generation
B. unstable training, mode collapse
C. blurry samples
D. many denoising steps

Answer: 1 → C, 2 → B, 3 → A, 4 → D

Working:

- VAE → blurry samples
- GAN → unstable training, mode collapse
- autoregressive → slow sequential generation
- diffusion → many denoising steps

Takeaway: Strengths: smooth latent, sharp, exact likelihood, quality.

Source: Book 3 (AI from zero), Chapter 24, p. 95

GENAI A76. [TF · Medium]

In the diffusion step with $\bar{\alpha}_t = 0.36$ the signal and noise weights are 0.6 and 0.8, and $0.6^2 + 0.8^2 = 1$.

Answer: True

Why: $\sqrt{\bar{\alpha}}$ and $\sqrt{(1 - \bar{\alpha})}$.

Takeaway: Variance preserved.

Source: Book 3 (AI from zero), Chapter 24, p. 95

GENAI A77. [MATCH · Medium]

Match AIMA's unsupervised/generative deep models.

1. autoencoder
2. variational autoencoder
3. GAN

A. encodes to a distribution and regularises it toward a prior
B. k-means
C. learns to reconstruct its input through a narrow code
D. generator and discriminator trained against each other

Answer: 1 → C, 2 → A, 3 → D

Working:

- autoencoder → learns to reconstruct its input through a narrow code
- variational autoencoder → encodes to a distribution and regularises it toward a prior
- GAN → generator and discriminator trained against each other

Takeaway: Three model families.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: unsupervised learning

GENAI A78. [TF · Medium]

(Extra, not in your slides) GANs give an explicit probability for each generated sample.

Answer: False

Why: They sample without an explicit density; evaluating them is hard.

Takeaway: Implicit models.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 21: GANs

Part B: calculation drills (96)

GENAI B1. [BLANK · Hard · calculation]

A VAE encoder outputs $\mu = [0, -0.8]$ and $\sigma = [1, 0.8]$. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.363

Working:

- $j=0$: $0^2 + 1^2 - 1 - \ln(1^2) = 0.0000$
- $j=1$: $-0.8^2 + 0.8^2 - 1 - \ln(0.8^2) = 0.7263$
- KL = ½ × sum = 0.3631.

Takeaway: KL is 0 only when $\mu = 0$ and $\sigma = 1$: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B2. [BLANK · Hard · calculation]

A VAE encoder outputs $\mu = [-0.6]$ and $\sigma = [1]$. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.180

Working:

- j=0: $-0.6^2 + 1^2 - 1 - \ln(1^2) = 0.3600$
- KL = ½ × sum = 0.1800.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B3. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.4, 0.5] and σ = [1, 1]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.205

Working:

- j=0: $-0.4^2 + 1^2 - 1 - \ln(1^2) = 0.1600$
- j=1: $0.5^2 + 1^2 - 1 - \ln(1^2) = 0.2500$
- KL = ½ × sum = 0.2050.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B4. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.2] and σ = [2]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.827

Working:

- j=0: $-0.2^2 + 2^2 - 1 - \ln(2^2) = 1.6537$
- KL = ½ × sum = 0.8269.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B5. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.1] and σ = [2]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.812

Working:

- j=0: $-0.1^2 + 2^2 - 1 - \ln(2^2) = 1.6237$
- KL = ½ × sum = 0.8119.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B6. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.4, 0.3] and σ = [1, 1]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.125

Working:

- j=0: $-0.4^2 + 1^2 - 1 - \ln(1^2) = 0.1600$

- j=1: $0.3^2 + 1^2 - 1 - \ln(1^2) = 0.0900$
- KL = ½ × sum = 0.1250.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B7. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [0, 0.3] and σ = [0.8, 2]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.895

Working:

- j=0: $0^2 + 0.8^2 - 1 - \ln(0.8^2) = 0.0863$
- j=1: $0.3^2 + 2^2 - 1 - \ln(2^2) = 1.7037$
- KL = ½ × sum = 0.8950.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B8. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.8, -0.8] and σ = [0.8, 1]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.683

Working:

- j=0: $-0.8^2 + 0.8^2 - 1 - \ln(0.8^2) = 0.7263$
- j=1: $-0.8^2 + 1^2 - 1 - \ln(1^2) = 0.6400$
- KL = ½ × sum = 0.6831.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B9. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.8] and σ = [2]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 1.127

Working:

- j=0: $-0.8^2 + 2^2 - 1 - \ln(2^2) = 2.2537$
- KL = ½ × sum = 1.1269.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B10. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.8, -0.7] and σ = [2, 1]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 1.372

Working:

- j=0: $-0.8^2 + 2^2 - 1 - \ln(2^2) = 2.2537$
- j=1: $-0.7^2 + 1^2 - 1 - \ln(1^2) = 0.4900$
- KL = ½ × sum = 1.3719.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B11. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [0.5] and σ = [1]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.125

Working:

- j=0: $0.5^2 + 1^2 - 1 - \ln(1^2) = 0.2500$
- KL = ½ × sum = 0.1250.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B12. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.1, -0.7] and σ = [2, 0.5]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 1.375

Working:

- j=0: $-0.1^2 + 2^2 - 1 - \ln(2^2) = 1.6237$
- j=1: $-0.7^2 + 0.5^2 - 1 - \ln(0.5^2) = 1.1263$
- KL = ½ × sum = 1.3750.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B13. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.3, -0.1] and σ = [1, 0.8]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.093

Working:

- j=0: $-0.3^2 + 1^2 - 1 - \ln(1^2) = 0.0900$
- j=1: $-0.1^2 + 0.8^2 - 1 - \ln(0.8^2) = 0.0963$
- KL = ½ × sum = 0.0931.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B14. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [0.6] and σ = [0.8]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.223

Working:

- j=0: $0.6^2 + 0.8^2 - 1 - \ln(0.8^2) = 0.4463$
- KL = ½ × sum = 0.2231.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B15. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.1, -0.2] and σ = [0.5, 1]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.343

Working:

- j=0: $-0.1^2 + 0.5^2 - 1 - \ln(0.5^2) = 0.6463$
- j=1: $-0.2^2 + 1^2 - 1 - \ln(1^2) = 0.0400$
- KL = ½ × sum = 0.3431.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B16. [BLANK · Hard · calculation]

A VAE encoder outputs μ = [-0.7] and σ = [1]. Compute $D_{KL}(q(z|x) \| N(0, I)) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$ using natural logs.

Answer: 0.245

Working:

- j=0: $-0.7^2 + 1^2 - 1 - \ln(1^2) = 0.4900$
- KL = ½ × sum = 0.2450.

Takeaway: KL is 0 only when μ = 0 and σ = 1: it pulls encodings toward the standard normal prior.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B17. [MCQ · Easy · calculation]

A VAE predicts μ = 0, σ = 0.2. A draw from N(0, 1) gives ε = 0.82. What sampled latent z does the reparameterisation trick produce?

A. 0.200
B. 0.820
C. 0.033
D. 0.164

Answer: D. 0.164

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0 + 0.2 \times 0.82 = 0.164$.

Option notes:

- A: $z = \mu + \sigma \cdot \varepsilon$.
- B: Mixed up the roles of μ and σ.
- C: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B18. [MCQ · Easy · calculation]

A VAE predicts μ = 0.6, σ = 0.5. A draw from N(0, 1) gives ε = -0.13. What sampled latent z does the reparameterisation trick produce?

A. 0.568
B. 0.535
C. 0.422
D. 0.170

Answer: B. 0.535

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0.6 + 0.5 \times -0.13 = 0.535$.

Option notes:

- A: Used the variance σ^2 instead of σ.
- C: $z = \mu + \sigma \cdot \varepsilon$.
- D: Mixed up the roles of μ and σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B19. [MCQ · Easy · calculation]

A VAE predicts μ = -0.2, σ = 2. A draw from N(0, 1) gives ε = 0.54. What sampled latent z does the reparameterisation trick produce?

A. 1.892
B. 0.880
C. 1.960
D. 0.140

Answer: B. 0.880

Working:

- $z = \mu + \sigma \cdot \varepsilon = -0.2 + 2 \times 0.54 = 0.880$.

Option notes:

- A: $z = \mu + \sigma \cdot \varepsilon$.
- C: Used the variance σ^2 instead of σ.
- D: Mixed up the roles of μ and σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B20. [MCQ · Easy · calculation]

A VAE predicts μ = 0, σ = 1.5. A draw from N(0, 1) gives ε = 0.78. What sampled latent z does the reparameterisation trick produce?

A. 0.780
B. 1.500
C. 1.755
D. 1.170

Answer: D. 1.170

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0 + 1.5 \times 0.78 = 1.170$.

Option notes:

- A: Mixed up the roles of μ and σ.
- B: $z = \mu + \sigma \cdot \varepsilon$.
- C: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B21. [MCQ · Easy · calculation]

A VAE predicts $\mu = 0.7$, $\sigma = 2$. A draw from N(0, 1) gives $\varepsilon = -0.16$. What sampled latent z does the reparameterisation trick produce?

A. 0.380
B. 0.060
C. 1.240
D. 1.888

Answer: A. 0.380

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0.7 + 2 \times -0.16 = 0.380$.

Option notes:

- B: Used the variance σ^2 instead of σ.
- C: Mixed up the roles of μ and σ.
- D: $z = \mu + \sigma \cdot \varepsilon$.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B22. [MCQ · Easy · calculation]

A VAE predicts $\mu = -0.8$, $\sigma = 2$. A draw from N(0, 1) gives $\varepsilon = 0.21$. What sampled latent z does the reparameterisation trick produce?

A. -1.390
B. 1.832
C. -0.380
D. 0.040

Answer: C. -0.380

Working:

- $z = \mu + \sigma \cdot \varepsilon = -0.8 + 2 \times 0.21 = -0.380$.

Option notes:

- A: Mixed up the roles of μ and σ.
- B: $z = \mu + \sigma \cdot \varepsilon$.
- D: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B23. [MCQ · Easy · calculation]

A VAE predicts μ = 0.5, σ = 2. A draw from N(0, 1) gives ε = -0.52. What sampled latent z does the reparameterisation trick produce?

A. -0.540
B. -1.580
C. 0.480
D. 1.740

Answer: A. -0.540

Working:

- z = μ + σ·ε = 0.5 + 2×-0.52 = -0.540.

Option notes:

- B: Used the variance σ² instead of σ.
- C: Mixed up the roles of μ and σ.
- D: z = μ + σ·ε.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B24. [MCQ · Easy · calculation]

A VAE predicts μ = 0.1, σ = 0.5. A draw from N(0, 1) gives ε = -0.66. What sampled latent z does the reparameterisation trick produce?

A. -0.610
B. -0.065
C. -0.230
D. 0.434

Answer: C. -0.230

Working:

- z = μ + σ·ε = 0.1 + 0.5×-0.66 = -0.230.

Option notes:

- A: Mixed up the roles of μ and σ.
- B: Used the variance σ² instead of σ.
- D: z = μ + σ·ε.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B25. [MCQ · Easy · calculation]

A VAE predicts μ = -0.9, σ = 2. A draw from N(0, 1) gives ε = -0.99. What sampled latent z does the reparameterisation trick produce?

A. -2.790
B. 2.891
C. -4.860
D. -2.880

Answer: D. -2.880

Working:

- $z = \mu + \sigma \cdot \varepsilon = -0.9 + 2 \times -0.99 = -2.880$.

Option notes:

- A: Mixed up the roles of μ and σ.
- B: $z = \mu + \sigma \cdot \varepsilon$.
- C: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B26. [MCQ · Easy · calculation]

A VAE predicts $\mu = 0$, $\sigma = 2$. A draw from N(0, 1) gives $\varepsilon = -0.51$. What sampled latent z does the reparameterisation trick produce?

A. -0.510
B. -1.020
C. -2.040
D. 2.000

Answer: B. -1.020

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0 + 2 \times -0.51 = -1.020$.

Option notes:

- A: Mixed up the roles of μ and σ.
- C: Used the variance σ^2 instead of σ.
- D: $z = \mu + \sigma \cdot \varepsilon$.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B27. [MCQ · Easy · calculation]

A VAE predicts $\mu = 0.1$, $\sigma = 1.5$. A draw from N(0, 1) gives $\varepsilon = -0.99$. What sampled latent z does the reparameterisation trick produce?

A. 1.401
B. -1.385
C. -2.128
D. -0.840

Answer: B. -1.385

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0.1 + 1.5 \times -0.99 = -1.385$.

Option notes:

- A: $z = \mu + \sigma \cdot \varepsilon$.
- C: Used the variance σ^2 instead of σ.
- D: Mixed up the roles of μ and σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B28. [MCQ · Easy · calculation]

A VAE predicts μ = -0.4, σ = 2. A draw from N(0, 1) gives ε = 0.67. What sampled latent z does the reparameterisation trick produce?

A. 1.732
B. -0.130
C. 2.280
D. 0.940

Answer: D. 0.940

Working:

- $z = \mu + \sigma \cdot \varepsilon = -0.4 + 2 \times 0.67 = 0.940$.

Option notes:

- A: $z = \mu + \sigma \cdot \varepsilon$.
- B: Mixed up the roles of μ and σ.
- C: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B29. [MCQ · Easy · calculation]

A VAE predicts μ = 0.6, σ = 2. A draw from N(0, 1) gives ε = -0.37. What sampled latent z does the reparameterisation trick produce?

A. -0.140
B. 0.830
C. 1.778
D. -0.880

Answer: A. -0.140

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0.6 + 2 \times -0.37 = -0.140$.

Option notes:

- B: Mixed up the roles of μ and σ.
- C: $z = \mu + \sigma \cdot \varepsilon$.
- D: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B30. [MCQ · Easy · calculation]

A VAE predicts μ = -0.6, σ = 1.5. A draw from N(0, 1) gives ε = -0.31. What sampled latent z does the reparameterisation trick produce?

A. -1.210
B. 1.686
C. -1.298
D. -1.065

Answer: D. -1.065

Working:

- $z = \mu + \sigma \cdot \varepsilon = -0.6 + 1.5 \times -0.31 = -1.065$.

Option notes:

- A: Mixed up the roles of μ and σ.
- B: $z = \mu + \sigma \cdot \varepsilon$.
- C: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B31. [MCQ · Easy · calculation]

A VAE predicts $\mu = 0.6$, $\sigma = 1.5$. A draw from $N(0, 1)$ gives $\varepsilon = 0.27$. What sampled latent z does the reparameterisation trick produce?

A. 1.005
B. 1.170
C. 1.662
D. 1.208

Answer: A. 1.005

Working:

- $z = \mu + \sigma \cdot \varepsilon = 0.6 + 1.5 \times 0.27 = 1.005$.

Option notes:

- B: Mixed up the roles of μ and σ.
- C: $z = \mu + \sigma \cdot \varepsilon$.
- D: Used the variance σ^2 instead of σ.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B32. [MCQ · Easy · calculation]

A VAE predicts $\mu = -0.9$, $\sigma = 0.5$. A draw from $N(0, 1)$ gives $\varepsilon = 0.31$. What sampled latent z does the reparameterisation trick produce?

A. -0.823
B. -0.140
C. -0.745
D. 0.221

Answer: C. -0.745

Working:

- $z = \mu + \sigma \cdot \varepsilon = -0.9 + 0.5 \times 0.31 = -0.745$.

Option notes:

- A: Used the variance σ^2 instead of σ.
- B: Mixed up the roles of μ and σ.
- D: $z = \mu + \sigma \cdot \varepsilon$.

Takeaway: Reparameterisation moves randomness into ε so gradients can flow through μ and σ.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B33. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.9$, pixel $x_0 = 0.8$ and noise $\varepsilon = -0.8$, what is x_t?

Answer: 0.506

Working:

- $\sqrt{0.9} = 0.9487$; $\sqrt{0.1} = 0.3162$.
- $x_t = 0.9487 \times 0.8 + 0.3162 \times -0.8 = 0.5060$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B34. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.36$, pixel $x_0 = 0.3$ and noise $\varepsilon = -0.1$, what is x_t?

Answer: 0.100

Working:

- $\sqrt{0.36} = 0.6000$; $\sqrt{0.64} = 0.8000$.
- $x_t = 0.6000 \times 0.3 + 0.8000 \times -0.1 = 0.1000$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B35. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.5$, pixel $x_0 = -0.9$ and noise $\varepsilon = -0.8$, what is x_t?

Answer: -1.202

Working:

- $\sqrt{0.5} = 0.7071$; $\sqrt{0.5} = 0.7071$.
- $x_t = 0.7071 \times -0.9 + 0.7071 \times -0.8 = -1.2021$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B36. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.9$, pixel $x_0 = -0.7$ and noise $\varepsilon = -0.9$, what is x_t?

Answer: -0.949

Working:

- $\sqrt{0.9} = 0.9487$; $\sqrt{0.1} = 0.3162$.
- $x_t = 0.9487 \times -0.7 + 0.3162 \times -0.9 = -0.9487$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B37. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.36$, pixel $x_0 = 0.8$ and noise $\varepsilon = -0.7$, what is x_t?

Answer: -0.080

Working:

- $\sqrt{0.36} = 0.6000$; $\sqrt{0.64} = 0.8000$.
- $x_t = 0.6000 \times 0.8 + 0.8000 \times -0.7 = -0.0800$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B38. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.81$, pixel $x_0 = -1$ and noise $\varepsilon = -0.4$, what is x_t?

Answer: -1.074

Working:

- $\sqrt{0.81} = 0.9000$; $\sqrt{0.19} = 0.4359$.
- $x_t = 0.9000 \times -1 + 0.4359 \times -0.4 = -1.0744$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B39. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.5$, pixel $x_0 = -1$ and noise $\varepsilon = -0.6$, what is x_t?

Answer: -1.131

Working:

- $\sqrt{0.5} = 0.7071$; $\sqrt{0.5} = 0.7071$.
- $x_t = 0.7071 \times -1 + 0.7071 \times -0.6 = -1.1314$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B40. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.64$, pixel $x_0 = -0.4$ and noise $\varepsilon = 0.8$, what is x_t?

Answer: 0.160

Working:

- $\sqrt{0.64} = 0.8000$; $\sqrt{0.36} = 0.6000$.
- $x_t = 0.8000 \times -0.4 + 0.6000 \times 0.8 = 0.1600$.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B41. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t = 0.64$, pixel $x_0 = 0.7$ and noise $\varepsilon = 0.4$, what is x_t?

Answer: 0.800

Working:

- √0.64 = 0.8000; √0.36 = 0.6000.
- x_t = 0.8000×0.7 + 0.6000×0.4 = 0.8000.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B42. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t$ = 0.64, pixel x_0 = 0.7 and noise ε = 0.1, what is x_t?

Answer: 0.620

Working:

- √0.64 = 0.8000; √0.36 = 0.6000.
- x_t = 0.8000×0.7 + 0.6000×0.1 = 0.6200.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B43. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t$ = 0.36, pixel x_0 = 0.8 and noise ε = 0.7, what is x_t?

Answer: 1.040

Working:

- √0.36 = 0.6000; √0.64 = 0.8000.
- x_t = 0.6000×0.8 + 0.8000×0.7 = 1.0400.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B44. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t$ = 0.81, pixel x_0 = -0.5 and noise ε = -1, what is x_t?

Answer: -0.886

Working:

- √0.81 = 0.9000; √0.19 = 0.4359.
- x_t = 0.9000×-0.5 + 0.4359×-1 = -0.8859.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B45. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t$ = 0.5, pixel x_0 = 0.8 and noise ε = -0.4, what is x_t?

Answer: 0.283

Working:

- √0.5 = 0.7071; √0.5 = 0.7071.
- x_t = 0.7071×0.8 + 0.7071×-0.4 = 0.2828.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B46. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t$ = 0.25, pixel x_0 = -0.3 and noise ε = 0.8, what is x_t?

Answer: 0.543

Working:

- √0.25 = 0.5000; √0.75 = 0.8660.
- x_t = 0.5000×-0.3 + 0.8660×0.8 = 0.5428.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B47. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t$ = 0.81, pixel x_0 = 0.2 and noise ε = -0.5, what is x_t?

Answer: -0.038

Working:

- √0.81 = 0.9000; √0.19 = 0.4359.
- x_t = 0.9000×0.2 + 0.4359×-0.5 = -0.0379.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B48. [BLANK · Medium · calculation]

Diffusion forward process: $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$. With $\bar{\alpha}_t$ = 0.81, pixel x_0 = 0 and noise ε = -0.5, what is x_t?

Answer: -0.218

Working:

- √0.81 = 0.9000; √0.19 = 0.4359.
- x_t = 0.9000×0 + 0.4359×-0.5 = -0.2179.

Takeaway: Small $\bar{\alpha}$ (late step) → mostly noise. The network is trained to predict ε with MSE $\|\hat{\varepsilon} - \varepsilon\|^2$.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B49. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.8 and D(G(z)) = 0.5. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.916

Working:

- ln(0.8) = -0.2231; ln(1 − 0.5) = ln(0.5) = -0.6931.
- Sum = -0.9163.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B50. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.6 and D(G(z)) = 0.4. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -1.022

Working:

- ln(0.6) = -0.5108; ln(1 − 0.4) = ln(0.6) = -0.5108.
- Sum = -1.0217.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B51. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.5 and D(G(z)) = 0.2. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.916

Working:

- ln(0.5) = -0.6931; ln(1 − 0.2) = ln(0.8) = -0.2231.
- Sum = -0.9163.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B52. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.6 and D(G(z)) = 0.1. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.616

Working:

- ln(0.6) = -0.5108; ln(1 − 0.1) = ln(0.9) = -0.1054.
- Sum = -0.6162.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B53. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.8 and D(G(z)) = 0.1. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.329

Working:

- ln(0.8) = -0.2231; ln(1 − 0.1) = ln(0.9) = -0.1054.
- Sum = -0.3285.

- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B54. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.9 and D(G(z)) = 0.1. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.211

Working:

- ln(0.9) = -0.1054; ln(1 − 0.1) = ln(0.9) = -0.1054.
- Sum = -0.2107.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B55. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.9 and D(G(z)) = 0.1. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.211

Working:

- ln(0.9) = -0.1054; ln(1 − 0.1) = ln(0.9) = -0.1054.
- Sum = -0.2107.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B56. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.9 and D(G(z)) = 0.2. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.329

Working:

- ln(0.9) = -0.1054; ln(1 − 0.2) = ln(0.8) = -0.2231.
- Sum = -0.3285.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B57. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.9 and D(G(z)) = 0.5. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.799

Working:

- ln(0.9) = -0.1054; ln(1 − 0.5) = ln(0.5) = -0.6931.
- Sum = -0.7985.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B58. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.8 and D(G(z)) = 0.5. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.916

Working:

- ln(0.8) = -0.2231; ln(1 − 0.5) = ln(0.5) = -0.6931.
- Sum = -0.9163.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B59. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.5 and D(G(z)) = 0.2. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.916

Working:

- ln(0.5) = -0.6931; ln(1 − 0.2) = ln(0.8) = -0.2231.
- Sum = -0.9163.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B60. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.9 and D(G(z)) = 0.1. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.211

Working:

- ln(0.9) = -0.1054; ln(1 − 0.1) = ln(0.9) = -0.1054.
- Sum = -0.2107.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B61. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.9 and D(G(z)) = 0.2. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.329

Working:

- ln(0.9) = -0.1054; ln(1 − 0.2) = ln(0.8) = -0.2231.
- Sum = -0.3285.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B62. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.6 and D(G(z)) = 0.5. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -1.204

Working:

- ln(0.6) = -0.5108; ln(1 − 0.5) = ln(0.5) = -0.6931.
- Sum = -1.2040.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B63. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.8 and D(G(z)) = 0.5. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.916

Working:

- ln(0.8) = -0.2231; ln(1 − 0.5) = ln(0.5) = -0.6931.
- Sum = -0.9163.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B64. [BLANK · Medium · calculation]

For one real sample and one fake sample, the discriminator outputs D(x) = 0.8 and D(G(z)) = 0.1. Evaluate log D(x) + log(1 − D(G(z))) (natural logs). D wants this large; G wants it small.

Answer: -0.329

Working:

- ln(0.8) = -0.2231; ln(1 − 0.1) = ln(0.9) = -0.1054.
- Sum = -0.3285.
- At equilibrium p_G = p_data and D(x) ≈ 0.5 everywhere: value = 2 ln 0.5 ≈ −1.386.

Takeaway: min over G, max over D of E[log D(x)] + E[log(1 − D(G(z)))].

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B65. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = 0$, $w = (-0.5, 0.5, -0.4)$ and $x_1, x_2, x_3 = 1, 0, 0$, compute $\hat{x}_4$.

Answer: 0.378

Working:

- $z = 0 + -0.5\times1 + 0.5\times0 + -0.4\times0 = -0.500$.
- $\sigma(-0.500) = 0.3775$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B66. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = 0.4$, $w = (0.5, -0.1, 0.6)$ and $x_1, x_2, x_3 = 0, 0, 1$, compute $\hat{x}_4$.

Answer: 0.731

Working:

- $z = 0.4 + 0.5\times0 + -0.1\times0 + 0.6\times1 = 1.000$.
- $\sigma(1.000) = 0.7311$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B67. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = 0.3$, $w = (-0.8, -0.1, 0.8)$ and $x_1, x_2, x_3 = 0, 1, 1$, compute $\hat{x}_4$.

Answer: 0.731

Working:

- $z = 0.3 + -0.8\times0 + -0.1\times1 + 0.8\times1 = 1.000$.
- $\sigma(1.000) = 0.7311$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B68. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.9$, $w = (-0.2, 0.7, 0.1)$ and $x_1, x_2, x_3 = 0, 1, 1$, compute $\hat{x}_4$.

Answer: 0.475

Working:

- $z = -0.9 + -0.2\times0 + 0.7\times1 + 0.1\times1 = -0.100$.
- $\sigma(-0.100) = 0.4750$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B69. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -1$, $w = (-0.8, 0, 0.8)$ and $x_1, x_2, x_3 = 1, 0, 0$, compute $\hat{x}_4$.

Answer: 0.142

Working:

- $z = -1 + -0.8 \times 1 + 0 \times 0 + 0.8 \times 0 = -1.800$.
- $\sigma(-1.800) = 0.1419$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B70. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.6$, $w = (0.7, 0.2, -0.1)$ and $x_1, x_2, x_3 = 1, 0, 0$, compute $\hat{x}_4$.

Answer: 0.525

Working:

- $z = -0.6 + 0.7 \times 1 + 0.2 \times 0 + -0.1 \times 0 = 0.100$.
- $\sigma(0.100) = 0.5250$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B71. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = 0.9$, $w = (0.7, -0.5, 0.2)$ and $x_1, x_2, x_3 = 0, 0, 1$, compute $\hat{x}_4$.

Answer: 0.750

Working:

- $z = 0.9 + 0.7 \times 0 + -0.5 \times 0 + 0.2 \times 1 = 1.100$.
- $\sigma(1.100) = 0.7503$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B72. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.6$, $w = (0.7, 0.1, -0.2)$ and $x_1, x_2, x_3 = 1, 1, 0$, compute $\hat{x}_4$.

Answer: 0.550

Working:

- $z = -0.6 + 0.7 \times 1 + 0.1 \times 1 + -0.2 \times 0 = 0.200$.
- $\sigma(0.200) = 0.5498$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B73. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.3$, $w = (-0.4, 0.1, -0.1)$ and $x_1, x_2, x_3 = 0, 1, 1$, compute $\hat{x}_4$.

Answer: 0.426

Working:

- $z = -0.3 + -0.4 \times 0 + 0.1 \times 1 + -0.1 \times 1 = -0.300$.
- $\sigma(-0.300) = 0.4256$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B74. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.9$, $w = (0.3, 0.4, -0.1)$ and $x_1, x_2, x_3 = 1, 0, 0$, compute $\hat{x}_4$.

Answer: 0.354

Working:

- $z = -0.9 + 0.3\times1 + 0.4\times0 + -0.1\times0 = -0.600$.
- $\sigma(-0.600) = 0.3543$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B75. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.1$, $w = (0.4, -0.9, -1)$ and $x_1, x_2, x_3 = 0, 0, 1$, compute $\hat{x}_4$.

Answer: 0.250

Working:

- $z = -0.1 + 0.4\times0 + -0.9\times0 + -1\times1 = -1.100$.
- $\sigma(-1.100) = 0.2497$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B76. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.4$, $w = (-0.5, 0.7, 0.6)$ and $x_1, x_2, x_3 = 0, 1, 0$, compute $\hat{x}_4$.

Answer: 0.574

Working:

- $z = -0.4 + -0.5\times0 + 0.7\times1 + 0.6\times0 = 0.300$.
- $\sigma(0.300) = 0.5744$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B77. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.1$, $w = (0.5, 0.1, -0.2)$ and $x_1, x_2, x_3 = 1, 1, 1$, compute $\hat{x}_4$.

Answer: 0.574

Working:

- $z = -0.1 + 0.5\times1 + 0.1\times1 + -0.2\times1 = 0.300$.
- $\sigma(0.300) = 0.5744$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B78. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.4$, $w = (-0.3, -0.4, -0.2)$ and $x_1, x_2, x_3 = 0, 1, 1$, compute $\hat{x}_4$.

Answer: 0.269

Working:

- $z = -0.4 + -0.3\times0 + -0.4\times1 + -0.2\times1 = -1.000$.
- $\sigma(-1.000) = 0.2689$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B79. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = 0.2$, $w = (0.5, -0.5, 0.3)$ and $x_1, x_2, x_3 = 0, 0, 1$, compute $\hat{x}_4$.

Answer: 0.622

Working:

- $z = 0.2 + 0.5\times0 + -0.5\times0 + 0.3\times1 = 0.500$.
- $\sigma(0.500) = 0.6225$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B80. [BLANK · Medium · calculation]

FVSBN: $\hat{x}_4 = P(x_4 = 1 \mid x_1, x_2, x_3) = \sigma(b_4 + w_{41}x_1 + w_{42}x_2 + w_{43}x_3)$. With $b_4 = -0.7$, $w = (-0.7, -0.1, -0.3)$ and $x_1, x_2, x_3 = 1, 0, 1$, compute $\hat{x}_4$.

Answer: 0.154

Working:

- $z = -0.7 + -0.7\times1 + -0.1\times0 + -0.3\times1 = -1.700$.
- $\sigma(-1.700) = 0.1545$.

Takeaway: Autoregressive: $p(x_1 \ldots x_n) = \Pi\, p(x_i \mid x_1 \ldots x_{i-1})$; each unit only sees EARLIER variables.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B81. [BLANK · Easy · calculation]

An autoencoder reconstructs $x = [0.5, 0.9, 0.9]$ as $\hat{x} = [0.3, 0.9, 0.9]$. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.040

Working:

- Differences: 0.2, 0, 0.
- Squares sum = 0.0400.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B82. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.8, 0, 0.4] as $\hat{x}$ = [0.9, 0.2, 0.3]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.060

Working:

- Differences: -0.1, -0.2, 0.1.
- Squares sum = 0.0600.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B83. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.4, 0.5, 0.8] as $\hat{x}$ = [0.4, 0.4, 0.6]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.050

Working:

- Differences: 0, 0.1, 0.2.
- Squares sum = 0.0500.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B84. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.5, 0.4, 0.9] as $\hat{x}$ = [0.2, 0.5, 0.8]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.110

Working:

- Differences: 0.3, -0.1, 0.1.
- Squares sum = 0.1100.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B85. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.8, 0.7, 0.4] as $\hat{x}$ = [1.1, 0.9, 0.6]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.170

Working:

- Differences: -0.3, -0.2, -0.2.
- Squares sum = 0.1700.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B86. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.1, 0.6, 0.6] as $\hat{x}$ = [0.1, 0.4, 0.9]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.130

Working:

- Differences: 0, 0.2, -0.3.
- Squares sum = 0.1300.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B87. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.8, 0.5, 0.2] as $\hat{x}$ = [1, 0.7, 0.5]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.170

Working:

- Differences: -0.2, -0.2, -0.3.
- Squares sum = 0.1700.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B88. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.5, 0.1, 0.2] as $\hat{x}$ = [0.7, -0.1, 0.5]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.170

Working:

- Differences: -0.2, 0.2, -0.3.
- Squares sum = 0.1700.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B89. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0, 0.6, 0] as $\hat{x}$ = [-0.2, 0.7, -0.1]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.060

Working:

- Differences: 0.2, -0.1, 0.1.
- Squares sum = 0.0600.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B90. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.5, 0.7, 0.2] as $\hat{x}$ = [0.8, 0.8, 0]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.140

Working:

- Differences: -0.3, -0.1, 0.2.
- Squares sum = 0.1400.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B91. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.5, 0.3, 0] as $\hat{x}$ = [0.6, 0.2, -0.2]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.060

Working:

- Differences: -0.1, 0.1, 0.2.
- Squares sum = 0.0600.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B92. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.3, 0.7, 0.8] as $\hat{x}$ = [0.1, 0.7, 1.1]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.130

Working:

- Differences: 0.2, 0, -0.3.
- Squares sum = 0.1300.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B93. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.8, 0.7, 0.3] as $\hat{x}$ = [0.7, 0.7, 0.5]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.050

Working:

- Differences: 0.1, 0, -0.2.
- Squares sum = 0.0500.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B94. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.2, 0.5, 0.3] as $\hat{x}$ = [-0.1, 0.6, 0.3]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.100

Working:

- Differences: 0.3, -0.1, 0.
- Squares sum = 0.1000.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B95. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0.8, 0.5, 0.6] as $\hat{x}$ = [0.7, 0.5, 0.8]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.050

Working:

- Differences: 0.1, 0, -0.2.
- Squares sum = 0.0500.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

GENAI B96. [BLANK · Easy · calculation]

An autoencoder reconstructs x = [0, 0.2, 0.7] as $\hat{x}$ = [0.3, -0.1, 0.7]. Compute $L(x, \hat{x}) = \|x - \hat{x}\|^2$ (sum of squared differences).

Answer: 0.180

Working:

- Differences: -0.3, 0.3, 0.
- Squares sum = 0.1800.

Takeaway: Autoencoder loss uses NO labels: the input is its own target.

Source: Lecture 9 (Generative AI: autoencoders, VAE, GAN, autoregressive, diffusion)

🔐 Logic & Inference (Lecture 10)

Part A: hand-written questions (121)

LOGIC A1. [MCQ · Easy]

When is P ⇒ Q false?

A. When both are false
B. When P is false
C. When Q is true
D. Only when P is true and Q is false

Answer: D. Only when P is true and Q is false

Option notes:

- A: True (vacuously).
- B: Then it's vacuously true.
- C: Always true then.

Takeaway: Implication = a promise.

Source: Lecture 10, slide 13

LOGIC A2. [TF · Medium]

'If 2 + 2 = 5, then x > 3' is false because its premise is false.

Answer: False

Why: A false premise makes the implication VACUOUSLY TRUE.

Takeaway: False premise → true implication.

Source: Lecture 10, slide 13

LOGIC A3. [MCQ · Easy]

Which is NOT a proposition?

A. It is raining.
B. Is it raining?
C. 2 + 2 = 5.
D. Sydney is in Australia.

Answer: B. Is it raining?

Option notes:

- A: Declarative: T/F.
- C: False, but a proposition.
- D: True proposition.

Takeaway: Questions and commands are not propositions.

Source: Lecture 10, slide 9

LOGIC A4. [MATCH · Medium]

Match each term to its definition.

1. Literal
2. Conjunction
3. Disjunction
4. Implication

A. P ⇒ Q, with premise P and conclusion Q
B. a sentence that is always false
C. two sentences connected by ∧
D. two sentences connected by ∨
E. an atomic sentence or a negated atomic sentence

Answer: 1 → E, 2 → C, 3 → D, 4 → A

Working:

- Literal → an atomic sentence or a negated atomic sentence
- Conjunction → two sentences connected by ∧
- Disjunction → two sentences connected by ∨
- Implication → P ⇒ Q, with premise P and conclusion Q

Takeaway: Lecture 10 slide 11.

Source: Lecture 10, slide 11

LOGIC A5. [TF · Medium]

P ⇔ Q is true whenever P and Q have the same truth value.

Answer: True

Why: Both T or both F.

Takeaway: Biconditional = necessary and sufficient.

Source: Lecture 10, slide 14

LOGIC A6. [MULTI · Medium]

Select ALL correct statements about a knowledge base (Lecture 10). (Select ALL that apply.)

A. It must be written in natural language
B. It stores facts and rules as sentences in a representation language
C. An inference engine derives new knowledge from it
D. From P, P ⇒ Q, Q ⇒ R we can infer R

Answer: B. It stores facts and rules as sentences in a representation language; C. An inference engine derives new knowledge from it; D. From P, P ⇒ Q, Q ⇒ R we can infer R

Option notes:

- A: Incorrect: A formal language.
- B: Correct:
- C: Correct:
- D: Correct: Chain.

Takeaway: KB + inference → action.

Source: Lecture 10, slide 5-7

LOGIC A7. [MCQ · Medium]

A formal language's grammar defines _____ and its semantic rules define _____.

A. their meaning; the syntax
B. the syntax of legal sentences; their meaning
C. the truth values; the symbols
D. the inference engine; the KB

Answer: B. the syntax of legal sentences; their meaning

Option notes:

- A: Reversed.
- C: Not quite.
- D: No.

Takeaway: Syntax vs semantics.

Source: Lecture 10, slide 8

LOGIC A8. [MCQ · Medium]

Which sentence is equivalent to P ⇒ Q?

A. Q ⇒ P
B. ¬P ⇒ ¬Q
C. P ∧ ¬Q
D. ¬Q ⇒ ¬P

Answer: D. ¬Q ⇒ ¬P

Option notes:

- A: Converse.
- B: Inverse.
- C: That's ¬(P ⇒ Q).

Takeaway: Contraposition.

Source: Lecture 10, slide 15

LOGIC A9. [MATCH · Medium]

Match each equivalence to its name.

1. α ⇒ β ≡ ¬α ∨ β
2. ¬(α ∧ β) ≡ ¬α ∨ ¬β
3. α ∨ (β ∧ γ) ≡ (α ∨ β) ∧ (α ∨ γ)
4. ¬(¬α) ≡ α

A. modus ponens
B. implication elimination
C. double-negation elimination
D. distributivity of ∨ over ∧
E. De Morgan

Answer: 1 → B, 2 → E, 3 → D, 4 → C

Working:

- α ⇒ β ≡ ¬α ∨ β → implication elimination
- ¬(α ∧ β) ≡ ¬α ∨ ¬β → De Morgan
- α ∨ (β ∧ γ) ≡ (α ∨ β) ∧ (α ∨ γ) → distributivity of ∨ over ∧
- ¬(¬α) ≡ α → double-negation elimination

Takeaway: Equivalence table.

Source: Lecture 10, slide 15

LOGIC A10. [TF · Medium]

¬(P ∨ Q) ≡ ¬P ∨ ¬Q.

Answer: False

Why: De Morgan: ¬(P ∨ Q) ≡ ¬P ∧ ¬Q (flip ∨ to ∧).

Takeaway: Flip the operator.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC A11. [MCQ · Medium]

Which FOL sentence says 'every king is a person'?

A. King(Person)
B. ∃x King(x) ⇒ Person(x)
C. ∀x King(x) ⇒ Person(x)
D. ∀x King(x) ∧ Person(x)

Answer: C. ∀x King(x) ⇒ Person(x)

Option notes:

- A: Person isn't an object here.
- B: Existential: too weak.
- D: Says everything is a king and a person.

Takeaway: ∀ pairs with ⇒.

Source: Lecture 10, slide 22

LOGIC A12. [TF · Medium]

∀x ∃y Loves(x, y) means the same as ∃y ∀x Loves(x, y).

Answer: False

Why: Order matters: 'everyone loves someone' vs 'someone is loved by everyone'.

Takeaway: Quantifier order.

Source: Lecture 10, slide 22

LOGIC A13. [MATCH · Medium]

Match each FOL expression to its category.

1. John
2. Father(John)
3. Brother(Richard, John)
4. Raining

A. binary predicate (true/false)
B. constant symbol (term)
C. quantifier
D. zero-ary predicate
E. function applied to a term (names an object)

Answer: 1 → B, 2 → E, 3 → A, 4 → D

Working:

- John → constant symbol (term)
- Father(John) → function applied to a term (names an object)
- Brother(Richard, John) → binary predicate (true/false)
- Raining → zero-ary predicate

Takeaway: Term names an object; predicate is true/false.

Source: Lecture 10, slide 20-21

LOGIC A14. [MCQ · Medium]

By De Morgan for quantifiers, 'All cats do not talk' (∀x ¬P(x)) is equivalent to…

A. There exists a cat that talks
B. All cats talk (∀x P(x))
C. Some cats do not talk (∃x ¬P(x))
D. There does not exist a cat that talks (¬∃x P(x))

Answer: D. There does not exist a cat that talks (¬∃x P(x))

Option notes:

- A: Opposite.
- B: Opposite.
- C: Weaker.

Takeaway: ∀x ¬P ⇔ ¬∃x P.

Source: Lecture 10, slide 23

LOGIC A15. [MCQ · Hard]

In Existential Instantiation, what must the replacement constant be?

A. A new constant that does not appear elsewhere in the KB
B. Any existing constant
C. A variable
D. The constant True

Answer: A. A new constant that does not appear elsewhere in the KB

Option notes:

- B: That's Universal Instantiation.
- C: Must be ground.
- D: No.

Takeaway: Fresh constant names an unspecified existing object.

Source: Lecture 10, slide 47

LOGIC A16. [MULTI · Medium]

Select ALL steps of FOL inference by propositionalisation. (Select ALL that apply.)

A. Apply Existential Instantiation with fresh constants
B. Treat ground atoms as propositions
C. Delete all predicates
D. Run a complete propositional inference method
E. Apply Universal Instantiation to create ground instances

Answer: A. Apply Existential Instantiation with fresh constants; B. Treat ground atoms as propositions; D. Run a complete propositional inference method; E. Apply Universal Instantiation to create ground instances

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: They become ground atoms.
- D: Correct:
- E: Correct:

Takeaway: EI → UI → propositionalise → infer.

Source: Lecture 10, slide 48

LOGIC A17. [TF · Medium]

Propositionalisation can be impractical because it may generate very many ground instances, especially with functions or large domains.

Answer: True

Why: Slide 45/48.

Takeaway: Practical limitation.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC A18. [MATCH · Medium]

Match each reasoning/inference type to its example.

1. Deductive reasoning
2. Inductive reasoning
3. Machine learning inference

A. Assume the opposite and find a contradiction
B. Last five lab-goers passed; Leo attends → Leo will probably pass
C. All students must submit; Mia is a student → Mia must submit
D. Trained flower classifier predicts Iris setosa for new measurements

Answer: 1 → C, 2 → B, 3 → D

Working:

- Deductive reasoning → All students must submit; Mia is a student → Mia must submit
- Inductive reasoning → Last five lab-goers passed; Leo attends → Leo will probably pass
- Machine learning inference → Trained flower classifier predicts Iris setosa for new measurements

Takeaway: Deduction certain; induction probable.

Source: Lecture 10, slide 25-26

LOGIC A19. [TF · Easy]

Deductive conclusions are certain if the premises are true, while inductive conclusions are only probable.

Answer: True

Why: Slide 25.

Takeaway: Key difference.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC A20. [MCQ · Medium]

KB ⊨ γ holds when…

A. γ is true in at least one model of KB
B. KB and γ have the same symbols
C. γ is true in every model in which KB is true
D. KB is true in every model of γ

Answer: C. γ is true in every model in which KB is true

Option notes:

- A: That's satisfiability of KB ∧ γ.
- B: Irrelevant.
- D: Reversed: M(KB) ⊆ M(γ).

Takeaway: No KB-true model makes γ false.

Source: Lecture 10, slide 28

LOGIC A21. [BLANKS · Medium · calculation]

Model | P | Q | P⇒Q | KB = P ∧ (P⇒Q) | Q
1 | T | T | T | T | T
2 | T | F | F | F | F
3 | F | T | T | F | T
4 | F | F | T | F | F

Rain example: KB = P ∧ (P ⇒ Q). In how many of the 4 models is KB true, and is Q true in them? (number, Yes/No)

(1) KB-true models = ____
(2) Q true there? = ____

Answer: KB-true models = 1; Q true there? = Yes

Takeaway: Only model 1 (P = T, Q = T).

Source: Lecture 10, slide 32

LOGIC A22. [MULTI · Medium]

Select ALL steps of model checking in order. (Select ALL that apply.)

A. Identify the symbols
B. Find models that satisfy KB
C. Negate the KB
D. Check the query in those models
E. Generate all models

Answer: A. Identify the symbols; B. Find models that satisfy KB; D. Check the query in those models; E. Generate all models

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: Not model checking.
- D: Correct:
- E: Correct:

Takeaway: List → generate → filter → check.

Source: Lecture 10, slide 31

LOGIC A23. [MCQ · Easy]

From P and P ⇒ Q we derive Q. This inference rule is…

A. Modus Ponens
B. Contraposition
C. And-elimination
D. Resolution

Answer: A. Modus Ponens

Option notes:

- B: An equivalence.
- C: That's α ∧ β ⊢ α.
- D: Uses clauses.

Takeaway: Theorem proving.

Source: Lecture 10, slide 35

LOGIC A24. [TF · Medium]

Theorem proving needs to enumerate all possible models.

Answer: False

Why: Theorem proving builds a proof chain WITHOUT enumerating all models.

Takeaway: That's its advantage over model checking.

Source: Lecture 10, slide 33

LOGIC A25. [MCQ · Medium]

To prove KB ⊨ γ by contradiction, which sentence do we show is unsatisfiable?

A. ¬KB ∧ γ
B. KB ∧ γ
C. KB ∧ ¬γ
D. ¬KB ∨ γ

Answer: C. KB ∧ ¬γ

Option notes:

- A: Don't negate the KB.
- B: Negate the query.
- D: That's KB ⇒ γ.

Takeaway: Negate only the query.

Source: Lecture 10, slide 42

LOGIC A26. [TF · Medium]

Deriving the empty clause □ by resolution proves that the starting clause set is satisfiable.

Answer: False

Why: □ proves the set is UNSATISFIABLE (a contradiction).

Takeaway: Empty clause = contradiction.

Source: Lecture 10, slide 41

LOGIC A27. [BLANK · Medium]

Resolving clauses (P ∨ Q) and (¬P ∨ R) gives the resolvent _____.

Answer: Q ∨ R
(Also accepted: Q v R, QvR, Q∨R, R ∨ Q)

Takeaway: Drop P and ¬P.

Source: Lecture 10, slide 41

LOGIC A28. [MCQ · Medium]

Which is in conjunctive normal form (CNF)?

A. ¬(P ∨ Q)
B. P ⇒ Q
C. (P ∨ Q) ∧ (¬P ∨ R)
D. (P ∧ Q) ∨ R

Answer: C. (P ∨ Q) ∧ (¬P ∨ R)

Option notes:

- A: Negation outside a clause.
- B: Contains ⇒.
- D: A disjunction of a conjunction.

Takeaway: CNF = AND of ORs of literals.

Source: Lecture 10, slide 37

LOGIC A29. [TF · Easy]

P ∧ ¬P is unsatisfiable.

Answer: True

Why: No assignment makes both true.

Takeaway: Classic contradiction.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC A30. [BLANK · Easy]

Facts and rules are stored as sentences in a knowledge _____.

Answer: base
(Also accepted: Base, KB)

Takeaway: Lecture 10 slide 5.

Source: Lecture 10 (Logic)

LOGIC A31. [MCQ · Medium]

KB: P (raining), P ⇒ Q (wet), Q ⇒ R (use indoor route). What does the inference chain conclude?

A. Q, then R: the robot takes the indoor route
B. Nothing without more facts
C. ¬R
D. Only P

Answer: A. Q, then R: the robot takes the indoor route

Option notes:

- B: Two Modus Ponens steps suffice.
- C: Contradicts the rules.
- D: P is the given fact.

Takeaway: Lecture 10 slide 7.

Source: Lecture 10 (Logic)

LOGIC A32. [TF · Hard]

Fuzzy logic is listed in Lecture 10 as one of the common knowledge representation languages, alongside propositional and first-order logic.

Answer: True

Why: Slide 8.

Takeaway: PL, FOL, fuzzy logic.

Source: Lecture 10 (Logic)

LOGIC A33. [TF · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔, the sentence ¬P ∨ Q ∧ R ⇒ S means ((¬P) ∨ (Q ∧ R)) ⇒ S.

Answer: True

Why: Slide 10.

Takeaway: ¬ first, then ∧.

Source: Lecture 10 (Logic)

LOGIC A34. [MCQ · Medium]

A model in propositional logic is…

A. a proof
B. a sentence in the KB
C. an assignment of True or False to every proposition symbol
D. a truth table of all sentences

Answer: C. an assignment of True or False to every proposition symbol

Option notes:

- A: No.
- B: That's a sentence.
- D: One row = one model.

Takeaway: Slide 12.

Source: Lecture 10 (Logic)

LOGIC A35. [MCQ · Hard]

Which sentence says 'Some student is enrolled in AI'?

A. ∀x Student(x) ∧ Enrolled(x, AI)
B. ∀x Student(x) ⇒ Enrolled(x, AI)
C. ∃x Student(x) ∧ Enrolled(x, AI)
D. ∃x Student(x) ⇒ Enrolled(x, AI)

Answer: C. ∃x Student(x) ∧ Enrolled(x, AI)

Option notes:

- A: Says everything is an enrolled student.
- B: Says every student is enrolled.
- D: With ∃, ⇒ is satisfied by any non-student: too weak.

Takeaway: ∃ pairs with ∧; ∀ pairs with ⇒.

Source: Lecture 10 (Logic)

LOGIC A36. [TF · Medium]

Machine learning inference means a trained model uses new input to produce a prediction.

Answer: True

Why: Slide 26.

Takeaway: Logical vs ML inference.

Source: Lecture 10 (Logic)

LOGIC A37. [BLANK · Easy]

Reasoning where a pattern in observations supports a likely (probable) conclusion is called _____ reasoning.

Answer: inductive
(Also accepted: Inductive)

Takeaway: Deductive = certain.

Source: Lecture 10 (Logic)

LOGIC A38. [MCQ · Hard]

'Rule: if a student passes every assessment they pass the subject. Fact: Mia passed every assessment. ⇒ Mia passed the subject.' This is…

A. model checking
B. logical inference (deductive)
C. inductive reasoning
D. machine learning inference

Answer: B. logical inference (deductive)

Option notes:

- A: No enumeration of models.
- C: No observed pattern; a rule is applied.
- D: No trained model.

Takeaway: Slide 26.

Source: Lecture 10 (Logic)

LOGIC A39. [TF · Medium]

And-elimination lets us infer α from α ∧ β.

Answer: True

Why: Slide 35.

Takeaway: Sound rule.

Source: Lecture 10 (Logic)

LOGIC A40. [BLANK · Easy]

The inference rule 'α ⇒ β, α ⊢ β' is called Modus _____.

Answer: Ponens
(Also accepted: ponens)

Takeaway: Theorem proving.

Source: Lecture 10 (Logic)

LOGIC A41. [MULTI · Hard]

Select ALL statements true of theorem proving (Lecture 10). (Select ALL that apply.)

A. It requires negating the KB
B. It applies sound inference rules directly to the KB
C. It constructs a proof without enumerating all models
D. Each step uses a valid rule or logical equivalence

Answer: B. It applies sound inference rules directly to the KB; C. It constructs a proof without enumerating all models; D. Each step uses a valid rule or logical equivalence

Option notes:

- A: Incorrect: That's not required.
- B: Correct:
- C: Correct:
- D: Correct:

Takeaway: Proof chain.

Source: Lecture 10 (Logic)

LOGIC A42. [BLANK · Easy]

Replacing a universally quantified variable with a ground term is called Universal _____.

Answer: Instantiation
(Also accepted: instantiation)

Takeaway: UI.

Source: Lecture 10 (Logic)

LOGIC A43. [TF · Hard]

UI may be applied repeatedly with different ground terms to generate multiple consequences, while EI is applied once to an existential sentence.

Answer: True

Why: Slides 46-47.

Takeaway: UI many times; EI once with a fresh constant.

Source: Lecture 10 (Logic)

LOGIC A44. [BLANK · Medium]

P ⇒ Q is false only when P is _____ and Q is false (one word: true/false).

Answer: true
(Also accepted: True, T)

Takeaway: Otherwise vacuously or genuinely true.

Source: Lecture 10 (Logic)

LOGIC A45. [MCQ · Medium]

With precedence ¬, ∧, ∨, ⇒, ⇔, ¬P ∨ Q ∧ R ⇒ S is read as:

A. ((¬P) ∨ (Q ∧ R)) ⇒ S
B. ¬P ∨ (Q ∧ (R ⇒ S))
C. (¬P ∨ Q) ∧ (R ⇒ S)
D. ¬(P ∨ Q) ∧ (R ⇒ S)

Answer: A. ((¬P) ∨ (Q ∧ R)) ⇒ S

Option notes:

- B: ⇒ is weakest, so it is applied last.
- C: ∧ binds tighter than ∨ and ⇒ is last.
- D: ¬ binds only P.

Takeaway: Not, and, or, implies, iff.

Source: Book 2 (Exam examples), Q315, p. 41

LOGIC A46. [TF · Easy]

'If 2 + 2 = 5, then x > 3' is true.

Answer: True

Why: A false premise makes the implication vacuously true.

Takeaway: F ⇒ anything = T.

Source: Book 2 (Exam examples), Q316, p. 41

LOGIC A47. [TF · Easy]

'If x > 5 then x < 3' is true for all x.

Answer: False

Why: x = 6 makes the premise true and conclusion false.

Takeaway: Only T ⇒ F is false.

Source: Book 2 (Exam examples), Q317, p. 41

LOGIC A48. [TF · Easy]

P ⇒ Q means P causes Q.

Answer: False

Why: Implication asserts no causation.

Takeaway: Truth, not cause.

Source: Book 2 (Exam examples), Q318, p. 41

LOGIC A49. [BLANKS · Medium · calculation]

P = T, Q = F, R = T. Evaluate (T/F): P ∧ Q; P ∨ Q; P ⇒ Q; Q ⇒ R; P ⇔ R; ¬P ∨ Q ∧ R ⇒ Q.

(1) P ∧ Q = _____
(2) P ∨ Q = _____
(3) P ⇒ Q = _____
(4) Q ⇒ R = _____
(5) P ⇔ R = _____
(6) ¬P ∨ Q ∧ R ⇒ Q = _____

Answer: P ∧ Q = F; P ∨ Q = T; P ⇒ Q = F; Q ⇒ R = T; P ⇔ R = T; ¬P ∨ Q ∧ R ⇒ Q = T

Takeaway: Last: left side (¬P) ∨ (Q ∧ R) = F, so F ⇒ Q is T.

Source: Book 2 (Exam examples), Q319, p. 41

LOGIC A50. [MCQ · Easy]

P ⇒ Q is equivalent to:

A. P ∨ ¬Q
B. Q ⇒ P
C. ¬P ∨ Q
D. ¬P ⇒ ¬Q

Answer: C. ¬P ∨ Q

Option notes:

- A: That is Q ⇒ P.
- B: That is the converse.
- D: That is the inverse.

Takeaway: Also ≡ ¬Q ⇒ ¬P (contrapositive).

Source: Book 2 (Exam examples), Q320, p. 41

LOGIC A51. [MCQ · Easy]

By De Morgan, ¬(P ∧ Q) ≡ ___ and ¬(P ∨ Q) ≡ ___.

A. ¬P ∧ ¬Q; ¬P ∨ ¬Q
B. ¬P ∧ Q; P ∨ ¬Q
C. ¬P ∨ ¬Q; ¬P ∧ ¬Q
D. P ∨ Q; P ∧ Q

Answer: C. ¬P ∨ ¬Q; ¬P ∧ ¬Q

Option notes:

- A: Swapped.
- B: Negate both.
- D: Negations dropped.

Takeaway: Flip the connective, negate each.

Source: Book 2 (Exam examples), Q321, p. 41

LOGIC A52. [MCQ · Easy]

P ⇔ Q ≡

A. P ∧ Q
B. ¬P ∨ Q
C. (P ⇒ Q) ∨ (Q ⇒ P)
D. (P ⇒ Q) ∧ (Q ⇒ P)

Answer: D. (P ⇒ Q) ∧ (Q ⇒ P)

Option notes:

- A: Misses the both-false case.
- B: That is P ⇒ Q only.
- C: That is always true.

Takeaway: Biconditional elimination.

Source: Book 2 (Exam examples), Q322, p. 41

LOGIC A53. [MATCH · Medium]

Match the term.

1. literal
2. clause
3. CNF
4. complementary literals

A. P and ¬P
B. a disjunction of literals
C. a conjunction of clauses
D. an atom or its negation

Answer: 1 → D, 2 → B, 3 → C, 4 → A

Working:

- literal → an atom or its negation
- clause → a disjunction of literals
- CNF → a conjunction of clauses
- complementary literals → P and ¬P

Takeaway: Resolution vocabulary.

Source: Book 2 (Exam examples), Q323, p. 41

LOGIC A54. [MULTI · Easy]

Which are propositions? (Select ALL that apply.)

A. 'Close the door.'
B. 'Is it cold?'
C. '2 + 2 = 4.'
D. 'It is raining.'

Answer: C. '2 + 2 = 4.'; D. 'It is raining.'

Option notes:

- A: Incorrect: a command
- B: Incorrect: a question
- C: Correct: true
- D: Correct: true or false

Takeaway: Declarative statements only.

Source: Book 2 (Exam examples), Q324, p. 42

LOGIC A55. [MCQ · Medium]

R = 'it rains', W = 'the road is wet', U = 'I take an umbrella'. 'If it rains, the road is wet and I take an umbrella.'

A. (R ⇒ W) ∧ U
B. R ∧ W ∧ U
C. (W ∧ U) ⇒ R
D. R ⇒ (W ∧ U)

Answer: D. R ⇒ (W ∧ U)

Option notes:

- A: U is part of the conclusion.
- B: No 'if'.
- C: Converse.

Takeaway: If-then → ⇒.

Source: Book 2 (Exam examples), Q325, p. 42

LOGIC A56. [MCQ · Easy]

'You pass if and only if your mark is at least 50' with P = pass, M = mark ≥ 50:

A. P ⇒ M
B. P ⇔ M
C. M ⇒ P
D. P ∧ M

Answer: B. P ⇔ M

Option notes:

- A: One direction only.
- C: One direction only.
- D: Not conditional.

Takeaway: Iff → ⇔.

Source: Book 2 (Exam examples), Q326, p. 42

LOGIC A57. [BLANKS · Medium · calculation]

KB = P ∧ (P ⇒ Q). Of the four models, KB is true in ___ model(s); in it Q is ___ (true/false), so KB ___ Q (entails / does not entail).

(1) models = ____
(2) Q = ____
(3) relation = ____

Answer: models = 1; Q = true; relation = entails

Takeaway: Model checking.

Source: Book 2 (Exam examples), Q327, p. 42

LOGIC A58. [TF · Medium]

KB = (P ∨ Q) ∧ ¬P entails Q.

Answer: True

Why: KB true only when P = F, then P ∨ Q needs Q = T.

Takeaway: Disjunctive syllogism.

Source: Book 2 (Exam examples), Q328, p. 42

LOGIC A59. [TF · Medium]

KB = P ⇒ Q entails Q.

Answer: False

Why: Model P = F, Q = F makes KB true and Q false.

Takeaway: Need P as well.

Source: Book 2 (Exam examples), Q329, p. 42

LOGIC A60. [BLANK · Easy · calculation]

How many models must be checked for a KB with 5 symbols?

Answer: 32

Takeaway: 2^5.

Source: Book 2 (Exam examples), Q330, p. 42

LOGIC A61. [MCQ · Medium]

α ⊨ β iff M(α) ___ M(β).

A. ⊆
B. =
C. ∩ = ∅
D. ⊇

Answer: A. ⊆

Option notes:

- B: Too strong.
- C: That would be contradiction.
- D: Backwards.

Takeaway: Every α-model is a β-model.

Source: Book 2 (Exam examples), Q331, p. 42

LOGIC A62. [TF · Medium]

α ⊨ β iff α ∧ ¬β is unsatisfiable.

Answer: True

Why: Basis of proof by contradiction.

Takeaway: Refutation.

Source: Book 2 (Exam examples), Q332, p. 42

LOGIC A63. [TF · Easy]

P ∧ ¬P is satisfiable.

Answer: False

Why: No model makes it true.

Takeaway: Contradiction.

Source: Book 2 (Exam examples), Q333, p. 42

LOGIC A64. [BLANK · Easy]

From P and P ⇒ Q, the rule ___ derives Q.

Answer: Modus Ponens
(Also accepted: modus ponens, MP)

Takeaway: Theorem proving.

Source: Book 2 (Exam examples), Q334, p. 42

LOGIC A65. [MATCH · Medium]

Match the inference rule / equivalence.

1. Modus Ponens
2. And-elimination
3. implication elimination
4. biconditional elimination

A. $\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$
B. from $\alpha \Rightarrow \beta$ and α infer β
C. $\alpha \Leftrightarrow \beta \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
D. from $\alpha \wedge \beta$ infer α

Answer: 1 → B, 2 → D, 3 → A, 4 → C

Working:

- Modus Ponens → from $\alpha \Rightarrow \beta$ and α infer β
- And-elimination → from $\alpha \wedge \beta$ infer α
- implication elimination → $\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$
- biconditional elimination → $\alpha \Leftrightarrow \beta \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$

Takeaway: Module 7 rules.

Source: Book 2 (Exam examples), Q335, p. 42

LOGIC A66. [BLANKS · Medium]

Resolution: clauses C1 P, C2 ¬P ∨ Q, C3 ¬Q. Resolving C1 and C2 gives ___; resolving that with C3 gives ___ (write 'empty').

(1) C1 + C2 = ____
(2) with C3 = ____

Answer: C1 + C2 = Q; with C3 = empty

Takeaway: Empty clause ⇒ KB ⊨ Q.

Source: Book 2 (Exam examples), Q336, p. 42

LOGIC A67. [MCQ · Medium]

Resolve (P ∨ Q ∨ R) with (¬Q ∨ S):

A. P ∨ R
B. P ∨ R ∨ S
C. empty clause
D. P ∨ Q ∨ R ∨ S

Answer: B. P ∨ R ∨ S

Option notes:

- A: S is kept.
- C: Other literals remain.
- D: Q and ¬Q cancel.

Takeaway: Drop the complementary pair, keep the rest.

Source: Book 2 (Exam examples), Q337, p. 43

LOGIC A68. [MCQ · Medium]

Resolve (A ∨ B) with (¬A ∨ B):

A. B
B. empty clause
C. A ∨ ¬A
D. B ∨ B ∨ A

Answer: A. B

Option notes:

- B: B remains.
- C: Not a resolvent.
- D: A cancels; duplicates merge.

Takeaway: Factor duplicates.

Source: Book 2 (Exam examples), Q338, p. 43

LOGIC A69. [MCQ · Medium]

Convert to CNF: A ⇒ (B ∧ C).

A. (A ∨ B) ∧ (A ∨ C)
B. ¬A ∧ B ∧ C
C. ¬A ∨ (B ∧ C)
D. (¬A ∨ B) ∧ (¬A ∨ C)

Answer: D. (¬A ∨ B) ∧ (¬A ∨ C)

Option notes:

- A: Lost the negation.
- B: Wrong distribution.
- C: Not CNF yet: distribute.

Takeaway: Eliminate ⇒, then distribute.

Source: Book 2 (Exam examples), Q339, p. 43

LOGIC A70. [MCQ · Medium]

Convert to CNF: ¬(A ∨ B) ⇒ C.

A. (¬A ∧ ¬B) ∨ C
B. A ∨ B ∨ C
C. ¬A ∨ ¬B ∨ C
D. A ∧ B ∧ C

Answer: B. A ∨ B ∨ C

Option notes:

- A: Not CNF and wrong sign.
- C: Double negation removes the ¬.
- D: Wrong connective.

Takeaway: ¬¬(A ∨ B) ∨ C.

Source: Book 2 (Exam examples), Q340, p. 43

LOGIC A71. [MCQ · Hard]

Convert to CNF (Wumpus style): B1,1 ⇔ (P1,2 ∨ P2,1).

A. (¬B1,1 ∨ P1,2 ∨ P2,1)
B. (B1,1 ∨ P1,2 ∨ P2,1) ∧ (¬P1,2 ∨ ¬B1,1)
C. (¬B1,1 ∨ P1,2) ∧ (¬B1,1 ∨ P2,1)
D. (¬B1,1 ∨ P1,2 ∨ P2,1) ∧ (¬P1,2 ∨ B1,1) ∧ (¬P2,1 ∨ B1,1)

Answer: D. (¬B1,1 ∨ P1,2 ∨ P2,1) ∧ (¬P1,2 ∨ B1,1) ∧ (¬P2,1 ∨ B1,1)

Option notes:

- A: Only one direction.
- B: Signs wrong.
- C: That is B ⇒ (P ∧ P).

Takeaway: ⇔ elim → ⇒ elim → De Morgan → distribute.

Source: Book 2 (Exam examples), Q341, p. 43

LOGIC A72. [MCQ · Easy]

In proof by contradiction you negate:

A. only the query
B. nothing
C. every clause
D. the whole KB

Answer: A. only the query

Option notes:

- B: You must negate the query.
- C: No.
- D: KB is assumed true.

Takeaway: KB ∧ ¬query.

Source: Book 2 (Exam examples), Q342, p. 43

LOGIC A73. [MULTI · Medium]

Three inference methods for propositional logic in this course: (Select ALL that apply.)

A. theorem proving
B. model checking
C. inference by enumeration
D. proof by contradiction with resolution

Answer: A. theorem proving; B. model checking; D. proof by contradiction with resolution

Option notes:

- A: Correct:

- B: Correct:
- C: Incorrect: that is for Bayesian networks
- D: Correct:

Takeaway: Three PL methods.

Source: Book 2 (Exam examples), Q343, p. 43

LOGIC A74. [MCQ · Medium]

'All students study' in FOL:

A. ∃x Student(x) ⇒ Studies(x)
B. Student(x) ⇒ Studies(x)
C. ∀x Student(x) ⇒ Studies(x)
D. ∀x Student(x) ∧ Studies(x)

Answer: C. ∀x Student(x) ⇒ Studies(x)

Option notes:

- A: Wrong quantifier.
- B: Free variable.
- D: Says everything is a studying student.

Takeaway: ∀ pairs with ⇒.

Source: Book 2 (Exam examples), Q344, p. 43

LOGIC A75. [MCQ · Medium]

'Some student is enrolled in AI':

A. ∀x Enrolled(x, AI) ⇒ Student(x)
B. ∃x Student(x) ∧ Enrolled(x, AI)
C. ∃x Student(x) ⇒ Enrolled(x, AI)
D. ∀x Student(x) ∧ Enrolled(x, AI)

Answer: B. ∃x Student(x) ∧ Enrolled(x, AI)

Option notes:

- A: Different claim.
- C: Vacuously true for any non-student.
- D: Too strong.

Takeaway: ∃ pairs with ∧.

Source: Book 2 (Exam examples), Q345, p. 43

LOGIC A76. [MCQ · Medium]

'Nothing is perfect':

A. ∃x ¬Perfect(x)
B. ∀x Perfect(x) ⇒ ¬Perfect(x)
C. ¬∃x Perfect(x)
D. ¬∀x Perfect(x)

Answer: C. ¬∃x Perfect(x)

Option notes:

- A: Only says something is imperfect.

- B: Odd but not the slide form.
- D: Not everything is perfect: weaker.

Takeaway: Equivalently ∀x ¬Perfect(x).

Source: Book 2 (Exam examples), Q346, p. 43

LOGIC A77. [MCQ · Easy]

'Every king is a person':

A. ∃x King(x) ∧ Person(x)
B. ∀x King(x) ⇒ Person(x)
C. ∀x King(x) ∧ Person(x)
D. ∀x Person(x) ⇒ King(x)

Answer: B. ∀x King(x) ⇒ Person(x)

Option notes:

- A: Only some.
- C: Everything is a king.
- D: Converse.

Takeaway: ∀ with ⇒.

Source: Book 2 (Exam examples), Q347, p. 43

LOGIC A78. [MCQ · Medium]

'Every parent is older than their child':

A. ∀x∀y Parent(x, y) ⇒ Older(x, y)
B. ∀x∃y Parent(x, y) ∧ Older(x, y)
C. ∀x Parent(x) ⇒ Older(x)
D. ∃x∀y Parent(x, y) ⇒ Older(x, y)

Answer: A. ∀x∀y Parent(x, y) ⇒ Older(x, y)

Option notes:

- B: Says everyone is a parent.
- C: Arity wrong.
- D: Only one x.

Takeaway: Two universals.

Source: Book 2 (Exam examples), Q348, p. 43

LOGIC A79. [MCQ · Hard]

'Everyone loves someone' vs 'There is someone everyone loves':

A. ∃y∀x Loves(x, y); ∀x∃y Loves(x, y)
B. ∀x∀y Loves(x, y); ∃x∃y Loves(x, y)
C. Both are ∀x∃y Loves(x, y)
D. ∀x∃y Loves(x, y); ∃y∀x Loves(x, y)

Answer: D. ∀x∃y Loves(x, y); ∃y∀x Loves(x, y)

Option notes:

- A: Swapped.
- B: Wrong quantifiers.

- C: Order matters.

Takeaway: Quantifier order matters.

Source: Book 2 (Exam examples), Q349, p. 43

LOGIC A80. [MCQ · Medium]

Why is ∀x Student(x) ∧ Studies(x) usually wrong for 'all students study'?

A. it is fine
B. ∧ is not allowed in FOL
C. it says everything is a student and studies
D. wrong quantifier

Answer: C. it says everything is a student and studies

Option notes:

- A: No.
- B: It is allowed.
- D: ∀ is right; the connective is wrong.

Takeaway: Use ⇒ with ∀.

Source: Book 2 (Exam examples), Q350, p. 44

LOGIC A81. [MCQ · Hard]

Why is ∃x Student(x) ⇒ Enrolled(x, AI) wrong for 'some student is enrolled'?

A. the predicate arity is wrong
B. it is true whenever some object is not a student (vacuous truth)
C. ∃ cannot be used
D. it is correct

Answer: B. it is true whenever some object is not a student (vacuous truth)

Option notes:

- A: Arity is fine.
- C: ∃ is right; connective wrong.
- D: No.

Takeaway: Use ∧ with ∃.

Source: Book 2 (Exam examples), Q351, p. 44

LOGIC A82. [MCQ · Medium]

By De Morgan for quantifiers, ∀x ¬P(x) ≡

A. ¬∃x P(x)
B. ∃x ¬P(x)
C. ∀x P(x)
D. ¬∀x P(x)

Answer: A. ¬∃x P(x)

Option notes:

- B: That is ¬∀x P(x).
- C: Negation lost.
- D: Different.

Takeaway: Push ¬ through, swap quantifier.

Source: Book 2 (Exam examples), Q352, p. 44

LOGIC A83. [MCQ · Medium]

¬∀x Passed(x) ('not everyone passed') ≡

A. ∀x ¬Passed(x)
B. ∃x Passed(x)
C. ¬∃x Passed(x)
D. ∃x ¬Passed(x)

Answer: D. ∃x ¬Passed(x)

Option notes:

- A: That says nobody passed.
- B: Someone passed.
- C: Also nobody passed.

Takeaway: Some x failed.

Source: Book 2 (Exam examples), Q353, p. 44

LOGIC A84. [MATCH · Medium]

Match each FOL expression to its kind.

1. Father(John)
2. Brother(Richard, John)
3. John
4. Raining

A. zero-ary predicate (propositional symbol)
B. function term naming an object
C. binary predicate (a sentence)
D. constant term

Answer: 1 → B, 2 → C, 3 → D, 4 → A

Working:

- Father(John) → function term naming an object
- Brother(Richard, John) → binary predicate (a sentence)
- John → constant term
- Raining → zero-ary predicate (propositional symbol)

Takeaway: A term names; a sentence is true/false.

Source: Book 2 (Exam examples), Q354, p. 44

LOGIC A85. [BLANK · Easy]

The number of arguments a predicate takes is its ___.

Answer: arity

Takeaway: Brother has arity 2.

Source: Book 2 (Exam examples), Q355, p. 44

LOGIC A86. [MCQ · Easy]

Universal instantiation of ∀x Student(x) ⇒ Studies(x) with Mia gives:

A. Student(Mia) ⇒ Studies(Mia)
B. ∀x Student(Mia)
C. Student(k) ⇒ Studies(k) for a new k
D. Studies(Mia)

Answer: A. Student(Mia) ⇒ Studies(Mia)

Option notes:

- B: Wrong substitution.
- C: That is EI style.
- D: Needs Modus Ponens and Student(Mia).

Takeaway: Replace x by a ground term.

Source: Book 2 (Exam examples), Q356, p. 44

LOGIC A87. [MCQ · Medium]

Existential instantiation of ∃x Student(x) ∧ Enrolled(x) gives Student(k) ∧ Enrolled(k), where k must be:

A. any constant already in the KB
B. every constant in turn
C. a new constant not appearing elsewhere in the KB
D. a variable

Answer: C. a new constant not appearing elsewhere in the KB

Option notes:

- A: Could assert a false identity.
- B: That is UI.
- D: Must be ground.

Takeaway: Skolem constant.

Source: Book 2 (Exam examples), Q357, p. 44

LOGIC A88. [TF · Medium]

UI can be applied many times with different ground terms; EI is applied once.

Answer: True

Why: EI introduces one fresh constant.

Takeaway: UI many, EI once.

Source: Book 2 (Exam examples), Q358, p. 44

LOGIC A89. [BLANKS · Medium]

KB: ∀x King(x) ∧ Greedy(x) ⇒ Evil(x), King(John), Greedy(John). Using UI with x ↦ ___ and Modus Ponens we infer ___.

(1) x ↦ = _____
(2) inferred = _____

Answer: x ↦ = John; inferred = Evil(John)

Takeaway: Classic example.

Source: Book 2 (Exam examples), Q359, p. 44

LOGIC A90. [MCQ · Medium]

The main practical problem with propositionalising FOL is:

A. EI needs a variable
B. UI is unsound
C. too many ground instances for large domains
D. entailment is lost

Answer: C. too many ground instances for large domains

Option notes:

- A: It needs a new constant.
- B: It is sound.
- D: Entailment is preserved.

Takeaway: Blow-up.

Source: Book 2 (Exam examples), Q360, p. 44

LOGIC A91. [MATCH · Easy]

Match the reasoning type.

1. deductive
2. inductive

A. pattern in observations supports a probable conclusion
B. general rule applied to a specific case; certain if premises true

Answer: 1 → B, 2 → A

Working:

- deductive → general rule applied to a specific case; certain if premises true
- inductive → pattern in observations supports a probable conclusion

Takeaway: Certain vs probable.

Source: Book 2 (Exam examples), Q361, p. 44

LOGIC A92. [MCQ · Easy]

'The last five students who attended every lab passed; Leo attended every lab; Leo will probably pass.' This is:

A. model checking
B. abductive
C. inductive
D. deductive

Answer: C. inductive

Option notes:

- A: A proof method.
- B: Not in the slides as this example.
- D: Not certain.

Takeaway: Probably → inductive.

Source: Book 2 (Exam examples), Q362, p. 45

LOGIC A93. [MATCH · Easy]

Match the kind of inference.

1. logical inference
2. machine-learning inference

A. a trained classifier labels a new flower Iris setosa
B. a rule applied to facts derives that Mia passed

Answer: 1 → B, 2 → A

Working:

- logical inference → a rule applied to facts derives that Mia passed
- machine-learning inference → a trained classifier labels a new flower Iris setosa

Takeaway: Rules vs trained models.

Source: Book 2 (Exam examples), Q363, p. 45

LOGIC A94. [BLANKS · Easy · calculation]

P = F, Q = T. P ⇒ Q ___; Q ⇒ P ___; P ⇔ Q ___ (T/F).

(1) P ⇒ Q = _____
(2) Q ⇒ P = _____
(3) P ⇔ Q = _____

Answer: P ⇒ Q = T; Q ⇒ P = F; P ⇔ Q = F

Takeaway: Only T ⇒ F is false.

Source: Book 2 (Exam examples), Mixed practice paper Q31, p. 55

LOGIC A95. [MCQ · Medium]

Resolve (¬A ∨ B) with (A ∨ C):

A. B ∨ C
B. A ∨ B ∨ C
C. empty clause
D. B ∧ C

Answer: A. B ∨ C

Option notes:

- B: A cancels.
- C: B, C remain.
- D: Resolvents are disjunctions.

Takeaway: Drop the complementary pair.

Source: Book 2 (Exam examples), Mixed practice paper Q32, p. 55

LOGIC A96. [MCQ · Medium]

'No student failed' in FOL:

A. ¬∀x Student(x) ⇒ Failed(x)
B. ∃x Student(x) ∧ ¬Failed(x)

C. ¬∃x Student(x) ∧ Failed(x)
D. ∀x Student(x) ∧ ¬Failed(x)

Answer: C. ¬∃x Student(x) ∧ Failed(x)

Option notes:

- A: Not all failed: weaker.
- B: Says some student passed.
- D: Says everything is a student.

Takeaway: Equivalently ∀x Student(x) ⇒ ¬Failed(x).

Source: Book 2 (Exam examples), Mixed practice paper Q33, p. 55

LOGIC A97. [MCQ · Medium]

Existential instantiation replaces the variable with:

A. any constant already in the KB
B. every constant in turn
C. a variable
D. a brand-new constant

Answer: D. a brand-new constant

Option notes:

- A: Could be false.
- B: That is UI.
- C: Must be ground.

Takeaway: Skolem constant.

Source: Book 2 (Exam examples), Mixed practice paper Q34, p. 55

LOGIC A98. [BLANKS · Easy]

P = F, Q = T: P ⇒ Q = ___; Q ⇒ P = ___ (T/F).

(1) P ⇒ Q = _____
(2) Q ⇒ P = _____

Answer: P ⇒ Q = T; Q ⇒ P = F

Takeaway: Only T ⇒ F is false.

Source: Book 3 (AI from zero), Practice 25.1, p. 102

LOGIC A99. [MCQ · Medium]

CNF of ¬(P ∧ Q) ∨ R:

A. P ∨ Q ∨ R
B. ¬P ∨ ¬Q ∨ R
C. ¬P ∧ ¬Q ∧ R
D. (¬P ∧ ¬Q) ∨ R

Answer: B. ¬P ∨ ¬Q ∨ R

Option notes:

- A: Negations lost.
- C: Wrong connectives.
- D: De Morgan on ∧ gives ∨.

Takeaway: One clause.

Source: Book 3 (AI from zero), Practice 25.2, p. 103

LOGIC A100. [MCQ · Medium]

Resolve (A ∨ ¬B) with (B ∨ C):

A. ¬B ∨ C
B. empty
C. A ∨ B ∨ C
D. A ∨ C

Answer: D. A ∨ C

Option notes:

- A: A remains.
- B: Literals remain.
- C: B cancels.

Takeaway: Drop B, ¬B.

Source: Book 3 (AI from zero), Practice 25.3, p. 103

LOGIC A101. [MCQ · Easy]

FOL: 'Every dog is an animal.'

A. ∃x Dog(x) ∧ Animal(x)
B. ∀x Animal(x) ⇒ Dog(x)
C. ∀x Dog(x) ⇒ Animal(x)
D. ∀x Dog(x) ∧ Animal(x)

Answer: C. ∀x Dog(x) ⇒ Animal(x)

Option notes:

- A: Only some.
- B: Converse.
- D: Everything is a dog.

Takeaway: ∀ with ⇒.

Source: Book 3 (AI from zero), Practice 25.4, p. 103

LOGIC A102. [BLANK · Easy · calculation]

How many models for 4 symbols?

Answer: 16

Takeaway: 2^4.

Source: Book 3 (AI from zero), Practice 25.5, p. 103

LOGIC A103. [MCQ · Easy]

Which instantiation needs a new constant?

A. existential
B. universal
C. neither
D. both

Answer: A. existential

Option notes:

- B: Any ground term.
- C: EI does.
- D: Only EI.

Takeaway: Skolem constant.

Source: Book 3 (AI from zero), Practice 25.6, p. 103

LOGIC A104. [TF · Medium]

With P = T, Q = F, R = T, (P ∨ Q) ⇒ ¬R is true.

Answer: False

Why: P ∨ Q = T, ¬R = F, T ⇒ F = F.

Takeaway: Evaluate inside out.

Source: Book 3 (AI from zero), Chapter 25, p. 99

LOGIC A105. [MCQ · Hard]

CNF of P ⇔ (Q ∨ R):

A. (¬P ∨ Q) ∧ (¬P ∨ R)
B. (¬P ∨ Q ∨ R) ∧ (¬Q ∨ P) ∧ (¬R ∨ P)
C. (P ∨ Q ∨ R)
D. (¬P ∨ Q ∨ R) ∧ (Q ∨ R ∨ P)

Answer: B. (¬P ∨ Q ∨ R) ∧ (¬Q ∨ P) ∧ (¬R ∨ P)

Working:

- (P ⇒ (Q ∨ R)) ∧ ((Q ∨ R) ⇒ P)
- (¬P ∨ Q ∨ R) ∧ (¬(Q ∨ R) ∨ P)
- (¬P ∨ Q ∨ R) ∧ ((¬Q ∧ ¬R) ∨ P)
- (¬P ∨ Q ∨ R) ∧ (¬Q ∨ P) ∧ (¬R ∨ P)

Option notes:

- A: That is P ⇒ (Q ∧ R).
- C: Lost the structure.
- D: Second direction wrong.

Takeaway: Three clauses.

Source: Book 3 (AI from zero), Chapter 25, p. 99

LOGIC A106. [TF · Easy]

The converse Q ⇒ P is equivalent to P ⇒ Q.

Answer: False

Why: Only the contrapositive ¬Q ⇒ ¬P is.

Takeaway: Wet road doesn't mean it rained.

Source: Book 3 (AI from zero), Chapter 25, p. 99

LOGIC A107. [MCQ · Medium]

KB: P, P ⇒ Q, Q ⇒ R. Resolution proof of R starts by adding:

A. ¬P
B. R
C. ¬R
D. ¬(P ⇒ Q)

Answer: C. ¬R

Option notes:

- A: Negate the query, not a fact.
- B: Adding the query proves nothing.
- D: Never negate the KB.

Takeaway: Then ¬Q, ¬P, □.

Source: Book 3 (AI from zero), Chapter 25, p. 99

LOGIC A108. [BLANKS · Medium · calculation]

Wumpus world, pits only in [1,2], [2,2], [3,1] considered: there are ___ possible models; the KB (no breeze in [1,1], breeze in [2,1]) is true in ___ of them.

(1) models = ____
(2) KB-true = ____

Answer: models = 8; KB-true = 3

Takeaway: α_1 = 'no pit in [1,2]' holds in all 3, so KB ⊨ α_1.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: wumpus entailment

LOGIC A109. [MATCH · Medium]

Match the term (AIMA).

1. valid
2. satisfiable
3. unsatisfiable

A. true in all models (a tautology)
B. true in at least one model
C. true in no model
D. true in exactly half the models

Answer: 1 → A, 2 → B, 3 → C

Working:

- valid → true in all models (a tautology)
- satisfiable → true in at least one model
- unsatisfiable → true in no model

Takeaway: Deduction theorem: α ⊨ β iff α ⇒ β is valid.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: validity and satisfiability

LOGIC A110. [TF · Medium]

Deduction theorem: α ⊨ β if and only if the sentence α ⇒ β is valid.

Answer: True

Why: And iff α ∧ ¬β is unsatisfiable (refutation).

Takeaway: Ties entailment to validity.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: deduction theorem

LOGIC A111. [TF · Hard]

(Extra, not in your slides) Propositional logic is monotonic: if KB ⊨ α then KB ∧ β ⊨ α for any β.

Answer: True

Why: Adding knowledge never removes conclusions.

Takeaway: Monotonicity.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: monotonicity

LOGIC A112. [MULTI · Medium]

Which are true of inference algorithms (AIMA)? (Select ALL that apply.)

A. resolution is refutation-complete for propositional logic
B. complete = can derive every entailed sentence
C. model checking is complete for first-order logic with infinite domains
D. sound = derives only entailed sentences

Answer: A. resolution is refutation-complete for propositional logic; B. complete = can derive every entailed sentence; D. sound = derives only entailed sentences

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: infinite models cannot be enumerated
- D: Correct:

Takeaway: Soundness and completeness.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: soundness/completeness

LOGIC A113. [MCQ · Hard]

(Extra, not in your slides) A Horn clause is a disjunction of literals with:

A. at most one positive literal
B. exactly two literals
C. no negative literals
D. only positive literals

Answer: A. at most one positive literal

Option notes:

- B: That is 2-CNF.
- C: Wrong.
- D: Wrong.

Takeaway: Forward/backward chaining decide Horn KBs in linear time.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: Horn clauses

LOGIC A114. [MCQ · Hard]

AIMA's CNF example: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ becomes:

A. $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$
B. $(\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}$
C. $(\neg B_{1,1} \vee P_{1,2}) \wedge (\neg B_{1,1} \vee P_{2,1})$
D. $(B_{1,1} \vee P_{1,2} \vee P_{2,1})$

Answer: A. $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

Option notes:

- B: Not CNF.
- C: That says both pits.
- D: Wrong sign and only one clause.

Takeaway: Eliminate $\Leftrightarrow$, $\Rightarrow$, push $\neg$, distribute.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: CNF conversion

LOGIC A115. [MCQ · Medium]

(Extra, not in your slides) Forward chaining is ___-driven; backward chaining is ___-driven.

A. random; goal
B. goal; data
C. data; goal
D. data; data

Answer: C. data; goal

Option notes:

- A: No.
- B: Reversed.
- D: Backward works from the query.

Takeaway: FC derives everything; BC only what the query needs.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 7: forward/backward chaining

LOGIC A116. [MCQ · Easy]

'Brothers are siblings' in FOL:

A. $\exists x,y$ Brother(x, y) $\Rightarrow$ Sibling(x, y)
B. Brother(x, y) $\Leftrightarrow$ Sibling(x, y)
C. $\forall x,y$ Brother(x, y) $\Rightarrow$ Sibling(x, y)
D. $\forall x,y$ Brother(x, y) $\wedge$ Sibling(x, y)

Answer: C. $\forall x,y$ Brother(x, y) $\Rightarrow$ Sibling(x, y)

Option notes:

- A: Wrong quantifier.
- B: Free variables, and siblings need not be brothers.
- D: Says everyone is everyone's brother.

Takeaway: $\forall$ with $\Rightarrow$.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 8: FOL sentences

LOGIC A117. [MCQ · Hard]

'King John has a crown' in FOL is:

A. ∀x Crown(x) ∧ OnHead(x, John)
B. Crown(John)
C. ∃x Crown(x) ⇒ OnHead(x, John)
D. ∃x Crown(x) ∧ OnHead(x, John)

Answer: D. ∃x Crown(x) ∧ OnHead(x, John)

Option notes:

- A: Says everything is his crown.
- B: Says John is a crown.
- C: True if anything is not a crown.

Takeaway: ∃ with ∧.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 8: quantifiers

LOGIC A118. [MATCH · Medium]

AIMA's FOL elements for 'Richard the Lionheart is King John's brother'.

1. Richard, John
2. Brother
3. LeftLeg

A. constant symbols (objects)
B. function symbol (maps an object to an object)
C. predicate symbol (relation)
D. quantifier

Answer: 1 → A, 2 → C, 3 → B

Working:

- Richard, John → constant symbols (objects)
- Brother → predicate symbol (relation)
- LeftLeg → function symbol (maps an object to an object)

Takeaway: Objects, relations, functions.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 8: FOL syntax

LOGIC A119. [BLANK · Hard]

(Extra, not in your slides) Unify Knows(John, x) and Knows(y, Mother(y)). The unifier is {y/John, x/___}.

Answer: Mother(John)
(Also accepted: mother(john))

Takeaway: Substitute y first.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 9: unification

LOGIC A120. [TF · Hard]

(Extra, not in your slides) Knows(John, x) and Knows(x, Elizabeth) cannot be unified as written, but can after standardising apart the variables.

Answer: True

Why: x cannot be both John and Elizabeth; rename one x to z.

Takeaway: Standardise apart.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 9: unification

LOGIC A121. [MCQ · Hard]

(Extra, not in your slides) AIMA crime KB: 'It is a crime for an American to sell weapons to hostile nations.' The rule is:

A. American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) ⇒ Criminal(x)
B. ∃x Criminal(x)
C. American(x) ∨ Weapon(y) ⇒ Criminal(x)
D. Criminal(x) ⇒ American(x) ∧ Weapon(y)

Answer: A. American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) ⇒ Criminal(x)

Option notes:

- B: Loses the conditions.
- C: Needs all conditions (∧).
- D: Reversed.

Takeaway: A definite clause; West is proved Criminal by chaining.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 9: forward chaining example

Part B: calculation drills (80)

LOGIC B1. [TF · Medium · calculation]

In the model P = T, Q = T, R = T, the sentence (Q ⇔ R) ⇒ R is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B2. [TF · Medium · calculation]

In the model P = T, Q = T, R = T, the sentence Q is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B3. [TF · Medium · calculation]

In the model P = F, Q = T, R = T, the sentence R ⇒ (Q ⇒ P) is true.

Answer: False

Why: It evaluates to F.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: F.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B4. [TF · Medium · calculation]

In the model P = T, Q = T, R = F, the sentence Q is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B5. [TF · Medium · calculation]

In the model P = T, Q = T, R = F, the sentence (Q ∧ ¬P) ∨ (Q ⇒ Q) is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B6. [TF · Medium · calculation]

In the model P = F, Q = F, R = T, the sentence (¬Q ⇒ Q) ∨ (¬Q ⇒ P) is true.

Answer: False

Why: It evaluates to F.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: F.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B7. [TF · Medium · calculation]

In the model P = F, Q = F, R = T, the sentence (¬P ⇒ Q) ⇒ R is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B8. [TF · Medium · calculation]

In the model P = F, Q = T, R = F, the sentence ¬Q is true.

Answer: False

Why: It evaluates to F.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: F.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B9. [TF · Medium · calculation]

In the model P = T, Q = T, R = F, the sentence P is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B10. [TF · Medium · calculation]

In the model P = F, Q = F, R = F, the sentence (Q ∨ R) ⇔ (Q ∧ R) is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.

- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B11. [TF · Medium · calculation]

In the model P = F, Q = T, R = F, the sentence P is true.

Answer: False

Why: It evaluates to F.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: F.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B12. [TF · Medium · calculation]

In the model P = F, Q = T, R = F, the sentence (Q ∧ ¬R) ∨ (¬R ⇒ R) is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B13. [TF · Medium · calculation]

In the model P = F, Q = T, R = T, the sentence (Q ∧ Q) ⇒ Q is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B14. [TF · Medium · calculation]

In the model P = T, Q = F, R = F, the sentence ¬Q is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B15. [TF · Medium · calculation]

In the model P = F, Q = T, R = F, the sentence Q ∨ ¬R is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B16. [TF · Medium · calculation]

In the model P = T, Q = T, R = T, the sentence Q ⇔ (R ∧ Q) is true.

Answer: True

Why: It evaluates to T.

Working:

- Evaluate innermost brackets first, then the main connective. Remember: A ⇒ B is false only when A = T and B = F; A ⇔ B is true when both sides match.
- Result: T.

Takeaway: Implication with a false premise is vacuously TRUE.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B17. [BLANKS · Hard · calculation]

KB = { R ⇒ P , ¬R ∨ ¬R }. Query γ = P ∧ R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 4; KB ⊨ γ? = No

Working:

- P | Q | R | R ⇒ P | ¬R ∨ ¬R | KB | γ
  T | T | T | T | F | F | T
  T | T | F | T | T | T | F
  T | F | T | T | F | F | T
  T | F | F | T | T | T | F
  F | T | T | F | F | F | F
  F | T | F | T | T | T | F
  F | F | T | F | F | F | F
  F | F | F | T | T | T | F

- KB is true in 4 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B18. [BLANKS · Hard · calculation]

KB = { R , R ⇔ P }. Query γ = R ∨ R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 2; KB ⊨ γ? = Yes

Working:

- P | Q | R | R | R ⇔ P | KB | γ
  T | T | T | T | T | T | T
  T | T | F | F | F | F | F
  T | F | T | T | T | T | T
  T | F | F | F | F | F | F
  F | T | T | T | F | F | T
  F | T | F | F | T | F | F
  F | F | T | T | F | F | T
  F | F | F | F | T | F | F
- KB is true in 2 model(s).
- γ is true in every one of them → KB ⊨ γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B19. [BLANKS · Hard · calculation]

KB = { Q ∧ P , R ⇒ ¬R }. Query γ = P ∨ R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 1; KB ⊨ γ? = Yes

Working:

- P | Q | R | Q ∧ P | R ⇒ ¬R | KB | γ
  T | T | T | T | F | F | T
  T | T | F | T | T | T | T
  T | F | T | F | F | F | T
  T | F | F | F | T | F | T
  F | T | T | F | F | F | T
  F | T | F | F | T | F | F
  F | F | T | F | F | F | T
  F | F | F | F | T | F | F
- KB is true in 1 model(s).
- γ is true in every one of them → KB ⊨ γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B20. [BLANKS · Hard · calculation]

KB = { Q ⇒ ¬P , Q ∨ ¬P }. Query γ = ¬R ∧ Q. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 4; KB ⊨ γ? = No

Working:

- P | Q | R | Q ⇒ ¬P | Q ∨ ¬P | KB | γ
  T | T | T | F | T | F | F
  T | T | F | F | T | F | T
  T | F | T | T | F | F | F
  T | F | F | T | F | F | F
  F | T | T | T | T | T | F
  F | T | F | T | T | T | T
  F | F | T | T | T | T | F
  F | F | F | T | T | T | F
- KB is true in 4 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B21. [BLANKS · Hard · calculation]

KB = { R ⇒ Q , ¬Q }. Query γ = ¬Q. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 2; KB ⊨ γ? = Yes

Working:

- P | Q | R | R ⇒ Q | ¬Q | KB | γ
  T | T | T | T | F | F | F
  T | T | F | T | F | F | F
  T | F | T | F | T | F | T
  T | F | F | T | T | T | T
  F | T | T | T | F | F | F
  F | T | F | T | F | F | F
  F | F | T | F | T | F | T
  F | F | F | T | T | T | T
- KB is true in 2 model(s).
- γ is true in every one of them → KB ⊨ γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B22. [BLANKS · Hard · calculation]

KB = { ¬P ⇒ ¬R , Q ⇒ P }. Query γ = ¬R ∧ ¬P. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 5; KB ⊨ γ? = No

Working:

- P | Q | R | ¬P ⇒ ¬R | Q ⇒ P | KB | γ
  T | T | T | T | T | T | F
  T | T | F | T | T | T | F
  T | F | T | T | T | T | F
  T | F | F | T | T | T | F
  F | T | T | F | F | F | F
  F | T | F | T | F | F | T
  F | F | T | F | T | F | F
  F | F | F | T | T | T | T
- KB is true in 5 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B23. [BLANKS · Hard · calculation]

KB = { ¬Q , Q ⇔ P }. Query γ = P ∨ R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 2; KB ⊨ γ? = No

Working:

- P | Q | R | ¬Q | Q ⇔ P | KB | γ
  T | T | T | F | T | F | T
  T | T | F | F | T | F | T
  T | F | T | T | F | F | T
  T | F | F | T | F | F | T
  F | T | T | F | F | F | T
  F | T | F | F | F | F | F
  F | F | T | T | T | T | T
  F | F | F | T | T | T | F
- KB is true in 2 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B24. [BLANKS · Hard · calculation]

KB = { ¬P ∧ R , P ⇒ ¬Q }. Query γ = Q ⇒ R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 2; KB ⊨ γ? = Yes

Working:

- P | Q | R | ¬P ∧ R | P ⇒ ¬Q | KB | γ
  T | T | T | F | F | F | T
  T | T | F | F | F | F | F
  T | F | T | F | T | F | T
  T | F | F | F | T | F | T
  F | T | T | T | T | T | T
  F | T | F | F | T | F | F
  F | F | T | T | T | T | T
  F | F | F | F | T | F | T
- KB is true in 2 model(s).
- γ is true in every one of them → KB ⊨ γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B25. [BLANKS · Hard · calculation]

KB = { Q ⇒ R , P ⇔ P }. Query γ = Q ⇒ ¬P. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 6; KB ⊨ γ? = No

Working:

- P | Q | R | Q ⇒ R | P ⇔ P | KB | γ
  T | T | T | T | T | T | F
  T | T | F | F | T | F | F
  T | F | T | T | T | T | T
  T | F | F | T | T | T | T
  F | T | T | T | T | T | T
  F | T | F | F | T | F | T
  F | F | T | T | T | T | T
  F | F | F | T | T | T | T
- KB is true in 6 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B26. [BLANKS · Hard · calculation]

KB = { R ⇔ P , ¬Q ⇒ Q }. Query γ = ¬R ⇔ Q. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 2; KB ⊨ γ? = No

Working:

- P | Q | R | R ⇔ P | ¬Q ⇒ Q | KB | γ
  T | T | T | T | T | T | F
  T | T | F | F | T | F | T
  T | F | T | T | F | F | T
  T | F | F | F | F | F | F
  F | T | T | F | T | F | F
  F | T | F | T | T | T | T
  F | F | T | F | F | F | T
  F | F | F | T | F | F | F
- KB is true in 2 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B27. [BLANKS · Hard · calculation]

KB = { P ⇒ P , ¬Q }. Query γ = R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 4; KB ⊨ γ? = No

Working:

- P | Q | R | P ⇒ P | ¬Q | KB | γ
  T | T | T | T | F | F | T
  T | T | F | T | F | F | F
  T | F | T | T | T | T | T
  T | F | F | T | T | T | F
  F | T | T | T | F | F | T
  F | T | F | T | F | F | F
  F | F | T | T | T | T | T
  F | F | F | T | T | T | F
- KB is true in 4 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B28. [BLANKS · Hard · calculation]

KB = { P ⇒ Q , ¬Q ⇒ ¬Q }. Query γ = ¬Q. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 6; KB ⊨ γ? = No

Working:

- P | Q | R | P ⇒ Q | ¬Q ⇒ ¬Q | KB | γ
  T | T | T | T | T | T | F

T | T | F | T | T | T | F
T | F | T | F | T | F | T
T | F | F | F | T | F | T
F | T | T | T | T | T | F
F | T | F | T | T | T | F
F | F | T | T | T | T | T
F | F | F | T | T | T | T

- KB is true in 6 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B29. [BLANKS · Hard · calculation]

KB = { R ∨ Q , R ∨ ¬R }. Query γ = R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 6; KB ⊨ γ? = No

Working:

- P | Q | R | R ∨ Q | R ∨ ¬R | KB | γ
  T | T | T | T | T | T | T
  T | T | F | T | T | T | F
  T | F | T | T | T | T | T
  T | F | F | F | T | F | F
  F | T | T | T | T | T | T
  F | T | F | T | T | T | F
  F | F | T | T | T | T | T
  F | F | F | F | T | F | F
- KB is true in 6 model(s).
- Some KB-true model makes γ false → KB does not entail γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B30. [BLANKS · Hard · calculation]

KB = { R ⇒ ¬Q , ¬Q ⇒ P }. Query γ = R ⇒ P. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 4; KB ⊨ γ? = Yes

Working:

- P | Q | R | R ⇒ ¬Q | ¬Q ⇒ P | KB | γ
  T | T | T | F | T | F | T
  T | T | F | T | T | T | T
  T | F | T | T | T | T | T
  T | F | F | T | T | T | T
  F | T | T | F | T | F | F

F | T | F | T | T | T | T
F | F | T | T | F | F | F
F | F | F | T | F | F | T

- KB is true in 4 model(s).
- γ is true in every one of them → KB ⊨ γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B31. [BLANKS · Hard · calculation]

KB = { P , P ∨ Q }. Query γ = P ∨ ¬R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 4; KB ⊨ γ? = Yes

Working:

- P | Q | R | P | P ∨ Q | KB | γ
  T | T | T | T | T | T | T
  T | T | F | T | T | T | T
  T | F | T | T | T | T | T
  T | F | F | T | T | T | T
  F | T | T | F | T | F | F
  F | T | F | F | T | F | T
  F | F | T | F | F | F | F
  F | F | F | F | F | F | T
- KB is true in 4 model(s).
- γ is true in every one of them → KB ⊨ γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B32. [BLANKS · Hard · calculation]

KB = { Q ⇒ R , Q ∧ R }. Query γ = R. By model checking over P, Q, R: (1) in how many of the 8 models is KB true? (2) Does KB ⊨ γ? (Yes/No)

(1) # KB-true models = ____
(2) KB ⊨ γ? = ____

Answer: # KB-true models = 2; KB ⊨ γ? = Yes

Working:

- P | Q | R | Q ⇒ R | Q ∧ R | KB | γ
  T | T | T | T | T | T | T
  T | T | F | F | F | F | F
  T | F | T | T | F | F | T
  T | F | F | T | F | F | F
  F | T | T | T | T | T | T
  F | T | F | F | F | F | F
  F | F | T | T | F | F | T
  F | F | F | T | F | F | F
- KB is true in 2 model(s).

- γ is true in every one of them → KB ⊨ γ.

Takeaway: KB ⊨ γ iff M(KB) ⊆ M(γ): no model where KB is true and γ is false.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B33. [MCQ · Medium]

Resolve the clauses (R ∨ ¬P) and (¬R ∨ Q). What is the resolvent?

A. ¬P ∨ Q
B. R ∨ ¬R
C. P ∨ ¬Q
D. R ∨ ¬P ∧ ¬R ∨ Q

Answer: A. ¬P ∨ Q

Working:

- Complementary pair: R and ¬R.
- Keep every other literal: ¬P ∨ Q.

Option notes:

- B: Resolution REMOVES the complementary pair, it doesn't keep it.
- C: Don't negate the remaining literals.
- D: That's just the conjunction of the inputs.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B34. [MCQ · Medium]

Resolve the clauses (S ∨ Q) and (¬S ∨ R). What is the resolvent?

A. S ∨ Q ∧ ¬S ∨ R
B. ¬Q ∨ ¬R
C. Q ∨ R
D. S ∨ ¬S

Answer: C. Q ∨ R

Working:

- Complementary pair: S and ¬S.
- Keep every other literal: Q ∨ R.

Option notes:

- A: That's just the conjunction of the inputs.
- B: Don't negate the remaining literals.
- D: Resolution REMOVES the complementary pair, it doesn't keep it.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B35. [MCQ · Medium]

Resolve the clauses (Q ∨ P) and (¬Q ∨ ¬S). What is the resolvent?

A. ¬P ∨ S
B. Q ∨ P ∧ ¬Q ∨ ¬S

C. Q ∨ ¬Q
D. P ∨ ¬S

Answer: D. P ∨ ¬S

Working:

- Complementary pair: Q and ¬Q.
- Keep every other literal: P ∨ ¬S.

Option notes:

- A: Don't negate the remaining literals.
- B: That's just the conjunction of the inputs.
- C: Resolution REMOVES the complementary pair, it doesn't keep it.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B36. [MCQ · Medium]

Resolve the clauses (Q ∨ ¬P) and (¬Q ∨ S). What is the resolvent?

A. P ∨ ¬S
B. Q ∨ ¬P ∧ ¬Q ∨ S
C. ¬P ∨ S
D. Q ∨ ¬Q

Answer: C. ¬P ∨ S

Working:

- Complementary pair: Q and ¬Q.
- Keep every other literal: ¬P ∨ S.

Option notes:

- A: Don't negate the remaining literals.
- B: That's just the conjunction of the inputs.
- D: Resolution REMOVES the complementary pair, it doesn't keep it.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B37. [MCQ · Medium]

Resolve the clauses (S ∨ Q) and (¬S ∨ P). What is the resolvent?

A. S ∨ Q ∧ ¬S ∨ P
B. S ∨ ¬S
C. Q ∨ P
D. ¬Q ∨ ¬P

Answer: C. Q ∨ P

Working:

- Complementary pair: S and ¬S.
- Keep every other literal: Q ∨ P.

Option notes:

- A: That's just the conjunction of the inputs.
- B: Resolution REMOVES the complementary pair, it doesn't keep it.

- D: Don't negate the remaining literals.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B38. [MCQ · Medium]

Resolve the clauses (P ∨ ¬Q) and (¬P ∨ ¬S). What is the resolvent?

A. ¬Q ∨ ¬S
B. P ∨ ¬Q ∧ ¬P ∨ ¬S
C. P ∨ ¬P
D. Q ∨ S

Answer: A. ¬Q ∨ ¬S

Working:

- Complementary pair: P and ¬P.
- Keep every other literal: ¬Q ∨ ¬S.

Option notes:

- B: That's just the conjunction of the inputs.
- C: Resolution REMOVES the complementary pair, it doesn't keep it.
- D: Don't negate the remaining literals.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B39. [MCQ · Medium]

Resolve the clauses (S ∨ R) and (¬S ∨ P). What is the resolvent?

A. S ∨ R ∧ ¬S ∨ P
B. S ∨ ¬S
C. R ∨ P
D. ¬R ∨ ¬P

Answer: C. R ∨ P

Working:

- Complementary pair: S and ¬S.
- Keep every other literal: R ∨ P.

Option notes:

- A: That's just the conjunction of the inputs.
- B: Resolution REMOVES the complementary pair, it doesn't keep it.
- D: Don't negate the remaining literals.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B40. [MCQ · Medium]

Resolve the clauses (R ∨ Q) and (¬R ∨ ¬S). What is the resolvent?

A. R ∨ Q ∧ ¬R ∨ ¬S
B. Q ∨ ¬S

C. R ∨ ¬R
D. ¬Q ∨ S

Answer: B. Q ∨ ¬S

Working:

- Complementary pair: R and ¬R.
- Keep every other literal: Q ∨ ¬S.

Option notes:

- A: That's just the conjunction of the inputs.
- C: Resolution REMOVES the complementary pair, it doesn't keep it.
- D: Don't negate the remaining literals.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B41. [MCQ · Medium]

Resolve the clauses (P ∨ ¬S) and (¬P ∨ Q). What is the resolvent?

A. S ∨ ¬Q
B. ¬S ∨ Q
C. P ∨ ¬S ∧ ¬P ∨ Q
D. P ∨ ¬P

Answer: B. ¬S ∨ Q

Working:

- Complementary pair: P and ¬P.
- Keep every other literal: ¬S ∨ Q.

Option notes:

- A: Don't negate the remaining literals.
- C: That's just the conjunction of the inputs.
- D: Resolution REMOVES the complementary pair, it doesn't keep it.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B42. [MCQ · Medium]

Resolve the clauses (R ∨ Q) and (¬R ∨ P). What is the resolvent?

A. ¬Q ∨ ¬P
B. R ∨ ¬R
C. R ∨ Q ∧ ¬R ∨ P
D. Q ∨ P

Answer: D. Q ∨ P

Working:

- Complementary pair: R and ¬R.
- Keep every other literal: Q ∨ P.

Option notes:

- A: Don't negate the remaining literals.
- B: Resolution REMOVES the complementary pair, it doesn't keep it.

- C: That's just the conjunction of the inputs.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B43. [MCQ · Medium]

Resolve the clauses (R ∨ Q) and (¬R ∨ ¬S). What is the resolvent?

A. R ∨ ¬R
B. ¬Q ∨ S
C. R ∨ Q ∧ ¬R ∨ ¬S
D. Q ∨ ¬S

Answer: D. Q ∨ ¬S

Working:

- Complementary pair: R and ¬R.
- Keep every other literal: Q ∨ ¬S.

Option notes:

- A: Resolution REMOVES the complementary pair, it doesn't keep it.
- B: Don't negate the remaining literals.
- C: That's just the conjunction of the inputs.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B44. [MCQ · Medium]

Resolve the clauses (Q ∨ ¬P) and (¬Q ∨ R). What is the resolvent?

A. Q ∨ ¬P ∧ ¬Q ∨ R
B. Q ∨ ¬Q
C. ¬P ∨ R
D. P ∨ ¬R

Answer: C. ¬P ∨ R

Working:

- Complementary pair: Q and ¬Q.
- Keep every other literal: ¬P ∨ R.

Option notes:

- A: That's just the conjunction of the inputs.
- B: Resolution REMOVES the complementary pair, it doesn't keep it.
- D: Don't negate the remaining literals.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B45. [MCQ · Medium]

Resolve the clauses (S ∨ ¬Q) and (¬S ∨ P). What is the resolvent?

A. ¬Q ∨ P
B. S ∨ ¬S

C. S ∨ ¬Q ∧ ¬S ∨ P
D. Q ∨ ¬P

Answer: A. ¬Q ∨ P

Working:

- Complementary pair: S and ¬S.
- Keep every other literal: ¬Q ∨ P.

Option notes:

- B: Resolution REMOVES the complementary pair, it doesn't keep it.
- C: That's just the conjunction of the inputs.
- D: Don't negate the remaining literals.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B46. [MCQ · Medium]

Resolve the clauses (P ∨ ¬S) and (¬P ∨ Q). What is the resolvent?

A. P ∨ ¬S ∧ ¬P ∨ Q
B. ¬S ∨ Q
C. S ∨ ¬Q
D. P ∨ ¬P

Answer: B. ¬S ∨ Q

Working:

- Complementary pair: P and ¬P.
- Keep every other literal: ¬S ∨ Q.

Option notes:

- A: That's just the conjunction of the inputs.
- C: Don't negate the remaining literals.
- D: Resolution REMOVES the complementary pair, it doesn't keep it.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B47. [MCQ · Medium]

Resolve the clauses (S ∨ ¬P) and (¬S ∨ R). What is the resolvent?

A. S ∨ ¬P ∧ ¬S ∨ R
B. ¬P ∨ R
C. S ∨ ¬S
D. P ∨ ¬R

Answer: B. ¬P ∨ R

Working:

- Complementary pair: S and ¬S.
- Keep every other literal: ¬P ∨ R.

Option notes:

- A: That's just the conjunction of the inputs.
- C: Resolution REMOVES the complementary pair, it doesn't keep it.

- D: Don't negate the remaining literals.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B48. [MCQ · Medium]

Resolve the clauses (P ∨ S) and (¬P ∨ ¬R). What is the resolvent?

A. P ∨ S ∧ ¬P ∨ ¬R
B. ¬S ∨ R
C. S ∨ ¬R
D. P ∨ ¬P

Answer: C. S ∨ ¬R

Working:

- Complementary pair: P and ¬P.
- Keep every other literal: S ∨ ¬R.

Option notes:

- A: That's just the conjunction of the inputs.
- B: Don't negate the remaining literals.
- D: Resolution REMOVES the complementary pair, it doesn't keep it.

Takeaway: Full resolution: drop the complementary pair, OR together everything that's left.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B49. [MCQ · Easy]

Which sentence is logically equivalent to P ⇔ Q?

A. P ∧ Q
B. (P ⇒ Q) ∧ (Q ⇒ P)
C. ¬P ⇒ ¬Q
D. Q ⇒ P

Answer: B. (P ⇒ Q) ∧ (Q ⇒ P)

Working:

- P ⇔ Q ≡ (P ⇒ Q) ∧ (Q ⇒ P) (biconditional elimination).

Option notes:

- A: Not equivalent: check a truth table.
- C: The inverse is not equivalent.
- D: The converse is not equivalent.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B50. [MCQ · Easy]

Which sentence is logically equivalent to P ∧ (Q ∨ R)?

A. (P ∧ Q) ∨ (P ∧ R)
B. P ∧ Q
C. Q ⇒ P
D. ¬P ⇒ ¬Q

Answer: A. (P ∧ Q) ∨ (P ∧ R)

Working:

- P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R) (distributivity of ∧ over ∨).

Option notes:

- B: Not equivalent: check a truth table.
- C: The converse is not equivalent.
- D: The inverse is not equivalent.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B51. [MCQ · Easy]

Which sentence is logically equivalent to P ∧ (Q ∨ R)?

A. P ∧ Q
B. ¬P ⇒ ¬Q
C. Q ⇒ P
D. (P ∧ Q) ∨ (P ∧ R)

Answer: D. (P ∧ Q) ∨ (P ∧ R)

Working:

- P ∧ (Q ∨ R) ≡ (P ∧ Q) ∨ (P ∧ R) (distributivity of ∧ over ∨).

Option notes:

- A: Not equivalent: check a truth table.
- B: The inverse is not equivalent.
- C: The converse is not equivalent.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B52. [MCQ · Easy]

Which sentence is logically equivalent to P ⇒ Q?

A. ¬P ∨ Q
B. ¬P ⇒ ¬Q
C. Q ⇒ P
D. P ∨ ¬Q

Answer: A. ¬P ∨ Q

Working:

- P ⇒ Q ≡ ¬P ∨ Q (implication elimination).

Option notes:

- B: That's the inverse, not equivalent.
- C: That's the converse, not equivalent.
- D: Negate the PREMISE: P ⇒ Q ≡ ¬P ∨ Q.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B53. [MCQ · Easy]

Which sentence is logically equivalent to P ⇒ Q?

A. ¬P ⇒ ¬Q
B. Q ⇒ P
C. P ∨ ¬Q
D. ¬P ∨ Q

Answer: D. ¬P ∨ Q

Working:

- P ⇒ Q ≡ ¬P ∨ Q (implication elimination).

Option notes:

- A: That's the inverse, not equivalent.
- B: That's the converse, not equivalent.
- C: Negate the PREMISE: P ⇒ Q ≡ ¬P ∨ Q.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B54. [MCQ · Easy]

Which sentence is logically equivalent to ¬(P ∧ Q)?

A. P ∨ Q
B. ¬P ∨ ¬Q
C. ¬P ∧ ¬Q
D. ¬P ∧ Q

Answer: B. ¬P ∨ ¬Q

Working:

- ¬(P ∧ Q) ≡ ¬P ∨ ¬Q (De Morgan).

Option notes:

- A: Negations can't just disappear.
- C: De Morgan flips ∧ ↔ ∨.
- D: Every literal must be negated.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B55. [MCQ · Easy]

Which sentence is logically equivalent to ¬(¬P)?

A. ¬P ⇒ ¬Q
B. Q ⇒ P
C. P
D. P ∧ Q

Answer: C. P

Working:

- ¬(¬P) ≡ P (double-negation elimination).

Option notes:

- A: The inverse is not equivalent.
- B: The converse is not equivalent.
- D: Not equivalent: check a truth table.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B56. [MCQ · Easy]

Which sentence is logically equivalent to ¬(P ∧ Q)?

A. P ∨ Q
B. ¬P ∨ ¬Q
C. ¬P ∧ ¬Q
D. ¬P ∧ Q

Answer: B. ¬P ∨ ¬Q

Working:

- ¬(P ∧ Q) ≡ ¬P ∨ ¬Q (De Morgan).

Option notes:

- A: Negations can't just disappear.
- C: De Morgan flips ∧ ↔ ∨.
- D: Every literal must be negated.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B57. [MCQ · Easy]

Which sentence is logically equivalent to ¬(P ∨ Q)?

A. P ∨ Q
B. ¬P ∧ ¬Q
C. ¬P ∧ Q
D. ¬P ∨ ¬Q

Answer: B. ¬P ∧ ¬Q

Working:

- ¬(P ∨ Q) ≡ ¬P ∧ ¬Q (De Morgan).

Option notes:

- A: Negations can't just disappear.
- C: Every literal must be negated.
- D: De Morgan flips ∧ ↔ ∨.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B58. [MCQ · Easy]

Which sentence is logically equivalent to P ⇒ Q?

A. P ∨ ¬Q
B. ¬P ∨ Q

C. ¬P ⇒ ¬Q
D. Q ⇒ P

Answer: B. ¬P ∨ Q

Working:

- P ⇒ Q ≡ ¬P ∨ Q (implication elimination).

Option notes:

- A: Negate the PREMISE: P ⇒ Q ≡ ¬P ∨ Q.
- C: That's the inverse, not equivalent.
- D: That's the converse, not equivalent.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B59. [MCQ · Easy]

Which sentence is logically equivalent to P ⇒ Q?

A. Q ⇒ P
B. P ∧ Q
C. ¬Q ⇒ ¬P
D. ¬P ⇒ ¬Q

Answer: C. ¬Q ⇒ ¬P

Working:

- P ⇒ Q ≡ ¬Q ⇒ ¬P (contraposition).

Option notes:

- A: The converse is not equivalent.
- B: Not equivalent: check a truth table.
- D: The inverse is not equivalent.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B60. [MCQ · Easy]

Which sentence is logically equivalent to ¬(P ∨ Q)?

A. ¬P ∧ ¬Q
B. ¬P ∧ Q
C. P ∨ Q
D. ¬P ∨ ¬Q

Answer: A. ¬P ∧ ¬Q

Working:

- ¬(P ∨ Q) ≡ ¬P ∧ ¬Q (De Morgan).

Option notes:

- B: Every literal must be negated.
- C: Negations can't just disappear.
- D: De Morgan flips ∧ ↔ ∨.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B61. [MCQ · Easy]

Which sentence is logically equivalent to P ⇒ Q?

A. ¬P ⇒ ¬Q
B. ¬Q ⇒ ¬P
C. Q ⇒ P
D. P ∧ Q

Answer: B. ¬Q ⇒ ¬P

Working:

- P ⇒ Q ≡ ¬Q ⇒ ¬P (contraposition).

Option notes:

- A: The inverse is not equivalent.
- C: The converse is not equivalent.
- D: Not equivalent: check a truth table.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B62. [MCQ · Easy]

Which sentence is logically equivalent to ¬(P ∨ Q)?

A. ¬P ∧ Q
B. P ∨ Q
C. ¬P ∧ ¬Q
D. ¬P ∨ ¬Q

Answer: C. ¬P ∧ ¬Q

Working:

- ¬(P ∨ Q) ≡ ¬P ∧ ¬Q (De Morgan).

Option notes:

- A: Every literal must be negated.
- B: Negations can't just disappear.
- D: De Morgan flips ∧ ↔ ∨.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B63. [MCQ · Easy]

Which sentence is logically equivalent to P ⇔ Q?

A. P ∧ Q
B. Q ⇒ P
C. (P ⇒ Q) ∧ (Q ⇒ P)
D. ¬P ⇒ ¬Q

Answer: C. (P ⇒ Q) ∧ (Q ⇒ P)

Working:

- P ⇔ Q ≡ (P ⇒ Q) ∧ (Q ⇒ P) (biconditional elimination).

Option notes:

- A: Not equivalent: check a truth table.
- B: The converse is not equivalent.
- D: The inverse is not equivalent.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B64. [MCQ · Easy]

Which sentence is logically equivalent to ¬(P ∧ Q)?

A. P ∨ Q
B. ¬P ∧ Q
C. ¬P ∧ ¬Q
D. ¬P ∨ ¬Q

Answer: D. ¬P ∨ ¬Q

Working:

- ¬(P ∧ Q) ≡ ¬P ∨ ¬Q (De Morgan).

Option notes:

- A: Negations can't just disappear.
- B: Every literal must be negated.
- C: De Morgan flips ∧ ↔ ∨.

Takeaway: Learn the equivalence table: it is the toolkit for CNF conversion.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B65. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∧ Q ⇒ R bracketed?

A. R ⇒ Q ∧ ¬P
B. ((¬P) ∧ Q) ⇒ R
C. ¬(P ∧ Q ⇒ R)
D. ¬P ∧ (Q ⇒ R)

Answer: B. ((¬P) ∧ Q) ⇒ R

Working:

- ¬P ∧ Q ⇒ R = ((¬P) ∧ Q) ⇒ R.

Option notes:

- A: Order of symbols cannot change.
- C: ¬ applies only to the symbol right after it.
- D: ∧ binds tighter than ∨ and ⇒.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B66. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is P ∧ Q ∨ R bracketed?

A. (P ∧ Q) ∨ R
B. P ∧ (Q ∨ R)

C. ¬(∧ Q ∨ R)
D. R ∨ Q ∧ P

Answer: A. (P ∧ Q) ∨ R

Working:

- P ∧ Q ∨ R = (P ∧ Q) ∨ R.

Option notes:

- B: ∧ binds tighter than ∨ and ⇒.
- C: ¬ applies only to the symbol right after it.
- D: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B67. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∨ Q ∧ R ⇒ S bracketed?

A. ¬(P ∨ Q ∧ R ⇒ S)
B. ((¬P) ∨ (Q ∧ R)) ⇒ S
C. S ⇒ R ∧ Q ∨ ¬P
D. ¬P ∨ Q ∧ (R ⇒ S)

Answer: B. ((¬P) ∨ (Q ∧ R)) ⇒ S

Working:

- ¬P ∨ Q ∧ R ⇒ S = ((¬P) ∨ (Q ∧ R)) ⇒ S.

Option notes:

- A: ¬ applies only to the symbol right after it.
- C: Order of symbols cannot change.
- D: ∧ binds tighter than ∨ and ⇒.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B68. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is P ∨ Q ⇒ R ⇔ S bracketed?

A. ¬(∨ Q ⇒ R ⇔ S)
B. P ∨ Q ⇒ R ⇔ S)
C. ((P ∨ Q) ⇒ R) ⇔ S
D. S ⇔ R ⇒ Q ∨ P

Answer: C. ((P ∨ Q) ⇒ R) ⇔ S

Working:

- P ∨ Q ⇒ R ⇔ S = ((P ∨ Q) ⇒ R) ⇔ S.

Option notes:

- A: ¬ applies only to the symbol right after it.
- B: ∧ binds tighter than ∨ and ⇒.
- D: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B69. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∨ Q ∧ R ⇒ S bracketed?

A. ¬P ∨ Q ∧ (R ⇒ S)
B. ¬(P ∨ Q ∧ R ⇒ S)
C. S ⇒ R ∧ Q ∨ ¬P
D. ((¬P) ∨ (Q ∧ R)) ⇒ S

Answer: D. ((¬P) ∨ (Q ∧ R)) ⇒ S

Working:

- ¬P ∨ Q ∧ R ⇒ S = ((¬P) ∨ (Q ∧ R)) ⇒ S.

Option notes:

- A: ∧ binds tighter than ∨ and ⇒.
- B: ¬ applies only to the symbol right after it.
- C: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B70. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∧ Q ⇒ R bracketed?

A. ¬(P ∧ Q ⇒ R)
B. ((¬P) ∧ Q) ⇒ R
C. ¬P ∧ (Q ⇒ R)
D. R ⇒ Q ∧ ¬P

Answer: B. ((¬P) ∧ Q) ⇒ R

Working:

- ¬P ∧ Q ⇒ R = ((¬P) ∧ Q) ⇒ R.

Option notes:

- A: ¬ applies only to the symbol right after it.
- C: ∧ binds tighter than ∨ and ⇒.
- D: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B71. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∧ Q ⇒ R bracketed?

A. ((¬P) ∧ Q) ⇒ R
B. ¬P ∧ (Q ⇒ R)
C. R ⇒ Q ∧ ¬P
D. ¬(P ∧ Q ⇒ R)

Answer: A. ((¬P) ∧ Q) ⇒ R

Working:

- ¬P ∧ Q ⇒ R = ((¬P) ∧ Q) ⇒ R.

Option notes:

- B: ∧ binds tighter than ∨ and ⇒.
- C: Order of symbols cannot change.
- D: ¬ applies only to the symbol right after it.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B72. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is P ∨ Q ⇒ R ⇔ S bracketed?

A. P ∨ Q ⇒ R ⇔ S)
B. ¬(∨ Q ⇒ R ⇔ S)
C. ((P ∨ Q) ⇒ R) ⇔ S
D. S ⇔ R ⇒ Q ∨ P

Answer: C. ((P ∨ Q) ⇒ R) ⇔ S

Working:

- P ∨ Q ⇒ R ⇔ S = ((P ∨ Q) ⇒ R) ⇔ S.

Option notes:

- A: ∧ binds tighter than ∨ and ⇒.
- B: ¬ applies only to the symbol right after it.
- D: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B73. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is P ∨ Q ⇒ R ⇔ S bracketed?

A. ¬(∨ Q ⇒ R ⇔ S)
B. P ∨ Q ⇒ R ⇔ S)
C. ((P ∨ Q) ⇒ R) ⇔ S
D. S ⇔ R ⇒ Q ∨ P

Answer: C. ((P ∨ Q) ⇒ R) ⇔ S

Working:

- P ∨ Q ⇒ R ⇔ S = ((P ∨ Q) ⇒ R) ⇔ S.

Option notes:

- A: ¬ applies only to the symbol right after it.
- B: ∧ binds tighter than ∨ and ⇒.
- D: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B74. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∧ Q ⇒ R bracketed?

A. R ⇒ Q ∧ ¬P
B. ((¬P) ∧ Q) ⇒ R

C. ¬P ∧ (Q ⇒ R)
D. ¬(P ∧ Q ⇒ R)

Answer: B. ((¬P) ∧ Q) ⇒ R

Working:

- ¬P ∧ Q ⇒ R = ((¬P) ∧ Q) ⇒ R.

Option notes:

- A: Order of symbols cannot change.
- C: ∧ binds tighter than ∨ and ⇒.
- D: ¬ applies only to the symbol right after it.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B75. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is P ∨ Q ⇒ R ⇔ S bracketed?

A. S ⇔ R ⇒ Q ∨ P
B. ((P ∨ Q) ⇒ R) ⇔ S
C. ¬(∨ Q ⇒ R ⇔ S)
D. P ∨ Q ⇒ R ⇔ S)

Answer: B. ((P ∨ Q) ⇒ R) ⇔ S

Working:

- P ∨ Q ⇒ R ⇔ S = ((P ∨ Q) ⇒ R) ⇔ S.

Option notes:

- A: Order of symbols cannot change.
- C: ¬ applies only to the symbol right after it.
- D: ∧ binds tighter than ∨ and ⇒.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B76. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∨ Q ∧ R ⇒ S bracketed?

A. ((¬P) ∨ (Q ∧ R)) ⇒ S
B. ¬P ∨ Q ∧ (R ⇒ S)
C. ¬(P ∨ Q ∧ R ⇒ S)
D. S ⇒ R ∧ Q ∨ ¬P

Answer: A. ((¬P) ∨ (Q ∧ R)) ⇒ S

Working:

- ¬P ∨ Q ∧ R ⇒ S = ((¬P) ∨ (Q ∧ R)) ⇒ S.

Option notes:

- B: ∧ binds tighter than ∨ and ⇒.
- C: ¬ applies only to the symbol right after it.
- D: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B77. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∨ Q ∧ R ⇒ S bracketed?

A. ¬P ∨ Q ∧ (R ⇒ S)
B. ((¬P) ∨ (Q ∧ R)) ⇒ S
C. S ⇒ R ∧ Q ∨ ¬P
D. ¬(P ∨ Q ∧ R ⇒ S)

Answer: B. ((¬P) ∨ (Q ∧ R)) ⇒ S

Working:

- ¬P ∨ Q ∧ R ⇒ S = ((¬P) ∨ (Q ∧ R)) ⇒ S.

Option notes:

- A: ∧ binds tighter than ∨ and ⇒.
- C: Order of symbols cannot change.
- D: ¬ applies only to the symbol right after it.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B78. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∧ Q ⇒ R bracketed?

A. ¬(P ∧ Q ⇒ R)
B. R ⇒ Q ∧ ¬P
C. ((¬P) ∧ Q) ⇒ R
D. ¬P ∧ (Q ⇒ R)

Answer: C. ((¬P) ∧ Q) ⇒ R

Working:

- ¬P ∧ Q ⇒ R = ((¬P) ∧ Q) ⇒ R.

Option notes:

- A: ¬ applies only to the symbol right after it.
- B: Order of symbols cannot change.
- D: ∧ binds tighter than ∨ and ⇒.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B79. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∧ Q ⇒ R bracketed?

A. ¬P ∧ (Q ⇒ R)
B. ¬(P ∧ Q ⇒ R)
C. ((¬P) ∧ Q) ⇒ R
D. R ⇒ Q ∧ ¬P

Answer: C. ((¬P) ∧ Q) ⇒ R

Working:

- ¬P ∧ Q ⇒ R = ((¬P) ∧ Q) ⇒ R.

Option notes:

- A: ∧ binds tighter than ∨ and ⇒.
- B: ¬ applies only to the symbol right after it.
- D: Order of symbols cannot change.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

LOGIC B80. [MCQ · Medium]

Using precedence ¬, ∧, ∨, ⇒, ⇔ (highest to lowest), how is ¬P ∧ Q ⇒ R bracketed?

A. ((¬P) ∧ Q) ⇒ R
B. ¬(P ∧ Q ⇒ R)
C. R ⇒ Q ∧ ¬P
D. ¬P ∧ (Q ⇒ R)

Answer: A. ((¬P) ∧ Q) ⇒ R

Working:

- ¬P ∧ Q ⇒ R = ((¬P) ∧ Q) ⇒ R.

Option notes:

- B: ¬ applies only to the symbol right after it.
- C: Order of symbols cannot change.
- D: ∧ binds tighter than ∨ and ⇒.

Takeaway: ¬ first, then ∧, ∨, ⇒, ⇔.

Source: Lecture 10 (Logical knowledge representation and inference)

Knowledge Representation (Lecture 12)

Part A: hand-written questions (87)

KR A1. [MATCH · Medium]

Match each hierarchy principle to its description.

1. Inheritance
2. Transitivity
3. Mutual exclusivity
4. Inclusiveness

A. the top class includes all subclasses
B. if A is a type of B and B of C, then A is a type of C
C. lower-level categories inherit properties of higher-level ones
D. any entity may belong to every class
E. sibling categories should not overlap

Answer: 1 → C, 2 → B, 3 → E, 4 → A

Working:

- Inheritance → lower-level categories inherit properties of higher-level ones
- Transitivity → if A is a type of B and B of C, then A is a type of C
- Mutual exclusivity → sibling categories should not overlap
- Inclusiveness → the top class includes all subclasses

Takeaway: Seven principles.

Source: Lecture 12, slide 6

KR A2. [TF · Medium]

Hierarchical classification works best when a domain is immature and class boundaries are unclear.

Answer: False

Why: It works best in MATURE, well-understood domains with CLEAR class boundaries.

Takeaway: Needs some theory to guide it.

Source: Lecture 12, slide 7

KR A3. [MCQ · Medium]

Species/differentia means a true hierarchy has…

A. overlapping siblings
B. no inheritance
C. many relationship types between levels
D. only one type of relationship between super- and subclasses: is-a

Answer: D. only one type of relationship between super- and subclasses: is-a

Option notes:

- A: That violates mutual exclusivity.
- B: Inheritance holds.
- C: Only is-a.

Takeaway: Generic is-a relation.

Source: Lecture 12, slide 6

KR A4. [BLANK · Easy]

Basketball ∈ Sports balls expresses _____ (one word: the relation between an object and its category).

Answer: membership
(Also accepted: member, MemberOf, member of)

Takeaway: ∈ = membership; ⊆ = subset.

Source: Lecture 12, slide 8

KR A5. [MCQ · Medium]

Basketballs ⊆ Sports balls is an example of…

A. an exception
B. the membership relation
C. the subset (subcategory) relation
D. a property value

Answer: C. the subset (subcategory) relation

Option notes:

- A: No.
- B: That's ∈ for objects.
- D: No.

Takeaway: Sub-categories inherit properties.

Source: Lecture 12, slide 8

KR A6. [TF · Medium]

Hierarchical representation supports inference from incomplete evidence.

Answer: True

Why: Slide 7: shared characteristics help place an entity even when evidence is missing.

Takeaway: Cat → mammal → bears live young.

Source: Lecture 12, slide 7

KR A7. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Pingu is a member of Penguin; Penguin is a subclass of Bird; Bird can fly; Penguin cannot fly. Can Pingu fly?

A. No: the specific 'cannot fly' overrides the inherited 'can fly'
B. Yes: Pingu inherits 'can fly' from Bird
C. Yes, because transitivity always wins
D. Unknown: the network is contradictory

Answer: A. No: the specific 'cannot fly' overrides the inherited 'can fly'

Option notes:

- B: The exception is more specific.
- C: Specific overrides general.
- D: Exceptions are allowed.

Takeaway: Exception overrides inherited knowledge.

Source: Lecture 12, slide 22

KR A8. [MULTI · Medium]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Which facts can be inferred for Tweety (member of Canary)? (Select ALL that apply.)

A. Tweety can fly
B. Tweety cannot fly
C. Tweety is yellow
D. Tweety is an Animal
E. Tweety has wings

Answer: A. Tweety can fly; C. Tweety is yellow; D. Tweety is an Animal; E. Tweety has wings

Option notes:

- A: Correct: Inherited from Bird.
- B: Incorrect: Only Penguin has that exception.
- C: Correct: Canary has colour Yellow.
- D: Correct: Canary → Bird → Animal.
- E: Correct: Bird has part Wings.

Takeaway: Follow links upward.

Source: Lecture 12, slide 11, 21

KR A9. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

Semantic network: Persons have Legs = 2; Mary ∈ Female Persons ⊆ Persons; no exception for Mary.
Mary has _____ legs.

Answer: 2
(Also accepted: two)

Takeaway: Inherited default.

Source: Lecture 12, slide 23

KR A10. [TF · Medium]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons —

John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

If Susan is Mary's mother, the HasMother link lets us infer that Susan is a member of Female Persons.

Answer: True

Why: Slide 24: HasMother links Persons to Female Persons.

Takeaway: Relations carry type information.

Source: Lecture 12, slide 24

KR A11. [MCQ · Medium]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

In the Persons network, John has an explicit Legs = 1. How many legs does the network say John has?

A. 3
B. 2, inherited from Persons
C. 1, because the explicit value on John overrides the default
D. Undefined

Answer: C. 1, because the explicit value on John overrides the default

Option notes:

- A: No.
- B: The specific value wins.
- D: There is a value.

Takeaway: Stop at the first value found going upward.

Source: Lecture 12, slide 12

KR A12. [MATCH · Medium]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

Match each link in the Persons semantic network to its type.

1. Female Persons → Persons
2. Mary → Female Persons
3. Mary → John
4. Persons → 2

A. Legs (property value)
B. InstanceOf Q146
C. SubsetOf (category hierarchy)
D. MemberOf (category membership)
E. SisterOf (relationship)

Answer: 1 → C, 2 → D, 3 → E, 4 → A

Working:

- Female Persons → Persons → SubsetOf (category hierarchy)
- Mary → Female Persons → MemberOf (category membership)
- Mary → John → SisterOf (relationship)
- Persons → 2 → Legs (property value)

Takeaway: Four kinds of link.

Source: Lecture 12, slide 12

KR A13. [MCQ · Medium]

What is the key difference between a semantic network and an ontology (Lecture 12)?

A. An ontology formally defines what concepts and relationships mean, enabling reliable automated reasoning
B. They are identical
C. Semantic networks can't represent categories
D. Ontologies can't represent individuals

Answer: A. An ontology formally defines what concepts and relationships mean, enabling reliable automated reasoning

Option notes:

- B: No.
- C: They can.
- D: They can.

Takeaway: Semantic net = graphical; ontology = formal, machine-interpretable.

Source: Lecture 12, slide 4

KR A14. [MULTI · Medium]

Which are parts of an ontology's basic structure? (Select ALL that apply.)

A. Classes and hierarchy
B. Axioms
C. Gradient updates
D. Properties
E. Individuals

Answer: A. Classes and hierarchy; B. Axioms; D. Properties; E. Individuals

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: Not KR.
- D: Correct:
- E: Correct:

Takeaway: Classes, properties, individuals, axioms.

Source: Lecture 12, slide 13

KR A15. [MCQ · Hard]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)

includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

Rule enrolsIn(x, y) requires Student(x) and Course(y). The assertion 'CB11.07 (a Room) enrols in Data Analytics' is…

A. accepted with a warning
B. accepted, because Data Analytics is a Course
C. rejected, because the domain constraint (Student) is violated
D. rejected, because the range is violated

Answer: C. rejected, because the domain constraint (Student) is violated

Option notes:

- A: Rule-based checking rejects it.
- B: The subject must also be a Student.
- D: The range (Course) is fine.

Takeaway: Domain = subject type.

Source: Lecture 12, slide 27

KR A16. [MCQ · Medium]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

Alice enrols in Data Analytics, which includes AI Foundations, owned by the School of CS. What does the property-chain rule infer?

A. Data Analytics is a Room
B. Alice studies in the School of CS department
C. School of CS is a Student
D. Alice teaches AI Foundations

Answer: B. Alice studies in the School of CS department

Option notes:

- A: No.
- C: No.
- D: No such link.

Takeaway: enrolsIn ∧ includes ∧ ownedBy ⇒ studiesInDepartment.

Source: Lecture 12, slide 29

KR A17. [TF · Medium]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

If Student is a subclass of Person and Alice is a Student, an ontology reasoner can infer Alice is a Person.

Answer: True

Why: Slide 28.

Takeaway: Class inference through the hierarchy.

Source: Lecture 12 (Hierarchical knowledge representation and reasoning)

KR A18. [MCQ · Hard]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

'Data Analytics scheduledIn School of CS' is flagged by the reasoner because…

A. Data Analytics is not a Subject
B. Rooms cannot have schedules
C. scheduledIn requires a Room as its object (range), but School of CS is a Department
D. Departments cannot own subjects

Answer: C. scheduledIn requires a Room as its object (range), but School of CS is a Department

Option notes:

- A: It is a Subject.
- B: Not the rule.
- D: Unrelated.

Takeaway: Range violation → inconsistency.

Source: Lecture 12, slide 30

KR A19. [MCQ · Easy]

A knowledge graph represents…

A. only class hierarchies with no individuals
B. pixels as nodes
C. probabilities on every edge
D. real-world entities as nodes and their relationships as labelled edges

Answer: D. real-world entities as nodes and their relationships as labelled edges

Option notes:

- A: KGs hold actual entities and facts.
- B: No.
- C: That's a Bayesian network.

Takeaway: Nodes, labelled edges, properties.

Source: Lecture 12, slide 15

KR A20. [BLANKS · Medium]

Wikidata: Douglas Adams is identified as Q_____ and 'instance of' is property P_____.

(1) Q-id = ____
(2) P-id = ____

Answer: Q-id = 42; P-id = 31

Takeaway: Q42; P31.

Source: Lecture 12, slide 16, 35

KR A21. [TF · Medium]

Each Wikidata item has a persistent Q identifier, while properties are prefixed with P.

Answer: True

Why: Slides 16/35.

Takeaway: Q = items, P = properties.

Source: Lecture 12 (Hierarchical knowledge representation and reasoning)

KR A22. [MCQ · Medium]

What does ?item wdt:P31 wd:Q146 . match?

A. Items that have an image
B. Labels of cats
C. Items that are instances of house cat
D. All cat breeds

Answer: C. Items that are instances of house cat

Option notes:

- A: That's P18.
- B: That's the label service.
- D: That would be Q1597324.

Takeaway: P31 = instance of, Q146 = house cat.

Source: Lecture 12, slide 35

KR A23. [TF · Medium]

In the cat query, a cat item without a P18 image still appears in the results with an empty ?pic.

Answer: False

Why: The triple pattern ?item wdt:P18 ?pic is required, so items without an image are excluded.

Takeaway: Every triple pattern must match.

Source: Lecture 12, slide 38

KR A24. [MATCH · Medium]

Match each SPARQL/RDF element to its role.

1. SELECT
2. WHERE
3. ?variable
4. Triple

A. placeholder replaced by values at query time
B. chooses the variables to return
C. defines the triple patterns data must match

D. deletes matching data
E. subject, predicate, object

Answer: 1 → B, 2 → C, 3 → A, 4 → E

Working:

- SELECT → chooses the variables to return
- WHERE → defines the triple patterns data must match
- ?variable → placeholder replaced by values at query time
- Triple → subject, predicate, object

Takeaway: SPARQL basics.

Source: Lecture 12, slide 32-36

KR A25. [MULTI · Medium]

Which are SPARQL query types named in Lecture 12? (Select ALL that apply.)

A. SELECT
B. DESCRIBE
C. BACKPROP
D. ASK
E. CONSTRUCT

Answer: A. SELECT; B. DESCRIBE; D. ASK; E. CONSTRUCT

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: Not SPARQL.
- D: Correct:
- E: Correct:

Takeaway: Four key query types.

Source: Lecture 12, slide 33

KR A26. [MCQ · Easy]

Green mottled skin, ovoid shape, red flesh with black seeds, in the fruit aisle → watermelon → good for fruit salad. This illustrates…

A. k-fold cross-validation
B. model checking
C. reasoning with categories: perceive → classify → predict
D. resolution

Answer: C. reasoning with categories: perceive → classify → predict

Option notes:

- A: ML evaluation.
- B: Logic inference.
- D: Logic.

Takeaway: Category reasoning.

Source: Lecture 12, slide 18

KR A27. [MATCH · Hard]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Which representation fits each scenario?

1. Reason about uncertain causes from evidence
2. Inherit default properties with exceptions (birds fly, penguins don't)
3. Share a formal vocabulary and check data consistency across systems
4. Prove a conclusion follows with certainty from rules

A. convolutional neural network
B. semantic network
C. propositional / first-order logic
D. Bayesian network
E. ontology

Answer: 1 → D, 2 → B, 3 → E, 4 → C

Working:

- Reason about uncertain causes from evidence → Bayesian network
- Inherit default properties with exceptions (birds fly, penguins don't) → semantic network
- Share a formal vocabulary and check data consistency across systems → ontology
- Prove a conclusion follows with certainty from rules → propositional / first-order logic

Takeaway: Choose the KR tool by the task (cross-topic).

Source: Lecture 12 (Hierarchical knowledge representation and reasoning)

KR A28. [BLANK · Hard]

In hierarchical KR, membership is decided by rules called necessary and _____ criteria.

Answer: sufficient
(Also accepted: Sufficient)

Takeaway: Slide 6.

Source: Lecture 12 (Knowledge representation)

KR A29. [TF · Easy]

Sub-categories inherit the properties of their super-categories.

Answer: True

Why: Slide 8.

Takeaway: Inheritance.

Source: Lecture 12 (Knowledge representation)

KR A30. [MCQ · Medium]

Why is organising objects into categories vital in structural KR?

A. Because objects cannot have properties
B. Much reasoning happens at the category level while interaction happens with individual objects

C. Because categories make data i.i.d.
D. Because categories remove the need for relations

Answer: B. Much reasoning happens at the category level while interaction happens with individual objects

Option notes:

- A: They can.
- C: Unrelated.
- D: Relations remain.

Takeaway: Slide 8 notes.

Source: Lecture 12 (Knowledge representation)

KR A31. [MATCH · Medium]

Match each symbol/relation to its meaning.

1. ∈
2. ⊆
3. is-a

A. a property value
B. a category is a subset of another category
C. the single generic relation in a true hierarchy
D. an object is a member of a category

Answer: 1 → D, 2 → B, 3 → C

Working:

- ∈ → an object is a member of a category
- ⊆ → a category is a subset of another category
- is-a → the single generic relation in a true hierarchy

Takeaway: Membership vs subset.

Source: Lecture 12 (Knowledge representation)

KR A32. [TF · Medium]

In the Lecture 12 semantic network, ovals represent concepts or objects and arrows represent labelled relations.

Answer: True

Why: Slide 11 notation.

Takeaway: Nodes + labelled links.

Source: Lecture 12 (Knowledge representation)

KR A33. [BLANK · Medium]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

```
Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)
```

In the university ontology, 'Mary → instance of → Student' makes Mary an _____ (one of: class, property, individual, axiom).

Answer: individual
(Also accepted: Individual)

Takeaway: Slide 13.

Source: Lecture 12 (Knowledge representation)

KR A34. [BLANK · Easy]

A knowledge-graph fact Student → enrolledIn → Course is called a _____.

Answer: triple
(Also accepted: Triple)

Takeaway: Slide 15.

Source: Lecture 12 (Knowledge representation)

KR A35. [MCQ · Medium]

What is the emphasis of a knowledge graph compared with an ontology?

A. Representing actual entities and facts, which may be organised by an ontology/schema
B. Defining only abstract class axioms
C. Storing images
D. Storing probabilities

Answer: A. Representing actual entities and facts, which may be organised by an ontology/schema

Option notes:

- B: That's the ontology side.
- C: No.
- D: Bayesian networks.

Takeaway: Slide 16.

Source: Lecture 12 (Knowledge representation)

KR A36. [MCQ · Easy]

In SPARQL, how is a variable written?

A. With a dollar sign, e.g. $item
B. With wd: prefix
C. With a question mark, e.g. ?item
D. In quotes, e.g. 'item'

Answer: C. With a question mark, e.g. ?item

Option notes:

- A: Not in the lecture syntax.
- B: That's an entity prefix.

- D: That's a literal.

Takeaway: Slide 34.

Source: Lecture 12 (Knowledge representation)

KR A37. [MCQ · Medium]

A semantic network's inheritance algorithm, starting from an object, …

A. searches every node and averages the values
B. follows member/subclass links upward and stops as soon as it finds a value
C. follows links downward to the leaves
D. uses theorem proving over all sentences

Answer: B. follows member/subclass links upward and stops as soon as it finds a value

Option notes:

- A: It stops at the first (most specific) value.
- C: Inheritance goes up toward general categories.
- D: Simpler and more efficient than theorem proving.

Takeaway: Slide 19 notes.

Source: Lecture 12 (Knowledge representation)

KR A38. [TF · Medium]

One attraction of semantic networks is that their inheritance-based inference is simpler and more efficient than logical theorem proving.

Answer: True

Why: Lecture 12 slide 19 notes.

Takeaway: Efficient category-based inference.

Source: Lecture 12 (Knowledge representation)

KR A39. [MULTI · Hard]

Select ALL things an ontology reasoner can do (Lecture 12). (Select ALL that apply.)

A. Infer properties via property chains
B. Learn new axioms by gradient descent
C. Check rule compliance (only Students may use enrolledIn)
D. Infer class membership (Alice is a Person)
E. Validate consistency against class/property constraints

Answer: A. Infer properties via property chains; C. Check rule compliance (only Students may use enrolledIn); D. Infer class membership (Alice is a Person); E. Validate consistency against class/property constraints

Option notes:

- A: Correct:
- B: Incorrect: Not ontology reasoning.
- C: Correct:
- D: Correct:
- E: Correct:

Takeaway: Infer + check.

Source: Lecture 12 (Knowledge representation)

KR A40. [BLANK · Medium]

In Wikidata SPARQL, the prefix for entities such as Q146 is _____ (e.g. wd:Q146).

Answer: wd
(Also accepted: wd:)

Takeaway: wd: entities, wdt: direct-claim properties.

Source: Lecture 12 (Knowledge representation)

KR A41. [MCQ · Hard]

Which statement best contrasts a knowledge graph and an ontology?

A. They are identical terms
B. An ontology stores only images
C. A knowledge graph cannot have labelled edges
D. An ontology defines the schema/meaning (classes, properties, axioms); a knowledge graph stores actual entities and facts that may follow that schema

Answer: D. An ontology defines the schema/meaning (classes, properties, axioms); a knowledge graph stores actual entities and facts that may follow that schema

Option notes:

- A: Different emphasis.
- B: No.
- C: Labelled edges are its core.

Takeaway: Schema vs data.

Source: Lecture 12 (Knowledge representation)

KR A42. [TF · Easy]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

In the Lecture 12 semantic network, Canary has colour Yellow.

Answer: True

Why: Canary —has colour→ Yellow.

Takeaway: Property value.

Source: Lecture 12 (Knowledge representation)

KR A43. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Bird network: can Tweety fly, and via which path?

A. Yes: Tweety → Canary → Bird (can Fly)
B. Unknown

C. Yes: Tweety → Penguin → Bird
D. No: Canary has no fly link

Answer: A. Yes: Tweety → Canary → Bird (can Fly)

Option notes:

- B: Inheritance answers it.
- C: Tweety is a Canary.
- D: Inherit from Bird.

Takeaway: Walk upward.

Source: Book 2 (Exam examples), Q402, p. 50

KR A44. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Bird network: can Pingu fly?

A. No: Penguin's explicit 'cannot Fly' overrides Bird's 'can Fly'
B. Yes: Pingu is a Bird
C. Unknown
D. Yes: Animals can fly

Answer: A. No: Penguin's explicit 'cannot Fly' overrides Bird's 'can Fly'

Option notes:

- B: The exception is found first.
- C: Answered by inheritance.
- D: No such link.

Takeaway: Specific overrides general.

Source: Book 2 (Exam examples), Q403, p. 50

KR A45. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Bird network: is Canary an Animal, and which property makes this valid?

A. Yes; mutual exclusivity
B. Yes; inclusiveness of Canary
C. No; Canary links only to Bird
D. Yes; transitivity of subclass (Canary ⊂ Bird ⊂ Animal)

Answer: D. Yes; transitivity of subclass (Canary ⊂ Bird ⊂ Animal)

Option notes:

- A: Wrong principle.
- B: Inclusiveness is about the top class.
- C: Transitivity.

Takeaway: Transitivity.

Source: Book 2 (Exam examples), Q404, p. 50

KR A46. [TF · Easy]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

In the bird network, Pingu has wings.

Answer: True

Why: Inherited from Bird; Penguin has no exception for wings.

Takeaway: Exceptions are property-specific.

Source: Book 2 (Exam examples), Q405, p. 50

KR A47. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Bird network: what colour is Tweety? ___

Answer: Yellow

Takeaway: Stated directly.

Source: Book 2 (Exam examples), Q406, p. 50

KR A48. [TF · Easy]

In a semantic network the inheritance algorithm stops at the first value found going upward.

Answer: True

Why: So the most specific value wins.

Takeaway: First value wins.

Source: Book 2 (Exam examples), Q407, p. 51

KR A49. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

People network (Persons have Legs = 2; John has Legs = 1). How many legs does Mary have? ___

Answer: 2

Takeaway: Inherited from Persons.

Source: Book 2 (Exam examples), Q408, p. 51

KR A50. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons —

John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

People network (Persons have Legs = 2; John has Legs = 1). How many legs does John have? ___

Answer: 1

Takeaway: Explicit value overrides the default.

Source: Book 2 (Exam examples), Q409, p. 51

KR A51. [BLANK · Medium]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

People network: HasMother links Persons to Female Persons. If Susan is Mary's mother, Susan is a member of ___.

Answer: Female Persons
(Also accepted: FemalePersons, female persons)

Takeaway: Range of HasMother.

Source: Book 2 (Exam examples), Q410, p. 51

KR A52. [TF · Easy]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

In the people network, Mary is a Mammal.

Answer: True

Why: Female Persons ⊂ Persons ⊂ Mammals.

Takeaway: Transitivity.

Source: Book 2 (Exam examples), Q411, p. 51

KR A53. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

People network: the relation between Mary and Female Persons is:

A. HasMother
B. MemberOf
C. SisterOf
D. SubsetOf

Answer: B. MemberOf

Option notes:

- A: That links a person to a mother.
- C: That links Mary and John.

- D: Mary is an object, not a category.

Takeaway: Object → category = membership.

Source: Book 2 (Exam examples), Q412, p. 51

KR A54. [MCQ · Easy]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

People network: the relation between Female Persons and Persons is:

A. MemberOf
B. instance of
C. SubsetOf
D. Legs

Answer: C. SubsetOf

Option notes:

- A: Both are categories.
- B: That is membership.
- D: A property.

Takeaway: Category → category = subset.

Source: Book 2 (Exam examples), Q413, p. 51

KR A55. [MATCH · Hard]

Match each structural requirement to its description.

1. inclusiveness
2. species/differentia
3. inheritance
4. transitivity
5. mutual exclusivity
6. necessary and sufficient criteria
7. systematic and predictable rules

A. the top class includes all subclasses
B. subclasses take on superclass attributes
C. membership rules say what must be true and what is enough
D. if A is a type of B and B of C, then A is a type of C
E. only the is-a relation between super- and subclass
F. an entity belongs to only one class
undefined. grouping and distinguishing rules fixed beforehand

Answer: 1 → A, 2 → E, 3 → B, 4 → D, 5 → F, 6 → C

Working:

- inclusiveness → the top class includes all subclasses
- species/differentia → only the is-a relation between super- and subclass
- inheritance → subclasses take on superclass attributes
- transitivity → if A is a type of B and B of C, then A is a type of C

- mutual exclusivity → an entity belongs to only one class
- necessary and sufficient criteria → membership rules say what must be true and what is enough
- systematic and predictable rules → grouping and distinguishing rules fixed beforehand

Takeaway: Seven requirements.

Source: Book 2 (Exam examples), Q414, p. 51

KR A56. [MULTI · Medium]

Hierarchical classification works best when: (Select ALL that apply.)

A. the domain is mature and well understood
B. entities are ill-defined
C. class boundaries are clear
D. we need to infer from incomplete evidence

Answer: A. the domain is mature and well understood; C. class boundaries are clear; D. we need to infer from incomplete evidence

Option notes:

- A: Correct:
- B: Incorrect: needs well-defined entities
- C: Correct:
- D: Correct:

Takeaway: Mature, clear, inference-friendly.

Source: Book 2 (Exam examples), Q415, p. 51

KR A57. [BLANKS · Easy]

Basketball9 ∈ Basketballs is a ___ relation; Basketballs ⊂ Balls is a ___ relation.

(1) ∈ = ____
(2) ⊂ = ____

Answer: ∈ = membership; ⊂ = subset

Takeaway: ∈ object, ⊂ category.

Source: Book 2 (Exam examples), Q416, p. 51

KR A58. [MCQ · Medium]

The key difference between a semantic network and an ontology:

A. none
B. ontologies cannot have hierarchies
C. an ontology formally defines what concepts and relations mean, enabling reliable reasoning
D. semantic networks are formal

Answer: C. an ontology formally defines what concepts and relations mean, enabling reliable reasoning

Option notes:

- A: There is a key difference.
- B: They do.
- D: They are informal.

Takeaway: Formal semantics.

Source: Book 2 (Exam examples), Q417, p. 52

KR A59. [MULTI · Easy]

Parts of an ontology: (Select ALL that apply.)

A. properties
B. axioms
C. gradients
D. classes and hierarchy
E. individuals

Answer: A. properties; B. axioms; D. classes and hierarchy; E. individuals

Option notes:

- A: Correct:
- B: Correct:
- C: Incorrect: not an ontology part
- D: Correct:
- E: Correct:

Takeaway: Classes, properties, individuals, axioms.

Source: Book 2 (Exam examples), Q418, p. 52

KR A60. [MCQ · Medium]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

Rule enrolsIn(x, y) requires Student(x) and Course(y). 'CB11.07 enrolsIn Data Analytics' is:

A. accepted, ontologies are open-world
B. accepted, Data Analytics is a Course
C. rejected, because CB11.07 is a Room so the domain constraint is violated
D. rejected, because Data Analytics is a Room

Answer: C. rejected, because CB11.07 is a Room so the domain constraint is violated

Option notes:

- A: Domain checks still flag it.
- B: The subject must be a Student.
- D: Data Analytics is a Course.

Takeaway: Domain/range checks.

Source: Book 2 (Exam examples), Q419, p. 52

KR A61. [BLANKS · Medium]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

enrolsIn(s, c) ∧ includes(c, u) ∧ ownedBy(u, d) ⇒ studiesInDepartment(s, d). Alice enrols in Data Analytics, which includes AI Foundations, owned by School of CS. The reasoner infers Alice studies in ___.

(1) department = _____

Answer: department = School of CS

Takeaway: Property chain.

Source: Book 2 (Exam examples), Q420, p. 52

KR A62. [MCQ · Medium]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

'Data Analytics scheduledIn School of CS' is inconsistent because:

A. scheduledIn requires the object to be a Room; School of CS is a Department
B. Data Analytics is not a Subject
C. it is consistent
D. School of CS has no rooms

Answer: A. scheduledIn requires the object to be a Room; School of CS is a Department

Option notes:

- B: It is a Course/Subject.
- C: Range is violated.
- D: Not the rule.

Takeaway: Range check.

Source: Book 2 (Exam examples), Q421, p. 52

KR A63. [MCQ · Easy]

Relation | Subject → Object
subclass of | Student → Person; Academic → Person
enrols in | Student → Course (e.g. Alice : Student enrols in Data Analytics : Course)
includes | Course → Subject (Data Analytics includes AI Foundations)
owned by | Subject → Department (AI Foundations owned by School of CS)
scheduled in | Subject → Room (e.g. CB11.07)

From Student ⊑ Person and Student(Alice) the reasoner infers:

A. Student(Person)
B. nothing
C. Alice ⊑ Person
D. Person(Alice)

Answer: D. Person(Alice)

Option notes:

- A: Category, not object.
- B: Class membership is inferred.
- C: Alice is an individual.

Takeaway: Class inference.

Source: Book 2 (Exam examples), Q422, p. 52

KR A64. [BLANKS · Easy]

A knowledge-graph triple has the form ___ — ___ — ___.

(1) 1st = _____
(2) 2nd = _____
(3) 3rd = _____

Answer: 1st = subject; 2nd = predicate; 3rd = object

Takeaway: Triples.

Source: Book 2 (Exam examples), Q423, p. 52

KR A65. [BLANK · Easy]

In Wikidata, Douglas Adams has identifier ___.

Answer: Q42

Takeaway: Persistent Q identifier.

Source: Book 2 (Exam examples), Q424, p. 52

KR A66. [MATCH · Easy]

SELECT ?item ?itemLabel ?pic
WHERE { ?item wdt:P31 wd:Q146 . ?item wdt:P18 ?pic .
SERVICE wikibase:label { bd:serviceParam wikibase:language "[AUTO_LANGUAGE],en" } }

Match the Wikidata identifiers in the query shown.

1. wdt:P31
2. wd:Q146
3. wdt:P18

A. house (domestic) cat
B. instance of
C. image
D. Douglas Adams

Answer: 1 → B, 2 → A, 3 → C

Working:

- wdt:P31 → instance of
- wd:Q146 → house (domestic) cat
- wdt:P18 → image

Takeaway: P = property, Q = item.

Source: Book 2 (Exam examples), Q425, p. 52

KR A67. [MCQ · Medium]

SELECT ?item ?itemLabel ?pic
WHERE { ?item wdt:P31 wd:Q146 . ?item wdt:P18 ?pic .
SERVICE wikibase:label { bd:serviceParam wikibase:language "[AUTO_LANGUAGE],en" } }

What does the query shown return?

A. individual cats that have an image, with label and image link
B. all images in Wikidata
C. all animals
D. all cat breeds

Answer: A. individual cats that have an image, with label and image link

Option notes:

- B: Only cats.
- C: Only house cats.
- D: Breeds use Q1597324.

Takeaway: Each line is a triple pattern.

Source: Book 2 (Exam examples), Q426, p. 52

KR A68. [TF · Medium]

SELECT ?item ?itemLabel ?pic
WHERE { ?item wdt:P31 wd:Q146 . ?item wdt:P18 ?pic .
SERVICE wikibase:label { bd:serviceParam wikibase:language "[AUTO_LANGUAGE],en" } }

In the query shown, a cat item without an image (P18) appears with an empty ?pic.

Answer: False

Why: The triple pattern must match, so it is excluded.

Takeaway: No OPTIONAL → excluded.

Source: Book 2 (Exam examples), Q427, p. 52

KR A69. [BLANK · Easy]

Variables in SPARQL start with ___.

Answer: ?
(Also accepted: question mark, a question mark, ?var)

Takeaway: ?item.

Source: Book 2 (Exam examples), Q428, p. 52

KR A70. [MULTI · Easy]

SPARQL query types: (Select ALL that apply.)

A. CONSTRUCT
B. SELECT
C. ASK
D. UPDATE-ALL
E. DESCRIBE

Answer: A. CONSTRUCT; B. SELECT; C. ASK; E. DESCRIBE

Option notes:

- A: Correct:
- B: Correct:
- C: Correct:
- D: Incorrect: not a query type

- E: Correct:

Takeaway: Four types.

Source: Book 2 (Exam examples), Q429, p. 53

KR A71. [BLANK · Easy]

To show at most 100 results add ___.

Answer: LIMIT 100
(Also accepted: limit 100)

Takeaway: LIMIT.

Source: Book 2 (Exam examples), Q430, p. 53

KR A72. [MATCH · Medium]

Match.

1. RDFS
2. OWL
3. URI

A. uniquely identifies a resource
B. RDF vocabulary for defining classes and properties
C. more expressive ontology language extending RDFS

Answer: 1 → B, 2 → C, 3 → A

Working:

- RDFS → RDF vocabulary for defining classes and properties
- OWL → more expressive ontology language extending RDFS
- URI → uniquely identifies a resource

Takeaway: Semantic web stack.

Source: Book 2 (Exam examples), Q431, p. 53

KR A73. [MULTI · Hard]

Build the network: 'Dogs and Cats are Mammals. Mammals are Animals. Animals breathe. Mammals have fur. Rex is a Dog. Tom is a Cat. Sphynx is a kind of Cat that has no fur. Kiki is a Sphynx.' Which are correct? (Select ALL that apply.)

A. Sphynx → Cat is a subclass-of link
B. Rex breathes (via Dog, Mammal, Animal)
C. Rex → Dog is a subclass-of link
D. Kiki has fur
E. Tom has fur (via Cat, Mammal)

Answer: A. Sphynx → Cat is a subclass-of link; B. Rex breathes (via Dog, Mammal, Animal); E. Tom has fur (via Cat, Mammal)

Option notes:

- A: Correct: 'is a kind of' → subclass
- B: Correct: inherited
- C: Incorrect: Rex is an individual: member-of
- D: Incorrect: Sphynx's 'has no fur' exception is found first

- E: Correct: inherited

Takeaway: Kinds → subclass; individuals → member; exceptions on the specific class.

Source: Book 2 (Exam examples), Q432, p. 53

KR A74. [MATCH · Easy]

Match the hierarchy requirement.

1. inclusiveness
2. inheritance
3. mutual exclusivity

A. subclasses take on superclass properties
B. siblings don't overlap
C. top class includes all subclasses

Answer: 1 → C, 2 → A, 3 → B

Working:

- inclusiveness → top class includes all subclasses
- inheritance → subclasses take on superclass properties
- mutual exclusivity → siblings don't overlap

Takeaway: Three of the seven.

Source: Book 2 (Exam examples), Mixed practice paper Q38, p. 55

KR A75. [TF · Easy]

[Diagram as text] Edges (label in brackets): Animal — Bird (subclass of); Bird — Penguin (subclass of); Bird — Canary (has part); Penguin — Pingu (member of); Canary — Tweety; Bird — Fly (can); Bird — Wings; Canary — Yellow (has colour)
Other labels: cannot (fly), member of

Penguin is a Bird; Bird can fly; Penguin cannot fly; Pingu is a Penguin. Pingu can fly.

Answer: False

Why: The specific exception on Penguin overrides the inherited property.

Takeaway: First value found wins.

Source: Book 2 (Exam examples), Mixed practice paper Q39, p. 55

KR A76. [MCQ · Easy]

Which requirement says an entity belongs to only one sibling class?

A. inclusiveness
B. inheritance
C. transitivity
D. mutual exclusivity

Answer: D. mutual exclusivity

Option notes:

- A: Top class.
- B: Properties flow down.
- C: Chains of is-a.

Takeaway: Siblings don't overlap.

Source: Book 3 (AI from zero), Practice 27.1, p. 110

KR A77. [TF · Easy]

Ostrich ⊆ Bird, Ostrich: cannot fly. Olly the ostrich can fly.

Answer: False

Why: The specific value wins.

Takeaway: Exceptions override.

Source: Book 3 (AI from zero), Practice 27.2, p. 110

KR A78. [BLANK · Easy]

wdt:P31 means ___.

Answer: instance of
(Also accepted: instance)

Takeaway: P31.

Source: Book 3 (AI from zero), Practice 27.3, p. 110

KR A79. [BLANKS · Easy]

The three parts of a triple:

(1) 1st = _____
(2) 2nd = _____
(3) 3rd = _____

Answer: 1st = subject; 2nd = predicate; 3rd = object

Takeaway: RDF.

Source: Book 3 (AI from zero), Practice 27.4, p. 110

KR A80. [TF · Hard]

OWL reasoning is closed-world: missing information is treated as false.

Answer: False

Why: It is open-world: missing means unknown.

Takeaway: No closed-world validation by itself.

Source: Book 3 (AI from zero), Chapter 27, p. 108

KR A81. [MCQ · Easy]

Green mottled skin, ovoid, red flesh, black seeds, in the fruit aisle ⇒ Watermelon ⇒ good for fruit salad. This illustrates:

A. reasoning with categories: perceive → classify → predict
B. inheritance exceptions
C. resolution
D. SPARQL

Answer: A. reasoning with categories: perceive → classify → predict

Option notes:

- B: No exception here.
- C: Logic proof method.
- D: Query language.

Takeaway: Categories let us predict.

Source: Book 1 (Compiled lecture notes), Chapter 12, p. 61

KR A82. [MCQ · Easy]

AIMA writes 'BB_9 is a basketball' and 'Basketballs are balls' as:

A. $BB_9 \subset$ Basketballs; Basketballs $\in$ Balls
B. Basketballs(BB_9) $\Rightarrow$ Balls
C. $BB_9 \in$ Basketballs; Basketballs $\subset$ Balls
D. BB_9 = Basketballs

Answer: C. $BB_9 \in$ Basketballs; Basketballs $\subset$ Balls

Option notes:

- A: Swapped.
- B: Mixed notation.
- D: Not identity.

Takeaway: Member vs subset.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 10: categories

KR A83. [MATCH · Hard]

(Extra, not in your slides) Match AIMA's category relations.

1. Disjoint
2. Exhaustive decomposition
3. Partition

A. disjoint exhaustive decomposition
B. no members in common (Animals, Vegetables)
C. subclass of exactly one category
D. every member of the parent is in at least one subclass

Answer: 1 → B, 2 → D, 3 → A

Working:

- Disjoint → no members in common (Animals, Vegetables)
- Exhaustive decomposition → every member of the parent is in at least one subclass
- Partition → disjoint exhaustive decomposition

Takeaway: Category structure.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 10: categories

KR A84. [TF · Medium]

(Extra, not in your slides) Semantic networks are a form of logic: their inheritance algorithm can be expressed in first-order logic, with default values that more specific values override.

Answer: True

Why: John's one leg overrides the Persons default of 2.

Takeaway: Defaults + overriding = nonmonotonic flavour.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 10: semantic networks

KR A85. [MCQ · Hard]

(Extra, not in your slides) Why is multiple inheritance a problem for semantic networks?

A. it breaks transitivity
B. networks cannot have two parents
C. it is too slow
D. an object can inherit conflicting values from two categories (e.g. Nixon: Quaker = pacifist, Republican = not)

Answer: D. an object can inherit conflicting values from two categories (e.g. Nixon: Quaker = pacifist, Republican = not)

Option notes:

- A: Not the issue.
- B: They can.
- C: The issue is conflict.

Takeaway: Nixon diamond.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 10: inheritance

KR A86. [BLANK · Easy]

[Diagram as text] Edges (label in brackets): Mammals — Persons (SubsetOf); Persons — Female Persons (SubsetOf); Persons — Male Persons (SubsetOf); Female Persons — Mary (MemberOf); Male Persons — John (MemberOf); Mary — John (SisterOf)
Other labels: Legs = 2, Legs = 1, HasMother → Female Persons

AIMA's persons network: John has Legs = 1 although Persons have Legs = 2. How many legs does the reasoner give John? ___

Answer: 1

Takeaway: Most specific value wins.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 10: semantic network defaults

KR A87. [MCQ · Hard]

(Extra, not in your slides) Description logics (the basis of OWL) are designed mainly to make which tasks easy?

A. subsumption (is one category a subset of another?) and classification (which category does an object belong to?)
B. computing probabilities
C. path finding
D. learning weights

Answer: A. subsumption (is one category a subset of another?) and classification (which category does an object belong to?)

Option notes:

- B: That is Bayesian networks.
- C: That is search.
- D: That is ML.

Takeaway: Tractable reasoning about categories.

Source: AIMA 4th ed. (Russell & Norvig 2022), Ch. 10: description logics

---

🧮 Maths Toolkit (Book 3) (Book 3 · Part I)

Part A: hand-written questions (45)

MATH A1. [MCQ · Easy]

Three actions have values 4, 9 and 7. What is argmax over the actions?

A. 7
B. 9
C. action 2 (the one worth 9)
D. 20

Answer: C. action 2 (the one worth 9)

Option notes:

- A: Neither max nor argmax.
- B: That is the max (a value).
- D: That is the sum.

Takeaway: argmax returns the input; max returns the value.

Source: Book 3 (AI from zero), Chapter 1, p. 9

MATH A2. [BLANK · Easy · calculation]

Order of operations: $2 + 3 \times 4^2 =$ ___.

Answer: 50

Takeaway: Powers, then multiply, then add.

Source: Book 3 (AI from zero), Chapter 1, p. 9

MATH A3. [MATCH · Easy]

Match the symbol to its usual meaning in this course.

1. α
2. γ
3. π
4. μ

A. a policy
B. learning rate
C. discount factor
D. the number 3.14
E. a mean

Answer: 1 → B, 2 → C, 3 → A, 4 → E

Working:

- α → learning rate
- γ → discount factor
- π → a policy
- μ → a mean

Takeaway: π is a policy here, not 3.14.

Source: Book 3 (AI from zero), Chapter 1, p. 9

MATH A4. [TF · Easy]

x_2 means x squared.

Answer: False

Why: A subscript is a label (the second x); x^2 is the power.

Takeaway: Low = label, high = power.

Source: Book 3 (AI from zero), Chapter 1, p. 9

MATH A5. [BLANKS · Medium · calculation]

Calculator check (3 s.f.): ln 2 = ___; e^{-1} = ___; 1/(1 + e^−0.6) = ___.

(1) ln 2 = _____
(2) e^{-1} = _____
(3) σ(0.6) = _____

Answer: ln 2 = 0.693; e^{-1} = 0.368; σ(0.6) = 0.646

Takeaway: Practise on your own calculator.

Source: Book 3 (AI from zero), Chapter 1, p. 9

MATH A6. [BLANKS · Easy · calculation]

If f(x) = 3x − 2, find f(4) and f(0).

(1) f(4) = _____
(2) f(0) = _____

Answer: f(4) = 10; f(0) = -2

Takeaway: Substitute.

Source: Book 3 (AI from zero), Practice 2.1, p. 14

MATH A7. [BLANKS · Easy]

Slope and intercept of y = −2x + 6?

(1) slope = _____
(2) intercept = _____

Answer: slope = -2; intercept = 6

Takeaway: $y = w_1x + w_0$.

Source: Book 3 (AI from zero), Practice 2.2, p. 14

MATH A8. [BLANKS · Medium · calculation]

Where is the minimum of $y = x^2 - 6x + 10$, and its value?

(1) x = _____
(2) y = _____

Answer: x = 3; y = 1

Takeaway: x = −b/2a = 3.

Source: Book 3 (AI from zero), Practice 2.3, p. 14

MATH A9. [BLANKS · Easy · calculation]

ReLU(−0.5) = ___, ReLU(0.5) = ___.

(1) ReLU(−0.5) = ____
(2) ReLU(0.5) = ____

Answer: ReLU(−0.5) = 0; ReLU(0.5) = 0.5

Takeaway: max(0, z).

Source: Book 3 (AI from zero), Practice 2.4, p. 14

MATH A10. [TF · Medium]

A local minimum is the lowest point of the whole function.

Answer: False

Why: Only lowest compared with its neighbours; the global minimum is lowest overall.

Takeaway: Hill climbing can stop at a local one.

Source: Book 3 (AI from zero), Chapter 2, p. 12

MATH A11. [MCQ · Medium]

Why does hill climbing on 8-queens maximise −cost?

A. −cost is the number of queens
B. hill climbing cannot minimise anything
C. cost is always negative
D. minimising f is the same as maximising −f

Answer: D. minimising f is the same as maximising −f

Option notes:

- A: No.
- B: It can, via −f.
- C: Cost ≥ 0.

Takeaway: min f = max(−f).

Source: Book 3 (AI from zero), Chapter 2, p. 12

MATH A12. [BLANKS · Easy · calculation]

$\log_2 16$ = ___, $\log_2 1$ = ___, $\log_2 0.5$ = ___, $\log_2 0.25$ = ___.

(1) $\log_2 16$ = ____
(2) $\log_2 1$ = ____
(3) $\log_2 0.5$ = ____
(4) $\log_2 0.25$ = ____

Answer: $\log_2 16 = 4$; $\log_2 1 = 0$; $\log_2 0.5 = -1$; $\log_2 0.25 = -2$

Takeaway: Powers of 2.

Source: Book 3 (AI from zero), Practice 3.1, p. 17

MATH A13. [BLANK · Medium · calculation]

$\log_2(2/3)$ using a calculator = ___ (3 d.p.).

Answer: -0.585

Takeaway: ln(0.6667)/ln 2.

Source: Book 3 (AI from zero), Practice 3.2, p. 17

MATH A14. [BLANKS · Medium · calculation]

σ(2) = ___ and σ(−2) = ___ (3 d.p.).

(1) σ(2) = _____
(2) σ(−2) = _____

Answer: σ(2) = 0.881; σ(−2) = 0.119

Takeaway: They add to 1.

Source: Book 3 (AI from zero), Practice 3.3, p. 17

MATH A15. [BLANK · Easy · calculation]

e^−0.5 (SA with Δ = 10, T = 20) = ___ (3 d.p.).

Answer: 0.607

Takeaway: Exponential.

Source: Book 3 (AI from zero), Practice 3.4, p. 17

MATH A16. [BLANK · Easy · calculation]

Simplify $\log_2(8 \times 4)$ using the log rules.

Answer: 5

Takeaway: 3 + 2.

Source: Book 3 (AI from zero), Practice 3.5, p. 17

MATH A17. [MCQ · Medium]

Why does entropy $H = -\Sigma\ p \log_2 p$ have a minus sign?

A. to make it a probability
B. it is a convention with no reason
C. every $\log_2 p$ is negative for $0 < p < 1$, so the minus makes H positive
D. because $\log_2$ is base 2

Answer: C. every $\log_2 p$ is negative for $0 < p < 1$, so the minus makes H positive

Option notes:

- A: Entropy can exceed 1 with many classes.
- B: There is a reason.
- D: Base does not set the sign.

Takeaway: Logs of fractions are negative.

Source: Book 3 (AI from zero), Chapter 3, p. 15

MATH A18. [TF · Medium]

Naive Bayes can add log-probabilities instead of multiplying, and the predicted class does not change.

Answer: True

Why: log is increasing, and log(abc) = log a + log b + log c.

Takeaway: Logs avoid underflow.

Source: Book 3 (AI from zero), Chapter 3, p. 15

MATH A19. [BLANK · Medium · calculation]

$0.8^{30} \approx$ ___ (2 s.f., e.g. 0.0034).

Answer: 0.0012
(Also accepted: 1.2e-3)

Takeaway: Multiplying by < 1 again and again shrinks fast.

Source: Book 3 (AI from zero), Chapter 3, p. 15

MATH A20. [BLANK · Easy · calculation]

$\Sigma_{i=1}^{5}\ 2i$ = ___.

Answer: 30

Takeaway: 2 + 4 + 6 + 8 + 10.

Source: Book 3 (AI from zero), Practice 4.1, p. 19

MATH A21. [BLANK · Easy · calculation]

$\Sigma_{k=0}^{3}\ 0.9^{k}$ = ___.

Answer: 3.439

Takeaway: 1 + 0.9 + 0.81 + 0.729.

Source: Book 3 (AI from zero), Practice 4.2, p. 19

MATH A22. [BLANK · Easy · calculation]

$\Pi_{i=1}^{4}\ i$ = ___.

Answer: 24

Takeaway: 4!.

Source: Book 3 (AI from zero), Practice 4.3, p. 19

MATH A23. [BLANK · Easy · calculation]

Weighted average of 10 and 20 with weights 0.3 and 0.7 = ___.

Answer: 17

Takeaway: 3 + 14.

Source: Book 3 (AI from zero), Practice 4.4, p. 19

MATH A24. [MATCH · Medium]

Match the formula to the operator it uses.

1. Naive Bayes score
2. weighted child entropy
3. attention output

A. Σ with branch shares as weights
B. $\lfloor \cdot \rfloor$ floor

C. Σ with attention weights
D. Π (product)

Answer: 1 → D, 2 → A, 3 → C

Working:

- Naive Bayes score → Π (product)
- weighted child entropy → Σ with branch shares as weights
- attention output → Σ with attention weights

Takeaway: Σ adds, Π multiplies.

Source: Book 3 (AI from zero), Chapter 4, p. 18

MATH A25. [TF · Medium]

Weighted child entropy is the plain average of the children's entropies.

Answer: False

Why: Each child is weighted by its share $|S_k|/|S|$.

Takeaway: Weighted, not plain.

Source: Book 3 (AI from zero), Chapter 4, p. 18

MATH A26. [BLANK · Easy · calculation]

$(2, -1, 3)\cdot(1, 4, 0)$ = ___.

Answer: -2

Takeaway: 2 − 4 + 0.

Source: Book 3 (AI from zero), Practice 5.1, p. 21

MATH A27. [BLANK · Easy · calculation]

$\|(1, 2, 2)\|$ = ___.

Answer: 3

Takeaway: √9.

Source: Book 3 (AI from zero), Practice 5.2, p. 21

MATH A28. [MCQ · Easy]

Shape of $(5 \times 2)(2 \times 7)$?

A. 2 × 2
B. 5 × 7
C. not defined
D. 5 × 2

Answer: B. 5 × 7

Option notes:

- A: Inner sizes vanish.
- C: Inner sizes match (2).
- D: Outer sizes remain.

Takeaway: (m×n)(n×p) = m×p.

Source: Book 3 (AI from zero), Practice 5.3, p. 21

MATH A29. [BLANKS · Medium · calculation]

[[0, 1], [1, 0]] × (3, 8) = (___, ___).

(1) 1st = _____
(2) 2nd = _____

Answer: 1st = 8; 2nd = 3

Takeaway: It swaps the entries.

Source: Book 3 (AI from zero), Practice 5.4, p. 21

MATH A30. [TF · Easy]

A negative dot product means the two vectors point in roughly opposite directions.

Answer: True

Why: Positive = similar, zero = perpendicular, negative = opposite.

Takeaway: Sign of alignment.

Source: Book 3 (AI from zero), Chapter 5, p. 20

MATH A31. [BLANK · Medium · calculation]

Cosine similarity of (1, 0) and (1, 1) = ___ (3 d.p.).

Answer: 0.707

Takeaway: 1/(1 × 1.414).

Source: Book 3 (AI from zero), Chapter 5, p. 20

MATH A32. [BLANK · Easy]

Derivative of $3x^2 + 2x$ = ___ (write in terms of x, e.g. 4x+1).

Answer: 6x+2
(Also accepted: 6x + 2)

Takeaway: Power rule.

Source: Book 3 (AI from zero), Practice 6.1, p. 25

MATH A33. [BLANK · Easy · calculation]

If σ(z) = 0.8, σ′(z) = ___.

Answer: 0.16

Takeaway: σ(1 − σ).

Source: Book 3 (AI from zero), Practice 6.2, p. 25

MATH A34. [BLANK · Medium · calculation]

$L = (w - 5)^2$, w = 1, α = 0.1. After one gradient step w = ___.

Answer: 1.8

Working:

- L′(1) = −8
- 1 − 0.1(−8) = 1.8

Takeaway: L′ = 2(w − 5) = −8.

Source: Book 3 (AI from zero), Practice 6.3, p. 25

MATH A35. [BLANK · Medium]

Chain rule: $y = u^2$, $u = 3x$. dy/dx = ___ (in terms of x).

Answer: 18x
(Also accepted: 18 x)

Takeaway: $2u \cdot 3 = 6u = 18x$.

Source: Book 3 (AI from zero), Practice 6.4, p. 25

MATH A36. [MCQ · Medium]

What happens if the learning rate α is too large?

A. training is just slower
B. steps overshoot the minimum and can diverge
C. the gradient becomes zero
D. the loss becomes convex

Answer: B. steps overshoot the minimum and can diverge

Option notes:

- A: That is α too small.
- C: No.
- D: α does not change the loss shape.

Takeaway: Pick α carefully.

Source: Book 3 (AI from zero), Chapter 6, p. 22

MATH A37. [TF · Medium]

The sigmoid's derivative is largest at z = 0, where it equals 0.25.

Answer: True

Why: 0.5×0.5.

Takeaway: Max slope 0.25.

Source: Book 3 (AI from zero), Chapter 6, p. 22

MATH A38. [BLANK · Hard · calculation]

Minimise $y = x^2 - 4x + 7$ by gradient descent from $x = -1.5$ with $\alpha = 0.3$. After one step x = ___.

Answer: 0.6

Working:

- $x \leftarrow -1.5 - 0.3(-7) = 0.6$

Takeaway: $y' = 2x - 4 = -7$.

Source: Book 3 (AI from zero), Chapter 6, p. 22

MATH A39. [BLANK · Easy · calculation]

Two dice: P(sum = 7) = ___.

Answer: 1/6
(Also accepted: 0.167, 6/36)

Takeaway: 6 of 36 outcomes.

Source: Book 3 (AI from zero), Practice 7.1, p. 28

MATH A40. [BLANKS · Medium · calculation]

P(A) = 0.3, P(B) = 0.5, independent. P(A ∧ B) = ___; P(A ∨ B) = ___.

(1) AND = _____
(2) OR = _____

Answer: AND = 0.15; OR = 0.65

Takeaway: Subtract the overlap.

Source: Book 3 (AI from zero), Practice 7.2, p. 28

MATH A41. [BLANK · Easy · calculation]

From the Lecture 11 joint table (rows 0.06, 0.09, 0.14, 0.11, 0.18, 0.12, 0.16, 0.14 for x1y1z1, x1y1z2, x1y2z1, x1y2z2, x2y1z1, ...), P(z1) = ___.

Answer: 0.54

Takeaway: 0.06 + 0.14 + 0.18 + 0.16.

Source: Book 3 (AI from zero), Practice 7.3, p. 28

MATH A42. [BLANK · Hard · calculation]

Prior 5%, sensitivity 80%, false-positive rate 10%. P(condition | +) = ___ (3 d.p.).

Answer: 0.296

Takeaway: 0.04/(0.04 + 0.095).

Source: Book 3 (AI from zero), Practice 7.4, p. 28

MATH A43. [BLANK · Easy · calculation]

A bet pays $10 with probability 0.2, else $0. Expected value = $___.

Answer: 2

Takeaway: 0.2 × 10.

Source: Book 3 (AI from zero), Practice 7.5, p. 28

MATH A44. [TF · Medium]

P(A ∨ B) = P(A) + P(B) always.

Answer: False

Why: Only when A and B cannot both happen; otherwise subtract P(A ∧ B).

Takeaway: General OR.

Source: Book 3 (AI from zero), Chapter 7, p. 26

MATH A45. [MATCH · Medium]

Match the distribution.

1. Bernoulli
2. categorical

3. Gaussian $N(\mu, \sigma^2)$

A. one yes/no outcome
B. uniform over pixels
C. bell curve: VAE prior, diffusion noise
D. one of k classes (softmax output)

Answer: 1 → A, 2 → D, 3 → C

Working:

- Bernoulli → one yes/no outcome
- categorical → one of k classes (softmax output)
- Gaussian $N(\mu, \sigma^2)$ → bell curve: VAE prior, diffusion noise

Takeaway: Distributions you will meet.

Source: Book 3 (AI from zero), Chapter 7, p. 26

📘 Book 2 mixed practice paper (40 questions, 100 minutes): answer key

1. A* frontier: node X has g = 120, h = 90; node Y has g = 150, h = 50. Which is expanded next?
   Answer: A. Y
2. Greedy best-first search with an admissible heuristic is guaranteed to find the cheapest path.
   Answer: False
3. A heuristic satisfying h(n) ≤ c(n, a, n′) + h(n′) for all edges is called ___.
   Answer: consistent
4. SA: a move is worse by Δ = 4 at temperature T = 2. Acceptance probability ___ (3 d.p.).
   Answer: 0.135
5. Match the local search algorithm.
   Answer: 1 → A, 2 → B, 3 → C
6. Entropy of a node with 3 positive and 1 negative example: ___ (3 d.p.).
   Answer: 0.811
7. Parent 5+/5−. A split gives L: 4+/1− and R: 1+/4−. Information gain ___ (3 d.p.).
   Answer: 0.278
8. Gini impurity of a node with 2 positive and 6 negative examples: ___.
   Answer: 0.375
9. TP = 30, FP = 20, FN = 10. Precision ___, recall ___, F1 ___ (3 d.p.).
   Answer: precision = 0.600; recall = 0.750; F1 = 0.667
10. Errors y − ŷ = (2, −2, 1, −1). RMSE ___ (3 d.p.); MAE ___.
    Answer: RMSE = 1.581; MAE = 1.5
11. Which metric is threshold-independent?
    Answer: A. AUC
12. Perceptron, w = (0, 1, −1) (bias first), x = (1, 0.5, 1), y = 1, α = 0.5, step activation. New w0, w1, w2 = ___.
    Answer: w0 = 0.5; w1 = 1.25; w2 = -0.5
13. Sigmoid derivatives are at most 0.25, which contributes to vanishing gradients in deep sigmoid networks.
    Answer: True
14. Reward 5 at each of three steps, γ = 0.5. Discounted return ___.
    Answer: 8.75
15. Option A: 60% +20, 40% −5. Option B: certain +8. EU(A) = ___; MEU chooses ___.
    Answer: EU(A) = 10; choice = A
16. TD update: U(s) = 0.2, R = 1, γ = 0.5, U(s′) = 0.6, α = 0.5. New U(s) = ___.
    Answer: 0.75
17. Q-learning: Q(s, a) = 1, R = 0, γ = 0.9, max_a′ Q(s′, a′) = 4, α = 0.25. New Q(s, a) = ___.
    Answer: 1.65
18. ADP differs from TD because ADP:
    Answer: B. learns P(s′|s, a) from counts and solves the Bellman equations
19. Conv output size for a 64 × 64 input, 5 × 5 kernel, padding 2, stride 2: ___.
    Answer: 32
20. 64 filters of 3 × 3 on a 32-channel input, with biases: ___ parameters.
    Answer: 18496
21. Receptive field side after four stacked 3 × 3 stride-1 convolutions: ___.
    Answer: 9

22. IoU of (0, 0)–(3, 3) and (1, 1)–(4, 4): ___ (3 d.p.).
    Answer: 0.286
23. NMS compares boxes of different classes against each other.
    Answer: False
24. RNN: x = 3, w_x = 0.2, h_prev = −0.5, w_h = 0.6, no bias, tanh(0.3) = 0.291 supplied. New hidden state ___.
    Answer: 0.291
25. Attention weights (0.1, 0.6, 0.3) on values [1, 1], [2, 0], [0, 3]. Output = [___, ___].
    Answer: 1st = 1.3; 2nd = 1.0
26. Naive Bayes: P(+) = P(−) = 0.5; P(great|+) = 0.2, P(great|−) = 0.05; P(boring|+) = 0.02, P(boring|−) = 0.1. 'great but boring': predicted class ___; P(+|text) = ___ (3 d.p.).
    Answer: class = negative; P(+|text) = 0.444
27. Match the text representation.
    Answer: 1 → A, 2 → B, 3 → C, 4 → D
28. The reparameterisation trick is needed in VAEs because:
    Answer: D. we cannot backpropagate through random sampling
29. KL term for one latent dimension with μ = 0.5, σ = 1: ___.
    Answer: 0.125
30. In a GAN, the discriminator is trained to maximise log D(x) + log(1 − D(G(z))).
    Answer: True
31. P = F, Q = T. P ⇒ Q ___; Q ⇒ P ___; P ⇔ Q ___ (T/F).
    Answer: P ⇒ Q = T; Q ⇒ P = F; P ⇔ Q = F
32. Resolve (¬A ∨ B) with (A ∨ C):
    Answer: A. B ∨ C
33. 'No student failed' in FOL:
    Answer: C. ¬∃x Student(x) ∧ Failed(x)
34. Existential instantiation replaces the variable with:
    Answer: D. a brand-new constant
35. Test: prior 5%, sensitivity 80%, false-positive rate 10%. P(condition | positive) = ___ (3 d.p.).
    Answer: 0.296
36. Burglary BN: P(¬j, m, a, b, ¬e) = ___ (3 s.f., e.g. 1.23e-5).
    Answer: 6.57e-5
37. In the burglary BN, Burglary and Earthquake are independent (no evidence observed).
    Answer: True
38. Match the hierarchy requirement.
    Answer: 1 → C, 2 → A, 3 → B
39. Penguin is a Bird; Bird can fly; Penguin cannot fly; Pingu is a Penguin. Pingu can fly.
    Answer: False
40. Which are true? (Select ALL that apply.)
    Answer: A. Q-learning is model-free; B. BFS uses a FIFO queue; D. ReLU(z) = max(0, z)